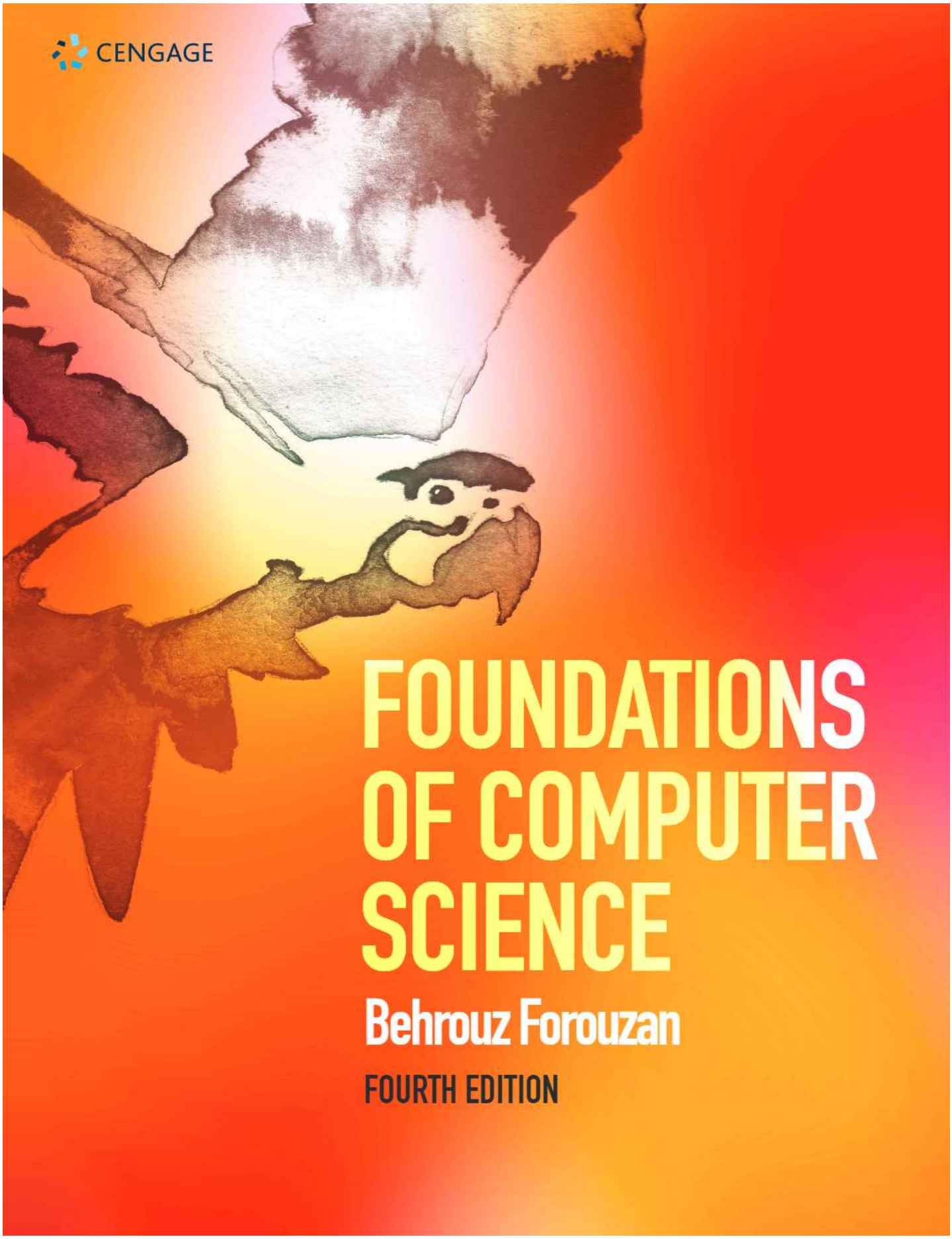
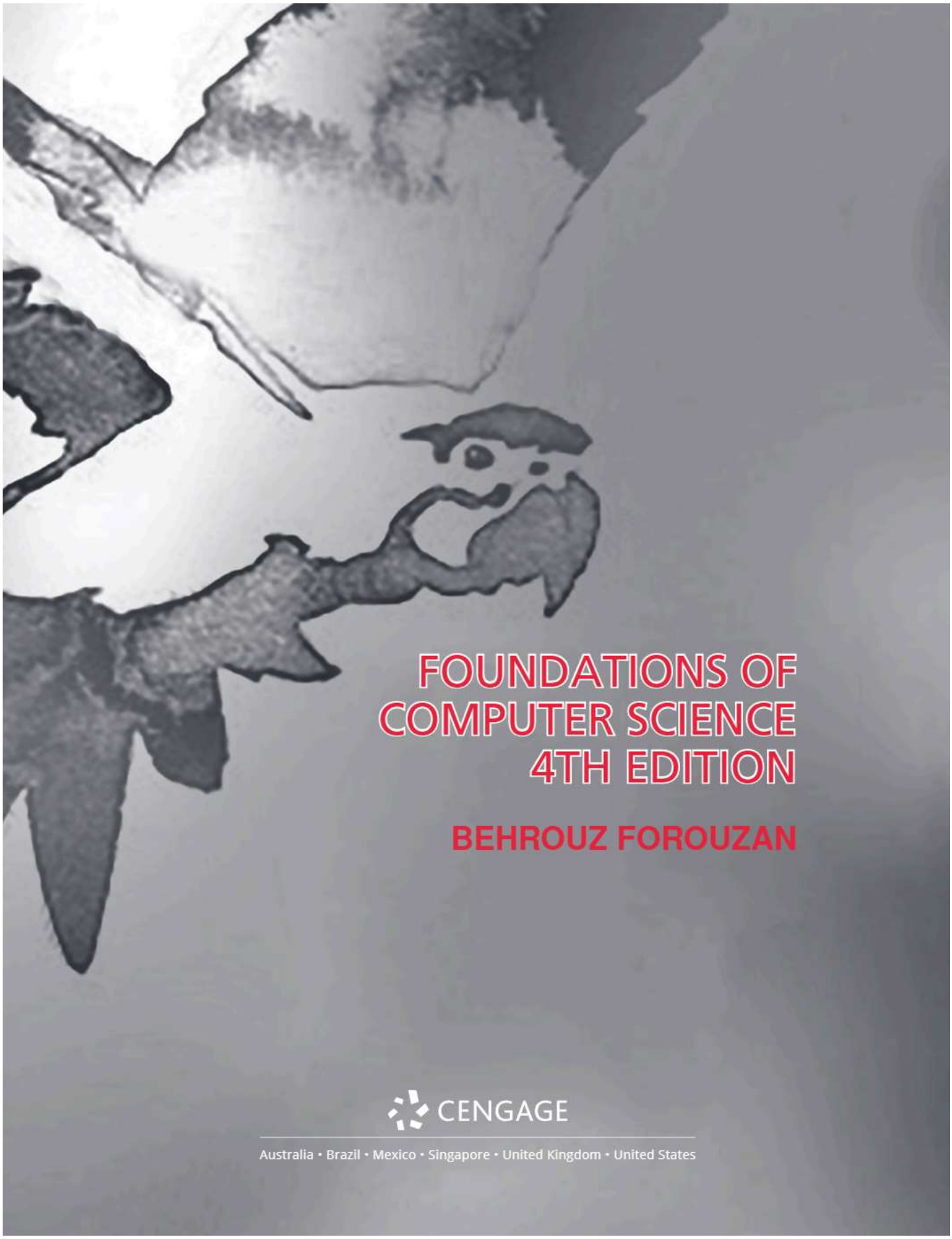

CENGAGE
FOUNDATIONS OF COMPUTER SCIENCE
Behrouz Forouzan
FOURTH EDITION


FOUNDATIONS OF
COMPUTER SCIENCE
4TH EDITION
BEHROUZ FOROUZAN
CENGAGE
Australia • Brazil • Mexico • Singapore • United Kingdom • United States

This is an electronic version of the print textbook. Due to electronic rights restrictions, some third party content may be suppressed. Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. The publisher reserves the right to remove content from this title at any time if subsequent rights restrictions require it. For valuable information on pricing, previous editions, changes to current editions, and alternate formats, please visit www.cengage.com/highered to search by ISBN#, author, title, or keyword for materials in your areas of interest.

Important Notice: Media content referenced within the product description or the product text may not be available in the eBook version.

CENGAGE

Foundations of Computer Science, **4th Edition**

Behrouz Forouzan

Publisher: Annabel Ainscow

List Manager: Jennifer Grene

Marketing Manager: Anna Reading

Content Project Manager: Phillipa Davidson-Blake

Manufacturing Buyer: Eyvett Davis

Typesetter: SPi Global

Cover Designer: Cyan Design

Cover Image: © Evgeny Turaev/ Shutterstock

© 2018, Cengage Learning EMEA

ALL RIGHTS RESERVED. No part of this work covered by the copyright herein may be reproduced or distributed in any form or by any means, except as permitted by U.S. copyright law, without the prior written permission of the copyright owner.

For product information and technology assistance, contact us at **emea.info@cengage.com.**

For permission to use material from this text or product, and for permission queries, email **emea.permissions@cengage.com.**

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library.

ISBN: 978-1-4737-5104-0

Cengage Learning EMEA
Cheriton House, North Way,
Andover, Hampshire, SP10 5BE
United Kingdom

Cengage Learning is a leading provider of customized learning solutions with employees residing in nearly 40 different countries and sales in more than 125 countries around the world. Find your local representative at: **www.cengage.co.uk**

Cengage Learning products are represented in Canada by Nelson Education Ltd.

For your course and learning solutions, visit **www.cengage.co.uk**

Purchase any of our products at your local college store or at our preferred online store **www.cengagebrain.com**

Printed in China by RR Donnelley
Print Number: 01 Print Year: 2017

To Ryan, Justin, William, and Benjamin

Contents

Preface *xi*
Trademarks *xvii*

1 *Introduction* *1*

Turing Model 2
Von Neumann Model 4
Computer Components 6
History 9
Computer Science as a Discipline 11
Outline of the Course 12
End-Chapter Materials 13
Practice Set 14

2 *Number Systems* *15*

Introduction 16
Positional Number Systems 16
Nonpositional Number Systems 31
End-Chapter Materials 32
Practice Set 34

3 *Data Storage* *39*

Data Types 40
Storing Numbers 42
Storing Text 60
Storing Audio 61
Storing Images 63
Storing Video 65
End-Chapter Materials 66
Practice Set 68

4 Operations on Data 73

Logic Operations 74
Shift Operations 79
Arithmetic Operations 82
End-Chapter Materials 86
Practice Set 87

5 Computer Organization 91

Introduction 92
Central Processing Unit 92
Main Memory 94
Input/Output Subsystem 97
Subsystem Interconnection 104
Program Execution 109
Different Architectures 113
A Simple Computer 117
End-Chapter Materials 126
Practice Set 130

6 Computer Networks and Internet 133

Overview 134
Application Layer 143
Transport Layer 156
Network Layer 159
Data-Link Layer 166
Physical Layer 174
Transmission Media 177
End-Chapter Materials 180
Practice Set 183

7 Operating Systems 187

Introduction 188
Evolution 189
Components 191
A Survey of Operating Systems 203
End-Chapter Materials 207
Practice Set 209

8 Algorithms 213

Concept 214
Three Constructs 218
Algorithm Representation 219
A More Formal Definition 223
Basic Algorithms 224

Subalgorithms 233
Recursion 234
End-Chapter Materials 236
Practice Set 238

9 *Programming Languages* 243

Evolution 244
Translation 246
Programming Paradigms 248
Common Concepts 257
End-Chapter Materials 267
Practice Set 269

10 *Software Engineering* 273

The Software Lifecycle 274
Analysis Phase 276
Design Phase 279
Implementation Phase 280
Testing Phase 283
Documentation 285
End-Chapter Materials 286
Practice Set 288

11 *Data Structure* 291

Arrays 292
Records 298
Linked Lists 301
End-Chapter Materials 314
Practice Set 315

12 *Abstract Data Types* 317

Background 318
Stacks 320
Queues 326
General Linear Lists 331
Trees 337
Graphs 343
End-Chapter Materials 344
Practice Set 346

13 *File Structure* 349

Introduction 350
Sequential Files 350
Indexed Files 354
Hashed Files 355
Directories 360

Text *versus* Binary 362
End-Chapter Materials 363
Practice Set 364

14 Databases *369*

Introduction 370
Database Architecture 372
Database Models 373
The Relational Database Model 374
Database Design 381
Other Database Models 385
End-Chapter Materials 386
Practice Set 388

15 Data Compression *391*

Introduction 392
Lossless Compression Methods 392
Lossy Compression Methods 400
End-Chapter Materials 406
Practice Set 407

16 Security *411*

Introduction 412
Confidentiality 415
Other Aspects of Security 428
Firewalls 442
End-Chapter Materials 445
Practice Set 447

17 Theory of Computation *451*

Simple Language 452
The Turing Machine 456
Gödel Numbers 463
The Halting Problem 464
The Complexity of Problems 467
End-Chapter Materials 469
Practice Set 470

18 Artificial Intelligence *473*

Introduction 474
Knowledge Representation 475
Expert Systems 485
Perception 487

Searching 494
Neural Networks 498
End-Chapter Materials 501
Practice Set 503

19 Introduction to Social Media 507

Introduction 508
Facebook 508
Twitter 514
End-Chapter Materials 521
Practice Set 522

20 Social and Ethical Issues 525

Ethical Principles 526
Intellectual Property 527
Privacy 528
Computer Crimes 528
Hackers 530
End-Chapter Materials 530
Practice Set 531

A Unicode 533

Planes 534
ASCII 535

B Unified Modeling Language (UML) 539

The User View 540
The Structural View 541
The Behavioral View 543
The Implementation View 550

C Pseudocode 553

Components 554

D Structure Chart 557

Structure Chart Symbols 557
Reading Structure Charts 560
Rules of Structure Charts 560

E Boolean Algebra and Logic Circuits 563

Boolean Algebra 563
Logic Circuits 574

F Examples of Programs in C, C++, and Java 581

Programs in C Language 581
Programs in C++ Language 584
Programs in Java Language 587

G Mathematical Review 591

Exponent and Logarithm 591
Modular Arithmetic 595
Discrete Cosine Transform 599

H Error Detection and Correction 601

Introduction 601
Block Coding 603
Linear Block Codes 606
Cyclic Codes 610
Checksum 613

I Addition and Subtraction for Sign-and-Magnitude Integers 617

Operations on Integers 617

J Addition and Subtraction for Reals 621

Operations on Reals 621

Acronyms 625

Glossary 629

Index 669

Preface

Computers play a large part in our everyday lives and will continue to do so in the future. Computer science is a young discipline that is evolving and progressing. Computer networks have connected people from far-flung points of the globe. Virtual reality is creating three-dimensional images that amaze the eyes. Space exploration owes part of its success to computers. Computer-generated special effects have changed the movie industry. Computers have played important roles in genetics.

Audience

This book is written for both academic and professional audience. The book can be used as a self-study guide for interested professionals. As a textbook, it can be used for a one-semester or one-quarter course. It is designed as the first course in computer science. This book is designed for a CS0 course based on the recommendations of the Association of Computing Machinery (ACM). It covers all areas of computer science in breadth. The book, totally or partially, can also be used in other disciplines where the students need to have a bird's-eye view approach to the computer science.

Changes in the fourth edition

I have made several categories of changes in this edition.

Revised chapters and appendices

Minor changes have been made to almost all the chapters. Two new chapters have been added (Chapters 19 and 20). Some materials have been removed from Chapter 4, expanded and inserted as two new appendices (Appendices I and J).

Organization

The book is made of 20 chapters and 10 appendices.

Chapters

Chapters are intended to provide the basic materials. However, not all chapters are needed for every audience. The professor who teaches the course can decide which chapters to use. We give guidance below.

Appendices

The appendices are intended to provide a quick reference or review of materials needed to understand the concepts discussed in the book. There are ten appendices that can be used by the students for reference and study.

Acronyms

The book contains a list of acronyms for finding the corresponding terms quickly.

Glossary

The book contains an extensive glossary giving full explanations of the terms used in the book.

Pedagogy

Several pedagogical features of this text are designed to make it particularly easy for students to understand the materials.

Visual approach

The book presents highly technical subject matter without complex formulas by using a balance of text and figures. More than 400 figures accompanying the text provide a visual and intuitive opportunity for understanding the material. Figures are particularly important in explaining the relationship between components of a whole. For many students, these concepts are more easily grasped visually than verbally.

Highlighted points

I have repeated important concepts in boxes for quick reference and immediate attention.

Examples and applications

Whenever appropriate, I have included examples that illustrate the concepts introduced in the text.

Algorithms

The inclusion of algorithms in the text helps students with problem solving and programming.

Unified Modeling Language (UML)

Throughout the book I have used UML diagrams to make students familiar with this tool, which is becoming the de facto standard in the industry.

End-of-chapter materials

Each chapter ends with a set of materials that includes the following:

Recommended reading

This section gives a brief list of references relative to the chapter. The references can be used to quickly find the corresponding literature.

Key terms

The new terms used in each chapter are listed at the end of the chapter and their definitions are included in the glossary.

Summary

Each chapter ends with a summary of the material covered by that chapter. The summary consolidates the important learning points in one place for ease of access by students.

Practice set

Each chapter includes a practice set designed to reinforce salient concepts and encourage students to apply them. It consists of three parts: quizzes, questions, and problems.

Quizzes

Quizzes, which are posted on the book website, provide quick concept checking. Students can take these quizzes to check their understanding of the materials. The feedback to the students' responses is given immediately.

Questions

This section contains simple questions about the concepts discussed in the book. Answers to the odd-numbered questions are posted on the book website to be checked by the student.

Problems

This section contains more difficult problems that need a deeper understanding of the materials discussed in the chapter. I strongly recommend that the student trys to solve all of these problems. Answers to the odd-numbered problems are also posted on the book website to be checked by the student.

Professor resources

The book contains complete resources for professors who teach the course. They can be downloaded from the book site. They include:

Presentations

The site includes a set of colorful and animated PowerPoint presentations for teaching the course.

Solutions to practice set

Solutions to all questions and problems are provided on the book website for the use of professors who teach the course.

Student resources

The book contains complete student resources on the book website. They include:

Quizzes

There are quizzes at the end of chapters that can be taken by the students. Students are encouraged to take these quizzes to test their general understanding of the materials presented in the corresponding chapter.

Solutions to odd-numbered practice sets

Solutions to all odd-number questions and problems are provided on the book website for the use of students.

How to use the book

The chapters in the book are organized to provide a great deal of flexibility. I suggest the following:

- Materials provided in Chapters 1 to 8 are essential to understand the rest of the book.
- Materials provided in Chapters 9 to 14 can be taught if the time allows. They can be skipped in a quarter system.
- Chapters 15 to 20 can be taught at the discretion of the professor and the majors of students.

Acknowledgments

It is obvious that the development of a book of this scope needs the support of many people.

Peer reviewers

I would like to acknowledge the contributions from peer reviewers to the development of the book. These reviewers are:

Sam Ssemugabi, UNISA	Steve Maybank, Birbeck College
Ronald Chikati, Botswana Accountancy College	Mario Kolberg, University of Stirling
Alex Dandadzi, University of Limpopo	Colin Price, University of Worcester
Tom Verhoeff, Eindhoven University of Technology	Boris Cogan, London Metropolitan University
Stefan Gruner, University of Pretoria	Thomas Mandl, University of Hildesheim
Harin Sellahwea, University of Buckingham	Daphne Becker, University of South Africa
John Newman, University of Wales	Lubna Fekry Abdulhai and Osama Abulnaja, King Abdulaziz University
	Katie Atkinson, University of Liverpool

Publisher staff

Special thanks go to the staff of the publisher.

Andrew Ashwin	Jennifer Grene
Annabel Ainscow	Phillipa Davidson-Blake

Behrouz A. Forouzan
Los Angeles, CA.
January 2018

Trademarks

Throughout the text we have used several trademarks. Rather than insert a trademark symbol with each mention of the trademark name, we acknowledge the trademarks here and state that they are used with no intention of infringing upon them. Other product names, trademarks, and registered trademarks are the property of their respective owners.

CHAPTER 1

Introduction

The phrase *computer science* has a very broad meaning today. However, in this book, we define the phrase as 'issues related to the computer'. This introductory chapter first tries to find out what a computer is, then investigates other issues directly related to computers. We look first at the **Turing model** as a mathematical and philosophical definition of computation. We then show how today's computers are based on the **von Neumann model**. The chapter ends with a brief history of this culture-changing device . . . the computer.

Objectives

After studying this chapter, the student should be able to:

- Define the Turing model of a computer.
- Define the von Neumann model of a computer.
- Describe the three components of a computer: hardware, data, and software.
- List topics related to computer hardware.
- List topics related to data.
- List topics related to software.
- Give a short history of computers.

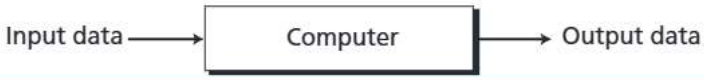
1.1 TURING MODEL

The idea of a universal computational device was first described by Alan Turing in 1936. He proposed that all computation could be performed by a special kind of a machine, now called a **Turing machine**. Although Turing presented a mathematical description of such a machine, he was more interested in the philosophical definition of computation than in building the actual machine. He based the model on the actions that people perform when involved in computation. He abstracted these actions into a model for a computational machine that has really changed the world.

1.1.1 Data processors

Before discussing the Turing model, let us define a computer as a **data processor**. Using this definition, a computer acts as a black box that accepts input data, processes the data, and creates output data (Figure 1.1). Although this model can define the functionality of a computer today, it is too general. In this model, a pocket calculator is also a computer (which it is, in a literal sense).

Figure 1.1 *A single-purpose computing machine*



Another problem with this model is that it does not specify the type of processing, or whether more than one type of processing is possible. In other words, it is not clear how many types or sets of operations a machine based on this model can perform. Is it a specific-purpose machine or a general-purpose machine?

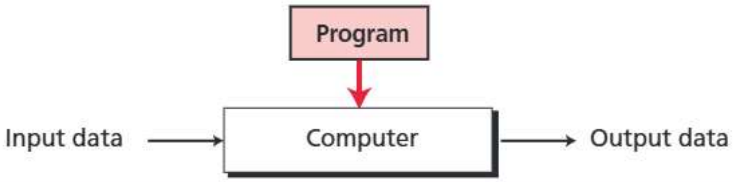
This model could represent a specific-purpose computer (or processor) that is designed to do a single job, such as controlling the temperature of a building or controlling the fuel usage in a car. However, computers, as the term is used today, are *general-purpose* machines. They can do many different types of tasks. This implies that we need to change this model into the Turing model to be able to reflect the actual computers of today.

1.1.2 Programmable data processors

The Turing model is a better model for a general-purpose computer. This model adds an extra element to the specific computing machine: the *program*. A **program** is a set of instructions that tells the computer what to do with data. Figure 1.2 shows the Turing model.

In the Turing model, the **output data** depends on the combination of two factors: the **input data** and the program. With the same input data, we can generate different output if we change the program. Similarly, with the same program, we can generate different

Figure 1.2 *A computer based on the Turing model: programmable data processor*

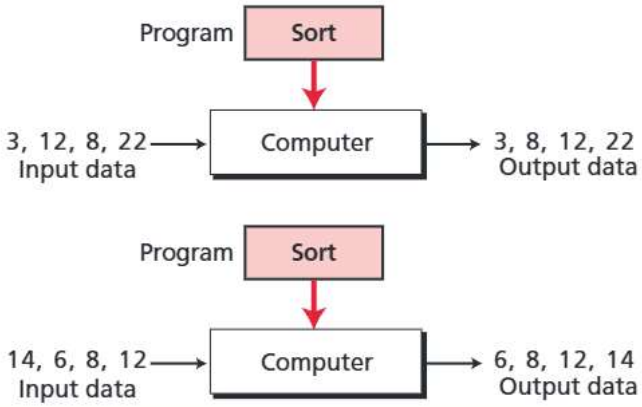


outputs if we change the input data. Finally, if the input data and the program remain the same, the output should be the same. Let us look at three cases.

Same program, different input data

Figure 1.3 shows the same sorting program with different input data. Although the program is the same, the outputs are different, because different input data is processed.

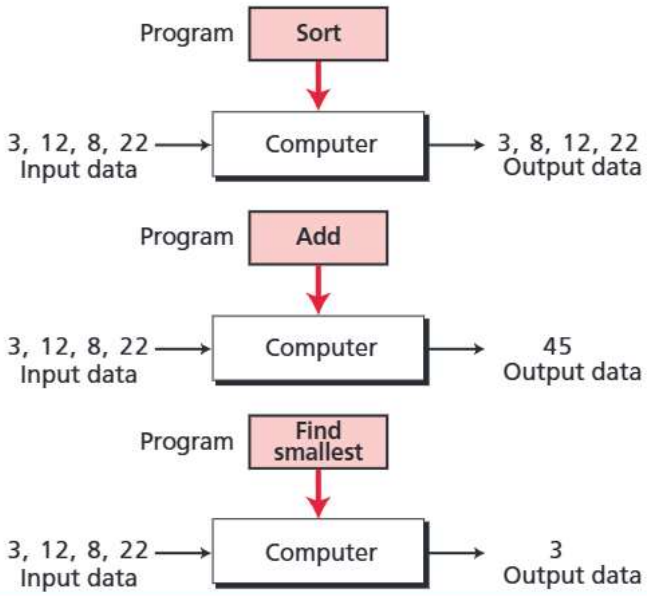
Figure 1.3 *The same program, different data*



Same input data, different programs

Figure 1.4 shows the same input data with different programs. Each program makes the computer perform different operations on the input data. The first program sorts the data, the second adds the data, and the third finds the smallest number.

Figure 1.4 *The same data, different programs*



Same input data, same program

We expect the same result each time if both input data and the program are the same, of course. In other words, when the same program is run with the same input data, we expect the same output.

1.1.3 The universal Turing machine

A *universal Turing machine*, a machine that can do any computation if the appropriate program is provided, was the first description of a modern computer. It can be proved that a very powerful computer and a universal Turing machine can compute the same thing. We need only provide the data and the program—the description of how to do the computation—to either machine. In fact, a universal Turing machine is capable of computing anything that is computable.

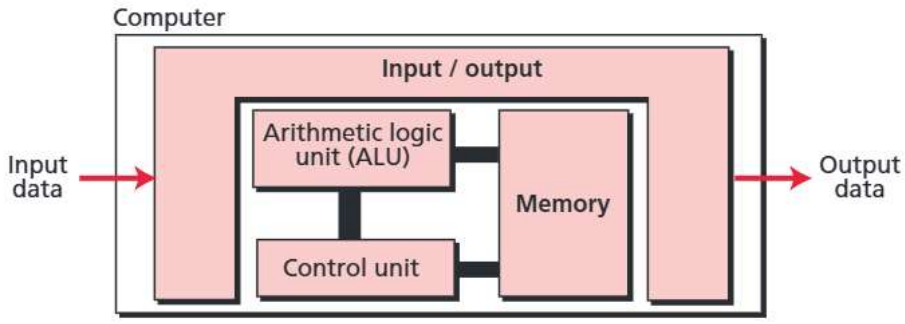
1.2 VON NEUMANN MODEL

Computers built on the Turing universal machine store data in their memory. Around 1944–1945, John von Neumann proposed that, since program and data are logically the same, programs should also be stored in the memory of a computer.

1.2.1 Four subsystems

Computers built on the von Neumann model divide the computer hardware into four subsystems: memory, arithmetic logic unit, control unit, and input/output (Figure 1.5).

Figure 1.5 *The Von Neumann model*



Memory

Memory is the storage area. This is where programs and data are stored during processing. We discuss the reasons for storing programs and data later in the chapter.

Arithmetic logic unit

The **arithmetic logic unit** (ALU) is where calculation and **logical operations** take place. For a computer to act as a data processor, it must be able to do arithmetic operations on data (such as adding a list of numbers). It should also be able to do logical operations on data, as we will see in Chapter 4.

Control unit

The **control unit** controls the operations of the **memory**, ALU, and the input/output subsystem.

Input / output

The **input subsystem** accepts input data and the program from outside the computer, while the **output subsystem** sends the result of processing to the outside world. The definition of the input/output subsystem is very broad: it also includes secondary storage devices such as disk or tape that stores data and programs for processing. When a disk stores data that results from processing, it is considered an output device: when it reads data from the disk, it is considered an input device.

1.2.2 The stored program concept

The von Neumann model states that the program must be stored in memory. This is totally different from the architecture of early computers in which only the data was stored in memory: the programs for their task were implemented by manipulating a set of switches or by changing the wiring system.

The memory of modern computers hosts both a program and its corresponding data. This implies that both the data and programs should have the same format, because they are stored in memory. In fact, they are stored as *binary* patterns in memory—a sequence of 0s and 1s.

1.2.3 Sequential execution of instructions

A program in the von Neumann model is made of a finite number of **instructions**. In this model, the control unit fetches one instruction from memory, decodes it, then executes it. In other words, the instructions are executed one after another. Of course, one instruction may request the control unit to jump to some previous or following instruction, but this does not mean that the instructions are not executed sequentially. Sequential execution of a program was the initial requirement of a computer based on the von Neumann model. Today's computers execute programs in the order that is the most efficient.

1.3 COMPUTER COMPONENTS

We can think of a computer as being made up of three components: computer hardware, data, and computer software.

1.3.1 Computer hardware

Computer hardware today has four components under the von Neumann model, although we can have different types of memory, different types of input/output subsystems, and so on. We discuss computer hardware in more detail in Chapter 5.

1.3.2 Data

The von Neumann model clearly defines a computer as a data processing machine that accepts the input data, processes it, and outputs the result.

Storing data

The von Neumann model does not define how data must be stored in a computer. If a computer is an electronic device, the best way to store data is in the form of an electrical signal, specifically its presence or absence. This implies that a computer can store data in one of two states.

Obviously, the data we use in daily life is not just in one of two states. For example, our numbering system uses digits that can take one of ten states (0 to 9). We cannot (as yet) store this type of information in a computer: it needs to be changed to another system that uses only two states (0 and 1). We also need to be able to process other types of data (text, image, audio, video). These also cannot be stored in a computer directly, but need to be changed to the appropriate form (0s and 1s).

In Chapter 3, we will learn how to store different types of data as a binary pattern, a sequence of 0s and 1s. In Chapter 4, we show how data is manipulated, as a binary pattern, inside a computer.

Organizing data

Although data should be stored only in one form inside a computer, a binary pattern, data outside a computer can take many forms. In addition, computers (and the notion of data processing) have created a new field of study known as *data organization*, which asks the question: can we organize our data into different entities and formats before storing them inside a computer? Today, data is not treated as a flat sequence of information. Instead, data is organized into small units, small units are organized into larger units, and so on. We will look at data from this point of view in Chapters 11–14.

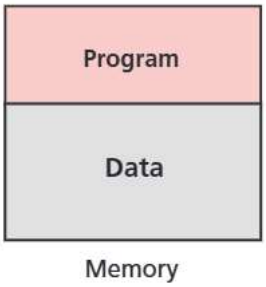
1.3.3 Computer software

The main feature of the Turing or von Neumann models is the concept of the *program*. Although early computers did not store the program in the computer's memory, they did use the concept of programs. *Programming* those early computers meant changing the wiring systems or turning a set of switches on or off. Programming was therefore a task done by an operator or engineer before the actual data processing began.

Programs must be stored

In the von Neumann model programs are stored in the computer's memory. Not only do we need memory to hold data, but we also need memory to hold the program (Figure 1.6).

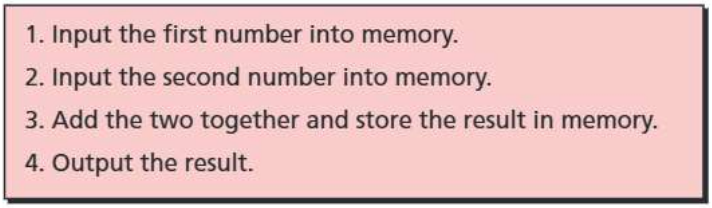
Figure 1.6 *Program and data in memory*



A sequence of instructions

Another requirement of the model is that the program must consist of a sequence of instructions. Each instruction operates on one or more data items. Thus, an instruction can change the effect of a previous instruction. For example, Figure 1.7 shows a program that inputs two numbers, adds them, and prints the result. This program consists of four individual instructions.

Figure 1.7 *A program made of instructions*



We might ask why a program must be composed of instructions. The answer is reusability. Today, computers do millions of tasks. If the program for each task was an independent entity without anything in common with other programs, programming would be difficult. The Turing and von Neumann models make programming easier by defining the different instructions that can be used by computers. A programmer can then combine these instructions to make any number of programs. Each program can be a different combination of different instructions.

Algorithms

The requirement for a program to consist of a sequence of instructions made programming possible, but it brought another dimension to using a computer. A programmer must not only learn the task performed by each instruction, but also learn how to combine these instructions to do a particular task. Looking at this issue differently, a programmer must first solve the problem in a step-by-step manner, then try to find the appropriate instruction (or series of instructions) to implement those steps. This step-by-step solution is called an **algorithm**. Algorithms play a very important role in computer science and are discussed in Chapter 8.

Languages

At the beginning of the computer age there was only one computer language, *machine language*. Programmers wrote instructions (using binary patterns) to solve a problem. However, as programs became larger, writing long programs using these patterns became tedious. Computer scientists came up with the idea of using symbols to represent binary patterns, just as people use symbols (words) for commands in daily life. Of course, the symbols used in daily life are different from those used in computers. So the concept of **computer languages** was born. A natural language such as English is rich and has many rules to combine words correctly: a computer language, on the other hand, has a more limited number of symbols and also a limited number of words. We will study computer languages in Chapter 9.

Software engineering

Something that was not defined in the von Neumann model is **software engineering**, which is the design and writing of **structured programs**. Today it is not acceptable just to write a program that does a task: the program must follow strict rules and principles. We discuss these principles, collectively known as *software engineering*, in Chapter 10.

Operating systems

During the evolution of computers, scientists became aware that there was a series of instructions common to all programs. For example, instructions to tell a computer where to receive data and where to send data are needed by almost all programs. It is more efficient to write these instructions only once for the use of all programs. Thus the concept of the **operating system** emerged. An operating system originally worked as a manager to facilitate access to the computer's components by a program, although today operating systems do much more. We will learn about them in Chapter 7.

1.4 HISTORY

In this section we briefly review the history of computing and computers. We divide this history into three periods.

1.4.1 Mechanical machines (before 1930)

During this period, several computing machines were invented that bear little resemblance to the modern concept of a computer.

- In the seventeenth century, Blaise Pascal, a French mathematician and philosopher, invented Pascaline, a mechanical calculator for addition and subtraction operations. In the twentieth century, when Niklaus Wirth invented a structured programming language, he called it Pascal to honor the inventor of the first mechanical calculator.
- In the late seventeenth century, German mathematician Gottfried Leibniz invented a more sophisticated mechanical calculator that could do multiplication and division as well as addition and subtraction. It was called the Leibniz Wheel.
- The first machine that used the idea of storage and programming was the Jacquard loom, invented by Joseph-Marie Jacquard at the beginning of the nineteenth century. The loom used punched cards (like a stored program) to control the raising of the warp threads in the manufacture of textiles.
- In 1823, Charles Babbage invented the Difference Engine, which could do more than simple arithmetic operations—it could solve polynomial equations, too. Later, he invented a machine called the Analytical Engine that, to some extent, parallels the idea of modern computers. It had four components: a mill (corresponding to a modern ALU), a store (memory), an operator (control unit), and output (input/output).
- In 1890, Herman Hollerith, working at the US Census Bureau, designed and built a programmer machine that could automatically read, tally, and sort data stored on punched cards.

1.4.2 The birth of electronic computers (1930–1950)

Between 1930 and 1950, several computers were invented by scientists who could be considered the pioneers of the electronic computer industry.

Early electronic computers

The early computers of this period did not store the program in memory—all were programmed externally. Five computers were prominent during these years:

- The first special-purpose computer that encoded information electrically was invented by John V. Atanasoff and his assistant Clifford Berry in 1939. It was called the ABC (Atanasoff Berry Computer) and was specifically designed to solve a system of linear equations.
- At the same time, a German mathematician called Konrad Zuse designed a general-purpose machine called Z1.
- In the 1930s, the US Navy and IBM sponsored a project at Harvard University under the direction of Howard Aiken to build a huge computer called Mark I. This computer used both electrical and mechanical components.
- In England, Alan Turing invented a computer called Colossus that was designed to break the German Enigma code.
- The first general-purpose, totally electronic computer was made by John Mauchly and J. Presper Eckert and was called ENIAC (Electronic Numerical Integrator and Calculator). It was completed in 1946. It used 18000 vacuum tubes, was 100 feet long by 10 feet high, and weighed 30 tons.

Computers based on the von Neumann model

The preceding five computers used memory only for storing data, and were programmed externally using wires or switches. John von Neumann proposed that the program and the data should be stored in memory. That way, every time we use a computer to do a new task, we need only change the program instead of rewiring the machine or turning hundreds of switches on and off.

The first computer based on von Neumann's ideas was made in 1950 at the University of Pennsylvania and was called EDVAC. At the same time, a similar computer called EDSAC was built by Maurice Wilkes at Cambridge University in England.

1.4.3 Computer generations (1950–present)

Computers built after 1950 more or less follow the von Neumann model. They have become faster, smaller, and cheaper, but the principle is almost the same. Historians divide this period into generations, with each generation witnessing some major change in hardware or software (but not in the model).

First generation

The first generation (roughly 1950–1959) is characterized by the emergence of commercial computers. During this time, computers were used only by professionals. They were locked in rooms with access limited only to the operator or computer specialist. Computers

were bulky and used vacuum tubes as electronic switches. At this time, computers were affordable only by big organizations.

Second generation

Second-generation computers (roughly 1959–1965) used transistors instead of vacuum tubes. This reduced the size of computers, as well as their cost, and made them affordable to small and medium-size corporations. Two high-level programming languages, FORTRAN and COBOL (see Chapter 9), were invented and made programming easier. These two languages separated the programming task from the computer operation task. A civil engineer, for example could write a FORTRAN program to solve a problem without being involved in the electronic details of computer architecture.

Third generation

The invention of the **integrated circuit** (transistors, wiring, and other components on a single chip) reduced the cost and size of computers even further. *Minicomputers* appeared on the market. Canned programs, popularly known as software packages, became available. A small corporation could buy a package, for example for accounting, instead of writing its own program. A new industry, the software industry, was born. This generation lasted roughly from 1965 to 1975.

Fourth generation

The fourth generation (approximately 1975–1985) saw the appearance of **microcomputers**. The first desktop calculator, the Altair 8800, became available in 1975. Advances in the electronics industry allowed whole computer subsystems to fit on a single circuit board. This generation also saw the emergence of computer networks (see Chapter 6).

Fifth generation

This open-ended generation started in 1985. It has witnessed the appearance of laptop and palmtop computers, improvements in secondary storage media (CD-ROM, DVD, and so on), the use of multimedia, and the phenomenon of virtual reality.

1.5 COMPUTER SCIENCE AS A DISCIPLINE

With the invention of computers, a new discipline has evolved: *computer science*. Like any other discipline, computer science has now divided into several areas. We can divide these areas into two broad categories: *systems areas* and *applications areas*. Systems areas cover those areas that directly related to the creation of hardware and software, such as *computer architecture*, *computer networking*, *security issues*, *operating systems*, *algorithms*, *programming languages*, and *software engineering*. Applications areas cover those that are related to the *use* of computers, such as *databases* and *artificial intelligence*. This book is a breadth-first approach to all of these areas. After reading the book, the reader should have enough information to select the desired area of specialty.

1.6 OUTLINE OF THE COURSE

After this introductory chapter, the book is divided into five parts.

1.6.1 Part I: Data representation and operation

This part includes Chapters 2, 3, and 4. Chapter 2 discusses number systems, how a quantity can be represented using symbols. Chapter 3 discusses how different data is stored inside the computer. Chapter 4 discusses some primitive operations on *bits*.

1.6.2 Part II: Computer hardware

This part includes Chapters 5 and 6. Chapter 5 gives a general idea of computer hardware, discussing different computer organizations. Chapter 6 shows how individual computers are connected to make computer networks, and *internetworks* (internets). In particular, this chapter explores some subjects related to the Internet and its applications.

1.6.3 Part III: Computer software

This part includes Chapters 7, 8, 9, and 10. Chapter 7 discusses operating systems, the system software that controls access to the hardware by users—either human or application programs. Chapter 8 shows how problem solving is reduced to writing an algorithm for the problem. Chapter 9 takes a journey through the list of contemporary programming languages. Finally, Chapter 10 is a review of software engineering, the engineering approach to the development of software.

1.6.4 Part IV: Data organization and abstraction

This part complements Part I. In computer science, *atomic* data is collected into records, files, and databases. Data *abstraction* allows the programmer to create abstract notions about data. Part IV includes Chapters 11, 12, 13, and 14. Chapter 11 discusses data structure, collecting data of the same or different type under one category. Chapter 12 discusses abstract data types. Chapter 13 shows how different file structures can be used for different purposes. Finally, Chapter 14 discusses databases.

1.6.5 Part V: Advanced topics

Part V gives an overview of advanced topics, topics that students of computer science will encounter later in their education. This part covers Chapters 15, 16, 17, and 18. Chapter 15 discusses data compression, which is prevalent in today's data communications. Chapter 16 explores some issues to do with security, which is becoming more and more important when we communicate over insecure channels. Chapter 17 discusses the theory of computation: what can and cannot be computed. Finally Chapter 18 gives some idea of artificial intelligence, a topic wih day-to-day challenges in computer science.

1.6.6 Part VI: Social media and social Issues

Part VI briefly discusses social media and social issues, two topics that students of computer science may be interested to explore.

1.7 END-CHAPTER MATERIALS

1.7.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Schneider, G. M. and Gersting, J. L. *Invitation to Computer Science*, Boston, MA: Course Technology, 2004
- Dale, N. and Lewis, J. *Computer Science Illuminated*, Sudbury, MA: Jones and Bartlett, 2004
- Patt, Y. and Patel, S. *Introduction to Computing Systems*, New York: McGraw-Hill, 2004

1.7.2 Key terms

algorithm 8	memory 5
arithmetic logic unit (ALU) 5	microcomputer 11
computer languages 8	operating system 9
control unit 5	output data 2
data processor 2	program 2
input data 2	structured programs 9
input/output subsystem 5	software engineering 9
instruction 6	Turing machine 2
integrated circuit 11	Turing model 1
logical operation 5	von Neumann model 1

1.7.3 Summary

- The idea of a universal computational device was first put forward by Alan Turing in 1936. He proposed that all computations can be performed by a special kind of a machine, now called a Turing machine.
- The von Neumann model defines a computer as four subsystems: memory, arithmetic logic unit, control unit, and input/output. The von Neumann model states that the program must be stored in memory.
- We can think of a computer as made up of three components: computer hardware, data, and computer software.
- The history of computing and computers can be divided into three periods: the period of mechanical machines (before 1930), the period of electronic computers (1930–1950), and the period that includes the five modern computer generations.
- With the invention of computers a new discipline has evolved, *computer science*, which is now divided into several areas.

1.8 PRACTICE SET

1.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

1.8.2 Review questions

Q1-1. Define a computer based on the Turing model.
Q1-2. Define a computer based on the von Neumann model.
Q1-3. What is the role of a program in a computer that is based on the Turing model?
Q1-4. What is the role of a program in a computer based on the von Neumann model?
Q1-5. What are the various subsystems of a computer?
Q1-6. What is the function of the memory subsystem in a computer?
Q1-7. What is the function of the ALU subsystem in a computer?
Q1-8. What is the function of the control unit subsystem in a computer?
Q1-9. What is the function of the input/output subsystem in a computer?
Q1-10. Briefly describe the five generations of computers.

1.8.3 Problems

P1-1. Explain why a computer cannot solve a problem for which there is no solution outside the computer.
P1-2. If a small cheap computer can do the same thing as a large expensive computer, why do people need to have a large one?
P1-3. Do some research and find out whether the Pascaline calculator is a computer according to the Turing model.
P1-4. Do some research and find out whether Leibnitz' Wheel is a computer according to the Turing model.
P1-5. Do some research and find out whether the Jacquard Loom is a computer according to the Turing model.
P1-6. Do some research and find out whether Babbage's Analytical Engine is a computer according to the von Neumann model.
P1-7. Do some research about the ABC computer and find out whether this computer followed the von Neumann model.
P1-8. Do some research and find out in which computer generation keyboards originated.

CHAPTER 2

Number Systems

This chapter is a prelude to Chapters 3 and 4. In Chapter 3 we will show how data is stored inside the computer. In Chapter 4 we will show how logic and arithmetic operations are performed on data. This chapter is a preparation for understanding the contents of Chapters 3 and 4. Readers who know about number systems can skip this chapter and move on to Chapter 3 without loss of continuity. Note that the number systems discussed in this chapter are 'paper and pencil representations': we show how these numbers are stored in a computer in Chapter 3.

Objectives

After studying this chapter, the student should be able to:

- Understand the concept of number systems.
- Distinguish between nonpositional and positional number systems.
- Describe the decimal system (base 10).
- Describe the binary system (base 2).
- Describe the hexadecimal system (base 16).
- Describe the octal system (base 8).
- Convert a number in binary, octal, or hexadecimal to a number in the decimal system.
- Convert a number in the decimal system to a number in binary, octal, or hexadecimal.
- Convert a number in binary to octal and *vice versa*.
- Convert a number in binary to hexadecimal and *vice versa*.
- Find the number of digits needed in each system to represent a particular value.

2.1 INTRODUCTION

A **number system** (or numeral system) defines how a number can be represented using distinct symbols. A number can be represented differently in different systems. For example, the two numbers $(2A)_{16}$ and $(52)_8$ both refer to the same quantity, $(42)_{10}$, but their representations are different. This is the same as using the words *cheval* (French) and *equus* (Latin) to refer to the same entity, a horse.

As we use symbols (characters) to create words in a language, we use symbols (digits) to represent numbers. However, we know that the number of symbols (characters) in any language is limited. We need to repeat characters and combine them to create words. It is the same for numbers: we have a limited number of symbols (digits) to represent numbers, which means that the digits need to be repeated.

Several number systems have been used in the past and can be categorized into two groups: positional and nonpositional systems. Our main goal is to discuss the positional number systems, but we also give examples of nonpositional systems.

2.2 POSITIONAL NUMBER SYSTEMS

In a **positional number system**, the position a symbol occupies in the number determines the value it represents. In this system, a number represented as:

$$\pm (S_{K-1} \ldots S_2 S_1 S_0 . S_{-1} S_{-2} \ldots S_{-L})_b$$

has the value of:

$$n = \pm \; S_{K-1} \times b^{K-1} + \ldots + S_1 \times b^1 + S_0 \times b^0 + S_{-1} \times b^{-1} + S_{-2} \times b^{-2} + \ldots + S_{-L} \times b^{-L}$$

in which S is the set of symbols, b is the **base** (or **radix**), which is equal to the total number of the symbols in the set S, and S_K and S_L are symbols in the whole and fraction parts of the number. Note that we have used an expression that can be extended from the right or from the left. In other words, the power of b can be 0 to $K - 1$ in one direction and -1 to $-L$ in the other direction. The terms with non negative powers of b are related to the integral part of the number, while the terms with negative power of b are related to the fractional part of the number. The ± sign shows that the number can be either positive or negative. We will study several positional number systems in this chapter.

2.2.1 The decimal system (base 10)

The first positional number system we discuss in this chapter is the **decimal system**. The word *decimal* is derived from the Latin root *decem* (ten). In this system the base b = 10 and we use ten symbols to represent a number. The set of symbols is S = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}. As we know, the symbols in this system are often referred to as **decimal digits** or just digits. In this chapter, we use ± to show that a number can be positive or negative, but remember that these signs are not stored in computers—computers handle the sign differently, as we discuss in Chapter 3.

Computers store positive and negative numbers differently.

In the decimal system, a number is written as:

$$\pm (S_{K-1} \ldots S_2 S_1 S_0 . S_{-1} S_{-2} \ldots S_{-L})_{10}$$

but for simplicity, we often drop the parentheses, the base, and the plus sign (if the number is positive). For example, we write the number $+(552.23)_{10}$ as 552.23—the base and plus signs are implicit.

Integers

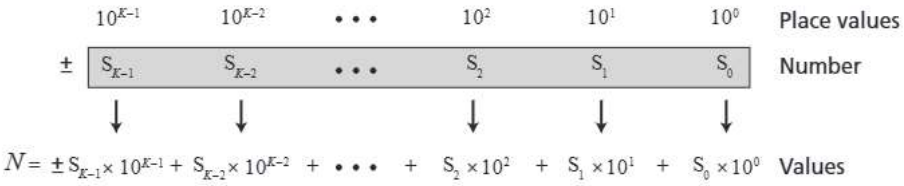
An **integer** (an integral number with no fractional part) in the decimal system is familiar to all of us—we use integers in our daily life. In fact, we have used them so much that they are intuitive. We represent an integer as $\pm S_{K-1} \ldots S_1 S_0$. The value is calculated as:

$$N = \pm \; S_{K-1} \times 10^{K-1} + S_{K-2} \times 10^{K-2} + \ldots + S_2 \times 10^2 + S_1 \times 10^1 + S_0 \times 10^0$$

in which S_i is a digit, b = 10 is the base, and K is the number of digits.

Another way to show an integer in a number system is to use **place values**, which are powers of 10 (10^0, 10^1, ..., 10^{K-1}) for decimal numbers. Figure 2.1 shows an integer in the decimal system using place values.

Figure 2.1 *Place value for an integer in decimal system*



Example 2.1

The following shows the place values for the integer +224 in the decimal system:

		10^2		10^1		10^0	Place values
		2		2		4	Number
N	= +	2×10^2	+	2×10^1	+	4×10^0	Values

Note that the digit in position 1 has the value 20, but the same digit in position 2 has the value 200. Also note that we normally drop the plus sign, but it is implicit.

Example 2.2

The following shows the place values for the decimal number –7508. We have used 1, 10, 100, and 1000 instead of powers of 10:

		1000		100		10		1	Place values
		7		5		0		8	Number
N =	–	$(7 \times 1000$	+	5×100	+	0×10	+	$8 \times 1)$	Values

Maximum value

Sometimes we need to know the maximum value of a decimal integer that can be represented by K digits. The answer is $N_{max} = 10^K - 1$. For example, if $K = 5$, then the maximum value is $N_{max} = 10^5 - 1 = 99\,999$.

Reals

A **real** (a number with a fractional part) in the decimal system is also familiar. For example, we use this system to show dollars and cents ($23.40). We can represent a real as $\pm S_{K-1} \ldots S_1 S_0 \bullet S_{-1} \ldots S_{-L}$. The value is calculated as:

	Integral part		Fractional part
$R = \pm$	$S_{K-1} \times 10^{K-1} + \ldots + S_1 \times 10^1 + S_0 \times 10^0$	+	$S_{-1} \times 10^{-1} + \ldots + S_{-L} \times 10^{-L}$

in which S_i is a digit, b = 10 is the base, K is the number of digits in the integral part, and L is the number of digits in the fractional part. The decimal point we use in our representation separates the fractional part from the integral part.

Example 2.3

The following shows the place values for the real number +24.13:

	10^1		10^0			10^{-1}		10^{-2}	Place values
	2		4		•	1		3	Number
$R = +$	2×10	+	4×1	+		1×0.1	+	3×0.01	Values

2.2.2 The binary system (base 2)

The second positional number system we discuss in this chapter is the **binary system**. The word **binary** is derived from the Latin root ***bini*** (or two by two). In this system the base b = 2 and we use only two symbols, S = {0, 1}. The symbols in this system are often referred to as **binary digits** or **bits** (binary digit). As we will see in Chapter 3, data and programs are stored in the computer using binary patterns, a string of bits. This is because the computer is made of electronic switches that can have only two states, on and off. The bit 1 represents one of these two states and the bit 0 the other.

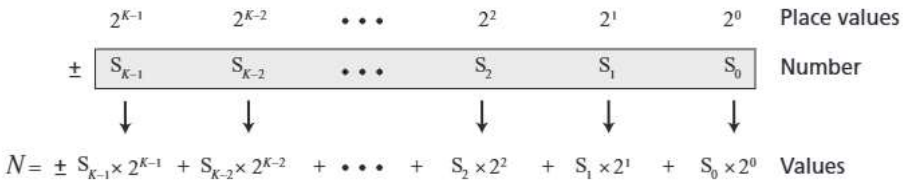
Integers

We can represent an integer as $\pm (S_{K-1} \ldots S_1 S_0)_2$. The value is calculated as:

$$N = \pm S_{K-1} \times 2^{K-1} + S_{K-2} \times 2^{K-2} + \ldots + S_2 \times 2^2 + S_1 \times 2^1 + S_0 \times 2^0$$

in which S_i is a digit, b = 2 is the base, and K is the number of bits. Another way to show a binary number is to use place values (2^0, 2^1, … 2^{K-1}). **Figure 2.2 shows a number in the binary number system using place values:**

Figure 2.2 *Place values in an integer in the binary system*



Example 2.4

The following shows that the number $(11001)_2$ in binary is the same as 25 in decimal. The subscript 2 shows that the base is 2:

	2^4		2^3		2^2		2^1		2^0	
	1		1		0		0		1	Number
$N =$	1×2^4	+	1×2^3	+	0×2^2	+	0×2^1	+	1×2^0	Decimal

Note that the equivalent decimal number is $N = 16 + 8 + 0 + 0 + 1 = 25$.

Maximum value

The maximum value of a binary integer with K digits is $N_{max} = 2^K - 1$. For example, if $K = 5$, then the maximum value is $N_{max} = 2^5 - 1 = 31$.

Reals

A real—a number with an optional fractional part—in the binary system can be made of K bits on the left and L bits on the right, $\pm (S_{K-1} \ldots S_1 S_0 \bullet S_{-1} \ldots S_{-L})_2$. The value can be calculated as:

$$R = \pm \underbrace{S_{K-1} \times 2^{K-1} \times \ldots \times S_1 \times 2^1 \times S_0 \times 2^0}_{\text{Integral part}} + \underbrace{S_{-1} \times 2^{-1} + \ldots + S_{-L} \times 2^{-L}}_{\text{Fractional part}}$$

in which S_i is a bit, b = 2 is the base, K is the number of bits to the left, and L is the number of bits to the right of the decimal point. Note that K starts from 0, but L starts from −1. The highest power is $K - 1$ and the lowest power is $-L$.

Example 2.5

The following shows that the number $(101.11)_2$ in binary is equal to the number 5.75 in decimal:

	2^2		2^1		2^0		2^{-1}		2^{-2}	Place values
	1		0		1	•	1		1	Number
R =	1×2^2	+	0×2^1	+	1×2^0	+	1×2^{-1}	+	1×2^{-2}	Values

Note that the value in the decimal system is $R = 4 + 0 + 1 + 0.5 + 0.25 = 5.75$.

2.2.3 The hexadecimal system (base 16)

Although the binary system is used to store data in computers, it is not convenient for representation of numbers outside the computer, as a number in binary notation is much longer than the corresponding number in decimal notation. However, the decimal system does not show what is stored in the computer as binary directly—there is no obvious relationship between the number of bits in binary and the number of decimal digits. Conversion from one to the other is not fast, as we will see shortly.

To overcome this problem, two positional systems were devised: hexadecimal and octal. We first discuss the **hexadecimal system**, which is more common. The word **hexadecimal** is derived from the Greek root *hex* (six) and the Latin root *decem* (ten). To be consistent with decimal and binary, it should really have been called *sexadecimal*, from the Latin roots *sex* and *decem*. In this system the base b = 16 and we use 16 symbols to represent a number. The set of symbols is S = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}. Note that the symbols A, B, C, D, E, F (uppercase or lowercase) are equivalent to 10, 11, 12, 13, 14, and 15 respectively. The symbols in this system are often referred to as **hexadecimal digits**.

Integers

We can represent an integer as $\pm S_{K-1} \ldots S_1 S_0$. The value is calculated as:

$$N = \pm S_{K-1} \times 16^{K-1} + S_{K-2} \times 16^{K-2} + \ldots + S_2 \times 16^2 + S_1 \times 16^1 + S_0 \times 16^0$$

in which S_i is a digit, b = 16 is the base, and K is the number of digits.

Another way to show a hexadecimal number is to use place values $(16^0, 16^1, \ldots, 16^{K-1})$. Figure 2.3 shows a number in the hexadecimal number system using place values.

Figure 2.3 *Place values in an integer in the hexadecimal system*

	16^{K-1}		16^{K-2}		• • •		16^2		16^1		16^0	Place values
±	S_{K-1}		S_{K-2}		• • •		S_2		S_1		S_0	Number
	↓		↓				↓		↓		↓	
N =	$\pm S_{K-1} \times 16^{K-1}$	+	$S_{K-2} \times 16^{K-2}$	+	• • •	+	$S_2 \times 16^2$	+	$S_1 \times 16^1$	+	$S_0 \times 16^0$	Values

Example 2.6

The following shows that the number $(2AE)_{16}$ in hexadecimal is equivalent to 686 in decimal:

		16^2		16^1		16^0	Place values
		2		A		E	Number
N	=	2×16^2	+	10×16^1	+	14×16^0	Values

Note that the value in the decimal system is $N = 512 + 160 + 14 = 686$.

Maximum value

The maximum value of a hexadecimal integer with K digits is $N_{max} = 16^K - 1$. For example, if $K = 5$, then the maximum value is $N_{max} = 16^5 - 1 = 1048575$.

Reals

Although a real number can be also represented in the hexadecimal system, it is not very common.

2.2.4 The octal system (base 8)

The second system that was devised to show the equivalent of the binary system outside the computer is the **octal system**. The word *octal* is derived from the Latin root *octo* (eight). In this system the base b = 8 and we use eight symbols to represent a number. The set of symbols is S = {0, 1, 2, 3, 4, 5, 6, 7}. The symbols in this system are often referred to as **octal digits**.

Integers

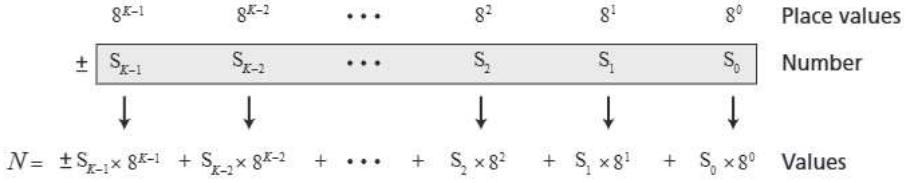
We can represent an integer as $\pm\ S_{K-1} \ldots S_1\ S_0$. The value is calculated as:

$$N = \pm S_{K-1} \times 8^{K-1} + S_{K-2} \times 8^{K-2} + \ldots + S_2 \times 8^2 + S_1 \times 8^1 + S_0 \times 8^0$$

in which S_i is a digit, b = 8 is the base, and K is the number of digits.

Another way to show an octal number is to use place values $(8^0, 8^1, \ldots, 8^{K-1})$. Figure 2.4 shows a number in the octal number system using place values.

Figure 2.4 *Place values in an integer in the octal system*



Example 2.7

The following shows that the number $(1256)_8$ in octal is the same as 686 in decimal:

	8^3		8^2		8^1		8^0	Place values
	1		2		5		6	Number
N =	1×8^3	+	2×8^2	+	5×8^1	+	6×8^0	Values

Note that the decimal number is $N = 512 + 128 + 40 + 6 = 686$.

Maximum Value

The maximum value of an octal integer with K digits is $N_{max} = 8^K - 1$. For example, if $K = 5$, then the maximum value is $N_{max} = 8^5 - 1 = 32767$.

Reals

Although a real number can be also represented in the octal system, it is not very common.

2.2.5 Summary of the four positional systems

Table 2.1 shows a summary of the four positional number systems discussed in this chapter.

Table 2.1 Summary of the four positional number systems

System	*Base*	*Symbols*	*Examples*
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	2345.56
Binary	2	0, 1	$(1001.11)_2$
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	$(156.23)_8$
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F	$(A2C.A1)_{16}$

Table 2.2 shows how the number 15 is represented with two digits in decimal, four digits in binary, two digits in octal, and only one digit in hexadecimal. The hexadecimal representation is definitely the shortest.

Table 2.2 Comparison of numbers in the four systems

Decimal	*Binary*	*Octal*	*Hexadecimal*
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4

Table 2.2 (Continued)

Decimal	Binary	Octal	Hexadecimal
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

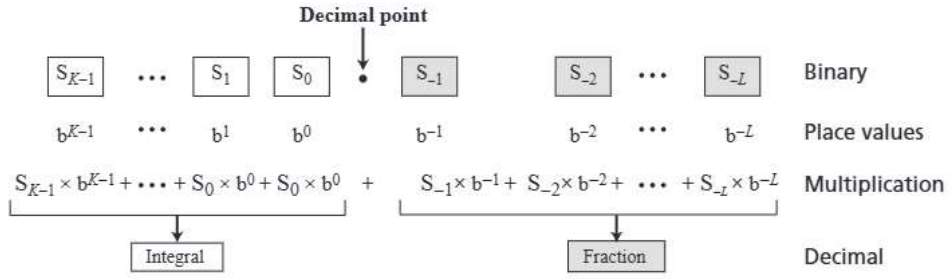
2.2.6 Conversion

We need to know how to convert a number in one system to the equivalent number in another system. Since the decimal system is more familiar than the other systems, we first show how to covert from any base to decimal. Then we show how to convert from decimal to any base. Finally, we show how we can easily convert from binary to hexadecimal or octal and *vice versa*.

Any base to decimal conversion

This type of conversion is easy and fast. We multiply each digit with its place value in the source system and add the results to get the number in the decimal system. Figure 2.5 shows the idea.

Figure 2.5 *Converting other bases to decimal*



Example 2.8

The following shows how to convert the binary number $(110.11)_2$ to decimal: $(110.11)_2 = 6.75$:

Binary	1		1		0	•	1		1
Place values	2^2		2^1		2^0		2^{-1}		2^{-2}
Partial results	4	+	2	+	0	+	0.5	+	0.25
Decimal: 6.75									

Example 2.9

The following shows how to convert the hexadecimal number $(1A.23)_{16}$ to decimal:

Hexadecimal	1		A	•	2		3
Place values	16^1		16^0		16^{-1}		16^{-2}
Partial result	16	+	10	+	0.125	+	0.012
Decimal: 26.137							

Note that the result in the decimal notation is not exact, because $3 \times 16^{-2} = 0.01171875$. We have rounded this value to three digits (0.012). In other words, $(1A.23)_{16} \approx 26.137$. When we convert a number in decimal to hexadecimal, we need to specify how many digits we allow to the right of the decimal point.

Example 2.10

The following shows how to convert $(23.17)_8$ to decimal:

Octal	2		3	•	1		7
Place values	8^1		8^0		8^{-1}		8^{-2}
Partial result	16	+	3	+	0.125	+	0.109
Decimal: 19.234							

This means that $(23.17)_8 \approx 19.234$ in decimal. Again, we have rounded up $7 \times 8^{-2} = 0.109375$.

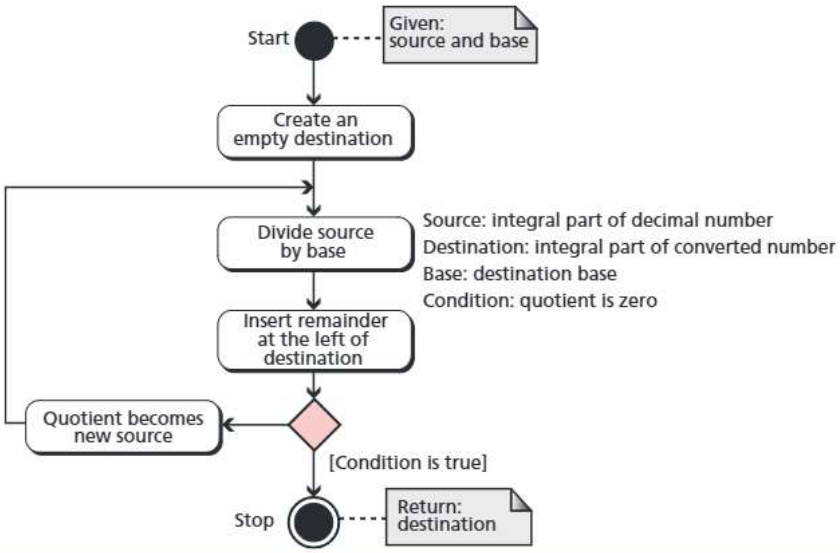
Decimal to any base

We can convert a decimal number to its equivalent in any base. We need two procedures, one for the integral part and one for the fractional part.

Converting the integral part

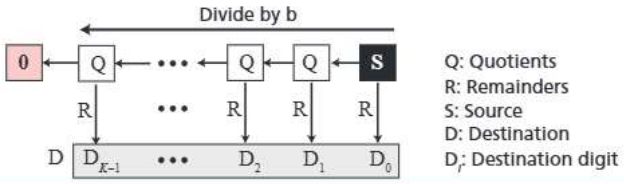
The integral part can be converted using repetitive division. Figure 2.6 shows the UML diagram for the process. We use UML diagrams through the book. For those readers not familiar with UML diagrams, please see Appendix B.

Figure 2.6 *Algorithm to convert the integral part*



We call the integral part of the decimal number the *source* and the integral part of the converted number the *destination*. We first create an empty destination. We then repetitively divide the source to get the quotient and the remainder. The remainder is inserted to the left of the destination. The quotient becomes a new source. Figure 2.7 shows how the destination is made with each repetition.

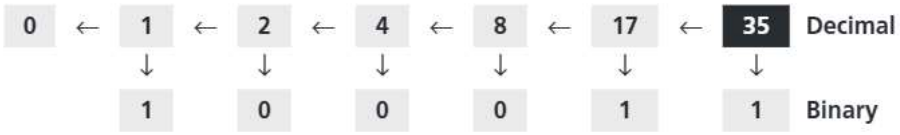
Figure 2.7 *Converting the integral part*



We use Figure 2.6 to illustrate the process manually with some examples.

Example 2.11

The following shows how to convert 35 in decimal to binary. We start with the number in decimal, we move to the left while continuously finding the quotients and the remainder of division by 2. The result is $35 = (100011)_2$:



Example 2.12

The following shows how to convert 126 in decimal to its equivalent in the octal system. We move to the right while continuously finding the quotients and the remainder of division by 8. The result is $126 = (176)_8$:

Example 2.13

The following shows how we convert 126 in decimal to its equivalent in the hexadecimal system. We move to the right while continuously finding the quotients and the remainder of division by 16. The result is $126 = (7E)_{16}$:

Converting the fractional part

The fractional part can be converted using repetitive multiplication. We call the fractional part of the decimal number the *source* and the fractional part of the converted number the *destination*. We first create an empty destination. We then repetitively multiply the source to get the result. The integral part of the result is inserted to the right of the destination, while the fractional part becomes the new source. Figure 2.8 shows the UML diagram for the process. Figure 2.9 shows how the destination is made in each repetition. We use the figure to illustrate the process manually with some examples.

Figure 2.8 *Algorithm to convert the fractional part*

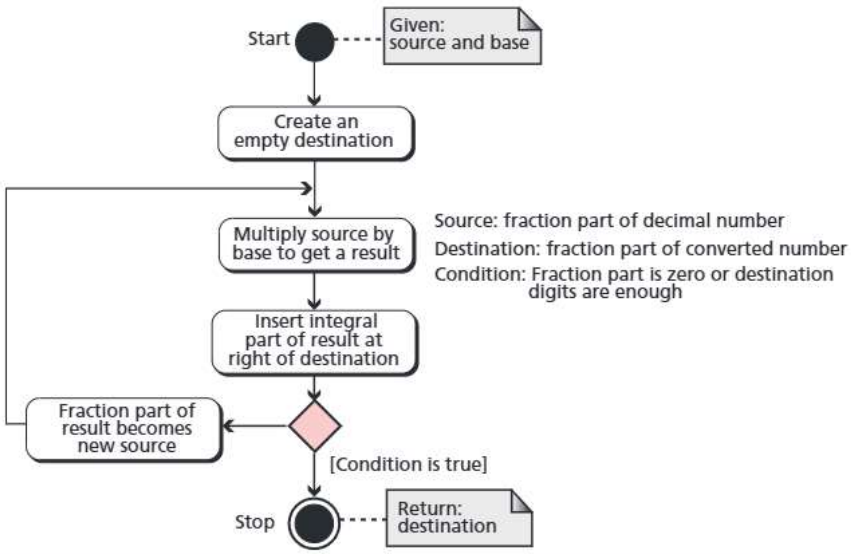
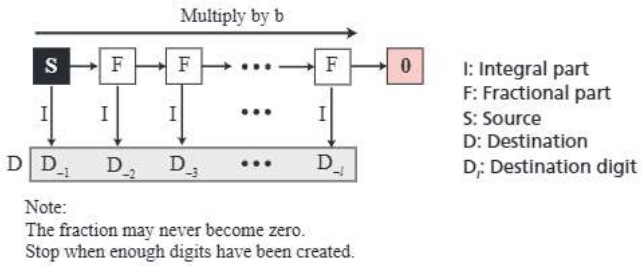


Figure 2.9 *Converting the fractional part*



Example 2.14

Convert the decimal number 0.625 to binary.

Solution

Since the number 0.625 has no integral part, the example shows how the fractional part is calculated. The base here is 2. Write the decimal number at the left corner. Multiply the number continuously by 2 and record the integral and fractional part of the result. The fractional part moves to the right, and the integral part is recorded under each operation. Stop when the fractional part is 0 or there are enough bits. The result is $(0.101)_2$:

Decimal		0.625	→	0.25	→	0.50	→	0.00
		↓		↓		↓		
Binary	•	1		0		1		

Example 2.15

The following shows how to convert 0.634 to octal using a maximum of four digits. The result is $0.634 = (0.5044)_8$. Note that we multiply by 8 (base octal):

Decimal		0.634	→	0.072	→	0.576	→	0.608	→	0.864
		↓		↓		↓		↓		
Octal	•	5		0		4		4		

Example 2.16

The following shows how to convert 178.6 in decimal to hexadecimal using only one digit to the right of the decimal point. The result is $178.6 = (B2.9)_{16}$ Note that we divide or multiply by 16 (base hexadecimal):

Decimal	0	←	11	←	178	•	0.6	→	0.6
			↓		↓		↓		
Hexadecimal			B		2	•	9		

Example 2.17

An alternative method for converting a small decimal integer (usually less than 256) to binary is to break the number as the sum of numbers that are equivalent to the binary place values:

Place values	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Decimal equivalent	128	64	32	16	8	4	2	1

Using this table, we can convert 165 to binary $(10100101)_2$ as shown below:

Decimal 165 =	128	+	0	+	32	+	0	+	0	+	4	+	0	+	1
Binary	1		0		1		0		0		1		0		1

Example 2.18

A similar method can be used to convert a decimal fraction to binary when the denominator is a power of two:

Place values	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
Decimal equivalent	½	¼	⅛	1/16	1/32	1/64	1/128

Using this table, we convert 27/64 to binary $(0.011011)_2$ as shown below:

Decimal = 27/64	16/64	+	8/64	+	2/64	+	1/64
	¼	+	⅛	+	1/32	+	1/64

Align these fractions according to decimal equivalent values. Note that since ½ and 1/16 are missing, we replace them with 0s:

Decimal 27/64 =	0	+	¼	+	⅛	+	0	+	1/32	+	1/64
Binary	0		1		1		0		1		1

Number of digits

We often need to know the number of digits before converting a number from decimal to other bases. In a positional number system with base b, we can always find the number of digits of an integer using the relation $K = \lceil \log_b N \rceil$, in which $\lceil x \rceil$ means the smallest integer greater than or equal to x (it is also called the *ceiling* of x), and N is the decimal value of the integer. For example, we can find the required number of bits in the decimal number 234 in all four systems as follows:

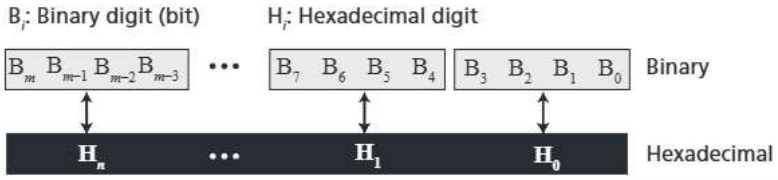
a. In decimal: $K_d = \lceil \log_{10} 234 \rceil = \lceil 2.37 \rceil = 3$, which is obvious.
b. In binary: $K_b = \lceil \log_2 234 \rceil = \lceil 7.8 \rceil = 8$. This is true because $234 = (11101010)_2$
c. In octal: $K_o = \lceil \log_8 234 \rceil = \lceil 2.62 \rceil = 3$. This is true because $234 = (352)_8$
d. In hexadecimal $K_h = \lceil \log_{16} 234 \rceil = \lceil 1.96 \rceil = 2$. This is true because $234 = (EA)_{16}$

See Appendix G for information on how to calculate $\log_b N$ if your calculator does not include logs to any base.

Binary–hexadecimal conversion

We can easily change a number from binary to hexadecimal and *vice versa*. The reason is that there is a relationship between the two bases: four bits in binary is one digit in hexadecimal. Figure 2.10 shows how this conversion can be done.

Figure 2.10 *Binary to hexadecimal and hexadecimal to binary*



Example 2.19

Show the hexadecimal equivalent of the binary number $(10011100010)_2$.

Solution

We first arrange the binary number in 4-bit patterns: 100 1110 0010. Note that the leftmost pattern can have one to four bits. We then use the equivalent of each pattern shown in Table 2.2 in section 2.2.5 to change the number to hexadecimal: $(4E2)_{16}$.

Example 2.20

What is the binary equivalent of $(24C)_{16}$?

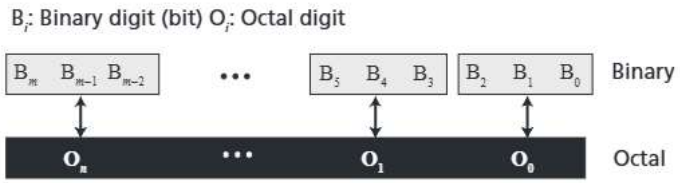
Solution

Each hexadecimal digit is converted to 4-bit patterns: 2 → 0010, 4 → 0100, and C → 1100. The result is $(001001001100)_2$.

Binary–octal conversion

We can easily convert a number from binary to octal and *vice versa*. The reason is that there is an interesting relationship between the two bases: three bits is one octal digit. Figure 2.11 shows how this conversion can be done.

Figure 2.11 Binary to octal conversion



Example 2.21

Show the octal equivalent of the binary number $(101110010)_2$.

Solution

Each group of three bits is translated into one octal digit. The equivalent of each 3-bit group is shown in Table 2.2 in section 2.2.5. The result is $(562)_8$.

Example 2.22

What is the binary equivalent of for $(24)_8$?

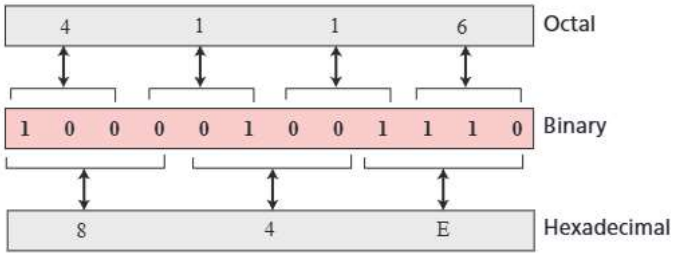
Solution

Write each octal digit as its equivalent bit pattern to get $(010100)_2$.

Octal–hexadecimal conversion

It is not difficult to convert a number in octal to hexadecimal or *vice versa*. We can use the binary system as the intermediate system. Figure 2.12 shows an example.

Figure 2.12 *Octal to hexadecimal and hexadecimal to octal conversion*



The following illustrates the process:

- To convert from octal to hexadecimal, we first convert the number in the octal system to binary. We then rearrange the bits in groups of four bits to find the hexadecimal equivalent.
- To convert from hexadecimal to octal, we first convert the number in the hexadecimal system to binary. We then rearrange the bits in groups of three to find the octal equivalent.

Number of digits

In conversion from one base to another, we often need to know the minimum number of digits we need in the destination system if we know the maximum number of digits in the source system. For example, if we know that we use at most six decimal digits in the source system, we want to know the minimum number of binary digits we need in the destination system. In general, assume that we are using K digits in base b_1 system. The maximum number we can represent in the source system is $b_1^K - 1$. The maximum number we can

have in the destination system is $b_2^x - 1$. Therefore, $b_2^x - 1 \geq b_1^K - 1$. This means $b_2^x \geq b_1^K$, which means:

$$x \geq K \times (\log b_1 / \log b_2) \quad \text{or} \quad x = \lceil K \times (\log b_1 / \log b_2) \rceil$$

Example 2.23

Find the minimum number of binary digits required to store decimal integers with a maximum of six digits.

Solution

$K = 6$, $b_1 = 10$, and $b_2 = 2$. Then $x = \lceil K \times (\log b_1 / \log b_2) \rceil = \lceil 6 \times (1 / 0.30103) \rceil = 20$. The largest six-digit decimal number is 999 999 and the largest 20-bit binary number is 1 048 575. Note that the largest number that can be represented by a 19-bit number is 524287, which is smaller than 999 999. We definitely need 20 bits.

2.3 NONPOSITIONAL NUMBER SYSTEMS

Although nonpositional number systems are not used in computers, we give a short review here for comparison with positional number systems. A **nonpositional number system** still uses a limited number of symbols in which each symbol has a value. However, the position a symbol occupies in the number normally bears no relation to its value—the value of each symbol is fixed. To find the value of a number, we add the value of all symbols present in the representation. In this system, a number is represented as:

$$S_{K-1} \ldots S_2 S_1 S_0 \bullet S_{-1} S_{-2} \ldots S_{-L}$$

and has the value of:

$$n = \pm \underbrace{S_{K-1} + \ldots + S_1 + S_0}_{\text{Integral part}} + \underbrace{S_{-1} + S_{-2} + \ldots + S_{-L}}_{\text{Fractional part}}$$

There are some exceptions to the addition rule we just mentioned, as shown in Example 2.24.

Example 2.24

The **Roman number system** is a good example of a nonpositional number system. This system was invented by the Romans and was used until the sixteenth century in Europe. Roman numerals are still used in sports events, clock dials, and other applications. This number system has a set of symbols S = {I, V, X, L, C, D, M}. The values of each symbol are shown in Table 2.3.

Table 2.3 Values of symbols in the Roman number system

Symbol	I	V	X	L	C	D	M
Value	1	5	10	50	100	500	1000

To find the value of a number, we need to add the value of symbols subject to specific rules:

1. When a symbol with a smaller value is placed after a symbol having an equal or larger value, the values are added.
2. When a symbol with a smaller value is placed before a symbol having a larger value, the smaller value is subtracted from the larger one.
3. A symbol S_1 cannot come before another symbol S_2 if $S_1 \leq 10 \times S_2$. For example, I or V cannot come before C.
4. For large numbers a bar is placed above any of the six symbols (all symbols except I) to express multiplication by 1000. For example, $\overline{V} = 5000$ and $\overline{M} = 1000000$.
5. Although Romans used the word *nulla* (nothing) to convey the concept of zero, the Roman numerals lack a zero digit in their system.

The following shows some Roman numbers and their values:

III	→	1 + 1 + 1	=	3
IV	→	5 − 1	=	4
VIII	→	5 + 1 + 1 + 1	=	8
XVIII	→	10 + 5 + 1 + 1 + 1	=	18
XIX	→	10 + (10 −1)	=	19
LXXII	→	50 + 10 + 10 + 1 + 1	=	72
CI	→	100 + 1	=	101
MMVII	→	1000 + 1000 + 5 + 1 + 1	=	2007
MDC	→	1000 + 500 + 100	=	1600

2.4 END-CHAPTER MATERIALS

2.4.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Stalling, W. *Computer Organization and Architecture,* Upper Saddle River, NJ: Prentice-Hall, 2000
- Mano, M. *Computer System Architecture,* Upper Saddle River, NJ: Prentice-Hall, 1993
- Null, L. and Lobur, J. *Computer Organization and Architecture,* Sudbury, MA: Jones and Bartlett, 2003
- Brown, S. and Vranesic, Z. *Fundamentals of Digital Logic with Verilog Design,* New York: McGraw-Hill, 2003

2.4.2 Key terms

base 16	nonpositional number system 31
binary digit 18	number system 16
binary system 18	octal digit 21
bit 18	octal system 21
decimal digit 16	place value 17
decimal system 16	positional number system 16
hexadecimal digit 20	radix 16
hexadecimal system 20	real 18
integer 17	Roman number system 31

2.4.3 Summary

- A number system (or numeral system) is a system that uses distinct symbols to represent a number. In a positional number system, the position a symbol occupies in the number determines the value it represents. Each position has a place value associated with it. A nonpositional number system uses a limited number of symbols in which each symbol has a value. However, the position a symbol occupies in the number normally bears no relation to its value: the value of each symbol is normally fixed.
- In the decimal system, the base b = 10 and we use ten symbols to represent numbers. The symbols in this system are often referred to as **decimal digits** or just **digits**. In the binary system, the base b = 2 and we use only two symbols to represent numbers. The symbols in this system are often referred to as **binary digits** or **bits**. In a hexadecimal system, the base = 16 and we use 16 symbols to represent numbers. The symbols in this system are often referred to as **hexadecimal digits**. In an octal system, the base b = 8 and we use eight symbols to represent numbers. The symbols in this system are often referred to as **octal digits**.
- We can convert a number in any system to decimal. We multiply each digit with its place value in the source system and add the result to get the number in the decimal system. We can convert a decimal number to its equivalent in any base using two different procedures, one for the integral part and one for the fractional part. The integral part needs repeated division and the fraction part needs repeated multiplication.
- Conversion from the binary system to the hexadecimal system and from the hexadecimal system to the binary system is very easy, because four bits in the binary system are represented as one digit in the hexadecimal system.
- Conversion from the binary system to the octal system and from the octal system to the binary system is very easy, because three bits in the binary system are represented as one digit in the octal system.

2.5 PRACTICE SET

2.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

2.5.2 Review questions

Q2-1. Define a number system.

Q2-2. Distinguish between positional and nonpositional number systems.

Q2-3. Define the base or radix in a positional number system. What is the relationship between a base and the number of symbols in a positional number system?

Q2-4. Explain the decimal system. Why is it called *decimal*? What is the base in this system?

Q2-5. Explain the binary system. Why is it called *binary*? What is the base in this system?

Q2-6. Explain the octal system. Why is it called *octal*? What is the base in this system?

Q2-7. Explain the hexadecimal system. Why is it called *hexadecimal*? What is the base in this system?

Q2-8. Why is it easy to convert from binary to hexadecimal and *vice versa*?

Q2-9. How many bits in the binary system are represented by one digit in the hexadecimal system?

Q2-10. How many bits in the binary system are represented by one digit in the octal system?

2.5.3 Problems

P2-1. Convert the following binary numbers to decimal without using a calculator, showing your work:

a. $(01101)_2$
b. $(1011000)_2$
c. $(011110.01)_2$
d. $(111111.111)_2$

P2-2. Convert the following hexadecimal numbers to decimal without using a calculator, showing your work:

a. $(AB2)_{16}$
b. $(123)_{16}$
c. $(ABB)_{16}$
d. $(35E.E1)_{16}$

P2-3. Convert the following octal numbers to decimal without using a calculator, showing your work:

a. $(237)_8$
b. $(2731)_8$
c. $(617.7)_8$
d. $(21.11)_8$

P2-4. Convert the following decimal numbers to binary without using a calculator, showing your work:

a. 1234
b. 88
c. 124.02
d. 14.56

P2-5. Convert the following decimal numbers to octal without using a calculator, showing your work:

a. 1156
b. 99
c. 11.4
d. 72.8

P2-6. Convert the following decimal numbers to hexadecimal without using a calculator, showing your work:

a. 567
b. 1411
c. 12.13
d. 16

P2-7. Convert the following octal numbers to hexadecimal without using a calculator, showing your work:

a. $(514)_8$
b. $(411)_8$
c. $(13.7)_8$
d. $(1256)_8$

P2-8. Convert the following hexadecimal numbers to octal without using a calculator, showing your work:

a. $(51A)_{16}$
b. $(4E1)_{16}$
c. $(BB.C)_{16}$
d. $(ABC.D)_{16}$

P2-9. Convert the following binary numbers to octal without using a calculator, showing your work:

a. $(01101)_2$
b. $(1011000)_2$
c. $(011110.01)_2$
d. $(111111.111)_2$

P2-10. Convert the following binary numbers to hexadecimal without using a calculator, showing your work:

a. $(01101)_2$
b. $(1011000)_2$
c. $(011110.01)_2$
d. $(111111.111)_2$

P2-11. Convert the following decimal numbers to binary using the alternative method discussed in Example 2.17, showing your work:

a. 121
b. 78
c. 255
d. 214

P2-12. Change the following decimal numbers into binary using the alternative method discussed in Example 2.18, showing your work:

a. $3\ 5/8$
b. $12\ 3/32$
c. $4\ 13/64$
d. $12\ 5/128$

P2-13. In a positional number system with base b, the largest integer number that can be represented using K digits is $b^K - 1$. Find the largest number in each of the following systems with *six* digits:

a. Binary
b. Decimal
c. Hexadecimal
d. Octal

P2-14. Without converting, find the minimum number of digits needed in the destination system for each of the following cases:

a. Five-digit decimal number converted to binary.
b. Four-digit decimal converted to octal.
c. Seven-digit decimal converted to hexadecimal.

P2-15. Without converting, find the minimum number of digits needed in the destination system for each of the following cases:

a. 5-bit binary number converted to decimal.

b. Three-digit octal number converted to decimal.

c. Three-digit hexadecimal converted to decimal.

P2-16. The following table shows how to rewrite a fraction so the denominator is a power of two (1, 4, 8, 16, and so on).

Original	*New*	*Original*	*New*
0.5	½	0.25	¼
0.125	⅛	0.0625	1⁄16
0.03125	1⁄32	0.015625	1⁄64

However, sometimes we need a combination of entries to find the appropriate fraction. For example, 0.625 is not in the table, but we know that 0.625 is 0.5 + 0.125. This means that 0.625 can be written as ½ + ⅛, or ⅝.

Change the following decimal fractions to a fraction with a power of 2.

a. 0.1875

b. 0.640625

c. 0.40625

d. 0.375

P2-17. Using the results of the previous problem, change the following decimal numbers to binary numbers.

a. 7.1875

b. 12.640625

c. 11.40625

d. 0.375

P2-18. Find the maximum value of an integer in each of the following cases:

a. b = 10, $K = 10$

b. b = 2, $K = 12$

c. b = 8, $K = 8$

d. b = 16, $K = 7$

P2-19. Find the minimum number of required bits to store the following integers:

a. less than 1000

b. less than 100000

c. less than 64

d. less than 256

P2-20. A number less than b^K can be represented using K digits in base b. Show the number of digits needed in each of the following cases.

a. Integers less than 2^{14} in binary

b. Integers less than 10^8 in decimal

c. Integers less than 8^{13} in octal

d. Integers less than 16^4 in hexadecimal

P2-21. A common base used on the Internet is b = 256. We need 256 symbols to represent a number in this system. Instead of creating this large number of symbols, the designers of this system have used decimal numbers to represent a symbol: 0 to 255. In other words, the set of symbols is S = {0, 1, 2, 3, …, 255}. A number in this system is always in the format $S_1.S_2.S_3.S_4$ with four symbols and three dots that separate the symbols. The system is used to define Internet addresses (see

Chapter 6). An example of an address in this system is 10.200.14.72, which is equivalent to $10 \times 256^3 + 200 \times 256^2 + 14 \times 256^1 + 72 \times 256^0 = 180883016$ in decimal. This number system is called *dotted decimal notation*. Find the decimal value of each of the following Internet addresses:

a. 17.234.34.14
b. 14.56.234.56
c. 110.14.56.78
d. 24.56.13.11

P2-22. Internet addresses described in the previous problem are also represented as patterns of bits. In this case, 32 bits are used to represent an address, eight bits for each symbol in dotted decimal notation. For example, the address 10.200.14.72 can also be represented as 00001010 11001000 00001110 01001000. Show the bit representation of the following Internet addresses:

a. 17.234.34.14
b. 14.56.234.56
c. 110.14.56.78
d. 24.56.13.11

P2-23. Write the decimal equivalent of the following Roman numbers:

a. XV
b. XXVII
c. VLIII
d. MCLVII

P2-24. Convert the following decimal numbers to Roman numbers:

a. 17
b. 38
c. 82
d. 999

P2-25. Find which of the following Roman numerals are not valid:

a. MMIM
b. MIC
c. CVC
d. VX

P2-26. Mayan civilization invented a positional vigesimal (base 20) numeral system, called the *Mayan numeral system*. They use base 20 probably because they used both their fingers and toes for counting. This system has 20 symbols that are constructed from three simpler symbols. The advanced feature of the system is that it has a symbol for zero, which is a shell. The other two symbols are a circle (or a pebble) for one and a horizontal bar (or a stick) for five. To represent a number greater than nineteen, numerals are written vertically. Search the Internet to answer the following: what are the decimal numbers 12, 123, 452, and 1256 in the Mayan numeral system?

P2-27. The *Babylonian* civilization is credited with developing the first positional numeral system, called the *Babylonian numeral system*. They inherited the Sumerian and Akkadian numeral system and developed it into positional sexagesimal system (base 60). This base is still used today for times and angles. For example, one hour is 60 minutes and one minute is 60 seconds: similarly, one degree is 60 minutes and one minute is 60 seconds. As a positional system with base b requires b symbols (digits), we expect a positional sexagesimal system to require 60 symbols. However, the Babylonians did not have a symbol for zero, and produced the other 59 symbols by stacking two symbols, those for one and ten. Search the Internet to answer the following questions:

a. Express the following decimal numbers in Babylonian numerals: 11 291, 3646, 3582.
b. Mention problems that might arise from not having a symbol for 0. Find how the Babylonian numeral system addresses the problem.

CHAPTER 3

Data Storage

As discussed in Chapter 1, a computer is a programmable data processing machine. Before we can talk about processing data, we need to understand the nature of data. In this chapter we discuss different data types and how they are stored inside a computer. In Chapter 4, we show how data is manipulated inside a computer.

Objectives

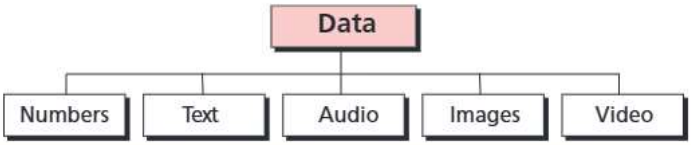
After studying this chapter, the student should be able to:

- List five different data types used in a computer.
- Describe how different data is stored inside the computer as bit patterns.
- Describe how integers are stored in a computer using unsigned format.
- Describe how integers are stored in a computer using sign-and-magnitude format.
- Describe how integers are stored in two's complement format.
- Describe how reals are stored in a computer using floating-point format.
- Describe how text is stored in a computer using one of the various encoding systems.
- Describe how audio is stored in a computer using sampling, quantization, and encoding.
- Describe how images are stored in a computer using raster and vector graphics schemes.
- Describe how video is stored in a computer as a representation of images changing in time.

3.1 DATA TYPES

Data today come in different forms including numbers, text, audio, images, and video (Figure 3.1).

Figure 3.1 *Different types of data*



People need to be able to process many different types of data:

- An engineering program uses a computer mainly to process numbers: to do arithmetic, to solve algebraic or trigonometric equations, to find the roots of a differential equation, and so on.
- A word processing program, on the other hand, uses a computer mainly to process text: justify, move, delete, and so on.
- A computer also handles audio data. We can play music on a computer and can record sound as data.
- An image processing program uses a computer to manipulate images: create, shrink, expand, rotate, and so on.
- Finally, a computer can be used not only to show movies, but also to create the special effects seen in movies.

The computer industry uses the term 'multimedia' to define information that contains numbers, text, audio, images, and video.

3.1.1 Data inside the computer

All data types are transformed into a uniform representation when they are stored in a computer and transformed back to their original form when retrieved. This universal representation is called a *bit pattern*, as discussed shortly.

Bits

A **bit (binary digit)** is the smallest unit of data that can be stored in a computer and has a value of 0 or 1. A bit represents the state of a device that can take one of two states. For example, a *switch* can be on or off. A convention can be established to represent the 'on' state as 1 and the 'off' state as 0, or *vice versa*. In this way, a switch can store one bit of information. Today, computers use various two-state devices to store data.

Bit patterns

To represent different types of data, we use a **bit pattern**, a sequence, or as it is sometimes called, a *string of bits*. Figure 3.2 shows a bit pattern made up of sixteen bits.

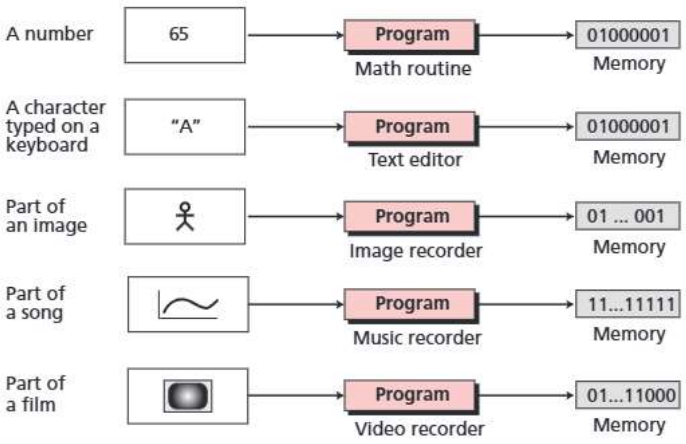
It is a combination of sixteen 0s and 1s. This means that if we need to store a bit pattern made of sixteen bits, we need sixteen electronic switches. If we need to store 1000 bit patterns, each sixteen bits long, we need 16000 switches, and so on. By tradition a bit pattern with eight bits is called a **byte**. Sometimes the term *word* is used to refer to a longer bit pattern.

Figure 3.2 *A bit pattern*

1 0 0 0 1 0 1 0 1 1 1 1 1 1 0 1

As Figure 3.3 shows, a piece of data belonging to different data types can be stored as the same pattern in the memory.

Figure 3.3 *Storage of different data types*



If we are using a **text editor** (a word processor), the character A typed on the keyboard can be stored as the 8-bit pattern 01000001. The same 8-bit pattern can represent the number 65 if we are using a mathematical routine. Moreover, the same pattern can represent part of an image, part of a song, or part of a scene in a film. The computer's memory stores all of them without recognizing what type of data they represent.

3.1.2 Data compression

To occupy less memory space, data is normally compressed before being stored in the computer. Data compression is a very broad and involved subject, so we have dedicated the whole of Chapter 15 to this subject.

Data compression is discussed in Chapter 15.

3.1.3 Error detection and correction

Another issue related to data is the detection and correction of errors during transmission or storage. We discuss this issue briefly in Appendix H.

Error detection and correction is discussed in Appendix H.

3.2 STORING NUMBERS

A number is changed to the binary system before being stored in the computer memory, as described in Chapter 2. However, there are still two issues that need to be handled:

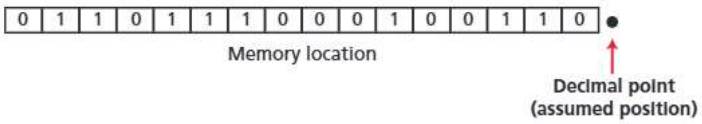
1. How to store the sign of the number.
2. How to show the decimal point.

There are several ways to handle the sign issue, discussed later in this chapter. For the decimal point, computers use two different representations: fixed-point and floating-point. The first is used to store a number as an integer—without a fractional part, the second is used to store a number as a real—with a fractional part.

3.2.1 Storing integers

Integers are whole numbers (numbers without a fractional part). For example, 134 and –125 are integers, whereas 134.23 and –0.235 are not. An integer can be thought of as a number in which the position of the decimal point is fixed: the decimal point is to the right of the least significant (rightmost) bit. For this reason, **fixed-point representation** is used to store an integer, as shown in Figure 3.4. In this representation the decimal point is assumed but not stored.

Figure 3.4 *Fixed-point representation of integers*



However, a user (or a program) may store an integer as a real with the fractional part set to zero. This may happen, for example, if an integer is too large to be stored in the size defined for an integer. To use computer memory more efficiently, unsigned and signed integers are stored inside the computer differently.

An integer is normally stored in memory using fixed-point representation.

Unsigned representation

An **unsigned integer** is an integer that can never be negative and can take only 0 or positive values. Its range is between 0 and positive infinity. However, since no computer can possibly represent all the integers in this range, most computers define a constant

called the *maximum unsigned integer*, which has the value of $(2^n - 1)$ where n is the number of bits allocated to represent an unsigned integer.

Storing unsigned integers

An input device stores an unsigned integer using the following steps:

1. The integer is changed to binary.
2. If the number of bits is less than n, 0s are added to the left of the binary integer so that there is a total of n bits. If the number of bits is greater than n, the integer cannot be stored. A condition referred to as *overflow* will occur, which we discuss later.

Example 3.1

Store 7 in an 8-bit memory location using unsigned representation.

Solution

First change the integer to binary, $(111)_2$. Add five 0s to make a total of eight bits, $(00000111)_2$. The integer is stored in the memory location. Note that the subscript 2 is used to emphasize that the integer is binary, but the subscript is not stored in the computer:

Change 7 to binary	→						1	1	1
Add five bits at the left	→	0	0	0	0	0	1	1	1

Example 3.2

Store 258 in a 16-bit memory location.

Solution

First change the integer to binary $(100000010)_2$. Add seven 0s to make a total of sixteen bits $(0000000100000010)_2$. The integer is stored in the memory location:

Change 258 to binary	→								1	0	0	0	0	0	0	1	0
Add seven bits at the left	→	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0

Retrieving unsigned integers

An output device retrieves a bit string from memory as a bit pattern and converts it to an unsigned decimal integer.

Example 3.3

What is returned from an output device when it retrieves the bit string 00101011 stored in the memory as an unsigned integer?

Solution

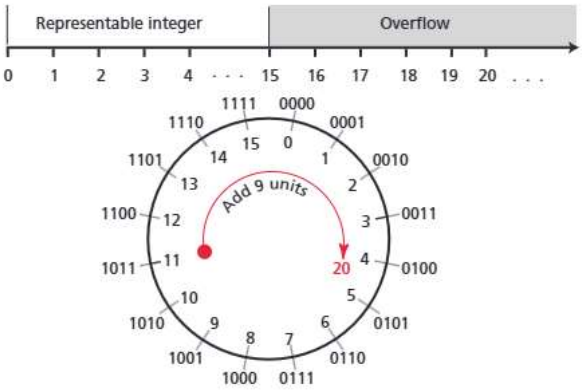
Using the procedure shown in Chapter 2, the binary integer is converted to the unsigned integer 43.

Overflow

Due to size limitations—the allocated number of bits—the range of integers that can be represented is limited. In an n-bit memory location we can only store an unsigned integer

between 0 and $2^n - 1$. Figure 3.5 shows what happens if we try to store an integer that is larger than $2^4 - 1 = 15$ in a memory location that can only hold four bits. This situation, called **overflow**, happens when, for example, we have stored the integer 11 in a memory location and then try to add 9 to the integer. The minimum number of bits we need to represent the decimal 20 is five bits. In other words, $20 = (10100)_2$, so the computer drops the leftmost bit and keeps the rightmost four bits $(0100)_2$. People are surprised when they see that the new integer is printed as 4 instead of 20. Figure 3.5 shows why this happens.

Figure 3.5 *Overflow in unsigned integers*



Applications of unsigned integers

Unsigned integer representation can improve the efficiency of storage because we do not need to store the sign of the integer. This means that the entire bit allocation can be used for storing the number. Unsigned integer representation can be used whenever we do not need negative integers. The following lists some cases:

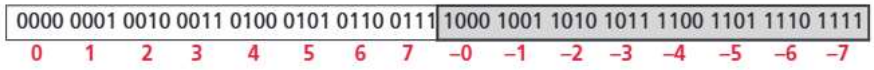
- **Counting**. When we count, we do not need negative numbers. We start counting from 1 (sometimes 0) and go up.
- **Addressing**. Some computer programs store the address of a memory location inside another memory location. Addresses are positive integers starting from 0 (the first memory location) and going up to an integer representing the total memory capacity. Here again, we do not need negative integers—unsigned integers can easily do the job.
- **Storing other data types**. Other data types (text, images, audio, and video), as we will discuss shortly, are stored as bit patterns, which can be interpreted as unsigned integers.

Sign-and-magnitude representation

Although the **sign-and-magnitude representation** format is not commonly used to store integers, this format is used to store part of a real number in a computer, as described in the next section. For this reason we briefly discuss this format here. In this method, the available range for unsigned integers (0 to $2^n - 1$) is divided into two equal subranges. The first half represents positive integers, the second half, negative integers. For example, if n is 4, the range is 0000 to 1111. This range is divided into two halves: 0000 to 0111 and

1000 to 1111 (Figure 3.6). The bit patterns are then assigned to negative and positive integers. Note that the negative numbers appear to the right of the positive numbers, which is contrary to conventional thinking about positive and negative numbers. Also note that we have two 0s: positive zero (0000) and negative zero (1000).

Figure 3.6 *Sign-and-magnitude representation*



Storing an integer in sign-and-magnitude format requires 1 bit to represent the sign (0 for positive, 1 for negative). This means that in an 8-bit allocation, we can only use seven bits to represent the absolute value of the number (number without the sign). Therefore, the maximum positive value is one half the unsigned value. The range of numbers that can be stored in an n-bit location is $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$. In an n-bit allocation, the leftmost bit is dedicated to store the sign (0 for positive, 1 for negative).

In sign-and-magnitude representation, the leftmost bit defines the sign of the integer. If it is 0, the integer is positive. If it is 1, the integer is negative.

Example 3.4

Store +28 in an 8-bit memory location using sign-and-magnitude representation.

Solution

The integer is changed to 7-bit binary. The leftmost bit is set to 0. The 8-bit number is stored:

Change 28 to 7-bit binary		0	0	1	1	1	0	0
Add the sign and store	0	0	0	1	1	1	0	0

Example 3.5

Store –28 in an 8-bit memory location using sign-and-magnitude representation.

Solution

The integer is changed to 7-bit binary. The leftmost bit is set to 1. The 8-bit number is stored:

Change 28 to 7-bit binary		0	0	1	1	1	0	0
Add the sign and store	1	0	0	1	1	1	0	0

Example 3.6

Retrieve the integer that is stored as 01001101 in sign-and-magnitude representation.

Solution

Since the leftmost bit is 0, the sign is positive. The rest of the bits (1001101) are changed to decimal as 77. After adding the sign, the integer is +77.

Example 3.7

Retrieve the integer that is stored as 10100001 in sign-and-magnitude representation.

Solution

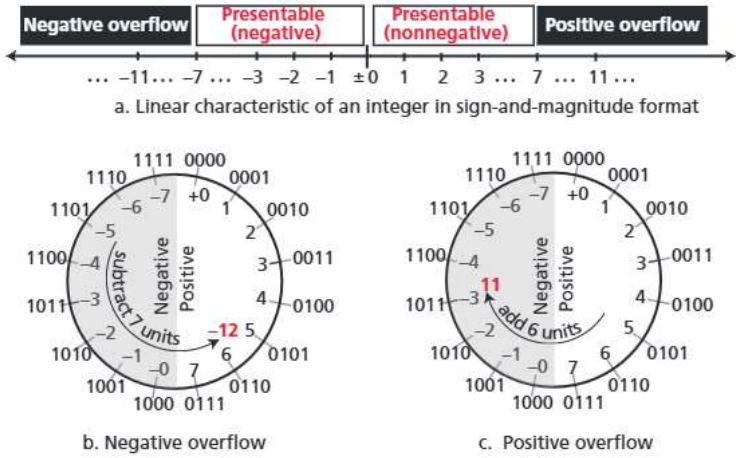
Since the leftmost bit is 1, the sign is negative. The rest of the bits (0100001) are changed to decimal as 33. After adding the sign, the integer is –33.

Overflow in sign-and-magnitude representation

Like unsigned integers, signed integers are also subjected to overflow. However, in this case, we may have both positive and negative overflow. Figure 3.7 shows both positive and negative overflow when storing an integer in sign-and-magnitude representation using a 4-bit memory location. Positive overflow occurs when we try to store a positive integer larger than 7. For example, assume that we have stored integer 5 in a memory location and we then try to add 6 to the integer. We expect the result to be 11, but the computer's response is –3. The reason is that if we start from 5 on a circular representation and go six units in the clockwise direction, we end up at –3. A positive overflow wraps the integer back to the range.

A negative overflow can happen when we try to store a integer that is less than –7, for example if we have stored the integer –5 in a memory and try to subtract 7 from it. We expect the result to be –12, but the computer's response is +6. The reason is that if we start from –5 on a circular representation and go seven units in the counterclockwise direction, we end up at +6.

Figure 3.7 *Overflow in sign-and-magnitude representation*



There are two 0s in sign-and-magnitude representation: +0 and –0.

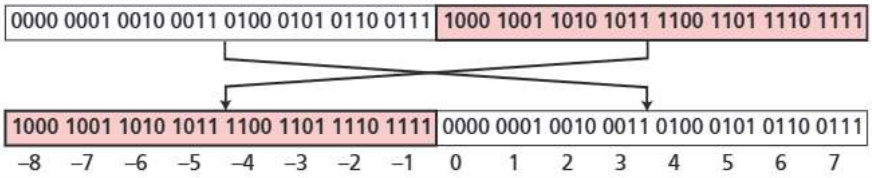
Applications of sign-and-magnitude representation

Sign-and-magnitude representation is not used to store integers. However, it is used to store part of real numbers, as we will see shortly. In addition, sign-and-magnitude representation is often used when we quantize an analog signal, such as audio.

Two's complement representation

Almost all computers use **two's complement representation** to store a signed integer in an n-bit memory location. In this method, the available range for an unsigned integer of (0 to $2^n - 1$) is divided into two equal subranges. The first subrange is used to represent nonnegative integers, the second half to represent negative integers. For example, if n is 4, the range is 0000 to 1111. This range is divided into two halves: 0000 to 0111 and 1000 to 1111. The two halves are swapped to be in agreement with the common convention of showing negative integers to the left of positive integers. The bit patterns are then assigned to negative and nonnegative (zero and positive) integers as shown in Figure 3.8.

Figure 3.8 *Two's complement representation*



Although the sign of the integer affects every bit in the binary integer stored, the first (leftmost) bit determines the sign. If the leftmost bit is 0, the integer is nonnegative: if the leftmost bit is 1, the integer is negative.

In two's complement representation, the leftmost bit defines the sign of the integer. If it is 0, the integer is positive. If it is 1, the integer is negative.

Two operations

Before we discuss this representation further, we need to introduce two operations. The first is called *one's completing* or *taking the one's complement of an integer*. The operation can be applied to any integer, positive or negative. This operation simply reverses (flips) each bit. A 0-bit is changed to a 1-bit, a 1-bit is changed to a 0-bit.

Example 3.8

The following shows how we take the **one's complement** of the integer 00110110:

Original pattern	0	0	1	1	0	1	1	0
After applying one's complement operation	1	1	0	0	1	0	0	1

Example 3.9

The following shows that we get the original integer if we apply the one's complement operations twice:

Original pattern	0	0	1	1	0	1	1	0
One's complementing once	1	1	0	0	1	0	0	1
One's complementing twice	0	0	1	1	0	1	1	0

The second operation is called *two's completing* or *taking the two's complement* of an integer in binary. This operation is done in two steps. First, we copy bits from the right until a 1 is copied, Then, we flip the rest of the bits.

Example 3.10

The following shows how we take the **two's complement** of the integer 00110100:

Original integer	0	0	1	1	0	1	0	0
Two's complementing once	1	1	0	0	1	1	0	0

Example 3.11

The following shows that we always get the original integer if we apply the two's complement operation twice:

Original integer	0	0	1	1	0	1	0	0
Two's complementing once	1	1	0	0	1	1	0	0
Two's complementing twice	0	0	1	1	0	1	0	0

An alternative way to take the two's complement of an integer is to first take the one's complement and then add 1 to the result (see Chapter 4 for binary addition).

Storing an integer in two's complement format

To store an integer in two's complement representation, the computer follows the steps below:

- The absolute value of the integer is changed to an n-bit binary.
- If the integer is positive or zero, it is stored as it is: if it is negative, the computer takes the two's complement of the integer and then stores it.

Retrieving an integer in two's complement format

To retrieve an integer in two's complement representation, the computer follows the steps below:

- If the leftmost bit is 1, the computer applies the two's complement operation to the integer. If the leftmost bit is 0, no operation is applied.
- The computer changes the integer to decimal.

Example 3.12

Store the integer 28 in an 8-bit memory location using two's complement representation.

Solution

The integer is positive (no sign means positive), so after decimal to binary transformation no more action is needed. Note that three extra 0s are added to the left of the integer to make it eight bits:

Change 28 to 8-bit binary	0	0	0	1	1	1	0	0

Example 3.13

Store –28 in an 8-bit memory location using two's complement representation.

Solution

The integer is negative, so after changing to binary, the computer applies the two's complement operation on the integer:

Change 28 to 8-bit binary	0	0	0	1	1	1	0	0
Apply two's complement operation	1	1	1	0	0	1	0	0

Example 3.14

Retrieve the integer that is stored as 00001101 in memory in two's complement format.

Solution

The leftmost bit is 0, so the sign is positive. The integer is changed to decimal and the sign is added:

Leftmost bit is 0. The sign is positive.	0	0	0	0	1	1	0	1
Integer changed to decimal.								13
Sign is added.								+13

Example 3.15

Retrieve the integer that is stored as 11100110 in memory using two's complement format.

Solution

The leftmost bit is 1, so the integer is negative. The integer needs to be two's complemented before changing to decimal:

Leftmost bit is 1. The sign is negative.	1	1	1	0	0	1	1	0
Apply two's complement operation.	0	0	0	1	1	0	1	0
Integer changed to decimal.								26
Sign is added.								–26

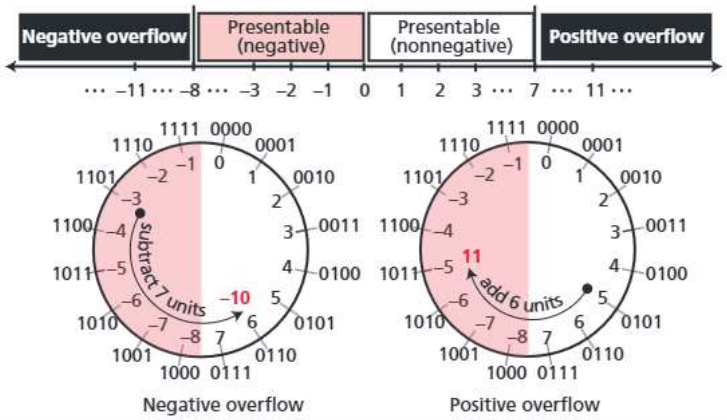
A very interesting point about two's complement is that there is only one zero in this representation. In sign-and-magnitude representation, there are two zeros (+0 and –0).

There is only one zero in two's complement notation.

Overflow in two's complement notation

Like other representations, integers stored in two's complement format are also subject to overflow. Figure 3.9 shows both positive and negative overflow when storing a signed integer in a 4-bit memory location. Positive overflow occurs when we try to store a positive integer larger than 7. For example, assume that we have stored an integer value 5 in a memory location and we then try to add 6 to the integer. We expect the result to be 11, but the computer's response is –5. The reason is if we start from 5 on the circular representation and move six units in the clockwise direction, we end up at –5. The positive overflow wraps the integer back to the range.

Figure 3.9 *Overflow in two's complement representation*



A negative overflow can happen when we try to store an integer that is less than –8, for example if we have stored –3 and try to subtract 7 from it. We expect the result to be –10, but the computer's response is +6. The reason is that if we start from –3 on a circular representation and go seven units in the counterclockwise direction, we end up at +6.

Applications of two's complement notation

Two's complement representation is the standard representation for storing integers in computers today. In the next chapter we will see why this is the case when we see the simplicity of operations using two's complement.

3.2.2 Comparison of the three systems

Table 3.1 shows a comparison between unsigned, two's complement, and sign-and-magnitude integers. A 4-bit memory location can store an unsigned integer between 0 and 15, and the same location can store two's complement signed integers between –8 and +7. It is very important that we store and retrieve an integer in the same format. For example, if the integer 13 is stored in signed format, it needs to be retrieved in signed format: the same integer is retrieved as –3 in two's complement format.

Table 3.1 Summary of integer representations

Contents of memory	*Unsigned*	*Sign-and-magnitude*	*Two's complement*
0000	0	0	+0
0001	1	1	+1
0010	2	2	+2
0011	3	3	+3
0100	4	4	+4
0101	5	5	+5
0110	6	6	+6
0111	7	7	+7
1000	8	–0	–8
1001	9	–1	–7
1010	10	–2	–6
1011	11	–3	–5
1100	12	–4	–4
1101	13	–5	–3
1110	14	–6	–2
1111	15	–7	–1

3.2.3 Storing reals

A **real** is a number with an integral part and a fractional part. For example, 23.7 is a real number—the integral part is 23 and the fractional part is 7/10. Although a fixed-point representation can be used to represent a real number, the result may not be accurate or it may not have the required precision. The next two examples explain why.

Example 3.16

In the decimal system, assume that we use a fixed-point representation with two digits at the right of the decimal point and fourteen digits at the left of the decimal point, for a total of sixteen digits. The precision of a real number in this system is lost if we try to represent a decimal number such as 1.00234: the system stores the number as 1.00.

Example 3.17

In the decimal system, assume that we use a fixed-point representation with six digits to the right of the decimal point and ten digits for the left of the decimal point, for a total of sixteen digits. The accuracy of a real number in this system is lost if we try to represent a decimal number such as 236154302345.00. The system stores the number as 6154302345.00: the integral part is much smaller than it should be.

Real numbers with very large integral parts or very small fractional parts should not be stored in fixed-point representation.

Floating-point representation

The solution for maintaining accuracy or precision is to use **floating-point representation**. This representation allows the decimal point to float: we can have different numbers of digits to the left or right of the decimal point. The range of real numbers that can be stored using this method increases tremendously: numbers with large integral parts or small fractional parts can be stored in memory. In floating-point representation, either decimal or binary, a number is made up of three sections, as shown in Figure 3.10.

Figure 3.10 *The three parts of a real number in floating-point representation*

Floating-point representation

The first section is the sign, either positive or negative. The second section shows how many places the decimal point should be shifted to the right or left to form the actual number. The third section is a fixed-point representation in which the position of the decimal is fixed.

A floating-point representation of a number is made up of three parts: a sign, a shifter, and a fixed-point number.

Floating-point representation is used in science to represent very small or very large decimal numbers. In this representation, which is called **scientific notation**, the fixed-point section has only one digit to the left of the decimal point and the shifter is the power of 10.

Example 3.18

The following shows the decimal number 7 425 000 000 000 000 000 000.00 in scientific notation (floating-point representation):

Solution

Actual number	→	+	7 425 000 000 000 000 000 000.00
Scientific notation	→	+	7.425×10^{21}

The three sections are the sign (+), the shifter (21), and the fixed-point part (7.425). Note that the shifter is the exponent. We can easily see the advantage of this. Even if we just want to write the number on a piece of paper, the scientific notation is shorter and takes less space. The notation uses the concept of floating-point because the position of the decimal point, which is near the right-hand end in the example, has moved 21 digits to the left to make the fixed-point part of the number. Some programming languages and calculators show the number as +7.425E21 because the base 10 is understood and does not need to be mentioned.

Example 3.19

Show the number –0.0000000000000232 in scientific notation.

Solution

We use the same approach as in the previous example—we move the decimal point after the digit 2, as shown below:

Actual number	→	–	0.0000000000000232
Scientific notation	→	–	2.32×10^{-14}

Note that the exponent is negative here because the decimal point in 2.32 needs to move to the left (fourteen positions) to form the original number. Again, we can say that the number in this notation is made of three parts: sign (–), the real number (2.32), and the negative integer (–14). Some programming languages and calculators show this as –2.32E–14.

Similar approaches have been used to represent very large or very small numbers (both integers and reals) in binary, to be stored in computers.

Example 3.20

Show the number $(10100100000000000000000000000000.00)_2$ in floating-point format.

Solution

We use the same idea, keeping only one digit to the left of the decimal point:

Actual number	→	+	$(10100100000000000000000000000000.00)_2$
Scientific notation	→	+	1.01001×2^{32}

Note that we don't have to worry about all those 0s at the right of the rightmost 1, because they are not significant when we use the real $(1.01001)_2$. The exponent is shown as 32, but it is actually stored in the computer in binary, as we will see shortly. We have also shown the sign as positive, but it would be stored as one bit.

Example 3.21

Show the number $-(0.00000000000000000000000101)_2$ in floating-point format.

Solution

We use the same idea, keeping only one non zero digit on the left-hand side of the decimal point:

Actual number	→	–	$(0.00000000000000000000000101)_2$
Scientific notation	→	–	1.01×2^{-24}

Note that exponent is stored as a negative binary in the computer.

Normalization

To make the fixed part of the representation uniform, both the scientific method (for the decimal system) and the floating-point method (for the binary system) use only one non-zero digit on the left of the decimal point. This is called **normalization**. In the decimal system this digit can be 1 to 9, while in the binary system it can only be 0 or 1. In the following, *d* is a non zero digit, *x* is a digit, and *y* is either 0 or 1:

Decimal	±	d.xxxxxxxxxxxxx	Note: d is 1 to 9 and each *x* is 0 to 9
Binary	±	1.yyyyyyyyyyyyy	Note: each *y* is 0 or 1

Sign, exponent, and mantissa

After a binary number is normalized, only three pieces of information about the number are stored: sign, exponent, and mantissa (the bits to the right of the decimal point). For example, +1000111.0101 becomes:

Sign	*Exponent*		*Mantissa*
+	2^6	×	1.0001110101
1	6		0001110101

Note that the point and the bit 1 to the left of the fixed-point section are not stored—they are implicit.

Sign

The sign of the number can be stored using 1 bit (0 or 1).

Exponent

The exponent (power of 2) defines the shifting of the decimal point. Note that the power can be negative or positive. The **Excess representation** (discussed later) is the method used to store the exponent.

Mantissa

The **mantissa** is the binary integer to the right of the decimal point. It defines the precision of the number. The mantissa is stored in fixed-point notation. If we think of the mantissa and the sign together, we can say this combination is stored as an integer in sign-and-magnitude format. However, we need to remember that it is not an integer—it is a fractional part that is stored like an integer. We emphasize this point because in a mantissa,

if we insert extra 0s to the *right* of the number, the value will not change, whereas in a real integer if we insert extra 0s to the *left* of the number, the value will not change.

The mantissa is a fractional part that, together with the sign, is treated like an integer stored in sign-and-magnitude representation.

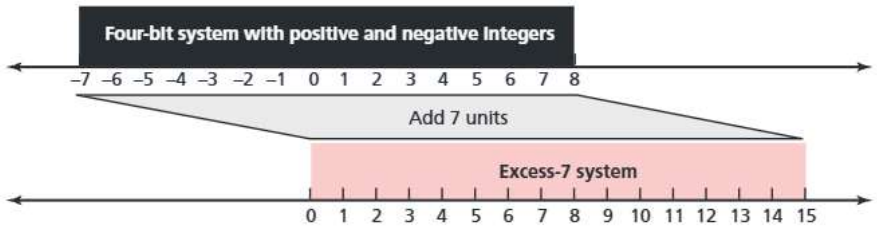
The Excess system

The mantissa can be stored as an unsigned integer. The exponent, the power that shows how many bits the decimal point should be moved to the left or right, is a signed number. Although this could have been stored using two's complement representation, a new representation, called the Excess system, is used instead. In the Excess system, both positive and negative integers are stored as unsigned integers. To represent a positive or negative integer, a positive integer (called a bias) is added to each number to shift them uniformly to the nonnegative side. The value of this bias is $2^{m-1} - 1$, where m is the size of the memory location to store the exponent.

Example 3.22

We can express sixteen integers in a number system with 4-bit allocation. Using one location for 0 and splitting the other fifteen (not quite equally) we can express integers in the range of –7 to 8, as shown in Figure 3.11. By adding seven units to each integer in this range, we can uniformly translate all integers to the right and make all of them positive without changing the relative position of the integers with respect to each other, as shown in the figure. The new system is referred to as Excess-7, or biased representation with biasing value of 7.

Figure 3.11 *Shifting in Excess representation*



The advantage of this new representation compared to that before the translation is that all integers in the Excess system are positive, so we don't need to be concerned about the sign when we are comparing or doing operations on the integers. For 4-bit allocation, the bias is $2^{4-1} - 1 = 7$, as we expected.

IEEE standards

The Institute of Electrical and Electronics Engineers (IEEE) has defined several standards for storing floating-point numbers. We discuss the two most common ones here, single

precision and double precision. These formats are shown in Figure 3.12. The numbers above the boxes are the number of bits for each field.

Figure 3.12 *IEEE standards for floating-point representation*

Single-precision format uses a total of 32 bits to store a real number in floating-point representation. The sign occupies one bit (0 for positive and 1 for negative), the exponent occupies eight bits (using a bias of 127), the mantissa uses 23 bits (unsigned number). This standard is sometimes referred to as Excess_127 because the bias is 127.

Double-precision format uses a total of 64 bits to store a real number in floating-point representation. The sign occupies one bit, the exponent occupies eleven bits (using a bias of 1023), and the mantissa uses 52 bits. The standard is sometimes referred to as Excess_1023 because the bias is 1023. Table 3.2 summarizes the specification of the two standards.

Table 3.2 Specifications of the two IEEE floating-point standards

Parameter	*Single Precision*	*Double Precision*
Memory location size (number of bits)	32	64
Sign size (number of bits)	1	1
Exponent size (number of bits)	8	11
Mantissa size (number of bits)	23	52
Bias (integer)	127	1023

Storage of IEEE standard floating-point numbers

A real number can be stored in one of the IEEE standard floating-point formats using the following procedure, with reference to Figure 3.12:

- Store the sign in S (0 or 1).
- Change the number to binary.
- Normalize.
- Find the values of E and M.
- Concatenate S, E, and M.

Example 3.23

Show the Excess_127 (single precision) representation of the decimal number 5.75.

Solution

a. The sign is positive, so S = 0.
b. Decimal to binary transformation: $5.75 = (101.11)_2$.
c. Normalization: $(101.11)_2 = (1.0111)_2 \times 2^2$.
d. $E = 2 + 127 = 129 = (10000001)_2$, M = 0111. We need to add 19 zeros at the right of M to make it 23 bits.
e. The presentation is shown below:

S	E	M
0	10000001	01110000000000000000000

The number is stored in the computer as 01000000101110000000000000000000.

Example 3.24

Show the Excess_127 (single precision) representation of the decimal number –161.875.

Solution

a. The sign is negative, so S = 1.
b. Decimal to binary transformation: $161.875 = (10100001.111)_2$.
c. Normalization: $(10100001.111)_2 = (1.0100001111)_2 \times 2^7$.
d. $E = 7 + 127 = 134 = (10000110)_2$ and $M = (0100001111)_2$.
e. Representation:

S	E	M
1	10000110	01000011110000000000000

The number is stored in the computer as 11000011010000111100000000000000.

Example 3.25

Show the Excess_127 (single precision) representation of the decimal number –0.0234375.

Solution

a. S = 1 (the number is negative).
b. Decimal to binary transformation: $0.0234375 = (0.0000011)_2$.
c. Normalization: $(0.0000011)_2 = (1.1)_2 \times 2^{-6}$.
d. $E = -6 + 127 = 121 = (01111001)_2$ and $M = (1)_2$.
e. Representation:

S	E	M
1	01111001	10000000000000000000000

The number is stored in the computer as 10111100110000000000000000000000.

Retrieving numbers stored in IEEE standard floating-point format

A number stored in one of the IEEE floating-point formats can be retrieved using the following method:

- Find the value of S, E, and M.
- If S = 0, set the sign to positive, otherwise, set the sign to negative.
- Find the shifter (E –127).
- Denormalize the mantissa.
- Change the denormalized number to binary to find the absolute value.
- Add the sign.

Example 3.26

The bit pattern $(11001010000000000111000100001111)_2$ is stored in memory in Excess_127 format. Show what is the value of the number in decimal notation.

Solution

a. The first bit represents S, the next eight bits, E and the remaining 23 bits, M:

S	E	M
1	10010100	00000000111000100001111

b. The sign is negative.

c. The shifter = E – 127 = 148 – 127 = 21.

d. Denormalization gives us $(1.00000000111000100001111)_2 \times 2^{21}$.

e. The binary number is $(10000000011100010000011.11)_2$.

f. The absolute value is 2 104 378.75.

g. The number is –2 104 378.75.

Example 3.27

The bit pattern 01000011111000000000000000000000 is stored in memory in Excess_127 format. Show the value of the number in decimal notation.

Solution

a. The first bit represents S, the next eight bits E, and the remaining 23 bits, M:

S	E	M
0	10000111	11000000000000000000000

b. The sign is positive.

c. The shifter = E – 127 = 135 – 127 = 8.

d. Denormalization gives us (1.11000000000000000000000)2 x 28.

e. The binary number is (111000000.00)2.

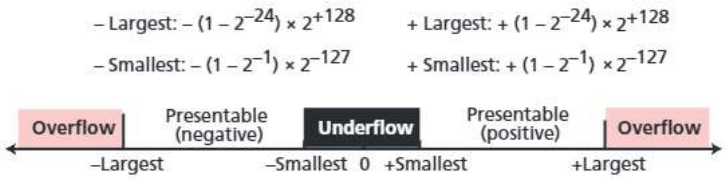
f. The absolute value is 448.

g. The number is +448.

Overflow and underflow

In the case of floating-point numbers, we can have both an overflow and **underflow**. Figure 3.13 shows the ranges of floating-point representations using 32-bit memory locations (Excess_127). This representation cannot store numbers with very small or very large absolute values. An attempt to store numbers with very small absolute values results in an underflow condition, while an attempt to store numbers with very large absolute values results to an overflow condition. We leave the calculation of boundary values (+largest, –largest, +smallest, and –smallest) as problems.

Figure 3.13 *Overflow and underflow in floating-point representation of reals*



Storing zero

You may have noticed that a real number with an integral part and the fractional part set to zero, that is, 0.0, cannot be stored using the steps discussed above. To handle this special case, it is agreed that in this case the sign, exponent, and the mantissa are set to 0s.

Truncation errors

When a real number is stored using floating-point representation, the value of the numbered stored may not be exactly as we expect it to be. For example, assume we need to store the number:

$$(1111111111111111.1111111111)_2$$

in memory using Excess_127 representation. After normalization, we have:

$$(1.1111111111111111111111111)_2$$

This means that the mantissa has 26 1s. This mantissa needs to be truncated to 23 1s. In other words, what is stored in the computer is:

$$(1.11111111111111111111111)_2$$

which means the original number is changed to:

$$(1111111111111111.11111111)_2$$

with the three 1s at the right of the fractional part truncated. The difference between the original number and what is retrieved is called the **truncation error**. This type of error is very important in areas in which very small or very large numbers are being used, such as

calculations in the space industry. In such cases we need to use larger memory locations and other presentations. The IEEE defines other standards with larger mantissas for these purposes.

3.3 STORING TEXT

A section of **text** in any language is a sequence of symbols used to represent an idea in that language. For example, the English language uses 26 symbols (A, B, C,..., Z) to represent uppercase letters, 26 symbols (a, b, c, ..., z) to represent lowercase letters, ten symbols (0, 1, 2, ..., 9) to represent numeric characters (not actual numbers—numbers are treated separately, as we explained in the previous section), and symbols (., ?, :, ; , ..., !) to represent punctuation. Other symbols such as blank, newline, and tab are used for text alignment and readability.

We can represent each symbol with a bit pattern. In other words, text such as 'CATS', which is made up from four symbols, can be represented as four *n*-bit patterns, each pattern defining a single symbol (Figure 3.14).

Figure 3.14 *Representing symbols using bit patterns*

Now the question is: how many bits are needed in a bit pattern to represent a symbol in a language? It depends on how many symbols are in the set used for the language. For example, if we create an imaginary language that uses only English uppercase letters, we need only 26 symbols. A bit pattern in this language needs to represent at least 26 symbols.

For another language, such as Chinese, we may need many more symbols. The length of the bit pattern that represents a symbol in a language depends on the number of symbols used in that language. More symbols mean a longer bit pattern.

Although the length of the bit pattern depends on the number of symbols, the relationship is not linear: it is logarithmic. If we need two symbols, the length is one bit ($\log_2 2$ is 1). If we need four symbols, the length is two bits ($\log_2 4$ is 2). Table 3.3 shows the relationship. A bit pattern of two bits can take four different forms: 00, 01, 10, and 11. Each of these forms can represent a symbol. In the same way, a bit pattern of three bits can take eight different forms: 000, 001, 010, 011, 100, 101, 110, and 111.

Table 3.3 Number of symbols and bit pattern length

Number of symbols	*Bit pattern length*	*Number of symbols*	*Bit pattern length*
2	1	128	7
4	2	256	8
8	3	65 536	16
16	4	4 294 967 296	32

3.3.1 Codes

Different sets of bit patterns have been designed to represent text symbols. Each set is called a **code**, and the process of representing symbols is called coding. In this section, we explain the common codes.

ASCII

The **American National Standards Institute (ANSI)** developed a code called **American Standard Code for Information Interchange (ASCII)**. This code uses seven bits for each symbol. This means that $2^7 = 128$ different symbols can be defined in this code. The full bit patterns for ASCII code are included in Appendix A. Today ASCII is part of Unicode, which is discussed next.

Unicode

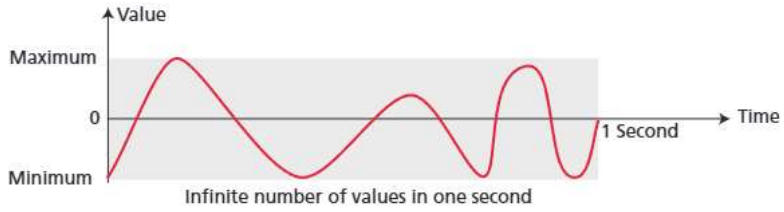
A coalition of hardware and software manufacturers have designed a code called **Unicode** that uses 32 bits and can therefore represent up to $2^{32} = 4\,294\,967\,296$ symbols. Different sections of the code are allocated to symbols from different languages in the world. Some parts of the code are used for graphical and special symbols. A brief set of Unicode symbols is listed in Appendix A. ASCII is part of Unicode today.

3.4 STORING AUDIO

Audio is a representation of sound or music. Audio, by nature, is different from the numbers or text we have discussed so far. Text is composed of countable entities (characters): we can count the number of characters in text. Text is an example of **digital** data. In contrast, audio is not countable. Audio is an entity that changes with time—we can only measure the intensity of the sound at each moment. When we discuss storing audio in computer memory, we mean storing the intensity of an audio signal, such as the signal from a microphone, over a period of time: one second, one hour.

Audio is an example of **analog** data. Even if we are able to measure all its values in a period of time, we cannot store these in the computer's memory, as we would need an infinite number of memory locations. Figure 3.15 shows the nature of an analog signal, such as audio, that varies with time.

Figure 3.15 *An audio signal*



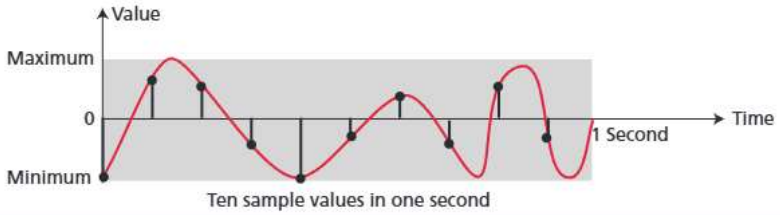
3.4.1 Sampling

If we cannot record all the values of an audio signal over an interval, we *can* record some of them. **Sampling** means that we select only a finite number of points on the analog signal, measure their values, and record them. Figure 3.16 shows a selection of ten samples from the signal: we can then record these values to represent the analog signal.

Sampling rate

The next logical question is, how many samples do we need in each second to be able to retrieve a replica of the original signal? The number of samples depends on the maximum number of changes in the analog signal. If the signal is smooth, we need fewer samples: if the signal is changing rapidly, we need more samples. It has been shown that a **sampling rate** of 40 000 samples per second is good enough to reproduce an audio signal.

Figure 3.16 *Sampling an audio signal*



3.4.2 Quantization

The value measured for each sample is a real number. This means that we can store 40 000 real values for each one second sample. However, it is simpler to use an unsigned number (a bit pattern) for each sample. **Quantization** refers to a process that rounds the value of a sample to the closest integer value. For example, if the real value is 17.2, it can be rounded down to 17: if the value is 17.7, it can be rounded up to 18.

3.4.3 Encoding

The next task is encoding. The quantized sample values need to be encoded as bit patterns. Some systems assign positive and negative values to samples, some just shift the curve to the positive part and assign only positive values. In other words, some systems use an unsigned integer to represent a sample, while others use signed integers to do so. However, the signed integers don't have to be in two's complement, they can be sign-and-magnitude values. The leftmost bit is used to represent the sign (0 for positive values and 1 for negative values), and the rest of the bits are used to represent the absolute values.

Bit per sample

The system needs to decide how many bits should be allocated for each sample. Although in the past only 8 bits were assigned to sound samples, today 16, 24, or even 32 bits per sample is normal. The number of bits per sample is sometimes referred to as the **bit depth**.

Bit rate

If we call the bit depth or number of bits per sample B, the number of samples per second, S, we need to store S × B bits for each second of audio. This product is sometimes referred to as **bit rate**, R. For example, if we use 40 000 samples per second and 16 bits per sample, the bit rate is R = 40 000 × 16 = 640 000 bits per second = 640 kilobits per second.

3.4.4 Standards for sound encoding

Today the dominant standard for storing audio is MP3 (short for MPEG Layer 3). This standard is a modification of the MPEG (Motion Picture Experts Group) compression method used for video. It uses 44 100 samples per second and 16 bits per sample. The result is a signal with a bit rate of 705 600 bits per second, which is compressed using a compression method that discards information that cannot be detected by the human ear. This is called lossy compression, as opposed to lossless compression: see Chapter 15.

3.5 STORING IMAGES

Images are stored in computers using two different techniques: raster graphics and vector graphics.

3.5.1 Raster graphics

Raster graphics (or **bitmap graphics**) is used when we need to store an analog image such as a photograph. A photograph consists of analog data, similar to audio information: the difference is that the intensity (color) of data varies in space instead of in time. This means that data must be sampled. However, sampling in this case is normally called **scanning**. The samples are called **pixels** (which stands for **picture elements**). In other words, the whole image is divided into small pixels where each pixel is assumed to have a single intensity value.

Resolution

Just like audio sampling, in image scanning we need to decide how many pixels we need to record for each square or linear inch. The scanning rate in image processing is called **resolution**. If the resolution is sufficiently high, the human eye cannot recognize the discontinuity in reproduced images.

Color depth

The number of bits used to represent a pixel, its **color depth**, depends on how a pixel's color is handled by different encoding techniques. The perception of color is how our eyes respond to a beam of light. Our eyes have different types of *photoreceptor* cells: some respond to the three primary colors red, green, and blue (often called **RGB**), while others merely respond to the intensity of light.

True-Color

One of the techniques used to encode a pixel is called **True-Color**, which uses 24 bits to encode a pixel. In this technique, each of the three primary colors (RGB) are represented

by eight bits. Since an 8-bit pattern can represent a number between 0 to 255 in this technique, each color is represented by three decimal numbers between 0 to 255. Table 3.4 shows the three values for some of the colors in this technique.

Table 3.4 Some colors defined in True-Color

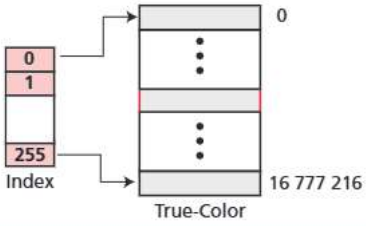
Color	*Red*	*Green*	*Blue*	*Color*	*Red*	*Green*	*Blue*
Black	0	0	0	Yellow	255	255	0
Red	255	0	0	Cyan	0	255	255
Green	0	255	0	Magenta	255	0	255
Blue	0	0	255	White	255	255	255

Note that the True-Color scheme can encode 2^{24} or 16 776 216 colors. In other words, the color intensity of each pixel is one of these values.

Indexed color

The True-Color scheme uses more than 16 million colors. Many applications do not need such a large range of colors. The **indexed color**—or **palette color**—scheme uses only a portion of these colors. In this scheme each application selects a few (normally 256) colors from the large set of colors and indexes them, assigning a number between 0 and 255 to each selected color. This is similar to the way in which an artist might have a great many colors in their studio, but at each moment use only a few on their palette. Figure 3.17 illustrates the idea of indexed color.

Figure 3.17 *Relationship of the indexed color scheme to the True-Color scheme*



The use of indexing reduces the number of bits required to store a pixel. For example, in the True-Color scheme 24 bits are needed to store a single pixel. The index color scheme normally uses 256 indexes, which needs only eight bits to store the same pixel. For example, a high-quality digital camera uses almost three million pixels for a 3 × 5 inch photo. The following shows the number of bits that need to be stored using each scheme:

True-Color:	**3 000 000**	×	**24**	=	**72 000 000**
Indexed-Color:	**3 000 000**	×	**8**	=	**24 000 000**

5/1/25, 9:57 PM study viewer

Standards for image encoding

Several de facto standards for image encoding are in use. **JPEG (Joint Photographic Experts Group)** uses the True-Color scheme, but compresses the image to reduce the number of bits (see Chapter 15). **GIF (Graphic Interchange Format)**, on the other hand, uses the indexed color scheme.

3.5.2 Vector graphics

Raster graphics has two disadvantages: the file size is big and rescaling is troublesome. To enlarge a raster graphics image means enlarging the pixels, so the image looks ragged when it is enlarged. The **vector graphic** image encoding method, however, does not store the bit patterns for each pixel. An image is decomposed into a combination of geometrical shapes such as lines, squares, or circles. Each geometrical shape is represented by a mathematical formula. For example, a line may be described by the coordinates of its endpoints, and a circle may be described by the coordinates of its center and the length of its radius. A vector graphic image is made up from a series of commands that defines how these shapes should be drawn.

When the image is to be displayed or printed, the size of the image is given to the system as an input. The system rescales the image to the new size and uses the same formulae to draw the image. In this case, each time an image is drawn, the formulae are reevaluated. For this reason, vector graphics are also called geometric modeling or object-oriented graphics.

For example, consider a circle of radius r. The main pieces of information a program needs to draw this circle are:

1. The radius r and equation of a circle.
2. The location of the center point of the circle.
3. The stroke line style and color.
4. The fill style and color.

When the size of the circle is changed, the program changes the value of the radius and recalculates the information to draw the circle again. Rescaling does not change the quality of the drawing.

Vector graphics is not suitable for storing the subtleties of photographic images. JPEG or GIF raster graphics provide much better and more vivid pictures. Vector graphics *is* suitable for applications that use mainly geometric primitives to create images. It is used in applications such as FLASH, and to create TrueType (Microsoft, Apple) and PostScript (Adobe) fonts. Computer-aided design (CAD) also uses vector graphics for engineering drawings.

3.6 STORING VIDEO

Video is a representation of images (called **frames**) over time. A movie consists of a series of frames shown one after another to create the illusion of motion. In other words, video is the representation of information that changes in space (single image) and in time (a series of images). So, if we know how to store an image inside a computer, we also know how to store video: each image or frame is transformed into a set of bit patterns and stored. The combination of the images then represents the

https://studylib.net/doc/26059254/foundations-computer-science-4th 85/706

video. Today video is normally compressed. In Chapter 15 we discuss MPEG, a common video compression technique.

3.7 END-CHAPTER MATERIALS

3.7.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Halsall, F. *Multimedia Communication*, Boston, MA: Addison-Wesley, 2001
- Koren, I. *Computer Arithmetic Algorithms*, Natick, MA: A K Peters, 2001
- Long, B. *Complete Digital Photography*, Hignham, MA: Charles River Media, 2003
- Mano, M. *Computer System Architecture*, Upper Saddle River, NJ: Prentice-Hall, 1993
- Miano, J. *Compressed Image File Formats*, Boston, MA: Addison-Wesley, 1999

3.7.2 Key terms

American National Standards Institute (ANSI) 61	one's complement 47
American Standard Code for Information Interchange (ASCII) 61	overflow 44
analog 61	palette color 64
audio 61	picture element 63
binary digit 40	pixel 63
bit 40	quantization 62
bit depth 62	raster graphic 63
bitmap graphic 63	real 51
bit pattern 40	resolution 63
bit rate 63	RGB 63
byte 41	sampling 62
code 61	sampling rate 62
color depth 63	scanning 63
digital 61	scientific notation 52
Excess_1023 56	sign-and-magnitude representation 44
Excess_127 56	text 60
Excess representation 54	text editor 41
floating-point representation 52	True-Color 63
frames 65	truncation error 59

Graphic Interchange Format (GIF) 65	two's complement 48
indexed color 64	two's complement representation 47
Joint Photographer Experts Group (JPEG) 65	underflow 59
mantissa 54	Unicode 61
MP3 63	unsigned integer 42
MPEG 63	vector graphic 65
normalization 54	video 65

3.7.3 Summary

- Data comes in different forms, including numbers, text, audio, image, and video. All data types are transformed into a uniform representation called a bit pattern.
- A number is changed to the binary system before being stored in computer memory. There are several ways to handle the sign. There are two ways to handle the decimal point: fixed-point and floating-point. An integer can be thought of as a number in which the position of the decimal point is fixed: the decimal point is at the right of the least significant bit. An unsigned integer is an integer that can never be negative. One of the methods used to store a signed integer is the sign-and-magnitude format. In this format, the leftmost bit is used to show the sign and the rest of the bits define the magnitude. Sign and magnitude are separated from each other. Almost all computers use the two's complement representation to store a signed integer in an n-bit memory location. In this method, the available range for unsigned integers is divided into two equal subranges. The first half is used to represent non negative integers, the second half is used to represent negative integers. In two's complement representation, the leftmost bit defines the sign of the integer, but sign and magnitude are not separated from each other. A real is a number with an integral part and a fractional part. Real numbers are stored in the computer using floating-point representation. In floating-point representation a number is made up of three sections: a sign, a shifter, and a fixed-point number.
- A piece of text in any language is a sequence of symbols. We can represent each symbol with a bit pattern. Different sets of bit patterns (codes) have been designed to represent text symbols. A coalition of hardware and software manufacturers have designed a code called Unicode that uses 32 bits to represent a symbol.
- Audio is a representation of sound or music. Audio is analog data. We cannot record an infinite number of values in an interval, we can only record some samples. The number of samples depends on the maximum number of changes in the analog signal. The values measured for each sample is a real number. Quantization refers to a process that rounds up the sample values to integers.
- Storage of images is done using two different techniques: raster graphics and vector graphics. Raster graphics are used when we need to store an analog image such as a photograph. The image is scanned (sampled) and pixels are stored. In the vector graphic method, an image is decomposed into a combination of geometrical shapes

such as lines, squares, or circles. Each geometrical shape is represented by a mathematical formula.

- Video is a representation of images (called frames) in time. A movie is a series of frames shown one after another to create the illusion of continuous motion. In other words, video is the representation of information that changes in space (single image) and in time (a series of images).

3.8 PRACTICE SET

3.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

3.8.2 Review questions

Q3-1. Name five types of data that a computer can process.

Q3-2. How is bit pattern length related to the number of symbols the bit pattern can represent?

Q3-3. How does the bitmap graphic method represent an image as a bit pattern?

Q3-4. What is the advantage of the vector graphic method over the bitmap graphic method? What is the disadvantage?

Q3-5. What steps are needed to convert audio data to bit patterns?

Q3-6. Compare and contrast the representation of positive integers in unsigned, sign-and-magnitude format, and two's complement format.

Q3-7. Compare and contrast the representation of negative integers in sign-and-magnitude and two's complement format.

Q3-8. Compare and contrast the representation of zero in sign-and-magnitude, two's complement, and Excess formats.

Q3-9. Discuss the role of the leftmost bit in sign-and-magnitude, and two's complement formats.

Q3-10. Answer the following questions about floating-point representations of real numbers:

a. Why is normalization necessary?

b. What is the mantissa?

c. After a number is normalized, what kind of information does a computer store in memory?

3.8.3 Problems

P3-1. How many distinct 5-bit patterns can we have?

P3-2. In some countries vehicle license plates have two decimal digits (0 to 9). How many distinct plates can we have? If the digit 0 is not allowed on the license plate, how many distinct plates can we have?

P3-3. Redo Problem P3-2 for a license plate that has two digits followed by three uppercase letters (A to Z).

P3-4. A machine has eight different cycles. How many bits are needed to represent each cycle?

P3-5. A student's grade in a course can be A, B, C, D, F, W (withdraw), or I (incomplete). How many bits are needed to represent the grade?

P3-6. A company has decided to assign a unique bit pattern to each employee. If the company has 900 employees, what is the minimum number of bits needed to create this system of representation? How many patterns are unassigned? If the company hires another 300 employees, should it increase the number of bits? Explain your answer.

P3-7. If we use a 4-bit pattern to represent the digits 0 to 9, how many bit patterns are wasted?

P3-8. An audio signal is sampled 8000 times per second. Each sample is represented by 256 different levels. How many bits per second are needed to represent this signal?

P3-9. Change the following decimal numbers to 8-bit unsigned integers.

a. 23
b. 121
c. 34
d. 342

P3-10. Change the following decimal numbers to 16-bit unsigned integers.

a. 41
b. 411
c. 1234
d. 342

P3-11. Change the following decimal numbers to 8-bit two's complement integers.

a. –12
b. –145
c. 56
d. 142

P3-12. Change the following decimal numbers to 16-bit two's complement integers.

a. 102
b. –179
c. 534
d. 62,056

P3-13. Change the following 8-bit unsigned numbers to decimal.

a. 01101011
b. 10010100
c. 00000110
d. 01010000

P3-14. Change the following 8-bit two's complement numbers to decimal.

a. 01110111
b. 11111100
c. 01110100
d. 11001110

P3-15. The following are two's complement binary numbers. Show how to change the sign of the number.

a. 01110111
b. 11111100
c. 01110111
d. 11001110

P3-16. If we apply the two's complement operation to a number twice, we should get the original number. Apply the two's complement operation to each of the following numbers and see if we can get the original number.

a. 01110111
b. 11111100
c. 01110100
d. 11001110

P3-17. Normalize the following binary floating-point numbers. Explicitly show the value of the exponent after normalization.

a. 1.10001
b. $2^3 \times 111.1111$
c. $2^{-2} \times 101.110011$
d. $2^{-5} \times 101101.00000110011000$

P3-18. Convert the following numbers in 32-bit IEEE format.

a. $-2^0 \times 1.10001$
b. $+2^3 \times 1.111111$
c. $+2^{-4} \times 1.01110011$
d. $-2^{-5} \times 1.01101000$

P3-19. Convert the following numbers in 64-bit IEEE format.

a. $-2^0 \times 1.10001$
b. $+2^3 \times 1.111111$
c. $+2^{-4} \times 1.01110011$
d. $-2^{-5} \times 1.01101000$

P3-20. Convert the following numbers in 32-bit IEEE format.

a. 7.1875
b. −12.640625
c. 11.40625
d. −0.375

P3-21. The following are sign-and-magnitude binary numbers in a 8-bit allocation. Convert them to decimal.

a. 01110111
b. 11111100
c. 01110100
d. 11001110

P3-22. Convert the following decimal integers to sign-and-magnitude with 8-bit allocation.

a. 53
b. −107
c. −5
d. 154

P3-23. One method of representing signed numbers in a computer is **one's complement representation**. In this representation, to represent a positive number, we store the binary number. To represent a negative number, we apply the one's complement operation on the number. Store the following decimal integers to one's complement with 8-bit allocation.

a. 53
b. −107
c. −5
d. 154

P3-24. The following are one's complement binary numbers in a 8-bit allocation. Convert them to decimal.

a. 01110111
b. 11111100
c. 01110100
d. 11001110

P3-25. If we apply the one's complement operation to a number twice, we should get the original number. Apply the one's complement operation twice to each of the following numbers and see if you can get the original number.

a. 01110111
b. 11111100
c. 01110100
d. 11001110

P3-26. An alternative method to find the two's complement of a number is to first take the one's complement of the number and then add 1 to the result. (Adding binary

integers is explained in Chapter 4). Try both methods using the following numbers. Compare and contrast the results.

a. 01110111
b. 11111100
c. 01110100
d. 11001110

P3-27. The equivalent of one's complement in the binary system is nine's complement in the decimal system (1 = 2 – 1 and 9 = 10 – 1). With n-digit allocation, we can represent nine's complement numbers in the range of: $-[(10^n/2) - 1]$ to $+[(10^n/2 - 1)]$. The nine's complement of a number with n digit allocation is obtained as follows. If the number is positive, the nine's complement of the number is itself. If the number is negative, we subtract each digit from 9. Answer the following questions for three-digit allocation:

a. What is the range of the numbers we can represent using nine's complement?
b. In this system, how can we determine the sign of a number?
c. Do we have two zeros in this system?
d. If the answer to c. is yes, what is the representation for +0 and –0?

P3-28. Assuming three-digit allocation, find the nine's complement of the following decimal numbers.

a. +234
b. +560
c. –125
d. –111

P3-29. The equivalent of two's complement in the binary system is ten's complement in the decimal system (in the binary system, 2 is the base, in the decimal system, 10 is the base). Using n-digit allocation, we can represent numbers in the range of:

$-(10^n/2)$ to $+(10^n/2 - 1)$

in ten's complement format. The ten's complement of a number with n-digit allocation is obtained by first finding the nine's complement of the number and then adding 1 to the result. Answer the following questions for three-digit allocation.

a. What is the range of the numbers we can represent using ten's complement?
b. In this system, how can we determine the sign of a number?
c. Do we have two zeros in this system?
d. If the answer to c. is yes, what is the representation for +0 and –0?

P3-30. Assuming three-digit allocation, find the ten's complement of the following decimal numbers.

a. +234
b. +560
c. –125
d. –111

P3-31. The equivalent of one's complement in the binary system is fifteen's complement in the hexadecimal system (1 = 2 – 1 and 15 = 16 – 1).

a. What range of numbers can we represent with three-digit allocation in fifteen's complement?
b. Explain how the fifteen's complement of a number is obtained in the hexadecimal system.

c. Do we have two zeros in this system?

d. If the answer to c. is yes, what is the representation for +0 and –0?

P3-32. Assuming three-digit allocation, find the fifteen's complement of the following hexadecimal numbers.

a. +B14

b. +FE1

c. –1A

d. –1E2

P3-33. The equivalent of two's complement in the binary system is sixteen's complement in the hexadecimal system.

a. What range of numbers can we represent with three-digit allocation in sixteen's complement?

b. Explain how a sixteen's complement of a number is obtained in the hexadecimal system.

c. Do we have two zeros in this system?

d. If the answer to c. is yes, what is the representation for +0 and –0?

P3-34. Assuming three-digit allocation, find the sixteen's complement of the following hexadecimal numbers.

a. +B14

b. +FE1

c. –1A

d. –1E2

CHAPTER 4

Operations on Data

In Chapter 3 we showed how to store different types of data in a computer. In this chapter, we show how to operate on data stored in a computer. Operations on data can be divided into three broad categories: logic operations, shift operations, and arithmetic operations.

Objectives

After studying this chapter, the student should be able to:

- List the three categories of operations performed on data.
- Perform unary and binary logic operations on bit patterns.
- Distinguish between logic shift operations and arithmetic shift operations.
- Perform logic shift operations on bit patterns.
- Perform arithmetic shift operations on integers stored in two's complement format.
- Perform addition and subtraction on integers when they are stored in two's complement format.
- Perform addition and subtraction on integers when they are stored in sign-and-magnitude format.
- Perform addition and subtraction operations on reals when they are stored in floating-point format.
- Understand some applications of logical and shift operations such as setting, unsetting, and flipping specific bits.

4.1 LOGIC OPERATIONS

In Chapter 3 we discussed the fact that data inside a computer is stored as patterns of bits. Logic operations refer to those operations that apply the same basic operation on individual bits of a pattern, or on two corresponding bits in two patterns. This means that we can define logic operations at the bit level and at the pattern level. A logic operation at the pattern level is *n* logic operations, of the same type, at the bit level where *n* is the number of bits in the pattern.

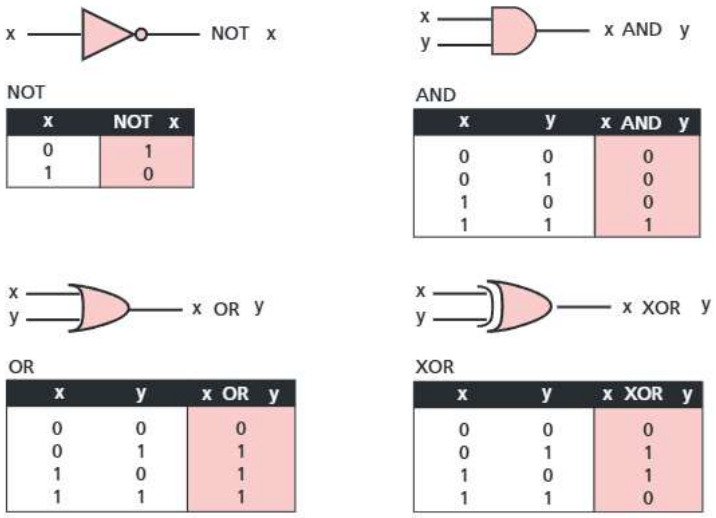
4.1.1 Logic operations at bit level

A bit can take one of the two values: 0 or 1. If we interpret 0 as the value *false* and 1 as the value *true*, we can apply the operations defined in **Boolean algebra** to manipulate bits. Boolean algebra, named in honor of George Boole, belongs to a special field of mathematics called *logic*. Boolean algebra and its application to building logic circuits in computers are briefly discussed in Appendix E. In this section, we show briefly four bit-level operations that are used to manipulate bits: NOT, AND, OR, and XOR.

Boolean algebra and logic circuits are discussed in Appendix E.

Figure 4.1 shows the symbols for these four bit-level operators and their truth tables. A **truth table** defines the values of output for each possible input or inputs. Note that the output of each operator is always one bit, but the input can be one or two bits.

Figure 4.1 *Logic operations at the bit level*



NOT

x	NOT x
0	1
1	0

AND

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	x OR y
0	0	0
0	1	1
1	0	1
1	1	1

XOR

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

NOT

The NOT operator is a unary operator: it takes only one input. The output bit is the complement of the input. If the input is 0, the output is 1, if the input is 1, the output is 0. In other words, the NOT operator flips its input. The truth table of the NOT operator has only two rows because the single input can be either 0 or 1: two possibilities.

AND

The AND operator is a binary operator: it takes two inputs. The output bit is 1 if both inputs are 1s and the output is 0 in the other three cases. The truth table of the AND operator has four rows because, with two inputs, there are four possible input combinations.

A property

One interesting point about the AND operator is that if a bit in one input is 0, we do not have to check the corresponding bit in the other input: we can quickly conclude that the result is 0. We use this property when we discuss the application of this operator in relation to a bit pattern:

$$\text{For } x = 0 \text{ or } 1 \qquad x \text{ AND } 0 \rightarrow 0 \qquad \text{and} \qquad 0 \text{ AND } x \rightarrow 0$$

OR

The OR operator is a also a binary operator: it takes two inputs. The output bit is 0 if both inputs are 0s and the output is 1 in the other three cases. The truth table of the OR operator has also four rows. The OR operator is sometimes called the *inclusive-or operator* because the output is 1 not only when one of the inputs is 1, but also when both inputs are 1s. This is in contrast to the operator we introduce next.

A property

One interesting point about the OR operator is that if a bit in one input is 1, we do not have to check the corresponding bit in the other input: we can quickly conclude that the result is 1. We use this property when we discuss the application of this operator in relation to a bit pattern:

$$\text{For } x = 0 \text{ or } 1 \qquad x \text{ OR } 1 \rightarrow 1 \qquad \text{and} \qquad 1 \text{ OR } x \rightarrow 1$$

XOR

The XOR operator (pronounced 'exclusive-or') is also a binary operator like the OR operator, with only one difference: the output is 0 if both inputs are 1s. We can look at this operator in another way: the output is 0 when both inputs are the same, and the output is 1 when the inputs are different.

Example 4.1

In English we use the conjunction 'or' sometimes to mean an inclusive-or, and sometimes to mean an exclusive-or.

a. The sentence 'I wish to have a car *or* a house' uses 'or' in the inclusive sense—I wish a car, a house, or both.

b. The sentence 'Today is either Monday or Tuesday' uses 'or' in the exclusive sense—today is either Monday or Tuesday, but it cannot be both.

Example 4.2

The XOR operator is not actually a new operator. We can always simulate it using the other three operators. The following two expressions are equivalent:

$$x \text{ XOR } y \quad \leftrightarrow \quad [x \text{ AND } (\text{NOT } y)] \text{ OR } [(\text{NOT } x) \text{ AND } y]$$

The equivalence can be proved if we make the truth table for both.

A property

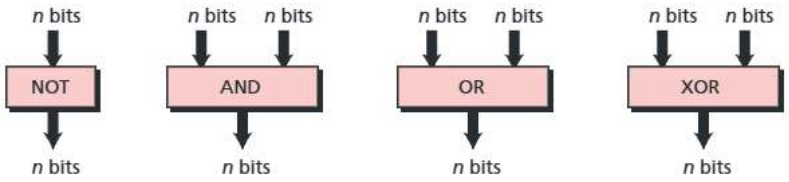
A property of XOR is that if a bit in one input is 1, the result is the complement of the corresponding bit in the other input. We use this property when we discuss the application of this operator in relation to a bit pattern:

$$\text{For } x = 0 \text{ or } 1 \qquad 1 \text{ XOR } x \rightarrow \text{NOT } x \qquad \text{and} \qquad x \text{ XOR } 1 \rightarrow \text{NOT } x$$

4.1.2 Logic operations at pattern level

The same four operators (NOT, AND, OR, and XOR) can be applied to an n-bit pattern. The effect is the same as applying each operator to each individual bit for NOT and to each corresponding pair of bits for other three operators. Figure 4.2 shows these four operators with input and output patterns.

Figure 4.2 *Logic operators applied to bit patterns*



Example 4.3

Use the NOT operator on the bit pattern 10011000.

Solution

The solution is shown below. Note that the NOT operator changes every 0 to 1 and every 1 to 0:

NOT	1	0	0	1	1	0	0	0	Input
	0	1	1	0	0	1	1	1	Output

Example 4.4

Use the AND operator on the bit patterns 10011000 and 00101010.

Solution

The solution is shown below. Note that only one bit in the output is 1, where both corresponding inputs are 1s:

	1	0	0	1	1	0	0	0	Input 1
AND	0	0	1	0	1	0	1	0	Input 2
	0	0	0	0	1	0	0	0	Output

Example 4.5

Use the OR operator on the bit patterns 10011001 and 00101110.

Solution

The solution is shown below. Note that only one bit in the output is 0, where both corresponding inputs are 0s:

	1	0	0	1	1	0	0	1	Input 1
OR	0	0	1	0	1	1	1	0	Input 2
	1	0	1	1	1	1	1	1	Output

Example 4.6

Use the XOR operator on the bit patterns 10011001 and 00101110.

Solution

The solution is shown below. Compare the output in this example with the one in Example 4.5. The only difference is that when the two inputs are 1s, the result is 0 (the effect of exclusion):

	1	0	0	1	1	0	0	1	Input 1
XOR	0	0	1	0	1	1	1	0	Input 2
	1	0	1	1	0	1	1	1	Output

Applications

Four logic operations can be used to modify a bit pattern.

Complementing

The only application of the NOT operator is to complement the whole pattern. Applying this operator to a pattern changes every 0 to 1 and every 1 to 0. This is sometimes referred to as a one's complement operation. Example 4.3 shows the effect of complementing.

Unsetting specific bits

One of the applications of the AND operator is to unset (force to 0) specific bits in a bit pattern. The second input in this case is called a **mask**. The 0-bits in the mask unset the

corresponding bits in the first input: the 1-bits in the mask leave the corresponding bits in the first input unchanged. This is due to the property we mentioned for the AND operator: if one of the inputs is 0, the output is 0 no matter what the other input is. Unsetting the bits in a pattern can have many applications. For example, if an image is using only one bit per pixel (a black and white image), then we can make a specific pixel black using a mask and the AND operator.

Example 4.7

Use a mask to unset (clear) the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution

The mask is 00000111. The result of applying the mask is:

	1	0	1	0	0	1	1	0	Input
AND	0	0	0	0	0	1	1	1	Mask
	0	0	0	0	0	1	1	0	Output

Note that the three rightmost bits remain unchanged, while the five leftmost bits are unset (changed to 0) no matter what their previous values.

Setting specific bits

One of the applications of the OR operator is to set (force to 1) specific bits in a bit pattern. Again we can use a mask, but a different one. The 1-bits in the mask set the corresponding bits in the first input, and the 0-bits in the mask leave the corresponding bits in the first input unchanged. This is due to the property we mentioned for the OR operator: if one of the inputs is 1, the output is 1 no matter what the other input is. Setting the bits in a pattern has many applications. For example, if an image is using only one bit per pixel (a black and white image), then we can make a specific pixel white using a mask and the OR operator.

Example 4.8

Use a mask to set the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution

The mask is 11111000. The result of applying the mask is:

	1	0	1	0	0	1	1	0	Input
OR	1	1	1	1	1	0	0	0	Mask
	1	1	1	1	1	1	1	0	Output

Flipping specific bits

One of the applications of the XOR operator is to flip (complement) specific bits in a bit pattern. Again we can use a mask, but a different one. The 1-bits in the mask flip the

corresponding bits in the first input, and the 0-bits in the mask leave the corresponding bits in the first input unchanged. This is due to the property we mentioned for the XOR operator: if one of the inputs is 1, the output is the complement of the corresponding bit. Note the difference between the NOT operator and the XOR operator. The NOT operator complements all the bits in the input, while the XOR operator complements only the specific bits in the first input as defined by the mask.

Example 4.9

Use a mask to flip the five leftmost bits of a pattern. Test the mask with the pattern 10100110.

Solution

The mask is 11111000. The result of applying the mask is:

	1	0	1	0	0	1	1	0	Input 1
XOR	1	1	1	1	1	0	0	0	Mask
	0	1	0	1	1	1	1	0	Output

4.2 SHIFT OPERATIONS

Shift operations move the bits in a pattern, changing the positions of the bits. They can move bits to the left or to the right. We can divide shift operations into two categories: logical shift operations and arithmetic shift operations.

4.2.1 Logical shift operations

A **logical shift operation** is applied to a pattern that does not represent a signed number. The reason is that these shift operation may change the sign of the number that is defined by the leftmost bit in the pattern. We distinguish two types of logical shift operations, as described below.

Simple shift

A simple right shift operation shifts each bit one position to the right. In an n-bit pattern, the rightmost bit is lost and a 0 fills the leftmost bit. A simple left shift operation shifts each bit one position to the left. In an n-bit pattern, the leftmost bit is lost and a 0 fills the rightmost bit. Figure 4.3 shows the simple right shift and simple left shift operations for an 8-bit pattern.

Figure 4.3 *Simple shift operations*

a. Simple right shift b. Simple left shift

Example 4.10

Use a simple left shift operation on the bit pattern 10011000.

Solution

The solution is shown below. The leftmost bit (white in the black background) is lost and a 0 is inserted as the rightmost bit (the bit in color):

←	1	0	0	1	1	0	0	0	Original
	0	0	1	1	0	0	0	0	After shift

Circular shift

A **circular shift operation** (or rotate operation) shifts bits, but no bit is lost or added. A circular right shift (or *right rotate*) shifts each bit one position to the right. The rightmost bit is circulated and becomes the leftmost bit. A circular left shift (or *left rotate*) shifts each bit one position to the left. The leftmost bit circulates and become the rightmost bit. Figure 4.4 shows the circular shift left and circular shift right operation.

Figure 4.4 *Circular shift operations*

a. Circular right shift b. Circular left shift

Example 4.11

Use a circular left shift operation on the bit pattern 10011000.

Solution

The solution is shown below. The leftmost bit (the white bit in the black background) is circulated and becomes the rightmost bit:

Original	1	0	0	1	1	0	0	0	Circular
After shift	0	0	1	1	0	0	0	1	

Example 4.12

Combining logic operations and logical shift operations gives us some tools for manipulating bit patterns. Assume that we have a pattern and we need to use the third bit (from the right) of this pattern in a decision-making process. We want to know if this particular bit is 0 or 1. The following shows how we can find out:

	h	g	f	e	d	c	b	a	
	0	h	g	f	e	d	c	b	One right shift
	0	0	h	g	f	e	d	c	Two right shifts
AND	0	0	0	0	0	0	0	1	Mask
	0	0	0	0	0	0	0	c	Result

We shift the pattern two bits to the right so that the target bit moves to the rightmost position. The result is then ANDed with a mask which has one 1 at the rightmost position. The result is a pattern with seven 0s and the target bit at the rightmost position. We can then test the result: if it is an unsigned integer 1, the target bit was 1, whereas if the result is an unsigned integer 0, the target bit was 0.

Arithmetic shift operations

Arithmetic shift operations assume that the bit pattern is a signed integer in two's complement format. Arithmetic right shift is used to divide an integer by two, while arithmetic left shift is used to multiply an integer by two (discussed later). These operations should not change the sign (leftmost) bit. An arithmetic right shift retains the sign bit, but also copies it into the next right bit, so that the sign is preserved. An arithmetic left shift discards the sign bit and accepts the bit to the left of the sign bit as the sign. If the new sign bit is the same as the previous one, the operation is successful, otherwise an overflow or underflow has occurred and the result is not valid. Figure 4.5 shows these two operations.

Figure 4.5 *Arithmetic shift operations*

a. Arithmetic right shift b. Arithmetic left shift

Example 4.13

Use an arithmetic right shift operation on the bit pattern 10011001. The pattern is an integer in two's complement format.

Solution

The solution is shown below. The leftmost bit is retained and also copied to its right neighbor bit (the white bit over the black background). The bit in color is lost:

Arithmetic Right	1	0	0	1	1	0	0	1	Original
	1	1	0	0	1	1	0	0	After shift

The original number was –103 and the new number is –52, which is the result of dividing –103 by 2 truncated to the smaller integer.

Example 4.14

Use an arithmetic left shift operation on the bit pattern 11011001. The pattern is an integer in two's complement format.

Solution

The solution is shown below. The leftmost bit (shown in color) is lost and a 0 (shown as white in the black background) is inserted as the rightmost bit:

Arithmetic Left	1	1	0	1	1	0	0	1	Original
	1	0	1	1	0	0	1	0	After shift

The original number was –39 and the new number is –78. The original number is multiplied by two. The operation is valid because no underflow occurred.

Example 4.15

Use an arithmetic left shift operation on the bit pattern 01111111. The pattern is an integer in two's complement format.

Solution

The solution is shown below. The leftmost bit (in color) is lost and a 0 (white in the black background) is inserted as the rightmost bit:

Arithmetic Left	0	1	1	1	1	1	1	1	Original
	1	1	1	1	1	1	1	0	After shift

The original number was 127 and the new number is –2. Here the result is not valid because an overflow has occurred. The expected answer $127 \times 2 = 254$ cannot be represented by an 8-bit pattern.

4.3 ARITHMETIC OPERATIONS

Arithmetic operations involve adding, subtracting, multiplying, and dividing. We can apply these operations to integers and floating-point numbers.

4.3.1 Arithmetic operations on integers

All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to integers. Although multiplication (division) of integers can be implemented using repeated addition (subtraction), the procedure is not efficient. There are more efficient procedures for multiplication and division, such as Booth procedures, but these are beyond the scope of this book. For this reason, we only discuss addition and subtraction of integers here.

Addition and subtraction for two's complement integers

We first discuss addition and subtraction for integers in two's complement representation, because it is more common. As we discussed in Chapter 3, integers are normally stored in

two's complement format. One of the advantages of two's complement representation is that there is no difference between addition and subtraction. When the subtraction operation is encountered, the computer simply changes it to an addition operation, but makes two's complement of the second number. In other words:

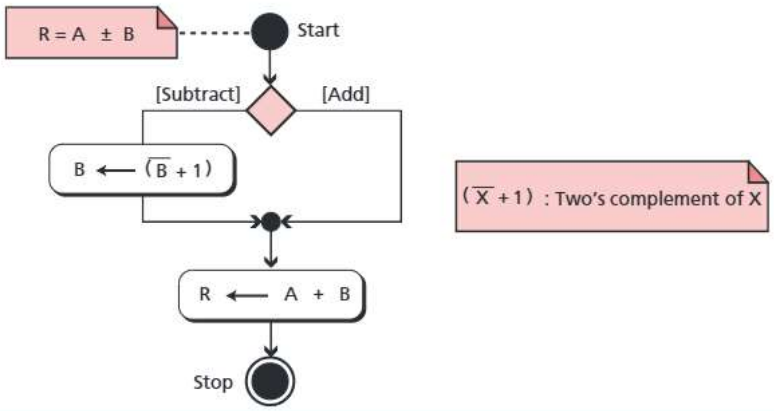
$$A - B \leftrightarrow A + (\overline{B} + 1) \quad \text{where } ((\overline{B} + 1)) \text{ means the two's complement of B}$$

This means that we only need to discuss addition. Adding numbers in two's complement is like adding the numbers in decimal: we add column by column, and if there is a carry, it is added to the next column. However, the carry produced from the last column is discarded.

We should remember that we add integers column by column. In each column, we have either two bits to add if there is no carry from the previous column, or three bits to add if there is a carry from the previous column.

Now we can show the procedure for addition or subtraction of two integers in two's complement format (Figure 4.6). Note that we use the notation $(\overline{X} + 1)$ to mean two's complement of X. This notation is very common in literature because $\overline{X}$ denotes the one's complement of X. If we add 1 to the one's complement of an integer, we get its two's complement.

Figure 4.6 *Addition and subtraction of integers in two's complement format*



The procedure is as follows:

1. If the operation is subtraction, we take the two's complement of the second integer. Otherwise, we move to the next step.
2. We add the two integers.

Example 4.16

Two integers A and B are stored in two's complement format. Show how B is added to A:

$$A = (00010001)_2 \quad B = (00010110)_2$$

Solution

The operation is adding. A is added to B and the result is stored in R:

			1					Carry
	0	0	0	1	0	0	0	1	A
+	0	0	0	1	0	1	1	0	B
	0	0	1	0	0	1	1	1	R

We check the result in decimal: (+17) + (+22) = (+39).

Example 4.17

Two integers A and B are stored in two's complement format. Show how B is added to A:

$$A = (00011000)_2 \quad B = (11101111)_2$$

Solution

The operation is adding. A is added to B and the result is stored in R. Note that the last carry is discarded because the size of the memory is only 8 bits:

1	1	1	1	1				Carry
	0	0	0	1	1	0	0	0	A
+	1	1	1	0	1	1	1	1	B
	0	0	0	0	0	1	1	1	R

Checking the result in decimal, (+24) + (–17) = (+7).

Example 4.18

Two integers A and B are stored in two's complement format. Show how B is subtracted from A:

$$A = (00011000)_2 \quad B = (11101111)_2$$

Solution

The operation is subtracting. A is added to $(\overline{B} + 1)$ and the result is stored in R:

			1					Carry
	0	0	0	1	1	0	0	0	A
+	0	0	0	1	0	0	0	1	$(\overline{B} + 1)$
	0	0	1	0	1	0	0	1	R

Checking the result in decimal, (+24) – (–17) = (+41).

Example 4.19

Two integers A and B are stored in two's complement format. Show how B is subtracted from A:

$$A = (11011101)_2 \quad B = (00010100)_2$$

Solution

The operation is subtracting. A is added to $(\overline{B} + 1)$ and the result is stored in R:

	1	1	1	1	1	1			Carry
		1	1	0	1	1	1	0	1	A
+	1	1	1	0	1	1	0	0	$(\overline{B} + 1)$
	1	1	0	0	1	0	0	1	R

Checking the result in decimal, (–35) – (+20) = (–55). Note that the last carry is discarded.

Example 4.20

Two integers A and B are stored in two's complement format. Show how B is added to A:

$$A = (01111111)_2 \quad B = (00000011)_2$$

Solution

The operation is adding. A is added to B and the result is stored in R:

	1	1	1	1	1	1	1		Carry
	0	1	1	1	1	1	1	1	A
+	0	0	0	0	0	0	1	1	B
	1	0	0	0	0	0	1	0	R

We expect the result to be 127 + 3 = 130, but the answer is –126. The error is due to overflow, because the expected answer (+130) is not in the range –128 to +127.

When we do arithmetic operations on numbers in a computer, we should remember that each number and the result should be in the range defined by the bit allocation.

Addition or subtraction for sign-and-magnitude integers

Addition or subtraction for integers in sign-and-magnitude representation looks very complex. We have four different combinations of signs (two signs, each of two values) for addition, and four different conditions for subtraction. This means that we need to consider eight different situations. For those interested readers, we describe these in more detail in Appendix I.

4.3.2 Arithmetic operations on reals

All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to reals stored in floating-point format. Multiplication of two reals involves multiplication of two integers in sign-and-magnitude representation. Division of two reals involves division of two integers in sign-and-magnitude representations. Since we did not discuss the multiplication or division of integers in sign-and-magnitude representation, we will not discuss the multiplication and division of reals, and only show addition and subtractions for reals in Appendix J.

4.4 END-CHAPTER MATERIALS

4.4.1 Recommended Reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Mano, M. *Computer System Architecture*, Upper Saddle River, NJ: Prentice-Hall, 1993
- Null, L. and Lobur, J. *Computer Organization and Architecture*, Sudbury, MA: Jones and Bartlett, 2003
- Stalling, W. *Computer Organization and Architecture*, Upper Saddle River, NJ: Prentice-Hall, 2000

4.4.2 Key terms

This chapter has introduced the following key terms, which are listed here with the pages on which they first occur:

AND operation 75	mask 77
arithmetic operation 82	NOT operation 75
arithmetic shift operation 81	OR operation 75
Boolean algebra 74	truth table 74
circular shift operation 80	XOR operation 75
logical shift operation 78	

4.4.3 Summary

- Operations on data can be divided into three broad categories: logic operations, shift operations, and arithmetic operations. Logic operations refer to those operations that apply the same basic operation to individual bits of a pattern or to two corresponding bits in two patterns. Shift operations move the bits in the pattern. Arithmetic operations involve adding, subtracting, multiplying, and dividing.

- The four logic operators discussed in this chapter (NOT, AND, OR, and XOR) can be used at the bit level or the pattern level. The NOT operator is a unary operator, while the AND, OR, and XOR operators are binary operators.
- The only application of the NOT operator is to complement the whole pattern. One of the applications of the AND operator is to unset (force to 0) specific bits in a bit pattern. One of the applications of the OR operator is to set (force to 1) specific bits in a bit pattern. One of the applications of the XOR operator is to flip (complement) specific bits in a bit pattern.
- Shift operations move the bits in the pattern: they change the positions of the bits. We can divide shift operations into two categories: logical shift operations and arithmetic shift operations. A logical shift operation is applied to a pattern that does not represent a signed number. Arithmetic shift operations assume that the bit pattern is a signed integer in two's complement format.
- All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to integers. Integers are normally stored in two's complement format. One of the advantages of two's complement representation is that there is no difference between addition and subtraction. When the subtraction operation is encountered, the computer simply changes it to an addition operation, but forms the two's complement of the second number. Addition and subtraction for integers in sign-and-magnitude representation looks very complex. We have eight situations to consider.
- All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to reals stored in floating-point format. Addition and subtraction of real numbers stored in floating-point numbers is reduced to addition and subtraction of two integers stored in sign-and-magnitude after the alignment of decimal points.

4.5 PRACTICE SET

4.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

4.5.2 Review questions

Q4-1. What is the difference between an arithmetic operation and a logical operation?

Q4-2. What happens to the carry from the leftmost column in the addition of integers in two's complement format?

Q4-3. Can n, the bit allocation, equal 1? Why, or why not?

Q4-4. Define the term *overflow*.

Q4-5. In the addition of floating-point numbers, how do we adjust the representation of numbers with different exponents?

Q4-6. What is the difference between a unary operation and a binary operation?

Q4-7. Name the logical binary operations.
Q4-8. What is a truth table?
Q4-9. What does the NOT operator do?
Q4-10. When is the result of an AND operator true?
Q4-11. When is the result of an OR operator true?
Q4-12. When is the result of an XOR operator true?
Q4-13. Mention an important property of the AND operator discussed in this chapter.
Q4-14. Mention an important property of the OR operator discussed in this chapter.
Q4-15. Mention an important property of the XOR operator discussed in this chapter.
Q4-16. What binary operation can be used to set bits? What bit pattern should the mask have?
Q4-17. What binary operation can be used to unset bits? What bit pattern should the mask have?
Q4-18. What binary operation can be used to flip bits? What bit pattern should the mask have?
Q4-19. What is the difference between simple and arithmetic shifts?

4.5.3 Problems

P4-1. Show the result of the following operations:
a. NOT $(99)_{16}$
b. NOT $(FF)_{16}$
c. NOT $(00)_{16}$
d. NOT $(01)_{16}$

P4-2. Show the result of the following operations:
a. $(99)_{16}$ AND $(99)_{16}$
b. $(99)_{16}$ AND $(00)_{16}$
c. $(99)_{16}$ AND $(FF)_{16}$
d. $(FF)_{16}$ AND $(FF)_{16}$

P4-3. Show the result of the following operations:
a. $(99)_{16}$ OR $(99)_{16}$
b. $(99)_{16}$ OR $(00)_{16}$
c. $(99)_{16}$ OR $(FF)_{16}$
d. $(FF)_{16}$ OR $(FF)_{16}$

P4-4. Show the result of the following operations:
a. NOT [$(99)_{16}$ OR $(99)_{16}$)]
b. $(99)_{16}$ OR [NOT $(00)_{16}$]
c. [$(99)_{16}$ AND $(33)_{16}$)] OR [$(00)_{16}$ AND $(FF)_{16}$]
d. $(99)_{16}$ OR $(33)_{16}$ AND [$(00)_{16}$ OR $(FF)_{16}$]

P4-5. We need to unset (force to 0) the four leftmost bits of a pattern. Show the mask and the operation.
P4-6. We need to set (force to 1) the four rightmost bits of a pattern. Show the mask and the operation.
P4-7. We need to flip the three rightmost and the two leftmost bits of a pattern. Show the mask and the operation.
P4-8. We need to unset the three leftmost bits and set the two rightmost bits of a pattern. Show the masks and operations.

P4-9. Use the shift operation to divide an integer by 4.

P4-10. Use the shift operation to multiply an integer by 8.

P4-11. Use a combination of logical and shift operations to extract the fourth and fifth bits from the left of an unsigned integer.

P4-12. Using an 8-bit allocation, first convert each of the following integers to two's complement, do the operation, and then convert the result to decimal.

a. 19 + 23
b. 19 – 23
c. –19 + 23
d. –19 – 23

P4-13. Using a 16-bit allocation, first convert each of the following numbers to two's complement, do the operation, and then convert the result to decimal.

a. 161 + 1023
b. 161 – 1023
c. –161 + 1023
d. –161 – 1023

P4-14. Which of the following operations creates an overflow if the numbers and the result are represented in 8-bit two's complement representation?

a. 11000010 + 00111111
b. 00000010 + 00111111
c. 11000010 + 11111111
d. 00000010 + 11111111

P4-15. Without actually doing the calculation, can we tell which of the following creates an overflow if the numbers and the result are in 8-bit two's complement representation?

a. 32 + 105
b. 32 – 105
c. –32 + 105
d. –32 – 105

P4-16. Show the result of the following operations assuming that the numbers are stored in 16-bit two's complement representation. Show the result in hexadecimal notation.

a. $(012A)_{16} + (0E27)_{16}$
b. $(712A)_{16} + (9E00)_{16}$
c. $(8011)_{16} + (0001)_{16}$
d. $(E12A)_{16} + (9E27)_{16}$

P4-17. Using an 8-bit allocation, first convert each of the following numbers to sign-and-magnitude representation, do the operation, and then convert the result to decimal.

a. 19 + 23
b. 19 – 23
c. –19 + 23
d. –19 – 23

P4-18. Show the result of the following floating-point operations using IEEE_127—see Chapter 3.

a. 34.75 + 23.125
b. –12.625 + 451.00
c. 33.1875 – 0.4375
d. –344.3125 – 123.5625

P4-19. In which of the following situations does an overflow never occur? Justify the answer.

a. Adding two positive integers.
b. Adding one positive integer to a negative integer.
c. Subtracting one positive integer from a negative integer.
d. Subtracting two negative integers.

P4-20. What is the result of adding an integer to its one's complement?

P4-21. What is the result of adding an integer to its two's complement?

CHAPTER 5

Computer Organization

In this chapter we discuss the organization of a stand-alone computer. We explain how every computer is made up of three subsystems. We also show how a simple, hypothetical computer can run a simple program to perform primitive arithmetic or logic operations.

Objectives

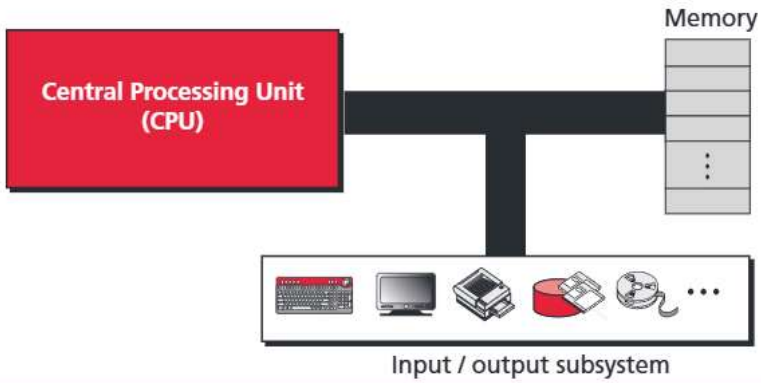
After studying this chapter, the student should be able to:

- List the three subsystems of a computer.
- Describe the role of the central processing unit (CPU) in a computer.
- Describe the fetch–decode–execute phases of a cycle in a typical computer.
- Describe the main memory and its addressing space.
- Distinguish between main memory and cache memory.
- Define the input/output subsystem.
- Understand the interconnection of subsystems and list different bus systems.
- Describe different methods of input/output addressing.
- Distinguish the two major trends in the design of computer architecture.
- Understand how computer throughput can be improved using pipelining.
- Understand how parallel processing can improve the throughput of computers.

5.1 INTRODUCTION

We can divide the parts that make up a computer into three broad categories or subsystems: the central processing unit (CPU), the main memory, and the input/output subsystem. The next three sections discuss these subsystems and how they are connected to make a standalone computer. Figure 5.1 shows the three subsystems of a standalone computer.

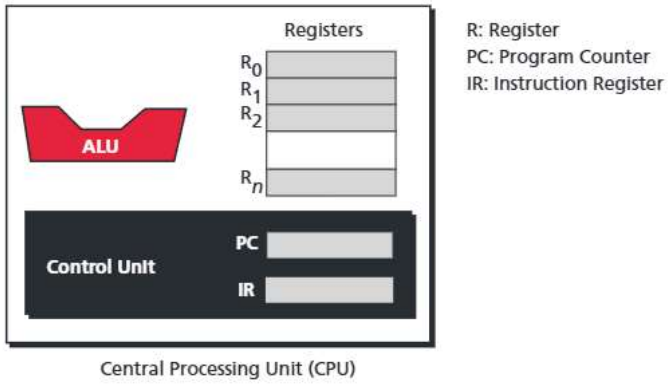
Figure 5.1 *Computer hardware (subsystems)*



5.2 CENTRAL PROCESSING UNIT

The **central processing unit** (CPU) performs operations on data. In most architectures it has three parts: an arithmetic logic unit (ALU), a control unit, and a set of registers, (Figure 5.2).

Figure 5.2 *Central processing unit (CPU)*



5/1/25, 9:57 PM study viewer

5.2.1 The arithmetic logic unit (ALU)

The **arithmetic logic unit** (ALU) performs logic, shift, and arithmetic operations on data.

Logic operations

We discussed several logic operations, such as NOT, AND, OR, and XOR, in Chapter 4. These operations treat the input data as bit patterns and the result of the operation is also a bit pattern.

Shift operations

We discussed two groups of shift operations on data in Chapter 4: logical shift operations and arithmetic shift operations. Logical shift operations are used to shift bit patterns to the left or right, while arithmetic operations are applied to integers. Their main purpose is to divide or multiply integers by two.

Arithmetic operation

We discussed some arithmetic operations on integers and reals on Chapter 4. We mentioned that some operations can be implemented more efficiently in hardware.

5.2.2 Registers

Registers are fast stand-alone storage locations that hold data temporarily. Multiple registers are needed to facilitate the operation of the CPU. Some of these registers are shown in Figure 5.2.

Data registers

In the past computers had only a few data registers to hold the input data and the result of the operations. Today, computers use dozens of registers inside the CPU to speed up their operations, because complex operations are done using hardware instead of software. These require several registers to hold the intermediate results. Data registers are named R_1 to R_n in Figure 5.2.

Instruction registers

Today computers store not only data, but also programs, in their memory. The CPU is responsible for fetching instructions one by one from memory, storing them in the **instruction register** (IR in Figure 5.2), decoding them, and executing them. We will discuss this issue later in the chapter.

Program counter

Another common register in the CPU is the **program counter** (PC in Figure 5.2). The program counter keeps track of the instruction currently being executed. After execution of the instruction, the counter is incremented to point to the address of the next instruction in memory.

https://studylib.net/doc/26059254/foundations-computer-science-4th 113/706

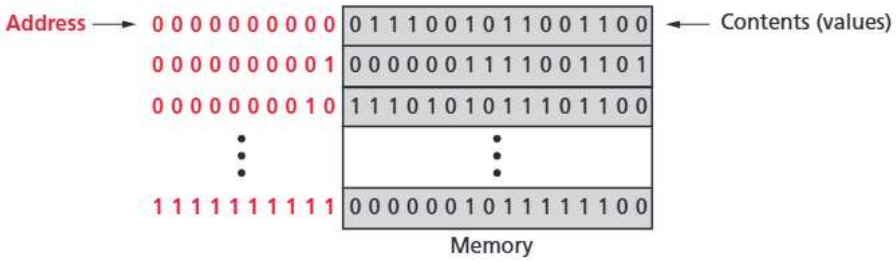
5.2.3 The control unit

The third part of any CPU is the control unit. The **control unit** controls the operation of each subsystem. Controlling is achieved through signals sent from the control unit to other subsystems.

5.3 MAIN MEMORY

Main memory is the second major subsystem in a computer (Figure 5.3). It consists of a collection of storage locations, each with a unique identifier, called an *address*. Data is transferred to and from memory in groups of bits called *words*. A word can be a group of 8 bits, 16 bits, 32 bits, or 64 bits (and growing). If the word is 8 bits, it is referred to as a *byte*. The term 'byte' is so common in computer science that sometimes a 16-bit word is referred to as a 2-byte word, or a 32-bit word is referred to as a 4-byte word.

Figure 5.3 *Main memory*



5.3.1 Address space

To access a word in memory requires an identifier. Although programmers use a name to identify a word (or a collection of words), at the hardware level each word is identified by an address. The total number of uniquely identifiable locations in memory is called the **address space**. For example, a memory with 64 kilobytes and a word size of 1 byte has an address space that ranges from 0 to 65 535.

Table 5.1 shows the units used to refer to memory. Note that the terminology is misleading: it approximates the number of bytes in powers of 10, but the actual number of bytes is in powers of 2. Units in powers of 2 facilitates addressing.

Table 5.1 Memory units

Unit	*Exact Number of Bytes*	*Approximation*
kilobyte	2^{10} (1024) bytes	10^3 bytes
megabyte	2^{20} (1 048 576) bytes	10^6 bytes
gigabyte	2^{30} (1 073 741 824) bytes	10^9 bytes
terabyte	2^{40} bytes	10^{12} bytes

Addresses as bit patterns

Because computers operate by storing numbers as bit patterns, a memory address is also represented as a bit pattern. So if a computer has 64 kilobytes (2^{16}) of memory with a word size of 1 byte, we need a bit pattern of 16 bits to define an address. Recall from Chapter 3 that addresses can be represented as unsigned integers (we do not have negative addresses). In other words, the first location is referred to as address 0000000000000000 (address 0), and the last location is referred to as address 1111111111111111 (address 65535). In general, if a computer has N words of memory, we need an unsigned integer of size $\log_2 N$ bits to refer to each memory location.

Memory addresses are defined using unsigned binary integers.

Example 5.1

A computer has 32 MB (megabytes) of memory. How many bits are needed to address any single byte in memory?

Solution

The memory address space is 32 MB, or 2^{25} ($2^5 \times 2^{20}$). This means that we need $\log_2 2^{25}$, or 25 bits, to address each byte.

Example 5.2

A computer has 128 MB of memory. Each word in this computer is eight bytes. How many bits are needed to address any single word in memory?

Solution

The memory address space is 128 MB, which means 2^{27}. However, each word is eight (2^3) bytes, which means that we have 2^{24} words. This means that we need $\log_2 2^{24}$, or 24 bits, to address each word.

5.3.2 Memory types

Two main types of memory exist: RAM and ROM.

RAM

Random access memory (RAM) makes up most of the main memory in a computer. In a random access device, a data item can be accessed randomly—using the address of the memory location—without the need to access all data items located before it. However, the term is confusing, because ROM can also be accessed randomly. What distinguishes RAM from ROM is that RAM can be read from and written to. The CPU can write something to RAM and later overwrite it. Another characteristic of RAM is that it is *volatile*: the information (program or data) is lost if the computer is powered down. In other words, all information in RAM is erased if you turn off the computer or if there is a power outage. RAM technology is divided into two broad categories: SRAM and DRAM.

SRAM

Static RAM (SRAM) technology uses traditional *flip-flop gates* (see Appendix E) to hold data. The gates hold their state (0 or 1), which means that data is stored as long as the power is on and there is no need to refresh memory locations. SRAM is fast but expensive.

DRAM

Dynamic RAM (DRAM) technology uses capacitors, electrical devices that can store energy, for data storage. If a capacitor is charged, the state is 1; if it is discharged, the state is 0. Because a capacitor loses some of its charge with time, DRAM memory cells need to be refreshed periodically. DRAMs are slow but inexpensive.

ROM

The contents of **read-only memory (ROM)** are written by the manufacturer, and the CPU can read from, but not write to, ROM. Its advantage is that it is *nonvolatile*—its contents are not lost if you turn off the computer. Normally, it is used for programs or data that must not be erased or changed even if you turn off the computer. For example, some computers come with ROM that holds the *boot program* that runs when we switch on the computer.

PROM

One variation of ROM is **programmable read-only memory (PROM)**. This type of memory is blank when the computer is shipped. The user of the computer, with some special equipment, can store programs on it. When programs are stored, it behaves like ROM and cannot be overwritten. This allows a computer user to store specific programs in PROM.

EPROM

A variation of PROM is **erasable programmable read-only memory (EPROM)**. It can be programmed by the user, but can also be erased with a special device that applies ultraviolet light. To erase EPROM memory requires physical removal and reinstallation of the EPROM.

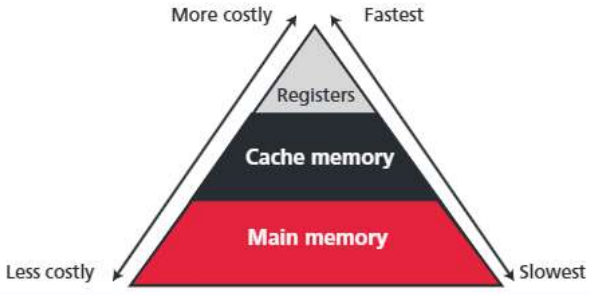
EEPROM

A variation of EPROM is **electrically erasable programmable read-only memory (EEPROM)**. EEPROM can be programmed and erased using electronic impulses without being removed from the computer.

5.3.3 Memory hierarchy

Computer users need a lot of memory, especially memory that is very fast and inexpensive. This demand is not always possible to satisfy—very fast memory is usually not cheap. A compromise needs to be made. The solution is hierarchical levels of memory (Figure 5.4). The hierarchy is based on the following:

Figure 5.4 *Memory hierarchy*

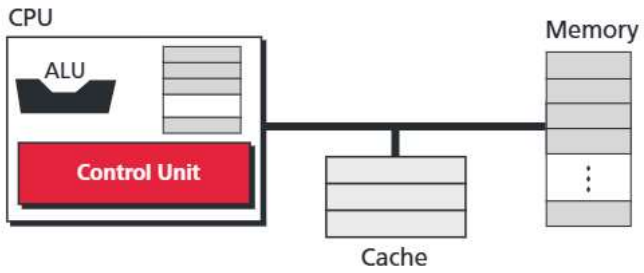


- Use a very small amount of costly high-speed memory where speed is crucial. The registers inside the CPU are of this type.
- Use a moderate amount of medium-speed memory to store data that is accessed often. *Cache memory*, discussed next, is of this type.
- Use a large amount of low-speed memory for data that is accessed less often. Main memory is of this type.

5.3.4 Cache memory

Cache memory is faster than main memory but slower than the CPU and its registers. Cache memory, which is normally small in size, is placed between the CPU and main memory (Figure 5.5).

Figure 5.5 *Cache memory*



Cache memory at any time contains a copy of a portion of main memory. When the CPU needs to access a word in main memory, it follows this procedure:

1. The CPU checks the cache.
2. If the word is there, it copies the word: if not, the CPU accesses main memory and copies a block of memory starting with the desired word. This block replaces the previous contents of cache memory.
3. The CPU accesses the cache and copies the word.

This procedure can expedite operations; if the word is in the cache, it is accessed immediately. If the word is not in the cache, the word and a whole block are copied to the cache. Since it is probable that the CPU, in its next **cycle**, will need to access the words following the first word, the existence of the cache speeds processing.

We might wonder why cache memory is so efficient despite its small size. The answer lies in the '80–20 rule'. It has been observed that most computers typically spend 80 per cent of their time accessing only 20 per cent of the data. In other words, the same data is accessed over and over again. Cache memory, with its high speed, can hold this 20 per cent to make access faster at least 80 per cent of the time.

5.4 INPUT/OUTPUT SUBSYSTEM

The third major subsystem in a computer is the collection of devices referred to as the **input/output (I/O) subsystem**. This subsystem allows a computer to communicate with

the outside world, and to store programs and data even when the power is off. Input/output devices can be divided into two broad categories: nonstorage and storage devices.

5.4.1 Nonstorage devices

Nonstorage devices allow the CPU/memory to communicate with the outside world, but they cannot store information.

Keyboard and monitor

Two of the more common nonstorage input/output devices are the **keyboard** and the **monitor**. The keyboard provides input, the monitor displays output and at the same time echoes input typed on the keyboard. Programs, commands, and data are input or output using strings of characters. The characters are encoded using a code such as ASCII (see Appendix A). Other devices that fall in this category are *mice*, *joysticks*, and so on.

Printer

A **printer** is an **output device** that creates a permanent record. A printer is a nonstorage device because the printed material cannot be directly entered into a computer again unless someone retypes or scans it.

5.4.2 Storage devices

Storage devices, although classified as I/O devices, can store large amounts of information to be retrieved at a later time. They are cheaper than main memory, and their contents are nonvolatile—that is, not erased when the power is turned off. They are sometimes referred to as *auxiliary storage devices*. We can categorize them as either magnetic or optical.

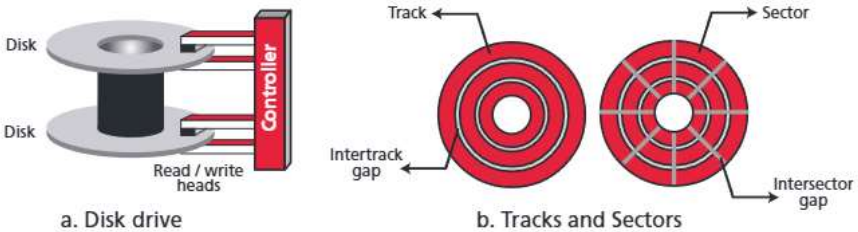
Magnetic storage devices

Magnetic storage devices use magnetization to store bits of data. If a location is magnetized, it represents 1, if not magnetized, it represents 0.

Magnetic disks

A **magnetic disk** consists of one or more disks stacked on top of each other. The disks are coated with a thin magnetic film. Information is stored on and retrieved from the surface of the disk using a **read/write head** for each magnetized surface of the disk. Figure 5.6 shows the physical layout of a magnetic disk drive and the organization of a disk.

Figure 5.6 *A magnetic disk*



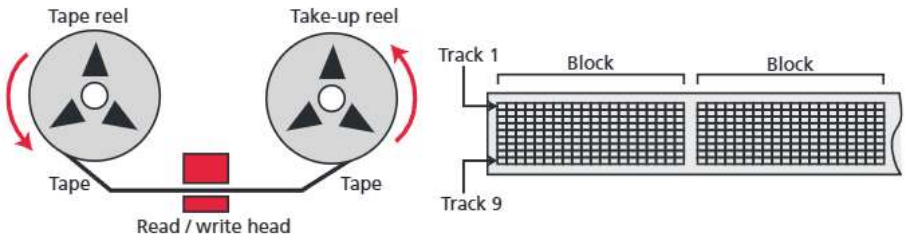
a. Disk drive b. Tracks and Sectors

- **Surface organization.** To organize data stored on the disk, each surface is divided into **tracks**, and each track is divided into **sectors** (Figure 5.6). The tracks are separated by an **intertrack gap**, and the sectors are separated by an **intersector gap**.
- **Data access.** A magnetic disk is considered a random access device. In a random access device, a data item can be accessed randomly without the need to access all other data items located before it. However, the smallest storage area that can be accessed at one time is a sector. A block of data can be stored in one or more sectors and retrieved without the need to retrieve the rest of the information on the disk.
- **Performance.** The performance of a disk depends on several factors, the most important being the rotational speed, the seek time, and the transfer time. The **rotational speed** defines how fast the disk is spinning. The **seek time** defines the time to move the read/write head to the desired track where the data is stored. The **transfer time** defines the time to move data from the disk to the CPU/memory.

Magnetic tape

Magnetic tape comes in various sizes. One common type is half-inch plastic tape coated with a thick magnetic film. The tape is mounted on two reels and uses a read/write head that reads or writes information when the tape is passed through it. Figure 5.7 shows the mechanical configuration of a magnetic tape drive.

Figure 5.7 *Magnetic tape*



a. Tape drive b. Surface orgnization

- **Surface organization.** The width of the tape is divided into nine tracks, each location on a track storing 1 bit of information. Nine vertical locations can store 8 bits of information related to a byte plus a bit for error detection (Figure 5.7).
- **Data access.** A magnetic tape is considered a sequential access device. Although the surface may be divided into blocks, there is no addressing mechanism to access each block. To retrieve a specific block on the tape, we need to pass through all the previous blocks.
- **Performance.** Although magnetic tape is slower than a magnetic disk, it is cheaper. Today, people use magnetic tape to back up large amounts of data.

Optical storage devices

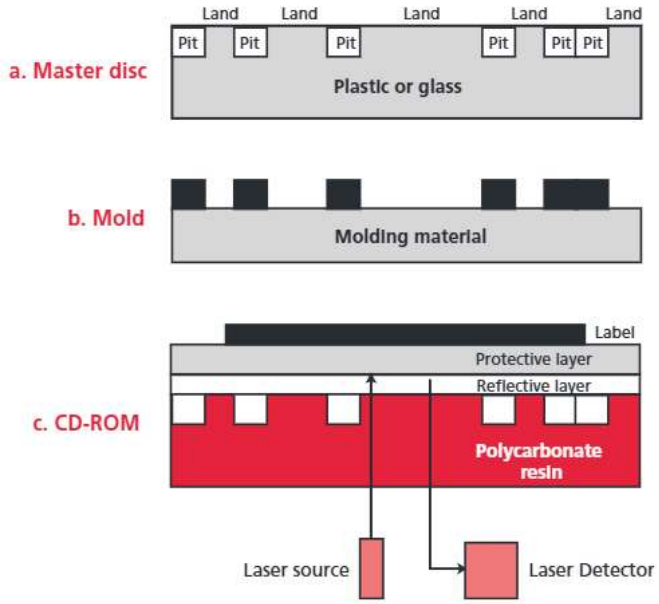
Optical storage devices, a relatively recent technology, use laser light to store and retrieve data. The use of optical storage technology followed the invention of the **compact disk (CD)** used to store audio information. Today, the same technology—slightly improved—is used

to store information in a computer. Devices that use this technology include CD-ROMs, CD-Rs, CD-RWs, and DVDs.

CD-ROMs

Compact disk read-only memory (CD-ROM) disks use the same technology as the audio CD, originally developed by Phillips and Sony for recording music. The only difference between these two technologies is enhancement: a CD-ROM drive is more robust and checks for errors. Figure 5.8 shows the steps involved in creating and using a CD-ROM.

Figure 5.8 *Creation and use of CD-ROMs*



- **Creation**. CD-ROM technology uses three steps to create a large number of discs:
  a. A **master disk** is created using a high-power infrared laser that creates bit patterns on coated plastic. The laser translates the bit patterns into a sequence of **pits** (holes) and **lands** (no holes). The pits usually represent 0s and the lands usually represent 1s. However, this is only a convention, and it can be reversed. Other schemes use a transition (pit to land or land to pit) to represent 1, and a lack of transition to represent 0.
  b. From the master disk, a mold is made. In the mold, the pits (holes) are replaced by bumps.
  c. Molten **polycarbonate resin** is injected into the mold to produce the same pits as the master disk. A very thin layer of aluminum is added to the polycarbonate to provide a reflective surface. On top of this, a protective layer of lacquer is applied and a label is added. Only this last step needs to be repeated for each disk.
- **Reading**. The CD-ROM is read using a low-power laser beam. The beam is reflected by the aluminum surface when passing through a land. It is reflected twice when it encounters a pit, once by the pit boundary and once by the aluminum boundary. The two reflections have a destructive effect, because the depth of the pit is chosen to be exactly

one-fourth of the beam wavelength. In other words, the sensor installed in the drive detects more light when the location is a land and less light when the location is a pit, so can read what was recorded on the original master disk and copied to the CD-ROM.

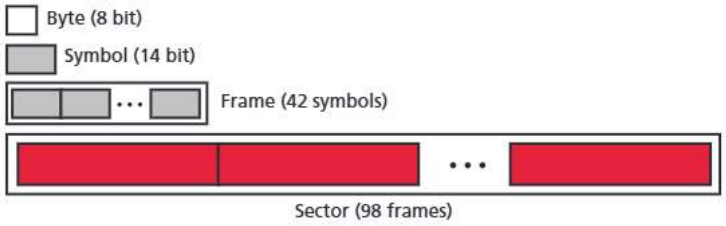
- **Format.** CD-ROM technology uses a different format than magnetic disk (Figure 5.9). The format of data on a CD-ROM is based on:
  a. A block of 8-bit data transformed into a 14-bit symbol using an error-correction method called Hamming code.
  b. A frame made up from 42 symbols (14 bits/symbol).
  c. A sector made up from 98 frames (2352 bytes).
- **Speed.** CD-ROM drives come in different speeds. Single speed is referred to as 1x, double speed 2x, and so on. If the drive is single speed, it can read up to 153 600 bytes per second. Table 5.2 shows the speeds and their corresponding data rates.

Table 5.2 CD-ROM speeds

Speed	*Data rate*	*Approximation*
1*x*	153 600 bytes per second	150 KB/s
2*x*	307 200 bytes per second	300 KB/s
4*x*	614 400 bytes per second	600 KB/s
6*x*	921 600 bytes per second	900 KB/s
8*x*	1 228 800 bytes per second	1.2 MB/s
12*x*	1 843 200 bytes per second	1.8 MB/s
16*x*	2 457 600 bytes per second	2.4 MB/s
24*x*	3 688 400 bytes per second	3.6 MB/s
32*x*	4 915 200 bytes per second	4.8 MB/s
40*x*	6 144 000 bytes per second	6 MB/s

- **Application.** The expense involved in creating a master disk, mold, and the actual disk can be justified if there are a large number of potential customers. In other words, this technology is economical if the discs are mass produced.

Figure 5.9 *CD-ROM format*



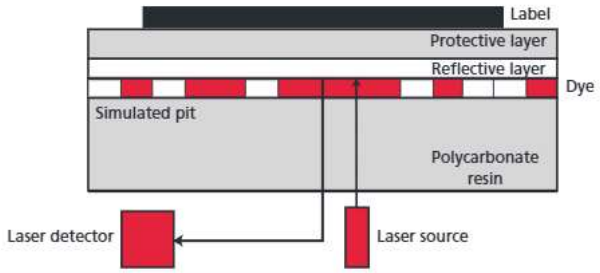
https://sanet.st/blogs/polatebooks/

CD-R

Clearly, CD-ROM technology is justifiable only if the manufacturer can create a large number of disks. On the other hand, the **compact disk recordable (CD-R)** format allows users to create one or more disks without going through the expense involved in creating CD-ROMs. It is particularly useful for making backups. You can write once to CD-R disks, but they can be read many times. This is why the format is sometimes called **write once, read many** (WORM).

- **Creation.** CD-R technology uses the same principles as CD-ROM to create a disk (Figure 5.10). The following lists the differences:
  a. There is no master disk or mold.
  b. The reflective layer is made of gold instead of aluminum.
  c. There are no physical pits (holes) in the polycarbonate: the pits and lands are only simulated. To simulate pits and lands, an extra layer of dye, similar to the material used in photography, is added between the reflective layer and the polycarbonate.
  d. A high-power laser beam, created by the CD burner of the drive, makes a dark spot in the dye, changing its chemical composition, which simulates a pit. The areas not struck by the beam become lands.
- **Reading.** CD-Rs can be read by a CD-ROM or a CD-R drive. This means that any differences should be transparent to the drive. The same low-power laser beam passes in front of the simulated pits and lands. For a land, the beam reaches the reflective layer and is reflected. For a simulated pit, the spot is opaque, so the beam cannot be reflected back.
- **Format and speed.** The format, capacity, and speed of CD-Rs are the same as CD-ROMs.
- **Application.** This technology is very attractive for the creation and distribution of a small number of disks. It is also very useful for making archive files and backups.

Figure 5.10 *Making a CD-R*

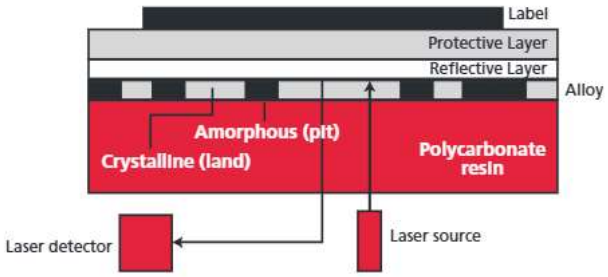


CD-RW

Although CD-Rs have become very popular, they can be written to only once. To overwrite previous materials, a new technology allows a new type of disk called **compact disk rewritable (CD-RW)**. It is sometimes called an *erasable optical disk*.

- **Creation**. CD-RW technology uses the same principles as CD-R to create the disk (Figure 5.11). The following lists the differences:

Figure 5.11 *Making a CD-RW*



  a. Instead of dye, the technology uses an alloy of silver, indium, antimony, and tellurium. This alloy has two stable states: crystalline (transparent) and amorphous (nontransparent).
  b. The drive uses high-power lasers to create simulated pits in the alloy (changing it from crystalline to amorphous).
- **Reading**. The drive uses the same type of low-power laser beam as CD-ROM and CD-R to detect pits and lands.
- **Erasing**. The drive uses a medium-power laser beam to change pits to lands. The beam changes a location from the amorphous state to the crystalline state.
- **Format and speed**. The format, capacity, and speed of CD-RWs are the same as CD-ROMs.
- **Application**. The technology is definitely more attractive than CD-R technology. However, CD-Rs are more popular for two reasons. First, blank CD-R discs are less expensive than blank CD-RW discs. Second, CD-Rs are preferable in cases where the created disk must not be changed, either accidentally or intentionally.

DVD

The industry has felt the need for digital storage media with even higher capacity. The capacity of a CD-ROM (650 MB) is insufficient to store video information. The latest optical memory storage device on the market is called a **digital versatile disk (DVD)**. It uses a technology similar to CD-ROM, but with the following differences:

  a. The pits are smaller: 0.4 microns in diameter instead of the 0.8 microns used in CDs.
  b. The tracks are closer to each other.
  c. The beam is a red laser instead of infrared.
  d. DVDs use one to two recording layers, and can be single-sided or double-sided.
- **Capacity**. These improvements result in higher capacities (Table 5.3).

Table 5.3 DVD capacities

Feature	*Capacity*
Single-sided, single-layer	4.7 GB
Single-sided, dual-layer	8.5 GB
Double-sided, single-layer	9.4 GB
Double-sided, dual-layer	17 GB

- **Compression**. DVD technology uses MPEG (see Chapter 15) for compression. This means that a single-sided, single-layer DVD can hold 133 minutes of video at high resolution. This also includes both audio and subtitles.
- **Application**. Today, the high capacity of DVDs attracts many applications that need to store a high volume of data.

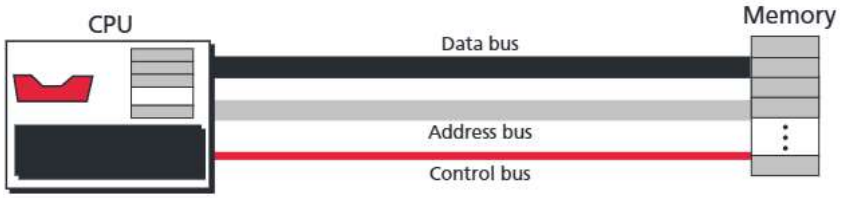
5.5 SUBSYSTEM INTERCONNECTION

The previous sections outlined the characteristics of the three subsystems (CPU, main memory, and I/O) in a stand-alone computer. In this section, we explore how these three subsystems are interconnected. The interconnection plays an important role because information needs to be exchanged between the three subsystems.

5.5.1 Connecting CPU and memory

The CPU and memory are normally connected by three groups of connections, each called a **bus**: data bus, address bus, and control bus (Figure 5.12).

Figure 5.12 *Connecting CPU and memory using three buses*



Data bus

The **data bus** is made of several connections, each carrying 1 bit at a time. The number of connections depends on the size of the word used by the computer. If the word is 32 bits (4 bytes), we need a data bus with 32 connections so that all 32 bits of a word can be transmitted at the same time.

Address bus

The **address bus** allows access to a particular word in memory. The number of connections in the address bus depends on the address space of the memory. If the memory has 2^n words, the address bus needs to carry n bits at a time. Therefore, it must have n connections.

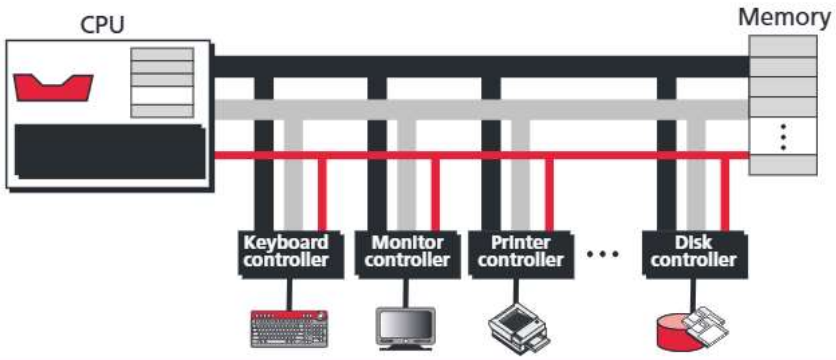
Control bus

The **control bus** carries communication between the CPU and memory. For example, there must be a code, sent from the CPU to memory, to specify a read or write operation. The number of connections used in the control bus depends on the total number of control commands a computer needs. If a computer has 2^m control actions, we need m connections for the control bus, because m bits can define 2^m different operations.

5.5.2 Connecting I/O devices

I/O devices cannot be connected directly to the buses that connect the CPU and memory because the nature of I/O devices is different from the nature of CPU and memory. I/O devices are electromechanical, magnetic, or optical devices, whereas the CPU and memory are electronic devices. I/O devices also operate at a much slower speed than the CPU/memory. There is a need for some sort of intermediary to handle this difference. Input/output devices are therefore attached to the buses through **input/output controllers** or interfaces. There is one specific **controller** for each input/output device (Figure 5.13).

Figure 5.13 *Connecting I/O devices to the buses*



Controllers

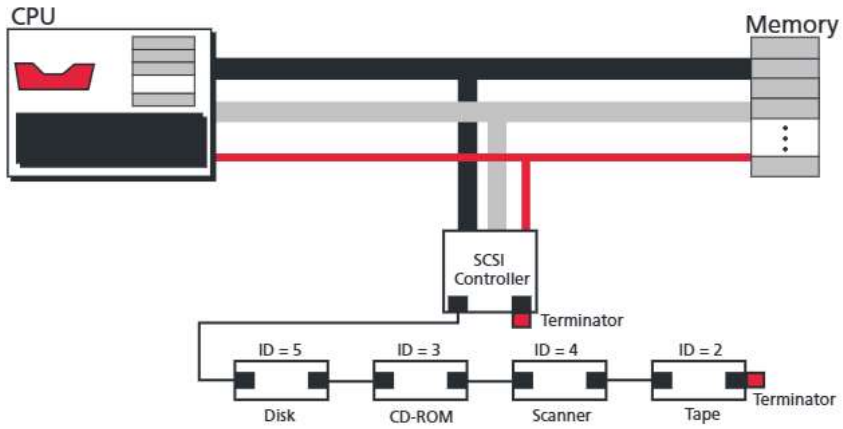
Controllers or interfaces bridge the gap between the nature of the I/O device and the CPU and memory. A controller can be a serial or parallel device. A serial controller has only one data wire, while a parallel controller has several data connections so that several bits can be transferred at a time.

Several kinds of controllers are in use. The most common ones today are SCSI, FireWire, USB, and HDMI.

SCSI

The **small computer system interface (SCSI)** was first developed for Macintosh computers in 1984. Today it is used in many systems. It has a parallel interface with 8, 16, or 32 connections. The SCSI interface provides a daisy-chained connection, as shown in Figure 5.14. Both ends of the chain must be connected to a special device called a *terminator*, and each device must have a unique address (target ID).

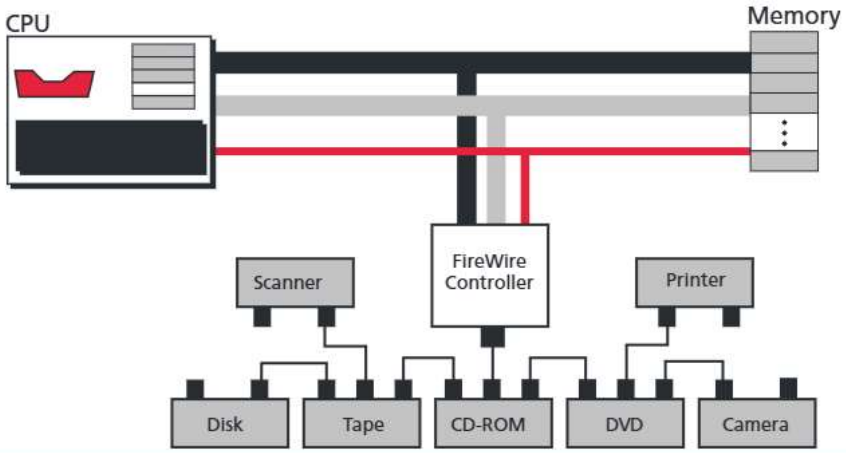
Figure 5.14 *SCSI controller*



FireWire

IEEE standard 1394 defines a serial interface commonly called **FireWire**. It is a high-speed serial interface that transfers data in packets, achieving a transfer rate of up to 50 MB/sec, or double that in the most recent version. It can be used to connect up to 63 devices in a daisy chain or a tree connection (using only one connection). Figure 5.15 shows the connection of input/output devices to a FireWire controller. There is no need for termination as there is for SCSI.

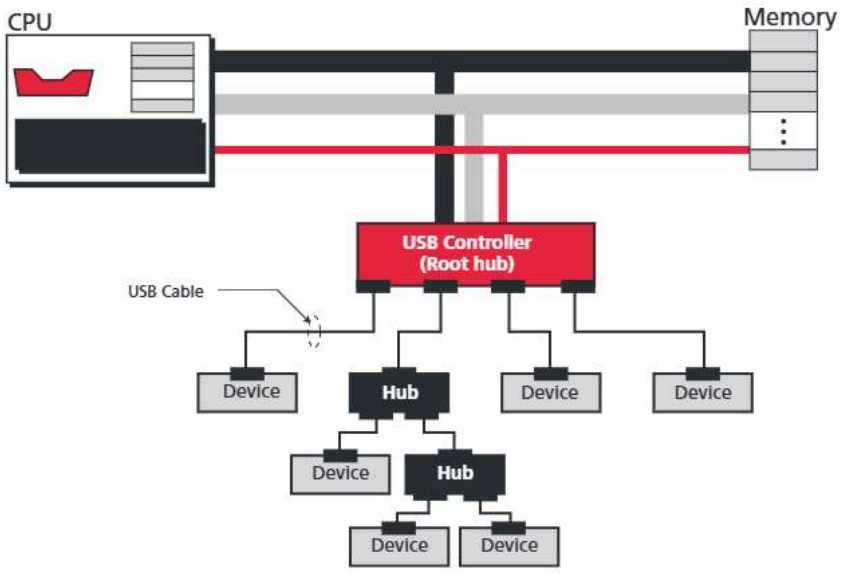
Figure 5.15 *FireWire controller*



USB

Universal Serial Bus (USB) is a competitor for FireWire. Although the nomenclature uses the term *bus*, USB is a serial controller that connects both low- and high-speed devices to the computer bus. Figure 5.16 shows the connection of the USB controller to the bus and the connection of devices to the controller.

Figure 5.16 *USB controller*



Multiple devices can be connected to a USB controller, which is also referred to as a *root hub*. USB-2 (USB Version 2.0) allows up to 127 devices to be connected to a USB controller using a tree-like **topology** with the controller as the root of the tree, **hubs** as the intermediate nodes, and the devices as the end nodes. The difference between the controller (root hub) and the other hubs is that the controller is aware of the presence of other hubs in the tree, but other hubs are passive devices that simply pass the data.

Devices can easily be removed or attached to the tree without powering down the computer. This is referred to as *hot-swappable*. When a hub is removed from the system, all devices and other hubs connected to it are also removed.

USB uses a cable with four wires. Two wires (+5 volts and ground) are used to provide power for low-power devices such as keyboards or mice. A high-power device needs to be connected to a power source. A hub gets its power from the bus and can provide power for low-power devices. The other two wires (twisted together to reduce noise) are used to carry data, addresses, and control signals. USB uses two different connectors: A and B. The A connector (downstream connector) is rectangular and is used to connect to the USB controller or the hub. The B connector (upstream connector) is close to square and is used to connect to the device. Recently two new connectors, mini A and mini B, have been introduced that are used for connecting to small devices and laptop computers.

USB-2 provides three data transfer rates: 1.5 Mbps (megabits per second), 12 Mbps, and 480 Mbps. The low data rate can be used with slow devices such as keyboards and mice, the medium data rate with printers, and the high data rate with mass storage devices.

Data is transferred over USB in packets (see Chapter 6). Each packet contains an address part (device identifier), a control part, and part of the data to be transmitted to that device. All devices will receive the same packet, but only those devices with the address defined in the packet will accept it.

USB 3.0 is another revision of the Universal Serial Bus (USB) standard for computer connectivity. USB 3.0 adds a new transfer mode called *SuperSpeed* capable of transferring data at up to 4.8 Gbit/s. It is promised to update USB 3.0 to 10 Gbit/s.

HDMI

HDMI (High-Definition Multimedia Interface) is a digital replacement for existing analog video standards. It can be used for transferring video data digital audio data from a source to a compatible computer monitor, video projector, digital television, or digital audio device. There are a number of HDMI-standard cables available including standard, enhanced, high definition, and 3D video signals; up to eight channels of compressed or uncompressed digital audio; a CEC (Consumer Electronics Control) connection; and an Ethernet data connection.

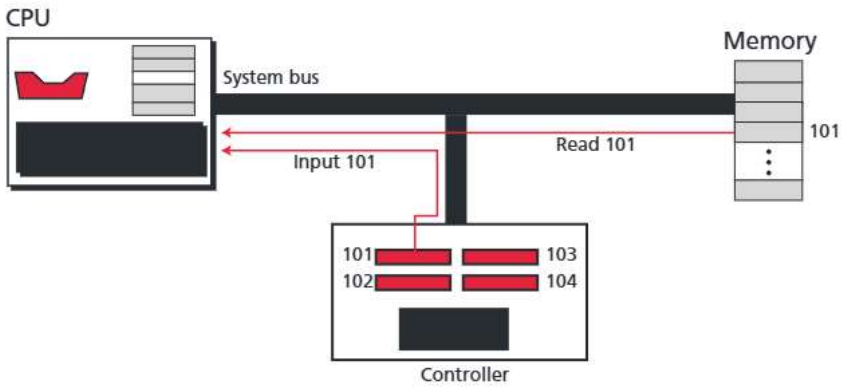
5.5.3 Addressing input/output devices

The CPU usually uses the same bus to read data from or write data to main memory and I/O device. The only difference is the instruction. If the instruction refers to a word in main memory, data transfer is between main memory and the CPU. If the instruction identifies an I/O device, data transfer is between the I/O device and the CPU. There are two methods for handling the addressing of I/O devices: isolated I/O and memory-mapped I/O.

Isolated I/O

In the **isolated I/O** method, the instructions used to read/write memory are totally different than the instructions used to read/write I/O devices. There are instructions to test, control, read from, and write to I/O devices. Each I/O device has its own address. The I/O addresses can overlap with memory addresses without any ambiguity because the instruction itself is different. For example, the CPU can use a command 'Read 101' to read from memory word 101, and it can use a command 'Input 101' to read from I/O device 101. There is no confusion, because the read command is for reading from memory and the input command is for reading from an I/O device (Figure 5.17).

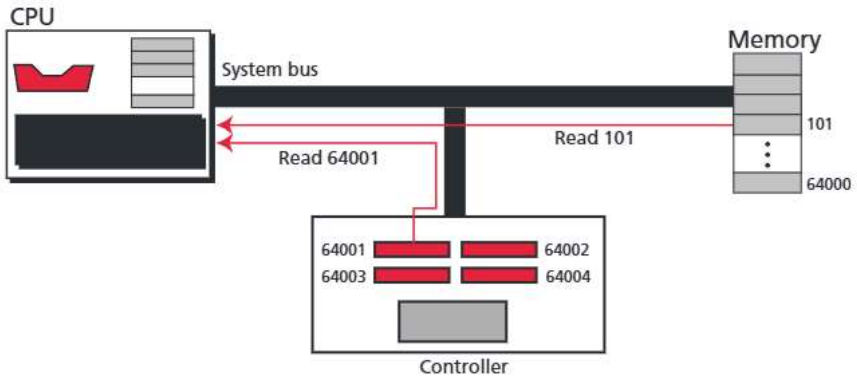
Figure 5.17 *Isolated I/O addressing*



Memory-mapped I/O

In the **memory-mapped I/O** method, the CPU treats each register in the I/O controller as a word in memory. In other words, the CPU does not have separate instructions for transferring data from memory and I/O devices. For example, there is only one 'Read' instruction. If the address defines a word from memory, the data is read from that word. If the address defines a register from an I/O device, the data is read from that register. The advantage of the memory-mapped configuration is a smaller number of instructions: all the memory instructions can be used by I/O devices. The disadvantage is that part of the memory address space is allocated to registers in I/O controllers. For example, if we have five I/O controllers and each has four registers, 20 addresses are used for this purpose. The size of the memory is reduced by 20 words. Figure 5.18 shows the memory-mapped I/O concept.

Figure 5.18 *Memory-mapped I/O addressing*



5.6 PROGRAM EXECUTION

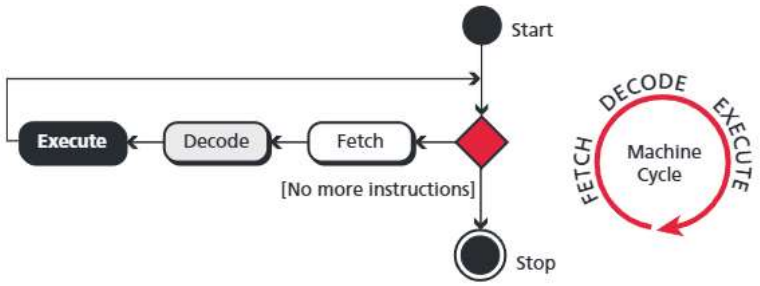
Today, general-purpose computers use a set of instructions called a *program* to process data. A computer executes the program to create output data from input data. Both the program and the data are stored in memory.

At the end of this chapter we give some examples of how a hypothetical simple computer executes a program.

5.6.1 Machine cycle

The CPU uses repeating machine cycles to execute instructions in the program, one by one, from beginning to end. A simplified cycle can consist of three phases: *fetch*, *decode*, and *execute* (Figure 5.19).

Figure 5.19 *The steps of a cycle*



Fetch

In the **fetch** phase, the control unit orders the system to copy the next instruction into the instruction register in the CPU. The address of the instruction to be copied is held in the program counter register. After copying, the program counter is incremented to refer to the next instruction in memory.

Decode

The second phase in the cycle is the **decode** phase. When the instruction is in the instruction register, it is decoded by the control unit. The result of this decode step is the binary code for some operation that the system will perform.

Execute

After the instruction is decoded, the control unit sends the task order to a component in the CPU. For example, the control unit can tell the system to load (read) a data item from memory, or the CPU can tell the ALU to add the contents of two input registers and put the result in an output register. This is the **execute** phase.

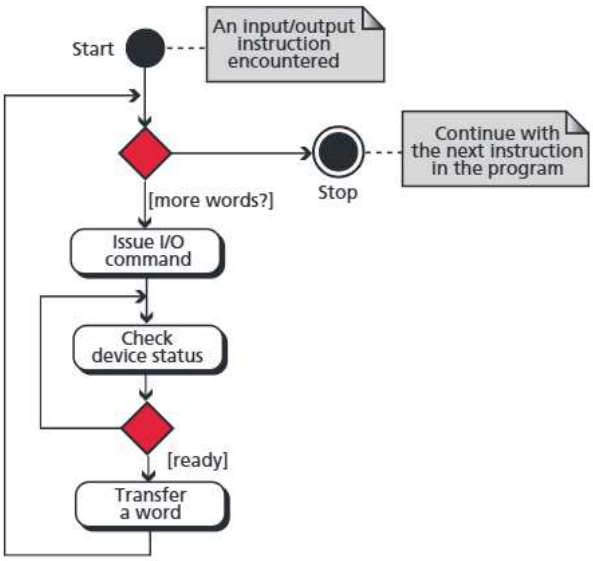
5.6.2 Input/output operation

Commands are required to transfer data from I/O devices to the CPU and memory. Because I/O devices operate at much slower speeds than the CPU, the operation of the CPU must be somehow synchronized with the I/O devices. Three methods have been devised for this synchronization: programmed I/O, interrupt-driven I/O, and direct memory access (DMA).

Programmed I/O

In the **programmed I/O** method, synchronization is very primitive: the CPU waits for the I/O device (Figure 5.20).

Figure 5.20 *Programmed I/O*



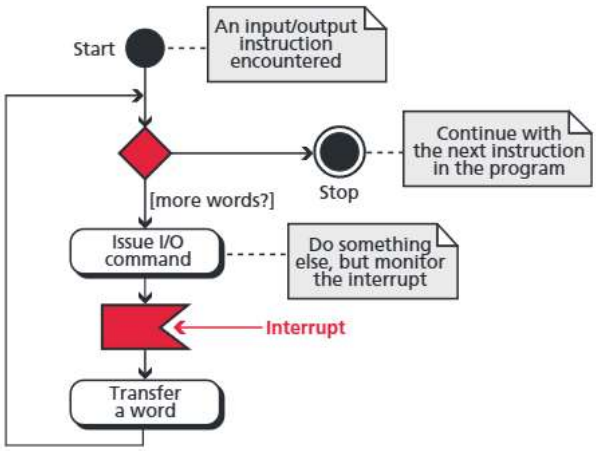
The transfer of data between the I/O device and the CPU is done by an instruction in the program. When the CPU encounters an I/O instruction, it does nothing else until the data transfer is complete. The CPU constantly checks the status of the I/O device: if the device is ready to transfer, data is transferred to the CPU. If the device is not ready, the CPU continues checking the device status until the I/O device is ready. The big issue here is that CPU time is wasted by checking the status of the I/O device for each unit of data to be transferred. Note that data is transferred to memory after the input operation, while data is transferred from memory before the output operation.

Interrupt-driven I/O

In the **interrupt-driven I/O** method, the CPU informs the I/O device that a transfer is going to happen, but it does not test the status of the I/O device continuously. The I/O device informs (interrupts) the CPU when it is ready. During this time, the CPU can do other jobs such as running other programs or transferring data from or to other I/O devices (Figure 5.21).

In this method, CPU time is not wasted—the CPU can do something else while the slow I/O device is finishing a task. Note that, like programmed I/O, this method also transfers data between the device and the CPU. Data is transferred to memory after the input operation. while data is transferred from memory before the output operation.

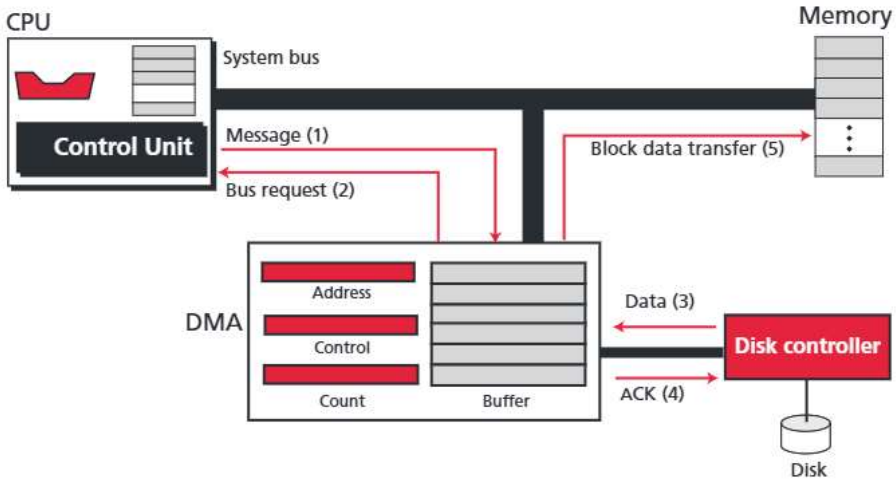
Figure 5.21 *Interrupt-driven I/O*



Direct memory access (DMA)

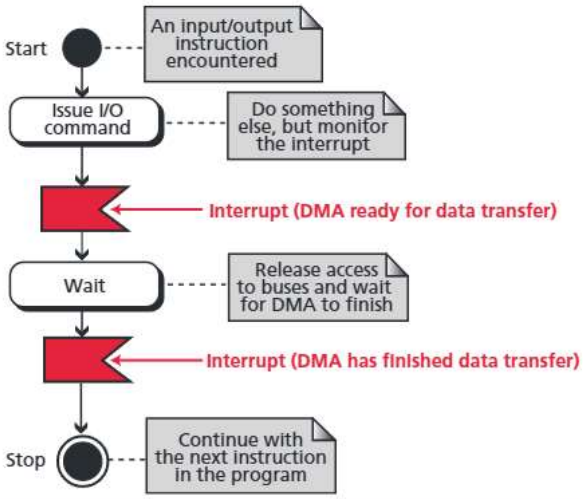
The third method used for transferring data is **direct memory access (DMA)**. This method transfers a large block of data between a high-speed I/O device, such as a disk, and memory directly without passing it through the CPU. This requires a DMA controller that relieves the CPU of some of its functions. The DMA controller has registers to hold a block of data before and after memory transfer. Figure 5.22 shows the DMA connection to the data, address, and control buses.

Figure 5.22 *DMA connection to the general bus*



Using this method for an I/O operation, the CPU sends a message to the DMA. The message contains the type of transfer (input or output), the beginning address of the memory location, and the number of bytes to be transferred. The CPU is then available for other jobs (Figure 5.23).

Figure 5.23 *DMA input/output*



When ready to transfer data, the DMA controller informs the CPU that it needs to take control of the buses. The CPU stops using the buses and lets the controller use them. After data transfer directly between the DMA and memory, the CPU continues its normal operation. Note that, in this method, the CPU is idle for a time. However, the duration of this idle period is very short compared to other methods—the CPU is idle only during the data transfer between the DMA and memory, not while the device prepares the data.

5.7 DIFFERENT ARCHITECTURES

The architecture and organization of computers have gone through many changes in recent decades. In this section we discuss some common architectures and organizations that differ from the simple computer architecture we discussed earlier.

5.7.1 CISC

CISC (pronounced *sisk*) stands for **complex instruction set computer (CISC)**. The strategy behind CISC architectures is to have a large set of instructions, including complex ones. Programming CISC-based computers is easier than in other designs because there is a single instruction for both simple and complex tasks. Programmers therefore do not have to write a set of instructions to do a complex task.

The complexity of the instruction set makes the circuitry of the CPU and the control unit very complicated. The designers of CISC architectures have come up with a solution to reduce this complexity: programming is done on two levels. An instruction in machine language is not executed directly by the CPU—the CPU performs only simple operations, called *microoperations*. A complex instruction is transformed into a set of these simple operations and then executed by the CPU. This necessitates the addition of a special memory called *micromemory* that holds the set of operations for each complex instruction in the instruction set. The type of programming that uses microoperations is called *microprogramming*.

One objection to CISC architecture is the overhead associated with microprogramming and access to micromemory. However, proponents of the architecture argue that this compensates for smaller programs at the machine level. An example of CISC architecture can be seen in the Pentium series of processors developed by Intel.

5.7.2 RISC

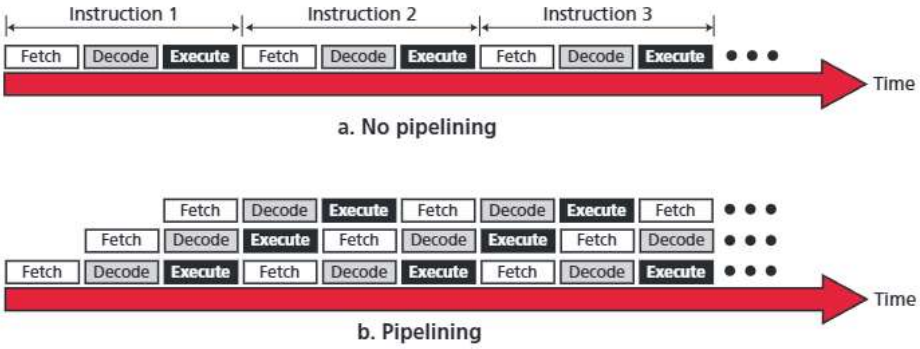
RISC (pronounced *risk*) stands for **reduced instruction set computer**. The strategy behind RISC architecture is to have a small set of instructions that do a minimum number of simple operations. Complex instructions are simulated using a subset of simple instructions. Programming in RISC is more difficult and time-consuming than in the other design because most of the complex instructions are simulated using simple instructions.

5.7.3 Pipelining

We have learned that a computer uses three phases of *fetch*, *decode*, and *execute* for each instruction. In early computers, these three phases needed to be done in series for each instruction. In other words, instruction n needs to finish all of these phases before the instruction $n + 1$ can start its own phases. Modern computers use a technique called pipelining to improve the **throughput** (the total number of instructions performed in each period of time). The idea is that if the control unit can do two or three of these phases simultaneously, the next instruction can start before the previous one is finished. Figure 5.24.a shows how three consecutive instructions are handled in a computer that uses no pipelining. Figure 5.24.b shows how pipelining can increase the throughput of the computer by allowing different types of phases belonging to different instructions to be done simultaneously. In other words, when the CPU is performing the decode phase of the first instruction, it can also perform the fetch phase of the second instruction. The first computer can perform on average 9 phases in the specific period of time, while the pipelined computer can perform 24 phases in the same period of time. If we assume that each phase uses the same amount of time, the first computer has done 9/3 = 3 instructions while the second computer has done 24/3 = 8 instructions. The throughput is therefore increased 8/3 or 266 per cent.

Of course, pipelining is not as easy as this. There are some problems, such as when a jump instruction is encountered. In this case, the instruction in the *pipe* should be discarded. However, new CPU designs have overcome most drawbacks. Some new CPU designs can even do several fetch cycles simultaneously.

Figure 5.24 *Pipelining*

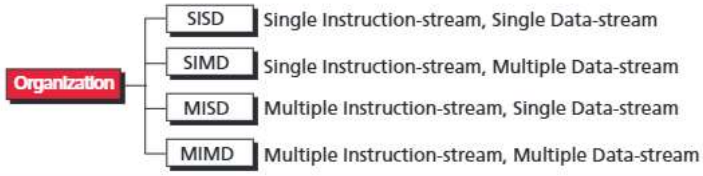


5.7.4 Parallel processing

Traditionally a computer had a single control unit, a single arithmetic logic unit, and a single memory unit. With the evolution in technology and the drop in the cost of computer hardware, today we can have a single computer with multiple control units, multiple arithmetic logic units and multiple memory units. This idea is referred to as *parallel processing*. Like pipelining, parallel processing can improve throughput.

Parallel processing involves many different techniques. A general view of parallel processing is given by the taxonomy proposed by M. J. Flynn. This taxonomy divides the computer's organization (in term of processing data) into four categories, as shown in Figure 5.25. According to Flynn, parallel processing may occur in the data stream, the instruction stream, or both.

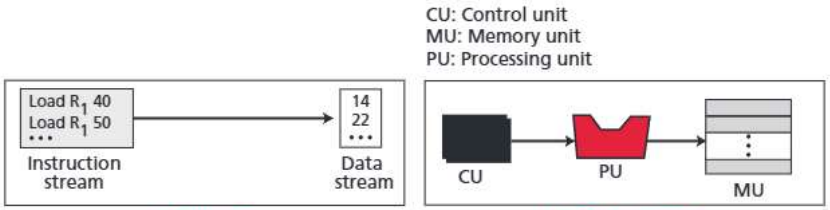
Figure 5.25 *A taxonomy of computer organization*



SISD organization

A **single instruction-stream, single data-stream (SISD)** organization represents a computer that has one control unit, one arithmetic logic unit, and one multiple memory units. The instructions are executed sequentially and each instruction may access one or more data items in the data stream. Our simple computer introduced earlier in the chapter is an example of SISD organization. Figure 5.26 shows the concept of configuration for an SISD organization.

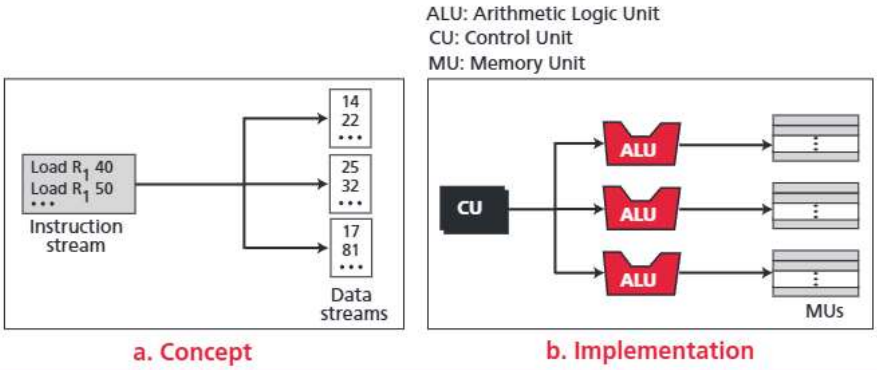
Figure 5.26 *SISD organization*



SIMD organization

A **single instruction-stream, multiple data-stream (SIMD)** organization represents a computer that has one control unit, multiple processing units, and multiple memory units. All processor units receive the same instruction from the control unit, but operate on different items of data. An array processor that simultaneously operates on an array of data belongs to this category. Figure 5.27 shows the concept and implementation of an SIMD organization.

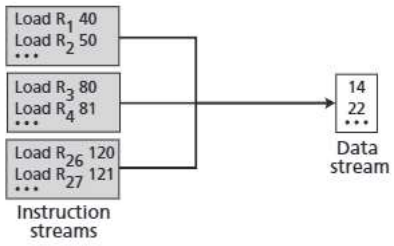
Figure 5.27 *SIMD organization*



MISD organization

A **multiple instruction-stream, single data-stream (MISD)** architecture is one in which several instructions belonging to several instruction streams simultaneously operate on the same data stream. Figure 5.28 shows the concept, but it has never been implemented.

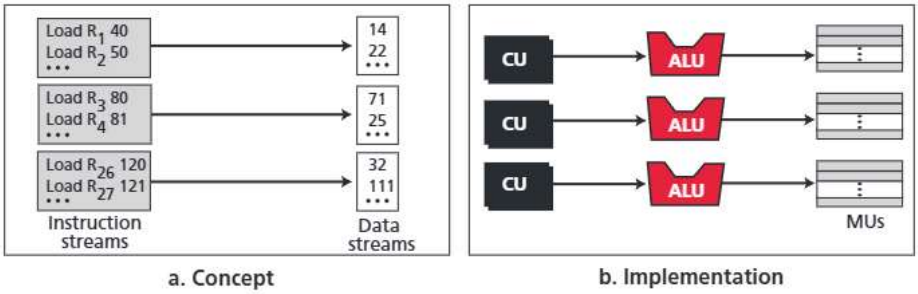
Figure 5.28 *MISD organization*



MIMD organization

A **multiple instruction-stream, multiple data-stream** (MIMD) architecture is one in which several instructions belonging to several instruction streams simultaneously operate on several data streams (each instruction on one data stream). Figure 5.29 shows the concept and implementation. MIMD organization is considered as a true parallel processing architecture by some experts. In this architecture several tasks can be performed simultaneously. The architecture can use a single shared memory or multiple memory sections.

Figure 5.29 *MIMD organization*

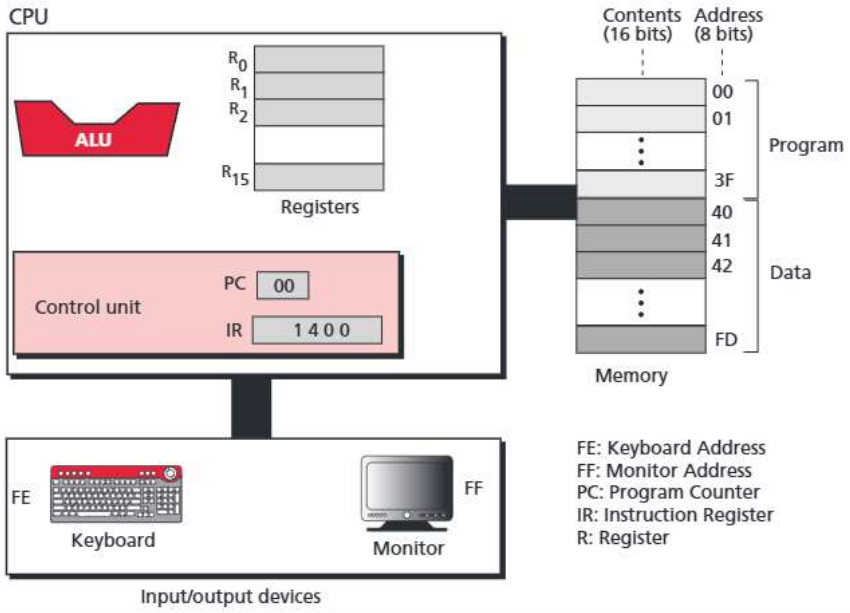


Parallel processing has found some applications, mostly in the scientific community, in which a task may take several hours or days if done using a traditional computer architecture. Some examples of this can be found in multiplication of very large matrices, in simultaneous processing of large amounts of data for weather prediction, or in space flight simulations.

5.8 A SIMPLE COMPUTER

To explain the architecture of computers as well as their instruction processing, we introduce a simple (unrealistic) computer, as shown in Figure 5.30. Our simple computer has three components: CPU, memory, and an input/output subsystem.

Figure 5.30 *The components of a simple computer*



5.8.1 CPU

The CPU itself is divided into three sections: data registers, arithmetic logic unit (ALU), and the control unit.

Data registers

There are sixteen 16-bit data registers with hexadecimal addresses $(0, 1, 2, \ldots, F)_{16}$, but we refer to them as R_0 to R_{15}. In most instructions, they hold 16-bit data, but in some instructions they may hold other information.

Control unit

The control unit has the circuitry to control the operations of the ALU, access to memory, and access to the I/O subsystem. In addition, it has two dedicated registers: program counter and instruction register. The program counter (PC), which can hold only eight bits, keeps track of which instruction is to be executed next. The contents of the PC points to the address of the memory location of the main memory that holds the next program instruction. After each machine cycle the program counter is incremented by one to point to the next program instruction. The instruction register (IR) holds a 16-bit value which is the encoded instruction for the current cycle.

5.8.2 Main memory

The main memory has 256 16-bit memory locations with binary addresses (00000000 to $11111101)_2$ or hexadecimal addresses (00 to $FD)_{16}$. The main memory holds both data and

program instruction. The first 64 locations $(00 \text{ to } 3F)_{16}$ are dedicated to program instructions. Program instructions for any program are stored in consecutive memory locations. Memory locations $(40 \text{ to } FD)_{16}$ are used for storing data.

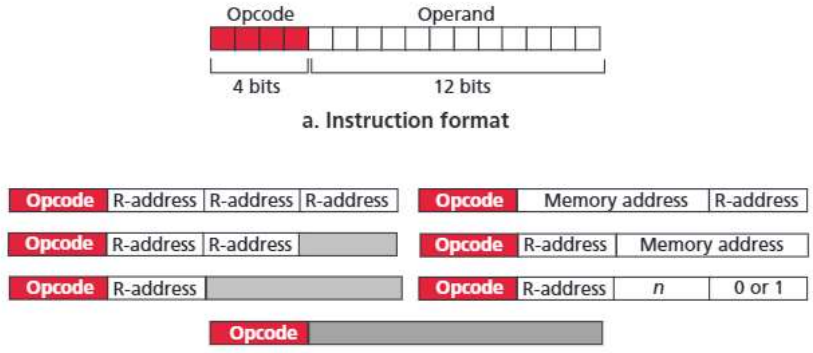
5.8.3 Input/output subsystem

Our simple computer has a very primitive input/output subsystem. The subsystem consists of a keyboard and a monitor. Although we show the keyboard and monitor in a separate box in Figure 5.30, the subsystem is part of the memory address-wise. These devices have memory-mapped addresses, as discussed earlier in the chapter. We assume that the keyboard (as the input device) and monitor (as the only output device) act like memory locations with addresses $(FE)_{16}$ and $(FF)_{16}$ respectively, as shown in the figure. In other words, we assume that they behave as 16-bit registers that interact with the CPU as a memory location would. These two devices transfer data from the outside world to the CPU and *vice versa*.

5.8.4 Instruction set

Our simple computer is capable of having a set of 16 instructions, although we are using only 14 of these instructions. Each computer instruction consists of two parts: the operation code (*opcode*) and the *operand(s)*. The opcode specifies the type of operation to be performed on the operand(s). Each instruction consists of 16 bits divided into four 4-bit fields. The leftmost field contains the opcode and the other three fields contain the operand or address of operand(s), as shown in Figure 5.31.

Figure 5.31 *Format and different instruction types*



The instructions are listed in Table 5.4 below. Note that not every instruction requires three operands. Any operand field not needed is filled with $(0)_{16}$. For example, all three operand fields of the halt instruction, and the last field of the move and NOT instructions, are filled with $(0)_{16}$. Also note that a register address is described by a single hexadecimal digit and thus uses a single field, but a memory location is described by two hexadecimal digits and uses two fields.

There are two add instructions: one for adding integers (ADDI) and one for adding floating points numbers (ADDF). The simple computer can take input from keyboard if we use

address $(FE)_{16}$ as the second operand of the LOAD instruction. Similarly, the computer sends output to the monitor if we use the address $(FF)_{16}$ as the second operand of the STORE instruction. If the third operand of the ROTATE instruction is 0, the instruction circularly rotates the bit pattern in R to the right *n* places: if the third operand is 1, it rotates it to the left. We have also included one increment (INC) and one decrement (DEC) instruction.

Table 5.4 List of instructions for the simple computer

Instruction	*Code*	*Operands*			*Action*
	d_1	d_2	d_3	d_4	
HALT	0				Stops the execution of the program
LOAD	1	R_D	M_S		$R_D \leftarrow M_S$
STORE	2	M_D		R_S	$M_D \leftarrow R_S$
ADDI	3	R_D	R_{S1}	R_{S2}	$R_D \leftarrow R_{S1} + R_{S2}$
ADDF	4	R_D	R_{S1}	R_{S2}	$R_D \leftarrow R_{S1} + R_{S2}$
MOVE	5	R_D	R_S		$R_D \leftarrow R_S$
NOT	6	R_D	R_S		$R_D \leftarrow \overline{R}_S$
AND	7	R_D	R_{S1}	R_{S2}	$R_D \leftarrow R_{S1}$ AND R_{S2}
OR	8	R_D	R_{S1}	R_{S2}	$R_D \leftarrow R_{S1}$ OR R_{S2}
XOR	9	R_D	R_{S1}	R_{S2}	$R_D \leftarrow R_{S1}$ XOR R_{S2}
INC	A	R			$R \leftarrow R + 1$
DEC	B	R			$R \leftarrow R - 1$
ROTATE	C	R	*n*	0 or 1	$Rot_n R$
JUMP	D	R	*n*		IF $R_0 \neq R$ then PC = *n*, otherwise continue

Key: R_S, R_{S1}, R_{S2}: Hexadecimal address of source registers
R_D: Hexadecimal address of destination register
M_S: Hexadecimal address of source memory location
M_D: Hexadecimal address of destination memory location
n: Hexadecimal number
d_1, d_2, d_3, d_4: First, second, third, and fourth hexadecimal digits

5.8.5 Processing the instructions

Our simple computer, like most computers, uses machine cycles. A cycle is made of three phases: *fetch*, *decode*, and *execute*. During the fetch phase, the instruction whose address is determined by the PC is obtained from the memory and loaded into the IR.

The PC is then incremented to point to the next instruction. During the *decode* phase, the instruction in the IR is decoded and the required operands are fetched from the register or from memory. During the *execute* phase, the instruction is executed and the results are placed in the appropriate memory location or the register. Once the third phase is completed, the control unit starts the cycle again, but now the PC is pointing to the next instruction. The process continues until the CPU reaches a HALT instruction.

An example

Let us show how our simple computer can add two integers A and B and create the result as C. We assume that integers are in two's complement format. Mathematically, we show this operation as:

$$C = A + B$$

To solve this problem with the simple computer, it is necessary for the first two integers to be held in two registers (for example, R_0 and R_1) and the result of the operation to be held in a third register (for example R_2). The ALU can only operate on the data that is stored in data registers in the CPU. However, most computers, including our simple computer, have a limited number of registers in the CPU. If the number of data items is large and they are supposed to stay in the computer for the duration of the program, it is better to store them in memory and only bring them to the registers temporarily. So we assume that the first two integers are stored in memory locations $(40)_{16}$ and $(41)_{16}$ and the result should be stored in memory location $(42)_{16}$. This means that two integers need to be loaded into the CPU and the result needs to be stored in the memory. Therefore, a simple program to do the simple addition needs five instructions, as shown below:

1. Load the contents of M_{40} into register R_0 ($R_0 \leftarrow M_{40}$).
2. Load the contents of M_{41} into register R_1 ($R_1 \leftarrow M_{41}$).
3. Add the contents of R_0 and R_1 and place the result in R_2 ($R_2 \leftarrow R_0 + R_1$).
4. Store the contents R_2 in M_{42} ($M_{42} \leftarrow$ R2).
5. Halt.

In the language of our simple computer, these five instructions are encoded as:

Code	*Interpretation*			
$(1040)_{16}$	1: LOAD	0: R_0	40: M_{40}	
$(1141)_{16}$	1: LOAD	1: R_1	41: M_{41}	
$(3201)_{16}$	3: ADDI	2: R_2	0: R_0	1: R_1
$(2422)_{16}$	2: STORE	42: M_{42}		2: R_2
$(0000)_{16}$	0: HALT			

5.8.6 Storing program and data

To follow the von Neumann model, we need to store the program and the data in memory. We can store the five-line program in memory starting from location $(00)_{16}$ to $(04)_{16}$. We already know that the data needs to be stored in memory locations $(40)_{16}$, $(41)_{16}$, and $(42)_{16}$.

5.8.7 Cycles

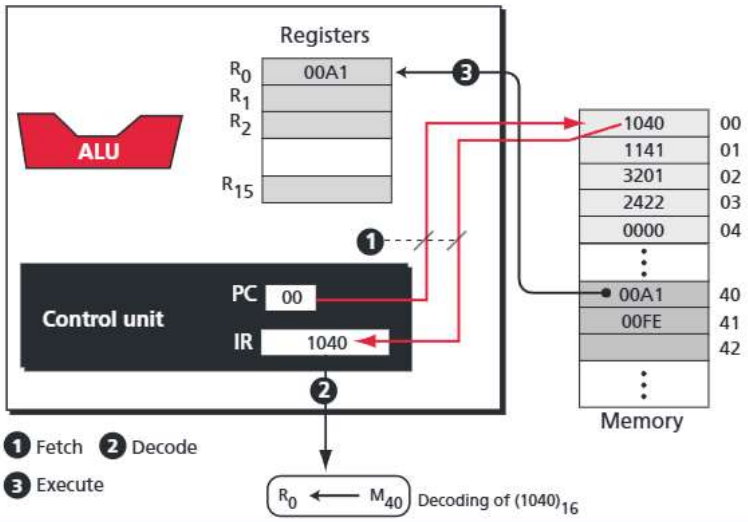
Our computer uses one cycle per instruction. If we have a small program with five instructions, we need five cycles. We also know that each cycle is normally made up of three steps: *fetch*, *decode*, *execute*. Assume for the moment that we need to add 161 + 254 = 415. The numbers are shown in memory in hexadecimal is, $(00A1)_{16}$, $(00FE)_{16}$, and $(019F)_{16}$.

Cycle 1

At the beginning of the first cycle (Figure 5.32), the PC points to the first instruction of the program, which is at memory location $(00)_{16}$. The control unit goes through three steps:

1. The control unit *fetches* the instruction stored in memory location $(00)_{16}$ and puts it in the IR. After this step, the value of the PC is incremented.
2. The control unit *decodes* the instruction $(1040)_{16}$ as $R_0 \leftarrow M_{40}$.
3. The control unit *executes* the instruction, which means that a copy of the integer stored in memory location (40) is loaded into register R_0.

Figure 5.32 *Status of cycle 1*

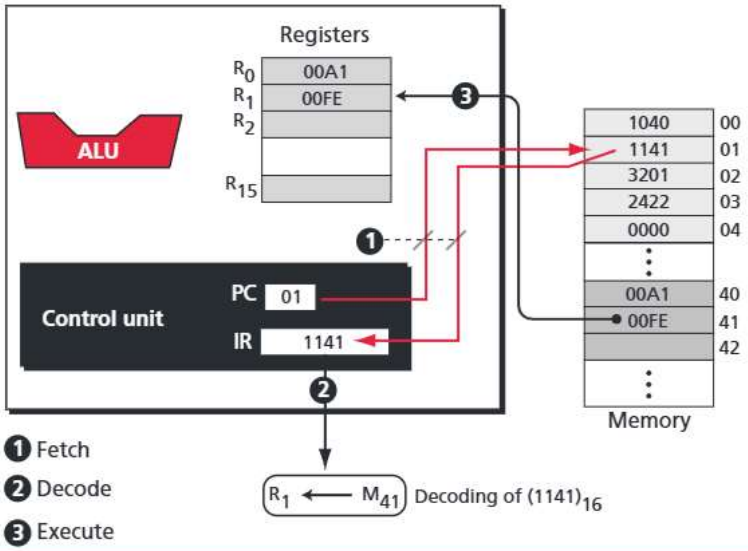


Cycle 2

At the beginning of the second cycle (Figure 5.33), the PC points to the second instruction of the program, which is at memory location $(01)_{16}$. The control unit goes through three steps:

1. The control unit *fetches* the instruction stored in memory location $(01)_{16}$ and puts it in the IR. After this step, the value of the PC is incremented.
2. The control unit *decodes* the instruction $(1141)_{16}$ as $R_1 \leftarrow M_{41}$.
3. The control unit *executes* the instruction, which means that a copy of integer stored in memory location $(41)_{16}$ is loaded into register R_1.

Figure 5.33 *Status of cycle 2*



Cycle 3

At the beginning of the third cycle (Figure 5.34), the PC points to the third instruction of the program, which is at memory location $(02)_{16}$. The control unit goes through three steps:

1. The control unit *fetches* the instruction stored in memory location $(02)_{16}$ and puts it in the IR. After this step, the value of the PC is incremented.
2. The control unit *decodes* the instruction $(3201)_{16}$ as $R_2 \leftarrow R_0 + R_1$.
3. The control unit *executes* the instruction, which means that the contents of R_0 is added to the content of R_1 (by the ALU) and the result is put in R_2.

Cycle 4

At the beginning of the fourth cycle (Figure 5.35), the PC points to the fourth instruction of the program, which is at memory location $(03)_{16}$. The control unit goes through three steps:

1. The control unit *fetches* the instruction stored in memory location $(03)_{16}$ and puts it in the IR. After this step, the value of the PC is incremented.
2. The control unit *decodes* the instruction $(2422)_{16}$ as $M_{42} \leftarrow R_2$.
3. The control unit *executes* the instruction, which means a copy of integer in register R_2 is stored in memory location $(42)_{16}$.

Figure 5.34 *Status of cycle 3*

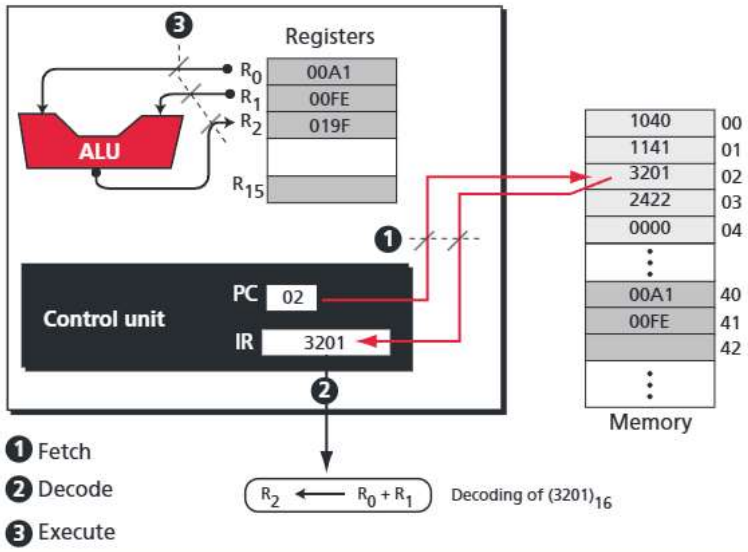
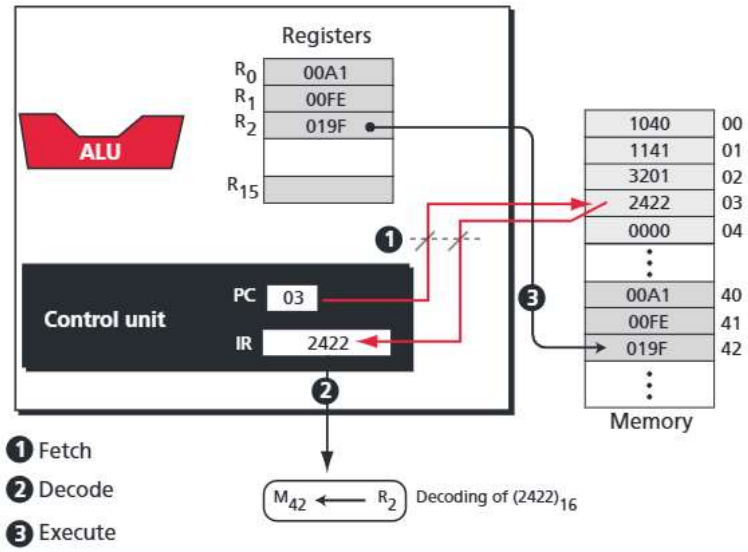


Figure 5.35 *Status of cycle 4*

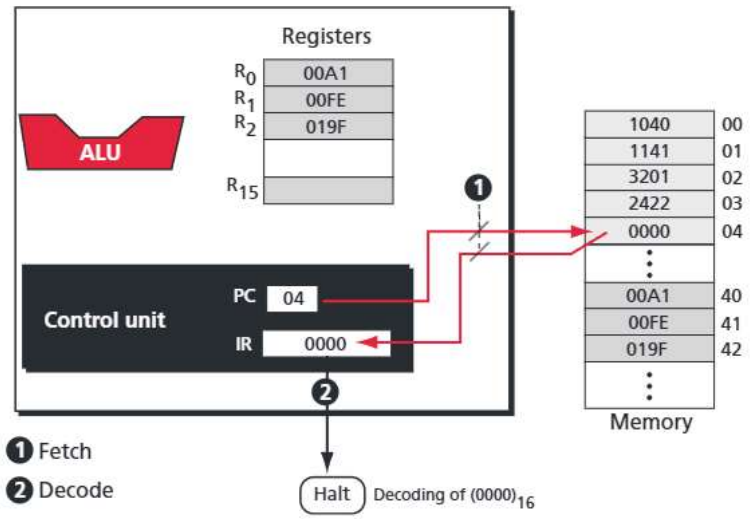


Cycle 5

At the beginning of the fifth cycle (Figure 5.36), the PC points to the fifth instruction of the program, which is at memory location $(04)_{16}$. The control unit goes through three steps:

1. The control unit *fetches* the instruction stored in memory location $(04)_{16}$ and puts it in the IR. After this step, the value of the PC is incremented.
2. The control unit *decodes* the instruction $(0000)_{16}$ as Halt.
3. The control unit *executes* the instruction, which means that the computer stops.

Figure 5.36 *Status of cycle 5*



5.8.8 Another example

In the previous example we assumed that the two integers to be added were already in memory. We also assume that the result of addition will be held in memory. You may ask how we can store the two integers we want to add in memory, or how we use the result when it is stored in the memory. In a real situation, we enter the first two integers into memory using an input device such as keyboard, and we display the third integer through an output device such as a monitor. Getting data via an input device is normally called a *read* operation, while sending data to an output device is normally called a *write* operation. To make our previous program more practical, we need to modify it as follows:

1. **Read an integer into M_{40}.**
2. **$R_0 \leftarrow M_{40}$.**
3. **Read an integer into M_{41}.**
4. **$R_1 \leftarrow M_{41}$.**
5. **$R_2 \leftarrow R_0 + R_1$.**
6. **$M_{42} \leftarrow R_2$.**
7. **Write the integer from M_{42}.**
8. **Halt.**

There are many ways to implement input and output. Most computers today do direct data transfer from an input device to memory and direct data transfer from memory to an

output device. However, our simple computer is not one of them. In our computer we can simulate read and write operations using the LOAD and STORE instruction. Furthermore, LOAD and STORE read data input to the CPU and write data from the CPU. We need two instructions to read data into memory or write data out of memory. The read operation is:

$R \leftarrow M_{FE}$ **Because the keyboard is assumed to be memory location** $(FE)_{16}$
$M \leftarrow R$

The write operation is:

$R \leftarrow M$
$M_{FF} \leftarrow R$ **Because the monitor is assumed to be memory location** $(FF)_{16}$

You may ask, if the operations are supposed to be done in the CPU, why do we transfer the data from the keyboard to the CPU, then to the memory, then to the CPU for processing? Could we directly transfer data to the CPU? The answer is that we can do this for this small problem, but we should not do it in principle. Think what happens if we need to add 1000 numbers or sort 1 000 000 integers. The number of registers in the CPU is limited (it may be hundreds in a real computer, but still not enough).

The input operation must always read data from an input device into memory: the output operation must always write data from memory to an output device.

With this in mind, the program is coded as:

1	$(1FFE)_{16}$	5	$(1040)_{16}$	9	$(1F42)_{16}$
2	$(240F)_{16}$	6	$(1141)_{16}$	10	$(2FFF)_{16}$
3	$(1FFE)_{16}$	7	$(3201)_{16}$	11	$(0000)_{16}$
4	$(241F)_{16}$	8	$(2422)_{16}$		

Operations 1 to 4 are for input and operations 9 and 10 are for output. When we run this program, it waits for the user to input two integers on the keyboard and press the enter key. The program then calculates the sum and displays the result on the monitor.

5.8.9 Reusability

One of the advantages of a computer over a non-programmable calculator is that we can use the same program over and over. We can run the program several times and each time enter different inputs and obtain a different output.

5.9 END-CHAPTER MATERIALS

5.9.1 Recommended reading

For more details about subjects discussed in this chapter, the following books are recommended:

- Englander, I. *The Architecture of Computer Hardware and Systems Software*, Hoboken, NJ: Wiley, 2003
- Mano, M. *Computer System Architecture*, Upper Saddle River, NJ: Prentice-Hall, 1993
- Null, L. and Lobur, J. *Computer Organization and Architecture*, Sudbury, MA: Jones and Bartlett, 2003
- Hamacher, C., Vranesic, Z. and Zaky, S. *Computer Organization*, New York: McGraw-Hill, 2002
- Warford, S. *Computer Systems*, Sudbury, MA: Jones and Bartlett, 2005
- Ercegovac, M., Lang, T. and Moreno, J. *Introduction to Digital Systems*, Hoboken, NJ: Wiley, 1998
- Cragon, H. *Computer Architecture and Implementation*, Cambridge: Cambridge University Press, 2000
- Stallings, W. *Computer Organization and Architecture*, Upper Saddle River, NJ: Prentice-Hall, 2002

5.9.2 Key terms

address bus 105	magnetic disk 98
address space 94	magnetic tape 99
arithmetic logic unit (ALU) 92	main memory 94
bus 104	master disk 100
cache memory 97	memory mapped I/O 109
central processing unit (CPU) 92	monitor 98
compact disk (CD) 99	multiple instruction-stream, multiple data-stream (MIMD) 117
compact disk read-only memory (CD-ROM) 100	multiple instruction-stream, single data-stream (MISD) 116
compact disk recordable (CD-R) 102	nonstorage device 98
complex instruction set computer (CISC) 113	optical storage device 99
control bus 105	output device 98
controller 105	parallel processing 115
control unit 94	pipelining 114
data bus 104	pit 100
decode 110	polycarbonate resin 100
digital versatile disk (DVD) 103	printer 98
direct memory access (DMA) 112	program counter 93

(*Continued*)

dynamic RAM (DRAM) 96	programmed I/O 110
electrically erasable programmable read-only memory (EEPROM) 96	random access memory (RAM) 95
erasable programmable read-only memory (EPROM) 96	read-only memory (ROM) 96
execute 110	read/write head 98
fetch 110	reduced instruction set computer (RISC) 114
FireWire 106	register 93
HDMI (High-Definition Multimedia Interface) 108	rotational speed 99
hub 107	sector 99
input/output controller 105	seek time 99
input/output subsystem 97	single instruction-stream, multiple data-stream (SIMD) 116
instruction register 93	static RAM (SRAM) 95
interrupt-driven I/O 111	storage device 98
intersector gap 99	throughput 114
intertrack gap 99	topology 107
isolated I/O 108	track 99
land 100	transfer time 99
machine cycle 109	Universal Serial Bus (USB) 107
programmable read-only memory (PROM) 96	write once, read many (WORM) 102

5.9.3 Summary

- The parts that make up a computer can be divided into three broad categories or subsystems: the central processing unit (CPU), the main memory, and the input/output subsystem.
- The central processing unit (CPU) performs operations on data. It has three parts: an arithmetic logic unit (ALU), a control unit, and a set of registers. The arithmetic logic unit (ALU) performs logic, shift, and arithmetic operations on data. Registers are fast stand-alone storage locations that hold data temporarily. The control unit controls the operation of each part of the CPU.

- Main memory is a collection of storage locations, each with a unique identifier called the *address*. Data is transferred to and from memory in groups of bits called *words*. The total number of uniquely identifiable locations in memory is called the *address space*. Two types of memory are available: random access memory (RAM) and read-only memory (ROM).
- The collection of devices referred to as the input/output (I/O) subsystem allows a computer to communicate with the outside world and to store programs and data even when the power is off. Input/output devices can be divided into two broad categories: nonstorage and storage devices. Nonstorage devices allow the CPU/memory to communicate with the outside world. Storage devices can store large amounts of information to be retrieved at a later time. Storage devices are categorized as either magnetic or optical.
- The interconnection of the three subsystems of a computer plays an important role, because information needs to be exchanged between these subsystems. The CPU and memory are normally connected by three groups of connections, each called a *bus*: data bus, address bus, and control bus. Input/output devices are attached to the buses through an *input/output controller* or interface. Several kinds of controllers are in use. The most common ones today are SCSI, FireWire, and USB.
- There are two methods of handling the addressing of I/O devices: isolated I/O and memory-mapped I/O. In the isolated I/O method, the instructions used to read/write to and from memory are different from the instructions used to read/write to and from input/output devices. In the memory-mapped I/O method, the CPU treats each register in the I/O controller as a word in memory.
- Today, general-purpose computers use a set of instructions called a *program* to process data. A computer executes the program to create output data from input data. Both the program and the data are stored in memory. The CPU uses repeating machine cycles to execute instructions in the program, one by one, from beginning to end. A simplified cycle can consist of three phases: *fetch*, *decode*, and *execute*.
- Three methods have been devised for synchronization between I/O devices and the CPU: programmed I/O, interrupt-driven I/O, and direct memory access (DMA).
- The architecture and organization of computers have gone through many changes during recent decades. We can divide computer architecture into two broad categories: CISC (complex instruction set computers) and RISC (reduced instruction set computers).
- Modern computers use a technique called *pipelining* to improve their throughput. The idea is to allow the control unit to perform two or three phases simultaneously, which means that processing of the next instruction can start before the previous one is finished.
- Traditionally, a computer had a single control unit, a single arithmetic logic unit, and a single memory unit. Parallel processing can improve throughput by using multiple instruction streams to handle multiple data streams.

5.10 PRACTICE SET

5.10.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

5.10.2 Review questions

Q5-1. What are the three subsystems that make up a computer?
Q5-2. What are the components of a CPU?
Q5-3. What is the function of the ALU?
Q5-4. What is the function of the control unit?
Q5-5. What is the function of main memory?
Q5-6. Define RAM, ROM, SRAM, DRAM PROM, EPROM, and EEPROM.
Q5-7. What is the purpose of cache memory?
Q5-8. Describe the physical components of a magnetic disk.
Q5-9. How are the surfaces of a magnetic disk and magnetic tape organized?
Q5-10. Compare and contrast CD-R, CD-RW, and DVD.
Q5-11. Compare and contrast SCSI, FireWire and USB controllers.
Q5-12. Compare and contrast the two methods for handling the addressing of I/O devices.
Q5-13. Compare and contrast the three methods for handling the synchronization of the CPU with I/O devices.
Q5-14. Compare and contrast CISC architecture with RISC architecture.
Q5-15. Describe pipelining and its purpose.
Q5-16. Describe parallel processing and its purpose.

5.10.3 Problems

P5-1. A computer has 64 MB (megabytes) of memory. Each word is 4 bytes. How many bits are needed to address each single word in memory?
P5-2. How many bytes of memory are needed to store a full screen of data if the screen is made of 24 lines with 80 characters in each line? The system uses ASCII code, with each ASCII character stored as a byte.
P5-3. An imaginary computer has 16 data registers (R0 to R15), 1024 words in memory, and 16 different instructions (add, subtract, and so on). What is the minimum size of an add instruction in bits if a typical instruction uses the following format: *add M R2*.
P5-4. If the computer in P5-3 uses the same size of word for data and instructions, what is the size of each data register?
P5-5. What is the size of the instruction register in the computer in P5-3?

P5-6. What is the size of the program counter in the computer in P5-3?

P5-7. What is the size of the data bus in the computer in P5-3?

P5-8. What is the size of the address bus in the computer in P5-3?

P5-9. What is the minimum size of the control bus in the computer in P5-3?

P5-10. A computer uses isolated I/O addressing. Its memory has 1024 words. If each controller has 16 registers, how many controllers can be accessed by this computer?

P5-11. A computer uses memory-mapped I/O addressing. The address bus uses ten lines (10 bits). If memory is made up of 1000 words, how many four-register controllers can be accessed by the computer?

P5-12. Using the instruction set of the simple computer in Section 5.8, write the code for a program that performs the following calculation: D ← A + B + C, in which A, B, C, and D are integers in two's complement format. The user types the value of A, B, and, C, and the value of D is displayed on the monitor.

P5-13. Using the instruction set of the simple computer, write the code for a program that performs the following calculation:

$$B \leftarrow A + 3$$

A and 3 are integers in two's complement format. The user types the value of A and the value of B is displayed on the monitor. (Hint: use the increment instruction.)

P5-14. Using the instruction set of the simple computer in Section 5.8, write the code for a program that performs the following calculation:

$$B \leftarrow A - 2$$

A and are integers in two's complement format. The user types the value of A and the value of B is displayed on the monitor. (Hint: use the decrement instruction.)

P5-15. Using the instruction set of the simple computer in Section 5.8, write the code for a program that adds n integers typed on the keyboard and displays their sum. You need first to type the value of n. (Hint: use decrement and jump instructions and repeat the addition n times.)

P5-16. Using the instruction set of the simple computer in Section 5.8, write the code for a program that accepts two integers from the keyboard. If the first integer is 0, the program increment the second integer, and if the first integer is 1, the programs decrement the second integer. The first integer must be only 0 or 1 otherwise the program fails. The program displays the result of the increment or decrement.

CHAPTER 6

Computer Networks and Internet

The development of the personal computer has brought about tremendous changes for business, industry, science, and education. A similar revolution has occurred in networking. Technological advances are making it possible for communication links to carry more and faster signals. As a result, services are evolving to allow use of this expanded capacity. Research in this area has resulted in new technologies. One goal is to be able to exchange data such as text, audio, and video from all parts of the world. We want to access the Internet to download and upload information quickly and accurately and at any time.

Objectives

After studying this chapter, the student should be able to:

- Describe local and wide area networks (LANs and WANs).
- Distinguish an internet from the Internet.
- Describe the TCP/IP protocol suite as the network model in the Internet.
- Define the layers in the TCP/IP protocol suite and their relationship.
- Describe some applications at the application layer.
- Describe the services provided by the transport-layer protocols.
- Describe the services provided by the network-layer protocols
- Describe different protocols used at the data-link layer.
- Describe the duties of the physical layer.
- Describe the different transmission media used in computer networking.

6.1 OVERVIEW

Although the goal of this chapter is to discuss the Internet, a system that interconnects billions of computers in the world, we think of the Internet not as a single network, but as an **internetwork**, a combination of networks. Therefore, we start our journey by first defining a network. We then show how we can connect networks to create small internetworks. Finally, we show the structure of the Internet and open the gate to study the Internet in the rest of this chapter.

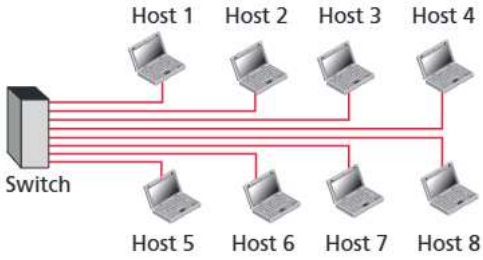
6.1.1 Networks

A **network** is the interconnection of a set of devices capable of communication. In this definition, a device can be a **host** (or an **end system** as it is sometimes called) such as a large computer, desktop, laptop, workstation, cellular phone, or security system. A device in this definition can also be a **connecting device** such as a **router** which connects the network to other networks, a **switch** which connects devices together, a modem (modulator-demodulator) that changes the form of data, and so on. These devices in a network are connected using wired or wireless transmission media such as cable or air. When we connect two computers at home using a plug-and-play router, we have created a network, although very small.

Local area network

A **local area network** (LAN) is usually privately owned and connects some hosts in a single office, building, or campus. Depending on the needs of an organization, a LAN can be as simple as two PCs and a printer in someone's home office, or it can extend throughout a company and include audio and video devices. Each host in a LAN has an identifier, an address, that uniquely defines the host in the LAN. A packet sent by a host to another host carries both the source host's and the destination host's addresses. Figure 6.1 shows an example of a LAN.

Figure 6.1 *Example of a LAN*

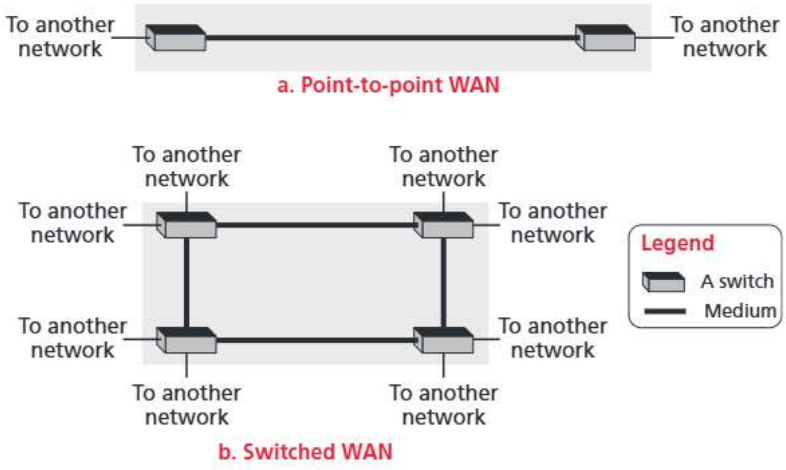


Wide area network

A **wide area network** (WAN) is also an interconnection of devices capable of communication. However, there are some differences between a LAN and a WAN. A LAN is normally

limited in size, spanning an office, a building, or a campus; a WAN has a wider geographical span, spanning a town, a state, a country, or even the world. A LAN interconnects hosts; a WAN interconnects connecting devices such as switches, routers, or modems. A LAN is normally privately owned by the organization that uses it; a WAN is normally created and run by communication companies and leased by an organization that uses it. We see two distinct examples of WANs today: **point-to-point** WANs and **switched** WANs as shown in Figure 6.2.

Figure 6.2 *A point-to-point and a switched WAN*



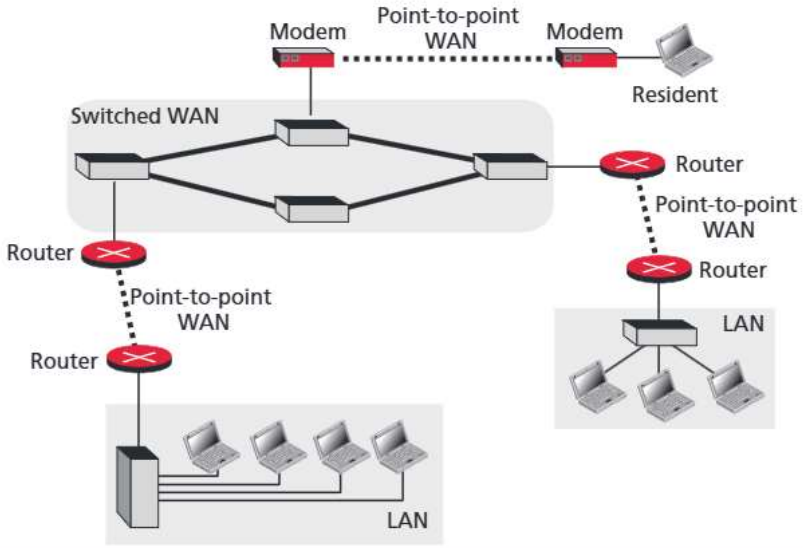
A point-to-point WAN is a network that connects two communicating devices through a transmission medium (cable or air).

A switched WAN is a network with more than two ends. A switched WAN, as we will see shortly, is used in the backbone of global communication today. We can say that a switched WAN is a combination of several point-to-point WANs that are connected by switches.

Internetwork

Today, it is very rare to see a LAN or a WAN in isolation; they are connected to one another. When two or more networks are connected, they make an internetwork, or **internet**. As an example, assume that an organization has two offices. Each office has a LAN that allows all employees in the office to communicate with each other. To make the communication between employees at different offices possible, the management leases a point-to-point dedicated WAN from a service provider, such as a telephone company, and connects the two LANs. Now the company has an internetwork, or a private internet (with lowercase *i*). Communication between offices is now possible. Figure 6.3 shows this internet.

Figure 6.3 *An internetwork made of two LANs and three WANs*



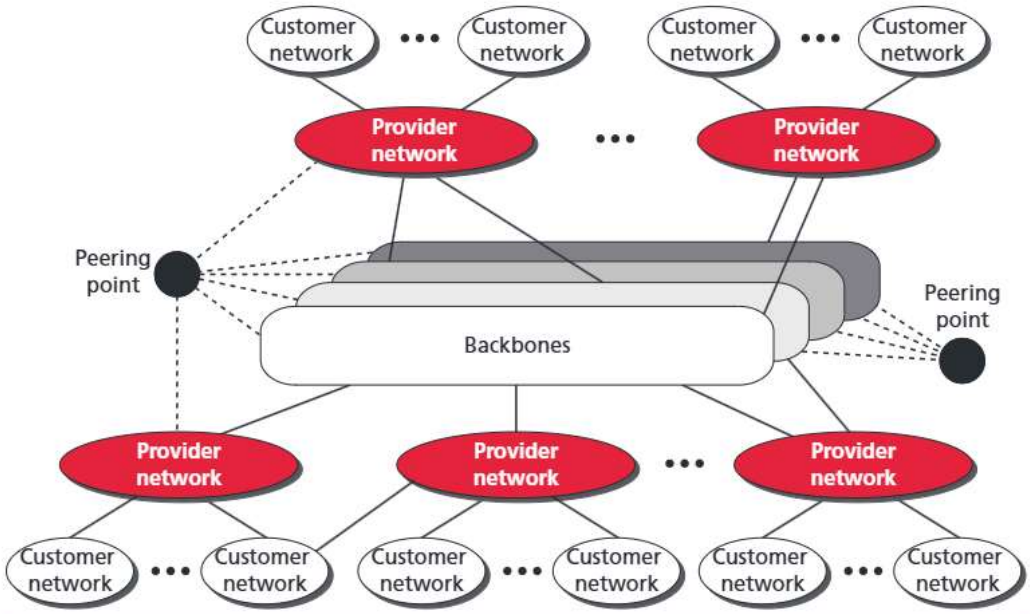
6.1.2 The Internet

As we discussed before, an internet (note the lowercase *i*) is two or more networks that can communicate with each other. The most notable internet is called the **Internet** (uppercase *I*), and is composed of thousands of interconnected networks. Figure 6.4 shows a conceptual (not geographical) view of the Internet.

The figure shows the Internet as several backbones, provider networks, and customer networks. At the top level, the *backbones* are large networks owned by some communication companies. The backbone networks are connected through some complex switching systems, called *peering points*. At the second level, there are smaller networks, called *provider networks*, that use the services of the backbones for a fee. The provider networks are connected to backbones and sometimes to other provider networks. The *customer networks* are networks at the edge of the Internet that actually use the services provided by the Internet. They pay fees to provider networks for receiving services.

Backbones and provider networks are also called **internet service providers (ISPs)**. The backbones are often referred to as international ISPs; the provider networks are often referred to as national or regional ISPs.

Figure 6.4 *The Internet today*



6.1.3 Hardware and software

We have given the overview of the Internet structure, which is made of small and large networks glued together with connecting devices. It should be clear, however, that if we only connect these pieces nothing will happen. For communication to happen, we need both **hardware** and **software**. This is similar to a complex computation in which we need both a computer and a program. In the next section, we show how these combinations of hardware and software are coordinated with each other using *protocol layering*.

6.1.4 Protocol layering

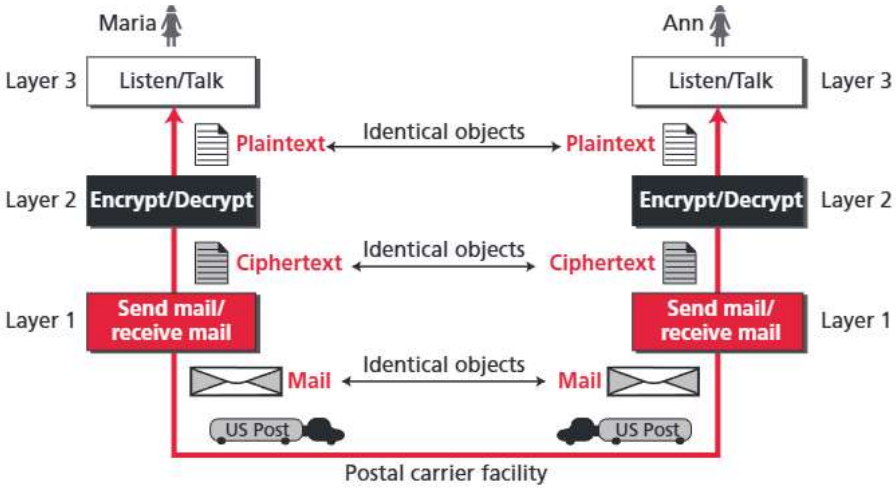
A word we hear all the time when we talk about the Internet is *protocol*. A **protocol** defines the rules that both the sender and receiver and all intermediate devices need to follow to be able to communicate effectively. When communication is simple, we may need only one simple protocol; when the communication is complex, we may need to divide the task between different layers, in which case we need a protocol at each layer, or **protocol layering**.

A scenario

Let us develop a simple scenario to better understand the need for protocol layering. We assume that Ann and Maria are neighbors with a lot of common ideas. They communicate with each other every time they meet about a project for the time when they will be retired.

Suddenly, Ann is offered a higher-level position in her company, but needs to move to another branch located in a city very far from Maria. The two friends still want to continue their communication and exchange ideas because they have come up with an innovative project to start a new business when they both retire. They decide to continue their conversation using regular mail through the post office. However, they do not want their ideas to be revealed to other people if the letters are intercepted. They agree on an encryption/decryption technique. The sender of the letter encrypts it to make it unreadable by an intruder; the receiver of the letter decrypts it to get the original letter. We discuss the encryption/decryption methods in Chapter 16, but for the moment we assume that Maria and Ann use one technique that makes it hard to decrypt the letter if one does not have the key for doing so. Now we can say that the communication between Maria and Ann takes place in three layers, as shown in Figure 6.5. We assume that Ann and Maria each have three machines (or robots) that can perform the task at each layer.

Figure 6.5 *A three-layer protocol*



Let us assume that Maria sends the first letter to Ann. Maria talks to the machine at the third layer as though the machine is Ann and is listening to her. The third-layer machine listens to what Maria says and creates the plaintext (a letter in English), which is passed to the second-layer machine. The second-layer machine takes the plaintext, encrypts it, and creates the **ciphertext**, which is passed to the first-layer machine. The first layer machine, presumably a robot, takes the ciphertext, puts it in an envelope, adds the sender and receiver addresses, and mails it.

At Ann's side, the first-layer machine picks up the letter from Ann's mail box, recognizing the letter from Maria by the sender address. The machine takes out the ciphertext from the envelope and delivers it to the second-layer machine. The second-layer machine decrypts the message, creates the plaintext, and passes the plaintext to the third-layer machine. The third-layer machine takes the plaintext and reads it as though Maria is speaking.

Protocol layering enables us to divide a complex task into several smaller and simpler tasks. For example, in Figure 6.5, we could have used only one machine to do the job of all three machines. However, if Maria and Ann decide that the encryption/decryption done by the machine is not enough to protect their secrecy, they have to change the whole machine. In the present situation, they need to change only the second-layer machine; the other two can remain the same. This is referred to as **modularity**. Modularity in this case means independent layers. A layer **(module)** can be defined as a black box with inputs and outputs, without concern about how inputs are changed to outputs. If two machines provide the same outputs when given the same inputs, they can replace each other. For example, Ann and Maria can buy the second-layer machine from two different manufacturers. As long as the two machines create the same ciphertext from the same plaintext and *vice versa*, they do the job.

One of the advantages of protocol layering is that it allows us to separate the services from the implementation. A layer needs to be able to receive a set of services from the lower layer and to give the services to the upper layer; we don't care about how the layer is implemented. For example, Maria may decide not to buy the machine (robot) for the first layer; she can do the job herself. As long as Maria can do the tasks provided by the first layer, in both directions, the communication system works.

Another advantage of protocol layering, which cannot be seen in our simple examples, but reveals itself when we discuss protocol layering in the Internet, is that communication does not always use only two end systems; there are intermediate systems that need only some layers, but not all layers. If we did not use protocol layering, we would have to make each intermediate system as complex as the end systems, which makes the whole system more expensive.

Is there any disadvantage to protocol layering? One can argue that having a single layer makes the job easier. There is no need for each layer to provide a service to the upper layer and give a service to the lower layer. For example, Ann and Maria could find or build one machine that could do all three tasks. However, as mentioned above, if one day they found that their code was broken, each would have to replace the whole machine with a new one instead of just changing the machine in the second layer.

Principles of protocol layering

Let us discuss some principles of protocol layering. The first principle dictates that if we want bidirectional communication, we need to make each layer so that it is able to perform two opposite tasks, one in each direction. For example, the third-layer task is to *listen* (in one direction) and *talk* (in the other direction). The second layer needs to be able to encrypt and decrypt. The first layer needs to send and receive mail.

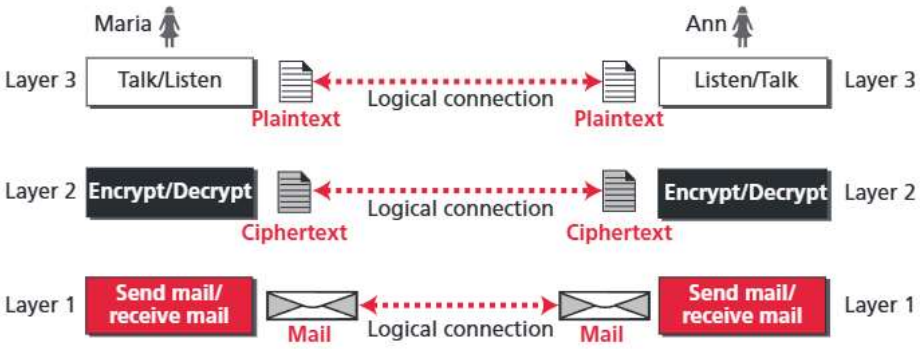
The second important principle that we need to follow in protocol layering is that the two objects under each layer at both sites should be identical. For example, the object under layer 3 at both sites should be a plaintext letter. The object under layer 2 at both sites should be a ciphertext letter. The object under layer 1 at both sites should be a piece of mail.

Logical connections

After following the above two principles, we can think about logical connection between each layer as shown in Figure 6.6. This means that we have layer-to-layer communication.

Maria and Ann can think that there is a logical (imaginary) connection at each layer through which they can send the object created from that layer. We will see that the concept of logical connection will help us better understand the task of layering we encounter in data communication and networking.

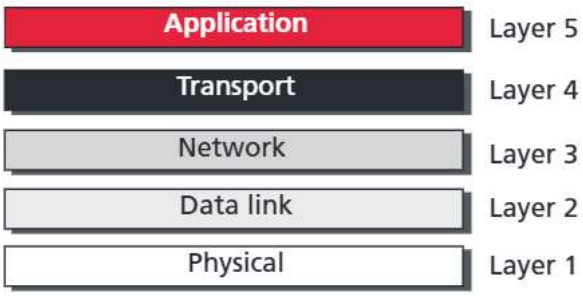
Figure 6.6 *Logical connection between peer layers*



6.1.5 TCP/IP protocol suite

Now that we know about the concept of protocol layering and the logical communication between layers in our second scenario, we can introduce the **TCP/IP (Transmission Control Protocol/Internet Protocol)**. TCP/IP is a protocol suite (a set of protocols organized in different layers) used in the Internet today. It is a hierarchical protocol made up of interactive modules, each of which provides a specific functionality. The term *hierarchical* means that each upper-level protocol is supported by the services provided by one or more lower-level protocols. The TCP/IP protocol suite is made of five layers as shown in Figure 6.7.

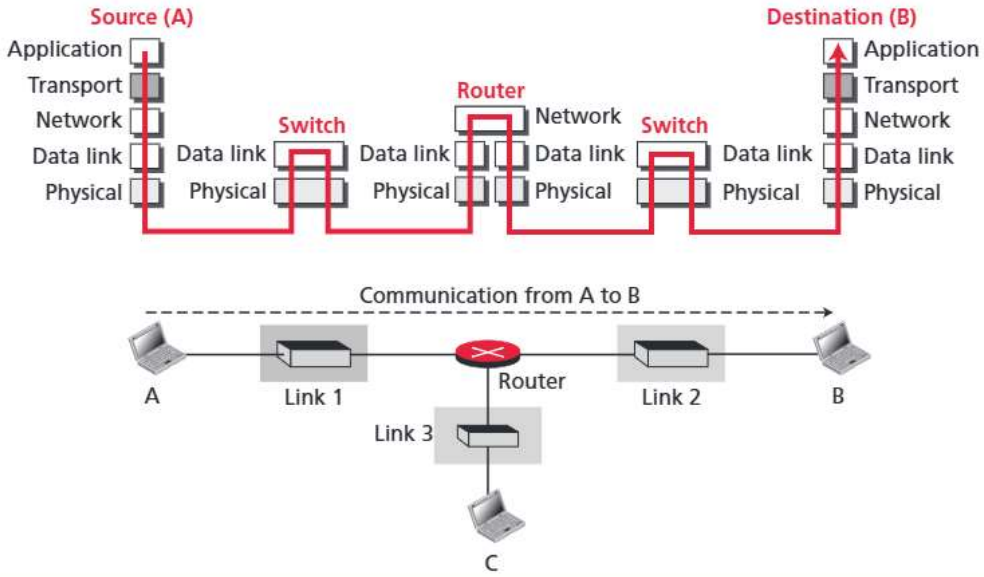
Figure 6.7 *Layers in the TCP/IP protocol suite*



Layered architecture

To show how the layers in the TCP/IP protocol suite are involved in communication between two hosts, we assume that we want to use the suite in a small internet made up of three LANs (links), each with a link-layer switch. We also assume that the links are connected by one router, as shown in Figure 6.8.

Figure 6.8 *Communication through an internet*



Let us assume that computer A communicates with computer B. As the figure shows, we have five communicating devices in this communication: source host (computer A), the link-layer switch in link 1, the router, the link-layer switch in link 2, and the destination host (computer B). Each device is involved with a set of layers depending on the role of the device in the internet. The two hosts are involved in all five layers; the source host needs to create a message in the **application layer** and send it down the layers so that it is physically sent to the destination host. The destination host needs to receive the communication at the **physical layer** and then deliver it through the other layers to the application layer.

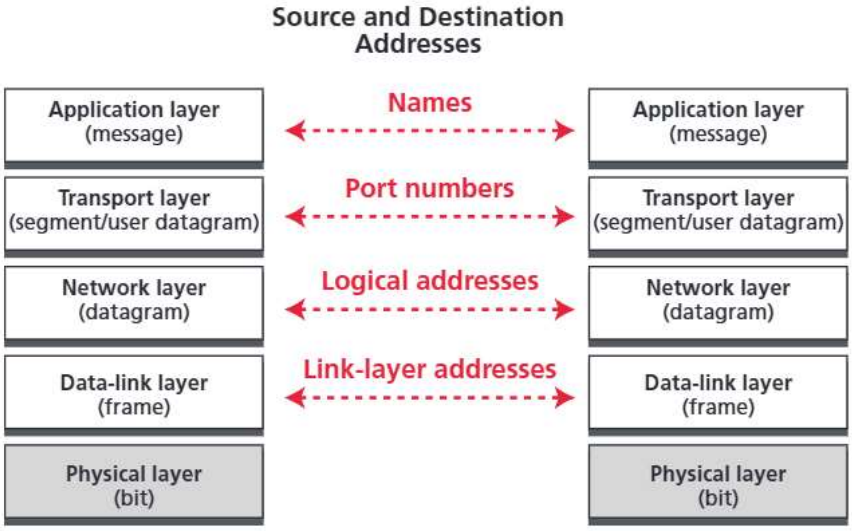
The router is involved only in three layers; there is no transport or application layer in a router as long as the router is used only for **routing**. Although a router is always involved in one network layer, it is involved in n combinations of link and physical layers in which n is the number of links the router is connected to. The reason is that each link may use its own data-link or physical protocol. For example, in the above figure, the router is involved in three links, but the message sent from source A to destination B is involved in two links. Each link may be using different link-layer and physical-layer protocols; the router needs to receive a packet from link 1 based on one pair of protocols and deliver it to link 2 based on another pair of protocols.

A link-layer switch in a link, however, is involved only in two layers, data-link and physical. Although each switch in the above figure has two different connections, the connections are in the same link, which uses only one set of protocols. This means that, unlike a router, a link-layer switch is involved only in one data-link and one physical layer.

Addressing and packet names

It is worth mentioning another two concepts related to protocol layering in the Internet: *addressing* and *packet names*. As we discussed before, we have logical communication between pairs of layers in this model. Any communication that involves two parties needs two addresses: source address and destination address. Although it looks as if we need five pairs of addresses, one pair per layer, we normally have only four because the physical layer does not need addresses; the unit of data exchange at the physical layer is a bit, which definitely cannot have an address. Figure 6.9 shows the addressing and the names of the packets at each layer.

Figure 6.9 *Addressing and packet names in the TCP/IP protocol suite*



As the figure shows, there is a relationship between the layer, the address used in that layer, and the packet name at that layer. At the application layer, we normally use names to define the site that provides services, such as *someorg.com*, or the email address, such as *somebody@coldmail.com*. At the transport layer, addresses are called **port numbers**, and these define the application-layer programs at the source and destination. Port numbers are local addresses that distinguish between several programs running at the same time. At the **network-layer**, the addresses are global, with the whole Internet as the scope. A network-layer address uniquely defines the connection of a device to the Internet. The link-layer

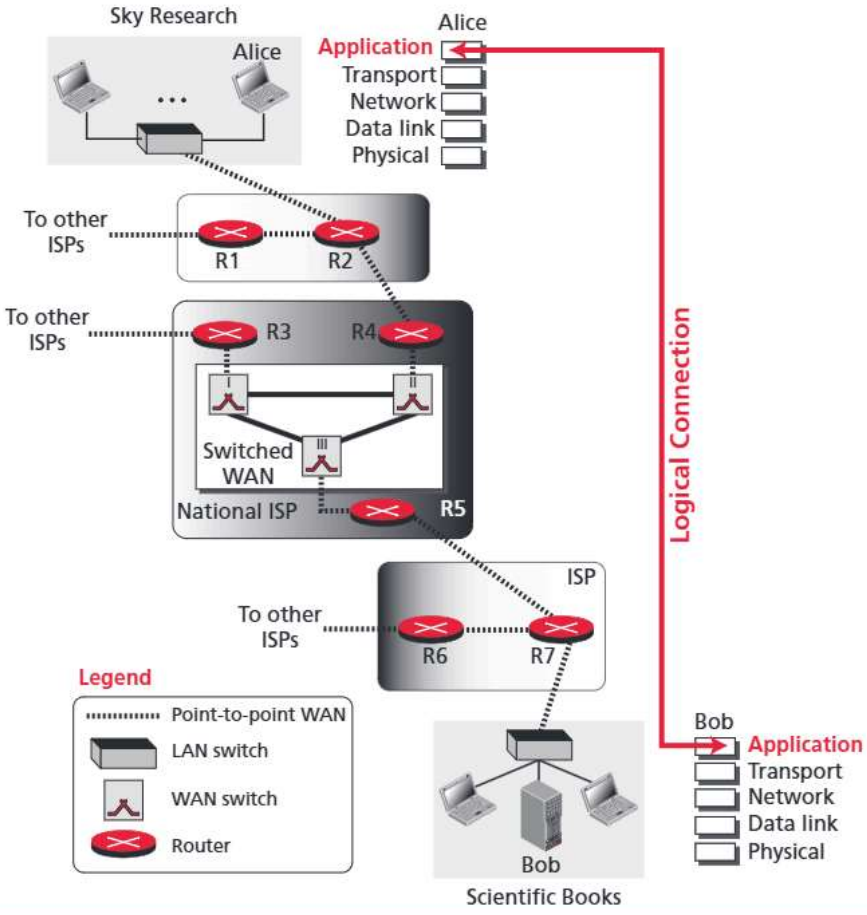
addresses, sometimes called **MAC addresses**, are locally defined addresses, each of which defines a specific host or router in a network (LAN or WAN).

6.2 APPLICATION LAYER

After the brief discussion of networks, internetworks, and the Internet, we are ready to give some discussion about each layer of the TCP/IP protocol. We start from the fifth layer and move to the first layer.

The fifth layer of the TCP/IP protocol is called the application layer. The application layer provides services to the user. Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages. Figure 6.10 shows the idea behind this logical connection.

Figure 6.10 *Logical connection at the application layer*



The figure shows a scenario in which a scientist working in a research company, Sky Research, needs to order a book related to her research from an online bookseller, Scientific Books. Logical connection takes place between the application layer at a computer at Sky Research and the application layer of a **server** at Scientific Books. We call the first host Alice and the second one Bob. The communication at the application layer is logical, not physical. Alice and Bob assume that there is a two-way logical channel between them through which they can send and receive messages. The actual communication, however, takes place through several devices (Alice, R2, R4, R5, R7, and Bob) and several physical channels as shown in the figure.

6.2.1 Providing services

The application layer is somehow different from other layers in that it is the highest layer in the suite. The protocols in this layer do not provide services to any other protocol in the suite; they only receive services from the protocols in the transport layer. This means that protocols can be removed from this layer easily. New protocols can be also added to this layer as long as the new protocol can use the service provided by one of the transport-layer protocols.

Since the application layer is the only layer that provides services to the Internet user, the flexibility of the application layer, as described above, allows new application protocols to be easily added to the Internet, which has been occurring during the lifetime of the Internet. When the Internet was created, only a few application protocols were available to the users; today we cannot give a number for these protocols because new ones are being added constantly.

6.2.2 Application-layer paradigms

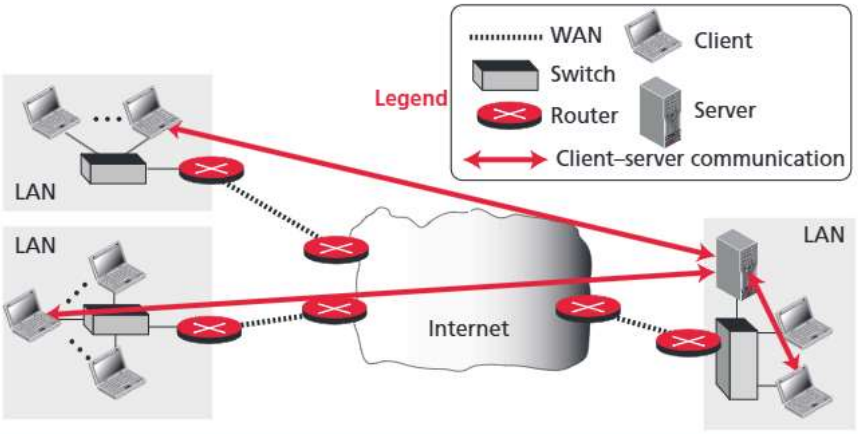
It should be clear that to use the Internet we need two application programs to interact with each other: one running on a computer somewhere in the world, the other running on another computer somewhere else in the world. The two programs need to send messages to each other through the Internet infrastructure. However, we have not discussed what the relationship should be between these programs. Should both application programs be able to request services and provide services, or should the application programs just do one or the other? Two paradigms have been developed during the lifetime of the Internet to answer this question: the *client–server paradigm* and the *peer-to-peer paradigm*. We briefly introduce these two paradigms here.

Traditional paradigm: client–server

The traditional paradigm is called the **client–server paradigm**. It was the most popular paradigm until a few years ago. In this paradigm, the service provider is an application program, called the server process; it runs continuously, waiting for another application program, called the client process, to make a connection through the Internet and ask for service. There are normally some server processes that can provide a specific type of service, but there are many clients that request service from any of these server processes. The server process must be running all the time; the client process is started when the client needs to receive service.

Although the communication in the client–server paradigm is between two application programs, the role of each program is totally different. In other words, we cannot run a client program as a server program or *vice versa*. Figure 6.11 shows an example of a client–server communication in which three clients communicate with one server to receive the services provided by this server.

Figure 6.11 *Example of a client–server paradigm*



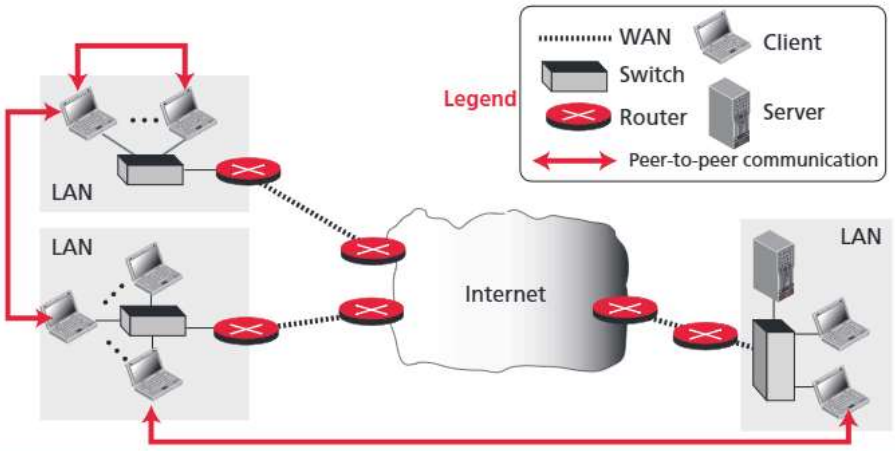
One problem with this paradigm is that the concentration of the communication load is on the shoulder of the server, which means the server should be a powerful computer. Even a powerful computer may become overwhelmed if a large number of clients try to connect to the server at the same time. Another problem is that there should be a service provider willing to accept the cost and create a powerful server for a specific service, which means the service must always return some type of income for the server in order to encourage such an arrangement.

Several traditional services are still using this paradigm, including the World Wide Web (WWW) and its vehicle HyperText Transfer Protocol (HTTP), file transfer protocol (FTP), secure shell (SSH), email, and so on. We discuss some of these protocols and applications later in the chapter.

New paradigm: peer-to-peer

A new paradigm, called the **peer-to-peer paradigm** (often abbreviated *P2P paradigm*) has emerged to respond to the needs of some new applications. In this paradigm, there is no need for a server process to be running all the time and waiting for the client processes to connect. The responsibility is shared between peers. A computer connected to the Internet can provide service at one time and receive service at another time. A computer can even provide and receive services at the same time. Figure 6.12 shows an example of communication in this paradigm.

Figure 6.12 *Example of a peer-to-peer paradigm*



One of the areas that really fits in this paradigm is Internet telephony. Communication by phone is indeed a peer-to-peer activity; no party needs to be running forever waiting for the other party to call. Another area in which the peer-to-peer paradigm can be used is when some computers connected to the Internet have something to share with each other. For example, if an Internet user has a file available to share with other Internet users, there is no need for the file holder to become a server and run a server process all the time waiting for other users to connect and retrieve the file.

Although the peer-to-peer paradigm has been proved to be easily scalable and cost-effective in eliminating the need for expensive servers to be running and maintained all the time, there are also some challenges. The main challenge has been **security**; it is more difficult to create secure communication between distributed services than between those controlled by some dedicated servers. The other challenge is applicability; it appears that not all applications can use this new paradigm. For example, not many Internet users are ready to become involved, if one day the Web can be implemented as a peer-to-peer service.

6.2.3 Standard client–server applications

During the lifetime of the Internet, several client–server application programs have been developed. We do not have to redefine them, but we need to understand what they do.

We have selected six standard application programs in this section. We start with HTTP and the World Wide Web because they are used by almost all Internet users. We then introduce file transfer and electronic mail applications which have high traffic loads on the Internet. Next, we explain remote logging and how it can be achieved using the TELNET and SSH protocols. Finally, we discuss DNS, which is used by all application programs to map the application layer identifier to the corresponding host IP address.

World Wide Web and HTTP

In this section, we first introduce the World Wide Web (abbreviated WWW or Web). We then discuss the HyperText Transfer Protocol (HTTP), the most common client–server application program used in relation to the Web.

World Wide Web

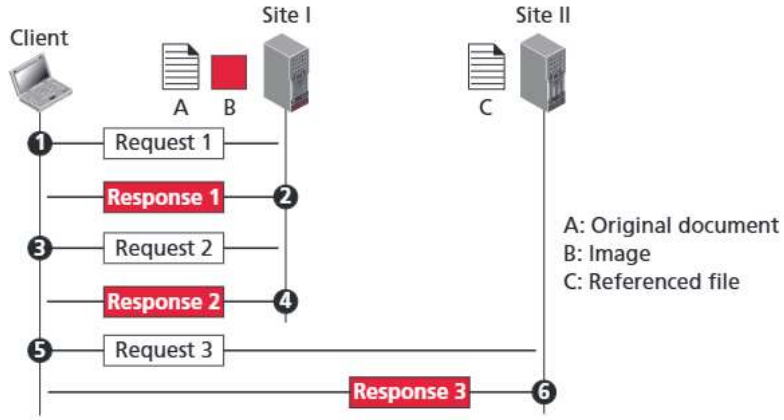
The Web today is a repository of information in which the documents, called Web pages, are distributed all over the world and related documents are linked together. The popularity and growth of the Web can be related to two terms in the above statement: distributed and linked. Distribution allows the growth of the Web. Each web server in the world can add a new web page to the repository and announce it to all Internet users without overloading a few servers. Linking allows one web page to refer to another web page stored in another server somewhere else in the world. The linking of web pages was achieved using a concept called **hypertext**, which was introduced many years before the advent of the Internet. The idea was to use a machine that automatically retrieved another document stored in the system when a link to it appeared in the document. The Web implemented this idea electronically: to allow the linked document to be retrieved when the link was clicked by the user. Today, the term hypertext, coined to mean linked text documents, has been changed to hypermedia, to show that a web page can be a text document, an image, an audio file, or a video file.

The WWW today is a distributed client–server service, in which a client using a browser can access a service using a server. However, the service provided is distributed over many locations called *sites*. Each site holds one or more documents, referred to as web pages. Each **web page**, however, can contain some links to other web pages in the same or other sites. In other words, a web page can be simple or composite. A simple web page has no links to other web pages; a composite web page has one or more links to other web pages. Each web page is a file with a name and address.

Example 6.1

Assume we need to retrieve a scientific document that contains one reference to another text file and one reference to a large image. Figure 6.13 shows the situation.

Figure 6.13 *Example 6.1*



The main document and the image are stored in two separate files in the same site (file A and file B); the referenced text file is stored in another site (file C). Since we are dealing with three different files, we need three transactions if we want to see the whole document. The first transaction (request/response) retrieves a copy of the main document (file A), which has references (pointers) to the second and third files. When a copy of the main document is retrieved and browsed, the user can click on the reference to the image to invoke the second transaction and retrieve a copy of the image (file B). If the user needs to see the contents of the referenced text file, he or she can click on its reference (pointer) invoking the third transaction and retrieving a copy of file C. Note that although files A and B both are stored in site I, they are independent files with different names and addresses. Two transactions are needed to retrieve them. A very important point we need to remember is that file A, file B, and file C in Example 6.1 are independent web pages, each with independent names and addresses. Although references to file B or C are included in file A, it does not mean that each of these files cannot be retrieved independently. A second user can retrieve file B with one transaction. A third user can retrieve file C with one transaction.

Web client (browser)

A variety of vendors offer commercial **browsers** that interpret and display a web page, and all of them use nearly the same architecture. Each browser usually consists of three parts: a controller, client protocols, and interpreters.

The controller receives input from the keyboard or the mouse and uses the client programs to access the document. After the document has been accessed, the controller uses one of the interpreters to display the document on the screen. The client protocol can be one of the protocols described later, such as HTTP or FTP. The interpreter can be **HyperText Markup Language (HTML)**, Java, or JavaScript, depending on the type of document. Some commercial browsers include Internet Explorer, Netscape Navigator, and Firefox.

Web server

The web page is stored at the server. Each time a request arrives, the corresponding document is sent to the client.

Uniform resource locator (URL)

A web page, as a file, needs to have a unique identifier to distinguish it from other web pages. To define a web page, we need three identifiers: *host*, *port*, and *path*. However, before defining the web page, we need to tell the browser what client–server application we want to use, which is called the *protocol*. This means we need four identifiers to define the web page. The first is the type of vehicle to be used to fetch the web page; the last three make up the combination that defines the destination object (web page).

- **Protocol**. The first identifier is the abbreviation for the client–server program that we need in order to access the web page.
- **Host identifier**. The host identifier can be the IP address of the server or the unique name given to the server.
- **Port number**. The port, a 16-bit integer, is normally predefined for the client–server application.
- **Path**. The path identifies the location and the name of the file in the underlying operating system. The format of this identifier normally depends on the operating system.

In UNIX, a path is a set of directory names followed by the file name, all separated by a slash.

To combine these four pieces together, the **uniform resource locator** (URL) has been designed; it uses three different separators between the four pieces as shown below:

protocol://host/path	Used most of the time
protocol://host:port/path	Used when port number is needed

HyperText Transfer Protocol (HTTP)

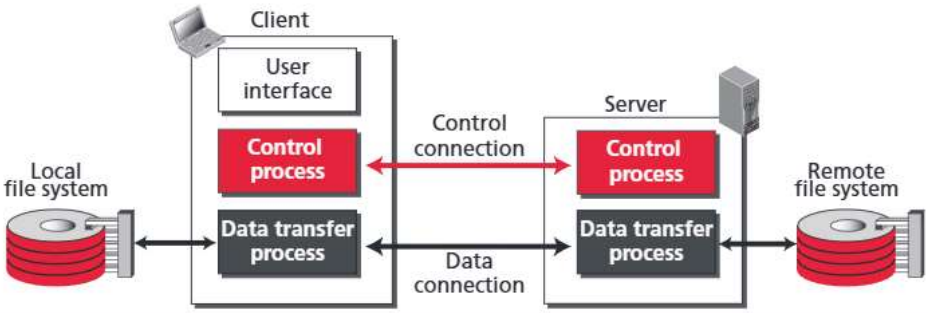
The **HyperText Transfer Protocol** (HTTP) is a protocol that is used to define how the client–server programs can be written to retrieve web pages from the Web. An HTTP client sends a request; an HTTP server returns a response. The server uses the port number 80; the client uses a temporary port number.

6.2.4 File Transfer Protocol (FTP)

File Transfer Protocol (FTP) is the standard protocol provided by TCP/IP for copying a file from one host to another. Although transferring files from one system to another seems simple and straightforward, some problems must be dealt with first. For example, two systems may use different file name conventions. Two systems may have different ways to represent data. Two systems may have different directory structures. All of these problems have been solved by FTP in a very simple and elegant approach.

Figure 6.14 shows the basic model of FTP. The client has three components: user interface, client control process, and the client data transfer process. The server has two components: the server control process and the server data transfer process. The control connection is made between the control processes. The data connection is made between the data transfer processes.

Figure 6.14 *FTP*



Separation of commands and data transfer makes FTP more efficient. The control connection uses very simple rules of communication. We need to transfer only a line of command or a line of response at a time. The data connection, on the other hand, needs more complex rules due to the variety of data types transferred.

Lifetimes of two connections

The two connections in FTP have different lifetimes. The control connection remains connected during the entire interactive FTP session. The data connection is opened and then closed for each file transfer activity. It opens each time commands that involve transferring files are used, and it closes when the file is transferred. In other words, when a user starts an FTP session, the control connection opens. While the control connection is open, the data connection can be opened and closed multiple times if several files are transferred.

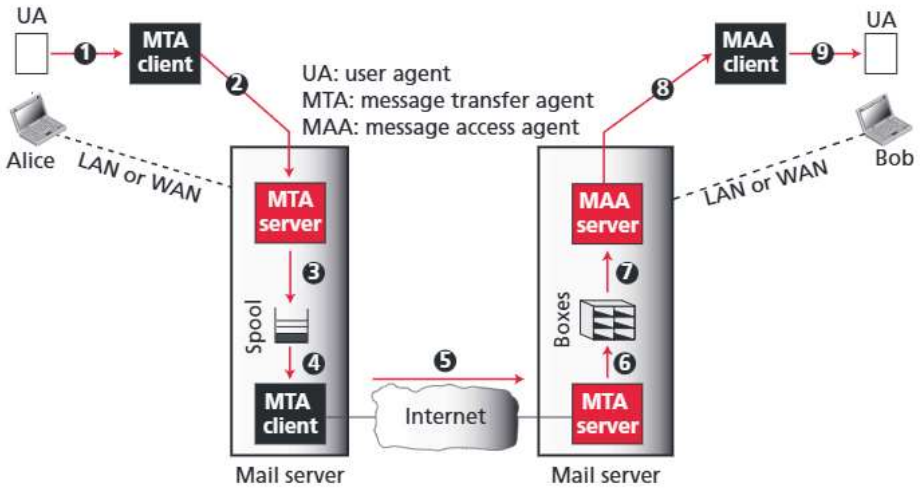
6.2.5 Electronic mail

Electronic mail (or email) allows users to exchange messages. The nature of this application, however, is different from other applications discussed so far. In an application such as HTTP or FTP, the server program is running all the time, waiting for a request from a client. When the request arrives, the server provides the service. There is a request and there is a response. In the case of electronic mail, the situation is different. First, email is considered a one-way transaction. When Alice sends an email to Bob, she may expect a response, but this is not a mandate. Bob may or may not respond. If he does respond, it is another one-way transaction. Second, it is neither feasible nor logical for Bob to run a server program and wait until someone sends an email to him. Bob may turn off his computer when he is not using it. This means that the idea of client/server programming should be implemented in another way: using some intermediate computers (servers). The users run only client programs when they want and the intermediate servers apply the client/server paradigm, as we discuss in the next section.

Architecture

To explain the architecture of email, we give a common scenario, as shown in Figure 6.15.

Figure 6.15 *Common scenario*



5/1/25, 9:57 PM study viewer

In the common scenario, the sender and the receiver of the email, Alice and Bob respectively, are connected via a LAN or a WAN to two mail servers. The administrator has created one mailbox for each user where the received messages are stored. A *mailbox* is part of a server hard drive, a special file with permission restrictions. Only the owner of the mailbox has access to it. The administrator has also created a queue (spool) to store messages waiting to be sent.

A simple email from Alice to Bob takes nine different steps, as shown in the figure. Alice and Bob use three different *agents*: a **user agent** (UA), a **Mail Transfer Agent** (MTA), and a **Message Access Agent** (MAA). When Alice needs to send a message to Bob, she runs a UA program to prepare the message and send it to her mail server. The mail server at her site uses a queue (spool) to store messages waiting to be sent. The message, however, needs to be sent through the Internet from Alice's site to Bob's site using an MTA. Here two **message transfer agents** are needed: one client and one server. Like most client–server programs on the Internet, the server needs to run all the time because it does not know when a client will ask for a connection. The client, on the other hand, can be triggered by the system when there is a message in the queue to be sent. The user agent at Bob's site allows Bob to read the received message. Bob later uses an MAA client to retrieve the message from an MAA server running on the second server.

Bob needs another pair of client–server programs: message access programs. This is because an MTA client–server program is a *push* program: the client pushes the message to the server. Bob needs a *pull* program. The client needs to pull the message from the server. We discuss more about MAAs shortly.

6.2.6 TELNET

A server program can provide a specific service to its corresponding client program. For example, the FTP server is designed to let the FTP client store or retrieve files on the server site. However, it is impossible to have a client/server pair for each type of service we need; the number of servers soon becomes intractable. The idea is not scalable. Another solution is to have a specific client/server program for a set of common scenarios, but to have some generic client/server programs that allow a user on the client site to log into the computer at the server site and use the services available there. For example, if a student needs to use the Java compiler program at her university lab, there is no need for a Java compiler client and a Java compiler server. The student can use a client logging program to log into the university server and use the compiler program at the university. We refer to these generic client/server pairs as **remote login** applications.

One of the original remote logging protocols is TELNET, which is an abbreviation for *TErminaL NETwork*. Although TELNET requires a logging name and password, it is vulnerable to hacking because it sends all data including the password in plaintext (not encrypted). A hacker can eavesdrop and obtain the logging name and password. Because of this security issue, the use of TELNET has diminished in favor of another protocol, Secure Shell (SSH), which we describe in the next section.

6.2.7 Secure Shell (SSH)

Although Secure Shell (SSH) is a secure application program that can be used today for several purposes such as remote logging and file transfer, it was originally designed to replace

https://studylib.net/doc/26059254/foundations-computer-science-4th 171/706

TELNET. There are two versions of SSH: SSH-1 and SSH-2, which are totally incompatible. The first version, SSH-1, is now deprecated because of security flaws in it. The current version is called SSH-2.

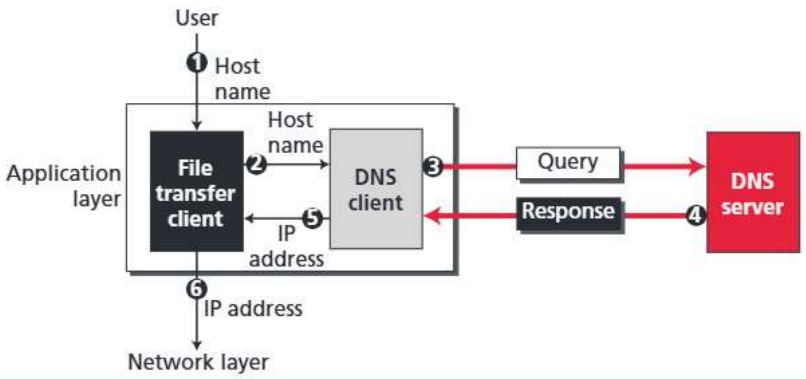
6.2.8 Domain Name System (DNS)

The last client–server application program we discuss has been designed to help other application programs. To identify an entity, TCP/IP protocols use the IP address, which uniquely identifies the connection of a host to the Internet. However, people prefer to use names instead of numeric addresses. Therefore, the Internet needs to have a directory system that can map a name to an address. This is analogous to the telephone network. A telephone network is designed to use telephone numbers, not names. People can either keep a private file to map a name to the corresponding telephone number or can call the telephone directory to do so.

Since the Internet is so huge today, a central directory system cannot hold all the mapping. In addition, if the central computer fails, the whole communication network will collapse. A better solution is to distribute the information among many computers in the world. In this method, the host that needs mapping can contact the closest computer holding the needed information. Figure 6.16 shows how TCP/IP uses a DNS client and a DNS server to map a name to an address. A user wants to use a file transfer client to access the corresponding file transfer server running on a remote host. The user knows only the file transfer server name, such as *afilesource.com*. However, the TCP/IP suite needs the IP address of the file transfer server to make the connection. The following six steps map the host name to an IP address:

1. The user passes the host name to the file transfer client.
2. The file transfer client passes the host name to the DNS client.
3. Each computer, after being booted, knows the address of one DNS server. The DNS client sends a message to a DNS server with a query that gives the file transfer server name using the known IP address of the DNS server.
4. The DNS server responds with the IP address of the desired file transfer server.
5. The DNS client passes the IP address to the file transfer server.
6. The file transfer client now uses the received IP address to access the file transfer server.

Figure 6.16 *DNS usage*



https://sanet.st/blogs/polatebooks/

Name space

To be unambiguous, the names assigned to machines must be carefully selected from a name space with complete control over the binding between the names and IP addresses. In other words, the names must be unique because the addresses are unique. A **name space** can map each address to a unique name and is normally organized hierarchically. In a *hierarchical name space*, each name is made of several parts. The first part can define the nature of the organization, the second part can define the name of an organization, the third part can define departments in the organization, and so on. In this case, the authority to assign and control the name spaces can be decentralized. A central authority can assign the part of the name that defines the nature of the organization and the name of the organization. The responsibility for the rest of the name can be given to the organization itself. The organization can add suffixes (or prefixes) to the name to define its host or resources. The management of the organization need not worry that the prefix chosen for a host is taken by another organization because, even if part of an address is the same, the whole address is different. For example, assume two organizations call one of their computers *caesar*. The first organization is given a name by the central authority, such as *first.com*, the second organization is given the name *second.com*. When each of these organizations adds the name *caesar* to the name they have already been given, the end result is two distinguishable names: *ceasar.first.com* and *ceasar.second.com*. The names are unique.

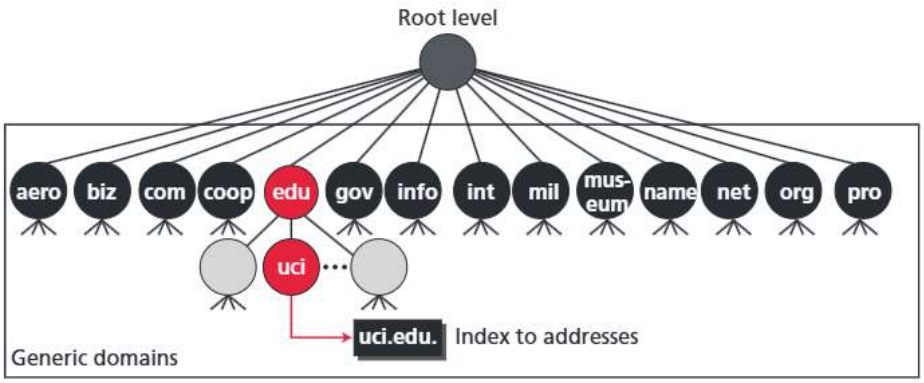
DNS in the Internet

DNS is a protocol that can be used in different platforms. In the Internet, the **domain name space** (tree) was originally divided into three different sections: generic domains, country domains, and the inverse domain. However, due to the rapid growth of the Internet, it became extremely difficult to keep track of the inverse domains, which could be used to find the name of a host when given the IP address. The inverse domains are now deprecated (see RFC 3425). We, therefore, concentrate on the first two.

Generic domains

The **generic domains** define registered hosts according to their generic behavior. Each node in the tree defines a domain, which is an index to the **domain name** space database (see Figure 6.17).

Figure 6.17 *Generic domains*



Looking at the tree, we see that the first level in the generic domains section allows 14 possible labels. These labels describe the organization types as listed in Table 6.1.

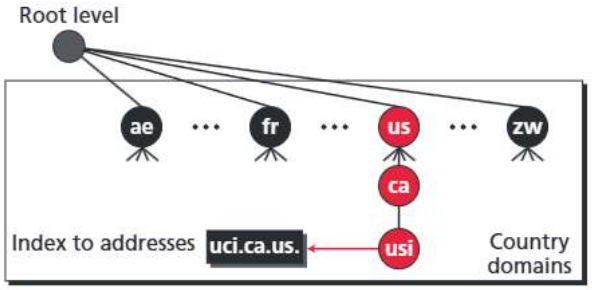
Table 6.1 Generic domain labels

Label	*Description*	*Label*	*Description*
aero	Airlines and aerospace	int	International organizations
biz	Businesses or firms	mil	Military groups
com	Commercial organizations	museum	Museums
coop	Cooperative organizations	name	Personal names (individuals)
edu	Educational institutions	net	Network support centers
gov	Government institutions	org	Nonprofit organizations
info	Information service providers	pro	Professional organizations

Country domains

The **country domains** section uses two-character country abbreviations (e.g., us for United States). Second labels can be organizational, or they can be more specific, national designations. The United States, for example, uses state abbreviations as a subdivision of us (e.g., ca.us.). Figure 6.18 shows the country domains section. The address uci.ca.us. can be translated to University of California, Irvine university in the state of California in the United States.

Figure 6.18 *Country domains*



6.2.9 Peer-to-peer paradigm

We discussed the client–server paradigm early in the chapter. The first instance of peer-to-peer file sharing goes back to December 1987 when Wayne Bell created *WWIVnet*, the network component of WWIV (World War Four) bulletin board software. In July 1999, Ian Clarke designed *Freenet*, a decentralized, censorship-resistant distributed data store, aimed to provide freedom of speech through a peer-to-peer network with strong protection of anonymity.

Peer-to-peer gained popularity with Napster (1999–2001), an online music file sharing service created by Shawn Fanning. Although free copying and distributing of music files by

the users led to a copyright violation lawsuit against Napster, and eventually closing of the service, it paved the way for peer-to-peer file-distribution models that came later. Gnutella had its first release in March 2000. It was followed by FastTrack (used by the Kazaa), BitTorrent, WinMX, and GNUnet in March, April, May, and November of 2001 respectively.

Internet users that are ready to share their resources become peers and form a network. When a peer in the network has a file (for example, an audio or video file) to share, it makes it available to the rest of the peers. An interested peer can connect itself to the computer where the file is stored and download it. After a peer downloads a file, it can make it available for other peers to download. As more peers join and download that file, more copies of the file become available to the group. Since lists of peers may grow and shrink, the question is how the paradigm keeps track of loyal peers and the location of the files. To answer this question, we first need to divide the P2P networks into two categories: centralized and decentralized.

Centralized networks

In a centralized P2P network, the directory system listing of the peers and what they offer uses the client–server paradigm, but the storing and downloading of the files are done using the peer-to-peer paradigm. For this reason, a centralized P2P network is sometimes referred to as a hybrid P2P network. Napster was an example of a centralized P2P. In this type of network, a peer first registers itself with a central server. The peer then provides its IP address and a list of files it has to share. To avoid system collapse, Napster used several servers for this purpose, but we show only one in Figure 6.18.

A peer, looking for a particular file, sends a query to a central server. The server searches its directory and responds with the IP addresses of nodes that have a copy of the file. The peer contacts one of the nodes and downloads the file. The directory is constantly updated as nodes join or leave the peer.

Centralized networks make the maintenance of the directory simple but have several drawbacks. Accessing the directory can generate huge traffic and slow down the system. The central servers are vulnerable to attack, and if all of them fail, the whole system goes down. The central component of the system was ultimately responsible for Napster's losing the copyright lawsuit and its closure in July 2001. Roxio brought back the New Napster in 2003; Napster version 2 is now a legal, pay-for-music site.

Decentralized network

A decentralized P2P network does not depend on a centralized directory system. In this model, peers arrange themselves into an *overlay network*, which is a logical network made on top of the physical network. Depending on how the nodes in the overlay network are linked, a decentralized P2P network is classified as either unstructured or structured.

In an unstructured P2P network, the nodes are linked randomly. A search in an unstructured P2P is not very efficient because a query to find a file must be flooded through the network, which produces significant traffic and still the query may not be resolved. Two examples of this type of network are Gnutella and Freenet. We next discuss the Gnutella network as an example.

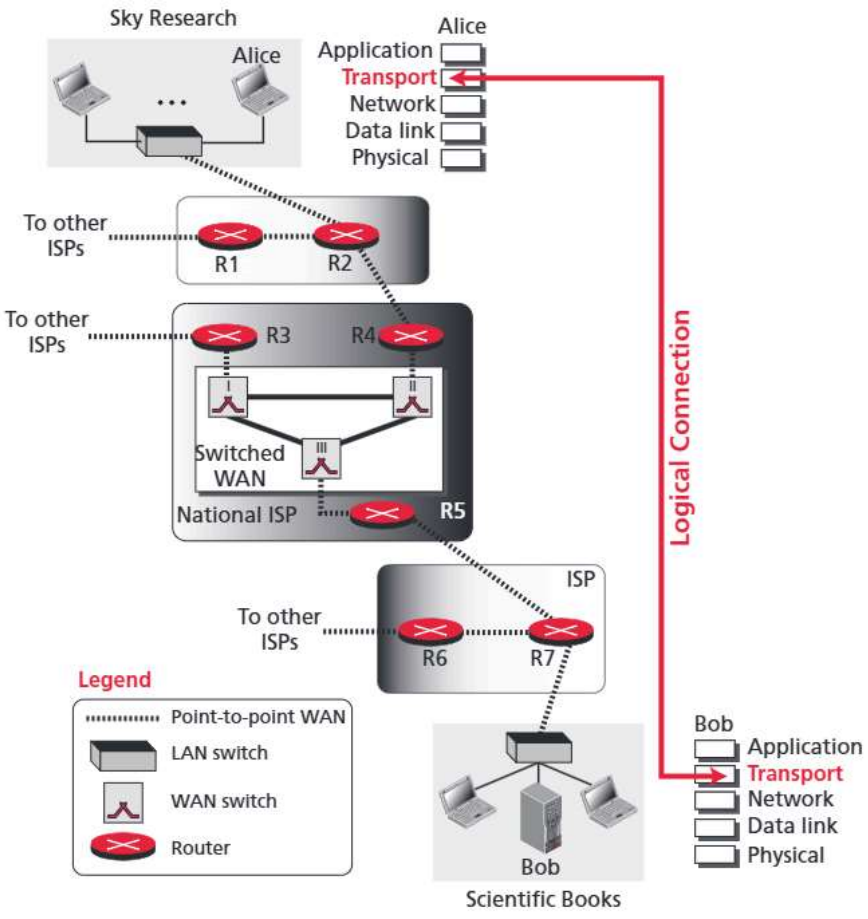
A structured network uses a predefined set of rules to link nodes so that a query can be effectively and efficiently resolved. The most common technique used for this purpose is the *Distributed Hash Table (DHT)*. DHT is used in many applications including Distributed Data Structure (DDS), Content Distributed Systems (CDS), Domain Name

System (DNS), and P2P file sharing. One popular P2P file sharing protocol that uses the DHT is BitTorrent. We discuss DHT independently in the next section as a technique that can be used both in structured P2P networks and in other systems.

6.3 TRANSPORT LAYER

The **transport layer** in the TCP/IP suite is located between the application layer and the network layer. It provides services to the application layer and receives services from the network layer. The transport layer acts as a liaison between a client program and a server program, a process-to-process connection. The transport layer is the heart of the TCP/IP protocol suite; it is the end-to-end logical vehicle for transferring data from one point to another in the Internet. Figure 6.19 shows the idea behind this logical connection.

Figure 6.19 *Logical connection at the transport layer*



The figure shows the same scenario we used for the application layer. Alice's host in the Sky Research company creates a logical connection with Bob's host in the Scientific Books company at the transport layer. The two companies communicate at the transport layer as though there is a real connection between them. Figure 6.19 shows that only the two end systems (Alice's and Bob's computers) use the service of the transport layer; all intermediate routers use only the first three layers.

6.3.1 Transport-layer services

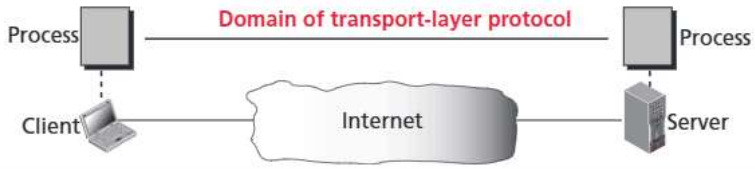
In this section, we discuss the services that can be provided by the transport layer; in the next section, we discuss several transport-layer protocols.

Process-to-process communication

The first duty of a transport-layer protocol is to provide **process-to-process communication.** A process is an application-layer entity (running program) that uses the services of the transport layer.

The network layer (discussed later) is responsible for communication at the computer level (host-to-host communication). A network-layer protocol can deliver the message only to the destination computer. However, this is an incomplete delivery. The message still needs to be handed to the correct process. This is where a transport-layer protocol takes over. A transport-layer protocol is responsible for delivery of the message to the appropriate process. Figure 6.20 shows the domains of a network layer and a transport layer.

Figure 6.20 *Network layer versus transport layer*

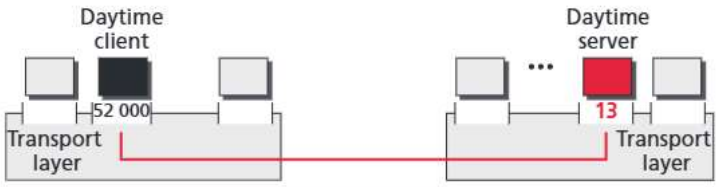


Addressing: port numbers

Although there are a few ways to achieve process-to-process communication, the most common is through the client–server paradigm (discussed before). A process on the local host, called a *client*, needs services from a process usually on the remote host, called a *server*. Both processes (client and server) have the same name. For example, to get the day and time from a remote machine, we need a daytime client process running on the local host and a daytime server process running on a remote machine. A remote computer can run several server programs at the same time, just as several local computers can run one or more client programs at the same time. For communication, we must define the local host, local process, remote host, and remote process. The local host and the remote host are defined using IP addresses (discussed in the next section). To define the processes, we need second identifiers, called **port numbers**. In the TCP/IP protocol suite, the port numbers are integers between 0 and 65 535 (16 bits).

The client program defines itself with a port number, called the **ephemeral port number**. The word ephemeral means *short-lived* and is used because the life of a client is normally short. An ephemeral port number is recommended to be greater than 1023 for some client/server programs to work properly. The server process must also define itself with a port number. This port number, however, cannot be chosen randomly. TCP/IP has decided to use universal port numbers for servers; these are called **well-known port numbers**. Every client process knows the well-known port number of the corresponding server process. For example, while the daytime client process, discussed above, can use an ephemeral (temporary) port number, 52 000, to identify itself, the daytime server process must use the well-known (permanent) port number 13. Figure 6.21 shows this concept.

Figure 6.21 *Port numbers*



6.3.2 Transport-layer protocols

Although the Internet uses several transport-layer protocols, we discuss only two in this section: UDP and TCP.

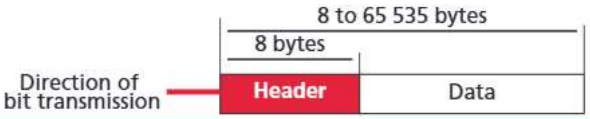
User Datagram Protocol (UDP)

The **User Datagram Protocol (UDP)** is a connectionless, unreliable transport protocol. It does not add anything to the services of network layer except for providing process-to-process communication instead of host-to-host communication. If UDP is so powerless, why would a process want to use it? With the disadvantages come some advantages. UDP is a very simple protocol using a minimum of overhead. If a process wants to send a small message and does not care much about reliability, it can use UDP. Sending a small message using UDP takes much less interaction between the sender and receiver than using TCP.

User datagrams

UDP packets, called **user datagrams**, have a fixed-size **header** of 8 bytes. Figure 6.22 shows the format of a user datagram. However, the total length needs to be less because a UDP user datagram is stored in an **IP datagram** with the total length of 65 535 bytes.

Figure 6.22 *User datagram packet format*



Transmission Control Protocol (TCP)

Transmission Control Protocol (TCP) is a connection-oriented, reliable protocol. TCP explicitly defines connection establishment, data transfer, and connection teardown phases to provide a connection-oriented service. Connection-oriented service here means that there is a connection (relationship) between all packets (segments) belonging to the same message (coming from the application layer). TCP uses sequence numbers to define the order of the segments. The sequence number is related to the number of bytes in each segment. For example, if the message is 6000 bytes, the first segment has a sequence number 0, the second has the sequence number 2000, and the third segment has the sequence number 4000 (the process is more complicated, we try to simplify it). In this way, if a segment is lost, the receiver holds the other two until the lost one is reset by the sender.

Segments

At the transport layer, TCP groups a number of bytes together into a packet called a **segment**. TCP adds a header to each segment (for control purposes) and delivers the segment to the network layer for transmission. The segments are encapsulated in an IP datagram and transmitted as shown in Figure 6.23.

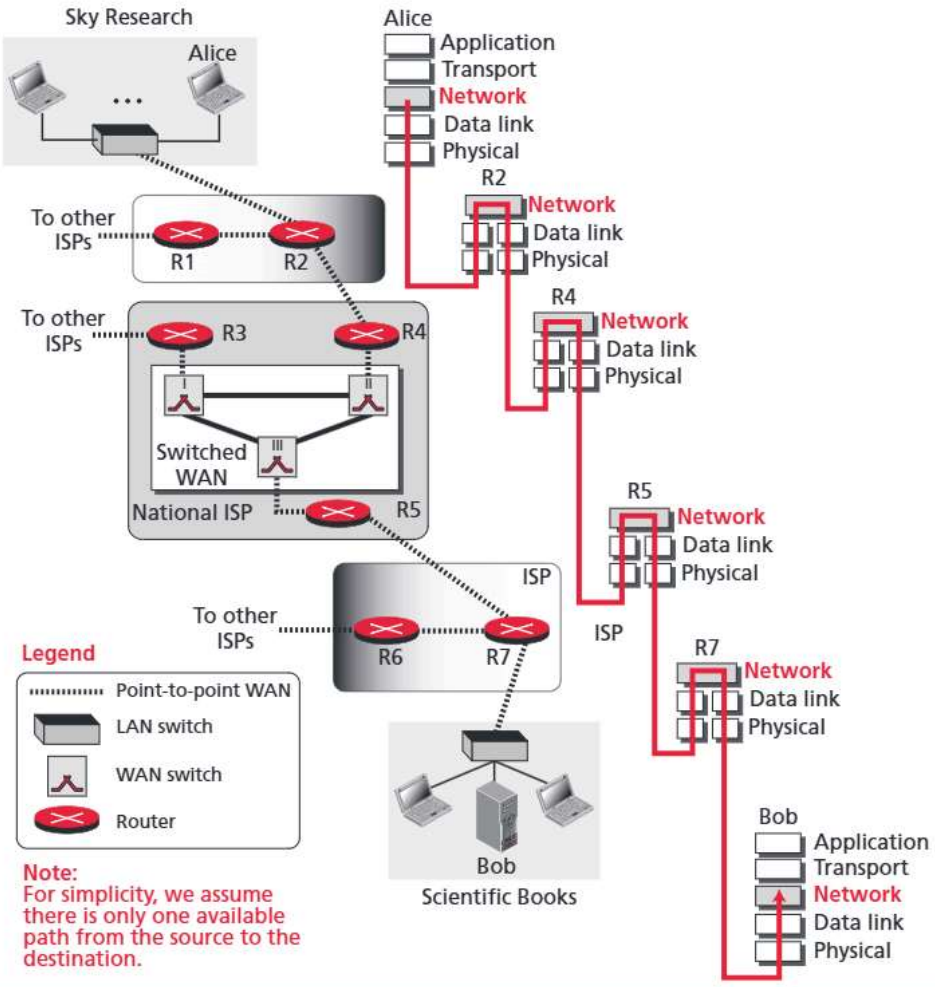
Figure 6.23 *TCP segments*

6.4 NETWORK LAYER

The network layer in the TCP/IP protocol suite is responsible for the host-to-host delivery of messages. Figure 6.24 shows the communication between Alice and Bob at the network layer assuming that there is only one path from Alice's computer to Bob's computer. This is the same scenario we used in the last two sections to show the communication at the application and the transport layers, respectively.

As the figure shows, the network layer is involved at the source host, destination host, and all routers in the path (R2, R4, R5, and R7). At the source host (Alice), the network layer accepts a packet from a transport layer, encapsulates the packet in a **datagram**, and delivers the packet to the data-link layer. At the destination host (Bob), the datagram is decapsulated, the packet is extracted and delivered to the corresponding transport layer. Although the source and destination hosts are involved in all five layers of the TCP/IP suite, the routers use three layers if they are routing packets only; however, they may need the transport and application layers for control purposes. A router in the path is normally shown with two data-link layers and two physical layers, because it receives a packet from one network and delivers it to another network.

Figure 6.24 *Communication at the network layer*



6.4.1 Services Provided by network layer

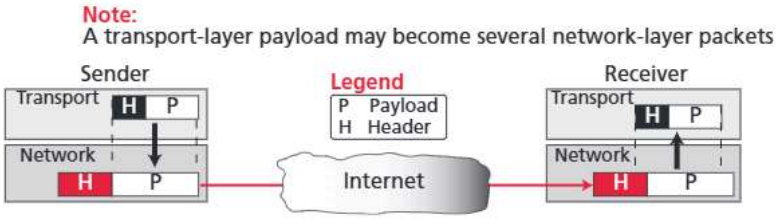
The network layer is located under the transport layer; this means that the network layer provides service to the transport layer. We discuss some aspects of this service below.

Packetizing

The first duty of the network layer is definitely **packetizing**: encapsulating the payload (data received from the upper layer) in a network-layer packet at the source and decapsulating the payload from the network-layer packet at the destination. In other words, one duty of the network layer is to carry a payload from the source to the destination without changing it or using it. The network layer is doing the service of a carrier such as the post

office, which is responsible for delivery of packages from a sender to a receiver without changing or using the contents. This is done in three steps as shown in Figure 6.25.

Figure 6.25 *Packetizing at the network layer*



1. The source network-layer protocol receives a packet from the transport-layer protocol, adds a header that contains the source and destination addresses and some other information that is required by the network-layer protocol.
2. The network layer protocol then logically delivers the packet to the network-layer protocol at the destination.
3. The destination host receives the network-layer packet, decapsulates the payload and delivers to the upper-layer protocol.

If the packet is fragmented at the source or at routers along the path, the network layer is responsible for waiting until all fragments arrive, reassembling them, and delivering them to the upper-layer protocol.

A transport-layer payload may be encapsulated in several network-layer packets.

Packet delivery

Packet delivery at the network layer is unreliable and connectionless. We briefly discuss these two concepts next.

Unreliable delivery

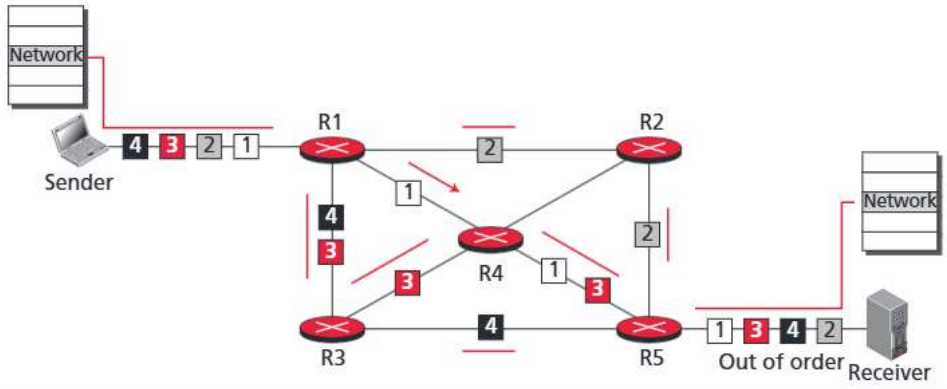
The delivery of packets at the network layer is *unreliable*. This means that the packets can be corrupted, lost, duplicated. In other words, the network layer provides a best-effort delivery, but there is no guarantee that a packet will reach the destination as we expect. This is the same service we receive from the post office when we mail a regular letter. The reason in both cases is the cost. If we need a guarantee from the post office, the cost is higher (registered mail for example). If we need a guarantee from the network layer, the delivery of packets will be delayed. Each packet needs to be checked at each router and destination and resent if corrupted. Checking the lost packets is even more costly. Does it mean that messages we send by the Internet are not reliable? The answer is that if we want to guarantee that the messages are not corrupted, we need to use the TCP protocol at the transport layer. If a payload at the transport layer is corrupted (because of unreliable

delivery at the data-link layer), TCP drops the packet and requests resending of the data as we discussed in the previous section.

Connectionless delivery

The delivery at the network layer is also connectionless, but the word *connectionless* here does not mean that there is no physical connection between the sender and receiver. It means that the network layer treats each packet independently (like the way the post office does with the letters). In other words, there is no relationship between packets belonging to the same transport-layer payload. If a transport-layer packet results in four network-layer packets, there is no guarantee that the packets arrive in the same order as sent because each packet may follow a different path to reach the destination. Figure 6.26 shows the reason for this problem.

Figure 6.26 *Packets travelling different paths*



A transport-layer packet is divided into four network-layer packets. They are sent in order (1, 2, 3, 4), but they are received out of order (2, 4, 3, 1). The transport layer at the destination is responsible for holding packets until all of them arrive before putting them in order and delivering them to the application layer.

Routing

Another duty of the network layer, which is as important as the others, is routing. The network layer is responsible for routing the packet from its source to the destination. A physical network is a combination of networks (LANs and WANs) and routers that connect them. This means that there is more than one route from the source to the destination. The network layer is responsible for finding the *best* one among these possible routes. The network layer needs to have some specific strategies for defining the best route. In the Internet today, this is done by running some *routing protocols* to help the routers coordinate their knowledge about the neighborhood and to come up with consistent tables to be used when a packet arrives.

6.4.2 Network-layer protocols

Although there are several protocols at the network layer, the main protocol is called the **Internet Protocol (IP)**. Other protocols are auxiliary protocols that help IP. Today, two versions of IP protocol are in use: IPv4 and IPv6. We discuss each in the next two sections.

Internet Protocol Version 4 (IPv4)

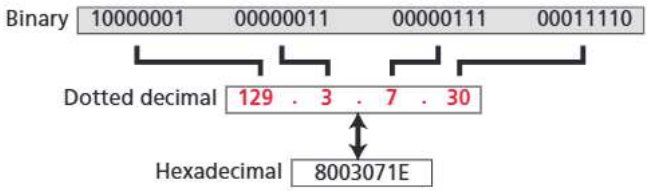
Today most systems are using the Internet Protocol Version 4 (IPv4), but this will be changed in future because of smaller address space and packet format of this protocol (among other reasons).

IPv4 addressing

The identifier used in the IPv4 layer of the TCP/IP protocol suite to identify the connection of each device to the Internet is called the **Internet address** or IP address. An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet. The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed. IPv4 addresses are unique in the sense that each address defines one, and only one, connection to the Internet. If a device, such as a router, has several connections to the Internet, via several networks, it has several IPv4 addresses. IPv4 addresses are universal in the sense that the addressing system must be accepted by any host that wants to be connected to the Internet.

There are three common notations to show an IPv4 address: binary notation (base 2), **dotted-decimal notation** (base 256), and hexadecimal notation (base 16). In *binary notation*, an IPv4 address is displayed as 32 bits. To make the address more readable, one or more spaces are usually inserted between each octet (8 bits). Each octet is often referred to as a byte. To make the IPv4 address more compact and easier to read, it is usually written in decimal form with a decimal point (dot) separating the bytes. This format is referred to as *dotted-decimal notation*. Note that because each byte (octet) is only 8 bits, each number in the dotted-decimal notation is between 0 and 255. We sometimes see an IPv4 address in hexadecimal notation. Each hexadecimal digit is equivalent to four bits. This means that a 32-bit address has 8 hexadecimal digits. This notation is often used in network programming. Figure 6.27 shows an IP address in the three discussed notations.

Figure 6.27 *Address notation*

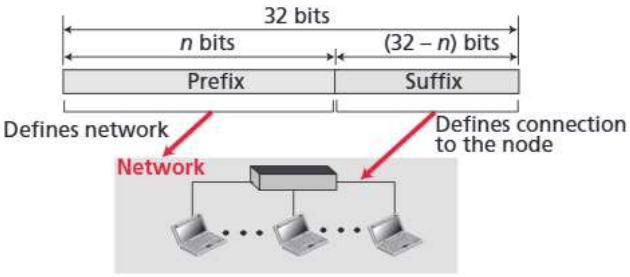


In any communication network that involves delivery, such as a telephone network or a postal network, the addressing system is hierarchical. In a postal network, the postal

address (mailing address) includes the country, state, city, street, house number, and the name of the mail recipient. Similarly, a telephone number is divided into the country code, area code, local exchange, and the connection.

A 32-bit IPv4 address is also hierarchical, but divided only into two parts. The first part of the address, called the *prefix*, defines the network; the second part of the address, called the *suffix*, defines the node (connection of a device to the Internet). Figure 6.28 shows the prefix and suffix of a 32-bit IPv4 address. The prefix length is *n* bits and the suffix length is (32 – *n*) bits. The prefix and suffix lengths depend on the site of the network (organization).

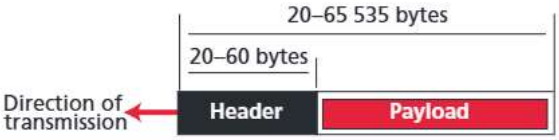
Figure 6.28 *Hierarchy in IPv4 addressing*



IPv4 datagram

Packets used by the IP are called *datagrams*. Figure 6.29 shows the IPv4 datagram format. A datagram is a variable-length packet consisting of two parts: header and payload (data). The header is 20 to 60 bytes in length and contains information essential to routing and delivery. Note that a byte is 8 bits.

Figure 6.29 *IPv4 datagram*



Internet Protocol Version 6 (IPv6)

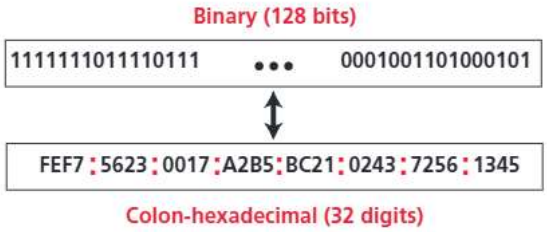
Some shortcomings of IPv4 such as address depletion prompted a new version of IP protocol in the early 1990s. The new version, which is called **Internet Protocol version 6 (IPv6)** or **IP new generation (IPng)** was a proposal to augment the address space of IPv4 and at the same time redesign the format of the IP packet and revise some auxiliary protocols. It is

interesting to know that IPv5 was a proposal that never materialized. The following shows the main changes in the IPv6 protocol.

IPv6 addressing

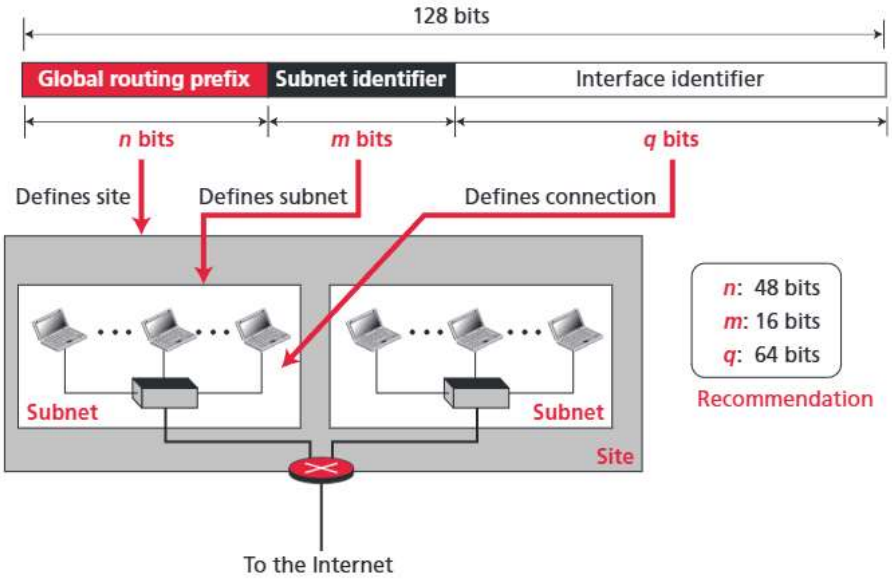
To prevent the address depletion, IPv6 uses 128 bits to define any device connected to the Internet. An address is represented as either binary or colon-hexadecimal form. The first form is used to store an address in the computer; the second form is used by humans. Figure 6.30 shows the two formats.

Figure 6.30 *IPv6 address notation*



The address in IPv6 actually defines three levels of hierarchy: site (organization), subnetwork, and connection to the host as shown in Figure 6.31.

Figure 6.31 *Hierarchy in IPv6 addressing*



IPv6 datagram

Figure 6.32 shows the IPv6 datagram format. A datagram in this version is also a variable-length packet consisting of two parts: header and payload (data). The header is 40 bytes. However, some extension headers are considered part of the payload in this version.

Figure 6.32 *IPv6 datagram*

6.5 DATA-LINK LAYER

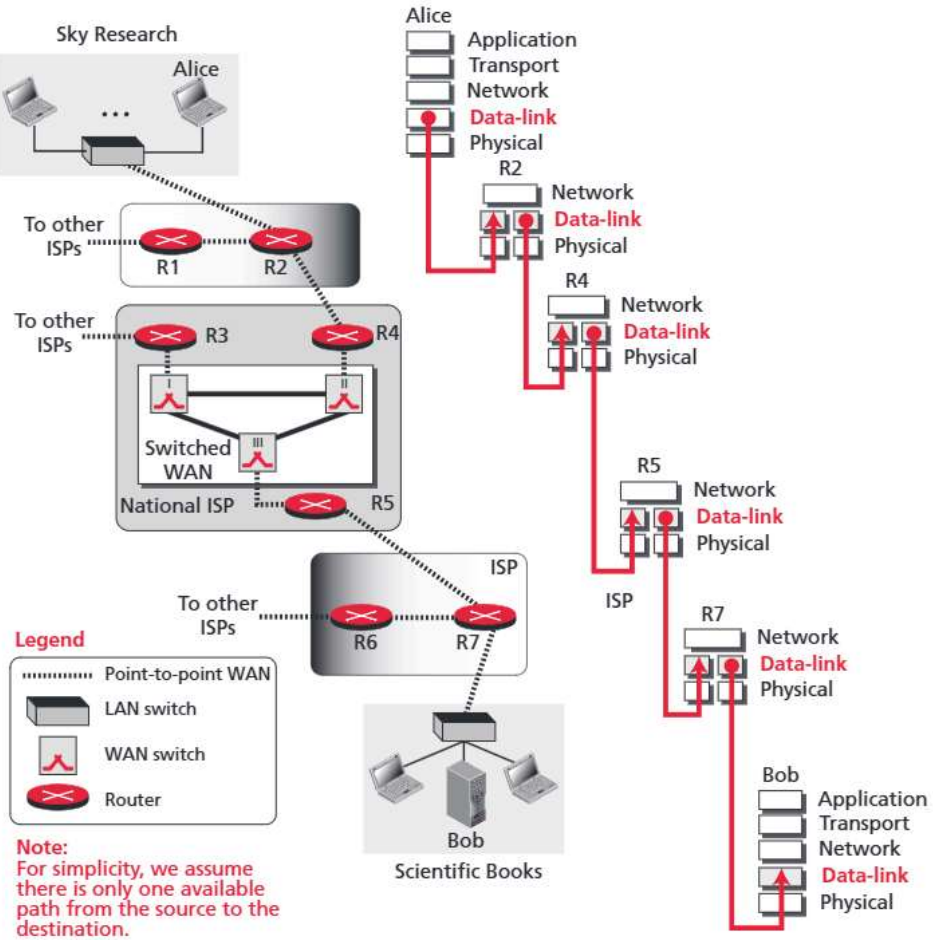
The TCP/IP suite does not define any protocol in the **data-link layer**. This layer is the territory of networks that when connected make up the Internet. These networks, wired or wireless, receive services and provide services to the network layer. This may give us a clue that there are several standard protocols in the market today.

In the previous section, we learned that communication at the network layer is host-to-host. The Internet, however, is a combination of networks glued together by connecting devices (routers or switches). If a datagram is to travel from a host to another host, it needs to pass through these networks.

Figure 6.33 shows communication between Alice and Bob, using the same scenario we followed in the last three sections. Communication at the data-link layer, however, is made up of five separate logical connections between the data-link layers in the path.

Only one data-link layer is involved at the source or the destination, but two data-link layers are involved at each router. The reason is that Alice's and Bob's computers are each connected to a single network; each router, however, takes input from one network and sends output to another network.

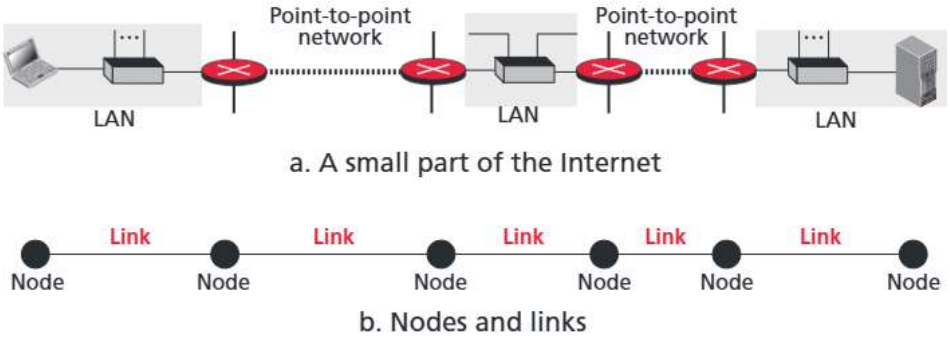
Figure 6.33 *Communication at the data-link layer*



6.5.1 Nodes and links

Although communication at the application, transport, and network layers is end-to-end, communication at the data-link layer is **node-to-node**. Data units from one point in the Internet need to pass through many networks (LANs and WANs) to reach another point. These LANs and WANs are connected by routers. It is customary to refer to the two end hosts and the routers as **nodes** and the networks in between as **links**. Figure 6.34 is a simple representation of links and nodes when the path of the data unit is only six nodes.

Figure 6.34 *Nodes and links*



The link that connects the nodes are either local area networks (LANs) or Wide Area Networks (WANs).

6.5.2 Local area networks (LANs)

In the beginning of this chapter, we learned that a local area network (LAN) is a computer network that is designed for a limited geographic area such as a building or a campus. Although a LAN can be used as an isolated network to connect computers in an organization for the sole purpose of sharing resources, most LANs today are also linked to a wide area network (WAN) or the Internet.

LANs can be wired or wireless networks. In the first group, the stations in the LANs are connected by wire; in the second group the stations are logically connected by air. We discuss each group separately.

Wired LANS: Ethernet

Although several wired LANs were invented in the past, only one has survived: the Ethernet. Maybe the reason is that Ethernet was upgraded several times according to the needs of the Internet community.

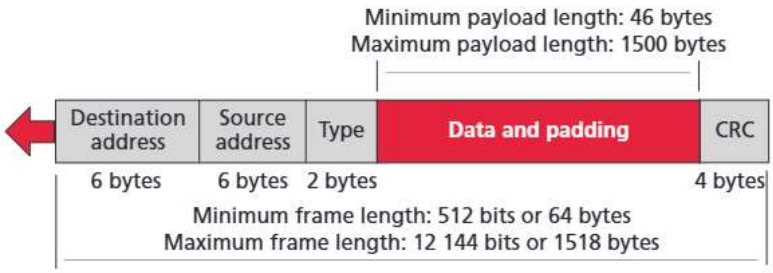
The Ethernet LAN was developed in the 1970s by Robert Metcalfe and David Boggs. Since then, it has gone through four generations: **Standard Ethernet** (10 Mbps), **Fast Ethernet** (100 Mbps), **Gigabit Ethernet** (1 Gbps), and **10 Gigabit Ethernet** (10 Gbps). The data rate, the speed in which bits are sent in each second, has been increased ten times in each generation.

Standard Ethernet

We refer to the original Ethernet technology with the data rate of 10 Mbps (ten million bits per second) as the Standard Ethernet. The data rate in this case defines the speed in which data can be sent out of the station to the LAN. In the case of the Ethernet, the speed is 10 million bits per second. The bits, however, are not sent one by one, a group of bits are packaged together and are referred to as a **frame**. A frame does not carry only data from the sender to the destination. It also carries some information such as the source

address (48 bits), the destination address (48 bits), the type of data, the actual data, and some other control bits as a guard to help checking the integrity of data during transition. If we can think of a frame as an envelope carrying a letter from the sender to the receiver, the data is inside the envelope, but there is other information, such as addresses on the envelope. In the case of the LAN all are encapsulated in a frame. Figure 6.35 shows an Ethernet LAN and the frame format.

Figure 6.35 *Ethernet frame*



Fast Ethernet (100 Mbps)

In the 1990s, Ethernet made a big jump by increasing the **transmission rate** to 100 Mbps, and the new generation was called the Fast Ethernet. The designers of the Fast Ethernet needed to make it compatible with the Standard Ethernet. Most of the protocol such as addressing, frame format remained unchanged. By increasing the transmission rate, features of the Standard Ethernet that depend on the transmission rate had to be revised.

Gigabit Ethernet

The need for an even higher data rate resulted in the design of the **Gigabit Ethernet** Protocol (1000 Mbps). The goals of the Gigabit Ethernet were to upgrade the data rate to 1 Gbps, but keep the address length, the frame format, and the maximum and minimum frame length the same.

10-Gigabit Ethernet

In recent years, there has been another look into the Ethernet for use in metropolitan areas. The idea is to extend the technology, the data rate, and the coverage distance so that the Ethernet can be used as LAN and MAN (**metropolitan area network**). The goals of the 10-Gigabit Ethernet design can be summarized as upgrading the data rate to 10 Gbps, keeping the same frame size and format, and allowing the interconnection of LANs, MANs, and WAN possible. This data rate is possible only with fiber-optic technology at this time.

Wireless LANs

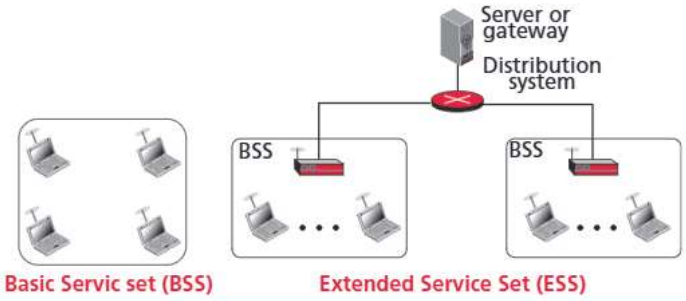
Wireless communication is one of the fastest-growing technologies. The demand for connecting devices without the use of cables is increasing everywhere. Wireless LANs can be found on college campuses, in office buildings, and in many public areas. The first difference

we can see between a wired and a wireless LAN is the medium. In a wired LAN, we use wires to connect hosts. In a wireless LAN, the medium is air, the signal is generally broadcast. When hosts in a wireless LAN communicate with each other, they are sharing the same medium (multiple access). Two technologies have been instrumental in this area: Wireless Ethernet and Bluetooth.

Wireless Ethernet (WiFi)

Institute of Electrical and Electronics Engineers (IEEE) has defined the specifications for a wireless LAN, which sometimes is referred to as wireless Ethernet or *WiFi* (short for wireless fidelity). WiFi, however, is a wireless LAN that is certified by the WiFi Alliance, a global, nonprofit industry association of more than 300 member companies. The standard defines two kinds of services: the basic service set (BSS) and the extended service set (ESS). The second service uses an extra device (access point or AP) that serves as a switch for connection to other LANs or WANs. Figure 6.36 shows both services.

Figure 6.36 *BSSs and ESSs*



Bluetooth

Bluetooth is a wireless LAN technology designed to connect devices of different functions such as telephones, notebooks, computers (desktop and laptop), cameras, printers, and even coffee makers when they are at a short distance from each other. A Bluetooth LAN is an *ad hoc* network, which means that the network is formed spontaneously; the devices, sometimes called gadgets, find each other and make a network called a piconet. A Bluetooth LAN can even be connected to the Internet if one of the gadgets has this capability. A Bluetooth LAN, by nature, cannot be large. If there are many gadgets that try to connect, there is chaos.

Bluetooth technology has several applications. Peripheral devices such as a wireless mouse or keyboard can communicate with the computer through this technology. Monitoring devices can communicate with sensor devices in a small health care center. Home security devices can use this technology to connect different sensors to the main security controller. Conference attendees can synchronize their laptop computers at a conference.

Bluetooth was originally started as a project by the Ericsson Company. It is named after Harald Blaatand, the King of Denmark (940–981) who united Denmark and Norway. *Blaatand* translates to *Bluetooth* in English.

6.5.3 Wide area networks (WANs)

As we discussed before, the networks connecting two nodes in the Internet can be a LAN or a WAN. As in the case of the LANs, WANs can be wired or wireless. We briefly discuss each separately.

Wired WANs

We have a variety of wired WANs in today's Ethernet. Some are point-to-point and some are switched WANs.

Point-to-point wireless WANs

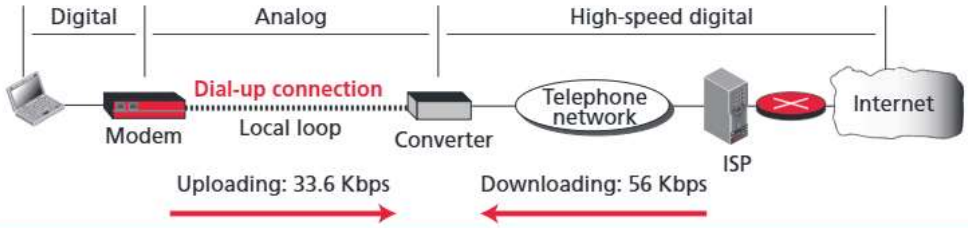
Today we can use several point-to-point wireless networks to provide what is called last-mile service to connect residents and businesses to the Internet.

Dial-up service

A dial-up network or connection uses the services provided by the telephone networks to transmit data. The telephone network had its beginnings in the late 1800s. The entire network was originally a system to transmit voice. With the advent of the computer era, the network, in the 1980s, began to carry data in addition to voice. During the last decade, the telephone network has undergone many technical changes. The need to communicate digital data resulted in the invention of the dial-up modem.

The term **modem** is a composite word that refers to the two functional entities that make up the device: a signal *mo*dulator and a signal *dem*odulator. A **modulator** creates signal from data. A **demodulator** recovers the data from the modulated signal. Figure 6.37 shows the idea behind a modem.

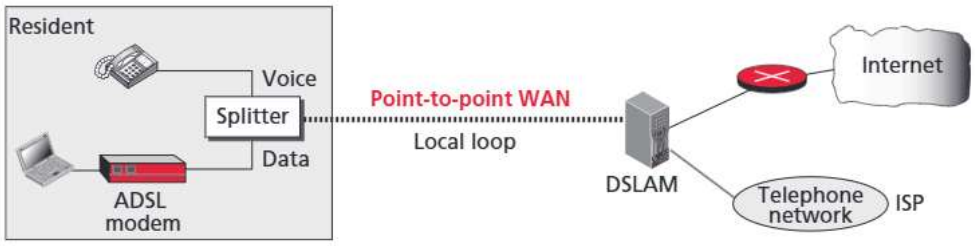
Figure 6.37 *Dial-up network to provide Internet access*



Digital subscriber line (DSL)

After traditional modems reached their peak data rate, telephone companies developed another technology, DSL, to provide higher-speed access to the Internet. **Digital subscriber line (DSL)** technology is one of the most promising for supporting high-speed communication over the existing telephone. DSL technology is a set of technologies, each differing in the first letter (ADSL, VDSL, HDSL, and SDSL). The set is often referred to as *x*DSL, where *x* can be replaced by A, V, H, or S. We just discuss the first, ADSL. The first technology in the set is *asymmetric DSL (ADSL)*. ADSL provides higher speed (**bit rate**) in the downstream direction (from the Internet to the resident) than in the upstream direction (from the resident to the Internet). That is the reason it is called asymmetric (see Figure 6.38).

Figure 6.38 *ASDL point-to-point network*



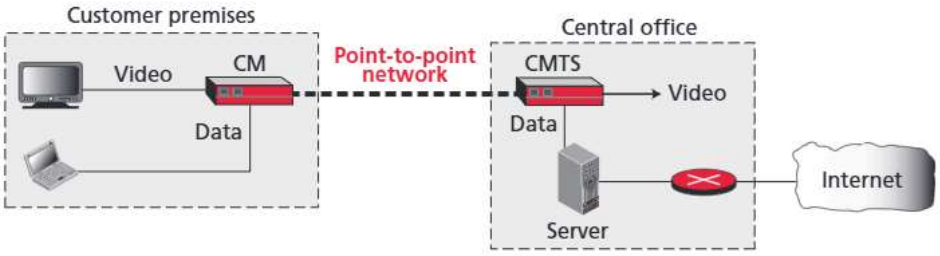
ADSL allows the subscriber to use the voice channel and the data channel at the same time. The rate for the upstream can reach 1.44 Mbps. However, the data rate is normally below 500 kbps because of the high-level noise in this channel. The downstream data rate can reach 13.4 Mbps. However, the data rate is normally below 8 Mbps because of noise in this channel. A very interesting point is that the telephone company in this case serves as the ISP, so services such as email or Internet access are provided by the telephone company itself.

Cable network

Cable networks were originally created to provide access to TV programs for those subscribers who had no reception because of natural obstructions such as mountains. Later the cable networks became popular with people who just wanted a better signal. In addition, cable networks enabled access to remote broadcasting stations via microwave connections. Cable TV also found a good market in Internet access provision, using some of the channels originally designed for video.

Cable companies are now competing with telephone companies for the residential customer who wants high-speed data transfer. DSL technology provides high-data-rate connections for residential subscribers over the local loop. However, DSL uses the existing unshielded twisted-pair cable, which is very susceptible to interference. This imposes an upper limit on the data rate. A solution is the use of the cable TV network. In this section, we briefly discuss this technology. Figure 6.39 shows an example of this service.

Figure 6.39 *Cable service*



Switched wired WANs

It is obvious that the Internet today cannot be operative with only point-to-point wired WANs that provide the last-mile connection. We need switched wired WANs to connect the backbone of the Internet. Several protocols in the past have been designed for this purpose such as SONET or ATM. However, these are complex networks, discussion of which is beyond the scope of this book.

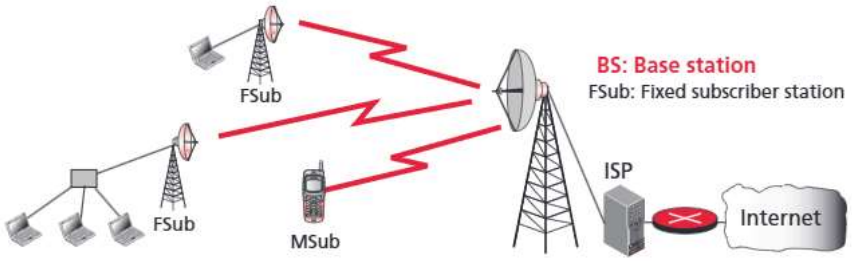
Wireless WANs

The service area of the Internet is so large today that sometimes using only wired WANs cannot provide services to each corner of the world. We definitely need wireless WANs. Several technologies have been used for this purpose as described below.

WiMax

The **Worldwide Interoperability Access (WiMax)** is the wireless version of DSL or Cable connection to the Internet. It provides two types of services (fixed WiMax) to connect the main station to fixed station or to mobile stations such as cellular phones. Figure 6.40 shows both type of connections.

Figure 6.40 *WiMax*



Cellular telephony network

Another wireless WAN today is the **cellular telephony** which was originally designed for voice communication, but it is also used today for Internet communication. We all know that the cellular network divides the earth into cells. The mobile stations communicate with the fixed antenna in the cell that they are inside at each moment. When the user moves to another cell, the communication is between the mobile device and the new antenna.

Satellite networks

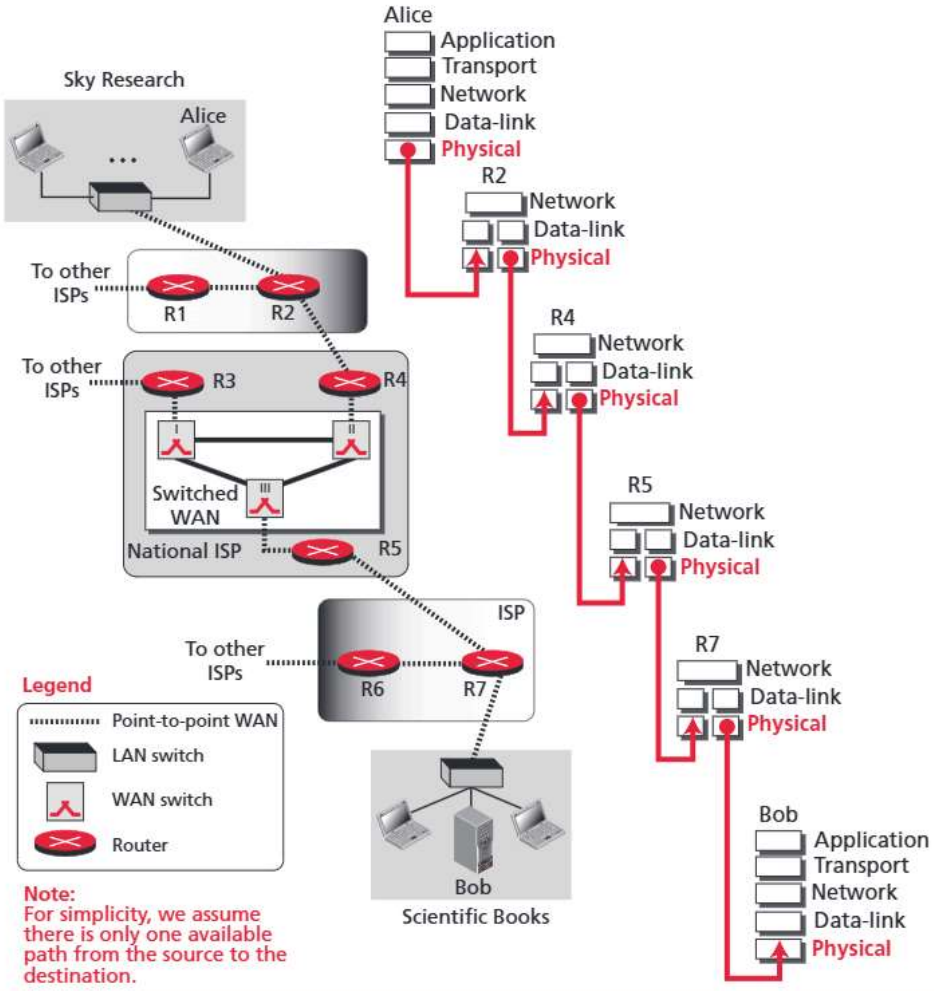
A *satellite network* is a combination of nodes, some of which are satellites, that provides communication from one point on the Earth to another. A node in the network can be a satellite, an Earth station, or an end-user terminal or telephone.

Satellite networks are like cellular networks in that they divide the planet into cells. Satellites can provide transmission capability to and from any location on Earth, no matter how remote. This advantage makes high-quality communication available to less well-developed parts of the world without requiring a huge investment in ground-based infrastructure.

6.6 PHYSICAL LAYER

Our discussion of the TCP/IP protocol suite would be incomplete without the discussion of the physical layer. The role of the physical layer is to transfer the bits received from the data-link layer and convert them to electromagnetic signals for transmission. After the bits are converted to signals, the signals are delivered to the transmission media, which are the subject of our discussion in the next section. Figure 6.41 uses the same scenario we showed in four earlier sections, but the communication is now at the physical layer.

Figure 6.41 *Communication at the physical layer*



6.6.1 Data and signals

At the physical layer, the communication is node-to-node, but the nodes exchange electromagnetic signals.

One of the major functions of the physical layer is to route bits between nodes. However, bits, as the representation of two possible values stored in the memory of a node (host, router, or switch), cannot be sent directly to the transmission medium (wire or air); the bits need to be changed to signals before transmission. So the main duty of the physical layer is to efficiently convert these bits into electromagnetic signals. We first need to understand the nature of the data and then the types of signals to see how we can do this conversion efficiently.

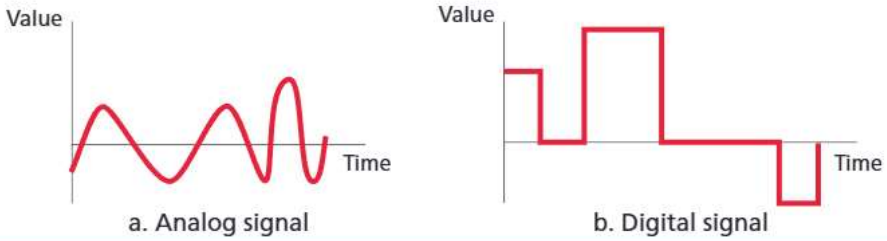
Analog and Digital

Data can be analog or digital. The term **analog data** refers to information that is continuous. Analog data, such as the sounds made by a human voice, take on continuous values. When someone speaks, an analog wave is created in the air. This can be captured by a microphone and converted to an analog signal or sampled and converted to a digital signal.

Digital data take on discrete values. For example, data are stored in computer memory in the form of 0s and 1s. They can be converted to a digital signal or modulated into an analog signal for transmission across a medium.

Like the data they represent, *signals* can be either analog or digital. An **analog signal** has infinitely many levels of intensity over a period of time. As the wave moves from value *A* to value *B*, it passes through and includes an infinite number of values along its path. A **digital signal**, on the other hand, can have only a limited number of defined values. Although each value can be any number, it is often as simple as 1 and 0. The simplest way to show signals is by plotting them on a pair of perpendicular axes. The vertical axis represents the value or strength of a signal. The horizontal axis represents time. Figure 6.42 illustrates an analog signal and a digital signal.

Figure 6.42 *Comparison of analog and digital signals*



a. Analog signal b. Digital signal

6.6.2 Digital transmission

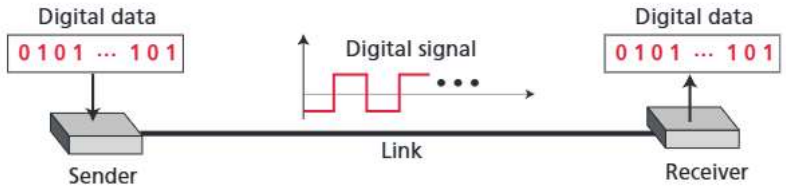
A computer network is designed to send information from one point to another. This information needs to be converted to either a digital signal or an analog signal for transmission. If data is digital, we need to use **digital-to-digital conversion** techniques, methods

which convert digital data to digital signals. If data is analog, we need to use **analog-to-digital conversion** techniques, methods which change an analog signal to a digital signal.

Digital–to–digital conversion

If our data is digital and we need to transmit digital signal, we can use digital-to-digital conversion to change the digital data to digital signal. Although there are many techniques for doing so, in its simplest form, a bit or group of bits is represented by a signal level as shown in Figure 6.43.

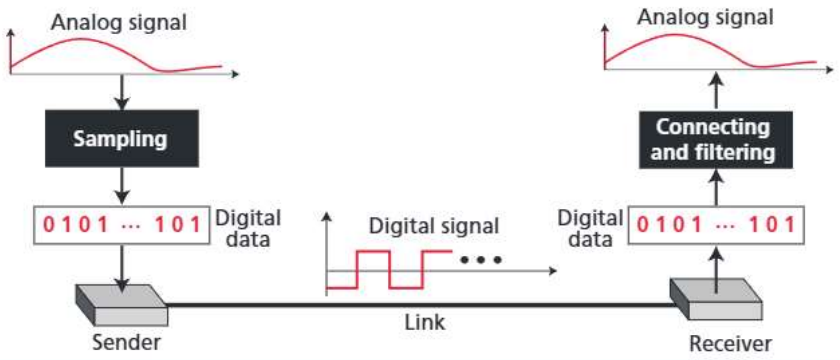
Figure 6.43 *Digital-to-digital conversion*



Analog-to-digital conversion

Sometimes we have an analog signal such as one created by a microphone or camera. The tendency today is to change an analog signal to digital data because the digital signal is less susceptible to noise. Although there are several techniques for doing so, the simplest one is to sample the analog signal to create a digital data and convert the digital data to digital signal as discussed before as shown in Figure 6.44.

Figure 6.44 *Analog-to-digital conversion*



6.6.3 Analog transmission

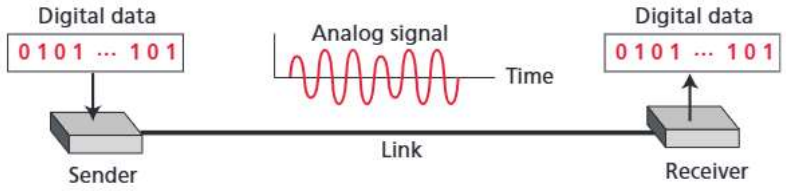
While digital transmission is desirable, it needs a dedicated channel; analog transmission is the only choice if we do not have a dedicated channel. For example, if we are broadcasting

in the air, the air belongs to everyone, so we can use only part of the channel available. Based on the data type available, we can use either digital-to-analog or analog-to analog conversion.

Digital-to-analog conversion

Digital-to-analog conversion is the process of changing one of the characteristics of an analog signal based on the information in digital data. Figure 6.45 shows the relationship between the digital information, the digital-to-analog conversion process, and the resultant analog signal.

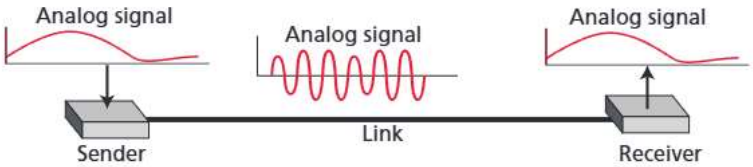
Figure 6.45 *Digital-to-analog conversion*



Analog-to-analog conversion

Analog-to-analog conversion is the process of changing one of the characteristics of an analog signal based on the information in digital data. Figure 6.46 shows the relationship between the digital information, the digital-to-analog conversion process, and the resultant analog signal.

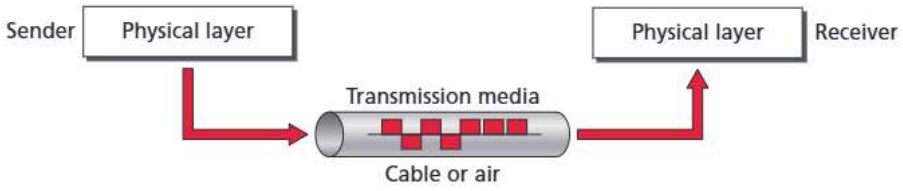
Figure 6.46 *Analog-to-analog conversion*



6.7 TRANSMISSION MEDIA

Electrical signals created at the physical layer need transmission media to go from one point to another. Transmission media are actually located below the physical layer and are directly controlled by the physical layer. We could say that transmission media belong to layer zero. Figure 6.47 shows the position of transmission media in relation to the physical layer.

Figure 6.47 Transmission media and physical layer



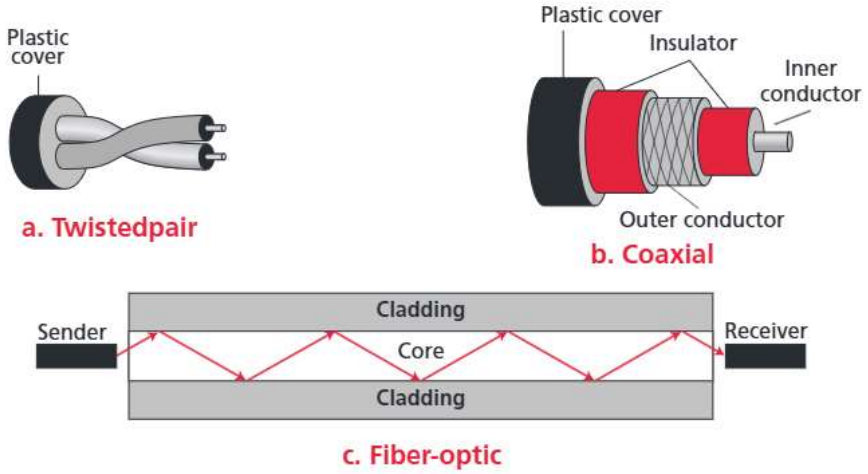
A **transmission medium** can be broadly defined as anything that can carry information from a source to a destination. For example, the transmission medium for two people having a dinner conversation is the air. The air can also be used to convey the message in a smoke signal or semaphore. For a written message, the transmission medium might be a mail carrier, a truck, or an airplane.

In telecommunications, transmission media can be divided into two broad categories: guided and unguided. Guided media include twisted-pair cable, coaxial cable, and fiber-optic cable. Unguided medium is free space.

6.7.1 Guided media

Guided media, which are those that provide a conduit from one device to another, include **twisted-pair cable**, **coaxial cable**, and **fiber-optic cable**. Figure 6.48 shows the three types of guided media.

Figure 6.48 *Guided media*



Twisted-pair cable

A twisted-pair consists of two conductors (normally copper), each with its own plastic insulation, twisted together. One of the wires is used to carry signals to the receiver, and

the other is used only as a ground reference. The receiver uses the difference between the two.

In addition to the signal from the sender, interference (noise) may affect both wires and create unwanted signals. If the two wires are parallel, the effect of these unwanted signals is not the same in both wires because they are at different locations relative to the noise sources. By twisting the pairs, a balance is maintained.

The DSL lines that are used by the telephone companies to provide high-data-rate connections are also twisted-pair cables.

Coaxial cable

Instead of having two wires, coax has a central core conductor of solid or stranded wire (usually copper) enclosed in an insulating sheath, which is, in turn, encased in an outer conductor of metal foil, braid, or a combination of the two. The outer metallic wrapping serves both as a shield against noise and as the second conductor, which completes the circuit. This outer conductor is also enclosed in an insulating sheath, and the whole cable is protected by a plastic cover.

Cable TV networks use coaxial cable. In the traditional cable TV network, the entire network used coaxial cable. Later, however, cable TV providers replaced most of the media with fiber-optic cable; hybrid networks use coaxial cable only at the network boundaries, near the consumer premises.

Fiber-optic cable

A fiber-optic cable is made of glass or plastic and transmits signals in the form of light. This technology uses the property of a beam of light that is refracted (comes back) when it encounters a medium of less density. Covering a glass or plastic medium by another less dense medium (called cladding) guides the light through the medium.

Fiber-optic cable is often found in backbone networks because its wide bandwidth is cost-effective.

6.7.2 Unguided media: wireless

Unguided media transport electromagnetic waves without using a physical conductor. This type of communication is often referred to as *wireless communication*. Signals are normally broadcast through free space and thus are available to anyone who has a device capable of receiving them.

Figure 6.49 shows the part of the electromagnetic spectrum, ranging from 3 kHz to 900 THz, used for wireless communication.

Figure 6.49 *Electromagnetic spectrum for wireless communication*

Communication today uses three different ranges of electromagnetic spectrum: radio waves, microwaves, and infrared.

Radio waves

Electromagnetic waves ranging in frequencies between 3 kHz and 1 GHz are normally called radio waves. They are used mostly for radio communication.

Microwaves

Electromagnetic waves having frequencies between 1 and 300 GHz are called microwaves. Microwaves are unidirectional. When an antenna transmits microwaves, they can be narrowly focused. This means that the sending and receiving antennas need to be aligned. The unidirectional property has an obvious advantage. A pair of antennas can be aligned without interfering with another pair of aligned antennas.

Infrared

Infrared waves, with frequencies from 300 GHz to 400 THz (wavelengths from 1 mm to 770 nm), can be used for short-range communication. Infrared waves, having high frequencies, cannot penetrate walls. This advantageous characteristic prevents interference between one system and another; a short-range communication system in one room cannot be affected by another system in the next room. When we use our infrared remote control, we do not interfere with the use of the remote by our neighbors. However, this same characteristic makes infrared signals useless for long-range communication. In addition, we cannot use infrared waves outside a building because the sun's rays contain infrared waves that can interfere with the communication.

6.8 END-CHAPTER MATERIALS

6.8.1 Recommended reading

For more details about subjects discussed in this chapter, the following books are recommended.

- Forouzan, B. and Mosharrf, F. *Computer Networks: A Top-Down Approach*, New York: McGraw-Hill Education, 2012
- Forouzan, B. *Data Communication and Networking*, New York: McGraw-Hill Education, 2013
- Forouzan, B. *TCP/IP Protocol Suite*, New York: McGraw-Hill Education, 2010
- Forouzan, B. *Local Area Networks*, New York: McGraw-Hill Education, 2003
- Kurose, J. and Ross, K. *Computer Networking*, Reading, MA: Addison-Wesley, 2007

6.8.2 Key terms

10-Gigabit Ethernet 168
analog-to-analog conversion 177
analog-to-digital conversion 176
analog data 175
analog signal 175
application layer 141
bit rate 171
Bluetooth 170
cellular telephony 173
ciphertext 138
client–server paradigm 144
coaxial cable 178
connecting device 134
connectionless protocol 182
country domain 154
demodulator 171
digital-to-analog conversion 177
digital-to-digital conversion 175
digital data 175
digital signal 175
digital subscriber line (DSL) 171
domain name 153
Domain name server (DNS) 152
domain name space 153
dotted-decimal notation 163
electronic mail (email) 150
end system 134
ephemeral port number 158
Fast Ethernet 168
fiber-optic cable 178
File Transfer Protocol (FTP) 149
frame 168
generic domain 153
Gigabit Ethernet 168
guided media 178
hardware 137
internet service provider (ISP) 136
Internetwork 134
IP address 146
IP datagram 158
IP new generation (IPng) 164
link 167
local area network (LAN) 134
Message Access Agent (MAA) 151
message transfer agent (MTA) 151
metropolitan area network (MAN) 169
modem 171
modularity 139
modulator 171
module 139
name space 153
network layer 142
node 167
packetizing 160
peer-to-peer (P2P) paradigm 145
physical layer 141
port number 142
protocol 137
protocol layering 137
remote login 151
router 134
Secure Shell (SHH) 151
segment 159
software 137
source to destination delivery 182
Standard Ethernet 168
switch 134
switched WAN 135
TCP/IP protocol suite 140
TELNET (terminal network) 151
Transmission Control Protocol (TCP) 159
transmission medium 178

header 158	transmission rate 169
host 134	twisted-pair cable 178
host identifier 148	unguided media 179
hypertext 147	uniform resource locator (URL) 149
HyperText Markup Language (HTML) 148	user agent (UA) 151
HyperText Transfer Protocol (HTTP) 149	user datagram 158
infrared waves 180	User Datagram Protocol (UDP) 158
internet 135	web page 147
Internet 136	well-known port number 158
Internet address 163	wide area network (WAN) 134
Internet Protocol 163	Worldwide Interoperability Access (WiMAX) 173
Internet Protocol version 6 (IPv6) 164	World Wide Web (WWW) 145

6.8.3 Summary

- A network is a set of devices connected by communication links. Today when we speak of networks, we are generally referring to two primary categories: local area networks and wide area networks. The Internet today is made up of many wide and local area networks joined by connecting devices and switching stations. A protocol is a set of rules that governs communication. TCP/IP is a hierarchical protocol suite made of five layers: application, transport, network, data-link, and physical.
- Applications in the Internet are designed using either a client–server paradigm or a peer-to-peer paradigm. The World Wide Web (WWW) is a repository of information linked together from points all over the world. The HyperText Transfer Protocol (HTTP) is the main protocol used to access data on the World Wide Web (WWW). File Transfer Protocol (FTP) is a TCP/IP client–server application for copying files from one host to another. Electronic mail is one of the most common applications on the Internet. TELNET is a client–server application that allows a user to log into a remote machine, giving the user access to the remote system. The Domain Name System (DNS) is a client–server application that identifies each host on the Internet with a unique name
- The main duty of a transport-layer protocol is to provide process-to-process communication. UDP is a transport protocol that provides unreliable and connectionless service. Transmission Control Protocol (TCP) is another transport-layer protocol that provides reliable and connection-oriented service.
- The network layer supervises the handling of packets by the underlying physical networks. IPv4 is an unreliable **connectionless protocol** responsible for **source-to-destination delivery**. The identifiers used in the IP layer of the TCP/IP protocol suite

are called the IP addresses. An IPv4 address is 32 bits long. IPv6, the latest version of the Internet Protocol, has a 128-bit address space.

- The data-link layer involves local and wide area networks (LANs and WANs). LANs and WANs can be wired or wireless. Ethernet is the most widely used wired local area network protocol. Dial-up service, DSL, and cable are mostly used for point-to-point wired WANs. Wireless LANs became formalized with wireless Ethernet. Bluetooth is a wireless LAN technology that connects devices (called gadgets) in a small area. WiMAX is a wireless access network that may replace DSL and cable in the future.
- Data must be transformed to electromagnetic signals to be transmitted. Analog data are continuous and take continuous values. Digital data have discrete states and take discrete values. Digital-to-digital conversion changes digital data to digital signal. Digital-to-analog conversion is the process of changing digital data to analog signal. Analog-to-digital conversion is the process of sampling analog data and changing it to digital signal. Analog-to-analog signal means changing analog data to analog signal.
- Transmission media lie below the physical layer. A guided medium provides a physical conduit from one device to another. Twisted-pair cable, coaxial cable, and optical fiber are the most popular types of guided media. Unguided media (free space) transport electromagnetic waves without the use of a physical conductor.

6.9 PRACTICE SET

6.9.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

6.9.2 Review questions

Q6-1. What is the first principle we discussed in this chapter for protocol layering that needs to be followed to make the communication bidirectional?

Q6-2. In the TCP/IP protocol suite, what are the identical objects at the sender and the receiver sites when we think about the logical connection at the application layer?

Q6-3. A host communicates with another host using the TCP/IP protocol suite. What is the unit of data sent or received at each of the following layers?

a. application layer b. network layer c. data-link layer

Q6-4. Which of the following data units is encapsulated in a frame?

a. a user datagram b. a datagram c. a segment

Q6-5. Which of the following data units is decapsulated from a user datagram?
a. a datagram b. a segment c. a message

Q6-6. Which of the following data units has an application-layer message plus the header from layer 4?
a. a frame b. a user datagram c. a bit

Q6-7. What are the types of addresses (identifiers) used in each of the following layers?
a. application layer b. network layer c. data-link layer

Q6-8. During the weekend, Alice often needs to access files stored on her office desktop, from her home laptop. Last week, she installed a copy of the FTP server process on her desktop at her office and a copy of the FTP client process on her laptop at home. She was disappointed when she could not access her files during the weekend. What could have gone wrong?

Q6-9. Most of the operating systems installed on personal computers come with several client processes, but normally no server processes. Explain the reason.

Q6-10. A new application is to be designed using the client–server paradigm. If only small messages need to be exchanged between the client and the server without the concern for message loss or corruption, what transport-layer protocol do you recommend?

Q6-11. Why is the routing the responsibility of the network layer? In other words, why can't the routing be done at the transport layer or the data-link layer?

Q6-12. Distinguish between communication at the network layer and communication at the data-link layer.

Q6-13. What is dial-up modem technology? List some of the common modem standards discussed in this chapter and give their data rates.

Q6-14. What is the reason that Bluetooth is normally called a wireless personal area network (WPAN) instead of a wireless local area network (WLAN)?

Q6-15. How can we find the period of a sine wave when its frequency is given?

Q6-16. Which of the following measures the value of a signal at any time?
a. amplitude b. frequency c. phase

Q6-17. Define analog transmission.

Q6-18. What is the position of the transmission media in the TCP/IP protocol suite?

Q6-19. Name the two major categories of transmission media.

Q6-20. What are the three major classes of guided media?

6.9.3 Problems

P6-1. Answer the following questions about Figure 6.5 when the communication is from Maria to Ann:
a. What is the service provided by layer 1 to layer 2 at Maria's site?
b. What is the service provided by layer 1 to layer 2 at Ann's site?

P6-2. Answer the following questions about Figure 6.5 when the communication is from Maria to Ann:
a. What is the service provided by layer 2 to layer 3 at Maria's site?
b. What is the service provided by layer 2 to layer 3 at Ann's site?

P6-3. Assume that the number of hosts connected to the Internet at year 2010 is five hundred million. If the number of hosts increases only 20 per cent per year, what is the number of hosts in year 2020?

P6-4. Assume a system uses five protocol layers. If the application program creates a message of 100 bytes and each layer (including the fifth and the first) adds a header of ten bytes to the data unit, what is the efficiency (the ratio of application-layer bytes to the number of bytes transmitted) of the system?

P6-5. Match the following to one or more layers of the TCP/IP protocol suite:

a. route determination

b. connection to transmission media

c. providing services for the end user

P6-6. Match the following to one or more layers of the TCP/IP protocol suite:

a. creating user datagrams

b. responsibility for handling frames between adjacent nodes

c. transforming bits to electromagnetic signals

P6-7. Protocol layering can be found in many aspects of our lives such as air travelling. Imagine you make a round-trip to spend some time on vacation at a resort. You need to go through some processes at your city airport before flying. You also need to go through some processes when you arrive at the resort airport. Show the protocol layering for the round trip using some layers such as baggage checking/claiming, boarding/unboarding, takeoff/landing.

P6-8. In an internet, we change the LAN technology to a new one. Which layers in the TCP/IP protocol suite need to be changed?

P6-9. Compare the range of 16-bit addresses, 0 to 65 535, with the range of 32-bit IP addresses, 0 to 4 294 967 295. Why do we need such a large range of IP addresses, but only a relatively small range of port numbers?

P6-10. Rewrite the following IP addresses using binary notation:

a. 110.11.5.88 b. 12.74.16.18 c. 201.24.44.32

P6-11. Rewrite the following IP addresses using dotted-decimal notation:

a. 01011110 10110000 01110101 00010101

b. 10001001 10001110 11010000 00110001

c. 01010111 10000100 00110111 00001111

P6-12. What is the hexadecimal equivalent of the following Ethernet address?

```
01011010 00010001 01010101 00011000 10101010 00001111
```

P6-13. A device is sending out data at the rate of 1000 bps.

a. How long does it take to send out 10 bits?

a. How long does it take to send out a character (8 bits)?

b. How long does it take to send a file of 100 000 characters?

CHAPTER 7

Operating Systems

This is the first chapter in this book to deal with computer software. In this chapter we explore the role of the operating system in a computer.

Objectives

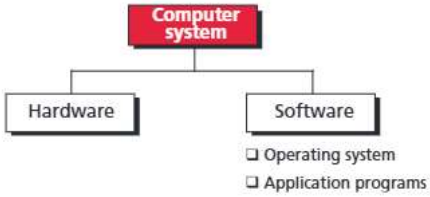
After studying this chapter, the student should be able to:

- Understand the role of the operating system in a computer system.
- Give a definition of an operating system.
- Understand the process of bootstrapping to load the operating system into memory.
- List the components of an operating system.
- Discuss the role of the memory manager in an operating system.
- Discuss the role of the process manager in an operating system.
- Discuss the role of the device manager in an operating system.
- Discuss the role of the file manager in an operating system.
- Understand the main features of three common operating systems: UNIX, Linux, and Windows.

7.1 INTRODUCTION

A computer is a system composed of two major components: *hardware* and *software*. Computer hardware is the physical equipment. Software is the collection of programs that allows the hardware to do its job. Computer **software** is divided into two broad categories: the *operating system* and *application programs* (Figure 7.1). Application programs use the computer hardware to solve users' problems. The operating system, on the other hand, controls the access to hardware by users.

Figure 7.1 *A computer system*



7.1.1 Operating system

An **operating system** is complex, so it is difficult to give a simple universal definition. Instead, here are some common definitions:

- An operating system is an interface between the hardware of a computer and the user (programs or humans).
- An operating system is a program (or a set of programs) that facilitates the execution of other programs.
- An operating system acts as a general manager supervising the activity of each component in the computer system. As a general manager, the operating system checks that hardware and software resources are used efficiently, and when there is a conflict in using a resource, the operating system mediates to solve it.

An operating system is an interface between the hardware of a computer and the user (programs or humans) that facilitates the execution of other programs and the access to hardware and software resources.

Two major design goals of an operating system are:

- Efficient use of hardware.
- Easy use of resources.

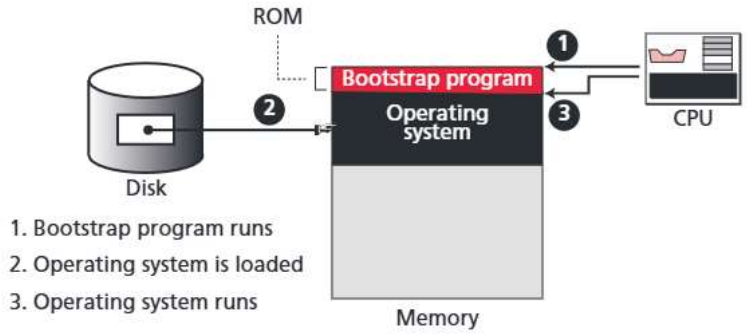
7.1.2 Bootstrap process

The operating system, based on the above definitions, provides support for other programs. For example, it is responsible for loading other programs into memory for execution. However, the operating system itself is a program that needs to be loaded into the memory and be run. How is this dilemma solved?

The problem can be solved if the operating system is stored (by the manufacturer) in part of memory using ROM technology. The program counter of the CPU (see Chapter 5) can be set to the beginning of this ROM memory. When the computer is turned on, the CPU reads instructions from ROM and executes them. This solution, however, is not very efficient, because a significant part of the memory would need to be composed of ROM and could not therefore be used by other programs. Today's technology needs to allocate just a small part of memory to part of the operating system.

The solution adopted today is a two-stage process. A very small section of memory is made of ROM and holds a small program called the bootstrap program. When the computer is turned on, the CPU counter is set to the first instruction of this bootstrap program and executes the instructions in this program. This program is only responsible for loading the operating system itself, or that part of it required to start up the computer, into RAM memory. When loading is done, the program counter in the CPU is set to the first instruction of the operating system in RAM and the operating system is executed. Figure 7.2 illustrates the bootstrap process.

Figure 7.2 *The bootstrap process*



7.2 EVOLUTION

Operating systems have gone through a long history of evolution, which we summarize next.

7.2.1 Batch systems

Batch operating systems were designed in the 1950s to control mainframe computers. At that time, computers were large machines that used punched cards for input, line printers for output, and tape drives for secondary storage media.

Each program to be executed was called a job. A programmer who wished to execute a job sent a request to the operating room along with punched cards for the program and data. The punched cards were fed into the computer by an operator. If the program was successful, a printout of the result was sent to the programmer—if not, a printout of the error was sent.

Operating systems during this era were very simple: they only ensured that all of the computer's resources were transferred from one job to the next.

7.2.2 Time-sharing systems

To use computer system resources efficiently, *multiprogramming* was introduced. The idea is to hold several jobs in memory at a time, and only assign a resource to a job that needs it on the condition that the resource is available. For example, when one program is using an input/output device, the CPU is free and can be used by another program. We discuss multiprogramming later in this chapter.

Multiprogramming brought the idea of time sharing: resources could be shared between different jobs, with each job being allocated a portion of time to use a resource. Because a computer is much faster than a human, time sharing is hidden from the user—each user has the impression that the whole system is serving them exclusively.

Multiprogramming, and eventually time sharing, improved the efficiency of computer systems tremendously. However, they required a more complex operating system. The operating system now had to do scheduling: allocating resources to different programs and deciding which program should use which resource, and when. During this era, the relationship between a computer and a user also changed. The user could directly interact with the system without going through an operator. A new term was also coined: process. A job is a program to be run, while a process is a program that is in memory and waiting for resources.

7.2.3 Personal systems

When personal computers were introduced, there was a need for an operating system for this new type of computer. During this era, **single-user operating systems** such as DOS (Disk Operating System) were introduced.

7.2.4 Parallel systems

The need for more speed and efficiency led to the design of **parallel systems:** multiple CPUs on the same machine. Each CPU can be used to serve one program or a part of a program, which means that many tasks can be accomplished in parallel instead of serially. The operating systems required for this are more complex than those that support single CPUs.

7.2.5 Distributed systems

Networking and internetworking, as we saw in Chapter 6, have created a new dimension in operating systems. A job that was previously done on one computer can now be shared between computers that may be thousands of miles apart. A program can be run partially on one computer and partially on another if they are connected through an internetwork such as the Internet. In addition, resources can be distributed. A program may need files located in different parts of the world. **Distributed systems** combine features of the previous generation with new duties such as controlling security.

7.2.6 Real-time systems

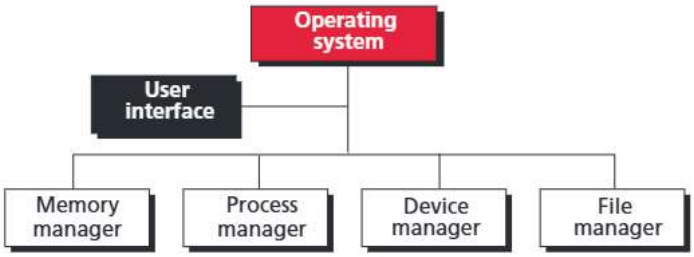
A **real-time system** is expected to do a task within specific time constraints. They are used with real-time applications, which monitor, respond to, or control external processes or environments. Examples can be found in traffic control, patient monitoring, or military control systems. The application program can sometimes be an embedded system such as a component of a large system, such as the control system in an automobile.

The requirements for a real-time operating system are often different than those for a general-purpose system. For this reason, we do not discuss them in this chapter.

7.3 COMPONENTS

Today's operating systems are very complex. An operating system needs to manage different resources in a computer system. It resembles an organization with several managers at the top level. Each manager is responsible for managing their department, but also needs to cooperate with others and coordinate activities. A modern operating system has at least four duties: memory manager, **process manager**, device manager, and file manager. Like many organizations that have a department that is not necessarily under any specific manager, an operating system also has such a component, which is usually called a user interface or a **shell**. The user interface is responsible for communication outside the operating system. Figure 7.3 shows the typical components of an operating system.

Figure 7.3 *Components of an operating system*



7.3.1 User interface

Each operating system has a **user interface**, a program that accepts requests from users (processes) and interprets them for the rest of the operating system. A user interface in some operating systems, such as UNIX, is called a **shell**. In others, it is called a window to denote that it is menu driven and has a **GUI (graphical user interface)** component.

7.3.2 Memory manager

One of the responsibilities of a modern computer system is **memory management**. Although the memory size of computers has increased tremendously in recent years, so has the size of the programs and data to be processed. Memory allocation must be managed to prevent applications from running out of memory. Operating systems can be divided into two broad categories of memory management: *monoprogramming* and *multiprogramming*.

Monoprogramming

Monoprogramming belongs to the past, but it is worth mentioning because it helps us to understand multiprogramming. In monoprogramming, most of the memory capacity is

dedicated to a single program (we consider the data to be processed by a program as part of the program): only a small part is needed to hold the operating system. In this configuration, the whole program is in memory for execution. When the program finishes running, the program area is occupied by another program (Figure 7.4).

Figure 7.4 *Monoprogramming*

The job of the memory manager is straightforward here. It loads the program into memory, runs it, and replaces it with the next program. However, there are several problems with this technique:

- The program must fit into memory. If the size of memory is less than the size of the program, the program cannot be run.
- When one program is being run, no other program can be executed. A program, during its execution, often needs to receive data from input devices and needs to send data to output devices. Input/output devices are slow compared with the CPU, so when the input/output operations are being carried out, the CPU is idle. It cannot serve another program because this program is not in memory. This is a very inefficient use of memory and CPU time.

Multiprogramming

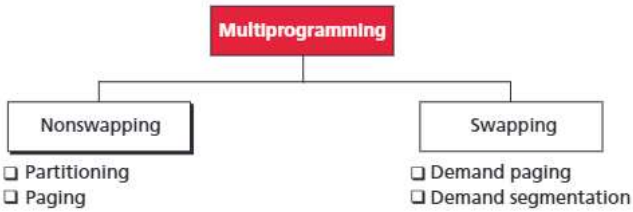
In **multiprogramming**, more than one program is in memory at the same time, and they are executed concurrently, with the CPU switching rapidly between the programs. Figure 7.5 shows memory in a multiprogramming environment.

Since the 1960s, multiprogramming has gone through several improvements that can be seen in the taxonomy in Figure 7.6.

Figure 7.5 *Multiprogramming*

We discuss each scheme very briefly in the next few sections. Two techniques belong to the *nonswapping* category, which means that the program remains in memory for the duration of execution. The other two techniques belong to the *swapping* category. This means that, during execution, the program can be swapped between memory and disk one or more times.

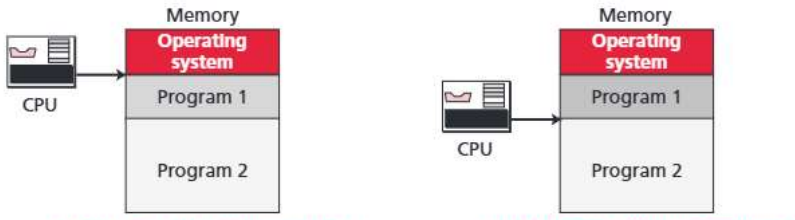
Figure 7.6 *Categories of multiprogramming*



Partitioning

The first technique used in multiprogramming is called **partitioning**. In this scheme, memory is divided into variable-length sections. Each section or partition holds one program. The CPU switches between programs. It starts with one program, executing some instructions until it either encounters an input/output operation or the time allocated for that program has expired. The CPU then saves the address of the memory location where the last instruction was executed and moves to the next program. The same procedure is repeated with the second program. After all the programs have been served, the CPU moves back to the first program. Priority levels can also be used to control the amount of CPU time allocated to each program (Figure 7.7).

Figure 7.7 *Partitioning*



With this technique, each program is entirely in memory and occupying contiguous locations. Partitioning improves the efficiency of the CPU, but there are still some issues:

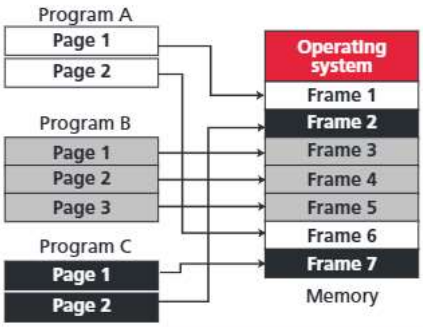
- The size of the partitions has to be determined beforehand by the memory manager. If partition sizes are small, some programs cannot be loaded into memory. If partition sizes are large, there might be some 'holes' (unused locations) in memory.
- Even if partitioning is perfect when the computer is started, there may be some holes after completed programs are replaced by new ones.

- When there are many holes, the memory manager can compact the partitions to remove the holes and create new partitions, but this creates extra overhead on the system.

Paging

Paging improves the efficiency of partitioning. In paging, memory is divided into equally sized sections called **frames**. Programs are also divided, into equally sized sections called **pages**. The size of a page and a frame is usually the same and equal to the size of the block used by the system to retrieve information from a storage device (Figure 7.8).

Figure 7.8 *Paging*



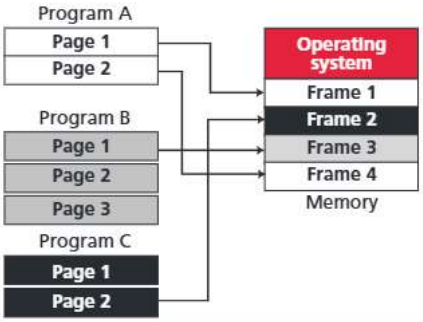
A page is loaded into a frame in memory. If a program has three pages, it occupies three frames in memory. With this technique, the program does not have to be contiguous in memory: two consecutive pages can occupy noncontiguous frames in memory. The advantage of paging over partitioning is that two programs, each using three noncontiguous frames, can be replaced by one program that needs six frames. There is no need for the new program to wait until six contiguous frames are free before being loaded into memory.

Paging improves efficiency to some extent, but the whole program still needs to be in memory before being executed. This means that a program that needs six frames, for example, cannot be loaded into memory if there are currently only four unoccupied frames.

Demand paging

Paging does not require that the program be in contiguous memory locations, but it does require that the entire program be in memory for execution. Demand paging has removed this last restriction. In **demand paging** the program is divided into pages, but the pages can be loaded into memory one by one, executed, and replaced by another page. In other words, memory can hold pages from multiple programs at the same time. In addition, consecutive pages from the same program do not have to be loaded into the same frame—a page can be loaded into any free frame. An example of demand paging is shown in Figure 7.9. Two pages from program A, one page from program B, and one page from program C are in the memory.

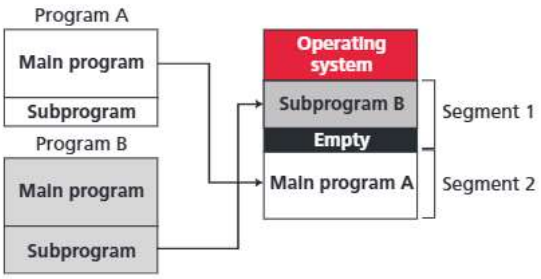
Figure 7.9 *Demand paging*



Demand segmentation

A technique similar to paging is *segmentation*. In paging, a program is divided into equally sized pages, which is not the way a programmer thinks—a programmer thinks in terms of modules. As we will see in later chapters, a program is usually made up of a main program and subprograms. In **demand segmentation**, the program is divided into segments that match the programmer's view. These are loaded into memory, executed, and replaced by another module from the same or a different program. An example of demand segmentation is shown in Figure 7.10. Since segments in memory are of equal size, part of a segment may remain empty.

Figure 7.10 *Demand segmentation*



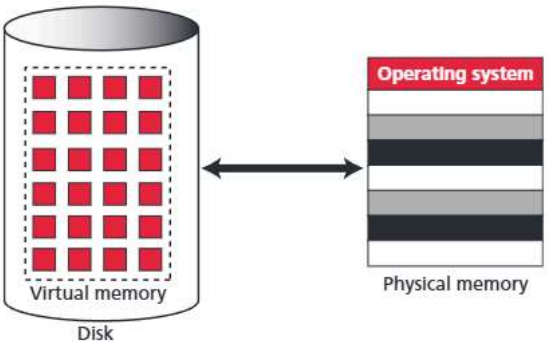
Demand paging and segmentation

Demand paging and segmentation can be combined to further improve the efficiency of the system. A segment may be too large to fit any available free space in memory. Memory can be divided into frames, and a module can be divided into pages. The pages of a module can then be loaded into memory one by one and executed.

Virtual memory

Demand paging and demand segmentation mean that, when a program is being executed, part of the program is in memory and part is on disk. This means that, for example, a memory size of 10 MB can execute ten programs, each of size 3 MB, for a total of 30 MB. At any moment, 10 MB of the ten programs are in memory and 20 MB are on disk. There is therefore an actual memory size of 10 MB, but a **virtual memory** size of 30 MB. Figure 7.11 shows the concept. Virtual memory, which implies demand paging, demand segmentation, or both, is used in almost all operating systems today.

Figure 7.11 *Virtual memory*



7.3.3 Process manager

A second function of an operating system is process management, but before discussing this concept, we need to define some terms.

Program, job, and process

Modern operating systems use three terms that refer to a set of instructions: *program*, *job*, and *process*. Although the terminology is vague and varies from one operating system to another, we can define these terms informally.

Program

A **program** is a nonactive set of instructions stored on disk (or tape). It may or may not become a job.

Job

A program becomes a **job** from the moment it is selected for execution until it has finished running and becomes a program again. During this time a job may or may not be executed. It may be located on disk waiting to be loaded to memory, or it may be loaded into memory and waiting for execution by the CPU. It may be on disk or in memory waiting for an input/output event, or it may be in memory while being executed by the CPU. The program is a job in all of these situations. When a job has finished executing (either normally or abnormally), it becomes a program and once again resides on the

disk. The operating system no longer governs the program. Note that every job is a program, but not every program is a job.

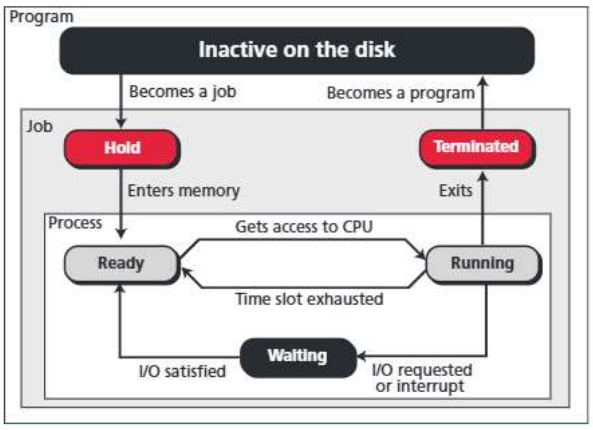
Process

A **process** is a program in execution. It is a program that has started but has not finished. In other words, a process is a job that is being run in memory. It has been selected among other waiting jobs and loaded into memory. A process may be executing or it may be waiting for CPU time. As long as the job is in memory, it is a process. Note that every process is a job, but not every job is a process.

State diagrams

The relationship between a program, a job, and a process becomes clearer if we consider how a program becomes a job and how a job becomes a process. This can be illustrated with a **state diagram** that shows the different states of each of these entities. Figure 7.12 is a state diagram using boundaries between a program, a job, and a process.

Figure 7.12 *State diagram with boundaries between program, job, and process*



A program becomes a job when selected by the operating system and brought to the **hold state**. It remains in this state until it can be loaded into memory. When there is memory space available to load the program totally or partially, the job moves to the **ready state**. It now becomes a process. It remains in memory and in this state until the CPU can execute it, moving to the **running state** at this time. When in the running state, one of three things can happen:

- The process executes until it needs I/O resources
- The process exhausts its allocated time slot
- The process terminates

In the first case, the process goes into the **waiting state** and waits until I/O is complete. In the second case, it goes directly to the ready state. In the third case, it goes into the

terminated state and is no longer a process. A process can move between the running, waiting, and ready states many times before it goes to the terminated state. Note that the diagram can be much more complex if the system uses virtual memory and swaps programs in and out of main memory.

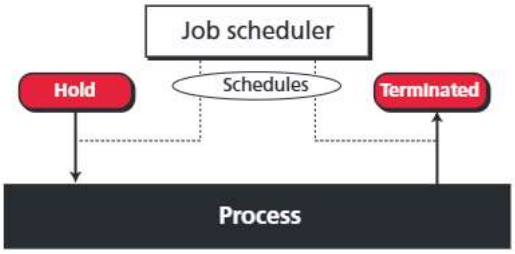
Schedulers

To move a job or process from one state to another, the process manager uses two **schedulers**: the job scheduler and the process scheduler.

Job scheduler

The **job scheduler** moves a job from the hold state to the ready state or from the running state to the terminated state. In other words, a job scheduler is responsible for creating a process from a job and terminating a process. Figure 7.13 shows the job scheduler.

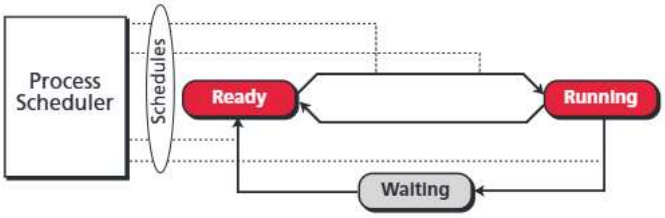
Figure 7.13 *Job scheduler*



Process scheduler

The **process scheduler** moves a process from one state to another. It moves a process from the running state to the waiting state when the process is waiting for some event to happen. It moves the process from the waiting state to the ready state when the event has occurred. It moves a process from the running state to the ready state if the process' time allotment has expired. When the CPU is ready to run the process, the process scheduler moves the process from the ready state to the running state. Figure 7.14 shows the process scheduler.

Figure 7.14 *Process scheduler*



Other schedulers

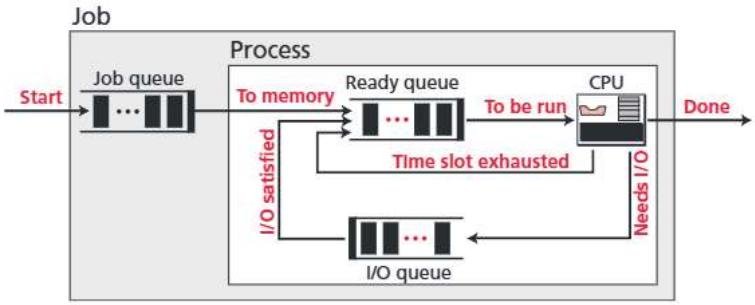
Some operating systems use other types of schedulers to make switching between processes more efficient.

Queuing

Our state diagram shows one job or process moving from one state to another. In reality, there are many jobs and many processes competing with each other for computer resources. For example, when some jobs are in memory, others must wait until space is available. Or when a process is running using the CPU, others must wait until the CPU is free. To handle multiple processes and jobs, the process manager uses **queues** (waiting lists). A *job control block* or *process control block* is associated with each job or process. This is a block of memory that stores information about that job or process. The process manager stores the job or process control block in the queues instead of the job or process itself. The job or process itself remains in memory or disk, as it is too big to be duplicated in a queue: the job control block or process control block is the representative of the waiting job or process.

An operating system can have several queues. For example, Figure 7.15 shows the circulation of jobs and processes through three queues: the job queue, the ready queue, and the I/O queue. The job queue holds the jobs that are waiting for memory. The ready queue holds the processes that are in memory, ready to be run and waiting for the CPU. The I/O queue holds the processes that are waiting for an I/O device (there can be several I/O queues, one for each input/output device, but we show only one for simplicity).

Figure 7.15 *Queues for process management*



The process manager can have different policies for selecting the next job or process from a queue: it could be first in, first out (FIFO), shortest length first, highest priority first, and so on.

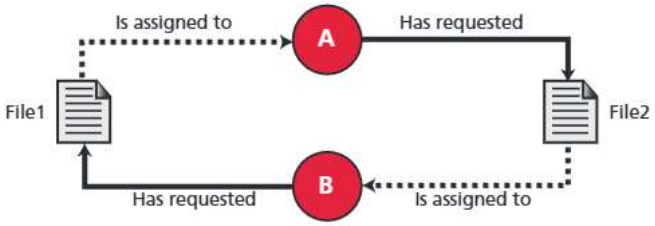
Process synchronization

The whole idea behind process management is to synchronize different processes with different resources. Whenever resources can be used by more than one user (or process, in this case), we can have two problematic situations: *deadlock* and *starvation*. A brief discussion of these two situations follows.

Deadlock

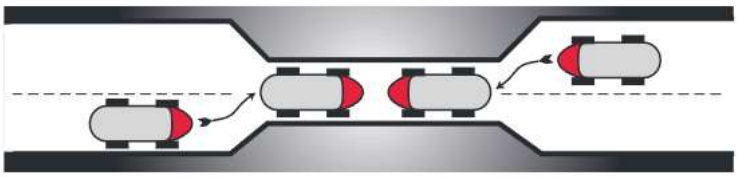
Instead of a formal definition of **deadlock**, we give an example. Assume that there are two processes, A and B. Process A is holding a file, File1 (that is, File1 is assigned to A) and cannot release it until it acquires another file, File2 (that is, A has requested File2). Process B is holding File2 (that is, File2 is assigned to B) and cannot release it until it has File1 (that is, B has requested File1). Files in most systems are not sharable—when in use by one process, a file cannot be used by another process. If there is no provision in this situation to force a process to release a file, deadlock is created (Figure 7.16).

Figure 7.16 *Deadlock*



As an analogy, Figure 7.17 shows deadlock on a narrow bridge. The situation is similar because the resource (part of the bridge) is held by a vehicle that does not release it until it gets the other part of the bridge, which is held by the other vehicle, and *vice versa*.

Figure 7.17 *Deadlock on a bridge*



Deadlock occurs if the operating system allows a process to start running without first checking to see if the required resources are ready, and allows a process to hold a resource as long as it wants. There should be some provision in the system to prevent deadlock. One solution is not to allow a process to start running until the required resources are free, but we will see later that this creates another problem. The second solution is to limit the time a process can hold a resource.

Deadlock occurs when the operating system does not put resource restrictions on processes.

Deadlock does not always occur. There are four necessary conditions for deadlock as shown below:

- **Mutual exclusion**. Only one process can hold a resource
- **Resource holding**. A process holds a resource even though it cannot use it until other resources are available
- **No preemption**. The operating system cannot temporarily reallocate a resource
- **Circular waiting**. All processes and resources involved form a loop, as in Figure 7.16

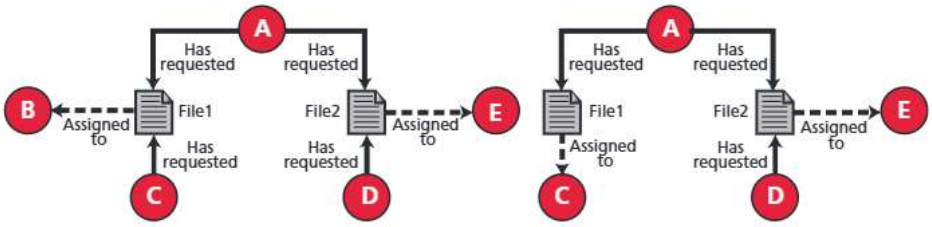
All four conditions are required for deadlock to occur. However, these conditions are only necessary preconditions, and are not sufficient to cause deadlock of themselves—they must be present for deadlock, but they might not be enough to cause it. If one of these conditions is missing, deadlock cannot occur. This gives us a method for preventing or avoiding deadlock: do not allow one of these conditions to happen.

Starvation

Starvation is the opposite of deadlock. It can happen when the operating system puts too many resource restrictions on a process. For example, imagine an operating system that specifies that a process must have possession of its required resources before it can be run.

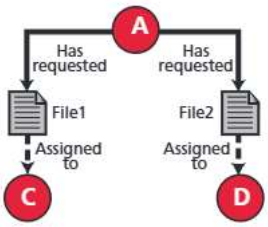
In Figure 7.18, imagine that process A needs two files, File1 and File2. File1 is being used by process B and File2 is being used by process E. Process B terminates first and releases File1. Process A cannot be started, because File2 is still not available. At this moment, process C, which needs only File1, is allowed to run. Now process E terminates and releases File2, but process A still cannot run because File1 is unavailable.

Figure 7.18 *Starvation*



a. Process A needs both File1 and File2

b. Process A still needs both File1 and File2



c. Process A still needs both File1 and File2 (starving)

Figure 7.19 *The dining philosophers problem*

A classic starvation problem is the one introduced by Edsger Dijkstra. Five philosophers are sitting at a round table (Figure 7.19). Each philosopher needs two chopsticks to eat a bowl of rice. However, one or both chopsticks could be used by a neighbor. A philosopher could starve if two chopsticks are not available at the same time.

Device manager

The **device manager**, or input/output manager, is responsible for access to input/output devices. There are limitations on the number and speed of input/output devices in a computer system. Because these devices are slower in speed compared with the CPU and memory, when a process accesses an input/output device, the device is not available to other processes for a period of time. The device manager is responsible for the efficient use of input/output devices.

A detailed discussion of device managers requires advanced knowledge of operating system principles and is beyond the scope of this book. However, we can briefly list the responsibilities of a device manager:

- The device manager monitors every input/output device constantly to ensure that the device is functioning properly. The manager also needs to know when a device has finished serving one process and is ready to serve the next process in the queue.
- The device manager maintains a queue for each input/output device or one or more queues for similar input/output devices. For example, if there are two fast printers in the system, the manager can have one queue for each or one queue for both.
- The device manager controls the different policies for accessing input/output devices. For example, it may use FIFO for one device and shortest length first for another.

7.3.4 File manager

Operating systems today use a file manager to control access to files. A detailed discussion of the file manager also requires advanced knowledge of operating system principles and file access concepts that are beyond the scope of this book. We discuss some issues related to file access in Chapter 13, but this is not adequate to understand the actual operation of a file manager. Here is a brief list of the responsibilities of a file manager:

5/1/25, 9:57 PM study viewer

- The file manager controls access to files. Access is permitted only by permitted applications and/or users, and the type of access can vary. For example, a process (or a user that calls a process) may be allowed to read from a file but is allowed to write to it (that is, change it). Another process may be allowed to execute a file and a process, but not allowed to read its contents, and so on.
- The file manager supervises the creation, deletion, and modification of files.
- The file manager can control the naming of files.
- The file manager supervises the storage of files: how they are stored, where they are stored, and so on.
- The file manager is responsible for archiving and backups.

7.4 A SURVEY OF OPERATING SYSTEMS

In this section we introduce some popular operating systems and encourage you to study them further. We have chosen three operating systems that are familiar to most computer users: UNIX, Linux, and Windows.

7.4.1 UNIX

UNIX was originally developed in 1969 by Thomson and Ritchie of the Computer Science Research Group at Bell Laboratories. UNIX has gone through many versions since then. It has been a popular operating system among computer programmers and computer scientists. It is a very powerful operating system with three outstanding features. First, UNIX is a portable operating system that can be moved from one platform to another without many changes. The reason is that it is written mostly in the C language (instead of a machine language specific to a particular computer system). Second, UNIX has a powerful set of utilities (commands) that can be combined (in an executable file called a *script*) to solve many problems that require programming in other operating systems. Third, it is device-independent, because it includes device drivers in the operating system itself, which means that it can be easily configured to run any device.

UNIX is a multiuser, multiprocessing, portable operating system designed to facilitate programming, text processing, communication, and many other tasks that are expected from an operating system. It contains hundreds of simple, single-purpose functions that can be combined to do virtually every processing task imaginable. Its flexibility is demonstrated by the fact that it is used in three different computing environments: stand-alone personal environments, time-sharing systems, and client–server systems.

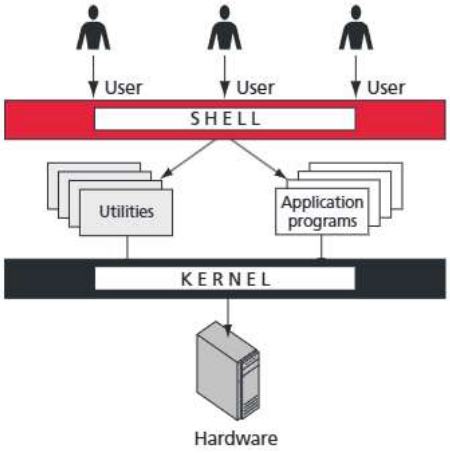
UNIX is a multiuser, multiprocessing, portable operating system.
It is designed to facilitate programming, text processing, and communication.

UNIX structure

UNIX consists of four major components: the *kernel*, the *shell*, a standard set of *utilities*, and *application programs*. These components are shown in Figure 7.20.

https://studylib.net/doc/26059254/foundations-computer-science-4th 223/706

Figure 7.20 *Components of the UNIX operating system*



The kernel

The **kernel** is the heart of the UNIX system. It contains the most basic parts of the operating system: memory management, process management, device management, and file management. All other components of the system call on the kernel to perform these services for them.

The shell

The shell is the part of UNIX that is most visible to the user. It receives and interprets the commands entered by the user. In many respects, this makes it the most important component of the UNIX structure. It is certainly the part that users get to know best. To do anything in the system, we must give the shell a command. If the command requires a utility, the shell requests that the kernel execute the utility. If the command requires an application program, the shell requests the kernel to run it. Some operating systems, such as UNIX, have several different shells.

Utilities

There are literally hundreds of UNIX utilities. A **utility** is a standard UNIX program that provides a support process for users. Three common utilities are text editors, search programs, and sort programs.

Many of the system utilities are actually sophisticated applications. For example, the UNIX email system is considered a utility, as are the three common text editors, **vi**, **emacs**, and **pico**. All four of these utilities are large systems in themselves. Other utilities are shorter, simpler functions. For example, the list (**ls**) utility displays the files in a disk directory.

Applications

Applications in UNIX are programs that are not a standard part of the operating system distribution. Written by systems administrators, professional programmers, or

users, they provide extended capabilities to the system. In fact, many of the standard utilities started out as applications years ago and proved so useful that they are now part of the system.

7.4.2 Linux

In 1991, Linus Torvalds, a Finnish student at the University of Helsinki at the time, developed a new operating system that is known today as **Linux**. The initial kernel, which was similar to a small subset of UNIX, has grown into a full-scale operating system today. The Linus 2.0 kernel, released in 1997, was accepted as a commercial operating system: it has all the features traditionally attributed to UNIX.

Components

Linux has the following components.

Kernel

The kernel is responsible for all duties attributed to a kernel, such as memory management, process management, device management, and file management.

System libraries

The system libraries hold a set of functions used by the application programs, including the shell, to interact with the kernel.

System utilities

The system utilities are individual programs that use the services provided by the system libraries to perform management tasks.

Networking capabilities

Linux supports the standard Internet protocols discussed in Chapter 6. It supports three layers: the socket interface, protocol drivers, and network device drivers.

Security

Linux' security mechanism provides the security aspects defined traditionally for UNIX, such as **authentication** and access control.

7.4.3 Windows

In the late 1980s, Microsoft started development of a new single-user operating system to replace MS-DOS (Microsoft Disk Operating System). **Windows** was the result. Several versions of Windows followed. We refer to all of these versions as Windows.

Design goals

Design goals released by Microsoft are *extensibility*, *portability*, *reliability*, *compatibility*, and *performance*.

Extensibility

Windows is designed as a modular architecture with several layers. The purpose is to let the higher layers to be changed with time without affecting the lower layers.

Portability

Windows, like UNIX, is mostly is written in C or C++ and the code is independent of the machine language of the computer on which it is running.

Reliability

Windows was designed to handle error conditions including protection from malicious software.

Compatibility

Windows was designed to run programs written for other operating systems and the earlier versions of Windows.

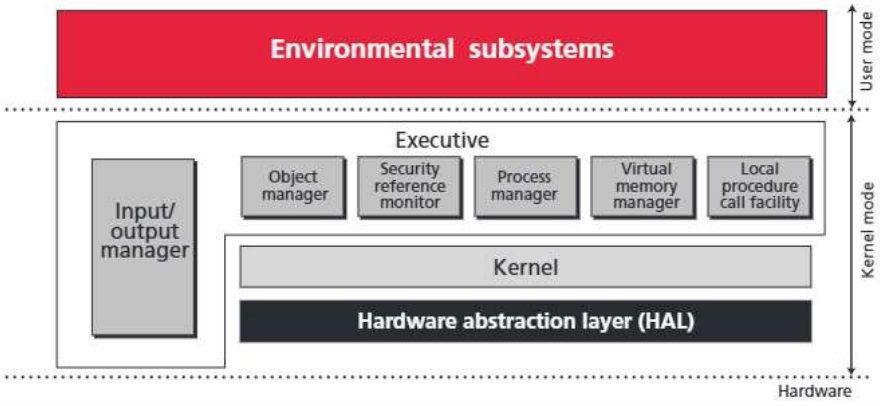
Performance

Windows was designed to have a fast response time to applications that run on top of the operating system.

Architecture

Windows uses a layered architecture, as shown in Figure 7.21.

Figure 7.21 *The architecture of Windows*



HAL

The **hardware abstraction layer (HAL)** hides hardware differences from the upper layers.

Kernel

The kernel is the heart of the operating system. It is an object-oriented piece of software that sees any entity as an object.

Executive

The Windows executive provides services for the whole operating system. It is made up of six subsystems: object manager, security reference monitor, process manager, virtual memory manager, local procedure call facility, and the I/O manager. Most of these subsystems are familiar from our previous discussions of operating subsystems. Some subsystems, like the object manager, are added to Windows because of its object-oriented nature. The executive runs in kernel (privileged) mode.

Environmental subsystems

These are subsystems designed to allow Windows to run application programs designed for Windows, for other operating systems, or for earlier versions of Windows. The native subsystem that runs applications designed for Windows is called Win32. The environment subsystems run in the user mode (a non-privileged mode).

7.5 END-CHAPTER MATERIALS

7.5.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Bic, L. and Shaw, A. *Operating Systems Principles*, Upper Saddle River, NJ: Prentice- Hall, 2003
- McHoes, A. and Flynn, I. *Understanding Operating Systems*, Boston, MA: Course Technology, 2007
- Nutt, G. *Operating Systems: A Modern Perspective*, Reading, MA: Addison-Wesley, 2001
- Silberschatz, A. and Galvin, P. *Operating System Concepts*, New York: Wiley, 2004

7.5.2 Key terms

authentication 205	partitioning 193
batch operating system 189	pico 204
bootstrap 188	portability 205
circular waiting 201	portability process scheduler 198
deadlock 200	process 197
demand paging 194	process manager 191
demand paging and segmentation 195	program 196
demand segmentation 195	queue 199
device manager 202	ready state 197
distributed system 190	real-time system 190

(Continued)

emacs 204	reliability 205
frame 194	resource holding 201
graphical user interface (GUI) 191	running state 197
hardware abstraction layer (HAL) 206	scheduler 198
hold state 197	scheduling 190
job 196	shell 191
job scheduler 198	single-user operating system 190
kernel 204	software 188
Linux 205	starvation 201
memory management 191	state diagram 197
Microsoft Disk Operating System (MS-DOS) 205	terminated state 198
monoprogramming 191	time sharing 190
multiprogramming 192	UNIX 203
mutual exclusion 201	user interface 191
no preemption 201	utility 204
operating system 188	vi 204
page 194	virtual memory 196
paging 194	waiting state 197
parallel system 190	Windows 205

7.5.3 Summary

- An operating system is an interface between the hardware of a computer and the user that facilitates the execution of programs and access to hardware and software resources. Two major design goals of an operating system are efficient use of hardware and ease of use of resources.
- Operating systems have gone through a long history of evolution: batch systems, time-sharing systems, personal systems, parallel systems, and distributed systems. A modern operating system has at least four functional areas: memory manager, process manager, device manager, and file manager. An operating system also provides a user interface.
- The first responsibility of a modern computer system is memory management. Memory allocation must be controlled by the operating system. Memory management techniques can be divided into two categories: monoprogramming and multiprogramming. In monoprogramming, most of the memory capacity is dedicated to one single program. In multiprogramming, more than one program can be in memory at the same time.

5/1/25, 9:57 PM study viewer

- The second responsibility of an operating system is process management. A process is a program in execution. The process manager uses schedulers and queues to manage processes. Process management involves synchronizing different processes with different resources. This may potentially create resource deadlock or starvation. Deadlock occurs when the operating system does not put resource restrictions on processes: starvation can happen when the operating system puts too many resource restrictions on a process.
- The third responsibility of an operating system is device or input/output management.
- The fourth responsibility of an operating system is file management. An operating system uses a file manager to control access to files. Access is permitted only by processes or users that are allowed access to specific files, and the type of access can vary.
- Two common operating systems with some similarities are UNIX and Linux. UNIX is a multiuser, multiprocessing, portable operating system made up from four parts: the kernel, the shell, a standard set of utilities, and application programs. Linux has three components: a kernel, a system utilities, and a system library.
- A popular family of operating systems from Microsoft is referred to as Windows. Windows is an object-oriented, multi-layer operating system. It uses several layers, including a hardware abstract layer (HAL), executive layer, and environment subsystem layer.

7.6 PRACTICE SET

7.6.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

7.6.2 Review questions

Q7-1. What is the difference between an application program and an operating system?

Q7-2. What are the components of an operating system?

Q7-3. What is the difference between monoprogramming and multiprogramming?

Q7-4. How is paging different from partitioning?

Q7-5. How is demand paging more efficient than regular paging?

Q7-6. How is a program related to a job? How is a job related to a process? How is a program related to a process?

Q7-7. Where does a program reside? Where does a job reside? Where does a process reside?

Q7-8. What is the difference between a job scheduler and a process scheduler?

https://studylib.net/doc/26059254/foundations-computer-science-4th 229/706

Q7-9. Why does an operating system need queues?

Q7-10. How does deadlock differ from starvation?

7.6.3 Problems

P7-1. A computer has a monoprogramming operating system. If the size of memory is 64 MB and the memory-resident part of the operating system needs 4 MB, what is the maximum size of a program that can be run by this computer?

P7-2. Redo Exercise 7-1 if the operating system automatically allocates 10 MB of memory to data.

P7-3. A monoprogramming operating system runs programs that on average need 10 microseconds access to the CPU and 70 microseconds access to the I/O devices. What percentage of time is the CPU idle?

P7-4. A multiprogramming operating system uses an apportioning scheme and divides the 60 MB of available memory into four partitions of 10 MB, 12 MB, 18 MB, and 20 MB. The first program to be run needs 17 MB and occupies the third partition. The second program needs 8 MB and occupies the first partition. The third program needs 10.5 MB and occupies the second partition. Finally, the fourth program needs 20 MB and occupies the fourth partition. What is the total memory used? What is the total memory wasted? What percentage of memory is wasted?

P7-5. Redo Problem P7-4 if all programs need 10 MB of memory.

P7-6. A multiprogramming operating system uses paging. The available memory is 60 MB divided into 15 frames, each of 4 MB. The first program needs 13 MB. The second program needs 12 MB. The third program needs 27 MB.

a. How many frames are used by the first program?
b. How many frames are used by the second program?
c. How many frames are used by the third program?
d. How many frames are unused?
e. What is the total memory wasted?
f. What percentage of memory is wasted?

P7-7. An operating system uses virtual memory but requires the whole program to be in physical memory during execution (no paging or segmentation). The size of physical memory is 100 MB. The size of virtual memory is 1 GB. How many programs of size 10 MB can be run concurrently by this operating system? How many of them can be in memory at any time? How many of them must be on disk?

P7-8. What is the status of a process in each of the following situations?

a. The process is using the CPU.
b. The process has finished printing and needs the attention of the CPU again.
c. The process has been stopped because its time slot is over.
d. The process is reading data from the keyboard.
e. The process is printing data.

P7-9. Three processes (A, B, and C) are running concurrently. Process A has acquired File1, but needs File2. Process B has acquired File3, but needs File1. Process C has acquired File2, but needs File3. Draw a diagram for these processes. Is this a deadlock situation?

P7-10. Three processes (A, B, and C) are running concurrently. Process A has acquired File1. Process B has acquired File2, but needs File1. Process C has acquired File3, but needs File2. Draw a diagram for these processes. Is this a deadlock situation? If your answer is 'no', show how the processes can eventually finish their tasks.

CHAPTER 8

Algorithms

In this chapter we introduce the concept of algorithms, step-by-step procedures for solving a problem. We then discuss the tools used to develop algorithms. Finally, we give some examples of common iterative and recursive algorithms.

Objectives

After studying this chapter, the student should be able to:

- Define an algorithm and relate it to problem solving.
- Define three constructs—sequence, selection, and repetition—and describe their use in algorithms.
- Describe UML diagrams and how they can be used when representing algorithms.
- Describe pseudocode and how it can be used when representing algorithms.
- List basic algorithms and their applications.
- Describe the concept of sorting and understand the mechanisms behind three primitive sorting algorithms.
- Describe the concept of searching and understand the mechanisms behind two common searching algorithms.
- Define subalgorithms and their relations to algorithms.
- Distinguish between iterative and recursive algorithms.

8.1 CONCEPT

In this section we informally define an **algorithm** and elaborate on the concept using an example.

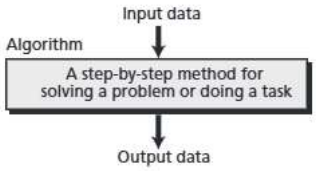
8.1.1 Informal definition

An informal definition of an algorithm is:

Algorithm: a step-by-step method for solving a problem or doing a task.

In this definition, an algorithm is independent of the computer system. More specifically, we should also note that the algorithm accepts **input data** and creates **output data** (Figure 8.1).

Figure 8.1 *Informal definition of an algorithm used in a computer*



8.1.2 Example

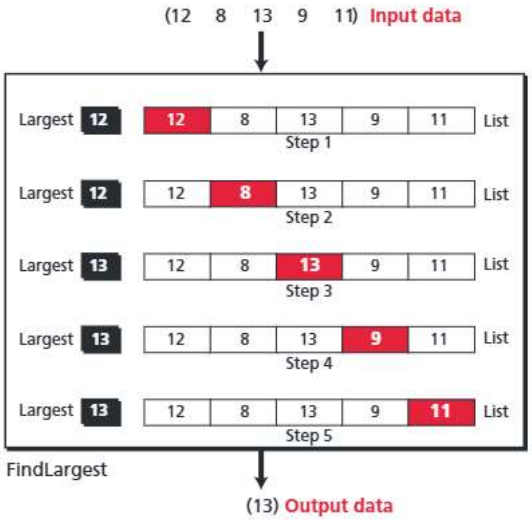
Let us elaborate on this simple definition with an example. We want to develop an algorithm for finding the largest integer among a list of positive integers. The algorithm should find the largest integer among a list of any size (for example 5, 1000, 10000, 1000000). The algorithm should be general and not depend on the number of integers.

It is obvious that finding the largest integer among many integers is a task that cannot be done in one step, either by a human or a computer. The algorithm needs to test each integer one by one.

To solve this problem, we need an intuitive approach. First use a small number of integers (for example, five), then extend the solution to any number of integers. Our solution for five integers follows the same principles and restrictions for one thousand or one million integers. Assume, even for a five-integer case, that the algorithm handles the integers one by one. It looks at the first integer without knowing the values of the remaining integers. After it handles the first one, it looks at the second integer, and so on. Figure 8.2 shows one way to solve this problem.

We call the algorithm FindLargest. Each algorithm has a name to distinguish it from other algorithms. The algorithm receives a list of five integers as input and gives the largest integer as output.

Figure 8.2 *Finding the largest integer among five integers*



Input

The algorithm accepts the list of five integers as input.

Processing

The algorithm uses the following five steps to find the largest integer:

Step 1

In this step, the algorithm inspects the first integer (12). Since it does not know the values of other integers, it decides that the largest integer (so far) is the first integer. The algorithm defines a data item, called Largest, and sets its value to the first integer (12).

Step 2

The largest integer so far is 12, but the new integer may change the situation. The algorithm makes a comparison between the value of Largest (12) and the value of the second integer (8). It finds that Largest is larger than the second integer, which means that Largest is still holding the largest integer. There is no need to change the value of Largest.

Step 3

The largest integer so far is 12, but the new integer (13) is larger than Largest. This means than the value of Largest is no longer valid. The value of Largest should be replaced by the third integer (13). The algorithm changes the value of Largest to 13 and moves to the next step.

Step 4

Nothing is changed in this step because Largest is larger than the fourth integer (9).

Step 5

Again nothing is changed because Largest is larger than the fifth integer (11).

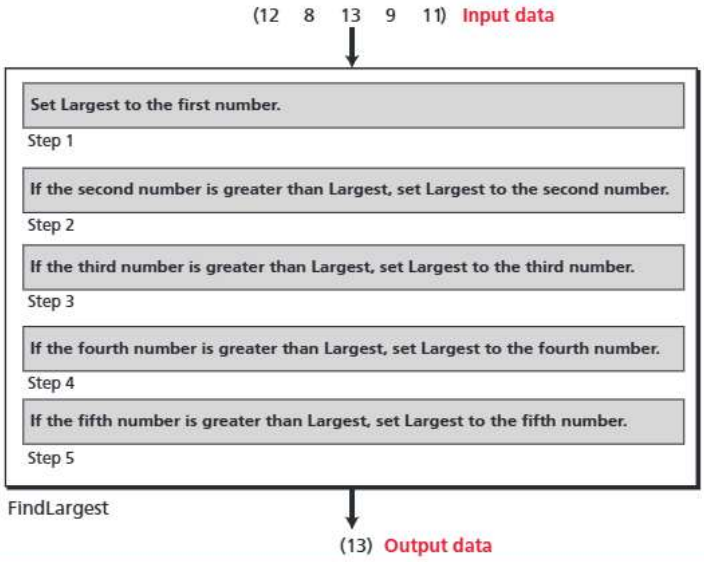
Output

Because there are no more integers to be processed, the algorithm outputs the value of Largest, which is 13.

8.1.3 Defining actions

Figure 8.2 does not show what should be done in each step. We can modify the figure to show more details. For example, in step 1, set Largest to the value of the first integer. In steps 2 to 5, however, additional actions are needed to compare the value of Largest with the current integer being processed. If the current integer is larger than Largest, set the value of Largest to the current integer (Figure 8.3).

Figure 8.3 *Defining actions in FindLargest algorithm*



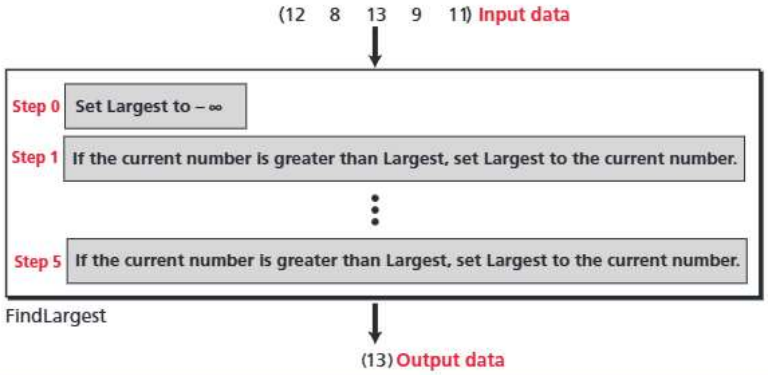
8.1.4 Refinement

This algorithm needs refinement to be acceptable to the programming community. There are two problems. First, the action in the first step is different than those for the other steps. Second, the wording is not the same in steps 2 to 5. We can easily redefine the algorithm to remove these two inconveniences by changing the wording in steps 2 to 5 to 'If the current integer is greater than Largest, set Largest to the current integer'. The reason that the first step is different than the other steps is because Largest is not initialized.

If we initialize Largest to –∞ (minus infinity), then the first step can be the same as the other steps, so we add a new step, calling it step 0 to show that it should be done before processing any integers.

Figure 8.4 shows the result of this refinement. Note that we do not have to show all the steps, because they are now the same.

Figure 8.4 *FindLargest refined*

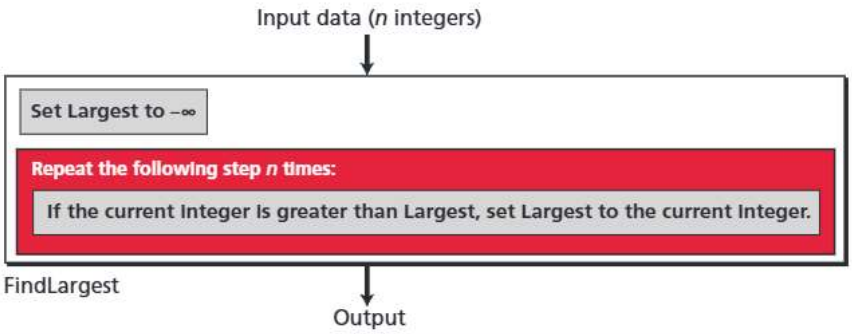


8.1.5 Generalization

Is it possible to generalize the algorithm? We want to find the largest of n positive integers, where n can be 1000, 1 000 000, or more. Of course, we can follow Figure 8.4 and repeat each step. But if we change the algorithm to a program, then we need to actually type the actions for n steps!

There is a better way to do this. We can tell the computer to repeat the steps n times. We now include this feature in our pictorial algorithm (Figure 8.5).

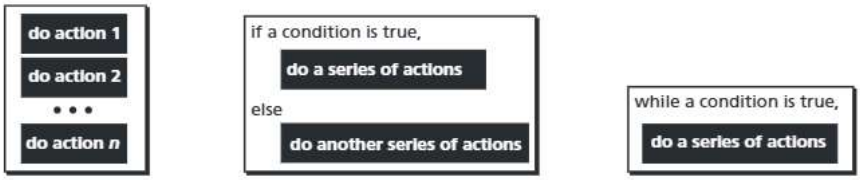
Figure 8.5 *Generalization of FindLargest*



8.2 THREE CONSTRUCTS

Computer scientists have defined three constructs for a structured program or algorithm. The idea is that a program must be made of a combination of only these three constructs: *sequence*, *decision*, and *repetition* (Figure 8.6). It has been proven there is no need for any other constructs. Using only these constructs makes a program or an algorithm easy to understand, debug, or change.

Figure 8.6 *Three constructs*



a. Sequence b. Decision c. Repetition

8.2.1 Sequence

The first construct is called the **sequence**. An algorithm, and eventually a program, is a sequence of instructions, which can be a simple instruction or either of the other two constructs.

8.2.2 Decision

Some problems cannot be solved with only a sequence of simple instructions. Sometimes we need to test a condition. If the result of testing is true, we follow a sequence of instructions: if it is false, we follow a different sequence of instructions. This is called the **decision (selection)** construct.

8.2.3 Repetition

In some problems, the same sequence of instructions must be repeated. We handle this with the **repetition** or **loop** construct. Finding the largest integer among a set of integers can use a construct of this kind.

8.3 ALGORITHM REPRESENTATION

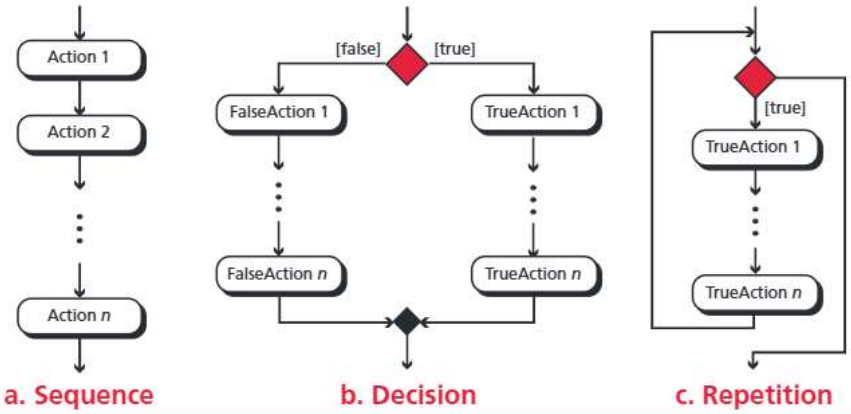
So far, we have used figures to convey the concept of an algorithm. During the last few decades, tools have been designed for this purpose. Two of these tools, UML and pseudocode, are presented here.

8.3.1 UML

Unified Modeling Language (UML) is a pictorial representation of an algorithm. It hides all the details of an algorithm in an attempt to give the 'big picture' and to show how the algorithm flows from beginning to end.

UML is covered in detail in Appendix B. Here we show only how the three constructs are represented using UML (Figure 8.7). Note that UML allows us a lot of flexibility, as shown in Appendix B. For example, the decision construct can be simplified if there are no actions on the false part.

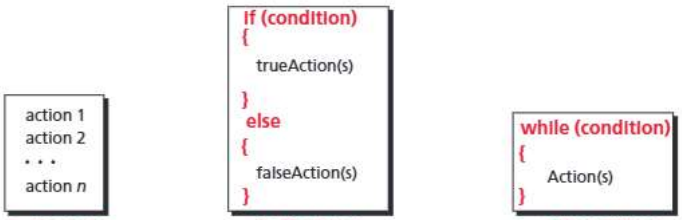
Figure 8.7 *UML for three constructs*



8.3.2 Pseudocode

Pseudocode is an English-language-like representation of an algorithm. There is no standard for pseudocode—some people use a lot of detail, others use less. Some use a code that is close to English, while others use a syntax like the Pascal programming language. Pseudocode is covered in detail in Appendix C. Here we show only how the three constructs can be represented by pseudocode (Figure 8.8).

Figure 8.8 *Pseudocode for three constructs*



Example 8.1

Write an algorithm in pseudocode that finds the sum of two integers.

Solution

This is a simple problem that can be solved using only the sequence construct. Note also that we name the algorithm, define the input to the algorithm and, at the end, we use a return instruction to return the sum (Algorithm 8.1).

Algorithm 8.1 Calculating the sum of two integers

```
Algorithm: SumOfTwo (first, second)
Purpose: Find the sum of two integers
Pre: Given: two integers (first and second)
Post: None
Return: The sum value
{
    sum ← first + second
    return sum
}
```

Example 8.2

Write an algorithm to change a numeric grade to a pass/no pass grade.

Solution

This problem cannot be solved with only the sequence construct. We also need the decision construct. The computer is given an integer between 0 and 100. It returns 'pass' if the integer is greater than or equal to 70, and returns 'no pass' if the integer is less than 70. Algorithm 8.2 shows the pseudocode for this algorithm.

Algorithm 8.2 Assigning pass/no pass grade

```
Algorithm: Pass/NoPass (score)
Purpose: Creates a pass/no pass grade given the score
Pre: Given: the score to be changed to grade
Post: None
Return: The grade
{
    if (score ≥ 70)
      grade ← "pass"
    else
      grade ← "nopass"
    return grade
}
```

Example 8.3

Write an algorithm to change a numeric grade (integer) to a letter grade.

Solution

This problem needs more than one decision. The pseudocode in Algorithm 8.3 shows one way to solve the problem—not the best one, but an easy one to understand. Again, an integer is given between 0 and 100, and we want to change it to a letter grade (A, B, C, D, or F).

Algorithm 8.3 Assigning a letter grade

```
Algorithm: LetterGrade (score)
Purpose: Find the letter grade corresponding to the given score
Pre: Given: a numeric score
Post: None
Return: A letter grade
{
    if (100 ≥ score ≥ 90)
      grade ← 'A'
    if (89 ≥ score ≥ 80)
      grade ← 'B'
    if (79 ≥ score ≥ 70)
      grade ← 'C'
    if (69 ≥ score ≥ 60)
      grade ← 'D'
    if (59 ≥ score ≥ 0)
      grade ← 'F'
    return grade
}
```

Note that the decision constructs do not need an *else* section, because we do nothing if the condition is false.

Example 8.4

Write an algorithm to find the largest of a set of integers. We do not know the number of integers.

Solution

We use the concept in Figure 8.5 on page 217 to write an algorithm for this problem (see Algorithm 8.4).

Algorithm 8.4 Finding the largest integer among a set of integers

```
Algorithm: FindLargest (list)
Purpose: Find the largest integer among a set of integers
Pre: Given: the set of integers
Post: None
Return: The largest integer
{
    largest ← −∞
    while (more integers to check)
    {
        current ← next integer
        if (current > largest)
            largest ← current
    }
    return largest
}
```

Example 8.5

Write an algorithm to find the smallest of the first 1000 integers in a set of integers.

Solution

Here we need a counter to count the number of integers. We initialize the counter to 1 and increment it in each repetition. When the counter is greater than 1000, we exit from the loop (see Algorithm 8.5). Note that there are more than 1000 integers in the list, but we want to find the smallest among the first 1000.

https://sanet.st/blogs/polatebooks/

Algorithm 8.5 Find the smallest integers among 1000 integers

```
Algorithm: FindSmallest (list)
Purpose: Find and return the smallest integer among the first 1000 integers
Pre: Given the set of integers with more than 1000 integers
Post: None
Return: The smallest integer
{
    smallest ← +∞
    counter ← 1
    while (counter ≤ 1000)
    {
        current ← next integer
        if (current < smallest)
            smallest ← current
        counter ← counter + 1
    }
    return smallest
}
```

8.4 A MORE FORMAL DEFINITION

Now that we have discussed the concept of an algorithm and shown its representation, here is a more formal definition. Let us elaborate on this definition.

Algorithm: An ordered set of unambiguous steps that produces a result and terminates in a finite time.

8.4.1 Well-Defined

An algorithm must be a well-defined, ordered set of instructions.

8.4.2 Unambiguous steps

Each step in an algorithm must be clearly and unambiguously defined. If one step is to *add two integers*, we must define both 'integers' as well as the 'add' operation: we cannot for example use the same symbol to mean addition in one place and multiplication somewhere else.

8.4.3 Produce a result

An algorithm must produce a result, otherwise it is useless. The result can be data returned to the calling algorithm, or some other effect (for example, printing).

8.4.4 Terminate in a finite time

An algorithm must terminate (halt). If it does not (that is, it has an infinite loop), we have not created an algorithm. In Chapter 17 we will discuss *solvable* and *unsolvable* problems, and we will see that a solvable problem has a solution in the form of an algorithm that terminates.

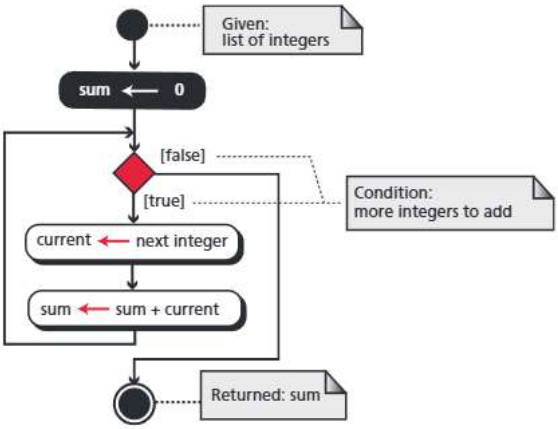
8.5 BASIC ALGORITHMS

Several algorithms are used in computer science so prevalently that they are considered 'basic'. We discuss the most common here. This discussion is very general: implementation depends on the language.

8.5.1 Summation

One commonly used algorithm in computer science is **summation**. We can add two or three integers very easily, but how can we add many integers? The solution is simple: we use the add operator in a loop (Figure 8.9).

Figure 8.9 *Summation algorithm*



A summation algorithm has three logical parts:

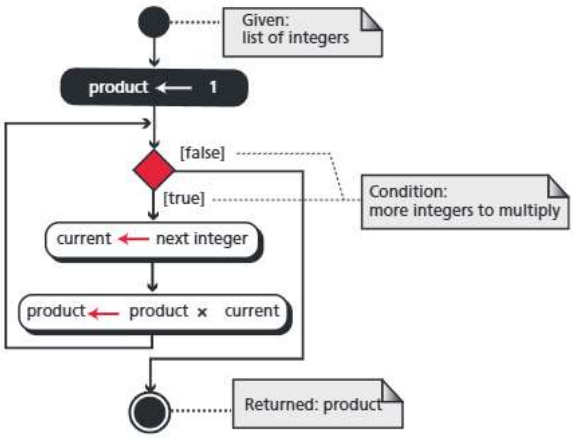
1. Initialization of the sum at the beginning.
2. The loop, which in each iteration adds a new integer to the sum.
3. Return of the result after exiting from the loop.

8.5.2 Product

Another common algorithm is finding the **product** of a list of integers. The solution is simple: use the multiplication operator in a loop (Figure 8.10). A product algorithm has three logical parts:

1. Initialization of the product at the beginning.
2. The loop, which in each iteration multiplies a new integer with the product.
3. Return of the result after exiting from the loop.

Figure 8.10 *Product algorithm*



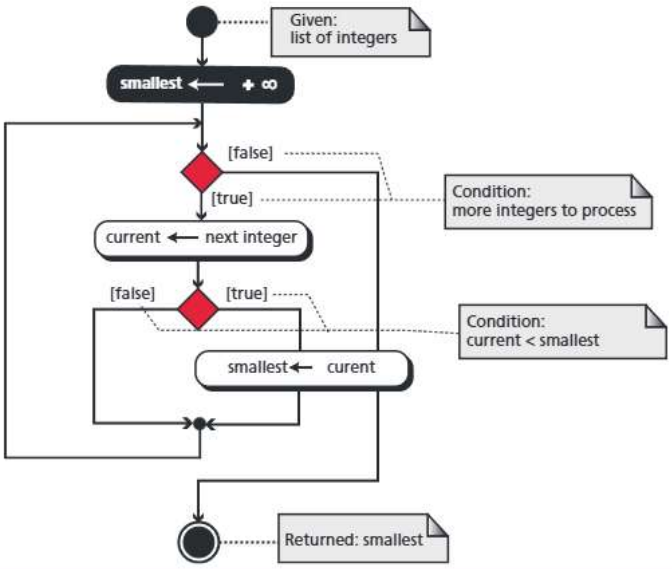
For example, the preceding algorithm can be used to calculate x^n using a minor modification—this is left as an exercise. As another example, the same algorithm can be used to calculate the factorial of an integer, which is discussed later in the chapter.

8.5.3 Smallest and largest

We discussed the algorithm for finding the largest among a list of integers at the beginning of this chapter. The idea was to write a decision construct to find the larger of two integers. If we put this construct in a loop, we can find the largest of a list of integers.

Finding the smallest integer among a list of integers is similar, with two minor differences. First, we use a decision construct to find the smaller of two integers. Second, we initialize with a very large integer instead of a very small one. Figure 8.11 shows the algorithm to find the smallest among a list of integers. The figure to find the largest is similar and left as an exercise.

Figure 8.11 *Finding the smallest data item*



8.5.4 Sorting

One of the most common applications in computer science is **sorting**, which is the process by which data is arranged according to its values. People are surrounded by data. If the data was not ordered, it would take hours and hours to find a single piece of information. Imagine the difficulty of finding someone's telephone number in a telephone book that is not ordered.

In this section, we introduce three sorting algorithms: selection sort, bubble sort, and insertion sort. These three sorting algorithms are the foundation of faster sorting algorithms used in computer science today.

Selection sorts

In a **selection sort**, the list to be sorted is divided into two sublists—sorted and unsorted—which are separated by an imaginary wall. We find the smallest element from the unsorted sublist and swap it with the element at the beginning of the unsorted sublist. After each selection and swap, the imaginary wall between the two sublists moves one element ahead, increasing the number of sorted elements and decreasing the number of unsorted ones. Each time we move one element from the unsorted sublist to the sorted sublist, we have completed a **sort pass**. A list of n elements requires $n - 1$ passes to completely rearrange the data. Selection sort is presented graphically in Figure 8.12.

Figure 8.12 *Selection sort*

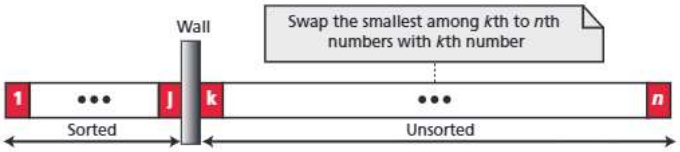
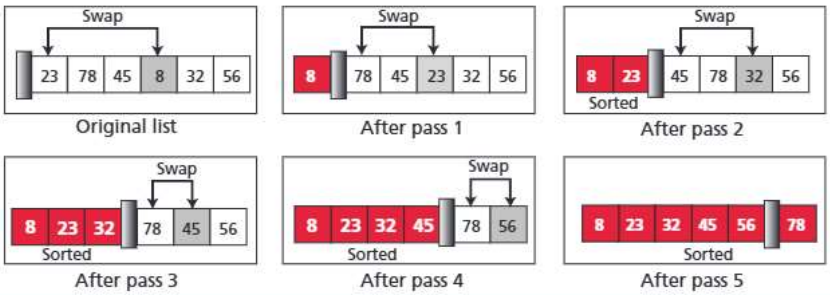


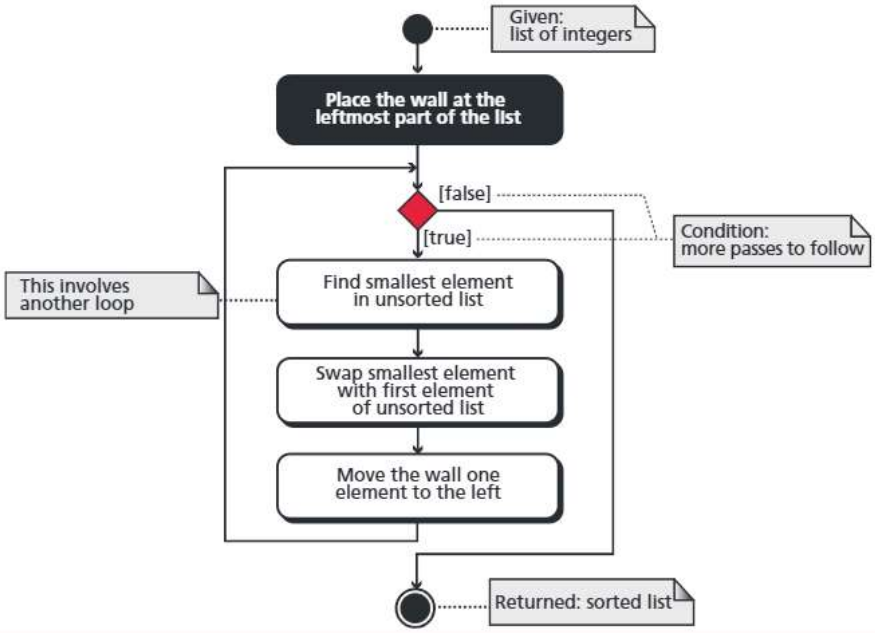
Figure 8.13 traces a set of six integers as we sort them.

Figure 8.13 *Example of selection sort*



The figure shows how the wall between the sorted and unsorted sublists moves in each pass. As we study the figure, we will see that the list is sorted after five passes, which is one less than the number of elements in the list. Thus, if we use a loop to control the sorting, the loop will have one less iteration than the number of elements to be sorted.

Figure 8.14 *Selection sort algorithm*



A selection sort algorithm

The algorithm uses two loops, one inside the other. The outer loop is iterated for each pass: the inner loop finds the smallest element in the unsorted list. Figure 8.14 shows the UML for the selection sort algorithm. The inner loop is not explicitly shown in the figure, but the first instruction in the loop is itself a loop. We leave the demonstration of the loop as an exercise.

Bubble sorts

In the **bubble sort** method, the list to be sorted is also divided into two sublists—sorted and unsorted. The smallest element is *bubbled up* from the unsorted sublist and moved to the sorted sublist. After the smallest element has been moved to the sorted list, the wall moves one element ahead, increasing the number of sorted elements and decreasing the number of unsorted ones. Each time an element moves from the unsorted sublist to the sorted sublist, one sort pass is completed (Figure 8.15). Given a list of n elements, bubble sort requires up to $n - 1$ passes to sort the data.

Figure 8.15 *Bubble sort*

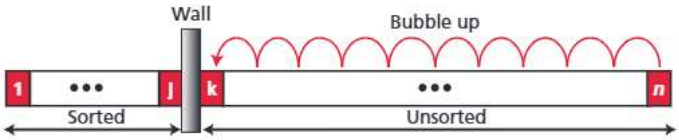
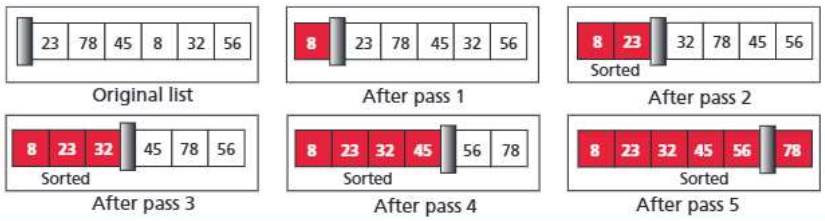


Figure 8.16 shows how the wall moves one element in each pass. Looking at the first pass, we start with 56 and compare it to 32. Since 56 is not less than 32, it is not moved, and we step down one element. No exchanges take place until we compare 45 to 8. Since 8 is less than 45, the two elements are exchanged, and we step down one element. Because 8 was moved down, it is now compared to 78, and these two elements are exchanged. Finally, 8 is compared to 23 and exchanged. This series of exchanges places 8 in the first location, and the wall is moved up one position. The algorithm gets its name from the way in which numbers—in this example, 8—appear to move to the start, or top, of the list in the same way that bubbles rise through water.

Note that we have stopped before the wall moves to the end of the list, because the list is already sorted. We can always include an indicator in the algorithm to stop the passes if no number exchanges occur in a pass. This fact can be used to improve the efficiency of the bubble sort by reducing the number of steps.

Figure 8.16 *Example of bubble sort*



The bubble sort was originally written to 'bubble down' the highest element in the list. From an efficiency point of view, it makes no difference whether high elements are moved or low elements are moved up. From a consistency point of view, however, it makes comparisons between the sort algorithms easier if all of them work in the same manner. For that reason, we have chosen to move the lowest value up in each pass.

A bubble sort algorithm

Bubble sorts also use two loops, one inside the other. The outer loop is iterated for each pass, while each iteration of the inner loop tries to bubble one element up to the top (left). We leave the UML and pseudocode as exercises.

Insertion sorts

The **insertion sort** algorithm is one of the most common sorting techniques, and it is often used by card players. Each card a player picks up is inserted into the proper place in their hand of cards to maintain a particular sequence. (Card sorting is an example of a sort that uses two criteria for sorting: suit and rank.)

Figure 8.17 *Insertion sort*

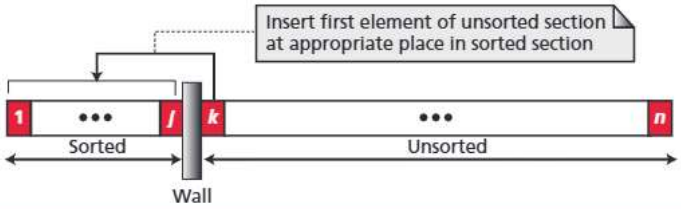
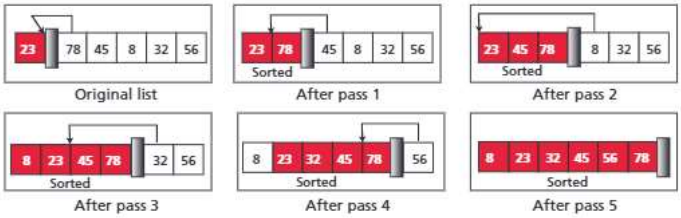


Figure 8.18 *Example of insertion sort*



In an insertion sort, as in the other two sorting algorithms discussed above, the list is divided into two parts—sorted and unsorted. In each pass, the first element of the unsorted sublist is transferred to the sorted sublist and inserted at the appropriate place (Figure 8.17). Note that a list of n elements will take $n - 1$ passes to sort the data.

Figure 8.18 traces an insertion sort through our list of six numbers. The wall moves with each pass as an element is removed from the unsorted sublist and inserted into the sorted sublist.

Insertion sort algorithm

The design of insertion sort follows the same pattern seen in both selection sort and bubble sort. The outer loop is iterated for each pass, and the inner loop finds the position of insertion. We leave the UML diagram and pseudocode as exercises.

Other sorting algorithms

The three sorting algorithms discussed here are the least efficient sorting algorithms, and should not be used if the list to be sorted has more than a few hundred elements. We have discussed these algorithms here for educational purposes, but they are not practical. There are however several reasons for discussing these sorting algorithms in an introductory book:

- They are the simplest algorithms to understand and analyze.
- They are the foundation of more efficient algorithms such as *quicksort*, *heap sort*, *Shell sort*, *bucket sort*, *merge sort*, *radix sort*, and so on.

Most such advanced sorting algorithms are discussed in books on data structures.

We may ask why there are so many sorting algorithms. The reason lies in the type of data that needs to be sorted. One algorithm may be more efficient for a list that is partially sorted, whereas another algorithm may be more efficient for a list that is completely unsorted. To decide which algorithm is best suited for a particular application, a measurement called the complexity of algorithms is needed. We discuss this issue in Chapter 17, but a thorough understanding requires additional courses in programming and data structures.

8.5.5 Searching

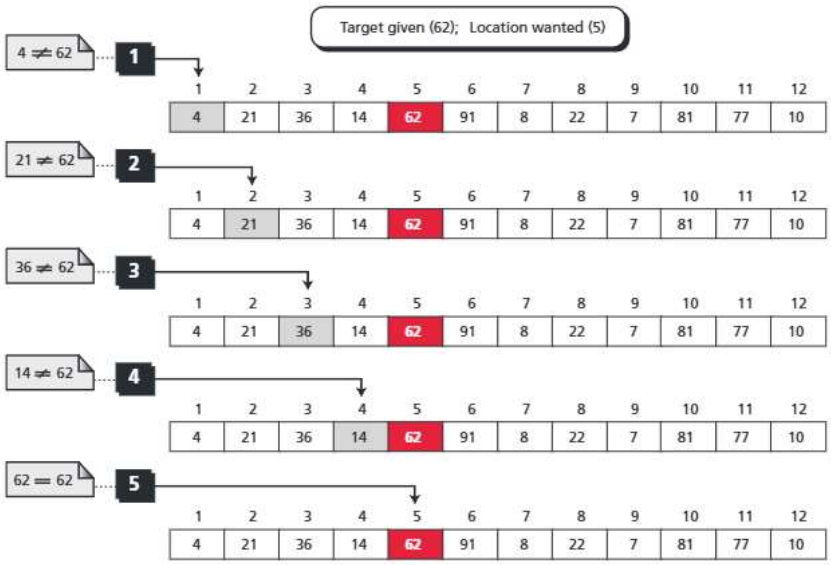
Another common algorithm in computer science is **searching**, which is the process of finding the location of a target among a list of objects. In the case of a list, searching means that given a value, we want to find the location of the first element in the list that contains that value. There are two basic searches for lists: **sequential search** and **binary search**. **Sequential search** can be used to locate an item in any list, whereas binary search requires the list first to be sorted.

Sequential search

Sequential search is used if the list to be searched is not ordered. Generally, we use this technique only for small lists, or lists that are not searched often. In other cases, the best approach is to first sort the list and then search it using the binary search discussed later.

In a sequential search, we start searching for the target from the beginning of the list. We continue until we either find the target or reach the end of the list. Figure 8.19 traces the steps to find the value 62. The search algorithm needs to be designed so that the search stops when we find the target or when we reach the end of the list.

Figure 8.19 *An example of a sequential search*

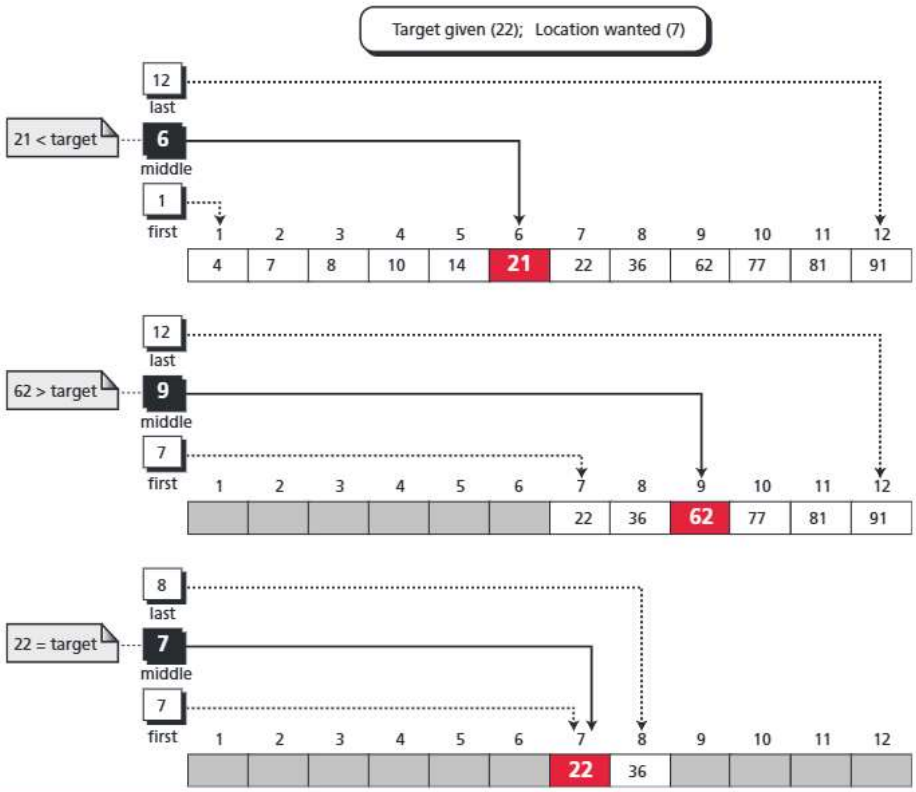


Binary search

The sequential search algorithm is very slow. If we have a list of a million elements, we must do a million comparisons in the worst case. If the list is not sorted, this is the only solution. If the list *is* sorted, however, we can use a more efficient algorithm called **binary search**. Generally speaking, programmers use a binary search when a list is large.

Please see the following page for an example of a binary search.

Figure 8.20 *Example of a binary search*



A binary search starts by testing the data in the element at the middle of the list. This determines whether the target is in the first half or the second half of the list. If it is in the first half, there is no need to further check the second half. If it is in the second half, there is no need to further check the first half. In other words, we eliminate half the list from further consideration.

We repeat this process until we either find the target or satisfy ourselves that it is not in the list. Figure 8.20 shows how to find the target, 22, in a list of 12 numbers using three references: first, mid, and last.

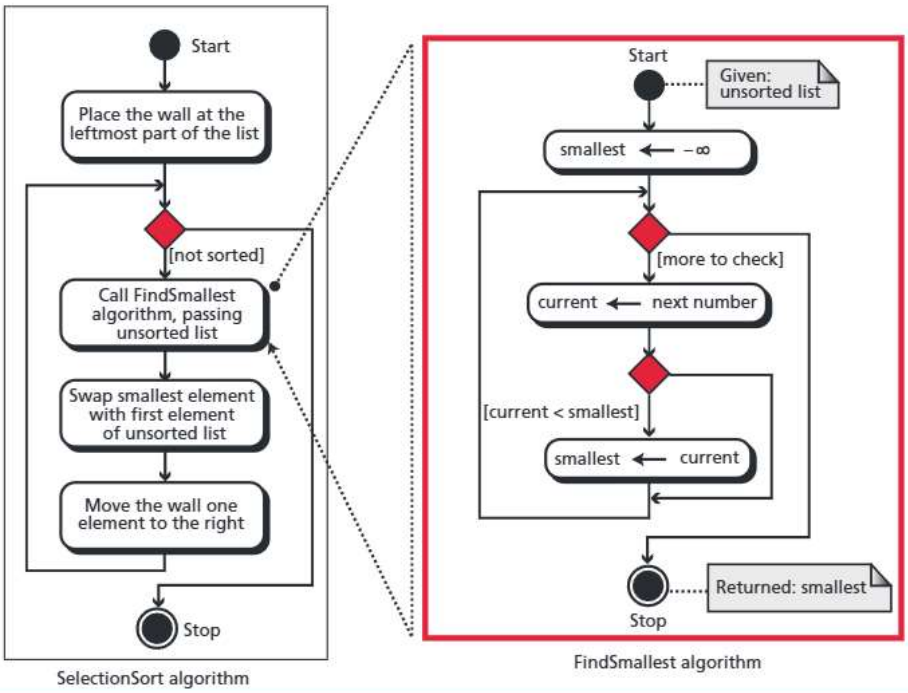
1. At the beginning, *first* shows 1 and *last* shows 12. Let *mid* show the middle position, (1 + 12) / 2, or 6 if truncated to an integer. Now compare the target (22) with data at position 6 (21). The target is greater than this value, so we ignore the first half of the list.
2. Move *first* after *mid*, to position 7. Let *mid* show the middle of the second half, (7 + 12) / 2, or 9. Now compare the target (22) with data at position 9 (62). The target is smaller than this value, so we ignore the integers from this value (62) to the end.
3. Move last before *mid* to position 8. Recalculate *mid* again, (8 + 7) / 2, or 7. Compare the target (22) with the value at this position (22). We have found the target and can quit.

The algorithm for binary search needs to be designed to find the target or to stop if the target is not in the list. It can be shown that if the target is not found in the list, the value of *last* becomes smaller than the value of *first*, an abnormal condition that helps us to know when to come out of the loop.

8.6 SUBALGORITHMS

The three programming constructs described in Section 8.2 allow us to create an algorithm for any solvable problem. The principles of structured programming, however, require that an algorithm be broken into small units called **subalgorithms**. Each subalgorithm is in turn divided into smaller subalgorithms. A good example is the algorithm for the selection sort in Figure 8.14. Finding the smallest integer in the unsorted sublist is an independent task that can be considered as a subalgorithm. (Figure 8.21). The algorithm SelectionSort calls the subalgorithm FindSmallest in each iteration.

Figure 8.21 *Concept of a subalgorithm*



Using subalgorithms has at least two advantages:

- It is more understandable. Looking at the SelectionSort algorithm, we can immediately see that a task (finding the smallest integer among the unsorted list) is repeated.
- A subalgorithm can be called many times in different parts of the main algorithm without being rewritten.

8.6.1 Structure chart

Another tool programmers use is the **structure chart**. A structure chart is a high-level design tool that shows the relationship between algorithms and subalgorithms. It is used mainly at the design level rather than at the programming level. We briefly discuss the structure chart in Appendix D.

8.7 RECURSION

In general, there are two approaches to writing algorithms for solving a problem. One uses iteration, the other uses *recursion*. **Recursion** is a process in which an algorithm calls itself.

8.7.1 Iterative definition

To study a simple example, consider the calculation of a factorial. The factorial of a integer is the product of the integral values from 1 to the integer. The definition is *iterative* (Figure 8.22). An algorithm is iterative whenever the definition does not involve the algorithm itself.

Figure 8.22 *Iterative definition of factorial*

$$\text{Factorial } (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1) \times (n-2) \cdots 3 \times 2 \times 1 & \text{if } n > 0 \end{cases}$$

8.7.2 Recursive definition

An algorithm is defined recursively whenever the algorithm appears within the definition itself. For example, the factorial function can be defined recursively as shown in Figure 8.23.

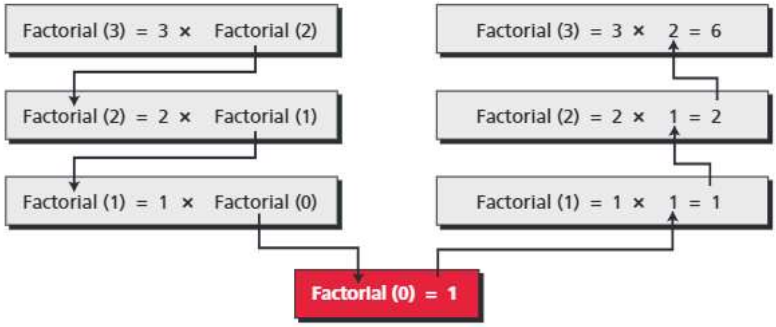
Figure 8.23 *Recursive definition of factorial*

$$\text{Factorial } (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times \text{Factorial } (n-1) & \text{if } n > 0 \end{cases}$$

The decomposition of factorial (3), using recursion, is shown in Figure 8.24. If we study the figure carefully, we will note that the recursive solution for a problem involves a

two-way journey. First we decompose the problem from top to bottom, and then we solve it from bottom to top.

Figure 8.24 *Tracing the recursive solution to the factorial problem*



Judging by this example, it looks as if the recursive calculation is much longer and more difficult. So why would we want to use the recursive method? Although the recursive calculation looks more difficult when using paper and pencil, it is often a much easier and more elegant solution when using computers. Additionally, it offers a conceptual simplicity to the creator and the reader.

Iterative solution

Let us write an algorithm to solve the factorial problem iteratively. This solution usually involves a loop such as seen in Algorithm 8.6.

Algorithm 8.6 Iteration solution to factorial problem

```
Algorithm: Factorial (n)
Purpose: Find the factorial of a number using a loop
Pre: Given: n
Post: None
Return: n!
{
    F ← 1
    i ← 1
    while (i ≤ n)
    {
        F ← F × i
        i ← i + 1
    }
    return F
}
```

Recursive solution

The recursive solution to factorials is shown in Algorithm 8.7. It does not need a loop, as the recursion concept itself involves repetition. In the recursive version, we let the algorithm Factorial call itself.

Algorithm 8.7 A recursive solution of the factorial problem

```
Algorithm: Factorial (n)
Purpose: Find the factorial of a number using recursion
Pre: Given: n
Post: None
Return: n!
{
      if (n = 0)
        return 1
      else
        return n × Factorial (n – 1)
}
```

8.8 END-CHAPTER MATERIALS

8.8.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Aho, A., Hopcroft, J. and Ullman, J. *The Design and Analysis of Computer Algorithms*, Boston, MA: Addison-Wesley, 1974
- Cormen, T., Leiserson, C. and Rivest, R. *Introduction to Algorithms*, New York: McGraw-Hill, 2003
- Gries, D. *The Science of Programming*, New York: Springer, 1998
- Tardos, E. and Kleinberg, J. *Algorithm Design*, Boston, MA: Addison-Wesley, 2006
- Roberts, E. *Thinking Recursively*, New York: Wiley, 1998

8.8.2 Key terms

algorithm 214	repetition 218
binary search 230	searching 230
bubble sort 228	selection 218

decision 218	selection sort 226
input data 214	sequence 218
insertion sort 229	sequential search 230
loop 218	sorting 226
output data 214	structure chart 234
product 224	subalgorithm 233
pseudocode 219	summation 224
recursion 234	Unified Modeling Language (UML) 219

8.8.3 Summary

- An algorithm can be informally defined as 'a step-by-step method for solving a problem or doing a task'. More formally, an algorithm is defined as 'an ordered set of unambiguous steps that produces a result and terminates in a finite time'.
- Computer scientists have defined three constructs for a structured program or algorithm: *sequence*, *decision* (selection), and *repetition* (loop).
- Several tools have been designed to show an algorithm: UML, pseudocode, and structure charts. UML is a pictorial representation of an algorithm. Pseudocode is an English-language-like representation of an algorithm. A structure chart is a high-level design tool that shows the relationship between algorithms and subalgorithms.
- Several algorithms are used in computer science so prevalently that they are considered basic. We discussed the most common in this chapter: *summation*, *product*, *finding the smallest and largest*, *sorting*, and *searching*.
- One of the most common applications in computer science is sorting, which is the process by which data is arranged according to its value. We introduced three primitive but fundamental, sorting algorithms: *selection sort*, *bubble sort*, and *insertion sort*. These three sorting algorithms are the foundation of the faster sorts used in computer science today.
- Another common algorithm in computer science is searching, which is the process of finding the location of a target among a list of objects. There are two basic searches for lists: *sequential search* and *binary search*. Sequential search can be used to locate an item in any list, whereas binary search requires the list to be sorted.
- The principles of structured programming require that an algorithm be broken into small units called *subalgorithms*. Each subalgorithm is in turn divided into smaller subalgorithms.
- In general, there are two approaches to writing algorithms to solve a problem. One uses *iteration*, the other uses *recursion*. An algorithm is iterative whenever the definition does not involve the algorithm itself. An algorithm is defined recursively whenever the algorithm appears within the definition itself.

8.9 PRACTICE SET

8.9.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

8.9.2 Review questions

Q8-1. What is the formal definition of an algorithm?
Q8-2. Define the three constructs used in structured programming.
Q8-3. How is a UML diagram related to an algorithm?
Q8-4. How is pseudocode related to an algorithm?
Q8-5. What is the purpose of a sorting algorithm?
Q8-6. What are the three basic sorting algorithms discussed in this chapter?
Q8-7. What is the purpose of a searching algorithm?
Q8-8. What are the two basic searching algorithms discussed in this chapter?
Q8-9. Give a definition and an example of an iterative process.
Q8-10. Give a definition and an example of a recursive process.

8.9.3 Problems

P8-1. Using the summation algorithm, make a table to show the value of the sum after each integer in the following list is processed:

20	12	70	81	45	13	81

P8-2. Using the product algorithm, make a table to show the value of the product after each integer in the following list is processed:

2	12	8	11	10	5	20

P8-3. Using the FindLargest algorithm, make a table to show the value of Largest after each integer in the following list is processed:

18	12	8	20	10	32	5

P8-4. Using the FindSmallest algorithm, make a table to show the value of Smallest after each integer in the following list is processed:

18	3	11	8	20	1	2

P8-5. Using the selection sort algorithm, manually sort the following list and show your work in each pass using a table:

14	7	23	31	40	56	78	9	2

P8-6. Using the bubble sort algorithm, manually sort the following list and show your work in each pass using a table:

14	7	23	31	40	56	78	9	2

P8-7. Using the insertion sort algorithm, manually sort the following list and show your work in each pass:

7	23	31	40	56	78	9	2

P8-8. A list contains the following elements. The first two elements have been sorted using the selection sort algorithm. What is the value of the elements in the list after three more passes of the selection sort?

7	8	26	44	13	23	98	57

P8-9. A list contains the following elements. The first two elements have been sorted using the bubble sort algorithm. What is the value of the elements in the list after three more passes of the bubble sort?

7	8	26	44	13	23	57	98

P8-10. A list contains the following elements. The first two elements have been sorted using the insertion sort algorithm. What is the value of the elements in the list after three more passes of the insertion sort?

3	13	7	26	44	23	98	57

P8-11. A list contains the following elements. Using the binary search algorithm, trace the steps followed to find 88. At each step, show the values of *first*, *last*, and *mid*:

8	13	17	26	44	56	88	97

P8-12. A list contains the following elements. Using the binary search algorithm, trace the steps followed to find 20. At each step, show the values of *first*, *last*, and *mid*:

17	26	44	56	88	97

P8-13. Using Figure 8.19 in section 8.5.1 (sequential search) show all the steps to try to find a target of 11 (which is not in the list).

P8-14. Using Figure 8.20 in section 8.5.5 (binary search) show all the steps try to find a target of 17 (which is not in the list).

P8-15. Apply the iterative definition of the Factorial algorithm to show the value of F in each step when finding the value of 6! (6 factorial).

P8-16. Apply the recursive definition of the Factorial algorithm to show the value of Factorial in each step when finding the value of 6!

P8-17. Write a recursive algorithm in pseudocode to find the greatest common divisor (gcd) of two integers using the definition in Figure 8.25. In this definition, the expression 'x mod y' means dividing x by y and using the remainder as the result of the operation.

Figure 8.25 *Problem P8-17*

$$\gcd(x, y) = \begin{cases} x & \text{if } y = 0 \\ \gcd(y, x \bmod y) & \text{otherwise} \end{cases}$$

P8-18. Using the definition of Figure 8.25, find the following:

a. gcd (7, 41)
b. gcd (12, 100)
c. gcd (80, 4)
d. gcd (17, 29)

P8-19. Write a recursive algorithm in pseudocode to find the combination of n objects taken k at a time using the definition in Figure 8.26.

Figure 8.26 *Problem P8-19*

$$C(n, k) = \begin{cases} 1 & \text{if } k = 0 \text{ or } n = k \\ C(n-1, k) + C(n-1, k-1) & \text{if } n > k > 0 \end{cases}$$

P8-20. Using the definition in Figure 8.26, find the following:

a. C (3, 2)
b. C (5, 5)
c. C (2, 7)
d. C (4, 3)

P8-21. The Fibonacci sequence, Fib (n), is used in science and mathematics as shown in Figure 8.27. Write a recursive algorithm in pseudocode to calculate the value of Fib (n).

Figure 8.27 *Problem P8-21*

$$\text{Fib}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2) & \text{if } n > 1 \end{cases}$$

P8-22. Using the definition of Figure 8.27, find the following:

a. Fib(2)
b. Fib(3)
c. Fib(4)
d. Fib(5)

P8-23. Draw a UML diagram for the selection sort algorithm that uses two loops. The nested loop is used to find the smallest element in the unsorted sublist.

P8-24. Draw a UML diagram for the bubble sort algorithm that uses two loops. The nested loop is used to swap adjacent items in the unsorted sublist.

P8-25. Draw a UML diagram for the insertion sort algorithm that uses two loops. The nested loop is used to do the insertion into the sorted sublist.

P8-26. Draw a UML diagram for the bubble sort algorithm that uses a subalgorithm. The subalgorithm bubbles the unsorted sublist.

P8-27. Draw a UML diagram for the insertion sort algorithm that uses a subalgorithm. The subalgorithm is used to do the insertion into the sorted sublist.

P8-28. Write an algorithm in pseudocode for the UML diagram in Figure 8.9 in section 8.5.1 (summation).

P8-29. Write an algorithm in pseudocode for the UML diagram in Figure 8.10 in section 8.5.5 (product).

P8-30. Write an algorithm in pseudocode for the selection sort using two nested loops.

P8-31. Write an algorithm in pseudocode for the selection sort using a subalgorithm to find the smallest integer in the unsorted sublist.

P8-32. Write an algorithm in pseudocode for the bubble sort using two nested loop.

P8-33. Write an algorithm in pseudocode for the bubble sort using a subalgorithm to do bubbling in the unsorted sublist.

P8-34. Write an algorithm in pseudocode for the insertion sort using two nested loop.

P8-35. Write an algorithm in pseudocode for the insertion sort using a subalgorithm to do insertion in the sorted sublist.

P8-36. Write an algorithm in pseudocode for the sequential search algorithm. Include the condition for algorithm termination if the target is found or not found.

P8-37. Write an algorithm in pseudocode for the binary search algorithm. Include the condition for algorithm termination if the target is found or not found.

P8-38. Using the UML diagram for the product algorithm, draw a diagram to calculate the value of x^n, when x and n are two given integers.

P8-39. Write an algorithm in pseudocode to find the value of x^n, when x and n are two given integers.

CHAPTER 9

Programming Languages

In Chapter 8 we discussed algorithms. We showed how we can write algorithms in UML or pseudocode to solve a problem. In this chapter, we examine programming languages that can implement pseudocode or UML descriptions of a solution into a programming language. This chapter is not designed to teach a particular programming language; it is written to compare and contrast different languages.

Objectives

After studying this chapter, the student should be able to:

- Describe the evolution of programming languages from machine language to high-level languages.
- Understand how a program in a high-level language is translated into machine language using an interpreter or a compiler.
- Distinguish between four computer language paradigms.
- Understand the procedural paradigm and the interaction between a program unit and data items in the paradigm.
- Understand the object-oriented paradigm and the interaction between a program unit and objects in this paradigm.
- Define functional paradigm and understand its applications.
- Define a declaration paradigm and understand its applications.
- Define common concepts in procedural and object-oriented languages.

9.1 EVOLUTION

To write a program for a computer, we must use a computer language. A **computer language** is a set of predefined words that are combined into a program according to predefined rules **(syntax)**. Over the years, computer languages have evolved from *machine language* to *high-level languages*.

9.1.1 Machine languages

In the earliest days of computers, the only **programming languages** available were **machine languages**. Each computer had its own machine language, which was made of streams of 0s and 1s. In Chapter 5 we showed that in a primitive hypothetical computer, we need to use 11 lines of code to read two integers, add them, and print the result. These lines of code, when written in machine language, make 11 lines of binary code, each of 16 bits, as shown in Table 9.1.

Table 9.1 Code in machine language to add two integers

Hexadecimal	*Code in machine language*			
$(1FEF)_{16}$	0001	1111	1110	1111
$(240F)_{16}$	0010	0100	0000	1111
$(1FEF)_{16}$	0001	1111	1110	1111
$(241F)_{16}$	0010	0100	0001	1111
$(1040)_{16}$	0001	0000	0100	0000
$(1141)_{16}$	0001	0001	0100	0001
$(3201)_{16}$	0011	0010	0000	0001
$(2422)_{16}$	0010	0100	0010	0010
$(1F42)_{16}$	0001	1111	0100	0010
$(2FFF)_{16}$	0010	1111	1111	1111
$(0000)_{16}$	0000	0000	0000	0000

Machine language is the only language understood by the computer hardware, which is made of electronic switches with two states: off (representing 0) and on (representing (1).

The only language understood by a computer is machine language.

Although a program written in machine language truly represents how data is manipulated by the computer, it has at least two drawbacks. First, it is machine-dependent. The machine language of one computer is different than the machine language of another computer if

they use different hardware. Second, it is very tedious to write programs in this language and very difficult to find errors. The era of machine language is now is referred to as the *first generation* of programming languages.

9.1.2 Assembly languages

The next evolution in programming came with the idea of replacing binary code for instruction and addresses with symbols or mnemonics. Because they used symbols, these languages were first known as **symbolic languages**. The set of these mnemonic languages were later referred to as **assembly languages**. The assembly language for our hypothetical computer to replace the machine language in Table 9.1 is shown in Table 9.2.

Table 9.2 Code in assembly language to add two integers

Code in assembly language				*Description*
LOAD	RF	Keyboard		Load from keyboard controller to register F
STORE	Number1	RF		Store register F into Number1
LOAD	RF	Keyboard		Load from keyboard controller to register F
STORE	Number2	RF		Store register F into Number2
LOAD	R0	Number1		Load Number1 into register 0
LOAD	R1	Number2		Load Number2 into register 1
ADDI	R2	R0	R1	Add registers 0 and 1 with result in register 2
STORE	Result	R2		Store register 2 into Result
LOAD	RF	Result		Load Result into register F
STORE	Monitor	RF		Store register F into monitor controller
HALT				Stop

A special program called an **assembler** is used to translate code in assembly language into machine language.

9.1.3 High-level languages

Although assembly languages greatly improved programming efficiency, they still required programmers to concentrate on the hardware they were using. Working with symbolic languages was also very tedious, because each machine instruction had to be individually coded. The desire to improve programmer efficiency and to change the focus from the computer to the problem being solved led to the development of **high-level languages**.

High-level languages are portable to many different computers, allowing the programmer to concentrate on the application rather than the intricacies of the computer's organization. They are designed to relieve the programmer from the details of assembly language.

High-level languages share one characteristic with symbolic languages: they must be converted to machine language. This process is called *interpretation* or *compilation* (described later in the chapter).

Over the years, various languages, most notably BASIC, COBOL, Pascal, Ada, C, C++, and Java, were developed. Program 9-1 shows the code for adding two integers as it would appear in the C++ language. Although the program looks longer, some of the lines are used for documentation (comments).

Program 9-1 Addition program in C++

```
/*      This program reads two integers from keyboard and prints their sum.
        Written by:
        Date:
*/
#include <iostream>
using namespace std;
int main ()
{
        // Local Declarations
        int number1;
        int number2;
        int result;
        // Statements
        cin >> number1;
        cin >> number2;
        result = number1 + number2;
        cout << result;
        return 0;
} // main
```

9.2 TRANSLATION

Programs today are normally written in one of the high-level languages. To run the program on a computer, the program needs to be translated into the machine language of the computer on which it will run. The program in a high-level language is called the **source program**. The translated program in machine language is called the **object program**. Two methods are used for translation: **compilation** and **interpretation**.

9.2.1 Compilation

A **compiler** normally translates the whole source program into the object program.

9.2.2 Interpretation

Some computer languages use an **interpreter** to translate the source program into the object program. Interpretation refers to the process of translating each line of the source

program into the corresponding line of the object program and executing the line. However, we need to be aware of two trends in interpretation: that used by some languages before Java and the interpretation used by Java.

First approach to interpretation

Some interpreted languages prior to Java (such as BASIC and APL) used a kind of interpretation process that we refer to as the *first approach* to interpretation, for the lack of any other name. In this type of interpretation, each line of the source program is translated into the machine language of the computer being used and executed immediately. If there are any errors in translation and execution, the process displays a message and the rest of the process is aborted. The program needs to be corrected and be interpreted and executed again from the beginning. This first approach was considered to be a slow process, which is why most languages use compilation instead of interpretation.

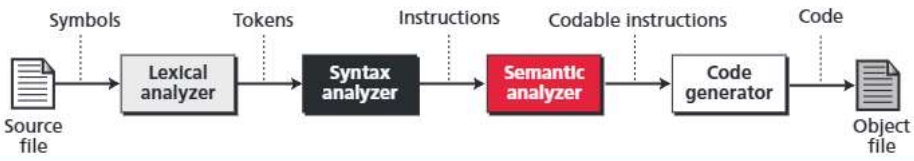
Second approach to interpretation

With the advent of Java, a new kind of interpretation process was introduced. The Java language is designed to be portable to any computer. To achieve portability, the translation of the source program to object program is done in two steps: compilation and interpretation. A Java source program is first compiled to create Java **bytecode**, which looks like code in a machine language, but is not the object code for any specific computer: it is the object code for a virtual machine, called the Java Virtual Machine or JVM. The bytecode then can be compiled or interpreted by any computer that runs a JVM emulator—that is, the computer that runs the bytecode needs only a JVM emulator, not the Java compiler.

9.2.3 Translation process

Compilation and interpretation differ in that the first translates the whole source code before executing it, while the second translates and executes the source code a line at a time. Both methods, however, follow the same translation process shown in Figure 9.1.

Figure 9.1 *Source code translation process*



Lexical analyzer

A **lexical analyzer** reads the source code, symbol by symbol, and creates a list of **tokens** in the source language. For example, the five symbols *w*, *h*, *i*, *l*, *e* are read and grouped together as the token *while* in the C, C++, or Java languages.

Syntax analyzer

The **syntax analyzer** parses a set of tokens to find instructions. For example, the tokens '*x*', '=', '0' are used by the syntax analyzer to create the assignment statement in the C language '*x* = 0'. We will discuss the function of a parser and a syntax analyzer in more detail when we describe language recognition in artificial intelligence in Chapter 18.

Semantic analyzer

The **semantic analyzer** checks the sentences created by the syntax analyzer to be sure that they contain no ambiguity. Computer languages are normally unambiguous, which means that this stage is either omitted in a **translator**, or its duty is minimal. We also discuss semantic analysis in more details in Chapter 18.

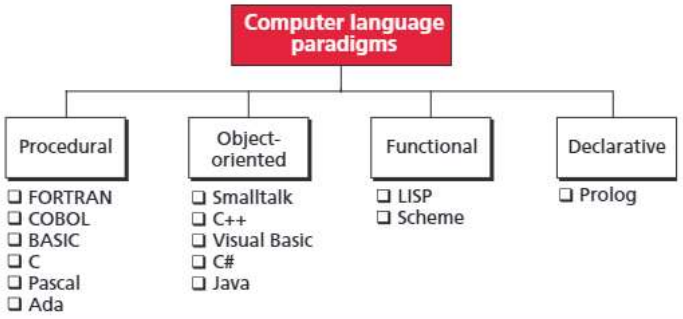
Code generator

After unambiguous instructions are created by the semantic analyzer, each instruction is converted to a set of machine language instructions for the computer on which the program will run. This is done by the **code generator**.

9.3 PROGRAMMING PARADIGMS

Today computer languages are categorized according to the approach they use to solve a problem. A *paradigm*, therefore, is a way in which a computer language looks at the problem to be solved. We divide computer languages into four paradigms: *procedural* (imperative), *object-oriented*, *functional*, and *declarative*. Figure 9.2 summarizes these.

Figure 9.2 *Categories of programming languages*



9.3.1 The procedural paradigm

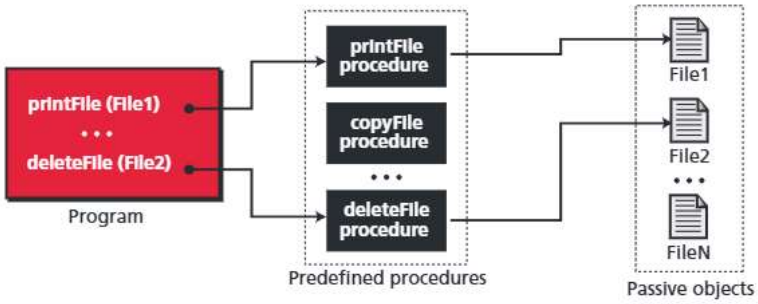
In the **procedural paradigm (or imperative paradigm)** we can think of a program as an *active agent* that manipulates *passive objects*. We encounter many passive objects in our

daily life: a stone, a book, a lamp, and so on. A passive object cannot initiate an action by itself, but it can receive actions from active agents.

A program in a procedural paradigm is an active agent that uses passive objects that we refer to as *data* or *data items*. Data items, as passive objects, are stored in the memory of the computer, and a program manipulates them. To manipulate a piece of data, the active agent (program) issues an action, referred to as a *procedure*. For example, think of a program that prints the contents of a file. To be printed, the file needs to be stored in memory (or some registers that act as memory). The file is a passive object or a collection of passive objects. To print the file, the program uses a procedure, which we call *print*. The procedure print has usually been written previously to include all the actions required to tell the computer how to print each character in the file. The program invokes or *calls* the procedure *print*. In a procedural paradigm, the object (*file*) and the procedure (*print*) are completely separate entities. The object (*file*) is an independent entity that can receive the *print* action, or some other actions, such as *delete*, *copy*, and so on. To apply any of these actions to the file, we need a procedure to act on the file. The procedure *print* (or *copy* or *delete*) is a separate entity that is written and the program only triggers it.

To avoid writing a new procedure each time we need to print a file, we can write a general procedure that can print any file. When we write this procedure, every reference to the file name is replaced by a symbol, such as F or FILE, or something else. When the procedure is called (triggered), we pass the name of the actual file to be printed to the procedure, so that we can write a procedure called *print* but call it twice in the program to print two different files. Figure 9.3 shows how a program can call different predefined procedures to print or delete different object files.

Figure 9.3 *The concept of the procedural paradigm*



We need to separate the procedure from its triggering by the program. The program does not define the procedure (as explained later), it only triggers or calls the procedure. The procedure must already exist.

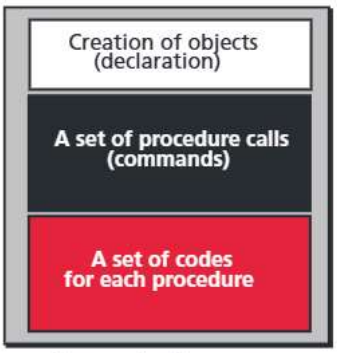
When we use a procedural high-level language, the program consists of nothing but a lot of procedure calls. Although it is not immediately obvious, even when we use a simple mathematical operator such as the addition operator (+), we are using a procedure call to a procedure that is already written. For example, when we use the expression A + B and expect the expression to add the value of two objects A and B, we are calling the procedure

add and passing the name of these two objects to the procedure. The procedure *add* needs two objects to act on. It adds the values of the two objects and returns the result. In other words, the expression A + B is a short cut for *add* (A, B). The designer of the language has written this procedure and we can call it.

If we think about the procedures and the objects to be acted upon, the concept of the procedural paradigm becomes simpler to understand. A program in this paradigm is made up of three parts: a part for object creation, a set of procedure calls, and a set of code for each procedure. Some procedures have already been defined in the language itself. By combining this code, the programmer can create new procedures.

Figure 9.4 shows these three components of a procedural program. There are also extra tokens in the language that are used for delimiting or organizing the calls, but these are not shown in the figure.

Figure 9.4 *The components of a procedural program*



A procedural program

Some procedural languages

Several high-level imperative (procedural) languages have been developed over the last few decades, such as FORTRAN, COBOL, Pascal, C, and Ada.

FORTRAN

FORTRAN (FORmula TRANslation), designed by a group of IBM engineers under the supervision of Jack Backus, became commercially available in 1957. FORTRAN was the first high-level language. During the last 40 years FORTRAN has gone through several versions: FORTRAN, FORTRAN II, FORTRAN IV, FORTRAN 77, FORTRAN 99, and HPF (High Performance FORTRAN). The newest version (HPF) is used in high-speed multiprocessor computer systems. FORTRAN has some features that, even after four decades, still make it an ideal language for scientific and engineering applications. These features can be summarized as:

- High-precision arithmetic
- Capability of handling complex numbers
- Exponentiation computation (a^b)

COBOL

COBOL (COmmon Business-Oriented Language) was designed by a group of computer scientists under the direction of Grace Hopper of the US Navy. COBOL had a specific design goal: to be used as a business programming language. The problems to be solved in a business environment are totally different from those in an engineering environment. The programming needs of the business world can be summarized as follows:

- Fast access to files and databases
- Fast updating of files and databases
- Large amounts of generated reports
- User-friendly formatted output

Pascal

Pascal was invented by Niklaus Wirth in 1971 in Zurich, Switzerland. It was named after Blaise Pascal, the 17th-century French mathematician and philosopher who invented the Pascaline calculator. Pascal was designed with a specific goal in mind: to teach programming to novices by emphasizing the structured programming approach. Although Pascal became the most popular language in academia, it never attained the same popularity in industry. Today's **procedural languages** owe a lot to this language.

C

The **C language** was developed in the early 1970s by Dennis Ritchie at Bell Laboratories. It was originally intended for writing operating systems and system software—most of the UNIX operating system is written in C. Later, it became popular among programmers for several reasons:

- C has all the high-level instructions a structured high-level programming language should have: it hides the hardware details from the programmer.
- C also has some low-level instructions that allow the programmer to access the hardware directly and quickly: C is closer to assembly language than any other high-level language. This makes it a good language for system programmers.
- C is a very efficient language: its instructions are short. This conciseness attracts programmers who want to write short programs.

Ada

Ada was named after Augusta Ada Byron, the daughter of Lord Byron and the assistant to Charles Babbage, the inventor of the Analytical Engine. It was created for the US Department of Defense (DoD) to be the uniform language used by all DoD contractors. Ada has three features that make it very popular for the DoD, and industry:

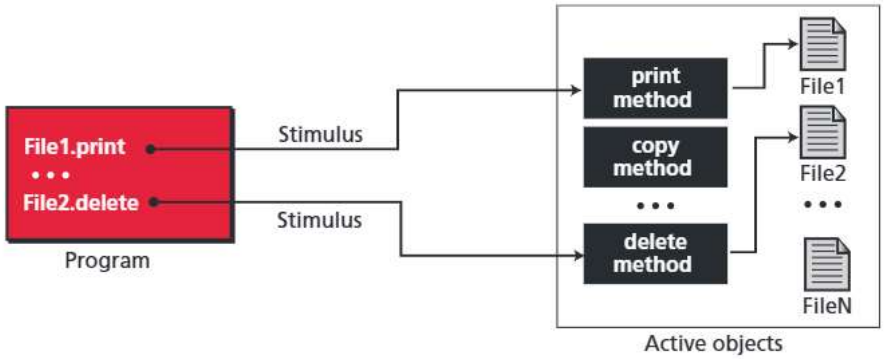
- Ada has high-level instructions like other procedural languages.
- Ada has instructions to allow real-time processing. This makes it suitable for process control.
- Ada has parallel-processing capabilities. It can be run on mainframe computers with multiple processors.

9.3.2 The object-oriented paradigm

The **object-oriented paradigm** deals with active objects instead of passive objects. We encounter many active objects in our daily life: a vehicle, an automatic door, a dishwasher, and so on. The actions to be performed on these objects are included in the object: the objects need only to receive the appropriate stimulus from outside to perform one of the actions.

Returning to our example in the procedural paradigm, a file in an object-oriented paradigm can be packed with all the procedures—called methods in the object-oriented paradigm—to be performed by the file: printing, copying, deleting, and so on. The program in this paradigm just sends the corresponding request to the object (print, delete, copy, and so on) and the file will be printed, copied, or deleted. Figure 9.5 illustrates the concept.

Figure 9.5 *The concept of an object-oriented paradigm*



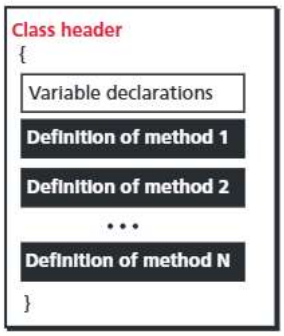
The methods are shared by all objects of the same type, and also for other objects that are inherited from these objects, as we discuss later. If the program wants to print File1, it just sends the required stimulus to the active objects and File1 will be printed.

Comparing the procedural paradigm with the object-oriented paradigm (Figure 9.3 and Figure 9.5), we see that the procedures in the procedural paradigm are independent entities, but the methods in object-oriented paradigm belong to the object's territory.

Classes

As Figure 9.5 shows, objects of the same type (files, for example) need a set of methods that show how an object of this type reacts to stimuli from outside the object's 'territories'. To create these methods, object-oriented languages such as C++, Java, and C# (pronounced 'C sharp') use a unit called a **class**, as shown in Figure 9.6. The exact format of this program unit is different for different object-oriented languages (see Appendix F).

Figure 9.6 *The components of a class*



Methods

In general, the formats of methods are very similar to the functions used in some procedural languages. Each **method** has its header, its local variables, and its statement. This means that most of the features we discussed for procedural languages are also applied to methods written for an object-oriented program. In other words, we can claim that object-oriented languages are actually an extension of procedural languages with some new ideas and some new features. The C++ language, for example, is an object-oriented extension of the C language. The C++ language can even be used as a procedural language with no or minimum use of objects. The Java language is an extension of C++ language, but it is totally an object-oriented language.

Inheritance

In the object-oriented paradigm, as in nature, an object can inherit from another object. This concept is called **inheritance**. When a general class is defined, we can define a more specific class that inherits some of the characteristics of the general class, but also has some new characteristics. For example, when an object of the type *GeometricalShapes* is defined, we can define a class called *Rectangles*. Rectangles are geometrical shapes with additional characteristics.

Polymorphism

Polymorphism means 'many forms'. Polymorphism in the object-oriented paradigm means that we can define several operations with the same name that can do different things in related classes. For example, assume that we define two classes, Rectangles and Circles, both inherited from the class GeometricalShapes. We define two operations both named *area*, one in Rectangles and one in Circles, that calculate the area of a rectangle or a circle. The two operations have the same name but do different things, as calculating the area of a rectangle and the area of a circle need different operands and operations.

Some object-oriented languages

Several **object-oriented languages** have been developed. We briefly discuss the characteristics of two: C++ and Java.

C++

The **C++ language** was developed by Bjarne Stroustrup at Bell Laboratory as an improvement of the C language. It uses *classes* to define the general characteristics of similar objects and the operations that can be applied to them. For example, a programmer can define a *GeometricalShapes* class and all the characteristics common to two-dimensional geometrical shapes, such as center, number of sides, and so on. The class can also define operations (functions or methods) that can be applied to a geometrical shape, such as calculating and printing the area, calculating and printing the perimeter, printing the coordinates of the center point, and so on. A program can then be written to create different objects of type *GeometricalShapes*. Each object can have a center located at a different point and a different number of sides. The program can then calculate and print the area, perimeter, and the location of the center for each object.

Three principles were used in the design of the C++ language: *encapsulation*, *inheritance*, and *polymorphism*.

Java

Java was developed at Sun Microsystems, Inc. It is based on C and C++, but some features of C++, such as multiple inheritance, are removed to make the language more robust. In addition, the language is totally class-oriented. In C++ one can solve a problem without ever defining a class, but in Java every data item belongs to a class.

A program in Java can either be an application or an **applet**. An application is a complete stand-alone program that can be run independently. An applet, on the other hand, is embedded HTML (see Chapter 6), stored on a server, and run by a browser. The browser can download the applet and run it locally.

In Java, an application program (or an applet) is a collection of classes and instances of those classes. One interesting feature of Java is the *class library*, a collection of classes. Although C++ also provides a class library, in Java the user can build new classes based on those provided by the library.

The execution of a program in Java is also unique. We create a class and pass it to the interpreter, which calls the class methods. Another interesting feature of Java is support for **multithreading**. A thread is a sequence of actions executed one after another. C++ allows only single threading—that is, the whole program is executed as a single process thread—but Java allows the concurrent execution of several lines of code.

9.3.3 The functional paradigm

In the **functional paradigm** a program is considered a mathematical function. In this context, a **function** is a black box that maps a list of inputs to a list of outputs (Figure 9.7).

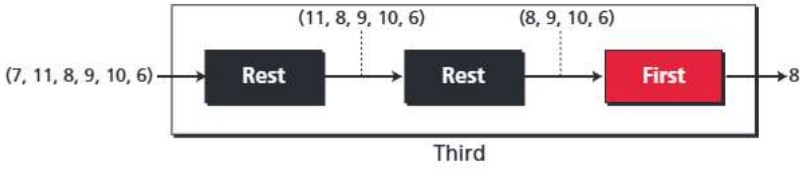
Figure 9.7 *A function in a functional language*

For example, *summation* can be considered as a function with *n* inputs and only one output. The function takes the *n* inputs, adds them, and creates the sum. A **functional language** does the following:

- Predefines a set of primitive (atomic) functions that can be used by any programmer.
- Allows the programmer to combine primitive functions to create new functions.

For example, we can define a primitive function called *first* that extracts the first element of a list. It may also have a function called *rest* that extracts all the elements except the first. A program can define a function that extracts the third element of a list by combining these two functions as shown in Figure 9.8.

Figure 9.8 *Extracting the third element of a list*



A functional language has two advantages over a procedural language: it encourages modular programming and allows the programmer to make new functions out of existing ones. These two factors help a programmer create large and less error-prone programs from already-tested programs.

Some functional languages

We briefly discuss LISP and Scheme as examples of functional languages.

LISP

LISP (LISt Programming) was designed by a team of researchers at MIT in the early 1960s. It is a list-processing programming language in which everything is considered a list.

Scheme

The LISP language suffered from a lack of standardization. After a while, there were different versions of LISP everywhere;. The de facto standard is the one developed by MIT in the early 1970s called **Scheme**.

The Scheme language defines a set of primitive functions that solves problems. The function name and the list of inputs to the function are enclosed in parentheses. The result is an output list, which can be used as the input list to another function. For example, there is a function, *car*, that extracts the first element of a list. There is a function, called *cdr*, that extracts the rest of the elements in a list except the first one. In other words, we have:

```
(car 2 3 7 8 11 17 20) → 2
(cdr 2 3 7 8 11 17 20) → 3 7 8 11 17 20
```

Now we can combine these two functions to extract the third element of any list:

```
(car (cdr (cdr  list)))
```

If we apply the above function to (2 3 7 8 11 17 20), it extracts 7 because the result of the innermost parentheses is 3 7 8 11 17 20. This becomes the input to the middle parentheses, with the result 7 8 11 17 20. This list now becomes the input to the car function, which takes out the first element, 7.

9.3.4 The declarative paradigm

A **declarative paradigm** uses the principle of logical reasoning to answer queries. It is based on formal logic defined by Greek mathematicians and later developed into *first-order predicate calculus*.

Logical reasoning is based on deduction. Some statements (facts) are given that are assumed to be true, and the logician uses solid rules of logical reasoning to deduce new statements (facts). For example, the famous rule of deduction in logic is:

```
If (A is B) and (B is C), then (A is C)
```

Using this rule and the two following facts:

```
Fact 1: Socrates is a human → A is B
Fact 2: A human is mortal → B is C
```

we can deduce a new fact:

```
Fact 3: Socrates is mortal → A is C
```

Programmers study the domain of their subject—that is, know all the facts in this domain—or get the facts from experts in the field. Programmers also need to be expert in logic to carefully define the rules. Then the program can deduce and create new facts.

One problem associated with **declarative languages** is that a program is specific to a particular domain, because collecting all the facts into one program makes it huge. This is the reason why declarative programming is limited so far to specific fields such as artificial intelligence. We discuss logic further in Chapter 18.

Prolog

One of the famous declarative languages is **Prolog (PROgramming in LOGic)** , developed by A. Colmerauer in France in 1972. A program in Prolog is made up of facts and rules. For example, the previous facts about human beings can be stated as:

```
human (John)
mortal (human)
```

The user can then ask:

```
?-mortal (John)
```

and the program will respond with *yes*.

9.4 COMMON CONCEPTS

In this section we conduct a quick navigation through some procedural languages to find common concepts. Some of these concepts are also available in most object-oriented languages because, as we explain, an object-oriented paradigm uses the procedural paradigm when creating methods.

9.4.1 Identifiers

One feature present in all procedural languages, as well as in other languages, is the **identifier**—that is, the name of objects. Identifiers allow us to name objects in the program. For example, each piece of data in a computer is stored at a unique address. If there were no identifiers to represent data locations symbolically, we would have to know and use data addresses to manipulate them. Instead, we simply give data names and let the compiler keep track of where they are physically located.

9.4.2 Data types

A **data type** defines a set of values and a set of operations that can be applied to those values. The set of values for each type is known as the *domain* for the type. Most languages define two categories of data types: *simple types* and *composite types*.

Simple data types

A **simple type** (sometimes called an *atomic type*, *fundamental type*, *scalar type*, or *built-in type*) is a data type that cannot be broken into smaller data types. Several simple data types have been defined in **imperative languages**:

- An *integer* type is a whole number, that is, a number without a fractional part. The range of values an integer can take depends on the language. Some languages support several integer sizes.
- A *real* type is a number with a fractional part.
- A *character* type is a symbol in the underlying character set used by the language, for example, ASCII or Unicode.
- A *Boolean* type is type with only two values, *true* or *false*.

Composite data types

A **composite type** is a set of elements in which each element is a simple type or a composite type (that is, a recursive definition). Most languages defines the following composite types:

- An *array* is a set of elements each of the same type.
- A *record* is a set of elements in which the element can be of different types.

9.4.3 Variables

Variables are names for memory locations. As discussed in Chapter 5, each memory location in a computer has an address. Although the addresses are used by the computer internally, it is very inconvenient for the programmer to use addresses for two reasons. First, the programmer does not know the relative address of the data item in memory. Second, a data item may occupy more than one location in memory. Names, as a substitute for addresses, free the programmer to think at the level at which the program is executed. A programmer can use a variable, such as *score*, to store the integer value of a score received in a test. Since a variable holds a data item, it has a type.

Variable declarations

Most procedural and object-oriented languages required that the variables be declared before being used. Declaration alerts the computer that a variable with a given name and type will be used in the program. The computer reserves the required storage area and names it. Declaration is part of the object creation we discussed in the previous section. For example, in C, C++, and Java we can declare three variables of type character, integer, and real as shown below:

```
char C;
int num;
double result;
```

The first line declares a variable C to be of type character. The second declares a variable num to be of type integer. The third line declares a variable named result to be of type real.

Variable initialization

Although the value of data stored in a variable may change during the program's execution, most procedural languages allow the initialization of the variables when they are declared. Initialization stores a value in the variable. The following shows how the variables can be declared and initialized at the same time:

```
char C ='Z';
int num = 123;
double result = 256,782;
```

9.4.4 Literals

A **literal** is a predetermined value used in a program. For example, if we need to calculate the area of a circle when the value of the radius is stored in the variable r, we can use the expression $3.14 \times r^2$, in which the approximate value of π (pi) is used as a literal. In most programming languages we can have integer, real, character, and Boolean literals. In most languages, we can also have string literals. To distinguish the character and string literals from the names of variables and other objects, most languages require that the character literals be enclosed in single quotes, such as 'A' and strings to be enclosed in double quotes such as "Anne".

9.4.5 Constants

The use of literals is not considered good programming practice unless we are sure that the value of the literal will not change with time (such as the value of π in geometry). However, most literals may change value with time. For example, if a sales tax is 8 per cent this year, it may not be the same next year. When we write a program to calculate the cost of items, we should not use the literal in our program:

$$cost \rightarrow price \times 1.08$$

For this reason, most programming languages defined **constants**. A constant, like a variable, is a named location that can store a value, but the value cannot be changed after it has been defined at the beginning of the program. However, if next year we want to use the program again, we can change just one line at the beginning of the program, the value of the constant. For example, in a C, or C++ program, the tax rate can be defined at the beginning and used during the program:

```
const float taxMultiplier = 1.08;
...
cost  = price * taxMultiplier;
```

Note that a constant, like a variable, has a type that must be defined when the constant is declared.

9.4.6 Input and Output

Almost every program needs to read and/or write data. These operations can be quite complex, especially when we read and write large files. Most programming languages use a predefined function for input and output.

Input

Data is **input** by either a statement or a predefined function. The C language has several input functions. For example, the *scanf* function reads data from the keyboard, formats it, and stores it in a variable. The following is an example:

```
scanf ("%d", &num);
```

When the program encounters this instruction, it waits for the user to type an integer. It then stores the value in the variable num. The %d tells the program to expect a decimal integer.

Output

Data is **output** by either a statement or a predefined function. The C language has several output functions. For example, the *printf* function displays a string on the monitor. The programmer can include the value of a variable or variables as part of the string. The following displays the value of a variable at the end of a literal string:

```
printf ("The value of the number is: %d", num);
```

https://sanet.st/blogs/polatebooks/

9.4.7 Expressions

An **expression** is a sequence of operands and operators that reduces to a single value. For example, the following is an expression with a value of 13:

```
2 * 5 + 3
```

Operator

An **operator** is a language-specific token that requires an action to be taken. The most familiar operators are drawn from mathematics. For example, multiply (*) is an operator—it indicates that two numbers are to be multiplied together. Every language has operators, and their use is rigorously specified in the syntax, or rules, of the language.

Arithmetic operators are used in most languages. Table 9.3 shows some arithmetic operators used in C, C++, and Java.

Table 9.3 Arithmetic operators

Operator	*Definition*	*Example*
+ – * / %	Addition Subtraction Multiplication Division (the result is the quotient) Division (the result is the remainder)	3 + 5 2 – 4 Num * 5 Sum / Count Count % 4
++ – –	Increment (add 1 to the value of the variable) Decrement (subtract 1 from the value of the variable)	Count++ Count– –

Relational operators compare data to see if a value is greater than, less than, or equal to another value. The result of applying relational operators is a Boolean value (true or false). C, C++, and Java use six relational operators, as shown in Table 9.4:

Table 9.4 Relational operators

Operator	*Definition*	*Example*
< <= > >= == !=	Less than Less than or equal to Greater than Greater than or equal to Equal to Not equal to	Num1 < 5 Num1 <= 5 Num2 > 3 Num2 >= 3 Num1 == Num2 Num1 != Num2

Logical operators combine Boolean values (true or false) to get a new value. The C language uses three logical operators, as shown in Table 9.5.

Table 9.5 Logical operators

Operator	*Definition*	*Example*
!	Not	! (Num1 < Num2)
&&	And	(Num1 < 5) && (Num2 > 10)
||	Or	(Num1 < 5) || (Num2 > 10)

Operand

An **operand** receives an operator's action. For any given operator, there may be one, two, or more operands. In our arithmetic example, the operands of division are the dividend and the divisor.

9.4.8 Statements

A **statement** causes an action to be performed by the program. It translates directly into one or more executable computer instructions. For example, C, C++, and Java define many types of statements. We discuss some of these in this section.

Assignment statements

An **assignment statement** assigns a value to a variable. In other words, it stores the value in the variable, which has already been created in the declaration section. We use the symbol ← in our algorithm to define assignment. Most languages like (C, C++, and Java) use the symbol = for assignment. Other languages such as Ada or Pascal use := for assignment.

Compound statements

A **compound statement** is a unit of code consisting of zero or more statements. It is also known as a *block*. A compound statement allows a group of statements to be treated as a single entity. A compound statement consists of an opening brace, an optional statement section, followed by a closing brace. The following shows the makeup of a compound statement:

```
{
   x = 1;
   y = 20;
}
```

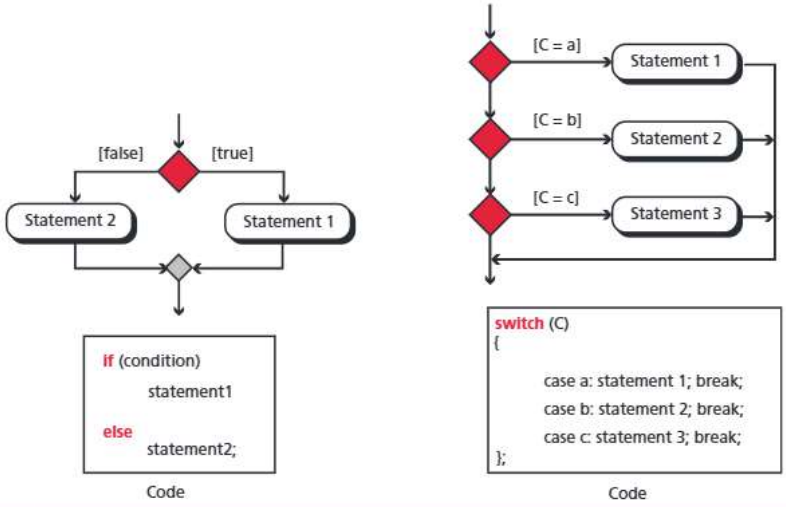
Control statements

A **control statement** is a program in a procedural language that is a set of statements. The statements are normally executed one after another. However, sometimes it is necessary to change this sequential order, for example to repeat a statement or a set of statements, or two different set of statements to be executed based on a Boolean value. The instruction provided in the machine languages of computers for this type of deviation from sequential execution is the *jump* instruction we discussed briefly in Chapter 5. Early imperative languages used the *go to* statement to simulate the *jump* instruction. Although the *go to* statement can be found in some imperative languages today, the principle of structured programming discourages its use. Instead, structured programming strongly recommends the use of the

three constructs of *sequence*, *selection*, and *repetition*, as we discussed in Chapter 8. Control statements in imperative languages are related to selection and repetition.

- Most imperative languages have two-way and multi-way selection statements. Two-way selection is achieved through the *if-else* statement, multi-way selection through the *switch* (or *case*) statement. The UML diagram and the code for the *if-else* statement is shown in Figure 9.9.

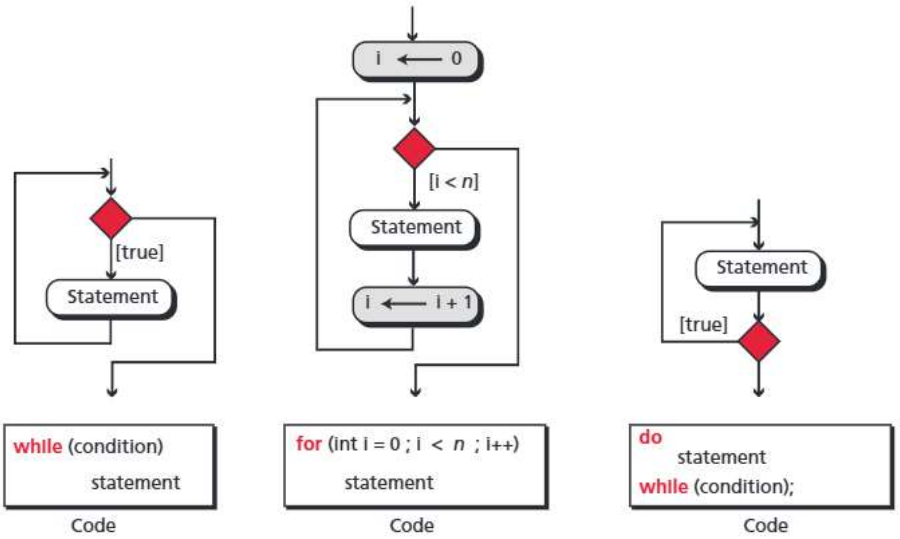
Figure 9.9 *Two-way and multi-way decisions*



In an *if-else* statement, if the condition is true, statement 1 is executed, while if the condition is false, statement 2 is executed. Both statements 1 and 2 can be any type of statement, including a null statement or a compound statement. Figure 9.9 also shows the code for the *switch* (or *case*) statement The value of C (a, b, or c) decide which of statement 1, statement 2, or statement 3 is executed.

- We discussed the repetition construct in Chapter 8. Most of the imperative languages define between one and three loop statements that can achieve repetition. C, C++ and Java define three loop statements, but all of them can be simulated using the *while* loop (Figure 9.10).

Figure 9.10 *Three types of repetition*



The main repetition construct in the C language is the *while* loop. A *while* loop is a *pretest* loop: it checks the value of a testing expression. If the value is true, the program goes through one iteration of the loop and tests the value again. The *while* loop is considered an event-controlled loop: the loop continues in iteration until an event happens that changes the value of the test expression from true to false.

The *for* loop is also a pretest loop. However, in contrast to the *while* loop, it is a counter-controlled loop. A counter is set to an initial value and is incremented or decremented in each iteration. The loop is terminated when the value of the counter matches a predetermined value.

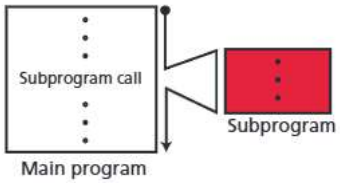
The *do-while* loop is also an event-controlled loop. However, in contrast to the *while* loop, it is a post-test loop. The loop does one iteration and tests the value of an expression. If it is false, it terminates, if true, it does one more iteration and tests again.

9.4.9 Subprograms

In Chapter 8 we showed that a selection sort algorithm can be written as a main program and a **subprogram**. All procedures that are needed to find the smallest item among an unsorted list can be grouped into a subprogram. The idea of subprograms is crucial in procedural languages and to a lesser extent in object-oriented languages. We explained that a program written in a procedural language is a set of procedures that are normally predefined, such as addition, multiplication, and so on. However, sometimes a subset of these procedures to accomplish a single task can be collected and placed in their own program unit, a subprogram. This is useful because the subprogram makes programming more structural: a subprogram to accomplish a specific task can be written once but called many times, just like predefined procedures in the programming language.

Subprograms also make programming easier: in incremental program development the programmer can test the program step by step by adding a subprogram at each step. This helps to detect errors before the next subprogram is written. Figure 9.11 illustrates the idea of a subprogram.

Figure 9.11 *The concept of a subprogram*



Local variables

In a procedural language, a subprogram, like the main program, can call predefined procedures to operate on local objects. These local objects or **local variables** are created each time the subprogram is called and destroyed when control returns from the subprogram. The local objects *belong* to the subprograms.

Parameters

It is rare for a subprogram to act only upon local objects. Most of the time the main program requires a subprogram to act on an object or set of objects created by the main program. In this case, the program and subprogram use **parameters**. These are referred to as **actual parameters** in the main program and **formal parameters** in the subprogram. A program can normally pass parameters to a subprogram in either of two ways:

- By *value*
- By *reference*

These are described below.

Pass by value

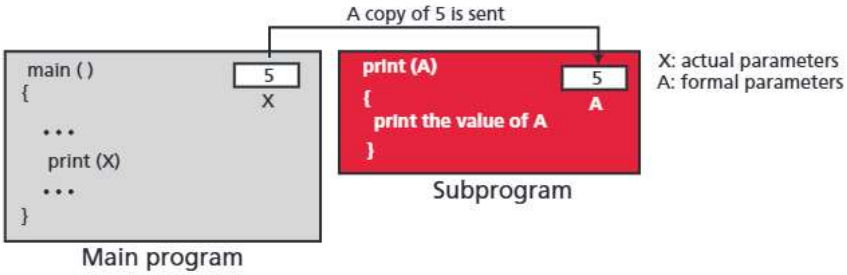
In **pass by value**, the main program and the subprogram create two different objects (variables). The object created in the program belongs to the program and the object created in the subprogram belongs to the subprogram. Since the territory is different, the corresponding objects can have the same or different names. Communication between the main program and the subprogram is one-way, from the main program to the subprogram. The main program sends the value of the actual parameter to be stored in the corresponding formal parameter in the subprogram: there is no communication of parameter value from the subprogram to the main program.

Example 9.1

Assume that a subprogram is responsible for carrying out printing for the main program. Each time the main program wants to print a value, it sends it to the subprogram to be

printed. The main program has its own variable X, the subprogram has its own variable A. What is sent from the main program to the subprogram is the *value* of variable X. This value is stored in the variable A in the subprogram and the subprogram will then print it (Figure 9.12).

Figure 9.12 *An example of pass by value*



Example 9.2

In Example 9.1, since the main program sends only a value to the subprogram, it does not need to have a variable for this purpose: the main program can just send a literal value to the subprogram. In other words, the main program can call the subprogram as print (X) or print (5).

Example 9.3

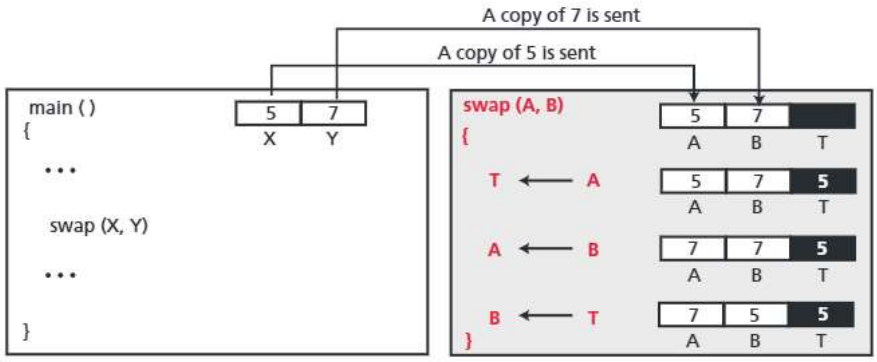
An analogy of pass by value in real life is when a friend wants to borrow and read a valued book that you wrote. Since the book is precious, possibly out of print, you make a copy of the book and pass it to your friend. Any harm to the copy therefore does not affect your book.

Pass by value has an advantage: the subprogram receives only a value. It cannot change, either intentionally or accidentally, the value of the variable in the main program. However, the inability of a subprogram to change the value of the variable in the main program is a disadvantage when the program actually needs the subprogram to do so.

Example 9.4

Assume that the main program has two variables X and Y that need to swap their values. The main program calls a subprogram called *swap* to do so. It passes the value of X and Y to the subprogram, which are stored in two variables A and B. The *swap* subprogram uses a local variable T (temporary) and swaps the two values in A and B, but the original values in X and Y remain the same: they are not swapped. This is illustrated in Figure 9.13.

Figure 9.13 *An example in which pass by value does not work*



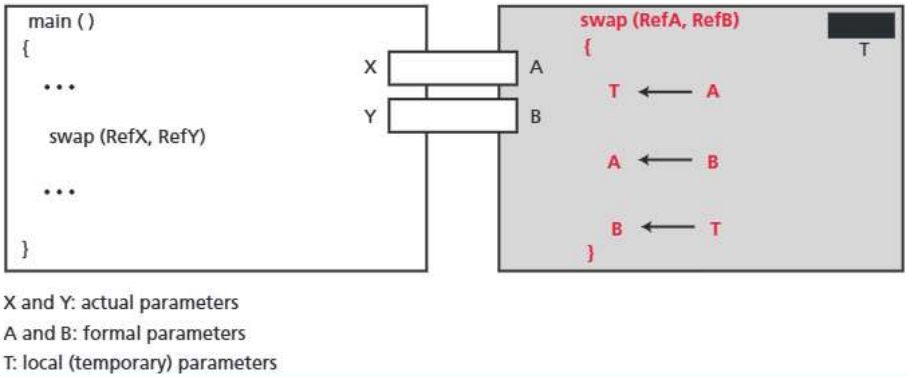
Pass by reference

Pass by reference was devised to allow a subprogram to change the value of a variable in the main program. In pass by reference, the variable, which in reality is a location in memory, is shared by the main program and the subprogram. The same variable may have different names in the main program and the subprogram, but both names refer to the same variable. Metaphorically, we can think of pass by reference as a box with two doors: one opens in the main program, the other opens in the subprogram. The main program can leave a value in this box for the subprogram, the subprogram can change the original value and leave a new value for the program in it.

Example 9.5

If we use the same *swap* subprogram but let the variables be passed by reference, the two values in X and Y are actually exchanged, as Figure 9.14 shows.

Figure 9.14 *An example of pass by reference*



5/1/25, 9:57 PM study viewer

Returning values

A subprogram can be designed to return a value or values. This is the way that predefined procedures are designed. When we use the expression C ← A + B, we actually call a procedure *add* (A, B) that return a value to be stored in the variable C.

Implementation

The concept of subprogram is implemented differently in different languages. In C and C++, the subprogram is implemented as a function.

9.5 END-CHAPTER MATERIALS

9.5.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Cooke, D. A. *Concise Introduction to Computer Languages*, Pacific Grove, CA: Brooks/Cole, 2003
- Tucker, A. and Noonan, R. *Programming Languages: Principles and Paradigms*, Burr Ridge, IL: McGraw-Hill, 2002
- Pratt, T. and Zelkowitz, M. *Programming Languages, Design and Implementation*, Englewood Cliffs, NJ: Prentice-Hall, 2001
- Sebesta, R. *Concepts of Programming Languages*, Boston, MA: Addison-Wesley, 2006

9.5.2 Key terms

actual parameter 264	Java 254
Ada 251	lexical analyzer 247
applet 254	LISt Programming Language (LISP) 255
arithmetic operator 260	literal 258
assembler 245	local variable 264
assembly language 245	logical operator 260
assignment statement 261	machine language 244
bytecode 247	method 253
C++ language 254	multithreading 254
C language 251	object-oriented language 253
class 252	object-oriented paradigm 252
code generator 248	object program 246
COmmon Business-Oriented Language (COBOL) 251	operand 261
compilation 246	operator 260
compiler 246	output 259

(Continued)

https://studylib.net/doc/26059254/foundations-computer-science-4th 287/706

composite type 257	parameter 264
compound statement 261	Pascal 251
computer language 244	pass by reference 266
constant 259	pass by value 264
control statement 261	polymorphism 253
data type 257	procedural paradigm 248
declarative language 256	Programming in LOGic (PROLOG) 256
declarative paradigm 256	programming language 244
expression 260	relational operator 260
formal parameter 264	Scheme 255
FORmula TRANslation (FORTRAN) 250	semantic analyzer 248
functional language 255	simple type 257
functional paradigm 254	source program 246
high-level language 255	statement 261
identifier 257	subprogram 263
imperative language 257	symbolic language 245
imperative paradigm 248	syntax 244
inheritance 253	syntax analyzer 248
input 259	token 247
interpretation 246	translator 248
interpreter 246	variable 258

9.5.3 Summary

- A computer language is a set of predefined words that are combined into a program according to predefined rules, the language's syntax. Over the years, computer languages have evolved from machine language to high-level languages. The only language understood by a computer is machine language.
- High-level languages are portable to many different computers, allowing the programmer to concentrate on the application rather than the intricacies of the computer's organization.
- To run a program on a computer, the program needs to be translated into the computer's native machine language. The program in the high-level language is called the *source program*. The translated program in machine language is called the *object program*. Two methods are used for translation: *compilation* and *interpretation*. A compiler translates the whole source program into the object program. Interpretation refers to the process of translating each line of the source program into the corresponding object program line by line and executing them.
- The translation process uses a lexical analyzer, a syntax analyzer, a semantic analyzer, and code generator, and a lexical analyzer to create a list of tokens.

5/1/25, 9:57 PM study viewer

- A paradigm describes a way in which a computer language can be used to approach a problem to be solved. We divide computer languages into four paradigms: *procedural*, *object-oriented*, *functional*, and *declarative*. A procedural paradigm considers a program as an active agent that manipulates passive objects. FORTRAN, COBOL, Pascal, C, and Ada are examples of procedural languages. The object-oriented paradigm deals with active objects instead of passive objects. C++ and Java are common object-oriented languages. In the functional paradigm, a program is considered as a mathematical function. In this context, a function is a black box that maps a list of inputs to a list of outputs. LISP and Scheme are common functional languages. A declarative paradigm uses the principle of logical reasoning to answer queries. One of the best-known declarative languages is PROLOG.
- Some common concepts in procedural and object-oriented languages are *identifiers*, *data types*, *variables*, *literals*, *constants*, *inputs* and *outputs*, *expressions*, and *statements*. Most languages use two categories of control statements: *decision* and *repetition*. *Sub-programming* is a common concept among procedural languages.

9.6 PRACTICE SET

9.6.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

9.6.2 Review questions

Q9-1. Distinguish between machine language and assembly language.
Q9-2. Distinguish between assembly language and a high-level language.
Q9-3. Which computer language is directly related and understood by a computer?
Q9-4. Distinguish between compilation and interpretation.
Q9-5. List four steps in programming language translation.
Q9-6. List four common computer language paradigms.
Q9-7. Compare and contrast a procedural paradigm with an object-oriented paradigm.
Q9-8. Define a class and a method in an object-oriented language. What is the relation between these two concepts and the concept of an object?
Q9-9. Define a functional paradigm.
Q9-10. Define a declarative paradigm.

9.6.3 Problems

P9-1. Declare three variables of type integers in C language.
P9-2. Define three variables of type real in C language and initialize them to three values.
P9-3. Declare three constants in C of type character, integer, and real respectively.
P9-4. Explain why a constant must be initialized when it is declared.

https://studylib.net/doc/26059254/foundations-computer-science-4th 289/706

P9-5. Find how many times the *statement* in the following code segment in C is executed:

```
A = 5
while (A < 8)
{
    statement;
    A = A + 2;
}
```

P9-6. Find how many times the *statement* in the following code segment in C is executed:

```
A = 5
while (A < 8)
{
    statement;
    A = A - 2;
}
```

P9-7. Find how many times the *statement* in the following code segment in C is executed:

```
for (int i = 5; i < 20, i++)
{
    statement;
    i = i + 1;
}
```

P9-8. Find how many times the *statement* in the following code segment in C is executed:

```
A = 5
do
{
    statement;
    A = A + 1;
} while (A < 10);
```

P9-9. Write the code in Problem P9-6 using *do-while* loop.
P9-10. Write the code in Problem P9-7 using *do-while* loop.
P9-11. Write the code in Problem P9-7 using *while* loop.
P9-12. Write the code in Problem P9-8 using a *for* loop.
P9-13. Write the code in Problem P9-6 using a *for* loop.
P9-14. Write a code fragment using a *while* loop that never executes its body.
P9-15. Write a code fragment using a *do* loop that never executes its body.
P9-16. Write a code fragment using a *for* loop that never executes its body.
P9-17. Write a code fragment using a *while* loop that never stops.

P9-18. Write a code fragment using a *do* loop that never stops.

P9-19. Write a code fragment using a *for* loop that never stops.

P9-20. In the following code, find all the literal values:

```
C = 12 * A + 4 * (B - 5)
```

P9-21. In the following code, find the variables and literals:

```
Hello = "Hello";
```

P9-22. Change the following segment of code to use a *switch* statement:

```
if (A = = 4)
{
    statement1;
}
else if (A == 6)
{
    statement 2;
}
else if (A == 8)
{
    statement 3;
}
else
{
    statement 4 ;
}
```

P9-23. If the subprogram *calculate (A, B, S, P)* accepts the value of A and B and calculates the their sum S and product P, which variable do you pass by value and which one by reference?

P9-24. If the subprogram *smaller(A, B, S)* accepts the value of A and B and finds the smaller of the two, which variable do you pass by value and which one by reference?

P9-25. If the subprogram *cube (A)* accepts the value of A and calculates its cube (A^3), should you pass A to the subprogram by value or by reference?

P9-26. If the subprogram needs to get a value for A from the keyboard and return it to the main program, should you pass A to the subprogram by value or by reference?

P9-27. If the subprogram needs to display the value of A on the monitor, should you pass A to the subprogram by value or by reference?

CHAPTER 10

Software Engineering

In this chapter we introduce the concept of software engineering. We begin with the idea of the software lifecycle. We then show two models used for the development process: the *waterfall* model and the *incremental* model. A brief discussion of four phases in the development process follows.

Objectives

After studying this chapter, the student should be able to:

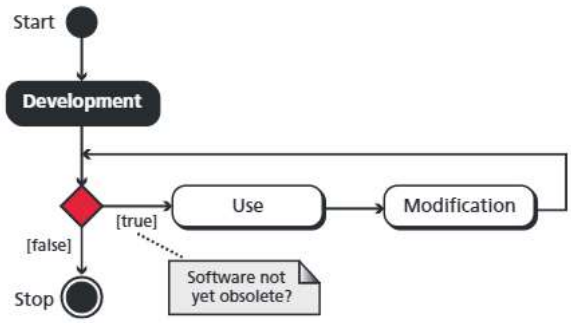
- Understand the concept of the software lifecycle in software engineering.
- Describe two major types of development process, the waterfall and incremental models.
- Understand the analysis phase and describe two separate approaches in the analysis phase: procedure-oriented analysis and object-oriented analysis.
- Understand the design phase and describe two separate approaches in the design phase: procedure-oriented design and object-oriented design.
- Describe the implementation phase and recognize the quality issues in this phase.
- Describe the testing phase and distinguish between glass-box testing and black-box testing.
- Recognize the importance of documentation in software engineering and distinguish between user documentation, system documentation, and technical documentation.

Software engineering is the establishment and use of sound engineering methods and principles to obtain reliable software. This definition, taken from the first international conference on software engineering in 1969, was proposed 30 years after the first computer was built.

10.1 THE SOFTWARE LIFECYCLE

A fundamental concept in **software engineering** is the **software lifecycle**. Software, like many other products, goes through a cycle of repeating phases (Figure 10.1).

Figure 10.1 *The software lifecycle*



Software is first developed by a group of developers. Usually it is in use for a while before modifications are necessary. Modification is often needed due to errors found in the software, changes in the rules or laws governing its design, or changes in the company itself. The software therefore needs to be modified before further use. These two steps, *use* and *modify*, continue until the software becomes obsolete. By 'obsolete', we mean that the software loses its validity because of inefficiency, obsolescence of the language, major changes in user requirements, or other factors.

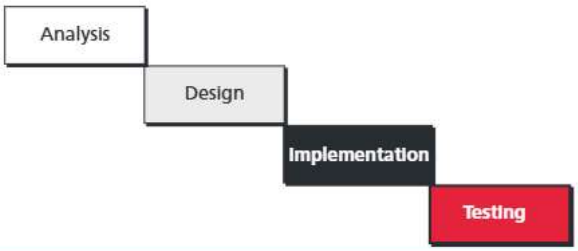
10.1.1 Development process models

Although software engineering involves all three processes in Figure 10.1, in this chapter we discuss only the **development process**, which is shown outside the cycle in Figure 10.1. The development process in the software lifecycle involves four phases: analysis, design, implementation, and testing. There are several models for the development process. We discuss the two most common here: the waterfall model and the incremental model.

The waterfall model

One very popular model for the software development process is known as the **waterfall model** (Figure 10.2). In this model, the development process flows in only one direction. This means that a phase cannot be started until the previous phase is completed.

Figure 10.2 *The waterfall model*



For example, the analysis phase of the whole project should be completed before its design phase is started. The entire design phase should be finished before the implementation phase can be started.

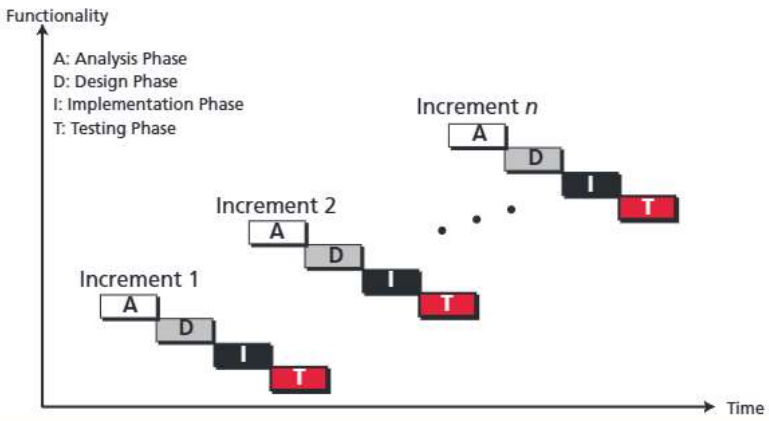
There are advantages and disadvantages to the waterfall model. One advantage is that each phase is completed before the next phase starts. The group that works on the design phase, for example, knows exactly what to do because they have the complete results of the analysis phase. The testing phase can test the whole system because the entire system under development is ready. However, a disadvantage of the waterfall model is the difficulty in locating a problem: if there is a problem in part of the process, the entire process must be checked.

The incremental model

In the **incremental model**, software is developed in a series of steps. The developers first complete a simplified version of the whole system. This version represents the entire system but does not include the details. Figure 10.3 shows the incremental model concept.

In the second version, more details are added, while some are left unfinished, and the system is tested again. If there is a problem, the developers know that the problem is with the new functionality. They do not add more functionality until the existing system works properly. This process continues until all required functionality has been added.

Figure 10.3 *The incremental model*



10.2 ANALYSIS PHASE

The development process starts with the **analysis phase**. This phase results in a specification document that shows *what* the software will do without specifying *how* it will be done. The analysis phase can use two separate approaches, depending on whether the implementation phase is done using a procedural programming language or an object-oriented language. We briefly discuss both in this section.

10.2.1 Procedure-oriented analysis

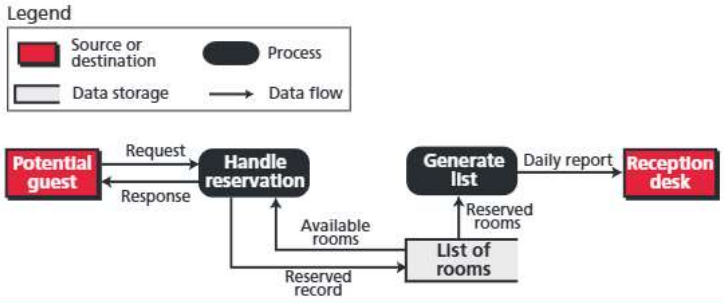
Procedure-oriented analysis—also called *structured analysis* or *classical analysis*—is the analysis process used if the system implementation phase will use a procedural language. The specification in this case may use several modeling tools, but we discuss only a few of them here.

Data flow diagrams

Data flow diagrams show the movement of data in the system. They use four symbols: a square box shows the source or destination of data, a rectangle with rounded corners shows the process (the action to be performed on the data), an open-ended rectangle shows where data is stored, and arrows show the flow of data.

Figure 10.4 shows a simplified version of a booking system in a small hotel that accepts reservations from potential guests through the Internet and confirms or rejects the reservation based on available vacancies.

Figure 10.4 *An example of a data flow diagram*



One of the processes in this diagram (handle reservation) checks the availability using the reservation file and accepts or rejects a reservation. If the reservation is accepted, it will be recorded in the reservation file.

Entity–relationship diagrams

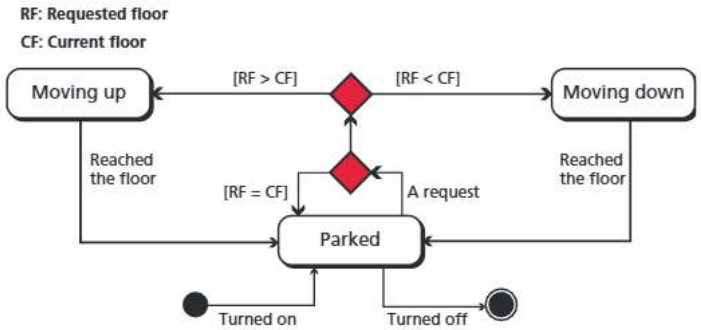
Another modeling tool used during the analysis phase is the **entity–relationship diagram**. Since this diagram is also used in database design, we discuss it in Chapter 14.

State diagrams

State diagrams (see Appendix B) provide another useful tool that is normally used when the state of the entity in the system will change in response to events. As an example of a state diagram, we show the operation of a one-passenger elevator. When a floor button is pushed, the elevator moves in the requested direction. It does not respond to any other request until it reach its destination.

Figure 10.5 shows a state diagram for this old-style elevator. The elevator can be in one of three states: moving up, moving down, or parked. Each of these states is represented by a rounded rectangle in the state diagram. When the elevator is in the parked state, it accepts a request. If the requested floor is the same as the current floor, the request is ignored—the elevator remains in the parked state. If the requested floor is above the current floor, the elevator starts moving up. If the requested floor is lower than the requested floor, the elevator starts moving down. Once moving, the elevator remains in one moving state until it reaches the requested floor.

Figure 10.5 *An example of a state diagram*



10.2.2 Object-oriented analysis

Object-oriented analysis is the analysis process used if the implementation uses an object-oriented language. The specification document in this case may use several tools, but we discuss only a few of them here.

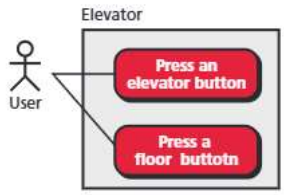
Use-case diagrams

A **use-case diagram** gives the user's view of a system: it shows how the user communicates with the system. A use-case diagram uses four components: system, use cases, actors, and relationships. A system, shown by a rectangle, performs a function. The actions in the system are shown by use cases, which are denoted by rounded rectangles. An actor is someone or something that uses the system. Although actors are represented by stick figures, they do not necessarily represent human beings.

Figure 10.6 shows the use-case diagram for the old-style elevator for which we gave a state diagram in Figure 10.5. The system in this figure is the elevator. The only actor is

the user of the elevator. There are two uses cases: pressing the elevator button (in the hall of each floor) and pressing the floor button inside the elevator. The elevator has only one button on each floor that gives the signal to the elevator to move to that floor.

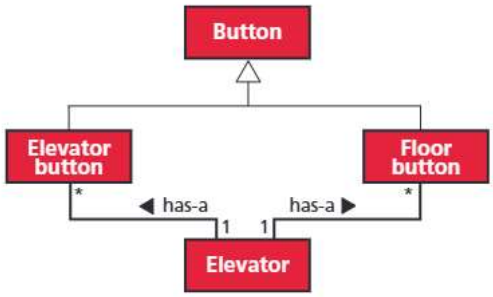
Figure 10.6 *An example of a use-case diagram*



Class diagrams

The next step in analysis is to create a **class diagram** for the system. For example, we can create a class diagram for our old-style elevator. To do so, we need to think about the entities involved in the system. In the elevator system we have two classes of entities: the buttons and the elevator itself. At first glance, therefore, it looks as if we have two classes: a button class and an elevator class. However, we have two types of buttons: the elevator buttons in the hallways and the floor buttons inside the elevator. It seems then that we can have a button class and two classes that inherit from the button class: an elevator button class and a floor button class. The first class diagram that we can create for the elevator problem is therefore that shown in Figure 10.7.

Figure 10.7 *An example of a class diagram*



Note that the elevator button class and the floor button class are subclasses of the button class. However, the relationship between the elevator class and the two button classes (elevator button and floor button) is a one-to-many relation (see Appendix B). The class diagram for the elevator system can of course be extended, but we leave this to books on software engineering.

State chart

After the class diagram is finalized, a **state chart** can be prepared for each class in the class diagram. A state chart in object-oriented analysis plays the same role as the state diagram in procedure-oriented analysis. This means that for the class diagram of Figure 10.7, we need to have a four-state chart.

10.3 DESIGN PHASE

The **design phase** defines *how* the system will accomplish *what* was defined in the analysis phase. In the design phase, all components of the system are defined.

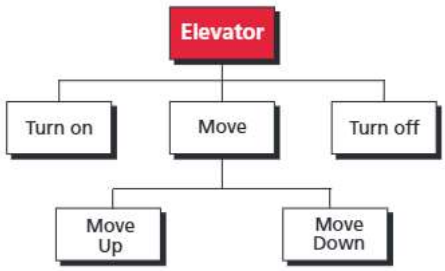
10.3.1 Procedure-oriented design

In **procedure-oriented design** we have both procedures and data to design. We discuss a category of design methods that concentrate on procedures. In procedure-oriented design, the whole system is divided into a set of procedure or modules.

Structure charts

A common tool for illustrating the relations between modules in procedure-oriented design is a **structure chart**. For example, the elevator system whose state diagram is shown in Figure 10.5 can be designed as a set of modules shown in the structure chart in Figure 10.8. Structure charts are discussed in Appendix D.

Figure 10.8 *A structure chart*



Modularity

Modularity means breaking a large project into smaller parts that can be understood and handled easily. In other words, modularity means dividing a large task into small tasks that can communicate with each other. The structure chart discussed in the previous section shows the modularity in the elevator system. There are two main concerns when a system is divided into modules: *coupling* and *cohesion*.

Coupling

Coupling is a measure of how tightly two modules are bound to each other. The more tightly coupled, the less independent they are. Since the objective is to make modules as independent as possible, we want them to be loosely coupled. There are at least three reasons why loose coupling is desirable.

- Loosely coupled modules are more likely to be reusable.
- Loosely coupled modules are less likely to create errors in related modules.
- When the system needs to be modified, loosely coupled modules allow us to modify only modules that need to be changed without affecting modules that do not need to change.

Coupling between modules in a software system must be minimized.

Cohesion

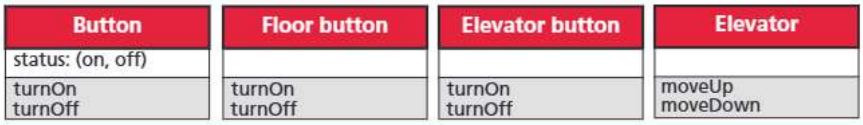
Another issue in modularity is cohesion. **Cohesion** is a measure of how closely the modules in a system are related. We need to have maximum possible cohesion between modules in a software system.

Cohesion between modules in a software system must be maximized.

10.3.2 Object-oriented design

In **object-oriented design**, the design phase continues by elaborating the details of classes. As we mentioned in Chapter 9, a class is made of a set of variables (attributes) and a set of methods (functions). The object-oriented design lists the details of these **attributes** and methods. Figure 10.9 shows an example of the details of our four classes used in the design of the old-style elevator.

Figure 10.9 *An example of classes with attributes and methods*



10.4 IMPLEMENTATION PHASE

In the waterfall model, after the design phase is completed, the **implementation phase** can start. In this phase the programmers write the code for the modules in procedure-oriented design, or write the program units to implement classes in object-oriented design. There are several issues to mention in each case.

10.4.1 Choice of language

In a procedure-oriented development, the project team needs to choose a language or a set of languages from among the procedural languages discussed in Chapter 9. Although some languages like C++ are considered to be both a procedure—and an object-oriented language—normally an implementation uses a purely procedural language such as C. In object-oriented cases, both C++ and Java are common.

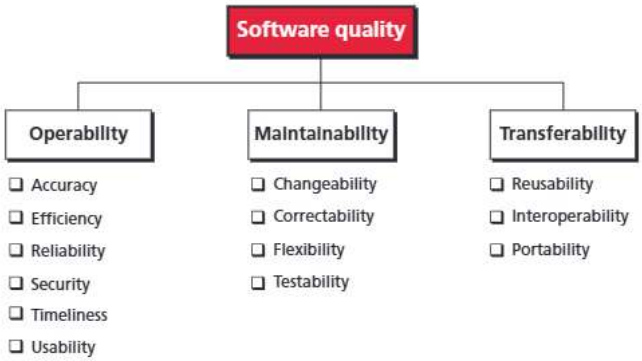
10.4.2 Software quality

The quality of software created at the implementation phase is a very important issue. A software system of high quality is one that satisfies the user's requirements, meets the operating standards of the organization, and runs efficiently on the hardware for which it was developed. However, if we want to achieve a software system of high quality, we must be able to define some attributes of quality.

Software quality factors

Software quality can be divided into three broad measures: *operability*, *maintainability*, and *transferability*. Each of these measures can be further broken down as shown in Figure 10.10.

Figure 10.10 *Quality factors*



Operability

Operability refers to the basic operation of a system. Several measures can be mentioned for operability, as shown in Figure 10.10: *accuracy*, *efficiency*, *reliability*, *security*, *timeliness*, and *usability*.

- A system that is not *accurate* is worse than no system at all. Any system that is developed, therefore, must be thoroughly tested both by a system's test engineer and the user. *Accuracy* can be measured by such metrics as mean time between failures, number of bugs per thousand lines of code, and number of user requests for change.

- *Efficiency* is a subjective term. In some cases, the user will specify a performance standard, such as a real-time response that must be received within 1 second 95 per cent of the time. This is certainly measurable.
- *Reliability* is really the sum of the other factors. If users count on the system to get their job done and are confident in it, then it is most likely reliable. On the other hand, some measures speak directly to a system's reliability, most notably, mean time between failures.
- How *secure* a system is refers to how easy it is for unauthorized people to access the system's data. Although this is a subjective area, there are checklists that assist in assessing the system's security. For example, does the system have and require passwords to identify users?
- *Timeliness* in software engineering can mean several different things. Does the system deliver its output in a timely fashion? For online systems, does the response time satisfy the users' requirements?
- *Usability* is another area that is highly subjective. The best measure of usability is to watch the users and see how they are using the system. User interviews will often reveal problems with the usability of a system.

Maintainability

Maintainability refers to the ease with which a system can be kept up to date and running correctly. Many systems require regular changes, not because they were poorly implemented, but because of changes in external factors. For example, the payroll system for a company might have to be changed often to meet changes in government laws and regulations.

- *Changeability* is a subjective factor. Experienced project leaders, however, are able to estimate how long a requested change will take to implement. If too long, it may indicate that the system is difficult to change. This is especially true of older systems. There are software measurement tools in the field today that will estimate a program's complexity and structure.
- One measure of *correctability* is *mean time to recovery*, which is the time it takes to get a program back into operation after it fails. Although this is a reactive definition, there are currently no predictors of how long it will take to correct a program when it fails.
- Users are constantly requesting changes to systems. *Flexibility* is a qualitative attribute that attempts to measure how easy it is to make these changes. If a program needs to be completely rewritten to effect a change, it is not flexible.
- We might think that **testability** is a highly subjective area, but test engineers have checklists of factors that can assess a program's testability.

Transferability

Transferability refers to the ability to move data and/or a system from one platform to another and to reuse code. In many situations this is not an important factor. On the other hand, if we are writing generalized software, it can be critical.

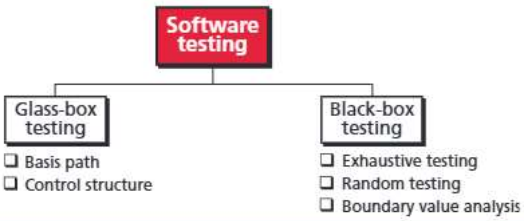
- If modules are written so that they can be reused in other systems, then they have high levels of *reusability*. Good programmers build libraries of functions that they can reuse for solving similar problems.

- *Interoperability* is the capability of sending data to other systems. In today's highly integrated systems, it is a desirable attribute. In fact, it has become so important that operating systems now support the ability to move data between systems, such as between a word processor and a spreadsheet.
- *Portability* is the ability to move software from one hardware platform to another.

10.5 TESTING PHASE

The goal of the **testing phase** is to find errors, which means that a good testing strategy is the one that finds most errors. There are two types of testing: *glass-box* and *black-box* (Figure 10.11).

Figure 10.11 *Software testing*



10.5.1 Glass-box testing

Glass-box testing (or **white-box testing**) is based on knowing the internal structure of the software. The testing goal is to check to determine whether all components of the software do what they are designed to do. Glass-box testing assumes that the tester knows everything about the software. In this case, the software is like a glass box in which everything inside the box is visible. Glass-box testing is done by the software engineer or a dedicated team. Glass-box testing that uses the structure of the software is required to guarantee that at least the following four criteria are met:

- All independent paths in every module are tested at least once.
- All the decision constructs (two-way and multiway) are tested on each branch.
- Each loop construct is tested.
- All data structures are tested.

Several testing methodologies have been designed in the past. We briefly discuss two of them: *basis path* testing and *control structure* testing.

Basis path testing

Basis path testing was proposed by Tom McCabe. This method creates a set of test cases that executes *every statement* in the software at least once.

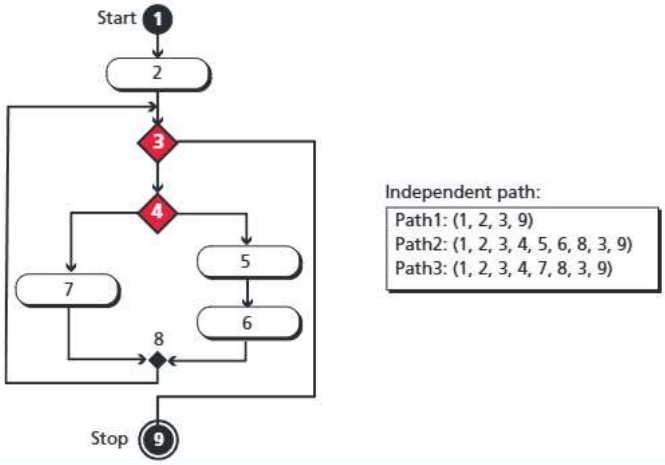
Basis path testing is a method in which each statement in the software is executed at least once.

Basis path testing uses *graph theory* (see Chapter 12) and *cyclomatic complexity* to find the independent paths that must be followed to guarantee that each statement is executed at least once.

Example 10.1

To give the idea of basis path testing and finding the independent paths in part of a program, assume that a system is made up of only one program and that the program is only a single loop with the UML diagram shown in Figure 10.12.

Figure 10.12 *An example of basis path testing*



In this simple program we have three independent paths. The first path is the case in which the loop is bypassed. The second path is where the loop is executed once through the right branch of the decision construct. The third path is when the loop is executed once but through the left branch of the decision construct. If there are more iterations, the paths created are not independent of these three paths: it is possible to prove this if we change the UML to a flow graph, but we must leave the proof to books on software engineering. The idea is to design test cases that cover all three paths in the basis path set, so that all statements are executed at least once.

Control structure testing

Control structure testing is more comprehensive than basis path testing and includes it. This method uses different categories of test that are briefly described below.

Condition testing

Condition testing applies to any condition expression in the module. A *simple condition* is a relational expression, while a *compound condition* is a combination of simple conditions and logical operators (see Chapter 9). Condition testing is designed to check whether all conditions are set correctly.

Data flow testing

Data flow testing is based on the flow of data through the module. This type of testing selects test cases that involve checking the value of variables when they are used on the left side of the assignment statement.

Loop testing

Loop testing uses test cases to check the validity of loops. All types of loops (*while*, *do*, and *for*) are carefully tested.

10.5.2 Black-box testing

Black-box testing gets its name from the concept of testing software without knowing what is inside it and without knowing how it works. In other words, the software is like a black box into which the tester cannot see. Black-box testing tests the functionality of the software in terms of what the software is supposed to accomplish, such as its inputs and outputs. Several methods are used in black-box testing, discussed below.

Exhaustive testing

The best black-box test method is to test the software for all possible values in the input domain. However, in complex software the input domain is so huge that it is often impractical to do so.

Random testing

In random testing, a subset of values in the input domain is selected for testing. It is very important that the subset be chosen in such a way that the values are distributed over the domain input. The use of random number generators can be very helpful in this case.

Boundary-value testing

Errors often happen when boundary values are encountered. For example, if a module defines that one of its inputs must be greater than or equal to 100, it is very important that module be tested for the boundary value 100. If the module fails at this boundary value, it is possible that some condition in the module's code such as $x > 100$ is written as $x > 100$.

10.6 DOCUMENTATION

For software to be used properly and maintained efficiently, documentation is needed. Usually, three separate sets of documentation are prepared for software: user documentation, system documentation, and technical documentation. However, note that documentation is an ongoing process. If the software has problems after release, they must be documented too. If the software is modified, all modifications and their relationship to the original package must also be documented. Documentation only stops when the package becomes obsolete.

Documentation is an ongoing process.

10.6.1 User documentation

To run the software system properly, the users need documentation, traditionally called a *user guide*, that shows how to use the software step by step. User guides usually contain a tutorial section to guide the user through each feature of the software.

A good user guide can be a very powerful marketing tool: the importance of user documentation in marketing cannot be overemphasized. User guides should be written for both the novice and the expert users, and a software system with good user documentation will definitely increase sales.

10.6.2 System documentation

System documentation defines the software itself. It should be written so that the software can be maintained and modified by people other than the original developers. System documentation should exist for all four phases of system development.

In the analysis phase, the information collected should be carefully documented. In addition, the analysts should define the sources of information. The requirements and methods chosen in this phase must be clearly stated with the rationale behind them.

In the design phase, the tools used in the final copy must be documented. For example, if a chart undergoes several changes, the final copy of the chart should be documented with complete explanations.

In the implementation phase, every module of the code should be documented. In addition, the code should be self-documenting as far as possible using comments and descriptive headers.

Finally, the developers must carefully document the testing phase. Each type of test applied to the final product should be mentioned along with its result. Even unfavorable results and the data that produced them must be documented.

10.6.3 Technical documentation

Technical documentation describe the installation and the servicing of the software system. Installation documentation defines how the software should be installed on each computer, for example, servers and clients. Service documentation defines how the system should be maintained and updated if necessary.

10.7 END-CHAPTER MATERIALS

10.7.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Braude, E. *Software Engineering – An Object-Oriented Perspective*, New York: Wiley, 2001
- Gustafson, D. *Software Engineering*, New York: McGraw-Hill, 2002
- Lethbridge, T. and Laganiere, R. *Object-Oriented Software Engineering*, New York: McGraw-Hill, 2005

- Pressman, R. *Software Engineering: A Practitioner's Approach*, New York: McGraw-Hill, 2005
- Schach, S. *Object-Oriented and Classical Software Engineering*, New York: McGraw-Hill, 2007

10.7.2 Key terms

analysis phase 276	object-oriented design 280
attribute 280	operability 281
basis path testing 283	procedure-oriented analysis 276
black-box testing 285	procedure-oriented design 279
class diagram 278	software engineering 274
cohesion 280	software lifecycle 274
control structure testing 284	software quality 281
coupling 280	state chart 279
data flow diagram 276	state diagram 277
design phase 279	structure chart 279
development process 274	technical documentation 286
entity–relationship diagram 276	testability 282
glass-box testing 283	testing phase 283
implementation phase 280	transferability 282
incremental model 275	use-case diagram 277
maintainability 282	waterfall model 274
modularity 279	white-box testing 283
object-oriented analysis 277	

10.7.3 Summary

- The software lifecycle is a fundamental concept in software engineering. Software, like many other products, goes through a cycle of repeating phases.
- The development process in the software lifecycle involves four phases: analysis, design, implementation, and testing. Several models have been used in relation to these phases. We discussed the two most common: the waterfall model and the incremental model.
- The development process starts with the analysis phase. The analyst prepares a specification document that shows *what* the software will do without specifying *how* it will be done. The analysis phase can be done in two ways: procedure-oriented analysis and object-oriented analysis.
- The design phase defines *how* the system will accomplish what was defined in the analysis phase. In procedure-oriented design, the whole project is divided into a set

of procedure or modules. In object-oriented design, the design phase continues by elaborating the details of classes.

- Modularity means breaking a large project into smaller parts that can be understood and handled easily. Two issues are important when a system is divided into modules: coupling and cohesion. Coupling is a measure of how tightly two modules are bound to each other. Coupling between modules in a software system must be minimized. Cohesion is a measure of how closely the modules in a system are related. Cohesion between modules in a software system should be maximized.
- In the implementation phase, programmers write the code for the modules in procedure-oriented design, or write the program units to implement classes in the object-oriented design.
- The quality of software is important. Software quality can be divided into three broad measures: operability, maintainability, and transferability.
- The goal of the testing phase is to find errors. There are two types of testing: glass-box and black-box. Glass-box testing (or white-box testing) is based on knowing the internal structure of the software. Glass-box testing assumes that the tester knows everything. Black-box testing means testing the software without knowing what is inside it and without knowing how it works.

10.8 PRACTICE SET

10.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

10.8.2 Review questions

Q10-1. Define 'software lifecycle'.

Q10-2. Distinguish between the waterfall model and the incremental development models.

Q10-3. List the four phases in the development process.

Q10-4. Define the purpose of the analysis phase and describe two trends in this phase.

Q10-5. Define the purpose of the design phase and describe two trends in this phase.

Q10-6. Describe modularity and mention two issues related to modularity.

Q10-7. Distinguish between coupling and cohesion.

Q10-8. Define the purpose of the implementation phase and describe the issue of quality in this phase.

Q10-9. Define the purpose of the testing phase and list two categories of testing.

Q10-10. Distinguish between glass-box testing and black-box testing.

10.8.3 Problems

P10-1. In Chapter 9 we explained that the use of constant values are preferred to literals. What it the effect of this preference on the software lifecycle?

P10-2. In Chapter 9 we showed that communication between two modules can take place either by pass-by-value or pass-by-reference. Which method provides less coupling between the two modules?

P10-3. In Chapter 9 we showed that communication between two modules can take place either by pass-by-value or pass-by-reference. Which method provides more cohesion between the two modules?

P10-4. Draw a use-case diagram for a simple library.

P10-5. Draw a use-case diagram for a small grocery store.

P10-6. Show the data flow diagram for a simple mathematical formula $x + y$.

P10-7. Show the data flow diagram for a simple mathematical formula $x \times y + z \times t$.

P10-8. Show the data flow diagram for a library.

P10-9. Show the data flow diagram for a small groceries store.

P10-10. Create a structure chart for Problem P10-8.

P10-11. Create a structure chart for Problem P10-9.

P10-12. Show a state diagram for a stack of fixed capacity (see Chapter 12).

P10-13. Show a state diagram for a queue of fixed capacity (see Chapter 12).

P10-14. Create a class diagram for a library.

P10-15. Create a class diagram for a small grocery store.

P10-16. Show the details of classes in Problem P10-14.

P10-17. Show the details of classes in Problem P10-15.

P10-18. The input data to a program is made up of a combination of three integers in the range 1000 to 1999 (inclusive). Find the number of exhaustive tests to test all combinations of these numbers.

P10-19. List the boundary-value tests required for the Problem P10-18.

P10-20. A random number generator creates a number between 0 and 0.999. How can this random number generator be used to do random testing for the system described in Problem P10-18.

CHAPTER 11

Data Structure

In the preceding chapters, we used variables that store a single entity. Although single variables are used extensively in programming languages, they cannot be used to solve complex problems efficiently. In this chapter, we introduce data structures. This chapter is a prelude to the next chapter, in which we introduce abstract data types (ADTs).

A **data structure** uses a collection of related variables that can be accessed individually or as a whole. In other words, a data structure represents a set of data items that share a specific relationship. We discuss three data structures in this chapter: *arrays*, *records*, and *linked lists*. Most programming languages have an implicit implementation of the first two. The third, however, is simulated using pointers and records.

Objectives

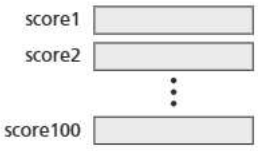
After studying this chapter, the student should be able to:

- Define a data structure.
- Define an array as a data structure and how it is used to store a list of data items.
- Distinguish between the name of an array and the names of the elements in an array.
- Describe operations defined for an array.
- Define a record as a data structure and how it is used to store attributes belonging to a single data element.
- Distinguish between the name of a record and the names of its fields.
- Define a linked list as a data structure and how it is implemented using pointers.
- Describe operations defined for a linked list.
- Compare and contrast arrays, records, and linked lists.
- Define the applications of arrays, records, and linked lists.

11.1 ARRAYS

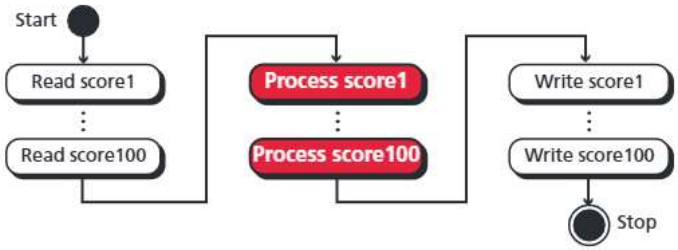
Imagine that we have 100 scores. We need to read them, process them, and print them. We must also keep these 100 scores in memory for the duration of the program. We can define a hundred *variables*, each with a different name, as shown in Figure 11.1.

Figure 11.1 *A hundred individual variables*



But having 100 different names creates other problems. We need 100 references to read them, 100 references to process them, and 100 references to write them. Figure 11.2 shows a diagram that illustrates this problem.

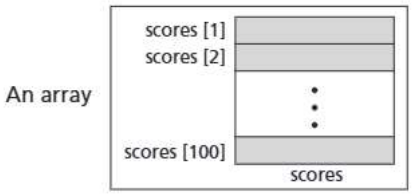
Figure 11.2 *Processing individual variables*



The number of instructions we need to handle even this relatively small number of scores is unacceptable. To process large amounts of data, we need a data structure such as an array.

An **array** is a sequenced collection of elements, of the same data type. We can refer to the elements in the array as the first element, the second element, and so forth until we get to the last element. If we were to put our 100 scores into an array, we could designate the elements as scores[1], scores[2], and so on. The **index** indicates the ordinal number of the element, counting from the beginning of the array. The elements of the array are individually addressed through their subscripts (Figure 11.3). The array as a whole has a name, *scores*, but each score can be accessed individually using its subscript.

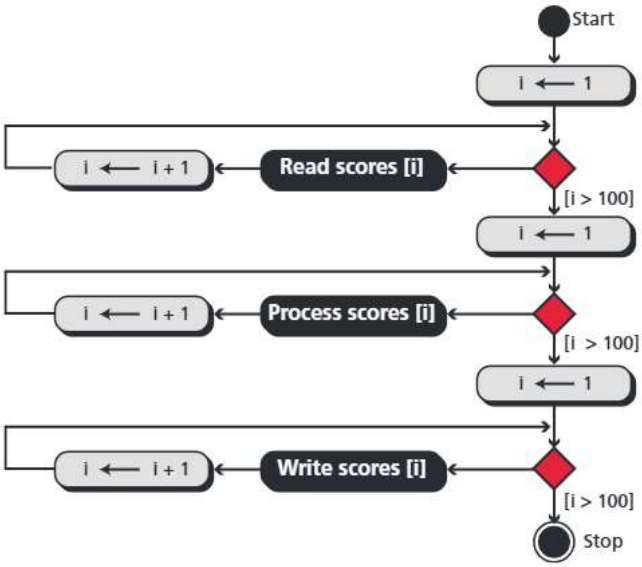
Figure 11.3 *Arrays with indexes*



We can use loops to read and write the elements in an array. We can also use loops to process elements. Now it does not matter if there are 100, 1000, or 10000 elements to be processed—loops make it easy to handle them all. We can use an integer variable to control the loop, and remain in the loop as long as the value of this variable is less than the total number of elements in the array (Figure 11.4).

We have used indexes that start from 1; some modern languages such as C, C++, and Java start indexes from 0.

Figure 11.4 *Processing an array*



Example 11.1

Compare the number of instructions needed to handle 100 individual elements in Figure 11.2 and the array with 100 in Figure 11.4. Assume that processing each score needs only one instruction.

https://sanet.st/blogs/polatebooks/

Solution

- In the first case, we need 100 instructions to read, 100 instructions to write, and 100 instructions to process. The total is 300 instructions.
- In the second case, we have three loops. In each loop we have two instructions, giving a total of six instructions. However, we also need three instructions for initializing the index and three instructions to check the value of the index. In total, we have 12 instructions.

Example 11.2

The number of cycles (fetch, decode, and execute phases) the computer needs to perform is not reduced if we use an array. The number of cycles is actually increased, because we have the extra overhead of initializing, incrementing, and testing the value of the index. But our concern is not the number of cycles: it is the number of lines we need to write the program.

Example 11.3

In computer science, one of the big issues is the reusability of programs—for example, how much needs to be changed if the number of data items is changed. Assume we have written two programs to process the scores as shown in Figure 11.2 and Figure 11.4. If the number of scores changes from 100 to 1000, how many changes do we need to make in each program?

In the first program we need to add $3 \times 900 = 2700$ instructions. In the second program, we only need to change three conditions ($I > 100$ to $I > 1000$). We can actually modify the diagram in Figure 11.4 to reduce the number of changes to one.

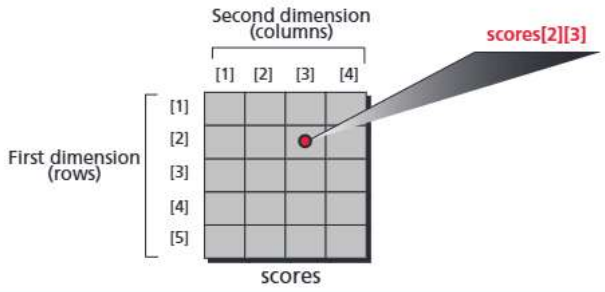
11.1.1 Array name versus element name

In an array we have two types of identifiers: the name of the array and the name of each individual element. The name of the array is the name of the whole structure, while the name of an element allows us to refer to that element. In the array of Figure 11.3, the name of the array is *scores* and the name of each element is the name of the array followed by the index, for example, scores[1], scores[2], and so on. In this chapter, we mostly need the names of the elements, but in some languages, such as C, we also need to use the name of the array.

11.1.2 Multidimensional arrays

The arrays discussed so far are known as **one-dimensional arrays** because the data is organized linearly in only one direction. Many applications require that data be stored in more than one dimension. One common example is a table, which is an array that consists of rows and columns. Figure 11.5 shows a table, which is commonly called a **two-dimensional array**.

Figure 11.5 *A two-dimensional array*



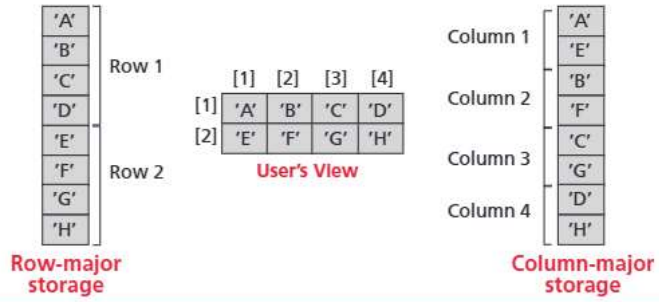
The array shown in Figure 11.5 holds the scores of students in a class. There are five students in the class and each student has four different scores for four quizzes. The variable scores[2][3] show the score of the second student in the third quiz. Arranging the scores in a two-dimensional array can help the teacher to find the average of scores for each student (the average over the row values) and find the average for each quiz (the average over the column values), as well as the average of all quizzes (the average of the whole table).

Multidimensional arrays—arrays with more than two dimensions—are also possible. However, we do not discuss arrays beyond two dimensions in this book.

11.1.3 Memory layout

The indexes in a one-dimensional array directly define the relative positions of the element in actual memory. A two-dimensional array, however, represents rows and columns. How each element is stored in memory depends on the computer. Most computers use **row-major storage**, in which an entire row of an array is stored in memory before the next row. However, a computer may store the array using **column-major storage**, in which the entire column is stored before the next column. Figure 11.6 shows a two-dimensional array and how it is stored in memory using row-major or column-major storage. Row-major storage is more common.

Figure 11.6 *Memory layout of arrays*



Example 11.4

We have stored the two-dimensional array *students* in the memory. The array is 100 × 4 (100 rows and 4 columns). Show the address of the element *students*[5][3] assuming that the element *student*[1][1] is stored in the memory location with address 1000 and each element occupies only one memory location. The computer uses row-major storage.

Solution

We can use the following formula to find the location of an element, assuming each element occupies one memory location:

$$y = x + Cols \times (i - 1) + (j - 1)$$

where x defines the start address, Cols defines the number of columns in the array, i defines the row number of the element, j defines the column number of the element, and y is the address we are looking for. In our example, x is 1000, Cols is 4, i is 5 and j is 3. We are looking for the value of y:

$$y = x + Cols \times (i - 1) + (j - 1) = 1000 + 4\,(5 - 1) + (3 - 1) = 1018$$

The answer makes sense because the element is at row 5 and column 3. There are four rows before this element that occupy 16 (4 × 4) memory locations. The previous two elements in row 5 have also occupied two memory locations. This means that all elements before the target elements occupy 18 memory locations. If the first element occupies the location 1000, the target element occupies the location 1018.

11.1.4 Operations on array

Although we can apply conventional operations defined for each element of an array (see Chapter 4), there are some operations that we can define on an array as a data structure. The common operations on arrays as structures are *searching*, *insertion*, *deletion*, *retrieval*, and *traversal*.

Searching for elements

We often need to find the index of an element when we know the value. This type of search was discussed in Chapter 8. We can use sequential search for unsorted arrays or binary search on sorted arrays. Searching is used for the next three operations.

Insertion of elements

Traditionally, computer languages require that the size of an array (the number of elements in the array) be defined at the time the program is written and prevent it from being changed during the execution of the program. Recently, some languages have allowed variable-size arrays. Even when the language allows variable-sized arrays, insertion of an element into an array needs careful attention.

Insertion at the end

If the insertion is at the end of an array and the language allows us to increase the size of the array, this can be done easily. For example, if an array has 30 elements, we increase the size of the array to 31 and insert the new item as the 31st item.

Insertion at the beginning or middle

If the insertion is to be at the beginning or in the middle of an array, the process is lengthy and time consuming. This happens when we need to insert an element in a sorted array. We first search the array, as described before. After finding the location of the insertion, we insert the new element. For example, if we want to insert an element as the 9th element in an array of 30 elements, elements 9 to 30 should be shifted one element towards the end of the array to open an empty element at position 9 for insertion. The following shows part of the pseudocode that needs to be applied to the array:

```
i ← 30
while (i ≥ 9)
{
  array [i + 1] ← array [i]
  i ← i – 1
}
array [i] ← newValue
```

Note that the shifting needs to take place from the end of the array to prevent losing the values of the elements. The code first copies the value of the 30th element into the 31st element, then copies the value of the 29th element into the 30th element, and so on. When the code comes out of the loop, the 9th element is already copied to the 10th element. The last line copies the value of the new item into the 9th element.

Deletion of elements

Deletion of an element in an array is as lengthy and involved as insertion. For example, if the ninth element should be deleted, we need to shift elements 10 to 30 one position towards the start of the array. We leave the pseudocode for this operation as an exercise, which is similar to the one for addition of an element.

Retrieving elements

Retrieving means randomly accessing an element for the purpose of inspecting or copying the data contained in the element. Unlike insertion and deletion operations, retrieving is an easy operation when a data structure is an array. In fact, an array is a *random-access* structure, which means that each element of an array can be accessed randomly without the need to access the elements before or after it. For example, if we want to retrieve the value of the 9th element in the array, we can do so using a single instruction, as shown below:

```
RetrievedValue ← array[9]
```

Traversal of arrays

Array traversal refers to an operation that is applied to all elements of the array, such as reading, writing, applying mathematical operation, and so on.

Algorithm 11.1 gives an example of finding the average of elements in an array whose elements are reals. The algorithm first finds the sum of the elements using a loop. After the loop is terminated, the average is calculated, which is the sum divided by the number of elements. Note that for proper calculation of the sum, the sum needs to be set to 0.0 before the loop.

Algorithm 11.1 Calculating the average of elements in an array

```
Algorithm: ArrayAverage (Array, n)
Purpose: Find the average value
Pre: Given the array Array and the number of elements, n.
Post: None
Return: The average value
{
```

Algorithm 11.1 Calculating the average of elements in an array (continued)

```
    sum ← 0.0
    i ← 1
    while (i ≤ n)
    {
        sum ← sum + Array [i]
        i ← i + 1
    }
    average ← sum / n
    Return ( average )
}
```

11.1.5 Strings

A **string**, as a set of characters, is treated in different languages differently. In C, a string is an array of characters. In C++, a string can be an array of characters, but there is a type named string. In Java, a string is a type.

11.1.6 Application

Thinking about the operations discussed in the previous section gives a clue to the application of arrays. If we have a list in which a lot of insertions and deletions are expected after the original list has been created, we should not use an array. An array is more suitable when the number of deletions and insertions is small, but a lot of **searching** and **retrieval** activities are expected.

An array is a suitable structure when a small number of insertions and deletions are required, but a lot of searching and retrieval is needed.

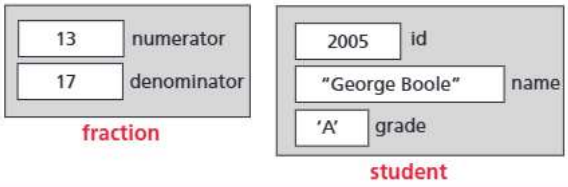
11.2 RECORDS

A **record** is a collection of related elements, possibly of different types, having a single name. Each element in a record is called a *field*. A **field** is the smallest element of named data that has meaning. A field has a type, and exists in memory. Fields can be assigned values, which in turn can be accessed for selection or manipulation. A field differs from a variable primarily in that it is part of a record.

Figure 11.7 contains two examples of records. The first example, *fraction*, has two fields, both of which are integers. The second example, *student*, has three fields made up of three different types.

The elements in a record can be of the same or different types, but all elements in the record must be related.

Figure 11.7 *Records*



The data in a record should all be related to one object. In Figure 11.7, the integers in the fraction both belong to the same fraction, and the data in the second example all relate to one student. (Note that we have placed string data between two double quotes and a single character between two single quotes. This is the convention used in most programming languages.)

11.2.1 Record name *versus* field name

Just like in an array, we have two types of identifier in a record: the name of the record and the name of each individual field inside the record. The name of the record is the name of the whole structure, while the name of each field allows us to refer to that field. For example, in the student record of Figure 11.7, the name of the record is *student*, the name of the fields are *student.id*, *student.name*, and *student.grade*. Most programming languages use a period (.) to separate the name of the structure (record) from the name of its components (fields). This is the convention we use in this book.

Example 11.5

The following shows how the value of fields in Figure 11.7 are stored:

```
student.id ← 2005    student.name ← "G. Boole"    student.grade ← 'A'
```

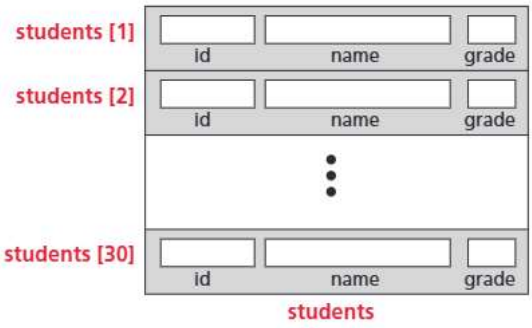
11.2.2 Comparison of records and arrays

We can conceptually compare an array with a record. This helps us to understand when we should use an array and when a record. An array defines a combination of elements, while a record defines the identifiable parts of an element. For example, an array can define a class of students (40 students), but a record defines different attributes of a student, such as id, name, or grade.

11.2.3 Array of records

If we need to define a combination of element and at the same time some attributes of each element, we can use an array of records. For example, in a class of 30 students, we can have an array of 30 records, each record representing a student. Figure 11.8 shows an array of 30 student records called *students*.

Figure 11.8 *Array of records*



In an array of records, the name of the array defines the whole structure, the group of students as a whole. To define each element, we need to use the corresponding index. To define the parts (attributes) of each element, we need to use the dot operator. In other words, first we need to define the element, then we can define part of that element. Therefore, the id of the third student is defined as:

```
(student[3]).id
```

Note that we use parentheses to emphasize that first a particular student should be chosen, then the id of that student. In other words, the parentheses tell us that the index operator has precedence over the dot operator. In some languages, there is no need to use parentheses, because the precedence is already established in the language itself, but using parentheses always guarantees the precedence.

Example 11.6

The following shows how we access the fields of each record in the students array to store values in them.

```
(students[1]).id ← 1001      (students[1]).name ← "J. Aron"      (students[1]).grade ← 'A'
(students[2]).id ← 2007      (students[2]).name ← "F. Bush"      (students[2]).grade ← 'F'
...                          ...                                 ...
(students[30]).id ← 3012     (students[30]).name ← "M. Blair"    (students[1]).grade ← 'B'
```

Example 11.7

However, we normally use a loop to read data into an array of record. Algorithm 11.2 shows part of the pseudocode for this process.

Algorithm 11.2 Part of the pseudocode to read student record

```
i ← 1
while (i < 31)
{
    read (students [i]).id
    read (students [i]).name
    read (students [i]).grade
    i ← i + 1
}
```

11.2.4 Arrays *versus* array of records

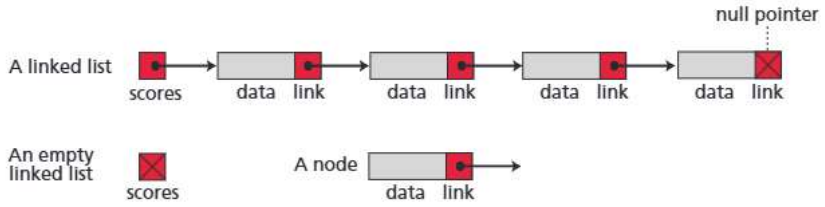
Both an array and an array of record represents a list of items. An array can be thought of as a special case of an array of records in which each element is a record with only a single field.

11.3 LINKED LISTS

A **linked list** is a collection of data in which each element contains the location of the next element—that is, each element contains two parts: data and **link**. The data part holds the value information: the data to be processed. The link is used to chain the data together, and contains a **pointer** (an address) that identifies the next element in the list. In addition, a pointer variable identifies the first element in the list. The name of the list is the same as the name of this pointer variable.

Figure 11.9 shows a linked list called *scores* that contains four elements. The link in each element, except the last, points to its successor. The link in the last element contains a **null pointer**, indicating the end of the list. We define an empty linked list to be only a null pointer: Figure 11.9 also shows an example of an empty linked list.

Figure 11.9 *Linked lists*



The elements in a linked list are traditionally called *nodes*. A **node** in a linked list is a record that has at least two fields: one contains the data, and the other contains the address of the next node in the sequence (the link). Figure 11.9 also shows a node.

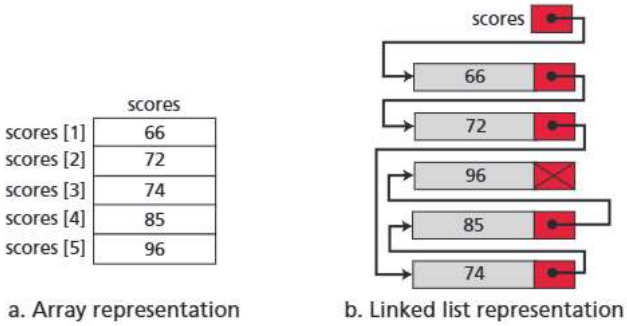
Before further discussion of linked lists, we need to explain the notation we use in the figures. We show the connection between two nodes using a line. One end of the line has an arrowhead, the other end has a solid circle. The arrowhead represents a copy of the address of the node to which the arrow head is pointed. The solid circle shows where this copy of the address is stored (Figure 11.10). The figure also shows that we can store a copy of the address in more than one place. For example, Figure 11.10 shows that two copies of the address are stored in two different locations. Understanding these concepts helps us to understand operations on a linked list better.

Figure 11.10 *The concept of copying and storing pointers*

11.3.1 Arrays *versus* linked lists

Both an array and a linked list are representations of a list of items in memory. The only difference is the way in which the items are linked together. In an array of records, the *linking tool* is the index. The element scores[3] is linked to the element scores[4] because the integer 4 comes after the integer 3. In a linked list, the *linking tool* is the link that points to the next element—the pointer or the address of the next element. Figure 11.11 compares the two representations for a list of five integers.

Figure 11.11 *Array* versus *linked list*



a. Array representation b. Linked list representation

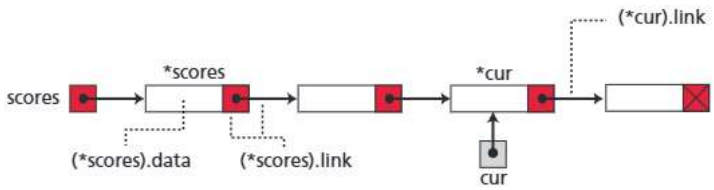
The elements of an array are stored one after another in the memory without a gap in between: the list is contiguous. The nodes of a linked list can be stored with gaps between them: the link part of the node 'glues' the items together. In other words, the computer has the option to store them contiguously or spread the nodes through the whole memory. This has an advantage: insertion and deletion in a linked list is much easier. The only thing that needs to be changed is the pointer to the address of the next element. However, this comes with an overhead: each node of a linked list has an extra field, the address of the next node in memory.

11.3.2 Linked list names *versus* nodes names

As for arrays and records, we need to distinguish between the name of the linked list and the names of the nodes, the elements of a linked list. A linked list must have a name.

The name of a linked list is the name of the head pointer that points to the first node of the list. Nodes, on the other hand, do not have explicit names in a linked list, just implicit ones. The name of a node is related to the name of the pointer that points to the node. Different languages handle the relation between the pointer and the node to which the pointer points differently. We use the convention used in the C language. If the pointer that points to a node is called **p**, for example, we call the node ***p**. Since the node is a record, we can access the fields inside the node using the name of the node. For example, the data part and the link part of a node pointed by a pointer p can be called **(*p).data** and **(*p).link**. This naming convention implies that a node can have more than one name. Figure 11.12 shows the name of the linked list and the names of the nodes.

Figure 11.12 *The name of a linked list* versus *the names of nodes*



11.3.3 Operations on linked lists

The same operations we defined for an array can be applied to a linked list.

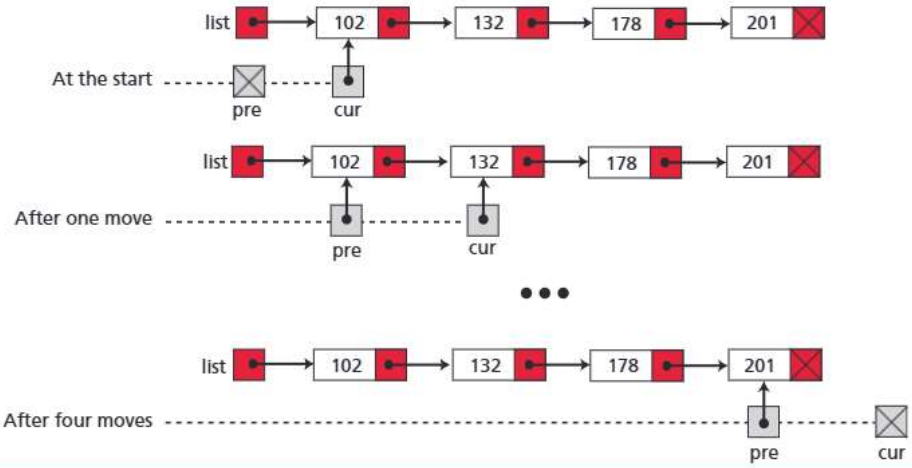
Searching a linked list

The search algorithm for a linked list can only be sequential (see Chapter 8) because the nodes in a linked list have no specific names (unlike the elements in an array) that can be found using a binary search. However, since nodes in a linked list have no names, we use two pointers, *pre* (for previous) and *cur* (for current).

At the beginning of the search, the *pre* pointer is null and the *cur* pointer points to the first node. The search algorithm moves the two pointers together towards the end of

the list. Figure 11.13 shows the movement of these two pointers through the list in an extreme case scenario: when the target value is larger than any value in the list. For example, in the five-node list, assume that our target value is 220, which is larger than any value in the list.

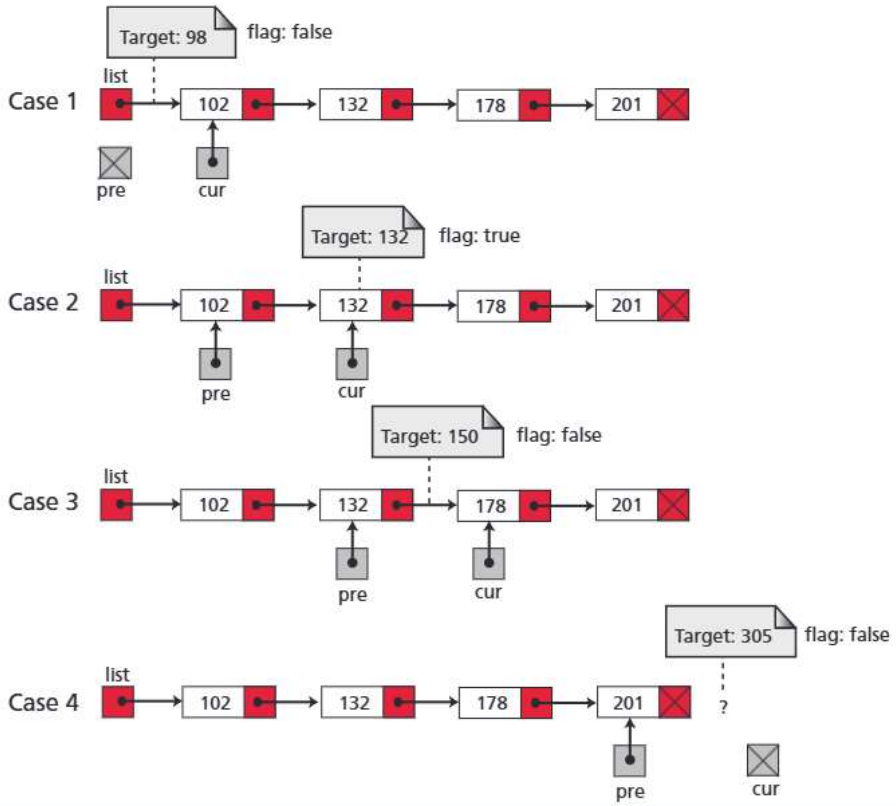
Figure 11.13 *Moving of pre and cur pointers in searching a linked list*



However, other situations can occur. The value of the target can be less that data value in the first node, or it can be equal to one of the data values in one of the nodes, and so on. In all situations, however, when the search stops, the *cur* pointer points to the node that stops the search and the *pre* pointer points to the previous node. If the target is found, the *cur* pointer points to the node that holds the target value. If the target value is not found, the *cur* pointer points to the node with a value larger than the target value. In other words, since the list is sorted, and may be very long, we never allow the two pointers to reach the end of the list if we are sure that we have passed the target value. The searching algorithm uses a flag (a variable that can take only *true* or *false* values). When the target is found, the flag is set to *true*: when the target is not found, the flag is set to *false*. When the flag is *true* the *cur* pointer points to the target value: when the flag is *false*, the *cur* pointer points to a value larger than the target value.

Figure 11.14 shows some different situations. In the first case, the target is 98. This value does not exist in the list and is smaller than any value in the list, so the search algorithm stops while *pre* is null and *cur* points to the first node. The value of the flag is false because the value was not found. In the second case, the target is 132, which is the value of the second node. The search algorithm stops while *cur* is pointing to the second node and *pre* is pointing to the first node. The value of the flag is true because the target is found. In the third and the fourth cases, the targets are not found so the value of the flag is *false*.

Figure 11.14 *Values of pre and cur pointers in different cases*



Algorithm 11.3 shows a simplified algorithm for the search. We need more conditions on the *while* loop, but we leave that for more advanced discussion of linked lists. Note how we move the two pointers forward together. In each move, we have:

pre ← cur and cur ← (*cur).link

This guarantees that the two pointers move together. The first assignment makes a copy of *cur* and stores it in *pre*. This means *pre* is taking the previous value of *cur*. In the second assignment, the node pointed by *cur* is selected and the value of its link field is copied and stored in *cur* (see Figure 11.12 for clarification). The search algorithm is used both by the insertion algorithm (if the target is not found) and by the delete algorithm (if the target is found).

Algorithm 11.3 Searching a linked list

```
Algorithm: SearchLinkedList
Purpose: Search the list using two pointers: pre and cur.
Pre: The linked list (head pointer) and target value
Post: None
Return: The position of pre and cur pointers and the value of the flag
{
    pre ← null
    cur ← list
    while (target < (*cur).data)
    {
        pre ← cur
        cur ← (*cur).link
    }
    if ((*cur).data = target)   flag ← true
    else   flag ← false
}
```

Inserting a node

Before insertion into a linked list, we first apply the searching algorithm. If the flag returned from the searching algorithm is false, we will allow insertion, otherwise we abort the insertion algorithm, because we do not allow data with duplicate values. Four cases can arise:

- Inserting into an empty list.
- Insertion at the beginning of the list.
- Insertion at the end of the list.
- Insertion in the middle of the list.

Insertion into an empty list

If the list is empty (list = null), the new item is inserted as the first element. One statement can do the job:

```
list ← new
```

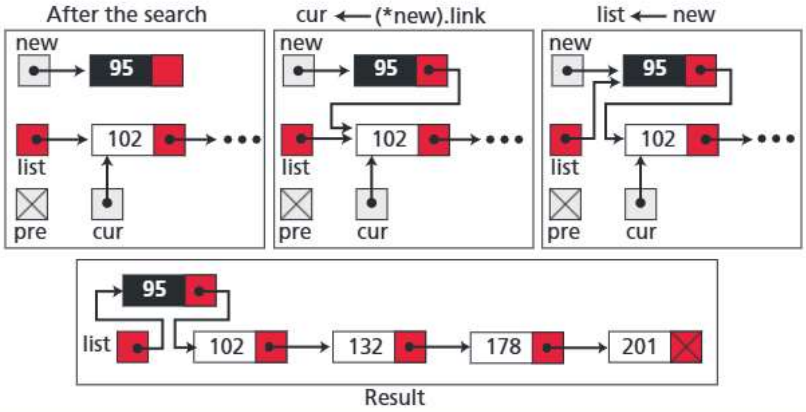
Insertion at the beginning

If the searching algorithm returns a flag with value of *false* and the value of the *pre* pointer is null, the data needs to be inserted at the beginning of the list. Two statements are needed to do the job:

(*new).link ← cur and list ← new

The first assignment makes the new node become the predecessor of the previous first node. The second statement makes the newly connected node the first node. Figure 11.15 shows the situation.

Figure 11.15 *Inserting a node at the beginning of a linked list*



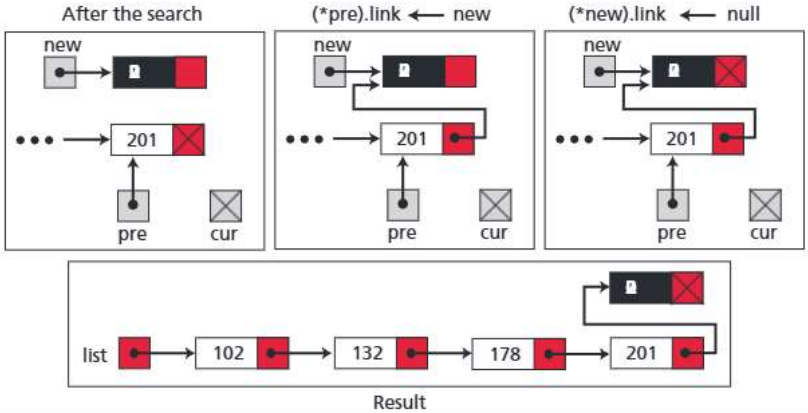
Insertion at the end

If the searching algorithm returns a flag with value of *false* and the value of the *cur* pointer is null, the data needs to be inserted at the end of the list. Two statements are needed to do the job:

(*pre).link ← new and (*new).link ← null

The first assignment connects the new node to the previous last node. The second statement makes the newly connected node become the last node. Figure 11.16 shows the situation.

Figure 11.16 *Inserting a node at the end of the linked list*



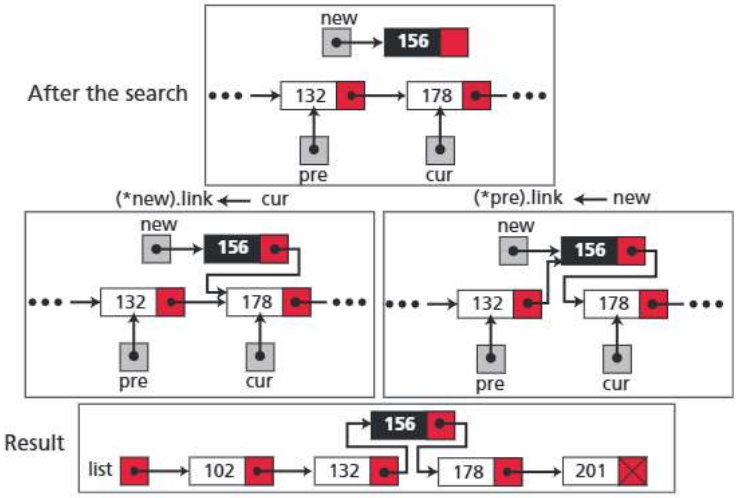
Insertion in the middle

If the searching algorithm returns a flag with a value of *false* and none of the returned pointers are null, the new data needs to be inserted at the middle of the list. Two statements are needed to do the job:

(*new).link ← cur and (*pre).link ← new

The first assignment connects the new node to its successor. The second statement connects the new node to its predecessor. Figure 11.17 shows the situation.

Figure 11.17 *Inserting a node at the middle of the linked list*



Algorithm 11.4 shows the pseudocode for inserting a new node in a linked list. The first section just adds a node to an empty list.

Algorithm 11.4 Inserting a node in a linked list

```
Algorithm: InsertLinkedList (list, target, new)
Purpose: Insert a node in the link list after searching the list
Pre: The linked list and the target data to be inserted
Post: None
Return: The new linked list
{
    searchlinkedlist (list, target, pre, cur, flag)
    // Given target and returning pre, cur, and flag
    if (flag = true)  // No duplicate
    {
```

Algorithm 11.4 Inserting a node in a linked list (continued)

```
        return list
    }
    if (list = null    // Insert into empty list
    {
        list ← new
    }
    if (pre = null)     // Insertion at the beginning
    {
        (*new).link ← cur
        list ← new
        return list
    }
    if (cur = null)    // Insertion at the end
    {
        (*pre).link ← new
        (*new).link ← null
        return list
    }
        (*new).link ← cur   // Insertion in the middle
        (*pre).link ← new
        return list
}
```

Deleting a node

Before deleting a node in a linked list, we apply the search algorithm. If the flag returned from the search algorithm is true (the node is found), we can delete the node from the linked list. However, deletion is simpler than insertion: we have only two cases—deleting the first node and deleting any other node. In other words, the deletion of the last and the middle nodes can be done by the same process.

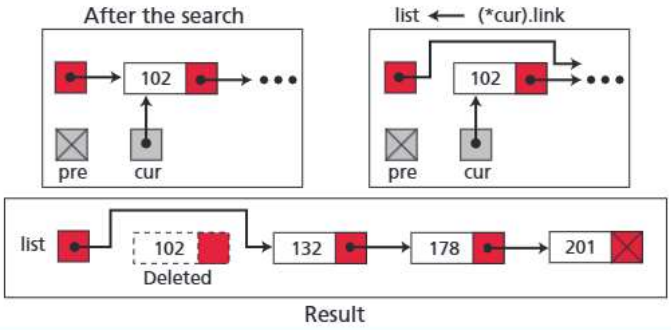
Deleting the first node

If the *pre* pointer is null, the first node is to be deleted. The *cur* pointer points to the first node and deleting can be done by one statement:

```
list ← (*cur).link
```

The statement connects the second node to the list pointer, which means that the first node is deleted. Figure 11.18 shows the case.

Figure 11.18 *Deleting the first node of a linked list*



Deleting the middle or the last node

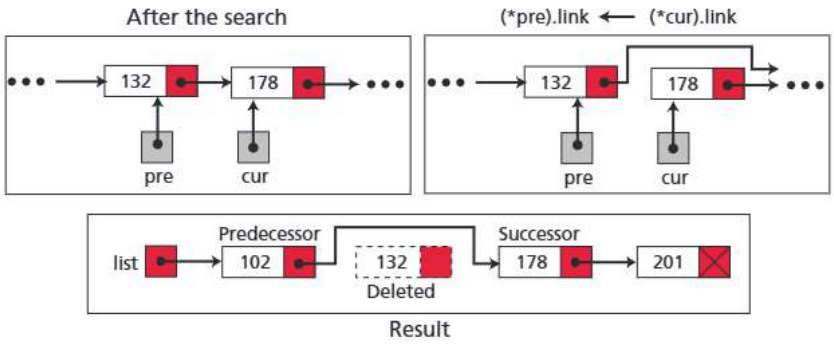
If neither of the pointers is null, the node to be deleted is either a middle node or the last node. The *cur* pointer points to the corresponding node and deleting can be done by one statement:

```
(*pre).link ← (*cur).link
```

The statement connects the successor node to the predecessor node, which means that the current node is deleted. Figure 11.19 shows the case.

Algorithm 11.5 shows the pseudocode for deleting a node. The algorithm is much simpler than the one for inserting. We have only two cases and each case needs only one statement.

Figure 11.19 *Deleting a node at the middle or end of a linked list*



Algorithm 11.5 Deleting a node in a linked list

```
Algorithm: DeleteLinkedList (list, target)
Purpose: Delete a node in a linked list after searching the list for the right node
Pre: The linked list and the target data to be deleted
Post: None
Return: The new linked list

{
     // Given target and returning pre, cur, and flag
    searchlinkedlist (list, target, pre, cur, flag)
    if (flag = false)
    {
    return list      // The node to be deleted not found
    }
    if (pre = null)       // Deleting the first node
    {
        list ← (*cur).link
        return list
    }
    (*pre).link ← (*cur).link   // Deleting other nodes
    return list
}
```

Retrieving a node

Retrieving means randomly accessing a node for the purpose of inspecting or copying the data contained in the node. Before retrieving, the linked list needs to be searched. If the data item is found, it is retrieved, otherwise the process is aborted. Retrieving uses only the *cur* pointer, which points to the node found by the search algorithm. Algorithm 11.6 shows the pseudocode for retrieving the data in a node. The algorithm is much simpler than the insertion or deletion algorithm.

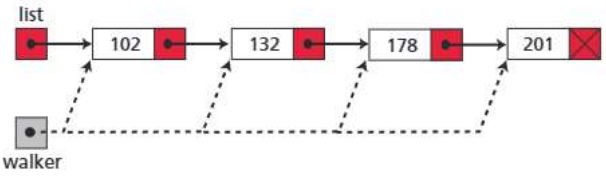
Algorithm 11.6 Retrieving a node in a linked list

```
Algorithm: RetrieveLinkedList (list, target)
Purpose: Retrieves the data in a node after searching the list for the right node
Pre: The linked list (head pointer) and the target (data to be retrieved)
Post: None
Return: Return the data retrieved

{
    searchlinkedlist (list, target, pre, cur, flag)
    if (flag = false)     // The node not found
    {
        return error
    }
    return (*cur).data
}
```

Traversing a linked list

To traverse the list, we need a 'walking' pointer, which is a pointer that moves from node to node as each element is processed. We start traversing by setting the walking pointer to the first node in the list. Then, using a loop, we continue until all of the data has been processed. Each iteration of the loop processes the current node, then advances the walking pointer to the next node. When the last node has been processed, the walking pointer becomes null and the loop terminates (Figure 11.20).

Figure 11.20 *Traversing a linked list*



Algorithm 11.7 shows the pseudocode for traversing a linked list.

Algorithm 11.7 Traversing a linked list

```
Algorithm: TraverseLinkedList (list)
Purpose: Traverse a linked list and process each data item
Pre: The linked list (head pointer)
Post: None
Return: The list

{
    walker ← list
    while (walker ≠ null)
    {
        Process (*walker).data
        walker ← (*walker).link
    }
    return list
}
```

11.3.4 Applications of linked lists

A linked list is a very efficient data structure for storing data that will go through many insertions and deletions. A linked list is a dynamic data structure in which the list can start with no nodes and then grow as new nodes are needed. A node can be easily deleted without moving other nodes, as would be the case with an array. For example, a linked list could be used to hold the records of students in a school. Each quarter or semester, new students enroll in the school and some students leave or graduate.

A linked list can grow infinitely and can shrink to an empty list. The overhead is to hold an extra field for each node. A linked list, however, is not a good candidate for data that must be searched often. This appears to be a dilemma, because each deletion or insertion needs a search. We will see that some abstract data types, discussed in the next chapter, have the advantages of an array for searching and the advantages of a link list for insertion and deletion.

A linked list is a suitable structure if a large number of insertions and deletions are needed, but searching a linked list is slower that searching an array.

11.4 END-CHAPTER MATERIALS

11.4.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Gilberg, R. and Forouzan, B. *Data Structures – A Pseudocode Approach with* C, Boston, MA: Course Technology, 2005
- Goodrich, M. and Tamassia, R. *Data Structures and Algorithms in Java*, New York: Wiley, 2005
- Neapolitan, R. and Naimipour, K. *Foundations of Algorithms Using* C++ *Pseudocode*, Sudbury, MA: Jones and Bartlett, 2004
- Main, M. and Savitch, W. *Data Structures and Other Objects Using* C++, Reading, MA: Addison-Wesley, 2004
- Standish, T. *Data Structures, Algorithms, and Software Principles*, Reading, MA: Addison-Wesley, 1994

11.4.2 Key terms

array 292	null pointer 301
column-major storage 295	one-dimensional array 294
data structure 291	pointer 301
field 298	record 298
index 292	retrieval 298
link 301	row-major storage 295
linked list 301	searching 298
multidimensional array 295	two-dimensional array 294
node 302	string 298

11.4.3 Summary

- A data structure uses a collection of related variables that can be accessed individually or as a whole. In other words, a data structure represents a set of data items that share a specific relationship. We discussed three data structures in this chapter: *arrays*, *records*, and *linked lists*.
- An array is a sequenced collection of elements normally of the same data type. We use indexes to refer to the elements of an array. In an array we have two types of identifiers: the name of the array and the name of each individual element.
- Many applications require that data is stored in more than one dimension. One common example is a table, which is an array that consists of rows and columns.

Two-dimensional arrays can be stored in memory using either row-major or column-major storage. The first is more common.

- The common operations on arrays as a structure are *searching*, *insertion*, *deletion*, *retrieval*, and *traversal*. An array is a suitable structure in applications where the number of deletions and insertions is small but a lot of searching and **retrieval** operations are required. An array is normally a static data structure and so is more suitable when the number of data items is fixed.
- A record is a collection of related elements, possibly of different types, having a single name. Each element in a record is called a *field*. A field is the smallest element of named data that has meaning in a record.
- A string is a set of characters that is treated like an array in some languages and as a type in others.
- A linked list is a collection of data in which each element contains the location of the next element; that is, each element contains two parts: *data* and *link*. The data part holds the useful information: the data to be processed. The link is used to chain the data together.
- The same operations defined for an array can be applied to a linked list. A linked list is a very efficient structure for data that will go through many insertions and deletions. A linked list is a dynamic data structure in which the list can start with no nodes and grow as new nodes are needed.

11.5 PRACTICE SET

11.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

11.5.2 Review questions

Q11-1. Name three types of data structures.
Q11-2. How is an element in an array different from an element in a record?
Q11-3. How is an element in an array different from an element in a linked list?
Q11-4. Why should we use indexes rather than subscripts to identify array elements?
Q11-5. How are the elements of an array stored in memory?
Q11-6. What is the definition of a field in a record?
Q11-7. What are the fields of a node in a linked list?
Q11-8. What is the function of the pointer in a linked list?
Q11-9. How do you point to the first node in a linked list?
Q11-10. What is the value of the link field in the last node of a linked list?

11.5.3 Problems

P11-1. There are two arrays, A and B, each of 10 integers. Write an algorithm that tests if every element of array A is equal to its corresponding element in array B.

P11-2. Write an algorithm that reverses the elements of an array so that the last element becomes the first, the second to the last becomes the second, and so forth.

P11-3. Write an algorithm to print the contents of a two-dimensional array of R rows and C columns.

P11-4. Write an algorithm to apply sequential search on an array of N elements.

P11-5. Write an algorithm to apply binary search on an array of N elements.

P11-6. Write an algorithm to insert an element into a sorted array. The algorithm must call a search algorithm to find the location for insertion.

P11-7. Write an algorithm to delete an element in a sorted array. The algorithm must call a search algorithm to find the location of insertion.

P11-8. Write an algorithm to multiply each element of an array by a constant.

P11-9. Write an algorithm to add a fraction (Fr1) to another fraction (Fr2).

P11-10. Write an algorithm to subtract a fraction (Fr1) from another fraction (Fr2).

P11-11. Write an algorithm to multiply a fraction (Fr1) by another fraction (Fr2).

P11-12. Write an algorithm to divide a fraction (Fr1) by another fraction (Fr2).

P11-13. Draw a diagram to show a linked list in which the data part is a student record with three fields: *id*, *name*, and *grade*.

P11-14. Show how the delete algorithm for a linked list (Algorithm 11.4, in section 11.3.3) can delete the only node in a linked list.

P11-15. Show how the insertion algorithm for a linked list (Algorithm 11.3, in section 11.3.3) can add a node to an empty linked list.

P11-16. Show how we can build a linked list from scratch using the insertion algorithm (Algorithm 11.3, in section 11.3.3).

P11-17. Write an algorithm to find the average of the numbers in a linked list of numbers.

P11-18. Show what happens if we apply the following statements to the linked list in Figure 11.9.

```
scores ← (*scores).link
```

P11-19. Show what happens if we apply the following statements to the linked list in Figure 11.13.

```
cur ← (*cur).link    and    pre ← (*pre).link
```

CHAPTER 12

Abstract Data Types

In this chapter we discuss abstract data types (ADTs), which are data types at a higher level of abstraction than the data structures we discussed in Chapter 11. ADTs use data structures for implementation. We begin this chapter with a brief background on ADTs. We then give a definition and propose a model. The remainder of the chapter discusses various ADTs, such as stacks, queues, general linear lists, trees, binary trees, binary search trees, and graphs.

Objectives

After studying this chapter, the student should be able to:

- Define the concept of an abstract data type (ADT).
- Define a stack, the basic operations on stacks, their applications, and how they can be implemented.
- Define a queue, the basic operations on queues, their applications, and how they can be implemented.
- Define a general linear list, the basic operations on lists, their applications, and how they can be implemented.
- Define a general tree and its application.
- Define a binary tree—a special kind of tree—and its applications.
- Define a binary search tree (BST) and its applications.
- Define a graph and its applications.

12.1 BACKGROUND

Problem solving with a computer means processing data. To process data, we need to define the data type and the operation to be performed on the data. For example, to find the sum of a list of numbers, we should select the type for the number (integer or real) and define the operation (addition). The definition of the data type and the definition of the operation to be applied to the data is part of the idea behind an **abstract data type** (ADT)—to hide how the operation is performed on the data. In other word, the user of an ADT needs only to know that a set of operations are available for the data type, but does not need to know how they are applied.

12.1.1 Simple ADTs

Many programming languages already define some simple ADTs as integral parts of the language. For example, the C language defines a simple ADT called an *integer*. The type of this ADT is integer with predefined ranges. C also defines several operations that can be applied on this data type (addition, subtraction, multiplication, division, and so on). C explicitly defines these operations on integers and what we expect as the results. A programmer who writes a C program to add two integers should know about the integer ADT and the operations that can be applied to it.

The programmer, however, does not need to know *how* these operations are actually implemented. For example, the programmer uses the expression $z \leftarrow x + y$ and expects the value of x (an integer) to be added to the value of y (an integer) and the result to be named z (an integer). The programmer does not need to know how the addition is performed. We learned in previous chapters that the way this addition is done by a computer is to store the two integers in two memory locations in two's complement format, to load them into the CPU register, to add them in binary, and to store the result back to another memory location. The programmer, however, does not need to know this. An integer in C is a simple abstract data type with predefined operations. How the operations are performed is not a concern for the programmer.

12.1.2 Complex ADTs

Although several simple ADTs, such as integer, real, character, pointer, and so on, have been implemented and are available for use in most languages, many useful complex ADTs are not. As we will see in this chapter, we need a list ADT, a stack ADT, a queue ADT, and so on. To be efficient, these ADTs should be created and stored in the library of the computer to be used. The user of a *list*, for example, should only need to know what operations are available for the list, not how these operations are performed.

Therefore, with an ADT, users are not concerned with *how* the task is done, but rather with *what* it can do. In other words, the ADT consists of a set of definitions that allow programmers to use the operation while their implementation is hidden. This generalization of operations with unspecified implementations is known as abstraction. We abstract the essence of the process and leave the implementation details hidden.

The concept of abstraction means:
1. **We know what a data type can do.**
2. **How it is done is hidden.**

12.1.3 Definition

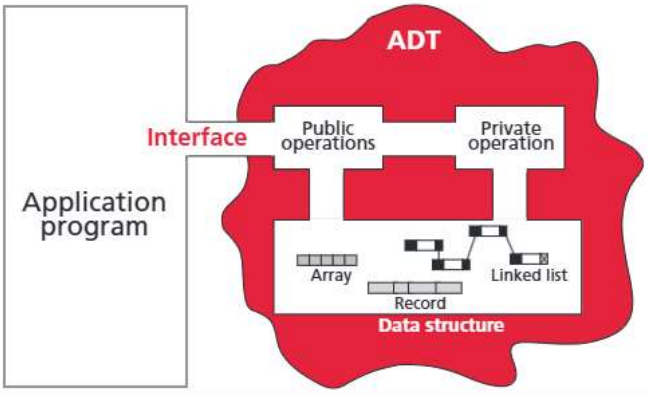
Let us now define an ADT. An abstract data type is a data type packaged with the operations that are meaningful for the data type. We then encapsulate the data and the operations on the data and hide them from the user.

Abstract Data Type
1. **Definition of data**
2. **Definition of operations**
3. **Encapsulation of data and operation**

12.1.4 Model for an abstract data type

The ADT model is shown in Figure 12.1. The colored area with an irregular outline represents the ADT. Inside the ADT are two different parts of the model: *data structure* and *operations* (public and private). The application program can only access the public operations through the **interface**. An interface is a list of public operations and data to be passed to or returned from those operations. The private operations are for internal use by the ADT. The data structures, such as arrays and linked lists, are inside the ADT and are used by the public and private operations.

Figure 12.1 *The model for an ADT*



Although the public operations and the interface should be independent of the implementation, the private operations are dependent on the data structures chosen during the implementation of the ADT. We will elaborate on this issue when we discuss some of the ADTs.

12.1.5 Implementation

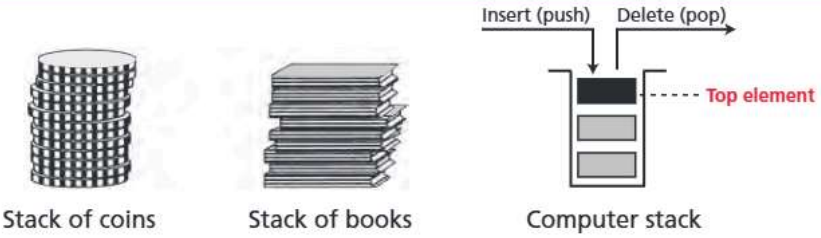
Computer languages do not provide ADT packages. To use an ADT, it is first implemented and kept in a library. The main purpose of this chapter is to introduce some common ADTs and their applications. However, we also give a brief discussion of each ADT implementation for the interested reader. We leave the pseudocode algorithms of the implementations as challenging exercises.

12.2 STACKS

A **stack** is a restricted **linear list** in which all additions and deletions are made at one end, the top. If we insert a series of data into a stack and then remove it, the order of the data is reversed. Data input as 5, 10, 15, 20, for example, would be removed as 20, 15, 10, and 5. This reversing attribute is why stacks are known as a **last in, first out (LIFO)** data structure.

We use many different types of stacks in our daily lives. We often talk of a stack of coins or a stack of books. Any situation in which we can only add or remove an object at the top is a stack. If we want to remove an object other than the one at the top, we must first remove all objects above it. Figure 12.2 shows three representations of stacks.

Figure 12.2 *Three representations of stacks*



12.2.1 Operations on stacks

Although we can define many operations for a stack, there are four basic operations, *stack*, *push*, *pop*, and *empty*, which we define in this chapter.

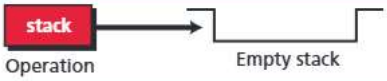
The stack operation

The stack operation creates an empty stack. The following shows the format:

```
stack (stackName)
```

stackName is the name of the stack to be created. This operation returns an empty stack. Figure 12.3 shows the pictorial representation of this operation.

Figure 12.3 *Stack operation*



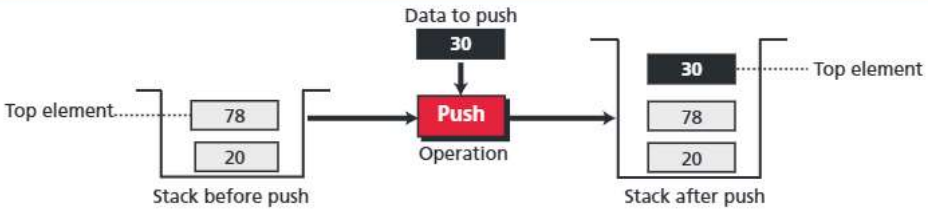
The push operation

The **push** operation inserts an item at the top of the stack. The following shows the format:

```
push (stackName, dataItem)
```

stackName is the name of the stack and *dataItem* is the data to be inserted at the top of the stack. After the push operation, the new item becomes the top item in the stack. This operation returns the new stack with *dataItem* inserted at the top. Figure 12.4 shows the pictorial representation of this operation.

Figure 12.4 *Push operation*



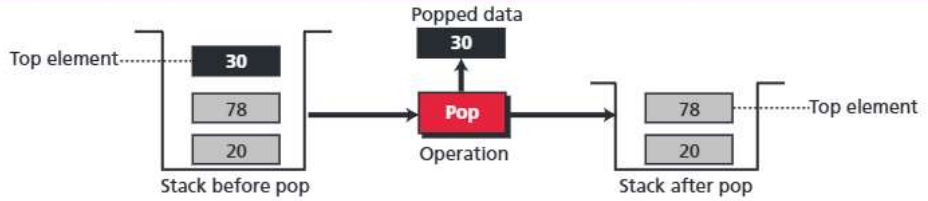
The pop operation

The **pop** operation deletes the item at the top of the stack. The following shows the format:

```
pop (stackName, dataItem)
```

stackName is the name of the stack and *dataItem* is the data that is deleted from the stack. Figure 12.5 shows the pictorial representation of this operation.

Figure 12.5 *Pop operation*



The deleted item can be used by the application program or can be just discarded. After the pop operation, the item that was under the top element before the deletion becomes the top element. This operation returns the new stack with one less element.

The empty operation

The *empty* operation checks the status of the stack. The following shows the format:

```
empty (stackName)
```

The *stackName* is the name of the stack. This operation returns *true* if the stack is empty and *false* if the stack is not empty.

12.2.2 Stack ADT

We define a stack as an ADT as shown below:

Stack ADT

Definition

A list of data items that can only be accessed at one end, called the top.

Operations

stack: Creates an empty stack.

push: Inserts an element at the top.

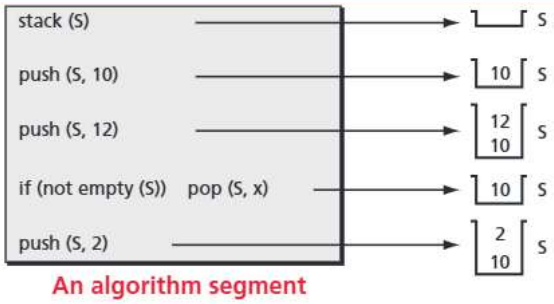
pop: Deletes the top element.

empty: Checks the status of the stack.

Example 12.1

Figure 12.6 shows a segment of an algorithm that applies the previously defined operations on a stack *S*. The fourth operation checks the status of the stack before trying to pop the top element. The value of the top element is stored in the variable *x*. However, we do not use this value: it will be automatically discarded at the end of the algorithm segment.

Figure 12.6 *Example 12.1*



An algorithm segment

12.2.3 Stack applications

Stack applications can be classified into four broad categories: reversing data, pairing data, postponing data usage, and backtracking steps. We discuss the first two in the sections that follow.

Reversing data items

Reversing data items requires that a given set of data items be reordered so that the first and last items are exchanged, with all of the positions between the first and last being relatively exchanged also. For example, the list (2, 4, 7, 1, 6, 8) becomes (8, 6, 1, 7, 4, 2).

Example 12.2

In Chapter 2 (Figure 2.6 in section 2.2.6) we gave a simple UML diagram to convert an integer from decimal to any base. Although the algorithm is very simple, if we print the digits of the converted integer as they are created, we will get the digits in reverse order. The print instruction in any computer language prints characters from left to right, but the algorithm creates the digits from right to left. We can use the reversing characteristic of a stack (LIFO structure) to solve the problem.

Algorithm 12.1 shows the pseudocode to convert a decimal integer to binary and print the result.

Algorithm 12.1 Example 2.2

```
Algorithm: DecimalToBinary (number)
Purpose: Print the binary equivalent of a given integer (absolute value)
Pre: Given the integer to be converted (number)
Post: The binary integer is printed
Return: None
{
    stack (S)
    while (number ≠ 0)
    {
        remainder ← number mod 2
        push (S, remainder)
        number ← number / 2
    }
    while (not empty (S))
    {
        pop (S, x)
        print (x)
    }
    return
}
```

We create an empty stack first. Then we use a *while* loop to create the bits, but instead of printing them, we push them into the stack. When all bits are created, we exit the loop. Now we use another loop to pop the bits from the stack and print them. Note that the bits are printed in the reverse order to that in which they have been created.

Pairing data items

We often need to pair some characters in an expression. For example, when we write a mathematical expression in a computer language, we often need to use parentheses to change the precedence of operators. The following two expressions are evaluated differently because of the parentheses in the second expression:

$$3 \times 6 + 2 = 20 \qquad 3 \times (6 + 2) = 24$$

In the first expression, the multiplication operator has precedence over the addition operator—it is calculated first. In the second expression, the parentheses ignore the precedence, so the addition is calculated first. When we type an expression with a lot of parentheses, we often forget to pair the parentheses. One of the duties of a compiler is to do the checking for us. The compiler uses a stack to check that all opening parentheses are paired with a closing parentheses.

Example 12.3

Algorithm 12.2 shows how we can check if every opening parenthesis is paired with a closing parenthesis.

Algorithm 12.2 Example 12.3

```
Algorithm: CheckingParentheses (expression)
Purpose: Check the pairing of parentheses in an expression
Pre: Given the expression to be checked
Post: Error messages if unpaired parentheses are found
Return: None
{
    stack (S)
    while (more character in the expression)
    {
        Char ← next character
        if (Char = '(')
        {
            push (S, Char)
        }
        else
        {
            if (Char = ')')
            {
                if (empty (S))
```

Algorithm 12.2 Example 12.3 (continued)

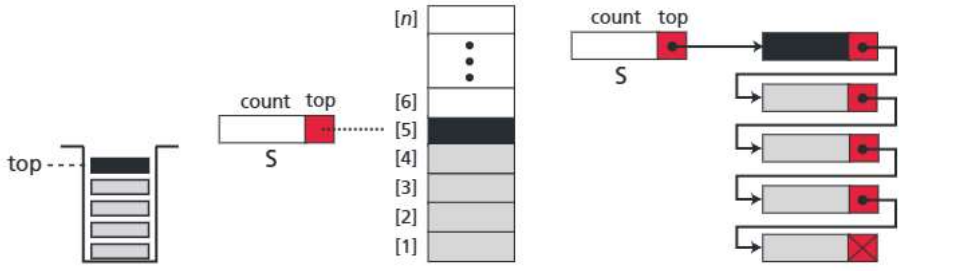
```
                {
                    print (unmatched opening parenthesis)
                }
                else
                {
                    pop (S, x)
                }
            }
        }
    }
    if (not empty (S))
    {
        print (a closing parenthesis not matched)
    }
    return
}
```

12.2.4 Stack implementation

In this section we describe the general ideas behind the implementation of a stack ADT. At the ADT level, we use the stack and its four operations (*stack*, *push*, *pop*, and *empty*): at the implementation level, we need to choose a data structure to implement it. Stack ADT can be implemented using either an array or a linked list. Figure 12.7 shows an example of a stack ADT with five items. The figure also shows how we can implement the stack.

In our array implementation, we have a record that has two fields. The first field can be used to store information about the array: we have used it as the count field, which at each moment shows the number of data items in the stack. The second field is an integer that holds the index of the top element. Note that the array is shown upside down to match the linked list implementation.

Figure 12.7 *Stack implementations*



The linked list implementation is similar: we have an extra node that has the name of the stack. This node also has two fields: a counter and a pointer that points to the top element.

Algorithms

We can write four algorithms in pseudocode for the four operations we defined for stack in each implementation. We showed algorithms to handle arrays and linked lists in Chapter 11: these algorithms can be modified to create the four algorithms we need for stacks: *stack*, *push*, *pop*, and *empty*. These algorithms are even easier than those presented in Chapter 11, because the insertion and deletion is done only at the top of stack. We leave the writing of these algorithms as exercises.

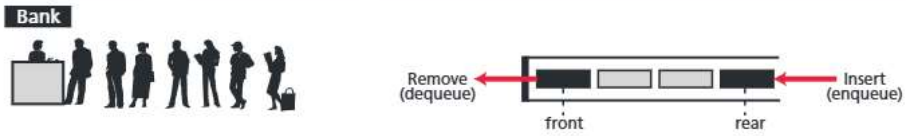
12.3 QUEUES

A **queue** is a linear list in which data can only be inserted at one end, called the **rear**, and deleted from the other end, called the **front**. These restrictions ensure that the data are processed through the queue in the order in which it is received. In other words, a queue is a **first in, first out (FIFO)** structure.

Queues are familiar from everyday life. A line of people waiting for the bus at a bus station is a queue, a list of calls put on hold to be answered by a telephone operator is a queue, and a list of waiting jobs to be processed by a computer is a queue.

Figure 12.8 shows two representations of queues, one a queue of people and the other a computer queue. Both people and data enter the queue at the rear and progress through the queue until they arrive at the front. Once they are at the front of the queue, they leave the queue and are served.

Figure 12.8 *Two representations of queues*



A queue of people A computer queue

12.3.1 Operations on queues

Although we can define many operations for a queue, four are basic: *queue*, *enqueue*, *dequeue*, and *empty*, as defined below.

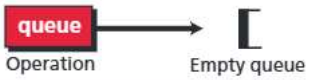
The queue operation

The *queue* operation creates an empty queue. The following shows the format:

```
queue (queueName)
```

queueName is the name of the queue to be created. This operation returns an empty queue. Figure 12.9 shows a pictorial representation of this operation.

Figure 12.9 *The queue operation*



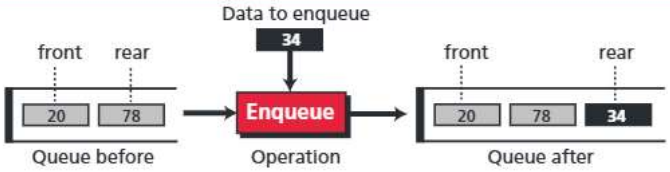
The enqueue operation

The **enqueue** operation inserts an item at the rear of the queue. The following shows the format:

```
enqueue (queueName, dataItem)
```

queueName is the name of the queue and *dataItem* is the data to be inserted at the rear of the queue. After the enqueue operation, the new item becomes the last item in the queue. This operation returns the new queue with *dataItem* inserted at the rear. Figure 12.10 shows the pictorial representation of this operation.

Figure 12.10 *The enqueue operation*



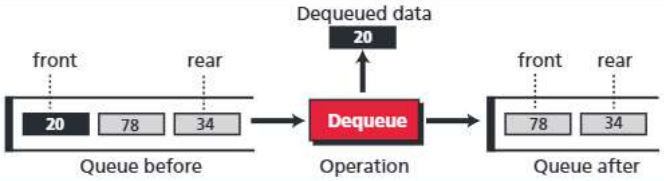
The dequeue operation

The **dequeue** operation deletes the item at the front of the queue. The following shows the format:

```
dequeue (queueName, dataItem)
```

queueName is the name of the queue and *dataItem* is the data that is deleted from the queue. The deleted item can be used by the application program or can be just discarded. After the dequeue operation, the item that followed the front element becomes the front element. This operation returns the new queue with one less element. Figure 12.11 shows the pictorial representation of this operation.

Figure 12.11 *The dequeue operation*



The empty operation

The *empty* operation checks the status of the queue. The following shows the format:

```
empty (queueName)
```

queueName is the name of the queue. This operation returns *true* if the queue is empty and *false* if the queue is not empty.

12.3.2 Queue ADT

We define a queue as an ADT as shown below:

Queue ADT

Definition

A list of data items in which an item can be deleted from one end, called the front and an item can be inserted at the other end called the rear.

Operations

queue: Creates an empty queue.

enqueue: Inserts an element at the rear.

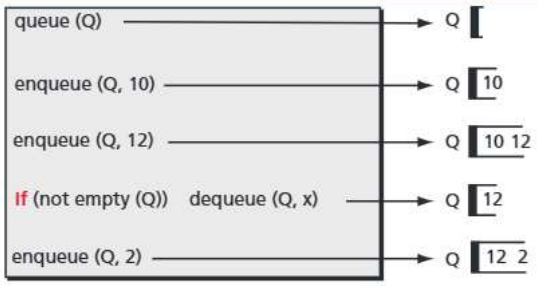
dequeue: Deletes an element from the front.

empty: Checks the status of the queue.

Example 12.4

Figure 12.12 shows a segment of an algorithm that applies the previously defined operations on a queue Q. The fourth operation checks the status of the queue before trying to dequeue the front element. The value of the front element is stored in the variable *x*. However, we do not use this value—it will automatically be discarded at the end of the algorithm segment.

Figure 12.12 *Example 12.4*



12.3.3 Queue applications

Queues are one of the most common of all data processing structures. They are found in virtually every operating system and network and in countless other areas. For example,

queues are used in online business applications such as processing customer requests, jobs, and orders. In a computer system, a queue is needed to process jobs and for system services such as print spools.

Example 12.5

Queues can be used to organize databases by some characteristic of the data. For example, imagine we have a list of sorted data stored in the computer belonging to two categories: less than 1000, and greater than 1000. We can use two queues to separate the categories and at the same time maintain the order of data in their own category. Algorithm 12.3 shows the pseudocode for this operation.

Algorithm 12.3 Example 12.5

```
Algorithm: Categorizer (list)
Purpose: Categorize data into two categories and create two separate lists.
Pre: Given: original list
Post: Prints the two lists
Return: None
{
        queue (Q1)
        queue (Q2)
        while (more data in the list)
        {
            if (data < 1000)
            {
                enqueue (Q1, data)
            }
            if (data ≥ 1000)
            {
                enqueue (Q2, data)
            }
        }
        while (not empty (Q1))
        {
            dequeue (Q1, x)
            print (x)
```

Algorithm 12.3 Example 12.5 (continued)

```
        }
        while (not empty (Q2))
        {
            dequeue (Q2, x)
            print (x)
        }
        return
    }
```

Example 12.6

Another common application of a queue is to adjust and create a balance between a fast producer of data and a slow consumer of data. For example, assume that a CPU is connected to a printer. The speed of a printer is not comparable with the speed of a CPU. If the CPU waits for the printer to print some data created by the CPU, the CPU would be idle for a long time. The solution is a queue. The CPU creates as many chunks of data as the queue can hold and sends them to the queue. The CPU is now free to do other jobs. The chunks are dequeued slowly and printed by the printer. The queue used for this purpose is normally referred to as a *spool queue*.

12.3.4 Queue implementation

At the ADT level, we use the queue and its four operations (*queue*, *enqueue*, *dequeue*, and *empty*): at the implementation level, we need to choose a data structure to implement it. A queue ADT can be implemented using either an array or a linked list. Figure 12.13 on page 331 shows an example of a queue ADT with five items. The figure also shows how we can implement it.

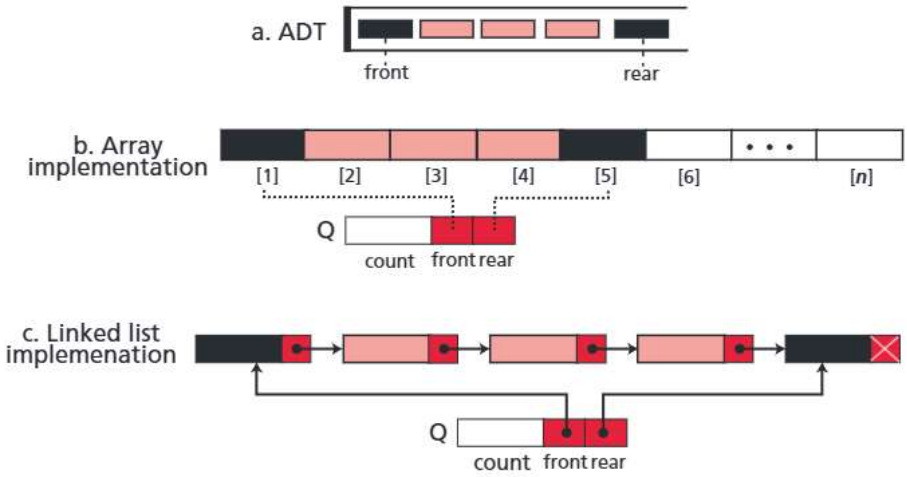
In the array implementation we have a record with three fields. The first field can be used to store information about the queue: we have used this as a count field that shows the current number of data items in the queue. The second field is an integer that holds the index of the front element. The third field is also an integer, which holds the index of the rear element.

The linked list implementation is similar: we have an extra node that has the name of the queue. This node also has three fields: a count, a pointer that points to the front element, and a pointer that points to the rear element.

Algorithms

We can write four algorithms in pseudocode for the four operations we defined for queues in each implementation. We described algorithms to handle arrays and linked lists in Chapter 11: we can modify those algorithms to create the four algorithms we need for queues: *queue*, *enqueue*, *dequeue*, and *empty*. These algorithms are easier than those presented in Chapter 11, because insertion is done only at the end of the queue and deletion is done only at the front of the queue. We leave the writing of these algorithms as exercises.

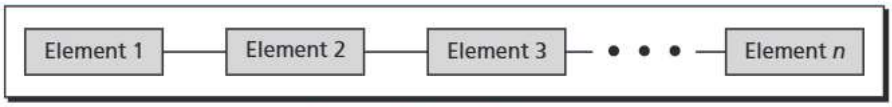
Figure 12.13 *Queue implementation*



12.4 GENERAL LINEAR LISTS

Stacks and queues defined in the two previous sections are *restricted linear lists*. A general linear list is a list in which operations, such as insertion and deletion, can be done anywhere in the list—at the beginning, in the middle, or at the end. Figure 12.14 shows a general linear list.

Figure 12.14 *General linear list*



General linear list

We define a **general linear list** as a collection of elements with the following properties:

- The elements are of the same type.
- The elements are arranged sequentially, which means that there is a first element and a last element.
- Each element except the first has a unique predecessor, each element except the last has a unique successor.
- Each element is a record with a key field.
- The elements are sorted based on the key value.

12.4.1 Operations on general linear lists

Although we can define many operations on a general linear list, we discuss only six common operations in this chapter: *list*, *insert*, *delete*, *retrieve*, *traverse*, and *empty*.

The list operation

The *list* operation creates an empty list. The following shows the format:

```
list (listName)
```

listName is the name of the general linear list to be created. This operation returns an empty list.

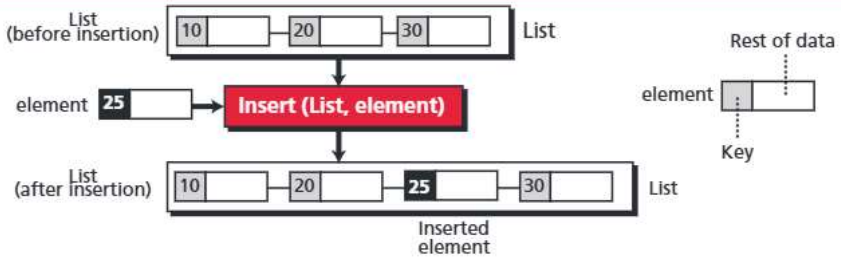
The insert operation

Since we assume that data in a general linear list is sorted, insertion must be done in such a way that the ordering of the elements is maintained. To determine where the elements are to be placed, searching is needed. However, searching is done at the implementation level, not at the ADT level. In addition, we assume for simplicity that duplicate data is not allowed in a general linear list. Therefore we insert an element in a location that preserves the order of the keys. The following shows the format:

```
insert (listName, element)
```

Insertion is shown graphically in Figure 12.15.

Figure 12.15 *The insert operation*



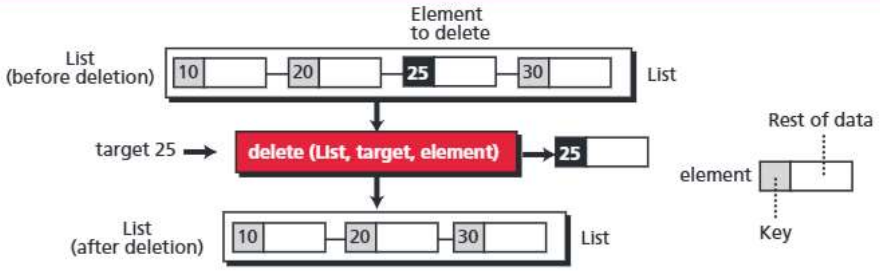
The delete operation

Deletion from a general list (Figure 12.16) also requires that the list be searched to locate the data to be deleted. After the location of the data is found, deletion can be done. The following shows the format:

```
delete (listName, target, element)
```

target is a data value of the same type as the key of the elements in the list. If an element with the key value equal to the target is found, that element is deleted. The **delete operation** is shown graphically in Figure 12.16.

Figure 12.16 *The delete operation*



Note that this operation returns the deleted element. This is necessary if we want, say, to change the value of some fields and reinsert the item into the list again—we have not defined any operation that changes the value of the fields in the list.

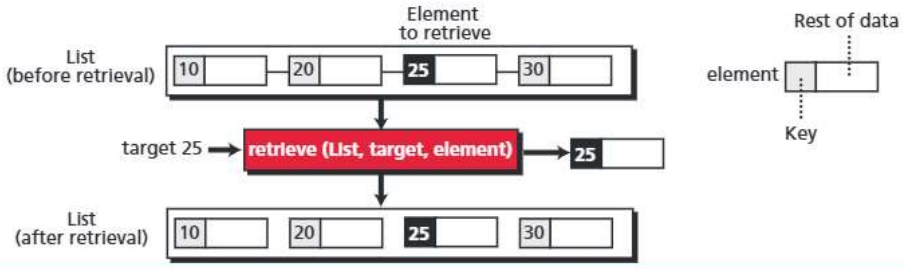
The retrieve operation

By retrieval, we mean access of a single element. Like insertion and deletion, the general list should be first searched, and if the data is found, it can be retrieved. The format of the retrieve operation is:

```
retrieve (listName, target, element)
```

target is a data value of the same type as the key of the elements in the list. Figure 12.17 shows the retrieve operation graphically. If an element with the key value equal to the target is found, a copy of the element is retrieved, but the element still remains in the list.

Figure 12.17 *The retrieve operation*



The traverse operation

Each of the previous operations involves a single element in the list, randomly accessing the list. List **traversal**, on the other hand, involves sequential access. It is an operation in which all elements in the list are processed one by one. The following shows the format:

```
traverse (listName, action)
```

The traverse operation accesses the elements of the list sequentially, while the action specifies the operation to be performed on each element. Some examples of actions are printing the data, applying some mathematical operation on the data, and so on.

The empty operation

The *empty* operation checks the status of the list. The following shows the format:

```
empty (listName)
```

listName is the name of the list. This operation returns *true* if the list is empty, or *false* if the list is not empty.

12.4.2 General linear list ADT

We define a general linear list as an ADT as shown below:

General Linear List ADT

Definition

A list of sorted data items, all of the same type.

Operations

list: Creates an empty list.

insert: Inserts an element in the list.

delete: Deletes an element from the list.

retrieve: Retrieves an element from the list.

traverse: Traverses the list sequentially.

empty: Checks the status of the list.

Example 12.7

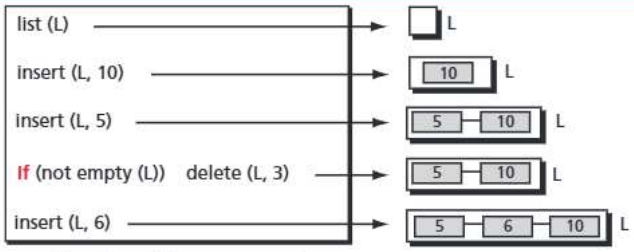
Figure 12.18 shows a segment of an algorithm that applies the previously defined operations on a list *L*. Note that the third operation inserts the new data at the correct position because the insert operation calls the search algorithm at the implementation level to find where the new data should be inserted.

The fourth operation is required to delete the data item 3 from the list. It calls the *empty* operation to be sure that the list is not empty. Since the list is not empty, this operation can proceed, but when it calls the search operation at the implementation level, the data item is not found in the list. The list is therefore returned without a change. Finally the final operation inserts 6 at the appropriate location.

12.4.3 General linear list applications

General linear lists are used in situations in which the elements are accessed randomly or sequentially. For example, in a college a linear list can be used to store information about students who are enrolled in each semester.

Figure 12.18 *Example 12.7*



Example 12.8

Assume that a college has a general linear list that holds information about the students and that each data element is a record with three fields: *ID*, *Name*, and *Grade*. Algorithm 12.4 shows an algorithm that helps a professor to change the grade for a student. The delete operation removes an element from the list, but makes it available to the program to allow the grade to be changed. The insert operation inserts the changed element back into the list. The element holds the whole record for the student, and the target is the *ID* used to search the list.

Algorithm 12.4 Example 12.8

```
Algorithm: ChangeGrade (StudentList, target, grade)
Purpose: Change the grade of a student
Pre: Given the list of students and the grade
Post: None
Return: None

{
    delete (StudentList, target, element)
    (element.data).Grade ← grade
    insert (StudentList, element)
    return
}
```

Example 12.9

Continuing with Example 12.8, assume that the tutor wants to print the record of all students at the end of the semester. Algorithm 12.5 can do this job.

Algorithm 12.5 Example 12.9

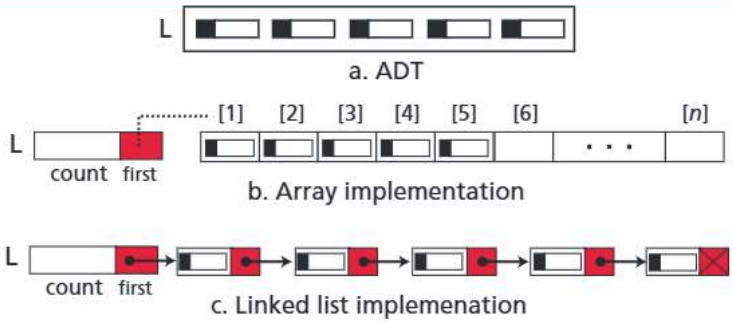
```
Algorithm: PrintRecord (StudentList)
Purpose: Print the record of all students in the StudentList
Pre: Given the list of students
Post: None
Return: None

{
    traverse (StudentList, Print)
    return
}
```

We assume that there is an algorithm called *Print* that prints the contents of the record. For each node, the list traverse calls the *Print* algorithm and passes the data to be printed to it.

12.4.4 General linear list implementation

At the ADT level, we use the list and its six operations (*list*, *insert*, *delete*, *retrieve*, *traverse*, and *empty*), but at the implementation level we need to choose a data structure to implement it. A general list ADT can be implemented using either an array or a linked list. Figure 12.19 shows an example of a list ADT with five items. The figure also shows how we can implement it.

Figure 12.19 *General linear list implementation*



In our array implementation we have a record with two fields. The first field can be used to store information about the array: we have used it as a count field that shows the current number of data item in a list. The second field is an integer that holds the index of the first element. The linked list implementation is similar: we have an extra node that has the name of the list. This node also has two fields, a counter and a pointer that points to the first element.

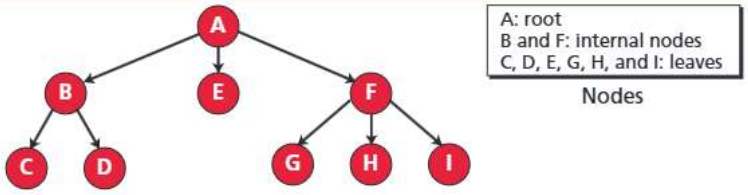
Algorithms

We can write six algorithms in pseudocode for the six operations we defined for a list in each implementation. We showed algorithms to handle arrays and linked lists in Chapter 11: these algorithms can be slightly modified to create the algorithms we need for a list. We leave these as an exercise.

12.5 TREES

A **tree** consists of a finite set of elements, called **nodes** (or **vertices**), and a finite set of directed lines, called **arcs**, that connect pairs of the nodes. If the tree is not empty, one of the nodes, called the **root**, has no incoming arcs. The other nodes in a tree can be reached from the root by following a unique **path**, which is a sequence of consecutive arcs. Tree structures are normally drawn upside down with the root at the top (see Figure 12.20).

Figure 12.20 *Tree representation*



We can divide the vertices in a tree into three categories: the root, **leaves**, and the **internal nodes**.

Table 12.1 shows the number of outgoing and incoming arcs allowed for each type of node.

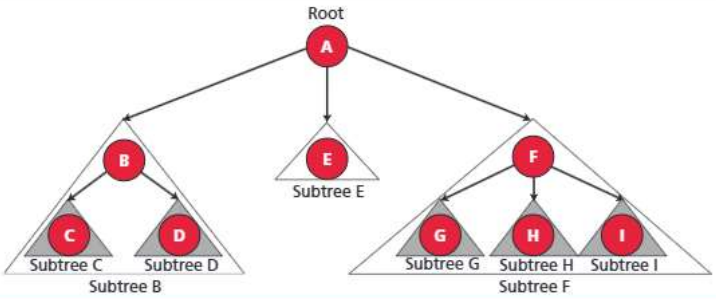
Table 12.1 Number of incoming and outgoing arcs

Type of node	*Incoming arc*	*Outgoing arc*
root	0	0 or more
leaf	1	0
internal	1	1 or more

A node that is directly accessible (through a single arc) from a given node is called the **child**: the node from which the child is directly accessible is called a **parent**. Nodes with a common parent are called **siblings**. **Descendents** of a node are all nodes that can be reached by that node, and a node from which all descendents can be reached is called an **ancestor**. Each node in a tree may have a **subtree**.

The subtree of each node includes one of its children and all descendents of that child. Figure 12.21 shows all subtrees for the tree in Figure 12.20.

Figure 12.21 *Subtrees*

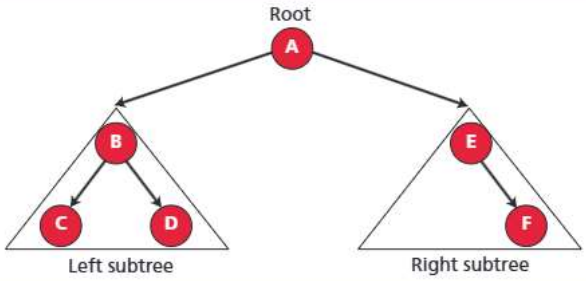


Although trees have many applications in computer science, such as index files, their study is beyond the scope of this book. We introduce trees as a prelude to discussing one special type of tree, *binary trees*.

12.5.1 Binary trees

A **binary tree** is a tree in which no node can have more than two subtrees. In other words, a node can have zero, one, or two subtrees. These subtrees are designated as the left subtree and the right subtree. Figure 12.22 shows a binary tree with its two subtrees. Note that each subtree is itself a binary tree.

Figure 12.22 *A binary tree*



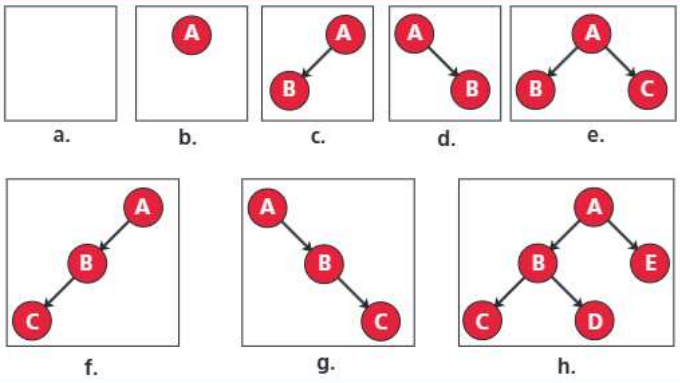
Recursive definition of binary trees

In Chapter 8 we introduced the recursive definition of an algorithm. We can also define a structure or an ADT recursively. The following gives the recursive definition of a binary tree. Note that, based on this definition, a binary tree can have a root, but each subtree can also have a root.

A binary tree is either empty or consists of a node, root, with two subtrees, in which each subtree is also a binary tree.

Figure 12.23 shows eight trees, the first of which is an empty binary tree (sometimes called a *null* binary tree).

Figure 12.23 *Examples of binary trees*



12.5.2 Operations on binary trees

The six most common operations defined for a binary tree are *tree* (creates an empty tree) *insert*, *delete*, *retrieve*, *empty* and *traversal*. The first five are complex and beyond the scope of this book. We discuss binary tree traversal in this section.

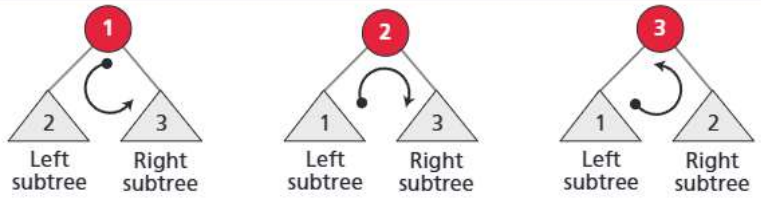
Binary tree traversals

A *binary tree traversal* requires that each node of the tree be processed once and only once in a predetermined sequence. The two general approaches to the traversal sequence are *depth-first* and *breadth-first* traversal.

Depth-first traversals

Given that a binary tree consists of a root, a left subtree, and a right subtree, we can define six different **depth-first traversal** sequences. Computer scientists have assigned standard names to three of these sequences in the literature: the other three are unnamed but are easily derived. The standard traversals are shown in Figure 12.24.

Figure 12.24 *Depth-first traversal of a binary tree*



a. Preorder traversal b. Inorder traversal c. Postorder traversal

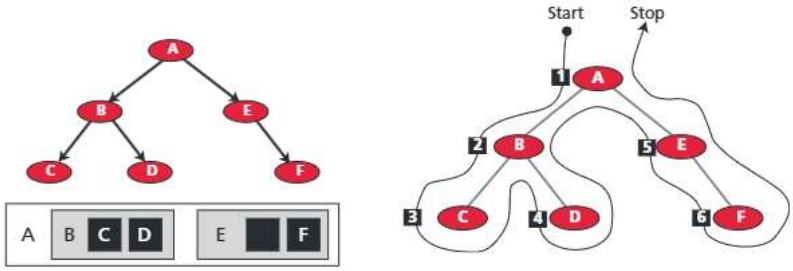
- **Preorder traversal**. In **preorder traversal** the root node is processed first, followed by the left subtree and then the right subtree. The prefix *pre* indicates that the root node is processed *before* the subtrees.

- **Inorder traversal.** In **inorder traversal** the left subtree is processed first, then the root node, and finally the right subtree. The prefix *in* indicates that the root node is processed *between* the subtrees.
- **Postorder traversal.** In **postorder traversal** the root node is processed after the left and right subtrees have been processed. The prefix *post* indicates that the root is processed *after* the subtrees.

Example 12.10

Figure 12.25 shows how we visit each node in a tree using preorder traversal. The figure also shows the *walking order*. In preorder traversal we visit a node when we pass from its left side. The nodes are visited in this order: A, B, C, D, E, F.

Figure 12.25 *Example 12.10*



a. Processing order b. "Walking" order

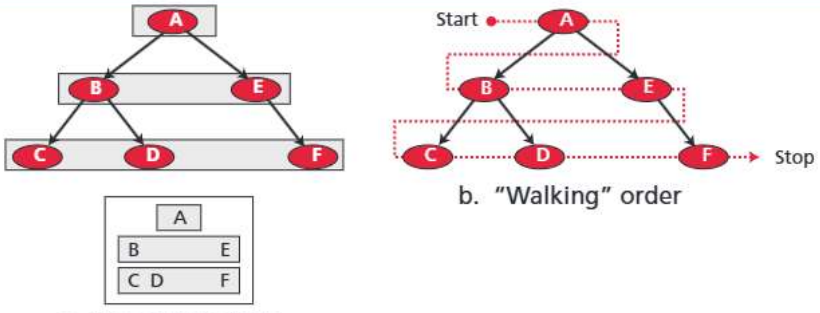
Breadth-first traversals

In **breadth-first traversal** of a binary tree we process all the children of a node before proceeding with the next generation. As with depth-first traversals, we can trace the traversal with a walk.

Example 12.11

Figure 12.26 shows how we visit each node in a tree using breadth-first traversal. The figure also shows the walking order. The traversal order is A, B, E, C, D, F.

Figure 12.26 *Example 12.11*



a. Processing order b. "Walking" order

12.5.3 Binary tree applications

Binary trees have many applications in computer science. In this section we mention only two of them: Huffman coding and expression trees.

Huffman coding

Huffman coding is a compression technique that uses binary trees to generate a variable length binary code from a string of symbols. We discuss Huffman coding in detail in Chapter 15.

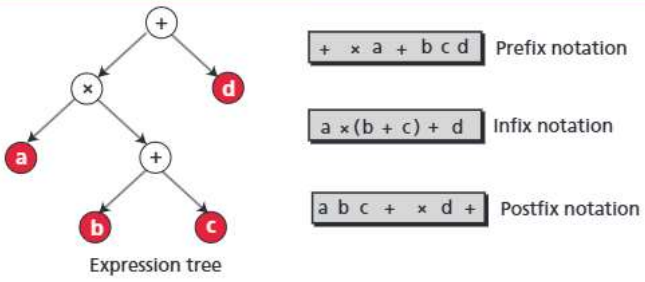
Expression trees

An arithmetic expression can be represented in three different formats: **infix**, **postfix**, and **prefix**. In an infix notation, the operator comes between the two operands. In postfix notation, the operator comes after its two operands, and in prefix notation it comes before the two operands. These formats are shown below for the addition of two operands A and B.

Prefix: + A B	Infix: A + B	Postfix: A B +

Although we use infix notation in our algorithms and in programming languages, the compiler often changes them to postfix notation before evaluating them. One way to do this conversion is to create an **expression tree**. In an expression tree, the root and the internal nodes are operators and the leaves are the operands. The three standard traversals (preorder, inorder, and postorder: Figure 12.4) then represent the three different expression formats: **infix**, **postfix**, and **prefix**. The inorder traversal produces the infix expression, the postorder traversal produces the postfix expression, and the preorder traversal produces the prefix expression. Figure 12.27 shows an expression and its expression tree. Note that only the infix notation needs parentheses.

Figure 12.27 *Expression tree*



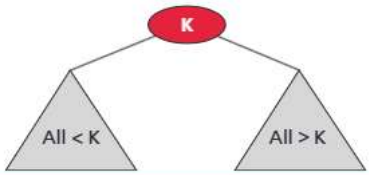
12.5.4 Binary tree implementation

Binary trees can be implemented using arrays or linked list. Link list implementation is more efficient for deletion and insertion and is more prevalent.

12.5.5 Binary search trees

A **binary search tree** (BST) is a binary tree with one extra property: the key value of each node is greater than the key values of all nodes in each left subtree and smaller than the value of all nodes in each right subtree. Figure 12.28 shows the idea.

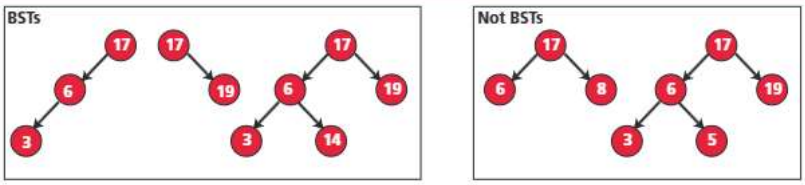
Figure 12.28 *Binary search tree (BST)*



Example 12.12

Figure 12.29 shows some binary trees that are BSTs and some that are not. Note that a tree is a BST if all its subtrees are BSTs and the whole tree is also a BST.

Figure 12.29 *Example 12.12*

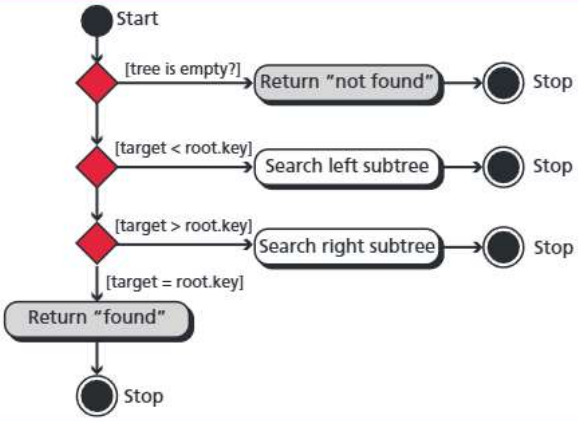


A very interesting property of a BST is that if we apply the inorder traversal of a binary tree, the elements that are visited are sorted in ascending order. For example, the three BSTs in Figure 12.29, when traversed in the order gives the list (3, 6, 17), (17, 19), and (3, 6, 14, 17, 19).

An inorder traversal of a BST creates a list that is sorted in ascending order.

Another feature that makes a BST interesting is that we can use a version of the binary search we used in Chapter 8 for a binary search tree. Figure 12.30 shows the UML for a BST search.

Figure 12.30 *Inorder traversal of a binary search tree*



https://sanet.st/blogs/polatebooks/

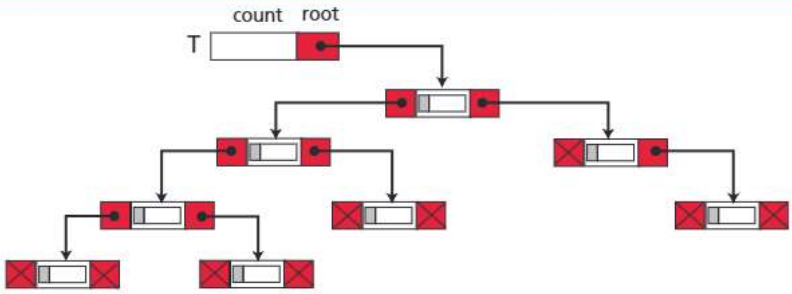
Binary search tree ADTs

The ADT for a binary search tree is similar to the one we defined for a general linear list with the same operation. As a matter of fact, we see more BST lists than general linear lists today. The reason is that searching a BST is more efficient that searching a linear list: a general linear list uses sequential searching, but BSTs use a version of binary search.

BST implementation

BSTs can be implemented using either arrays or linked lists. However, linked list structures are more common and more efficient. A linear implementation uses nodes with two pointers, *left* and *right*. The left pointer points to the left subtree and the right pointer points to the right subtree. If the left subtree is empty, the left pointer is null: if the right subtree is empty, the right pointer is null. Like a linked-list implementation of a general linear list, a BST linked-list implementation uses a dummy node that has the same name as the BST. The data section of this dummy node can hold information about the tree, such as the number of nodes in the tree. The pointer section points to the root of the tree. Figure 12.31 shows a BST in which the data field of each node is a record.

Figure 12.31 *BST implementation*

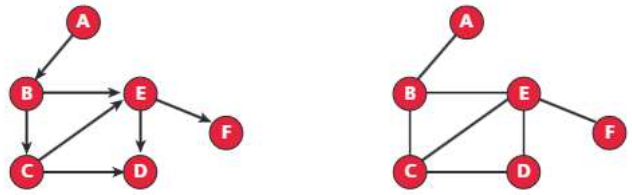


12.6 GRAPHS

A **graph** is an ADT made of a set of nodes, called vertices, and set of lines connecting the vertices, called **edges** or arcs. Whereas a tree defines a hierarchical structure in which a node can have only one single parent, each node in a graph can have one or more parents. Graphs may be either *directed* or *undirected*. In a **directed graph**, or **digraph**, each edge, which connects two vertices, has a direction (shown in the figure by an arrowhead) from one vertex to the other. In an **undirected graph**, there is no direction. Figure 12.32 shows an example of both a directed graph (a) and an undirected graph (b).

The vertices in a graph can represent objects or concepts and the edges or arcs can represent a relationship between those objects or concepts. If a graph is directed, the relations are one-way: if a graph is undirected, the relation is two-way.

Figure 12.32 *Graph*



a. Directed graph b. Undirected graph

Example 12.13

A map of cities and the roads connecting the cities can be represented in a computer using an undirected graph. The cities are vertices and the undirected edges are the roads that connect them. If we want to show the distances between the cities, we can use *weighted graphs*, in which each edge has a weight that represent the distance between two cities connected by that edge.

Example 12.14

Another application of graphs is in computer networks (Chapter 6). The vertices can represent the nodes or hubs; the edges can represent the route. Each edge can have a weight that defines the cost of reaching from one hub to the adjacent hub. A router can use graph algorithms to find the shortest path between itself and the final destination of a packet.

12.7 END-CHAPTER MATERIALS

12.7.1 Recommended reading

For more details about subjects discussed in this chapter, the following books are recommended:

- Gilberg, R. and Forouzan, B. *Data Structures – A Pseudocode Approach with* C, Boston, MA: Course Technology, 2005
- Goodrich, M. and Tamassia, R. *Data Structures and Algorithms in Java*, New York: Wiley, 2005
- Nyhoff, L. *ADTs, Data Structures, and Problem Solving with* C++, Upper Saddle River, NJ: Prentice-Hall, 2005

12.7.2 Key terms

abstract data type (ADT) 318	internal node 337
ancestor 337	last in, first out (LIFO) 320
arc 337	leaf 337
binary search tree (BST) 341	linear list 320
binary tree 338	node 337

breadth-first traversal 340	parent 337
child 337	path 337
delete operation 332	pop 321
depth-first traversal 339	postfix 341
dequeue 327	postorder traversal 340
descendent 337	prefix 341
digraph 343	preorder traversal 339
directed graph 343	push 321
edge 343	queue 326
enqueue 327	rear 326
expression tree 341	root 337
first in, first out (FIFO) 326	sibling 337
front 326	stack 320
general linear list 331	subtree 337
graph 343	traversal 333
Huffman coding 341	tree 337
infix 341	undirected graph 343
inorder traversal 340	vertex 337
interface 319	

12.7.3 Summary

Although several simple data types have been implemented in all programming languages, most languages do not define complex data types. An abstract data type (ADT) is a package that defines a new data type, defines operations on that data type, and encapsulates the data and the operations.

- A stack is a restricted linear list in which all additions and deletions are made at one end, called the *top*. If we inserted a series of data items into a stack and then removed them, the order of the data is reversed. This reversing attribute is why stacks are known as a last in, first out (LIFO) structure. We defined four basic operations on a stack: *stack*, *push*, *pop*, and *empty*.
- A queue is a linear list in which data can only be inserted at one end, called the *rear*, and deleted from the other end, called the *front*. These restrictions ensure that data is processed through the queue in the order in which it is received. In other words, a queue is a first in, first out (FIFO) structure. We defined four basic operations for a queue: *queue*, *enqueue*, *dequeue*, and *empty*. A general linear list is a list in which operations, such as insertion and deletion, can be done anywhere in the list—at the beginning, in the middle, or at the end. We defined six operations for a general linear list: *list*, *insert*, *delete*, *retrieve*, *traverse*, and *empty*.

- A tree consists of a finite set of elements, called *nodes* (or *vertices*), and a finite set of directed lines, called *arcs*, that connect pairs of nodes. If the tree is not empty, one of the nodes, called the *root*, has no incoming arcs. A binary tree is a tree in which no node can have more than two subtrees. In other words, a node can have zero, one, or two subtrees. A binary tree traversal requires that each node of the tree be processed once and only once in a predetermined sequence. The two general approaches to the traversal sequence are *depth first* and *breadth first*. A binary search tree (BST) is a binary tree with one extra property: the key value of each node is greater than the key values of all nodes in each left subtree and smaller than the value of all nodes in each right subtree.
- A graph is an ADT made up of a set of nodes, called *vertices*, and set of lines connecting the vertices, called *edges* or *arcs*. Whereas a tree defines a hierarchical structure in which a node can only have a single parent, each node in a graph can have one or more parents. Graphs may be either directed or undirected.

12.8 PRACTICE SET

12.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

12.8.2 Review questions

Q12-1. What is an abstract data type? In an ADT, what is known and what is hidden?

Q12-2. What is a stack? What are the four basic stack operations defined in this chapter?

Q12-3. What is a queue? What are the four basic queue operations defined in this chapter?

Q12-4. What is a general linear list? What are the six basic operations defined for a general linear list in this chapter?

Q12-5. Define a tree. Distinguish between a tree and a binary tree. Distinguish between a binary tree and a binary search tree.

Q12-6. Distinguish between a depth-first traversal and breadth-first traversal of a binary tree.

Q12-7. What is a graph? Distinguish between a directed graph and an undirected graph.

Q12-8. List some applications of stacks and queues.

Q12-9. List some applications of general linear lists.

Q12-10. List some applications of binary trees and binary search trees.

12.8.3 Problems

P12-1. Write an algorithm segment using while loops to empty the contents of stack S2.

P12-2. Write an algorithm segment using while loops to move the contents of stack S1 to S2. After the operation, stack S1 should be empty.

P12-3. Write an algorithm segment using while loops to copy the contents of stack S1 to S2. After the operation, the contents of stacks S1 and S2 should be the same.

P12-4. Write an algorithm segment using while loops to concatenate the contents of stack S2 with the contents of stack S1. After the concatenation, the elements of stack S2 should be above the elements of stack S1 and stack S2 should be empty.

P12-5. Show the contents of stack S1 and the value of variables *x* and *y* after the following algorithm segment is executed.

```
stack (S1)
push (S1, 5)
push (S1, 3)
push (S1, 2)
if (not empty (S1))
{
    pop (S1, x)
}
if (not empty (S1))
{
    pop (S1, y)
}
push (S1, 6)
```

P12-6. A palindrome is a string that can be read backwards and forwards with the same result. For example, the following is a palindrome if we ignore spaces:

Able was I ere I saw Elba

Write an algorithm in pseudocode using a stack to test whether a string is a palindrome.

P12-7. Write an algorithm in pseudocode to compare the contents of two stacks.

P12-8. Use a while loop to empty the contents of queue Q.

P12-9. Use while loops to move the contents of queue Q1 to queue Q2. After the operation, queue Q1 should be empty.

P12-10. Use while loops to copy the contents of queue Q1 to queue Q2. After the operation, the contents of queue Q1 and queue Q2 should be the same.

P12-11. Use while loops to concatenate the contents of queue Q2 to the contents of queue Q1. After the concatenation, the elements of queue Q2 should be at the end of the elements of queue Q1. Queue Q2 should be empty.

P12-12. Write an algorithm to compare the contents of two queues.

P12-13. Find the root of each of the following binary trees:

a. Tree with postorder traversal: FCBDG

b. Tree with preorder traversal: IBCDFEN

c. Tree with postorder traversal: CBIDFGE

P12-14. A binary tree has ten nodes. The inorder and preorder traversal of the tree are shown below:

Preorder: JCBADEFIGH	**Inorder:** ABCEDFJGIH

Draw the tree.

P12-15. A binary tree has eight nodes. The inorder and postorder traversal of the tree follow:

Postorder: FECHGDBA	**Inorder:** FECABHDG

Draw the tree.

P12-16. A binary tree has seven nodes. The following shows the inorder and postorder traversal of a tree. Can we draw the tree? If not, explain why not:

Postorder: GFDABEC	**Inorder:** ABDCEFG

P12-17. Create the ADT package in pseudocode to implement the four operations defined for a stack in this chapter using an array as the data structure.

P12-18. Create the ADT package in pseudocode to implement the four operations defined for a stack in this chapter using a linked list as the data structure.

P12-19. Create the ADT package in pseudocode to implement the four operations defined for a queue in this chapter using an array as the data structure.

P12-20. Create the ADT package in pseudocode to implement the four operations defined for a queue in this chapter using a linked list as the data structure.

P12-21. Create the ADT package in pseudocode to implement the six operations defined for a general linear list in this chapter using an array as the data structure.

P12-22. Create the ADT package in pseudocode to implement the six operations defined for a general linear list in this chapter using a linked list as the data structure.

CHAPTER 13

File Structure

In this chapter we discuss file structures. Based on the application, files are stored in auxiliary storage devices using various methods. We also discuss how individual records are retrieved. This chapter is a prelude to the following chapter, which discusses how a collection of related files, called a *database*, is organized and accessed.

Objectives

After studying this chapter, the student should be able to:

- Define two categories of access methods: sequential access and random access.
- Understand the structure of sequential files and how they are updated.
- Understand the structure of indexed files and the relation between the index and the data file.
- Understand the idea behind hashed files and describe some hashing methods.
- Describe address collisions and how they can be resolved.
- Define directories and how they can be used to organize files.
- Distinguish between text and binary files.

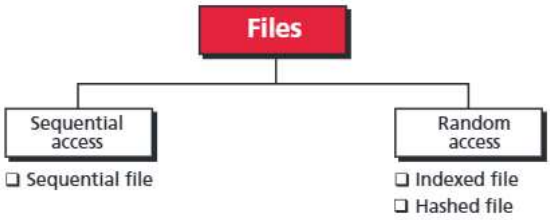
13.1 INTRODUCTION

Files are stored on **auxiliary** or **secondary storage devices**. The two most common forms of secondary storage are disk and tape. Files in secondary storage can be both read from and written to. Files can also exist in forms that the computer can write to but not read. For example, the display of information on the system monitor is a form of file, as is data sent to a printer. In a general sense, the keyboard is also a file, although it cannot store data.

For our purposes, a file is a collection of data records in which each record consists of one or more fields, as defined in Chapter 11.

When we design a file, the important issue is how we will retrieve information (a specific record) from the file. Sometimes we need to process records one after another, whereas sometimes we need to access a specific record quickly without retrieving the preceding records. The **access method** determines how records can be retrieved: *sequentially* or *randomly*.

Figure 13.1 *A taxonomy of file structures*



13.1.1 Sequential access

If we need to access a file sequentially—that is, one record after another, from beginning to end—we use a **sequential file** structure.

13.1.2 Random access

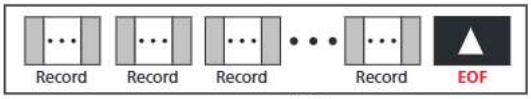
If we need to access a specific record without having to retrieve all records before it, we use a file structure that allows **random access**. Two file structures allow this: *indexed files* and *hashed files*. This taxonomy of file structures is shown in Figure 13.1.

13.2 SEQUENTIAL FILES

A sequential file is one in which records can only be accessed one after another from beginning to end. Figure 13.2 shows the layout of a sequential file. Records are stored one after another in auxiliary storage, such as tape or disk, and there is an EOF (end-of-file) marker after the last record. The operating system has no information about the record

addresses, it only knows where the whole file is stored. The only thing known to the operating system is that the records are sequential.

Figure 13.2 *A sequential file*



Sequential file

Algorithm 13.1 shows how records in a sequential file are processed. We process the records one by one. After the operating system processes the last record, the EOF is detected and the loop is exited.

Algorithm 13.1 Pseudocode for processing records in a sequential file

```
Algorithm: SequentialFileProcessing (file)
Purpose: Process all records in a sequential file
Pre: Given the beginning address of the file on the auxiliary storage
Post: None
Return: None

{
    while (Not EOF)
    {
        Read the next record from the auxiliary storage into memory
        Process the record
    }
}
```

Sequential files are used in applications that need to access all records from beginning to end. For example, if personal information about each employee in a company is stored in a file, we can use **sequential access** to retrieve each record at the end of the month to print the paychecks. Because we have to process each record, sequential access is more efficient and easier than random access.

However, the sequential file is not efficient for random access. For example, if all customer records in a bank can only be accessed sequentially, a customer who needs to get money from an ATM would have to wait as the system checks each record from the beginning of the file until it reaches the customer's record. If this bank has a million customers, the system, on average, would retrieve half a million records before reaching the customer's record. This is very inefficient.

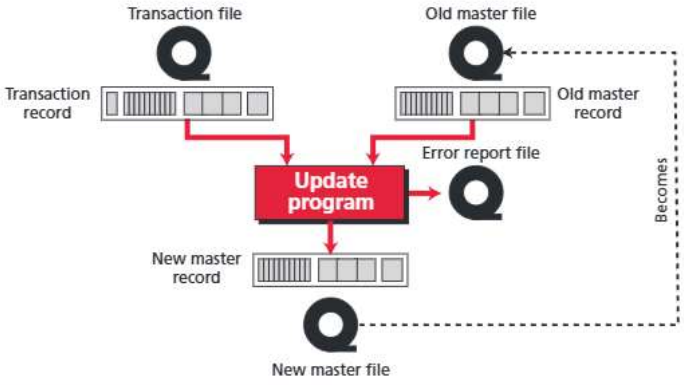
13.2.1 Updating sequential files

Sequential files must be updated periodically to reflect changes in information. The updating process is very involved because all the records need to be checked and updated (if necessary) sequentially.

Files involved in updating

There are four files associated with an update program: the new master file, the old master file, the transaction file, and the error report file. All these files are sorted based on key values. Figure 13.3 is a pictorial representation of a sequential file update. In this figure, we see the four files discussed above. Although we use the tape symbol for the files, we could just as easily have represented them with a hard disk symbol. Note that after the update program completes, the new master file is sent to offline storage, where it is kept until needed again. When the file is to be updated, the **master file** is retrieved from offline storage and becomes the old master.

Figure 13.3 *Updating a sequential file*



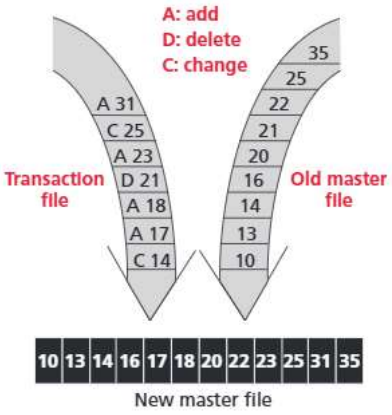
- **New master file**. The new permanent data file or, as it is commonly known, the **new master file**, contains the most current data.
- **Old master file**. The **old master file** is the permanent file that should be updated. Even after updating, the old master file is normally kept for reference.
- **Transaction file**. The third file is the **transaction file**. This contains the changes to be applied to the master file. There are three basic types of changes in all file updates. *Add transactions* contain data about a new record to be added to the master file. *Delete transactions* identify records to be deleted from the file. *Change transactions* contain revisions to specific records in the file. To process any of these transactions, we need a *key*. A **key** is one or more fields that uniquely identify the data in the file. For example, in a file of students, the key could be student ID. In an employee file, the key could be social security number.

- **Error report file.** The fourth file needed in an update program is an **error report file**. It is very rare that an update process does not produce at least one error. When an error occurs, we need to report it. The *error report* contains a listing of all errors discovered during the update process and is presented for corrective action. The next section describes some cases that can cause errors.

Processing file updates

To make the updating process efficient, all files are sorted on the same key. This updating process is shown in Figure 13.4.

Figure 13.4 *Updating process*



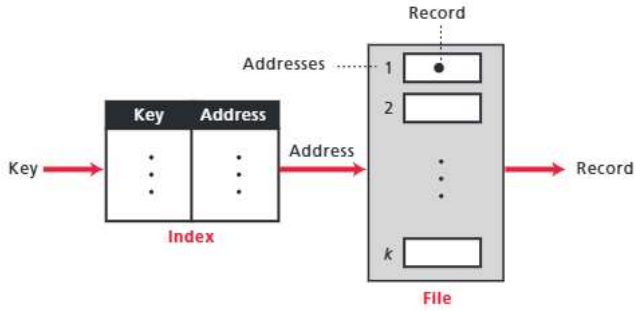
The update process requires that we compare the keys on the transaction and master files and, assuming that there are no errors, follow one of three actions:

1. If the transaction file key is less than the master file key and the transaction is an add (A), add the transaction to the new master.
2. If the transaction file key is equal to the master file key, either:
   a. Change the contents of the master file data if the transaction is a change (C).
   b. Remove the data from the master file if the transaction is a deletion (D).
3. If the transaction file key is greater than the master file key, write the old master file record to the new master file.
4. Several cases may create an error and be reported in the error file:
   a. If the transaction defines adding a record that already exists in the old master file (same key values).
   b. If the transaction defines deleting or changing a record that does not exist in the old master file.

13.3 INDEXED FILES

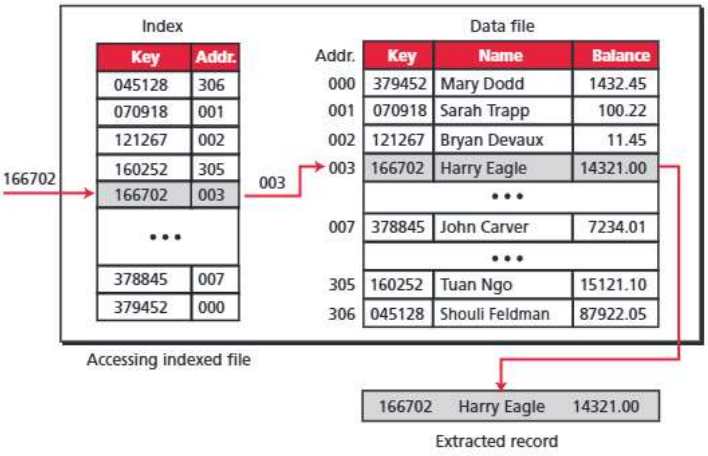
To access a record in a file randomly, we need to know the address of the record. For example, suppose a customer wants to check their bank account. Neither the customer nor the teller knows the address of the customer's record. The customer can only give the teller their account number (key). Here, an indexed file can relate the account number (key) to the record address (Figure 13.5).

Figure 13.5 *Mapping in an indexed file*



An **indexed file** is made of a **data file**, which is a sequential file, and an **index**. The index itself is a very small file with only two fields: the key of the sequential file and the address of the corresponding record on the disk. The index is sorted based on the key values of the data files. Figure 13.6 shows the logical view of an indexed file.

Figure 13.6 *Logical view of an indexed file*



Accessing a record in the file requires these steps:

1. The entire index file is loaded into main memory (the file is small and uses little memory).
2. The index entries are searched, using an efficient search algorithm such as a binary search, to find the desired key.
3. The address of the record is retrieved.
4. Using the address, the data record is retrieved and passed to the user.

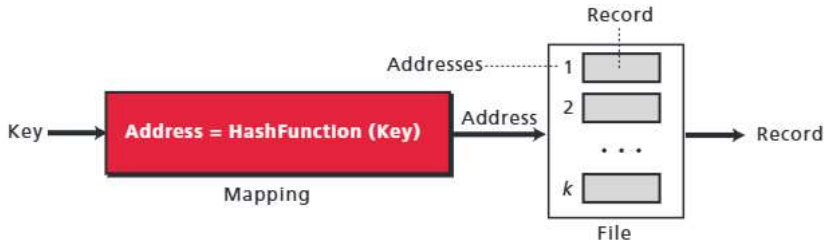
13.3.1 Inverted files

One of the advantages of indexed files is that we can have more than one index, each with a different key. For example, an employee file can be retrieved based on either social security number or last name. This type of indexed file is usually called an **inverted file**.

13.4 HASHED FILES

In an indexed file, the index maps the key to the address. A **hashed file** uses a mathematical function to accomplish this mapping. The user gives the key, the function maps the key to the address and passes it to the operating system, and the record is retrieved (Figure 13.7).

Figure 13.7 *Mapping in a hashed file*



The hashed file eliminates the need for an extra file (the index). In an indexed file, we must keep the index on file in the disk, and when we need to process the data file, we must first load the index into memory, search it to find the address of the data record, and then access the data file to access the record. In a hashed file, finding the address is done through the use of a function, so there is no need for an index and all of the overhead associated with it. However, we will see that hashed files have their own drawbacks.

13.4.1 Hashing methods

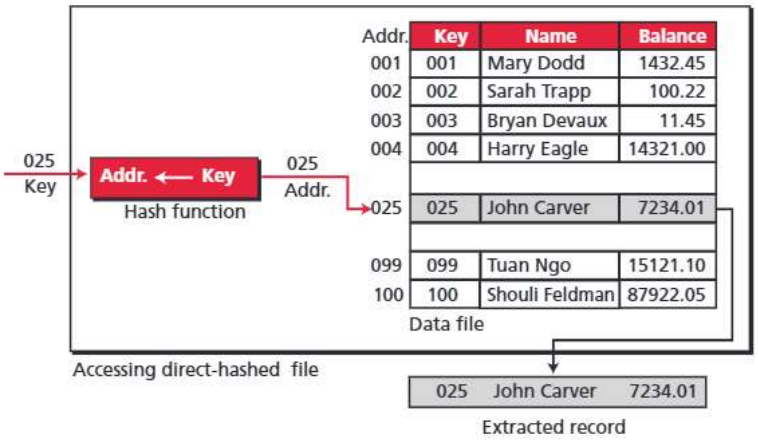
For key-address mapping, we can select one of several **hashing methods**. We discuss a few of them here.

Direct hashing

In **direct hashing**, the key is the data file address without any algorithmic manipulation. The file must therefore contain a record for every possible key. Although situations suitable for direct hashing are limited, it can be very powerful because it guarantees that there are no *synonyms* or *collisions* (discussed later in this chapter), as with other methods.

Let's look at a trivial example. Imagine that an organization has fewer than 100 employees. Each employee is assigned a number between 1 and 100 (their employee ID). In this case, if we create a file of 100 employee records, the employee number can be directly used as the address of any individual record. This concept is shown in Figure 13.8. The record with key 025 (John Carver…) is hashed to address (sector) 025. Note that not every element in the file contains an employee record. Some of the space is wasted.

Figure 13.8 *Direct hashing*



Although this is the ideal method, its application is very limited. For example, it is very inefficient to use long identifiers as keys, because they must have several digits. For example, if the identifier is nine digits, we need a huge file with 999 999 999 records, but we would use fewer than 100. Let's turn our attention, therefore, to hashing techniques that map a large population of possible keys to a small address space.

Modulo division hashing

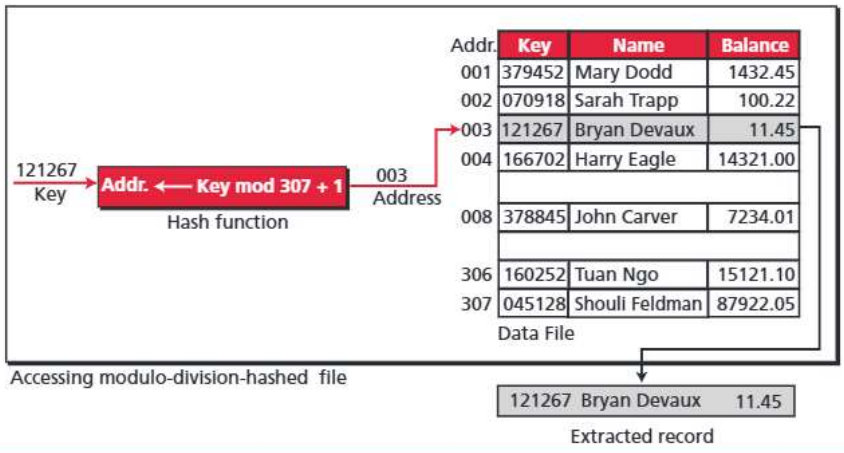
Also known as **division remainder hashing**, the **modulo division** method divides the key by the file size and uses the remainder plus 1 for the address. This gives the simple hashing algorithm that follows, where *list_size* is the number of elements in the file. The reason for adding a 1 to the mod operation result is that our list starts with 1 instead of 0:

```
address = key mod list_size + 1
```

Although this algorithm works with any list size, a list size that is a prime number produces fewer collisions than other list sizes. Therefore, whenever possible, try to make the file size a prime number.

As our company begins to grow, we realize that soon we will have more than 100 employees. Planning for the future, we create a new employee numbering system that will handle one million employees. We also decide that we want to provide data space for up to 300 employees. The first prime number greater than 300 is 307. We therefore choose 307 as our list (file) size. Our new employee list and some of its hashed addresses are shown in Figure 13.9. In this case, Bryan Devaux, with key 121267, is hashed to address 003 because 121267 mod 307 = 2, and we add 1 to the result to get the address (003).

Figure 13.9 *Modulo division*



Digit extraction hashing

Using **digit extraction hashing**, selected digits are extracted from the key and used as the address. For example, using our six-digit employee number to hash to a three-digit address (000–999), we could select the first, third, and fourth digits (from the left) and use them as the address. Using the keys from Figure 13.9, we hash them to the following addresses:

125870	→	158	122801	→	128	121267	→	112

Other hashing methods

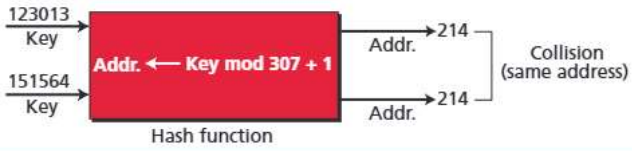
Other popular methods exist, such as the midsquare method, folding methods, the rotational method, and the pseudorandom method. We leave the exploration of these as exercises.

13.4.2 Collision

Generally, the population of keys for a hashed list is greater than the number of records in the data file. For example, if we have a file of 50 students for a class in which the students are identified by the last four digits of their social security number, then there are 200 possible keys for each element in the file (10000/50). Because there are many keys for each address in the file, there is a possibility that more than one key will hash to the same address in the file. We call the set of keys that hash to the same address in our list **synonyms**. The collision concept is illustrated in Figure 13.10.

In the figure, when we calculate the address for two different records, we obtain the same address (214). Obviously, the two records cannot be stored in the same address. We need to resolve the situation, as discussed in the next section.

Figure 13.10 *Collision*



If the actual data that we insert into our list contains two or more synonyms, we will have collisions. A **collision** is the event that occurs when a hashing algorithm produces an address for an insertion key but that address is already occupied. The address produced by the hashing algorithm is known as the **home address**. The part of the file that contains all the home addresses is known as the **prime area**. When two keys collide at a home address, we must resolve the collision by placing one of the keys and its data in another location, outside the prime area.

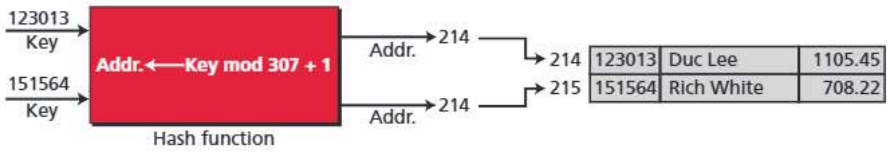
Collision resolution

With the exception of the direct method, none of the methods we have discussed for hashing creates one-to-one mappings. This means that when we hash a new key to an address, we may create a collision. There are several methods for handling collisions, each of them independent of the hashing algorithm. That is, any hashing method can be used with any **collision resolution** method. In this section, we discuss some of these methods.

Open addressing

The first collision resolution method, **open addressing resolution**, resolves collisions in the prime area. When a collision occurs, the prime area addresses are searched for an open or unoccupied record where the new data can be placed. One simple strategy for data that cannot be stored in the home address is to store it in the next address (home address + 1). Figure 13.11 shows how to solve the collision in Figure 13.10 using this method. The first record is stored in address 214, and the second is stored in address 215 if it is not occupied.

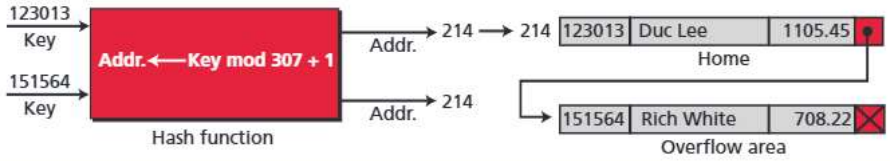
Figure 13.11 *Open addressing resolution*



Linked list resolution

A major disadvantage of open addressing is that each collision resolution increases the probability of future collisions. This disadvantage is eliminated in another approach to collision resolution, **linked list resolution**. In this method, the first record is stored in the home address, but contains a pointer to the second record. Figure 13.12 shows how to resolve the situation in Figure 13.10.

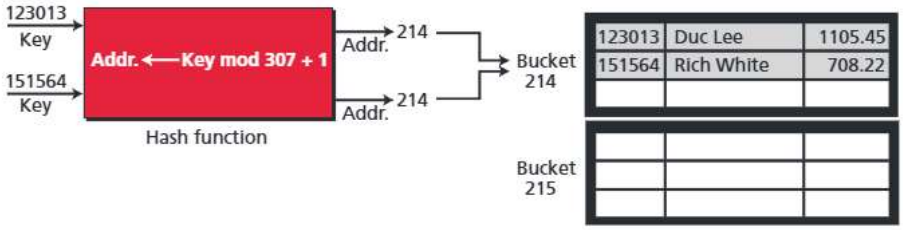
Figure 13.12 *Linked list resolution*



Bucket hashing

Another approach to handling the problem of collisions is to hash to **buckets**. Figure 13.13 shows how to solve the collision in Figure 13.10 using **bucket hashing**. A bucket is a node that can accommodate more than one record. The disadvantage of this method is that there may be a lot of wasted (unoccupied) locations.

Figure 13.13 *Bucket hashing resolution*



Combination approaches

There are several approaches to resolving collisions. As with hashing methods, a complex implementation will often use multiple approaches.

13.5 DIRECTORIES

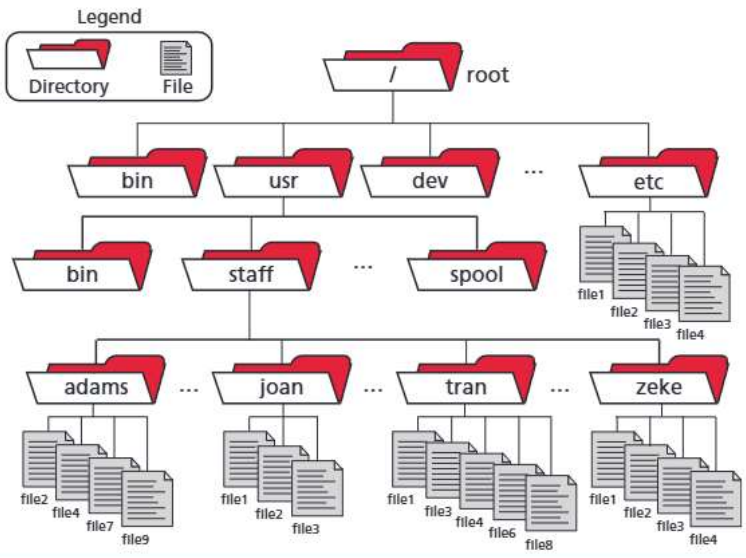
Directories are provided by most operating systems for organizing files. A directory performs the same function as a folder in a filing cabinet. However, a directory in most operating systems is represented as a special type of file that holds information about other files. A directory not only serves as a kind of index that tells the operating system where files are located on an auxiliary storage device, but can also contain other information about the files it contains, such as who has access to each file, or the date when each file was created, accessed, or modified.

Directories in most operating systems are organized like the *tree* abstract data type (ADT) we discussed in Chapter 12, in which each directory except the root directory has a parent. A directory contained in another directory is called a *subdirectory* of the container directory.

13.5.1 Directories in the UNIX operating system

In UNIX the directory system is organized as shown in Figure 13.14.

Figure 13.14 *An example of directory system in UNIX*



At the top of the directory structure is a directory called the *root*. Although its name is root, in commands related to directories it is typed as one slash (/). In turn, each directory can contain subdirectories and files.

Special directories

There are four special types of directory that play an important role in the directory structure in UNIX: the root directory, home directories, working directories, and parent directories.

Root directory

The **root directory** is the highest level in the file system hierarchy. It is the root of the whole file structure, and therefore does not have a parent directory. In a UNIX environment, the root directory always has several levels of subdirectories. The root directory belongs to the system administrator and can be changed only by the system administrator.

Home directory

We use our **home directory** when we first log into the system. This contains any files we create while in it and may contain personal system files. Our home directory is also the beginning of our personal directory structure. Each user has a home directory.

Working directory

The **working directory** (or **current directory**) is the directory we are 'in' at any point in a user session. When we first log in, the working directory is our home directory. If we have subdirectories, we will most likely move from our home directory to one or more subdirectories as needed during a session. When we change directory, our working directory changes automatically.

Parent directory

The **parent directory** is the directory immediately above the working directory. When we are in our home directory, its parent is one of the system directories.

Paths and pathnames

Every directory and file in a file system must have a name. If we examine Figure 13.14 carefully, however, we will note that there are some files that have the same names as files in other directories. It should be obvious, therefore, that we need more than just the filename to identify them. To uniquely identify a file, therefore, we need to specify the file's **path** from the root directory to the file. The file's path is specified by its **absolute pathname**, a list of all directories separated by a slash character (/).

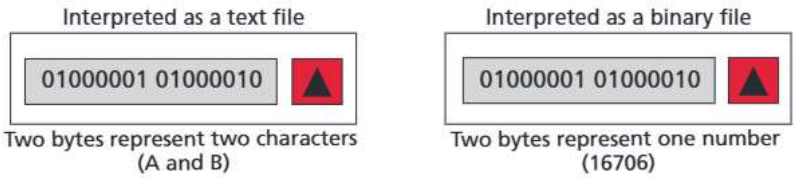
The absolute pathname for a file or a directory is like an address of a person. If we know only the person's name, we cannot easily find that person. On the other hand, if we know a person's name, street address, city, state, and country, then we can locate anyone in the world. This full or absolute pathname can get quite long. For that reason, UNIX also provides a shorter pathname under certain circumstances, known as a **relative pathname**, which is the path relative to the working directory. For example if our working directory in Figure 13.14 is *staff*, the file3 under the *joan* directory can be selected using both relative and absolute pathnames:

Relative Pathname:	**joan/file3**
Absolute pathname:	**/usr/staff/joan/file3**

13.6 TEXT *VERSUS* BINARY

Before closing this chapter, we discuss two terms used to categorize files: *text* files and *binary* files. A file stored on a storage device is a sequence of bits that can be interpreted by an application program as a text file or a binary file, as shown in Figure 13.15.

Figure 13.15 *Text and binary interpretations of a file*



13.6.1 Text files

A **text file** is a file of characters. It cannot contain integers, floating-point numbers, or any other data structures in their internal memory format. To store these data types, they must be converted to their character equivalent formats.

Some files can only use character data types. Most notable are file streams (input/output objects in some object-oriented language like C++) for keyboards, monitors, and printers. This is why we need special functions to format data that is input from or output to these devices.

Let's look at an example. When data (a file stream) is sent to the printer, the printer takes eight bits, interprets them as a byte, and decodes them into the encoding system of the printer (ASCII or EBCDIC). If the character belongs to the printable category, it will be printed, otherwise some other activity takes place, such as printing a space. The printer takes the next eight bits and repeats the process. This is done until the file stream is exhausted.

13.6.2 Binary files

A **binary file** is a collection of data stored in the internal format of the computer. In this definition, data can be an integer (including other data types represented as unsigned integers, such as image, audio, or video), a floating-point number, or any other structured data (except a file).

Unlike text files, binary files contain data that is meaningful only if it is properly interpreted by a program. If the data is textual, one byte is used to represent one character. But if the data is numeric, two or more bytes are considered a data item. For example, assume we are using a personal computer that uses two bytes to store an integer. In this case, when we read or write an integer, two bytes are interpreted as one integer.

13.7 END-CHAPTER MATERIALS

13.7.1 Recommended reading

For more details about subjects discussed in this chapter, the following books are recommended:

- Forouzan, B. and Gilberg, R. *Computer Science: A Structured Programming Approach Using* C, Boston, MA: Course Technology, 2007
- Forouzan, B. and Gilberg, R. *UNIX and Shell Programming*, Pacific Grove, CA: Brooks/Cole, 2003
- Gilberg, R. and Forouzan, B. *Data Structures – A Pseudocode Approach with* C, Boston, MA: Course Technology, 2005

13.7.2 Key terms

absolute pathname 361	inverted file 355
access method 350	key 352
auxiliary storage 350	linked list resolution 359
binary file 362	master file 352
bucket 359	modulo division 356
bucket hashing 359	new master file 352
collision 358	old master file 352
collision resolution 358	open addressing resolution 358
current directory 361	parent directory 361
data file 354	path 361
digit extraction method 357	prime area 358
direct hashing 356	random access 350
directory 360	relative pathname 361
division remainder method 356	root directory 361
error report file 353	secondary storage device 350
hashed file 355	sequential access 351
hashing method 355	sequential file 350
home address 358	synonym 358
home directory 361	text file 362
index 354	transaction file 352
indexed file 354	working directory 361

13.7.3 Summary

- A file is an external collection of related data treated as a single unit. The primary purpose of a file is to store data. Since the contents of main memory are lost when the computer is shut down, we need files to store data in a more permanent form. Files are stored in auxiliary or secondary storage devices.
- The access method determines how records can be retrieved: sequentially or randomly. If we need to access a file sequentially, we use a sequential file structure. If we need to access one specific record without having to retrieve all records before it, we use a random file structure.
- A sequential file is one in which records can only be accessed sequentially—one after another—from beginning to end. Sequential files must be updated periodically to reflect changes in information. There are four files associated with an update program: the new master file, the old master file, the transaction file, and the error report file.
- To access a record in a file randomly, we need to know the address of the record. Two types of files are normally used for accessing records randomly: an indexed file and a hashed file.
- An indexed file is made up of a data file, which is a sequential file, and an index. The index itself is a very small file with only two fields: the key of the sequential file and the address of the corresponding record on disk. The index is sorted based on the key values of the data files. In a hashed file, the key is mapped to the record address using a hashing function.
- Several methods have been used for hashing. In the direct method, the key is the address without any algorithmic manipulation. In the modulo division method, the key is divided by the file size and the remainder plus 1 is used for the address. In digit extraction hashing, selected digits are extracted from the key and used as the address.
- In hashing there is a possibility that more than one key will hash to the same address in the file, resulting in a collision. We discussed a few collision resolution methods: open addressing, linked list resolution, and bucket hashing.
- Directories are provided by most operating systems for organizing files. A directory performs the same function as a folder in a filing cabinet. However, a directory in most operating systems is represented as a special type of file that holds information about other files.
- A file stored on a storage device is a sequence of bits that can be interpreted by an application program as a text file or a binary file. A text file is a file of characters. A binary file is a collection of data stored in the internal format of the computer.

13.8 PRACTICE SET

13.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

13.8.2 Review questions

Q13-1. What are the two general types of file access methods?

Q13-2. What is the relationship between the new master file and the old master file?

Q13-3. What is the purpose of the transaction file in updating a sequential file?

Q13-4. Describe the function of the address in a randomly accessed file.

Q13-5. How is the index related to the data file in indexed files?

Q13-6. What is the relationship between the key and the address in direct hashing of a file?

Q13-7. What is the relationship between the key and the address in modulo division hashing of a file?

Q13-8. What is the relationship between the key and the address in digit extraction hashing of a file?

Q13-9. List and describe three collision resolution methods.

Q13-10. What is the difference between a text file and a binary file?

13.8.3 Problems

P13-1. Given the old master file and the transaction file in Figure 13.16, find the new master file. If there are any errors, create an error file too.

Figure 13.16 *Problem P13-1*

Old master file

Key	Name	Pay rate
14	John Wu	17.00
16	George Brown	18.00
17	Duc Lee	11.00
20	Li Nguyen	12.00
26	Ted White	23.00
31	Joanne King	27.00
45	Brue Wu	12.00
89	Mark Black	19.00
92	Betsy Yellow	14.00

Transaction file

Action	Key	Name	Pay rate
A	17	Martha Kent	17.00
D	20		
C	31		28.00
D	45		
A	90	Orva Gilbert	20.00

P13-2. Create an index file for Table 13.1.

Table 13.1 Problem P13-2

Key	*Name*	*Department*
123453	John Adam	CIS
114237	Ted White	MTH
156734	Jimmy Lions	ENG
093245	Sophie Grands	BUS
077654	Eve Primary	CIS
256743	Eva Lindens	ENG
423458	Bob Bauer	ECO

P13-3. A hash file uses a modulo division method with 41 as the divisor. What is the address for each of the following keys?

a. 14232
b. 12560
c. 13450
d. 15341

P13-4. In the midsquare hashing method, the key is squared and the address is selected from the middle of the result. Use this method to select the address from each of the following keys. Use digits 3 and 4, counting from the left.

a. 142
b. 125
c. 134
d. 153

P13-5. In the fold shift hashing method, the key is divided into parts. The parts are added to obtain the address. Use this method to find the address from the following keys. Divide the key into two-digit parts and add them to find the address.

a. 1422
b. 1257
c. 1349
d. 1532

P13-6. In the fold boundary hashing method, the key is divided into parts. The left and right parts are reversed and added to the middle part to obtain the address. Use this method to find the address from the following keys. Divide the key into three two-digit parts, reverse the digits in the first and the third part, and then add the parts to obtain the address.

a. 142234
b. 125711
c. 134914
d. 153213

P13-7. Find the address of the following keys using the modulo division method and a file of size 411. If there is a collision, use open addressing to resolve it. Draw a figure to show the position of the records.

a. 10278
b. 08222
c. 20553
d. 17256

P13-8. Redo P13-7 using linked list resolution.

P13-9. One common algorithm in file processing is to merge two sequential files sorted on the key values to create a new sequential file, which is also sorted. The merge algorithm can be quite simple if each file has a dummy record at the end with a unique key value that is larger than any key values in either file. The unique key value is referred to as a *sentinel*. In this hypothetical situation, there is no need to check the files for EOF markers. Draw a UML diagram to merge two files for this hypothetical situation.

P13-10. Write an algorithm in pseudocode for P13-9.

P13-11. Draw a UML diagram to update a sequential file based on a transaction file if the two files have sentinel values as discussed in P13-9.

P13-12. Write an algorithm in pseudocode for P13-11.

P13-13. The hypothetical situation of Problem P13-9 can be applied to real situation (files with an EOF marker) if we can artificially create the sentinel during the

processing. Draw a UML diagram to merge two sequential files with EOF markers but no sentinel.

P13-14. Write an algorithm in pseudocode for Problem P13-13.

P13-15. Draw a UML diagram to update a sequential file based on a transaction file if the two files use EOF markers but no sentinel. Use the idea described in Problem P13-13.

P13-16. Write an algorithm in pseudocode for Problem P13-15.

CHAPTER 14

Databases

In this chapter we discuss databases and database management systems (DBMS). We present the three-level architecture for a DBMS, focusing on the **relational database** model, with examples of its operation. We also discuss a language (Standard Query Language) that operates on relational databases. We briefly touch on the design of the databases, and finally mention other database models.

Objectives

After studying this chapter, the student should be able to:

- Define a database and a database management system (DBMS) and describe the components of a DBMS.
- Describe the architecture of a DBMS based on the ANSI/SPARC definition.
- Define the three traditional database models: hierarchical, networking, and relational.
- Describe the relational model and relations.
- Understand operations on a relational database based on commands available in SQL.
- Describe the steps in database design.
- Define ERM and E-R diagrams and explain the entities and relationships in this model.
- Define the hierarchical levels of normalization and understand the rationale for normalizing the relations.
- List database types other than the relational model.

14.1 INTRODUCTION

Data storage traditionally used individual, unrelated files, sometimes called **flat-files**. In the past, each application program in an organization used its own file. In a university, for example, each department might have its own set of files: the record office kept a file about the student information and their grades, the financial aid office kept its own file about students that needed financial aid to continue their education, the scheduling office kept the name of the professors and the courses they were teaching, the payroll department kept its own file about the whole staff (including professors), and so on. Today, however, all of these flat-files can be combined in a single entity, the database for the whole university.

14.1.1 Definition

Although it is difficult to give a universally agreed definition of a **database**, we use the following common definition:

Definition: A database is a collection of related, logically coherent, data used by the application programs in an organization.

14.1.2 Advantages of databases

Comparing the flat-file system, we can mention several advantages of a database system.

Less redundancy

In a flat-file system there is a lot of redundancy. For example, in the flat-file system for a university, the names of professors and students are stored in more than one file.

Inconsistency avoidance

If the same piece of information is stored in more than one place, then any changes in the data need to occur in all places that data is stored. For example, if a female student marries and accepts the last name of her husband, the last name of the student needs to be changed in all files that hold information about the student. Lack of care may create inconsistency in the data.

Efficiency

A database is usually more efficient that a flat-file system, because a piece of information is stored in fewer locations.

Data integrity

In a database system it is easier to maintain data integrity (see Chapter 16) because a piece of data is stored in fewer locations.

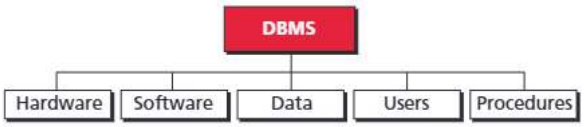
Confidentiality

It is easier to maintain the confidentiality of the information if the storage of data is centralized in one location.

14.1.3 Database management systems

A **database management system** (DBMS) defines, creates, and maintains a database. The DBMS also allows controlled access to data in the database. A DBMS is a combination of five components: hardware, software, data, users, and procedures (Figure 14.1).

Figure 14.1 *DBMS components*



Hardware

The hardware is the physical computer system that allows access to data. For example, the terminals, hard disk, main computer, and workstations are considered part of the hardware in a DBMS.

Software

The software is the actual program that allows users to access, maintain, and update data. In addition, the software controls which user can access which parts of the data in the database.

Data

The data in a database is stored physically on the storage devices. In a database, data is a separate entity from the software that accesses it. This separation allows the organization to change the software without having to change the physical data or the way in which it is stored. If an organization decides to use a DBMS, then all the information needed by the organization should be kept together as one entity, to be accessible by the software in the DBMS.

Users

The term **users** in a DBMS has a broad meaning. We can divide users into two categories: end users and application programs.

End users

End users are those humans who can access the database directly to get information. There are two types of end users: database administrators (DBAs) and normal users. Database administrators have the maximum level of privileges and can control other users and their access to the DBMS, grant some of their privileges to somebody else, but retain the ability to revoke them at any time. A normal user, on the other hand, can only use part of the database and has limited access.

Application programs

The other users of data in a database are **application programs**. Applications need to access and process data. For example, a payroll application program needs to access part of the data in a database to create paychecks at the end of the month.

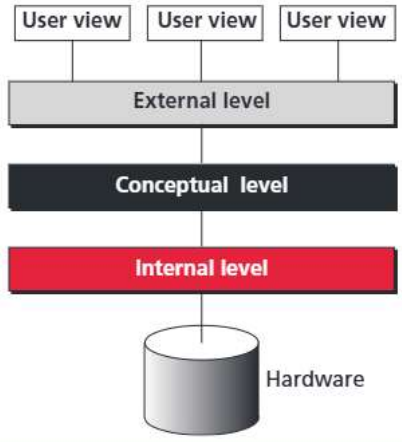
Procedures

The last component of a DBMS is a set of procedures or rules that should be clearly defined and followed by the users of the database.

14.2 DATABASE ARCHITECTURE

The American National Standards Institute/Standards Planning and Requirements Committee (ANSI/SPARC) has established a three-level architecture for a DBMS: internal, conceptual, and external (Figure 14.2).

Figure 14.2 *Database architecture*



14.2.1 Internal level

The **internal level** determines where data is actually stored on the storage devices. This level deals with low-level access methods and how bytes are transferred to and from storage devices. In other words, the internal level interacts directly with the hardware.

14.2.2 Conceptual level

The **conceptual level** defines the logical view of the data. The data model is defined on this level, and the main functions of the DBMS, such as queries, are also on this level. The DBMS changes the internal view of data to the external view that users need to see. The conceptual level is an intermediary and frees users from dealing with the internal level.

14.2.3 External level

The **external level** interacts directly with the user (end users or application programs). It changes the data coming from the conceptual level to a format and view that is familiar to the users.

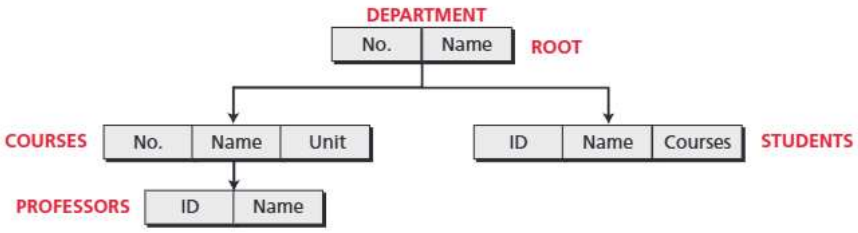
14.3 DATABASE MODELS

A **database model** defines the logical design of data. The model also describes the relationships between different parts of the data. In the history of database design, three models have been in use: the hierarchical model, the network model, and the relational model.

14.3.1 Hierarchical database model

In the **hierarchical model**, data is organized as an inverted tree. Each entity has only one parent but can have several children. At the top of the hierarchy, there is one entity, which is called the *root*. Figure 14.3 shows a logical view of an example of the hierarchical model. As the hierarchical model is obsolete, no further discussion of this model is necessary.

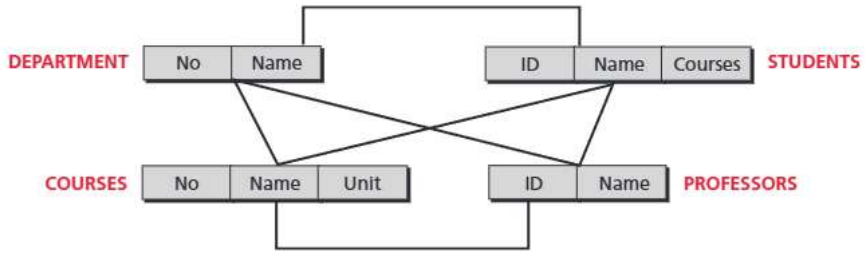
Figure 14.3 *An example of the hierarchical model representing a university*



14.3.2 Network database model

In the **network model**, the entities are organized in a graph, in which some entities can be accessed through several paths (Figure 14.4). There is no hierarchy. This model is also obsolete and needs no further discussion.

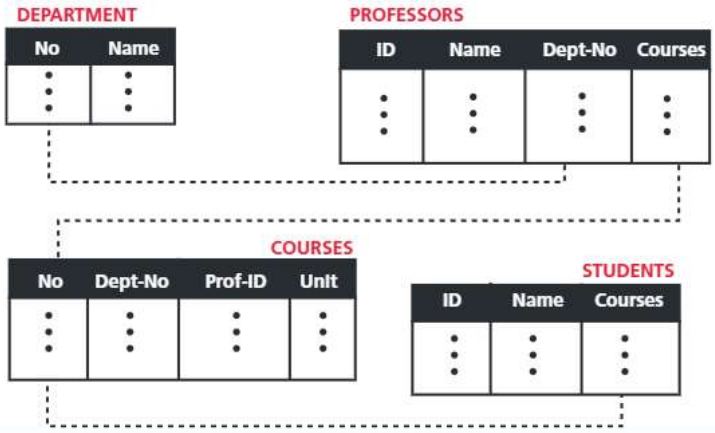
Figure 14.4 *An example of the network model representing a university*



14.3.3 Relational database model

In the **relational model**, data is organized in two-dimensional tables called relations. There is no hierarchical or network structure imposed on the data. The tables or *relations* are, however, related to each other, as we see in Figure 14.5.

Figure 14.5 *An example of the relational model representing a university*



The relational model is one of the common models in use today, and we devote most of this chapter to it. In the last section, we briefly discuss the other two common models that are derived from the relational model: the distributed model and the object-oriented model.

14.4 THE RELATIONAL DATABASE MODEL

In the **relational database management system (RDBMS)**, the data is represented as a set of relations.

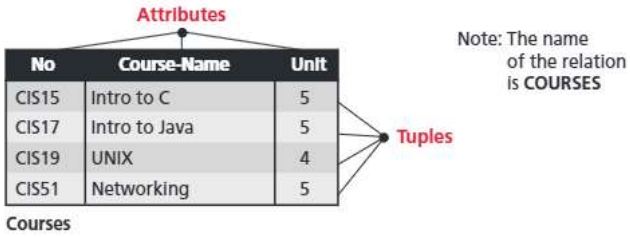
14.4.1 Relation

A **relation**, in appearance, is a two-dimensional table. The RDBMS organizes the data so that its external view is a set of relations or tables. This does not mean that data are stored as tables: the physical storage of the data is independent of the way in which the data is logically organized. Figure 14.6 shows an example of a relation.

A relation in an RDBMS has the following features:

- **Name**. Each relation in a relational database should have a **name** that is unique among other relations.
- **Attributes**. Each column in a relation is called an **attribute**. The attributes are the column headings in the table in Figure 14.6. Each attribute gives meaning to the data

Figure 14.6 *An example of a relation*



stored under it. Each column in the table must have a name that is unique in the scope of the relation. The total number of attributes for a relation is called the degree of the relation. For example, in Figure 14.6, the relation has a degree of 3. Note that the attribute names are not stored in the database: the conceptual level uses the attributes to give meaning to each column.

- **Tuples**. Each row in a relation is called a **tuple**. A tuple defines a collection of attribute values. The total number of rows in a relation is called the **cardinality** of the relation. Note that the cardinality of a relation changes when tuples are added or deleted. This makes the database dynamic.

14.4.2 Operations on relations

In a relational database we can define several operations to create new relations based on existing ones. We define nine operations in this section: *insert*, *delete*, *update*, *select*, *project*, *join*, *union*, *intersection*, and *difference*. Instead of discussing these operations in the abstract, we describe each operation as defined in the database query language SQL (Structured Query Language).

Structured Query Language

Structured Query Language (SQL) is the language standardized by the American National Standards Institute (ANSI) and the **International Organization for Standardization (ISO)** for use on relational databases. It is a declarative rather than procedural language, which means that users declare what they want without having to write a step-by-step procedure. The SQL language was first implemented by the Oracle Corporation in 1979, with various versions of SQL being released since then.

Insert

The **insert operation** is a **unary operation**—that is, it is applied to a single relation. The operation inserts a new tuple into the relation. The insert operation uses the following format:

```
insert into  RELATION-NAME  values  (..., ..., ...)
```

The *values* clause defines all the attribute values for the corresponding tuple to be inserted. For example, Figure 14.7 shows how this operation can be applied to a relation. Note that in SQL string values are enclosed in quotation marks, numeric values are not.

Figure 14.7 *An example of an insert operation*

Delete

The **delete operation** is also a unary operation. The operation deletes a tuple defined by a criterion from the relation. The delete operation uses the following format:

```
delete from RELATION-NAME where criteria
```

The criteria for deletion are defined in the *where* clause. For example, Figure 14.8 shows how one tuple can be deleted from a relation called COURSES. Note that the criteria is No = "CIS19".

Figure 14.8 *An example of a delete operation*

Update

The **update operation** is also a unary operation that is applied to a single relation. The operation changes the value of some attributes of a tuple. The update operation uses the following format:

```
update   RELATION-NAME
set attribute1 = value1,   attribute2 = value2, ...
where   criteria
```

The attribute to be changed is defined in the set clause and the criteria for updating in the *where* clause. For example Figure 14.9 shows how the number of units in one tuple is updated.

Figure 14.9 *An example of an update operation*

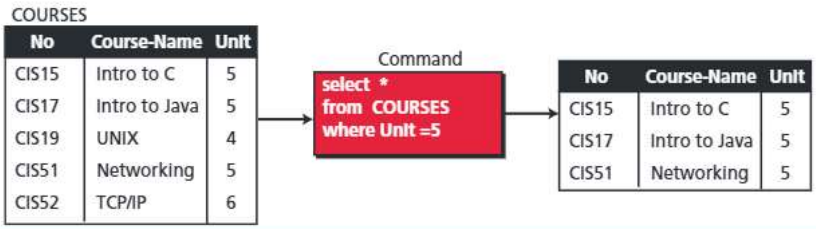
Select

The **select operation** is a unary operation—that is, is applied to a single relation—and creates another relation. The tuples (rows) in the resulting relation are a subset of the tuples in the original relation. The select operation uses some criteria to select some of the tuples from the original relation. The select operation uses the following format:

```
select  *
from RELATION-NAME
where   criteria
```

The asterisk signifies that all attributes are chosen. Figure 14.10 shows an example of the select operation. In this figure, there is a relation that shows courses offered by a small department. The select operation allows the user to select only the five-unit courses.

Figure 14.10 *An example of a select operation*



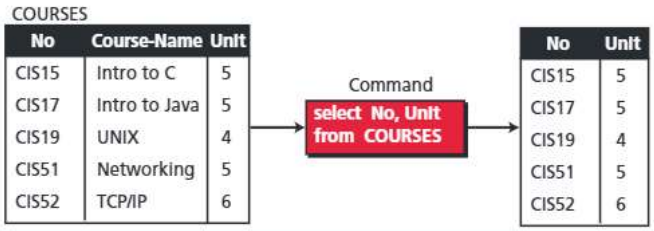
Project

The **project operation** is also a unary operation, and creates another relation. The attributes (columns) in the resulting relation are a subset of the attributes in the original relation. The project operation creates a relation in which each tuple has fewer attributes. The number of tuples (rows) in this operation remains the same. The project operation uses the following format:

```
select   attribute-list
from RELATION-NAME
```

The names of the columns for the new relation are explicitly listed. Figure 14.11 shows an example of a project operation that creates a relation with only two columns.

Figure 14.11 *An example of a project operation*



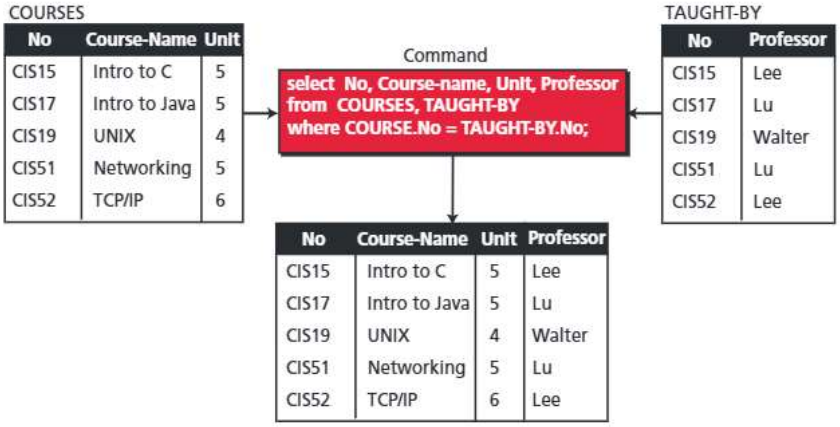
Join

The **join operation** is a **binary operation**—it takes two relations and combines them based on common attributes. The join operation uses the following format:

```
select   attribute-list
from RELATION1, RELATION2
where criteria
```

The attribute list is the combination of attributes from the two input relations: criteria explicitly define the attributes used as common attributes. The join operation is complex and has many variations. In Figure 14.12, we show a very simple example in which the COURSES relation is combined with the TAUGHT-BY relation to create a new relation that shows full information about the courses, including the names of the professors that teach them. In this case, the common attribute is the course number (No).

Figure 14.12 *An example of a join operation*



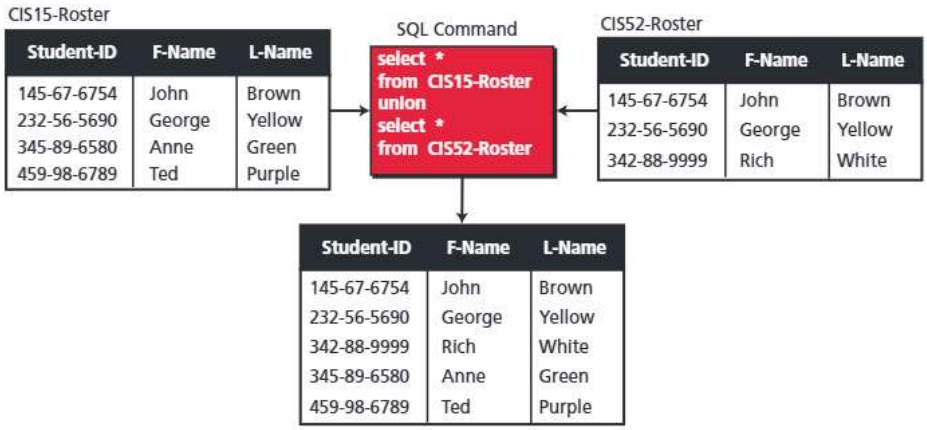
Union

The **union operation** is also a binary operation, taking two relations and creating a new relation. However, there is a restriction on the two relations: they must have the same attributes. The union operation, as defined in set theory, creates a new relation in which each tuple is either in the first relation, in the second, or in both. The union operation uses the following format:

```
select *
from RELATION1
union
select *
from RELATION2
```

Again, asterisks signify that all attributes are selected. For example, Figure 14.13 shows two relations. On the upper left is the roster for course CIS15, on the upper right is the roster for course CIS52. The result is a relation with information about students that take either CIS15, CIS52, or both.

Figure 14.13 *An example of a union operation*



Intersection

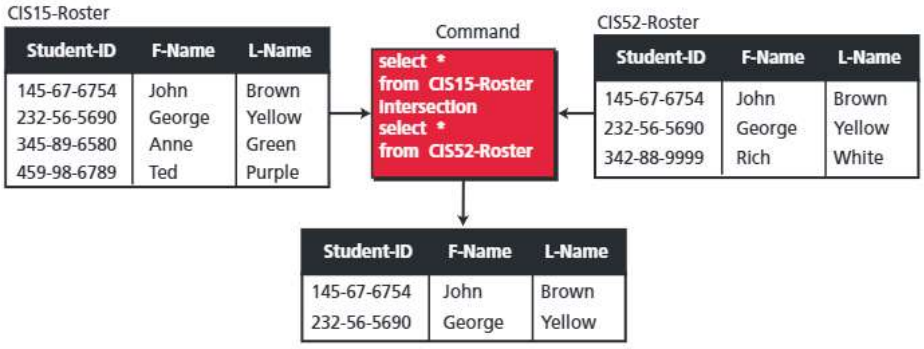
The **intersection operation** is also a binary operation, taking two relations and creating a new relation. Like the union operation, the two relations must have the same attributes. The intersection operation, as defined in set theory, creates a new relation in which each tuple is a member in both relations. The intersection operation uses the following format:

```
select *
from RELATION1
intersection
select *
from RELATION2
```

Again, asterisks signify that all attributes are selected. For example, the intersection operation in Figure 14.14 shows that all attributes are selected.

For example, Figure 14.14 shows two input relations. The result of the intersection operation is a relation with information about students taking both courses CIS15 and CIS52.

Figure 14.14 *An example of an intersection operation*



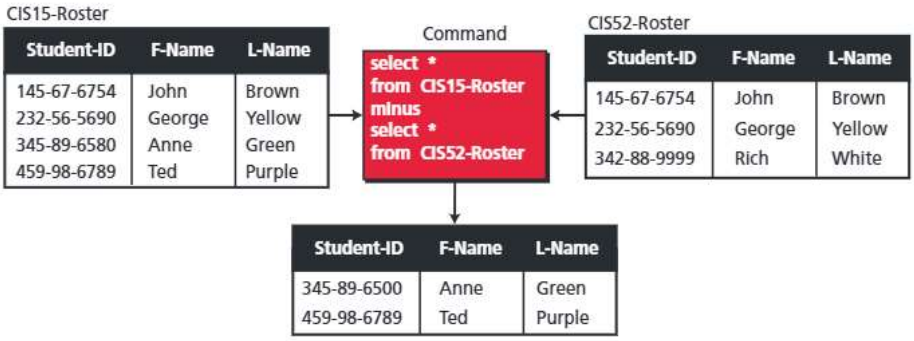
Difference

The **difference operation** is also a binary operation. It is applied to two relations with the same attributes. The tuples in the resulting relation are those that are in the first relation but not the second. The difference operation uses the following format:

```
select *
from RELATION1
minus
select *
from RELATION2
```

Again, asterisks signify that all attributes are selected. For example, Figure 14.15 shows two input relations. The result of the difference operation is a relation with information about students taking course CIS15 but not course CIS52.

Figure 14.15 *An example of a difference operation*



Combination of statements

The SQL language allows us to combine the forgoing statements to extract more complex information from a database.

14.5 DATABASE DESIGN

The design of any database is a lengthy and involved task that can only be done through a step-by-step process. The first step normally involves a lot of interviewing of potential users of the database, for example in a university, to collect the information needed to be stored and the access requirements of each department. The second step is to build an **entity–relation model (ERM)** that defines the entities for which some information must be maintained, the attributes of these entities, and the relationship between these entities.

The next step in design is based on the type of database to be used. In a relational database, the next step is to build relations based on the ERM and normalize the relations. In this introductory course, we just give some idea about ERMs and normalization.

14.5.1 Entity–relation model (ERM)

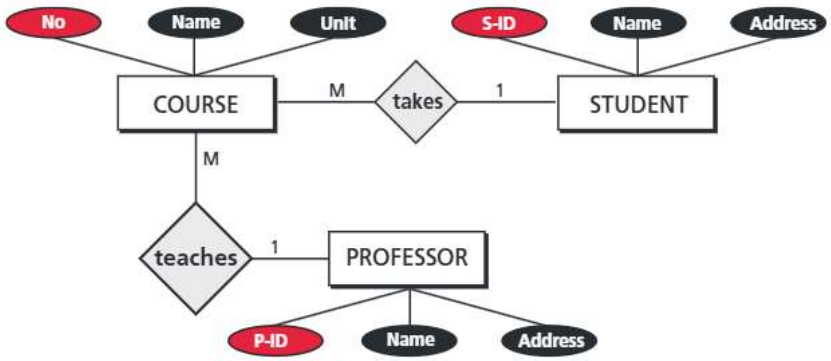
In this step, the database designer creates an **entity–relationship (E-R) diagram** to show the entities for which information needs to be stored and the relationship between those entities. E-R diagrams uses several geometric shapes, but we use only a few of them here:

- *Rectangles* represent entity sets
- *Ellipses* represent attributes
- *Diamonds* represent relationship sets
- *Lines* link attributes to entity sets and link entity sets to relationship sets

Example 14.1

Figure 14.16 shows a very simple E-R diagram with three entity sets, their attributes, and the relationship between the entity sets.

Figure 14.16 *Entities, attributes, and relationships in an E-R diagram*



The relationships, which are shown by diamonds, can be one-to-one, one-to-many, many-to-one, and many-to-many. In Figure 14.16 the relationship between the STUDENT set and the COURSE set is one-to-many (shown by 1-M in the diagram), which means each student in the set of students can take many courses in the set of courses. If we change the relationship from *takes* to *is taken*, then the relationship between the STUDENT set and the COURSE set can be many-to-one.

Some of the attributes in Figure 14.16 are shaded. These are attributes in each set that are considered key for that set. Note that the relationship sets can also have some entities, but we have shown no attributes for the relationship set, to make the discussion easier.

14.5.2 From E-R diagrams to relations

After the E-R diagram has been finalized, relations (tables) in the relational database can be created.

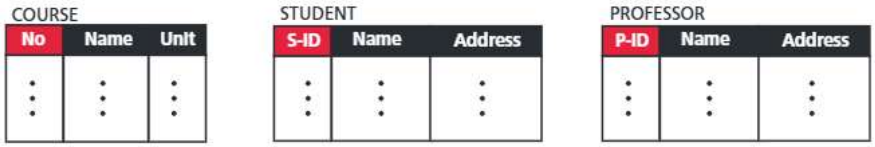
Relations for entity sets

For each entity set in the E-R diagram, we create a relation (table) in which there are n columns related to the n attributes defined for that set.

Example 14.2

We can have three relations (tables), one for each entity set defined in Figure 14.16, as shown in Figure 14.17.

Figure 14.17 *Relations for entity set in Figure 14.16*



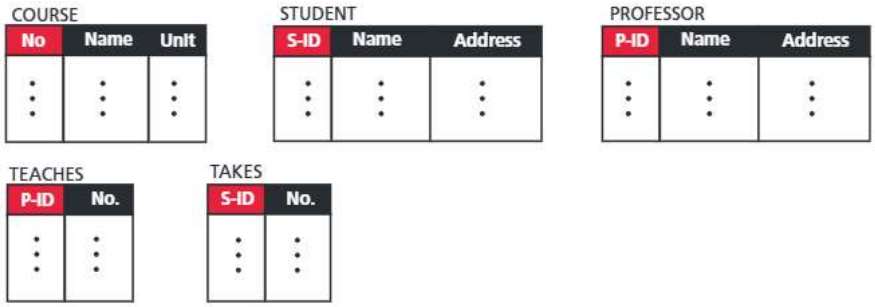
Relations for relationship sets

For each relationship set in the E-R diagram, we create a relation (table). This relation has one column for the key of each entity set involved in this relationship and also one column for each attribute of the relationship itself if the relationship has attributes (not in our case).

Example 14.3

There are two relationship sets in Figure 14.16, *teaches* and *takes*, each connected to two entity sets. The relations for these relationship sets are added to the previous relations for the entity set and shown in Figure 14.18.

Figure 14.18 *Relations for E-R diagram in Figure 14.16*



14.5.3 Normalization

Normalization is the process by which a given set of relations are transformed to a new set of relations with a more solid structure. Normalization is needed to allow any relation in the database to be represented, to allow a languages like SQL to use powerful retrieval operations composed of atomic operations, to remove anomalies in insertion, deletion, and updating, and reduce the need for restructuring the database as new data type are added.

The normalization process defines a set of hierarchical **normal forms** (NFs). Several normal forms have been proposed, including 1NF, 2NF, 3NF, BCNF (Boyce–Codd Normal Form), 4NF, PJNF (Projection/Joint Normal Form), 5NF, and so on. The discussion of

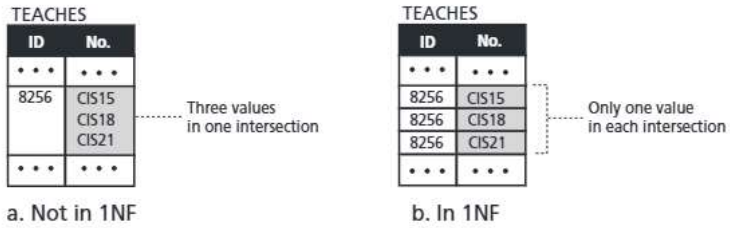
these normal forms (except 1NF) involves the discussion of functional dependencies, a theoretical discipline, which is beyond the scope of this book, although we briefly discuss some of them here for interest. However, one important point that we need to know is that these normal forms form a hierarchical structure. In other words, if the relations in a database are in 3NF, it should have been first in 2NF.

First normal form (1NF)

When we transform entities or relationships into tabular relations, there may be some relations in which there are more values in the intersection of a row or column. For example, in our set of relations in Figure 14.18, there are two relations, *teaches* and *takes*, that are not in first normal form. A professor can teach more than one course, and a student can take more than one course. These two relations can be normalized by repeating the rows in which this problem exists.

Figure 14.19 shows how normalization is done for the relation *teaches*. A relation that is not in the first normal form may suffers from many problems. For example, if the professor with ID 8256 is not teaching the course CIS15, we need to delete only part of the record for this professor in the *teaches* relation. In a database system, we should always delete a whole record, not part of a record.

Figure 14.19 *An example of 1NF*



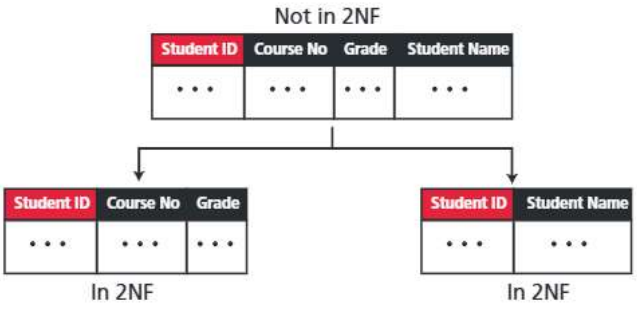
Second normal form (2NF)

In each relation we need to have a key (called a *primary* key) on which all other attributes (column values) needs to depend. For example, if the ID of a student is given, it should be possible to find the student's name. However, it may happen that when relations are established based on the E-R diagram, we may have some composite keys (a combination of two or more keys). In this case, a relation is in second normal form if every non-key attribute depends on the whole composite key.

If some attributes depend on part of the composite key, the relation is not in second normal form. As a simple example, assume that we have a relation in which there are four attributes (Student ID, Course No, Student Grade, and Student Name) in which the first two make up a composite key. The student's grade depends on the whole key, but the name depends on only part of the key. We can apply the 2NF process and divide the relation into two, both in the second normal form.

https://sanet.st/blogs/polatebooks/

Figure 14.20 *An example of 2NF*



A relation that is not in second normal form may also suffer from problems. For example, in Figure 14.20, we cannot add a student to the database if the student does not have a grade at least in one course. But if we have two relations, the student can be added to the second relation. The information about this student is added to the first relation when they take a course and complete it with a grade.

Other normal forms

Other normal forms use more complicated dependencies among attributes. We leave these dependencies to books dedicated to the discussion of database topics.

14.6 OTHER DATABASE MODELS

The relational database is not the only database model in use today. Two other common models are *distributed databases* and *object-oriented databases*. We briefly discuss these here.

14.6.1 Distributed databases

The **distributed database** model is not a new model, but is based on the relational model. However, the data is stored on several computers that communicate through the Internet or a private wide area network. Each computer (or *site*) maintains either part of the database or the whole database. In other words, data is either fragmented, with each fragment stored at one site, or data is replicated at each site.

Fragmented distributed databases

In a **fragmented distributed database**, data is localized—locally used data is stored at the corresponding site. However, this does not mean that a site cannot access data stored at another site, access is mostly local, but occasionally global. Although each site has complete control over its local data, there is global control through the Internet or a wide area network.

For example, a pharmaceutical company may have multiple sites in many countries. Each site has a database with information about its own employees, but a central personnel department could have control of all the databases.

Replicated distributed databases

In a **replicated distributed database**, each site holds an exact replica of another site. Any modification to data stored in one site is repeated exactly at every site. The reason for having such a database is security. If the system at one site fails, users at the site can access data at another site.

14.6.2 Object-oriented databases

The relational database has a specific view of data that is based on the nature of the database's tuples and attributes. The smallest unit of data in a relational database is the intersection of a tuple and an attribute. However, some applications need to look at data in other forms, for example to see data as a structure (see Chapter 11), such as a record composed of fields.

An **object-oriented database** tries to keep the advantages of the relational model and at the same time allows applications to access structured data. In an object-oriented database, objects and their relations are defined. In addition, each object can have attributes that can be expressed as fields.

For example, in an organization, one could define object types for employee, department, and customer. The employee class could define the attributes of an employee object (first name, last name, social security number, salary, and so on) and how they can be accessed. The department object could define the attributes of the department and how they can be accessed. In addition, the database could create a relation between an employee object and a department object to denote that the employee works in that department.

XML

The query language normally used for objected-oriented databases is XML **(Extensible Markup Language)**. XML was originally designed to add markup information to text documents, but it also has found its application as a query language in databases. XML can represent data with nested structure.

14.7 END-CHAPTER MATERIALS

14.7.1 Recommended reading

For more details about subjects discussed in this chapter, the following books are recommended:

- Alagic, S. *Relational Database Technology*, New York: Springer, 1986
- Dietrich, S. *Understanding Relational Database Query Language*, Upper Saddle River, NJ: Prentice-Hall, 2001
- Elmasri, R. and Navathe, S. *Fundamentals of Database Systems*, Reading, MA: Addison-Wesley, 2006
- Mannino, M. *Database Application Development and Design*, New York: McGraw-Hill, 2001

- Ramakrishnan, R. and Gehrke, J. *Database Management Systems*, New York: McGraw-Hill, 2003
- Silberschatz, A., Korth, H. and Sudarshan, S. *Databases: System Concepts*, New York: McGraw-Hill, 2005

14.7.2 Key Terms

application programs 372	International Organization for Standardization (ISO) 375
attribute 374	intersection operation 380
binary operation 378	join operation 378
cardinality 375	name 374
conceptual level 372	network model 373
database 370	normal form (NF) 383
database management system (DBMS) 371	normalization 383
database model 373	object-oriented database 386
delete operation 376	project operation 378
difference operation 380	relation 374
distributed database 385	relational database management system (RDBMS) 374
End users 371	relational model 374
Entity-Relation Model (E-R) 381	replicated distributed database 386
Entity–Relationship (E-R) diagram 381	select operation 377
external level 373	Structured Query Language (SQL) 375
flat-files 370	tuple 375
fragmented distributed database 385	unary operation 375
hierarchical model 373	union operation 379
insert operation 375	update operation 376
internal level 372	users 371

14.7.3 Summary

- A database is a collection of data that is logically, but not necessarily physically, coherent—its various parts can be physically separated. A database management system (DBMS) defines, creates, and maintains a database.
- The American National Standards Institute/Standards Planning and Requirements Committee (ANSI/SPARC) has established a three-level architecture for a DBMS: internal, conceptual, and external. The internal level determines where data is actually stored on storage devices. The conceptual level defines the logical view of the data. The external level interacts directly with the user.

- Traditionally, three types of database model were defined: hierarchical, network, and relational. Only the last, relational model, has survived.
- In the relational model, data is organized in two-dimensional tables called relations. A relation has the following features: name, attributes, and tuples.
- In a relational database we can define several operations to create new relations based on existing ones. We mentioned nine operations in the context of the database query language SQL (Structured Query Language): insert, delete, update, select, project, join, union, intersection, and difference.
- The design of a database, for example for an organization, is often a lengthy task that can only be done through a step-by-step process. The first step often involves interviewing potential users of the database to collect the information that needs to be stored. The second step is to build an Entity-Relation Model (ERM) that defines the entities for which information must be maintained. The next step is to build relations based on the ERM.
- Normalization is the process by which a given set of relations are transformed to a new set of relations with a more solid structure. Normalization is required to allow any relation in the database to be represented, to allow a query language such as SQL to use powerful retrieval operations composed of atomic operations, to remove anomalies in insertion, deletion, and updating, and to reduce the need for restructuring the database as new data types are to be added.
- The relational database is not the only model of database in use today. The other two common models are distributed databases and object-oriented databases.

14.8 PRACTICE SET

14.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

14.8.2 Review questions

Q14-1. What are the five necessary components of a DBMS?

Q14-2. What are the three database models? Which is the most popular today?

Q14-3. What is a relation in a relational database?

Q14-4. In a relation, what is an attribute? What is a tuple?

Q14-5. List some unary operations in relational databases.

Q14-6. List some binary operations in relational databases.

Q14-7. What is SQL? What is XML? Which one is a query language for relational databases? Which one is a query language for the objected-oriented language?

14.8.3 Problems

Figure 14.21 is used in problems P14-1 through P14-5.

Figure 14.21 *Relations for Problem 14-1 through Problem 14-5*

A

A1	A2	A3
1	12	100
2	16	102
3	16	103
4	19	104

B

B1	B2
22	214
24	216
27	284
29	216

C

C1	C2	C3
31	401	1006
32	401	1025
33	405	1065

P14-1. You have relations A, B, and C as shown in Figure 14.21. Show the resulting relation if you apply the following SQL statements:

```
select *
from A
where A2 = 16
```

P14-2. You have relations A, B, and C as shown in Figure 14.21. Show the resulting relation if you apply the following SQL statements:

```
select A1 A2
from A
where A2 = 16
```

P14-3. You have relations A, B, and C as shown in Figure 14.21. Show the resulting relation if you apply the following SQL statements:

```
select A3
from A
```

P14-4. You have relations A, B, and C as shown in Figure 14.21. Show the resulting relation if you apply the following SQL statements:

```
select B1
from B
where B2 = 216
```

P14-5. You have relations A, B, and C as shown in Figure 14.21. Show the resulting relation if you apply the following SQL statements:

```
update C
set C1 = 37
where C1 = 31
```

P14-6. Using the model in Figure 14.5 in section 14.3.3, show the SQL statement that creates a new relation containing only the course number and the number of units for each course.

P14-7. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing only the student ID and student name.

P14-8. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing only the professor's name.

P14-9. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing only the department name.

P14-10. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing the courses taken by the student with ID 2010.

P14-11. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing the courses taught by Professor Blake.

P14-12. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing only courses that have three units.

P14-13. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing only the name of students taking course CIS015.

P14-14. Using the model in Figure 14.5, show the SQL statement that creates a new relation containing the department number of the Computer Science Department.

P14-15. Is the following relation in first normal form (1NF)? If not, change the table to make it pass 1NF criteria.

A	*B*	*C*	*D*
1	70	65	14
2	25, 32, 71	24	12, 18
3	32	6, 11	18

P14-16. Create an E-R diagram for a public library. Show the outline of the relations that can be created from that diagram.

P14-17. Create an E-R diagram for a real estate company, then show the outline of the relations that can be created from that diagram.

P14-18. Create an E-R diagram for three entities FLIGHT, AIRCRAFT, and PILOT in an airline, then show the outlines of the relations in this company.

P14-19. Use references or the Internet to find some information about third normal form (3NF). What kind of functional dependency is involved in this normal form?

P14-20. Use references or the Internet and find some information about Boyce–Codd Normal Form (BCNF). What kind of functional dependency is involved in this normal form?

CHAPTER 15

Data Compression

In recent years technology has changed the way we transmit and store data. For example, fiber-optic cable allows us to transmit data much faster, and DVDs allow us to store huge amounts of data on a physically small medium. However, as in other aspects of life, the rate of demand from the public is ever increasing. Today, we want to download more and more data in a shorter and shorter amount of time. We also want to store more and more data in a smaller space.

Compressing data can reduce the amount of data to be sent or stored by partially eliminating inherent redundancy. Redundancy is created when we produce data. Through data compression, we make transmission and storage more efficient, and at the same time, we preserve the integrity of the data.

Objectives

After studying this chapter, the student should be able to:

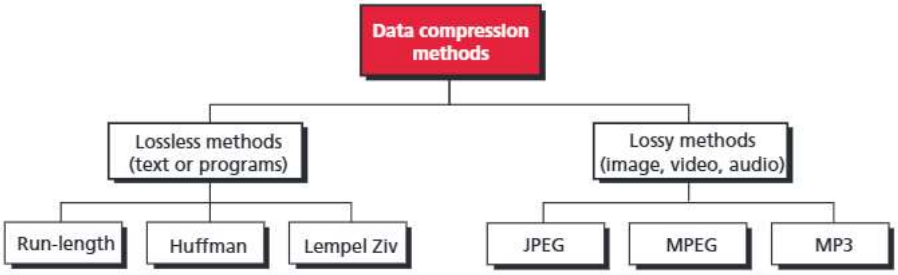
- Distinguish between lossless and lossy compression.
- Describe run-length encoding and how it achieves compression.
- Describe Huffman coding and how it achieves compression.
- Describe Lempel Ziv encoding and the role of the dictionary in encoding and decoding.
- Describe the main idea behind the JPEG standard for compressing still images.
- Describe the main idea behind the MPEG standard for compressing video and its relation to JPEG.
- Describe the main idea behind the MP3 standard for compressing audio.

15.1 INTRODUCTION

Data compression implies sending or storing a smaller number of bits. Although many methods are used for this purpose, in general these methods can be divided into two broad categories: lossless and lossy methods. Figure 15.1 shows the two categories and common methods used in each category.

We first discuss lossless compression methods, as they are simpler and easier to understand. We then present lossy compression methods.

Figure 15.1 *Data compression methods*



15.2 LOSSLESS COMPRESSION METHODS

In **lossless data compression**, the integrity of the data is preserved. The original data and the data after compression and decompression are exactly the same because, in these methods, the compression and decompression algorithms are exact inverses of each other: no part of the data is lost in the process. Redundant data is removed in compression and added during decompression.

Lossless compression methods are normally used when we cannot afford to lose any data. For example, we must not lose data when we compress a text file or an application program.

We discuss three lossless compression methods in this section: *run-length encoding*, *Huffman coding*, and the *Lempel Ziv algorithm*.

15.2.1 Run-length encoding

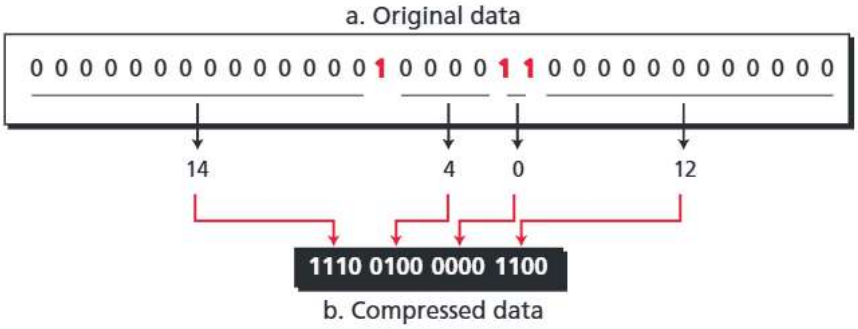
Run-length encoding is probably the simplest method of compression. It can be used to compress data made of any combination of symbols. It does not need to know the frequency of occurrence of symbols (as is necessary for Huffman coding) and can be very efficient if data is represented as 0s and 1s.

The general idea behind this method is to replace consecutive repeating occurrences of a symbol by one occurrence of the symbol followed by the number of occurrences. For example, AAAAAAAA can be replaced by A08. Figure 15.2 shows an example of this simple compression method. Note that we use a fixed number of digits (two) to represent the count.

Figure 15.2 *Run-length encoding example*

The method can be even more efficient if the data uses only two symbols (for example 0 and 1) in its bit pattern and one symbol is more frequent than the other. For example, let's say we have an image represented by mostly 0s and some 1s. In this case, we can reduce the number of bits by sending (or storing) the number of 0s occurring between two 1s (Figure 15.3).

Figure 15.3 *Run-length encoding for two symbols*



We have represented the counts as a 4-bit binary number (unsigned integer). In an actual situation, we would find an optimal number of bits to avoid introducing extra redundancy. In Figure 15.3, there are fourteen 0s before the first 1. These fourteen 0s are compressed to the binary pattern 1110 (14 in binary). The next set of 0s is compressed to 0100 because there are four 0s. Next we have two 1s in the original data, which are represented by 0000 in the compressed data. Finally, the last twelve 0s in the data are compressed to 1100.

Note that, given a 4-bit binary compression, if there are more than fifteen 0s, they are broken into two or more groups. For example, a sequence of twenty-five 0s is encoded as 1111 1010. Now the question is how the decoding algorithm knows that this consists of twenty-five 0s and not fifteen 0s, then a 1, and then ten 0s. The answer is that if the first count is 1111, the receiver knows the next 4-bit pattern is a continuation of 0s. Now another question is raised: what if there are exactly fifteen 0s between two 1s? In this case, the pattern is 1111 followed by 0000.

15.2.2 Huffman coding

Huffman coding assigns shorter codes to symbols that occur more frequently and longer codes to those that occur less frequently. For example, imagine we have a text file that uses only five characters (A, B, C, D, E). We chose only five characters to make the discussion simpler, but the procedure is equally valid for a smaller or greater number of characters.

Before we can assign bit patterns to each character, we assign each character a weight based on its frequency of use. In this example, assume that the frequency of the characters is as shown in Table 15.1. Character A occurs 17 per cent of the time, character B occurs 12 per cent of the time, and so on.

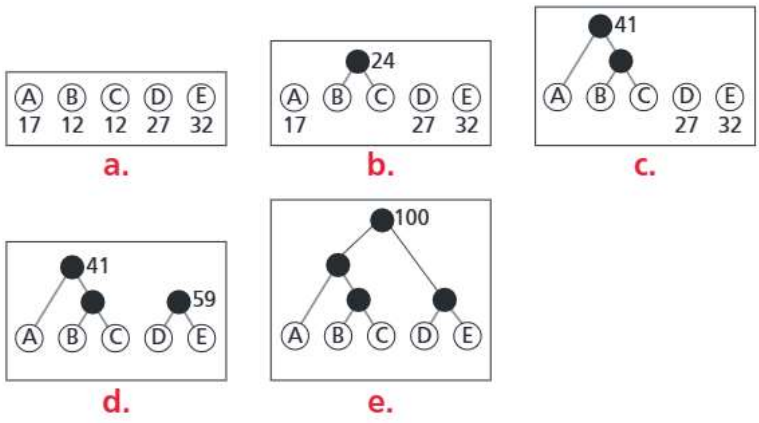
Table 15.1 Frequency of characters

Character	A	B	C	D	E
Frequency	17	12	12	27	32

Once the weight of each character is established, we build a tree based on those values. The process for building this tree is shown in Figure 15.4. It follows three basic steps:

1. Put the entire character set in a row. Each character is now a node at the lowest level of the tree.
2. Find the two nodes with the smallest weights and join them to form a third node, resulting in a simple two-level tree. The weight of the new node is the combined weights of the original two nodes. This node, one level up from the leaves, is eligible for combination with other nodes. Remember that the sum of the weights of the two nodes chosen must be smaller than the combination of any other possible choices.
3. Repeat step 2 until all of the nodes, on every level, are combined into a single tree.

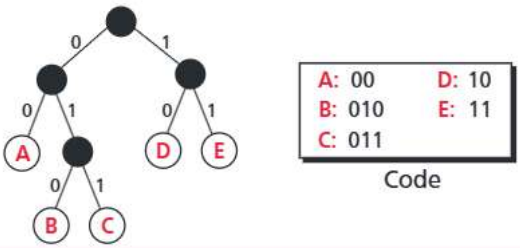
Figure 15.4 *Huffman coding*



Once the tree is complete, use it to assign codes to each character. First, assign a bit value to each branch. Starting from the root (top node), assign 0 to the left branch and 1 to the right branch and repeat this pattern at each node.

A character's code is found by starting at the root and following the branches that lead to that character. The code itself is the bit value of each branch on the path, taken in sequence. Figure 15.5 shows the final tree with bits added to each branch. Note that we moved the leaf nodes to make the tree look like a binary tree.

Figure 15.5 *Final tree and code*

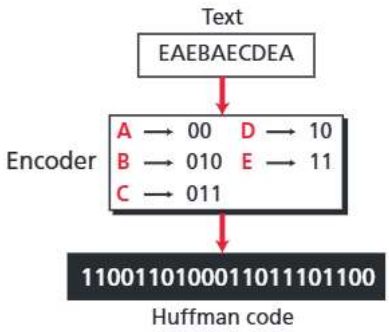


Note these points about the codes. First, the characters with higher frequencies receive a shorter code (A, D, and E) than the characters with lower frequencies (B and C). Compare this with a code that assigns equal bit lengths to each character. Second, in this coding system, no code is a prefix of another code. The 2-bit codes, 00, 10, and 11, are not the prefixes of any of the two other codes (010 and 011). In other words, we do not have a 3-bit code beginning with 00, 10, or 11. This property makes Huffman code an instantaneous code. We will explain this property when we discuss encoding and decoding in Huffman coding.

Encoding

Let us see how to encode text using the code for our five characters. Figure 15.6 shows the original and the encoded text.

Figure 15.6 *Huffman encoding*



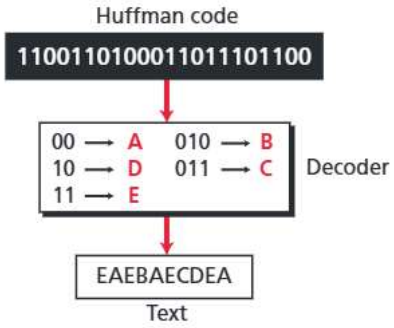
Two points about this figure are worth mentioning. First, notice that there is a sense of compression even in this small and unrealistic code. If we want to send the text without using Huffman coding, we need to assign a 3-bit code to each character. You would have sent 30 bits, whereas with Huffman coding, we send only 22 bits.

Second, notice that we have not used any delimiters between the bits that encode each character. We write the codes one after another. The beauty of Huffman coding is that no code is the prefix of another code. There is therefore no ambiguity in encoding, so the decoding algorithm can decode the received data without ambiguity.

Decoding

The recipient has a very easy job in decoding the data that it receives. Figure 15.7 shows how decoding takes place. When the recipient receives the first 2 bits, it does not have to wait for the next bit to make a decision—it knows that these 2 bits encode the letter E. This is because these 2 bits are not the prefix of any 3-bit code (there is no 3-bit code that starts with 11). Likewise, when the receiver receives the next 2 bits (00), it also knows that the character must be A. The next 2 bits are interpreted the same way (11 must be E). However, when it receives bits 7 and 8, it knows that it must wait for the next bit, because this code (01) is not in the list of codes. After receiving the next bit (0), it interprets the 3 bits together (010) as B. This is why Huffman code is called an instantaneous code—the decoder can unambiguously decode the bits instantaneously, using the minimum number of bits.

Figure 15.7 *Huffman decoding*



15.2.3 Lempel Ziv encoding

Lempel Ziv (LZ) encoding, named after its inventors (Abraham Lempel and Jacob Ziv), is an example of a category of algorithms called **dictionary-based encoding**. The idea is to create a dictionary (a table) of strings used during the communication session. If both the sender and the receiver have a copy of the dictionary, then previously encountered strings can be substituted by their index in the dictionary to reduce the amount of information transmitted.

Although the idea appears simple, several difficulties surface in the implementation. First, how can a dictionary be created for each session? It cannot be universal, due to its length. Second, how can the recipient acquire the dictionary created by the sender—if we send the dictionary, we are sending extra data, which defeats the whole purpose of compression?

The Lempel Ziv (LZ) algorithm is a practical algorithm that uses the idea of adaptive dictionary-based encoding. The algorithm has gone through several versions (LZ77, LZ78). We introduce the basic idea of this algorithm with an example, but do not delve into the details of different versions and implementations. In the example, assume that the following string is to be sent. We have chosen this specific string to simplify the discussion:

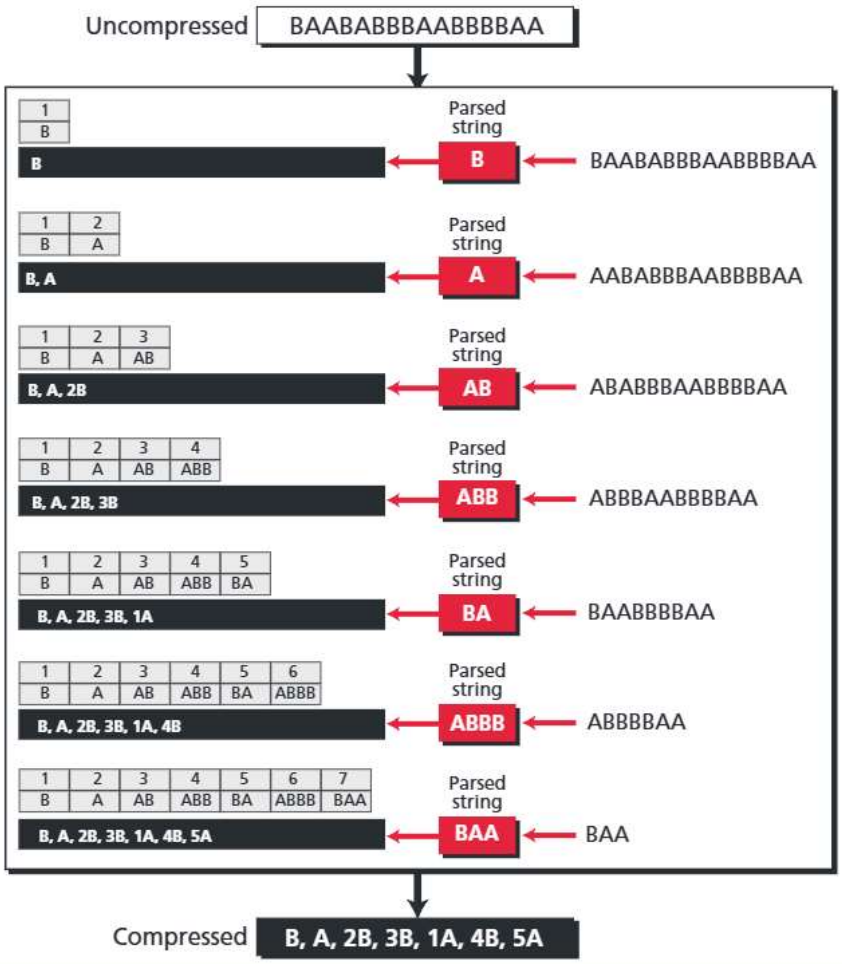
BAABABBBAABBBBAA

Using our simple version of the LZ algorithm, the process is divided into two phases: compressing the string and decompressing the string.

Compression

In this phase there are two concurrent events: building an indexed dictionary and compressing a string of symbols. The algorithm extracts the smallest substring that cannot be found in the dictionary from the remaining uncompressed string. It then stores a copy of this substring in the dictionary as a new entry and assigns it an index value. Compression occurs when the substring, except for the last character, is replaced with the index found in the dictionary. The process then inserts the index and the last character of the substring into the compressed string. For example, if the substring is ABBB, we search for ABB in the dictionary. You find that the index for ABB is 4, so the compressed substring is therefore 4B. Figure 15.8 shows the process for our sample string.

Figure 15.8 *An example of Lempel Ziv encoding*



Let us go through a few steps in this figure:

Step 1

The encoding process extracts the smallest substring from the original string that is not in the dictionary. Because the dictionary is empty, the smallest character is one character (the first character, B). The process stores a copy of it as the first entry in the dictionary with an index of 1. No part of this substring can be replaced with an index from the dictionary, as it is only one character. The process inserts B in the compressed string. So far, the compressed string has only one character, B. The remaining uncompressed string is the original string without the first character.

Step 2

The encoding process extracts the next smallest substring that is not in the dictionary from the remaining string. This substring is the character A, which is not in the dictionary. The process stores a copy of it as the second entry in the dictionary. No part of this substring can be replaced with an index from the dictionary, as it is only one character. The process inserts A in the compressed string. So far, the compressed string has two characters: B and A (we have placed commas between the substrings in the compressed string to show the separation).

Step 3

The encoding process extracts the next smallest substring that is not in the dictionary from the remaining string. This situation differs from the two previous steps: the next character (A) is in the dictionary, so the process extracts two characters (AB), which are not in the dictionary. The process stores a copy of AB as the third entry in the dictionary. The process now finds the index of an entry in the dictionary that is the substring without the last character (AB without the last character is A). The index for A is 2, so the process replaces A with 2 and inserts 2B in the compressed string.

Step 4

Next the encoding process extracts the substring ABB (because A and AB are already in the dictionary). A copy of ABB is stored in the dictionary with an index of 4. The process finds the index of the substring without the last character (AB), which is 3. The combination 3B is inserted into the compressed string.

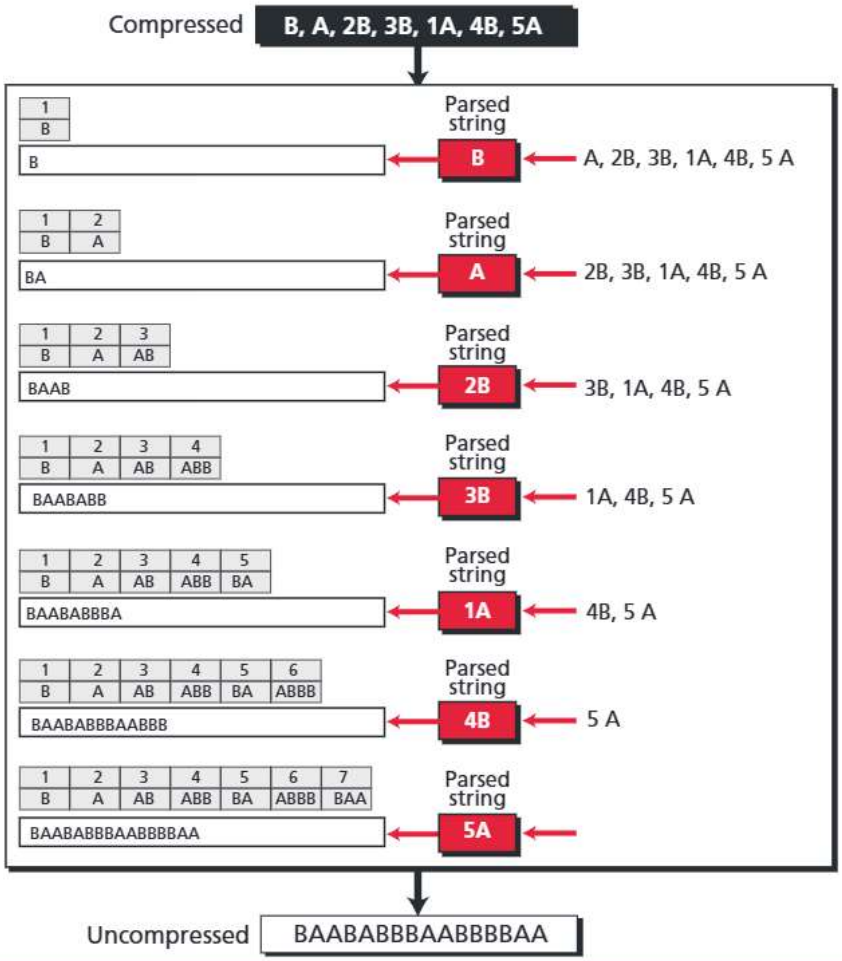
You may have noticed that in the three previous steps, we have not actually achieved any compression, because we have replaced one character by one character (A by A in the first step and B by B in the second step) and two characters by two characters (AB by 2B in the third step). But in this step, we have reduced the number of characters (ABB becomes 3B). If the original string has many repetitions (which is true in most cases), we can greatly reduce the number of characters.

The remaining steps are all similar to one of the preceding four steps. Note that the dictionary was only used by the encoding program to find the indexes. It is not sent to the recipient, and the recipient must create the dictionary for itself, as we will see in the next section.

Decompression

Decompression is the inverse of the compression process. The process extracts the substrings from the compressed string and tries to replace the indexes with the corresponding entry in the dictionary, which is empty at first and built up gradually. The idea is that when an index is received, there is already an entry in the dictionary corresponding to that index. Figure 15.9 shows the decompression process.

Figure 15.9 *An example of Lempel Ziv decoding*



Let us go through a few steps in the figure:

Step 1

The first substring of the compressed string is examined. It is B without an index. Because the substring is not in the dictionary, it is added to the dictionary. The substring (B) is inserted into the decompressed string.

Step 2

The second substring (A) is examined: the situation is similar to step 1. Now the decompressed string has two characters (BA), and the dictionary has two entries.

Step 3

The third substring (2B) is examined. The process searches the dictionary and replaces the index 2 with the substring A. The new substring (AB) is added to the decompressed string, and AB is added to the dictionary.

Step 4

The fourth substring (3B) is examined. The process searches the dictionary and replaces the index 3 with the substring AB. The substring ABB is now added to the decompressed string, and ABB is added to the dictionary.

We leave the exploration of the last three steps as an exercise. As we have noticed, we used a number such as 1 or 2 for the index. In reality, the index is a binary pattern (possibly variable in length) for better efficiency. Also note that LZ encoding leaves the last character uncompressed (which means less efficiency). A version of LZ encoding, called **Lempel Ziv Welch (LZW) encoding**, compresses even this single character. However, we leave the discussion of this algorithm to more specialized textbooks.

15.3 LOSSY COMPRESSION METHODS

Loss of information is not acceptable in a text file or a program file. It is, however, acceptable in an image, video, or audio file. The reason is that our eyes and ears cannot distinguish subtle changes. In such cases, we can use a **lossy data compression** method. These methods are cheaper—they take less time and space when it comes to sending millions of bits per second for images and video.

Several methods have been developed using lossy compression techniques. **JPEG (Joint Photographic Experts Group)** encoding is used to compress pictures and graphics, **MPEG (Moving Picture Experts Group)** encoding is used to compress video, and **MP3 (MPEG audio layer 3)** for audio compression.

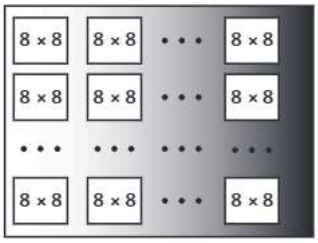
15.3.1 Image compression—JPEG encoding

As discussed in Chapter 2, an image can be represented by a two-dimensional array (table) of picture elements (pixels). For example, $640 \times 480 = 307\,200$ pixels. If the picture is grayscale, each pixel can be represented by an 8-bit integer, giving 256 levels of gray. If the picture is color, each pixel can be represented by 24 bits (3×8 bits), with each 8 bits representing one of the colors in the RBG color system. To simplify the discussion, we concentrate on a grayscale picture with 640×480 pixels. You can see why we need compression. A grayscale picture of 307 200 pixels is represented by 2 457 600 bits, and a color picture is represented by 7 372 800 bits.

In JPEG, a grayscale picture is divided into blocks of 8×8 pixel blocks (Figure 15.10). The purpose of dividing the picture into blocks is to decrease the number of calculations because, as we will see shortly, the number of mathematical operations for each picture is the square of the number of units. That is, for the entire image, we need $307\,200^2$ operations

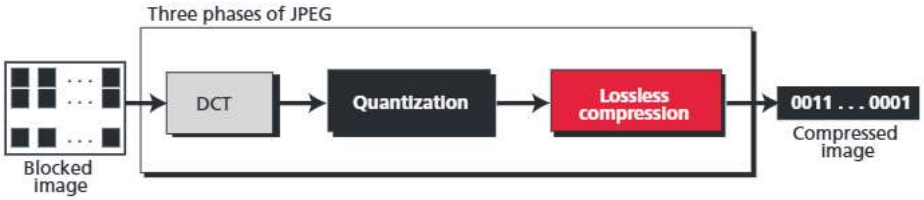
(94 371 840 000 operations). If we use JPEG, we need 64^2 operations for each block, a total of $64^2 \times 80 \times 60$, or 19 660 800 operations. This decreases by 4800 times the number of operations.

Figure 15.10 *JPEG grayscale example, 640 × 480 pixels*



The whole idea of JPEG is to change the picture into a linear (vector) set of numbers that reveals the redundancies. The redundancies (lack of changes) can then be removed using one of the lossless compression methods we studied previously. A simplified version of the process is shown in Figure 15.11.

Figure 15.11 *The JPEG compression process*



Discrete cosine transform (DCT)

In this step, each block of 64 pixels goes through a transformation called the **discrete cosine transform (DCT)**. The transformation changes the 64 values so that the relative relationships between pixels are kept but the redundancies are revealed. The formula is given in Appendix G. $P(x, y)$ defines one value in the block, while $T(m, n)$ defines the value in the transformed block.

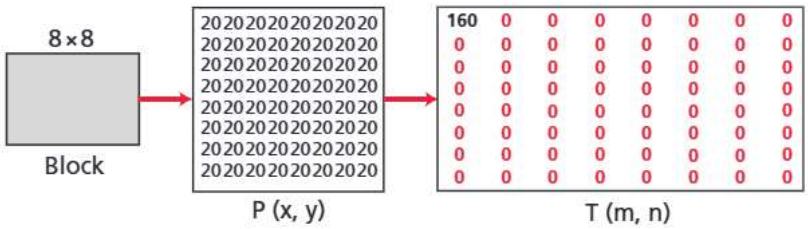
To understand the nature of this transformation, let us show the result of the transformations for three cases.

Case 1

In this case, we have a block of uniform grayscale, and the value of each pixel is 20. When we do the transformations, we get a nonzero value for the first element (upper left corner). The rest of the pixels have a value of 0 because, according to the formula, the value

of $T(0,0)$ is the average of the other values. This is called the **DC value** (direct current, borrowed from electrical engineering). The rest of the values, called **AC values**, in $T(m, n)$ represent changes in the pixel values. But because there are no changes, the rest of the values are 0s (Figure 15.12).

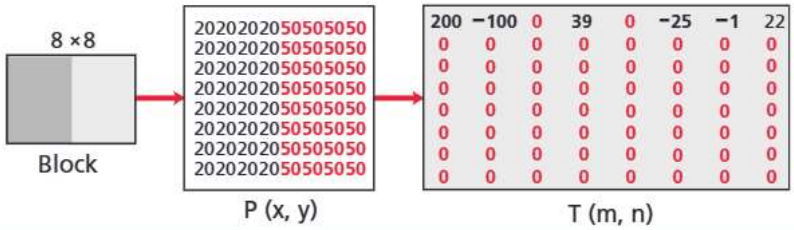
Figure 15.12 *Case 1: uniform grayscale*



Case 2

In the second case, we have a block with two different uniform grayscale sections. There is a sharp change in the values of the pixels (from 20 to 50). When we do the transformations, we get a DC value as well as nonzero AC values. However, there are only a few nonzero values clustered around the DC value. Most of the values are 0 (Figure 15.13).

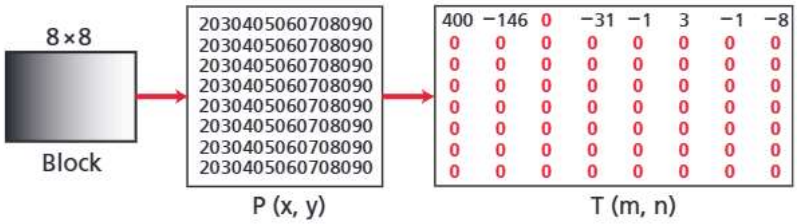
Figure 15.13 *Case 2: two sections*



Case 3

In the third case, we have a block that changes gradually. That is, there is no sharp change between the values of neighboring pixels. When we do the transformations, we get a DC value, with many nonzero AC values also (Figure 15.14).

Figure 15.14 *Case 3: gradient grayscale*



From Figures 15.12, 15.13, and 15.14, we can state the following:

- The transformation creates table *T* from table *P*.
- The DC value gives the average value of the pixels.
- The AC values gives the changes.
- Lack of changes in adjacent pixels creates 0s.

Note that the DCT transformation is reversible. Appendix G also shows the mathematical formula for a reverse transformation.

Quantization

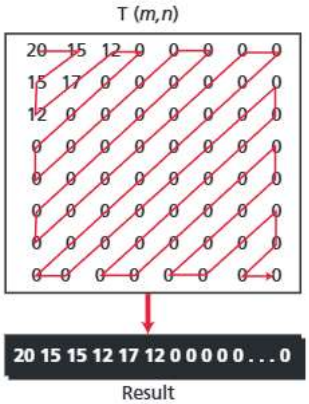
After the *T* table is created, the values are quantized to reduce the number of bits needed for encoding. Quantization divides the number of bits by a constant and then drops the fraction. This reduces the required number of bits even more. In most implementations, a quantizing table (8 by 8) defines how to quantize each value. The divisor depends on the position of the value in the *T* table. This is done to optimize the number of bits and the number of 0s for each particular application.

Note that the only phase in the process that is not reversible is the quantizing phase. You lose some information here that is not recoverable. The only reason that JPEG is a lossy compression method is because of the quantization phase.

Compression

After quantization the values are read from the table, and redundant 0s are removed. However, to cluster the 0s together, the process reads the table diagonally in a zigzag fashion rather than row by row or column by column. The reason is that if the picture does not have fine changes, the bottom right corner of the *T* table is all 0s. Figure 15.15 shows the process. JPEG usually uses run-length encoding at the compression phase to compress the bit pattern resulting from the zigzag linearization.

Figure 15.15 *Reading the table*



15.3.2 Video compression—MPEG encoding

The Moving Picture Experts Group (MPEG) method is used to compress video. In principle, a motion picture is a rapid sequence of a set of frames in which each frame is a picture. In other words, a frame is a spatial combination of pixels, and a video is a temporal combination of frames that are sent one after another. Compressing video, then, means spatially compressing each frame and temporally compressing a set of frames.

Spatial compression

The **spatial compression** of each frame is done with JPEG, or a modification of it. Each frame is a picture that can be independently compressed.

Temporal compression

In **temporal compression**, redundant frames are removed. When we watch television, for example, we receive 30 frames per second. However, most of the consecutive frames are almost the same. For example, in a static scene in which someone is talking, most frames are the same except for the segment around the speaker's lips, which changes from one frame to the next.

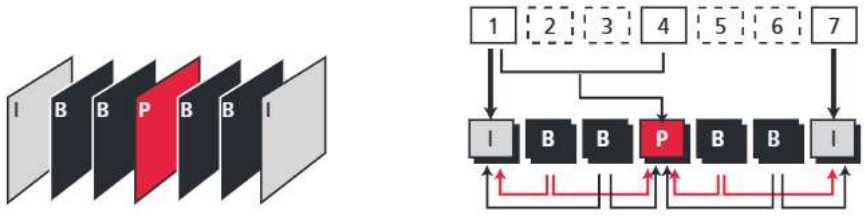
A rough calculation points to the need for temporal compression for video. A 20:1 JPEG compression of one frame sends 368 640 bits per frame: at 30 frames per second, this is 11 059 200 bits per second. We need to reduce this number!

To temporally compress data, the MPEG method first divides frames into three categories: I-frames, P-frames, and B-frames.

- **I-frames**. An **intracoded frame (I-frame)** is an independent frame that is not related to any other frame—that is, not to the frame sent before or to the frame sent after. They are present at regular intervals (for example, every ninth frame is an I-frame). An I-frame must appear periodically due to some sudden change in the frame that the previous and following frames cannot show. Also, when a video is broadcast, a viewer may tune in their receiver at any time. If there is only one I-frame at the beginning of the broadcast, the viewer who tunes in late will not receive a complete picture. I-frames are independent of other frames and cannot be constructed from other frames.
- **P-frames**. A **predicted frame ((P-frame)** is related to the preceding I-frame or P-frame. In other words, each P-frame contains only the changes from the preceding frame. The changes, however, cannot cover a big segment of the image. For example, for a fast-moving object, the new changes may not be recorded in a P-frame. P-frames can be constructed only from previous I- or P-frames. P-frames carry much less information than other frame types and carry even fewer bits after compression.
- **B-frames**. A **bidirectional frame (B-frame)** is relative to the preceding and following I-frame or P-frame. In other words, each B-frame is relative to the past and the future. Note that a B-frame is never related to another B-frame.

Figure 15.16 shows a sample sequence of frames and how they are constructed. Note that for decoding, the decoding process should receive the P frames before the B frames. For this reason, the order of transmission of frames is different than the order in which they displayed at the receiving application. The frames are sent as I, P, B, B, P, B, B, I.

Figure 15.16 *MPEG frames*



a. Frames b. Frame construction

Versions

MPEG has gone through several versions. The discussion above is related to MPEG-1. MPEG-2 was introduced in 1991, is more capable than MPEG and can be used for video storage as well as TV broadcasting, including high definition TV (HDTV). A more recent version of MPEG is called MPEG-7, which is named 'Multimedia Content Description Interface'. MPEG-7 is mostly a standard that uses XML to describe *metadata* (data about data), the description of what is included in the video.

15.3.3 Audio compression

Audio compression can be used for speech or music. For speech we need to compress a 64-kHz digitized signal, while for music we need to compress a 1.411-MHz signal. Two categories of techniques are used for audio compression: predictive encoding and perceptual encoding.

Predictive encoding

In **predictive encoding**, the differences between samples are encoded instead of encoding all the sampled values. This type of compression is normally used for speech. Several standards have been defined such as GSM (13 kbps), G.729 (8 kbps), and G.723.3 (6.4 or 5.3 kbps). Detailed discussions of these techniques are beyond the scope of this book.

Perceptual encoding: MP3

The most common compression technique used to create CD-quality audio is based on the **perceptual encoding** technique. This type of audio needs at least 1.411 Mbps, which cannot be sent over the Internet without compression. MP3 (MPEG audio layer 3), a part of the MPEG standard (discussed in the video compression section), uses this technique.

Perceptual encoding is based on the science of psychoacoustics, which is the study of how people perceive sound. The idea is based on flaws in our auditory system: some sounds can mask other sounds. Masking can happen in both frequency and time. In **frequency masking**, a loud sound in one frequency range can partially or totally mask a

5/1/25, 9:57 PM study viewer

softer sound in another frequency range. For example, we cannot hear what our dance partner says in a room in which a loud heavy metal band is performing. In **temporal masking**, a loud sound can reduce the sensitivity of our hearing for a short time even after the sound has stopped.

MP3 uses these two phenomena, frequency and temporal masking, to compress audio signals. The technique analyzes and divides the audio spectrum into several groups. Zero bits are allocated to frequency ranges that are totally masked, a small number of bits are allocated to frequency ranges that are partially masked, and a larger number of bits are allocated to frequency ranges that are not masked.

MP3 produces three data rates: 96 kbps, 128 kbps, and 160 kbps. The rate is based on the range of the frequencies in the original analog audio.

15.4 END-CHAPTER MATERIALS

15.4.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Drozdek, A. *Elements of Data Compression*, Boston, MA: Course Technology, 2001
- Symes, P. *Video Compression*, New York: McGraw-Hill, 1998
- Haskell, B., Puri, A. and Netravali, A. *Digital Video: An Introduction to MPEG2*, New York: Chapman and Hall, 1997
- Pennebaker, W. and Mitchell, J. *JPEG Still Image Data Compression Standard*, New York: Van Nostrand Reinhold, 1993

15.4.2 Key terms

AC value 402	lossless data compression 392
bidirectional frame (B-frame) 404	lossy data compression 400
data compression 392	Moving Picture Experts Group (MPEG) 400
DC value 402	MPEG audio layer 3 (MP3) 400
dictionary-based encoding 396	perceptual encoding 405
discrete cosine transform (DCT) 401	predicted frame (P-frame) 404
frequency masking 405	predictive encoding 405
Huffman coding 393	run-length encoding 392
intracoded frame (I-frame) 404	spatial compression 404
Joint Photographic Experts Group (JPEG) 400	temporal compression 404
Lempel Ziv (LZ) encoding 396	temporal masking 406
Lempel Ziv Welch (LZW) encoding 400	

https://studylib.net/doc/26059254/foundations-computer-science-4th 426/706

15.4.3 Summary

- Data compression methods are either lossless (all information is recoverable) or lossy (some information is lost).
- In lossless compression methods, the received data is an exact replica of the sent data. Three lossless compression methods are run-length encoding, Huffman coding, and Lempel Ziv (LZ) encoding.
- In run-length encoding, repeated occurrences of a symbol are replaced by a symbol and the number of occurrences of the symbol.
- In Huffman coding, the code length is a function of symbol frequency: more frequent symbols have shorter codes than less frequent symbols.
- In LZ encoding, repeated strings or words are stored in memory locations. An index to the memory location replaces the string or word. LZ encoding requires a dictionary and an algorithm at both sender and receiver.
- In lossy compression methods, the received data need not be an exact replica of the sent data. Three lossy compression methods were discussed in this chapter: JPEG, MPEG, and MP3.
- JPEG (Joint Photographic Experts Group) compression is a method of compressing pictures and graphics. The JPEG process involves blocking, the discrete cosine transform, quantization, and lossless compression.
- MPEG (Moving Pictures Experts Group) compression is a method of compressing video. MPEG involves both spatial compression and temporal compression. The former is similar to JPEG, while the latter removes redundant frames.
- MP3 (MPEG audio layer 3) is a part of the MPEG standard. MP3 uses perceptual encoding techniques to compress CD-quality audio.

15.5 PRACTICE SET

15.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

15.5.2 Review questions

Q15-1. What are the two categories of data compression methods?
Q15-2. What is the difference between lossless compression and lossy compression?
Q15-3. What is run-length encoding?
Q15-4. How does Lempel Ziv encoding reduce the amount of bits transmitted?
Q15-5. What is Huffman coding?
Q15-6. What is the role of the dictionary in LZ encoding?

Q15-7. What is the advantage of LZ encoding over Huffman coding?
Q15-8. Name three lossy compression methods.
Q15-9. When would you use JPEG? When would you use MPEG?
Q15-10. How is MPEG related to JPEG?
Q15-11. In JPEG, what is the function of blocking?
Q15-12. Why is the discrete cosine transform needed in JPEG?
Q15-13. How does quantization contribute to compression?
Q15-14. What is a frame in MPEG compression?
Q15-15. What is spatial compression compared to temporal compression?
Q15-16. Discuss the three types of frames used in MPEG.

15.5.3 Problems

P15-1. Encode the following bit pattern using run-length encoding with 5-bit codes:

18 zeros, 11, 56 zeros, 1, 15 zeros, 11

P15-2. Encode the following bit pattern using run-length encoding with 5-bit codes:

1, 8 zeros, 1, 45 zeros, 11

P15-3. Encode the following characters using Huffman coding with the given frequencies:

A (12), B (8), C (9), D (20), E (31), F (14), G (8)

P15-4. Encode the following characters using Huffman coding. Each character has the same frequency (1):

A, B, C, D, E, F, G, H, I, J

P15-5. Can the following be a Huffman code? Explain.

A: 0 B: 10 C:11

P15-6. Can the following be a Huffman code? Explain.

A: 0 B:1 C: 00 D: 01 E: 10 F: 11

P15-7. Encode the message BAABBBBAACAA using the following Huffman code:

A: 0 B: 10 C: 11

P15-8. Decode the message 0101000011110 using the following Huffman code:

A: 0 B: 10 C: 11

P15-9. Encode the message BAABBBBAACAA using the Lempel Ziv method, then decode the encoded message to get the original message.

P15-10. Encode the string AAAABBCCCBBB (part of a message) using the Lempel Ziv method if the dictionary contains ABB. Show the final contents of the dictionary.

P15-11. DCT evaluation requires a lot of calculation and is normally performed using a computer program. Instead of DCT, use the following rule to transform a 2×2 table:

$$T(0,0) = (1/16)\,[P(0,0) + P(0,1) + P(1,0) + P(1,1)]$$
$$T(0,1) = (1/16)\,[0.95P(0,0) + 0.9P(0,1) + 0.85P(1,0) + 0.80P(1,1)]$$
$$T(1,0) = (1/16)\,[0.90P(0,0) + 0.85P(0,1) + 0.80P(1,0) + 075P(1,1)]$$
$$T(1,1) = (1/16)\,[0.85P(0,0) + 0.80P(0,1) + 0.75P(1,0) + 0.70P(1,1)]$$

If $P(0,0) = 64$, $P(0,1) = 32$, $P(1,0) = 128$, and $P(1,1) = 148$, find $T(0,0)$, $T(0,1)$, $T(1,0)$, and $T(1,1)$.

CHAPTER 16

Security

The topic of security is very broad and involves some specific areas of mathematics such as number theory. In this chapter, we try to give a very simple introduction to this topic to prepare the background for more study.

Objectives

After studying this chapter, the student should be able to:

- Define security goals: confidentiality, integrity, and availability.
- Show how confidentiality can be achieved using symmetric-key and asymmetric-key cipher.
- Discuss other aspects of security: message integrity, message authentication, digital signature, entity authentication, and key management.
- Discuss the use of firewalls to protect a system from harmful messages.

16.1 INTRODUCTION

We are living in the information age. We need to keep information about every aspect of our lives. In other words, information is an asset that has a value like any other asset. As an asset, information needs to be secured from attacks. To be secure, information needs to be hidden from unauthorized access (*confidentiality*), protected from unauthorized change (*integrity*), and available to an authorized entity when it is needed (*availability*).

During the last three decades, computer networks have created a revolution in the use of information. Information is now distributed. Authorized people can send and retrieve information from a distance using computer networks. Although the three above-mentioned requirements—confidentiality, integrity, and availability—have not changed, they now have some new dimensions. Not only should information be confidential when it is stored; there should also be a way to maintain its confidentiality when it is transmitted from one computer to another.

In this section, we first discuss the three major goals of information security. We then see how attacks can threaten these three goals. We then discuss the security services in relation to these **security goals**. Finally, we define two techniques to implement the security goals and prevent attacks.

16.1.1 Security goals

Let us first discuss the three security goals: confidentiality, integrity, and availability.

Confidentiality

Confidentiality is probably the most common aspect of information security. We need to protect our confidential information. An organization needs to guard against those malicious actions that endanger the confidentiality of its information. Confidentiality not only applies to the storage of information, it also applies to the transmission of information. When we send a piece of information to be stored in a remote computer or when we retrieve a piece of information from a remote computer, we need to conceal it during transmission.

Integrity

Information needs to be changed constantly. In a bank, when a customer deposits or withdraws money, the balance of their account needs to be changed. **Integrity** means that changes need to be done only by authorized entities and through authorized mechanisms. Integrity violation is not necessarily the result of a malicious act; an interruption in the system, such as a power surge, may also create unwanted changes in some information.

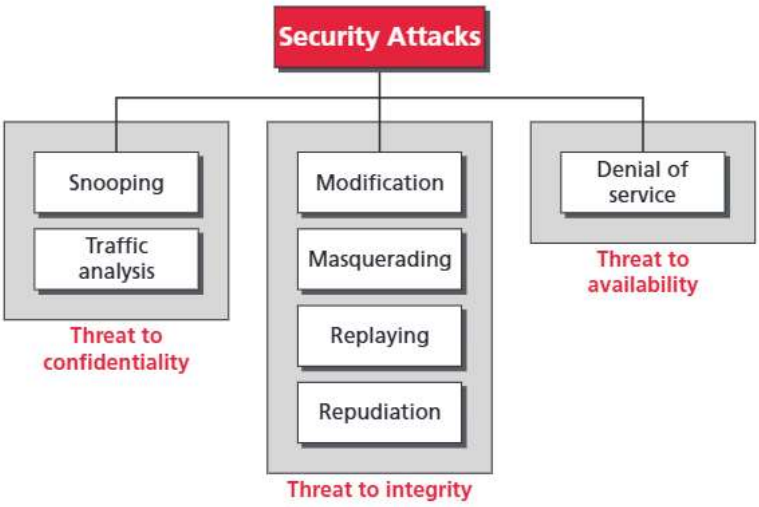
Availability

The third component of information security is **availability**. The information created and stored by an organization needs to be available to authorized entities. Information is useless if it is not available. Information needs to be constantly changed, which means it must be accessible to authorized entities. The unavailability of information is just as harmful for an organization as the lack of confidentiality or integrity. Imagine what would happen to a bank if the customers could not access their accounts for transactions.

16.1.2 Attacks

Our three goals of security—confidentiality, integrity, and availability—can be threatened by **security attacks**. Although the literature uses different approaches to categorizing the attacks, we divide them into three groups related to the security goals. Figure 16.1 shows the taxonomy.

Figure 16.1 *Taxonomy of attacks with relation to security goals*



Attacks threatening confidentiality

In general, two types of attacks threaten the confidentiality of information: **snooping** and **traffic analysis**.

Snooping

Snooping refers to unauthorized access to or interception of data. For example, a file transferred through the Internet may contain confidential information. An unauthorized entity may intercept the transmission and use the contents for their own benefit. To prevent snooping, the data can be made nonintelligible to the intercepter by using encipherment techniques discussed later.

Traffic analysis

Although encipherment of data may make it nonintelligible for the intercepter, they can obtain some other type of information by monitoring online traffic. For example, they can find the electronic address (such as the email address) of the sender or the receiver. They can collect pairs of requests and responses to help them guess the nature of the transaction.

Attacks threatening integrity

The integrity of data can be threatened by several kinds of attacks: *modification*, *masquerading*, *replaying*, and *repudiation*.

Modification

After intercepting or accessing information, the attacker modifies the information to make it beneficial to them. For example, a customer sends a message to a bank to initiate some transaction. The attacker intercepts the message and changes the type of transaction to benefit them. Note that sometimes the attacker simply deletes or delays the message to harm the system or to benefit from it.

Masquerading

Masquerading, or **spoofing**, happens when the attacker impersonates somebody else. For example, an attacker might steal the bank card and PIN of a bank customer and pretend that they are that customer. Sometimes the attacker pretends instead to be the receiver entity. For example, a user tries to contact a bank, but another site pretends that it is the bank and obtains some information from the user.

Replaying

Replaying is another attack. The attacker obtains a copy of a message sent by a user and later tries to replay it. For example, a person sends a request to their bank to ask for payment to the attacker, who has done a job for them. The attacker intercepts the message and sends it again to receive another payment from the bank.

Repudiation

This type of attack is different from others because it is performed by one of the two parties in the communication: the sender or the receiver. The sender of the message might later deny that they have sent the message; the receiver of the message might later deny that they have received the message. An example of denial by the sender would be a bank customer asking their bank to send some money to a third party but later denying that they have made such a request. An example of denial by the receiver could occur when a person buys a product from a manufacturer and pays for it electronically, but the manufacturer later denies having received the payment and asks to be paid.

Attacks threatening availability

We mention only one attack threatening availability: *denial of service*.

Denial of service

Denial of service (DoS) is a very common attack. It may slow down or totally interrupt the service of a system. The attacker can use several strategies to achieve this. They might send so many bogus requests to a server that the server crashes because of the heavy load. The attacker might intercept and delete a server's response to a client, making the client believe that the server is not responding. The attacker may also intercept requests from the clients, causing the clients to send requests many times and overload the system.

16.1.3 Services and techniques

ITU-T defines some security services to achieve security goals and prevent attacks. Each of these services is designed to prevent one or more attacks while maintaining security goals. The actual implementation of security goals needs some techniques.

Two techniques are prevalent today: one is very general (cryptography) and one is specific (steganography).

Cryptography

Some security services can be implemented using cryptography. **Cryptography**, a word with Greek origins, means 'secret writing'. However, we use the term to refer to the science and art of transforming messages to make them secure and immune to attacks. Although in the past *cryptography* referred only to the **encryption** and **decryption** of messages using **secret keys**, today it is defined as involving three distinct mechanisms: symmetric-key encipherment, asymmetric-key encipherment, and hashing. We will discuss all these mechanisms later in the chapter.

Steganography

Although this chapter and the next are based on cryptography as a technique for implementing security mechanisms, another technique that was used for secret communication in the past is being revived at the present time: steganography. The word **steganography**, with origins in Greek, means 'covered writing', in contrast to *cryptography*, which means 'secret writing'. *Cryptography* means concealing the contents of a message by enciphering; *steganography* means concealing the message itself by covering it with something else. We leave the discussion of steganography to those books dedicated to this topic.

16.2 CONFIDENTIALITY

We now look at the first goal of security, confidentiality. Confidentiality can be achieved using ciphers. **Ciphers** can be divided into two broad categories: symmetric-key and asymmetric-key.

16.2.1 Symmetric-key ciphers

A **symmetric-key cipher** uses the same key for both encryption and decryption, and the key can be used for bidirectional communication, which is why it is called *symmetric*. Figure 16.2 shows the general idea behind a symmetric-key cipher.

Figure 16.2 *General idea of a symmetric-key cipher*

https://sanet.st/blogs/polatebooks/

Symmetric-key ciphers are also called secret-key ciphers.

In Figure 16.2, an entity, Alice, can send a message to another entity, Bob, over an insecure channel with the assumption that an adversary, Eve, cannot understand the contents of the message by simply eavesdropping over the channel.

The original message from Alice to Bob is called **plaintext**; the message that is sent through the channel is called **ciphertext**. To create the ciphertext from the plaintext, Alice uses an **encryption algorithm** and a *shared secret key*.

To create the plaintext from ciphertext, Bob uses a **decryption algorithm** and the same secret key. We refer to encryption and decryption algorithms as **ciphers**. A **key** is a set of values (numbers) that the cipher, as an algorithm, operates on.

Note that the symmetric-key encipherment uses a single key (the key itself may be a set of values) for both encryption and decryption. In addition, the encryption and decryption algorithms are inverses of each other. If P is the plaintext, C is the ciphertext, and K is the key, the encryption algorithm $E_k(x)$ creates the ciphertext from the plaintext; the decryption algorithm $D_k(x)$ creates the plaintext from the ciphertext. We assume that $E_k(x)$ and $D_k(x)$ are inverses of each other: they cancel the effect of each other if they are applied one after the other on the same input. We have:

Encryption: $C = E_k(P)$	Decryption: $P = D_k(C)$

in which, $D_k(E_k(x)) = E_k(D_k(x)) = x$. We need to emphasize that it is better to make the encryption and decryption public but keep the shared key secret. This means that Alice and Bob need another channel, a secured one, to exchange the secret key. Alice and Bob can meet once and exchange the key personally. The secured channel here is the face-to-face exchange of the key. They can also trust a third party to give them the same key. They can create a temporary secret key using another kind of cipher—asymmetric-key ciphers—which will be described later.

Encryption can be thought of as locking the message in a box; decryption can be thought of as unlocking the box. In symmetric-key encipherment, the same key locks and unlocks, as shown in Figure 16.3. Later sections show that the *asymmetric-key* encipherment needs two keys, one for locking and one for unlocking.

Figure 16.3 *Symmetric-key encipherment as locking and unlocking with the same key*

The symmetric-key ciphers can be divided into traditional ciphers and modern ciphers. Traditional ciphers are simple, character-oriented ciphers that are not secured based on

today's standard. Modern ciphers, on the other hand, are complex, bit-oriented ciphers that are more secure. We briefly discuss the traditional ciphers to pave the way for discussing more complex modern ciphers.

Traditional symmetric-key ciphers

Traditional ciphers belong to the past. However, we briefly discuss them here because they can be thought of as the components of the modern ciphers. To be more exact, we can divide traditional ciphers into substitution ciphers and transposition ciphers.

Substitution ciphers

A **substitution cipher** replaces one symbol with another. If the symbols in the plaintext are alphabetic characters, we replace one character with another. For example, we can replace letter A with letter D and letter T with letter Z. If the symbols are digits (0 to 9), we can replace 3 with 7 and 2 with 6.

A substitution cipher replaces one symbol with another.

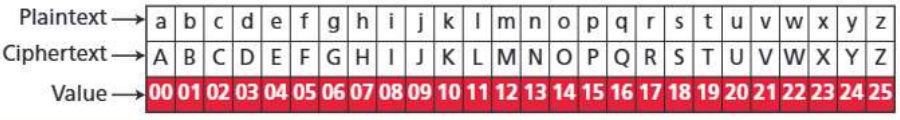
Substitution ciphers can be categorized as either monoalphabetic ciphers or polyalphabetic ciphers.

Monoalphabetic ciphers

In a **monoalphabetic cipher**, a character (or a symbol) in the plaintext is always changed to the same character (or symbol) in the ciphertext regardless of its position in the text. For example, if the algorithm says that letter A in the plaintext is changed to letter D, every letter A is changed to letter D. In other words, the relationship between letters in the plaintext and the ciphertext is one-to-one.

The simplest monoalphabetic cipher is the **additive cipher** (or **shift cipher**). Assume that the plaintext consists of lowercase letters (a to z), and that the ciphertext consists of uppercase letters (A to Z). To be able to apply mathematical operations on the plaintext and ciphertext, we assign numerical values to each letter (lower- or uppercase), as shown in Figure 16.4.

Figure 16.4 *Representation of plaintext and ciphertext characters in modulo 26*



In Figure 16.4 each character (lowercase or uppercase) is assigned an integer in modulo 26. The secret key between Alice and Bob is also an integer in modulo 26. The encryption algorithm adds the key to the plaintext character; the decryption algorithm subtracts the key from the ciphertext character. All operations are done in modulo 26.

In additive cipher, the plaintext, ciphertext, and key are integers in modulo 26.

Historically, additive ciphers are called shift ciphers because the encryption algorithm can be interpreted as 'shift *key* characters down' and the encryption algorithm can be interpreted as 'shift *key* characters up'. Julius Caesar used an additive cipher, with a key of 3, to communicate with his officers. For this reason, additive ciphers are sometimes referred to as the **Caesar cipher.**

Example 16.1

Use the additive cipher with key = 15 to encrypt the message 'hello'.

Solution

We apply the encryption algorithm to the plaintext, character by character:

Plaintext: h → 07	Encryption: (07 + 15) mod 26	Ciphertext: 22 → W
Plaintext: e → 04	Encryption: (04 + 15) mod 26	Ciphertext: 19 → T
Plaintext: l → 11	Encryption: (11 + 15) mod 26	Ciphertext: 00 → A
Plaintext: l → 11	Encryption: (11 + 15) mod 26	Ciphertext: 00 → A
Plaintext: o → 14	Encryption: (14 + 15) mod 26	Ciphertext: 03 → D

The result is 'WTAAD'. Note that the cipher is monoalphabetic because two instances of the same plaintext character (*l*) are encrypted as the same character (*A*).

Example 16.2

Use the additive cipher with key = 15 to decrypt the message 'WTAAD'.

Solution

We apply the decryption algorithm to the plaintext character by character:

Ciphertext: W →22	Decryption: (22 → 15) mod 26	Plaintext: 07 → h
Ciphertext: T → 19	Decryption: (19 → 15) mod 26	Plaintext: 04 → e
Ciphertext: A → 00	Decryption: (00 → 15) mod 26	Plaintext: 11 → l
Ciphertext: A → 00	Decryption: (00 → 15) mod 26	Plaintext: 11 → l
Ciphertext: D → 03	Decryption: (03 → 15) mod 26	Plaintext: 14 → o

The result is 'hello'. Note that the operation is in modulo 26, which means that we need to add 26 to a negative result (for example—15 becomes 11).

Polyalphabetic ciphers

In a **polyalphabetic cipher,** each occurrence of a character may have a different substitute. The relationship between a character in the plaintext to a character in the ciphertext is one-to-many. For example, 'a' could be enciphered as 'D' at the beginning of the text, but as 'N' in the middle. Polyalphabetic ciphers have the advantage of hiding the letter frequency of the underlying language. Eve cannot use single-letter frequency statistics to break the ciphertext.

To create a polyalphabetic cipher, we need to make each ciphertext character dependent on both the corresponding plaintext character and the position of the plaintext character in the message. This implies that our key should be a stream of subkeys, in which

each subkey depends somehow on the position of the plaintext character that uses that subkey for encipherment. In other words, we need to have a key stream $k = (k_1, k_2, k_3, \ldots)$ in which k_i is used to encipher the ith character in the plaintext to create the ith character in the ciphertext.

To see the position dependency of the key, let us discuss a simple polyalphabetic cipher called the **autokey cipher.** In this cipher, the key is a stream of subkeys, in which each subkey is used to encrypt the corresponding character in the plaintext. The first subkey is a predetermined value secretly agreed upon by Alice and Bob. The second subkey is the value of the first plaintext character (between 0 and 25). The third subkey is the value of the second plaintext character, and so on:

$$P = P_1P_2P_3 \ldots \qquad C = C_1C_2C_3 \ldots \qquad k = (k_1, P_1, P_2, \ldots)$$

$$\text{Encryption: } C_i = (P_i + k_i) \bmod 26 \qquad \text{Decryption: } P_i = (C_i - k_i) \bmod 26$$

The name of the cipher, *autokey*, implies that the subkeys are automatically created from the plaintext cipher characters during the encryption process.

Example 16.3

Assume that Alice and Bob agreed to use an autokey cipher with initial key value $k_1 = 12$. Now Alice wants to send Bob the message 'Attack is today'. Enciphering is done character by character. Each character in the plaintext is first replaced by its integer value. The first subkey is added to create the first ciphertext character. The rest of the key is created as the plaintext characters are read. Note that the cipher is polyalphabetic because the three occurrences of 'a' in the plaintext are encrypted differently. The three occurrences of 't' are encrypted differently:

Plaintext:	a	t	t	a	c	k	i	s	t	o	d	a	y
P's Values:	00	19	19	00	02	10	08	18	19	14	03	00	24
Key stream:	*12*	*00*	*19*	*19*	*00*	*02*	*10*	*08*	*18*	*19*	*14*	*03*	*00*
C's Values:	12	19	12	19	02	12	18	00	11	7	17	03	24
Ciphertext:	M	T	M	T	C	M	S	A	L	H	R	D	Y

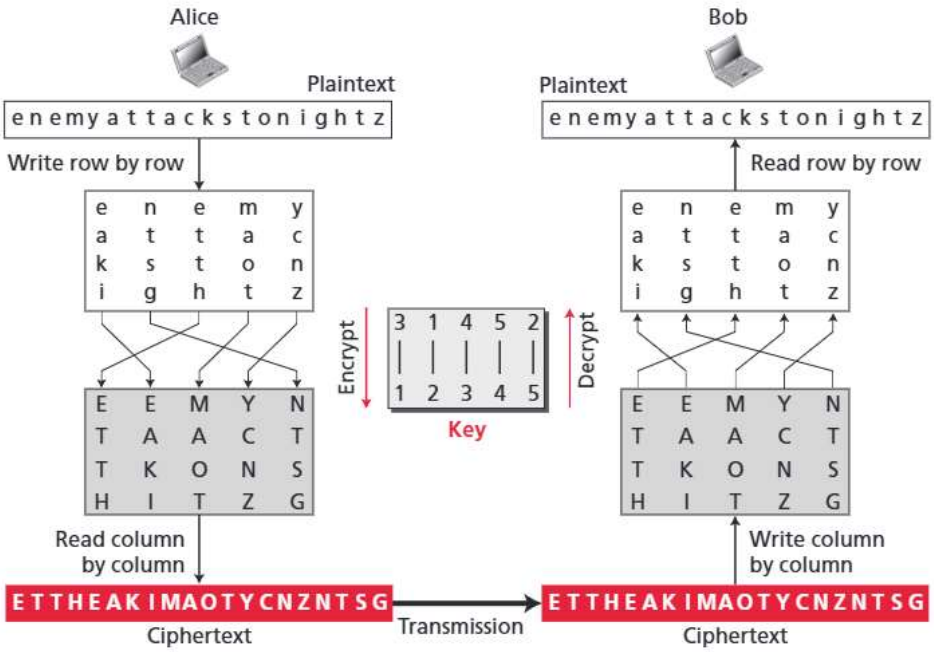
Transposition ciphers

A **transposition cipher** does not substitute one symbol for another; instead it changes the location of the symbols. A symbol in the first position of the plaintext may appear in the tenth position of the ciphertext. A symbol in the eighth position in the plaintext may appear in the first position of the ciphertext. In other words, a transposition cipher reorders (transposes) the symbols.

A transposition cipher reorders symbols.

Suppose Alice wants to secretly send the message 'Enemy attacks tonight' to Bob. The encryption and decryption is shown in Figure 16.5. Note that we added an extra character (z) to the end of the message to make the number of characters a multiple of 5.

Figure 16.5 *Transposition cipher*



The first table is created by Alice writing the plaintext row by row. The columns are permuted using a key. The ciphertext is created by reading the second table column by column. Bob does the same three steps in the reverse order. He writes the ciphertext column by column into the first table, permutes the columns, and then reads the second table row by row. Note that the same key is used for encryption and decryption, but the algorithm uses the key in reverse order.

Stream and block ciphers

The literature divides the symmetric ciphers into two broad categories: stream ciphers and block ciphers.

Stream cipher

In a **stream cipher,** encryption and decryption are done one symbol (such as a character or a bit) at a time. We have a plaintext stream, a ciphertext stream, and a key stream. Call the plaintext stream P, the ciphertext stream C, and the key stream K:

$$P = P_1P_2P_3, \ldots \qquad C = C_1C_2C_3, \ldots \qquad K = (k_1, k_2, k_3, \ldots)$$

$$C_1 = E_{k1}(P_1) \qquad C_2 = E_{k2}(P_2) \qquad C_3 = E_{k3}(P_3) \ldots$$

Block ciphers

In a **block cipher**, a group of plaintext symbols of size m ($m > 1$) are encrypted together, creating a group of ciphertext of the same size. Based on the definition, in a block cipher, a single key is used to encrypt the whole block even if the key is made of multiple values. In a block cipher, a ciphertext block depends on the whole plaintext block.

Combination

In practice, blocks of plaintext are encrypted individually, but they use a stream of keys to encrypt the whole message block by block. In other words, the cipher is a block cipher when looking at the individual blocks, but it is a stream cipher when looking at the whole message, considering each block as a single unit. Each block uses a different key that may be generated before or during the encryption process.

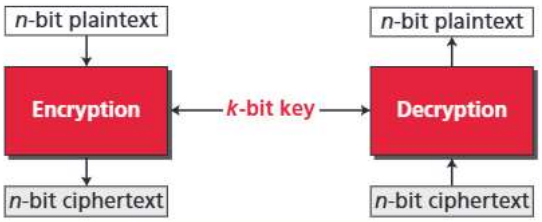
Modern symmetric-key ciphers

The traditional symmetric-key ciphers that we have studied so far are *character-oriented ciphers*. With the advent of the computer, we need *bit-oriented ciphers*. This is because the information to be encrypted is not just text; it can also consist of numbers, graphics, audio, and video data. It is convenient to convert these types of data into a stream of bits, to encrypt the stream, and then to send the encrypted stream. In addition, when text is treated at the bit level, each character is replaced by 8 (or 16) bits, which means that the number of symbols becomes 8 (or 16) times larger. Mixing a larger number of symbols increases security. A modern cipher can be either a block cipher or a stream cipher.

Modern block ciphers

A symmetric-key *modern block cipher* encrypts an n-bit block of plaintext or decrypts an n-bit block of ciphertext. The encryption or decryption algorithm uses a k-bit key. The decryption algorithm must be the inverse of the encryption algorithm, and both operations must use the same secret key so that Bob can retrieve the message sent by Alice. Figure 16.6 shows the general idea of encryption and decryption in a modern block cipher.

Figure 16.6 *A modern block cipher*



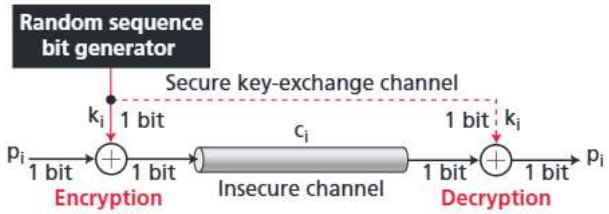
If the message has fewer than n bits, padding must be added to make it an n-bit block; if the message has more than n bits, it should be divided into n-bit blocks and the appropriate padding must be added to the last block if necessary. The common values for n are 64, 128, 256, and 512 bits.

Modern stream ciphers

In addition to modern block ciphers, we can also use modern stream ciphers. The differences between modern stream ciphers and modern block ciphers are similar to the differences between traditional stream and block ciphers, which we explained in the previous section. In a *modern stream cipher,* encryption and decryption are done r bits at a time. We have a plaintext bit stream $P = p_n \ldots p_2 p_1$, a ciphertext bit stream $C = c_n \ldots c_2 c_1$, and a key bit stream $K = k_n \ldots k_2 k_1$, in which p_i, c_i, and k_i are r-bit words. Encryption is $c_i = E(k_i, p_i)$, and decryption is $p_i = D(k_i, c_i)$. Stream ciphers are faster than block ciphers. The hardware implementation of a stream cipher is also easier. When we need to encrypt binary streams and transmit them at a constant rate, a stream cipher is the better choice to use. Stream ciphers are also more immune to the corruption of bits during transmission.

The simplest and the most secure type of synchronous stream cipher is called the **one-time pad,** which was invented and patented by Gilbert Vernam. A one-time pad cipher uses a key stream that is randomly chosen for each encipherment. The encryption and decryption algorithms each use a single exclusive-OR operation. Based on properties of the exclusive-OR operation, the encryption and decryption algorithms are inverses of each other. It is important to note that in this cipher the exclusive-OR operation is used one bit at a time. Note also that there must be a secure channel so that Alice can send the key stream sequence to Bob (Figure 16.7).

Figure 16.7 *One-time pad*



The one-time pad is an ideal cipher. It is perfect. There is no way that an adversary can guess the key or the plaintext and ciphertext statistics. There is no relationship between the plaintext and ciphertext, either. In other words, the ciphertext is a true random stream of bits even if the plaintext contains some patterns. Eve cannot break the cipher unless she tries all possible random key streams, which would be 2^n if the size of the plaintext is n bits. However, there is an issue here. How can the sender and the receiver share a one-time pad key each time they want to communicate? They need to somehow agree on the random key. So this perfect and ideal cipher is very difficult to achieve. However, there are some feasible, less secured, versions. One of the common alternatives is called a *feedback*

shift register (FSR), but we leave the discussion of this interesting cipher to the books dedicated to the security topic.

16.2.2 Asymmetric-key ciphers

In previous sections we discussed symmetric-key ciphers. In this section, we start the discussion of **asymmetric-key ciphers**. Symmetric- and asymmetric-key ciphers will exist in parallel and continue to serve the community. We actually believe that they are complements of each other; the advantages of one can compensate for the disadvantages of the other.

The conceptual differences between the two systems are based on how these systems keep a secret. In symmetric-key cryptography, the secret must be shared between two persons. In asymmetric-key cryptography, the secret is personal (unshared); each person creates and keeps his or her own secret.

In a community of n people, $n\ (n - 1)/2$ shared secrets are needed for symmetric-key cryptography; only n personal secrets are needed in asymmetric-key cryptography. For a community with a population of 1 million, symmetric-key cryptography would require half a billion shared secrets; asymmetric-key cryptography would require one million personal secrets.

Symmetric-key cryptography is based on sharing secrecy; asymmetric-key cryptography is based on personal secrecy.

There are some other aspects of security besides encipherment that need asymmetric-key cryptography. These include authentication and digital signatures. Whenever an application is based on a personal secret, we need to use asymmetric-key cryptography.

Whereas symmetric-key cryptography is based on substitution and permutation of symbols (characters or bits), asymmetric-key cryptography is based on applying mathematical functions to numbers. In symmetric-key cryptography, the plaintext and ciphertext are thought of as a combination of symbols. Encryption and decryption permute these symbols or substitute one symbol for another. In asymmetric-key cryptography, the plaintext and ciphertext are numbers; encryption and decryption are mathematical functions that are applied to numbers to create other numbers.

In symmetric-key cryptography, symbols are permuted or substituted; in asymmetric-key cryptography, numbers are manipulated.

Asymmetric-key cryptography uses two separate keys: one private and one public. If encryption and decryption are thought of as locking and unlocking padlocks with keys, then the padlock that is locked with a public key can be unlocked only with the corresponding private key. Figure 16.8 shows that if Alice locks the padlock with Bob's public key, then only Bob's private key can unlock it.

Figure 16.8 *Locking and unlocking in asymmetric-key cryptosystem*

The figure shows that, unlike symmetric-key cryptography, there are distinctive keys in asymmetric-key cryptography: a **private key** and a **public key.** Although some books use the term *secret key* instead of *private key,* we use the term *secret key* only for symmetric-key cryptography and the terms *private key* and *public key* for asymmetric-key cryptography. We even use different symbols to show the three keys. In other words, we want to show that a *secret key* is not exchangeable with a *private key;* there are two different types of secrets.

Asymmetric-key ciphers are sometimes called public-key ciphers.

General idea

Figure 16.9 shows the general idea of asymmetric-key cryptography as used for encipherment.

Figure 16.9 *General idea of asymmetric-key cryptosystem*

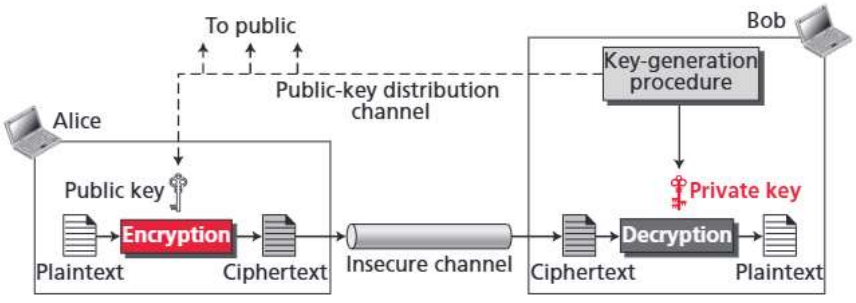


Figure 16.9 shows several important facts. First, it emphasizes the asymmetric nature of the cryptosystem. The burden of providing security is mostly on the shoulders of the receiver (Bob, in this case). Bob needs to create two keys: one private and one public. Bob is responsible for distributing the public key to the community. This can

be done through a public-key distribution channel. Although this channel is not required to provide secrecy, it must provide authentication and integrity. Eve should not be able to advertise her public key to the community pretending that it is Bob's public key.

Second, asymmetric-key cryptography means that Bob and Alice cannot use the same set of keys for two-way communication. Each entity in the community should create its own private and public keys. Figure 16.9 shows how Alice can use Bob's public key to send encrypted messages to Bob. If Bob wants to respond, Alice needs to establish her own private and public keys.

Third, asymmetric-key cryptography means that Bob needs only one private key to receive all correspondence from anyone in the community, but Alice needs n public keys to communicate with n entities in the community, one public key for each entity. In other words, Alice needs a ring of public keys.

Plaintext/ciphertext

Unlike in symmetric-key cryptography, plaintext and ciphertext in asymmetric-key cryptography are treated as integers. The message must be encoded as an integer (or a set of integers) before encryption; the integer (or the set of integers) must be decoded into the message after decryption. Asymmetric-key cryptography is normally used to encrypt or decrypt small pieces of information, such as the cipher key for a symmetric-key cryptography. In other words, asymmetric-key cryptography is normally used for ancillary goals instead of message encipherment. However, these ancillary goals play a very important role in cryptography today.

Asymmetric-key cryptography is normally used to encrypt or decrypt small pieces of information.

Encryption/decryption

Encryption and decryption in asymmetric-key cryptography are mathematical functions applied over the numbers representing the plaintext and ciphertext. The ciphertext can be thought of as $C = f(K_{public}, P)$; the plaintext can be thought of as $P = g(K_{private}, C)$. The decryption function f is used only for encryption; the decryption function g is used only for decryption.

Need for both

There is a very important fact that is sometimes misunderstood: the advent of asymmetric-key (public-key) cryptography does not eliminate the need for symmetric-key (secret-key) cryptography. The reason is that asymmetric-key cryptography, which uses mathematical functions for encryption and decryption, is much slower than symmetric-key cryptography. For encipherment of large messages, symmetric-key cryptography is still needed. On the other hand, the speed of symmetric-key cryptography does not eliminate the need for asymmetric-key cryptography. Asymmetric-key cryptography is still needed for authentication, digital signatures, and secret-key exchanges. This means that, to be able to use all aspects of security today, we need both symmetric-key and asymmetric-key cryptography. One complements the other.

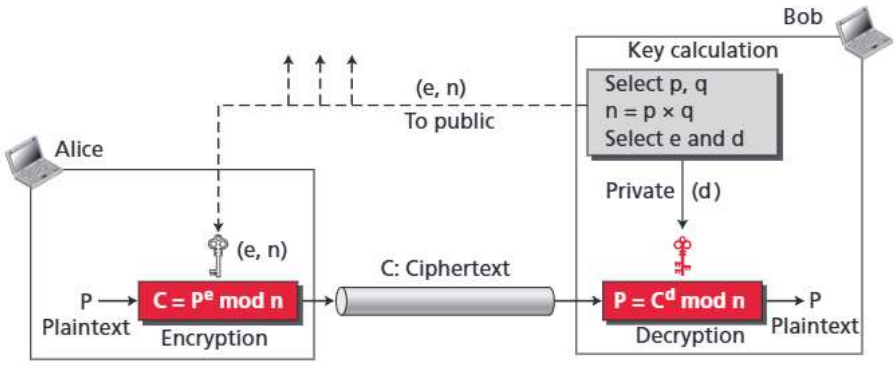
RSA cryptosystem

Although there are several asymmetric-key cryptosystems, one of the common public-key algorithms is the **RSA cryptosystem,** named for its inventors (Rivest, Shamir, and Adleman). RSA uses two exponents, e and d, where e is public and d is private. Suppose P is the plaintext and C is the ciphertext. Alice uses $C = P^e \bmod n$ to create ciphertext C from plaintext P; Bob uses $P = C^d \bmod n$ to retrieve the plaintext sent by Alice. The modulus n, a very large number, is created during the key generation process.

Procedure

Figure 16.10 shows the general idea behind the procedure used in RSA.

Figure 16.10 *Encryption, decryption, and key generation in RSA*



Bob chooses two large numbers, p and q, and calculates $n = p \times q$ and $\phi = (p - 1) \times (q - 1)$. Bob then selects e and d such that $(e \times d) \bmod \phi = 1$. Bob advertises e and n to the community as the public key; Bob keeps d as the private key. Anyone, including Alice, can encrypt a message and send the ciphertext to Bob, using $C = (P^e) \bmod n$; only Bob can decrypt the message, using $P = (C^d) \bmod n$. An intruder such as Eve cannot decrypt the message if p and q are very large numbers (she does not know d).

Example 16.4

For the sake of demonstration, let Bob choose 7 and 11 as p and q and calculate $n = 7 \times 11 = 77$. The value of $\phi(n) = (7 - 1)(11 - 1)$, or 60. If he chooses e to be 13, then d is 37. Note that $e \times d \bmod 60 = 1$. Now imagine that Alice wants to send the plaintext 5 to Bob. She uses the public exponent 13 to encrypt 5. This system is not safe because p and q are small.

Plaintext: 5
$C = 5^{13} = 26 \bmod 77$
Ciphertext: 26

Ciphertext: 26
$P = 26^{37} = 5 \bmod 77$
Plaintext: 5

Example 16.5

Here is a more realistic example calculated using a computer program in Java. We choose a 512-bit p and q, and calculate n and $\phi(n)$. We then choose e and calculate d. Finally, we show the results of encryption and decryption. The integer p is a 159-digit number:

```
p =  961303453135835045741915812806154279093098455949962158225831508796479404550564706384912571601803475031209866660649242019180878066742109606335421992666120 9
```

The integer q is a 160-digit number:

```
q =  120601919572314469182767942044508960015559250546370339360617983217314821484837646592153894532091752252732268301071206956046025138871455249690003596600456 17
```

The modulus $n = p \times q$. It has 309 digits:

```
n =  115935041739676149688925098646158875237714573754541447754855261376147885408326350817276878815968325168468849300625485764111250162414552339182927162507656772727460097082714127730434960500556347274566628060099924037102991424472292215772798531727033839381334692684137327622000966676671831831088373420823444370953
```

$\phi(n) = (p - 1)(q - 1)$ has 309 digits:

```
φ(n) =  115935041739676149688925098646158875237714573754541447754855261376147885408326350817276878815968325168468849300625485764111250162414552339182927162507656751054233608492916752034482627988117554787657013923444405716989581728196098226361075467211864612171359107358640614008885170265377277264467341066243857664128
```

Bob chooses $e = 35535$ (the ideal is 65537). He then finds d:

```
e =  35535

d =  580083028600377639360936612896779175946690620896509621804228661113805938528223587317062869100300217108590443384021707298690876006115306202524959884448047568240966247081485817130463240644077704833134010850947385295645071936774061197326557424237217617674620776371642076003370853332885321447088595513667029483 1
```

Alice wants to send the message 'THIS IS A TEST', which can be changed to a numeric value using the 00–26 encoding scheme (26 is the *space* character):

```
P =  19070818260818260026190418 19
```

The ciphertext calculated by Alice is $C = P^e$, which is shown below:

```
C =  4753091236462268272063655506105451809423717960704917165232392430544529606131993285666178434183591141511974112520056829797945717360361012782188478927415660904800235071907152771859149751884658886321011483541033616578984679683867637337657774656250792805211481418404081418443081277305900469287424855916646210865 6
```

Bob can recover the plaintext from the ciphertext using $P = C^d$, which is shown below:

```
P =  190708182608182600261904181 9
```

The recovered plaintext is 'THIS IS A TEST' after decoding.

Applications

Although RSA can be used to encrypt and decrypt actual messages, it is very slow if the message is long. RSA, therefore, is useful for short messages. In particular, we will see that RSA is used in digital signatures and other cryptosystems that often need to encrypt a small message without having access to a symmetric key. RSA is also used for authentication, as we will see later in the chapter.

16.3 OTHER ASPECTS OF SECURITY

The cryptography systems that we have studied so far provide confidentiality. However, in modern communication, we need to take care of other aspects of security, such as integrity, message and entity authentication, non-repudiation, and key management. We briefly discuss these issues in this section.

16.3.1 Message integrity

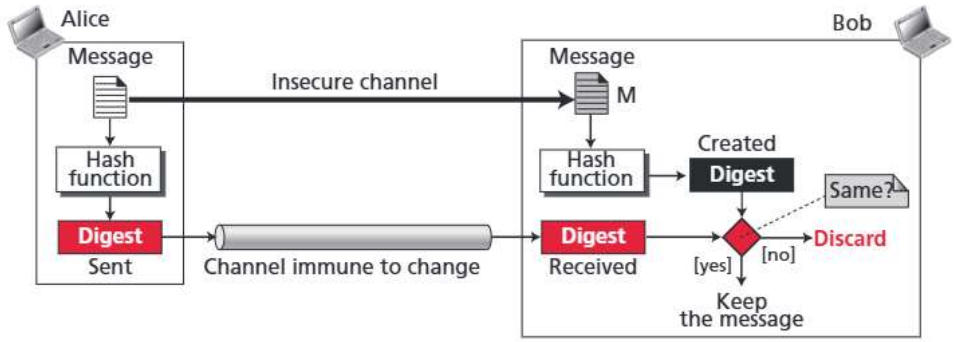
There are occasions where we may not even need secrecy but instead must have integrity: the message should remain unchanged. For example, Alice may write a will to distribute her estate upon her death. The will does not need to be encrypted. After her death, anyone can examine the will. The integrity of the will, however, needs to be preserved. Alice does not want the contents of the will to be changed.

Message and message digest

One way to preserve the integrity of a document is through the use of a *fingerprint*. If Alice needs to be sure that the contents of her document will not be changed, she can put her fingerprint at the bottom of the document. Eve cannot modify the contents of this document or create a false document because she cannot forge Alice's fingerprint. To ensure that the document has not been changed, Alice's fingerprint on the document can be compared to Alice's fingerprint on file. If they are not the same, the document is not from Alice. The electronic equivalent of the document and fingerprint pair is the *message* and *digest* pair. To preserve the integrity of a message, the message is passed through an

algorithm called a **cryptographic hash function**. The function creates a compressed image of the message, called a **digest**, that can be used like a fingerprint. To check the integrity of a message or document, Bob runs the cryptographic hash function again and compares the new digest with the previous one. If both are the same, Bob is sure that the original message has not been changed. Figure 16.11 shows the idea.

Figure 16.11 Message and digest



The two pairs (document/fingerprint) and (message/message digest) are similar, with some differences. The document and fingerprint are physically linked together. The message and message digest can be unlinked (or sent separately), and, most importantly, the message digest needs to be safe from change.

The message digest needs to be safe from change.

Hash functions

A cryptographic hash function takes a message of arbitrary length and creates a message digest of fixed length. All cryptographic hash functions need to create a fixed-size digest out of a variable-size message. Creating such a function is best accomplished using iteration. Instead of using a hash function with variable-size input, a function with fixed-size input is created and is used a necessary number of times. The fixed-size input function is referred to as a *compression function*. It compresses an n-bit string to create an m-bit string where n is normally greater than m. The scheme is referred to as an *iterated cryptographic hash function*.

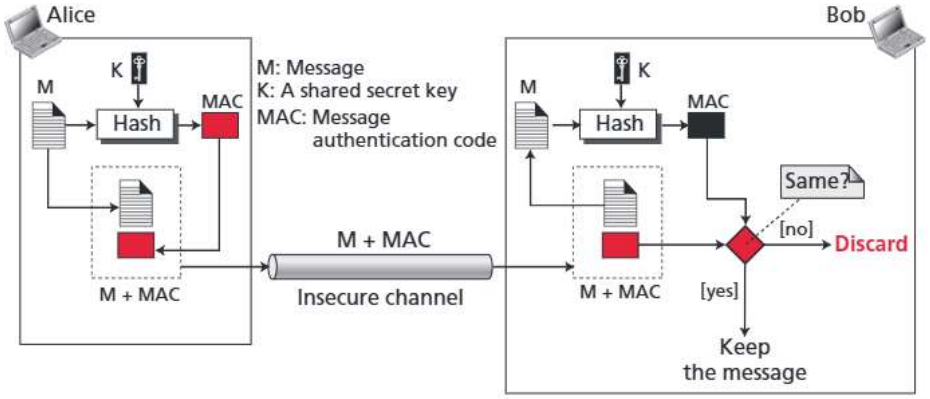
Several hash algorithms were designed by Ron Rivest. These are referred to as *MD2*, *MD4*, and *MD5*, where *MD* stands for **message digest.** The last version, MD5, is a strengthened version of MD4 that divides the message into blocks of 512 bits and creates a 128-bit digest. It turns out, however, that a message digest of size 128 bits is too small to resist attack.

In response to the insecurity of MD hash algorithms, the Secure Hash Algorithm was invented. The **Secure Hash Algorithm (SHA)** is a standard that was developed by the National Institute of Standards and Technology (NIST). SHA has gone through several versions.

16.3.2 Message authentication

A digest can be used to check the integrity of a message—that the message has not been changed. To ensure the integrity of the message and the data origin authentication—that Alice is the originator of the message, not somebody else—we need to include a secret shared by Alice and Bob (that Eve does not possess) in the process; we need to create a **message authentication code (MAC).** Figure 16.12 shows the idea.

Figure 16.12 *Message authentication code*



Alice uses a hash function to create a MAC from the concatenation of the key and the message, h (K + M). She sends the message and the MAC to Bob over the insecure channel. Bob separates the message from the MAC. He then makes a new MAC from the concatenation of the message and the secret key. Bob then compares the newly created MAC with the one received. If the two MACs match, the message is authentic and has not been modified by an adversary.

Note that there is no need to use two channels in this case. Both the message and the MAC can be sent on the same insecure channel. Eve can see the message, but she cannot forge a new message to replace it because Eve does not possess the secret key between Alice and Bob. She is unable to create the same MAC that Alice did.

A MAC provides message integrity and message authentication using a combination of a hash function and a secret key.

HMAC

The National Institute of Standards and Technology (NIST) has issued a standard for a nested MAC that is often referred to as HMAC (hashed MAC). The implementation of HMAC is much more complex than the simplified MAC and is not covered in this text.

16.3.3 Digital signature

Another way to provide message integrity and message authentication (and some more security services, as we will see shortly) is a digital signature. A MAC uses a secret key to protect the digest; a digital signature uses a pair of private–public keys.

A digital signature uses a pair of private–public keys.

We are all familiar with the concept of a signature. A person signs a document to show that it originated from her or was approved by her. The signature is proof to the recipient that the document comes from the correct entity. When a customer signs a check, the bank needs to be sure that the check is issued by that customer and nobody else. In other words, a signature on a document, when verified, is a sign of authentication—the document is authentic. Consider a painting signed by an artist. The signature on the art, if authentic, means that the painting is probably authentic.

When Alice sends a message to Bob, Bob needs to check the authenticity of the sender; he needs to be sure that the message comes from Alice and not Eve. Bob can ask Alice to sign the message electronically. In other words, an electronic signature can prove the authenticity of Alice as the sender of the message. We refer to this type of signature as a **digital signature.**

Comparison

Let us begin by looking at the differences between conventional signatures and digital signatures.

Inclusion

A conventional signature is included in the document; it is part of the document. When we write a check, the signature is on the check; it is not a separate document. But when we sign a document digitally, we send the signature as a separate document.

Verification method

The second difference between the two types of signature is the method of verifying the signature. For a conventional signature, when the recipient receives a document, they compare the signature on the document with the signature on file. If they are the same, the document is authentic. The recipient needs to have a copy of this signature on file for comparison. For a digital signature, the recipient receives the message and the signature. A copy of the signature is not stored anywhere. The recipient needs to apply a verification technique to the combination of the message and the signature to verify the authenticity.

Relationship

For a conventional signature, there is normally a one-to-many relationship between a signature and documents. A person uses the same signature to sign many documents. For a digital signature, there is a one-to-one relationship between a signature and a message. Each message has its own signature. The signature of one message cannot be used in another message. If Bob receives two messages, one after another, from Alice, he cannot use the signature of the first message to verify the second. Each message needs a new signature.

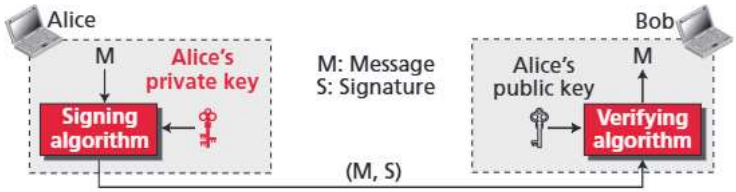
Duplicity

Another difference between the two types of signatures is a quality called *duplicity*. With a conventional signature, a copy of the signed document can be distinguished from the original one on file. With a digital signature, there is no such distinction unless there is a factor of time (such as a timestamp) on the document. For example, suppose Alice sends a document instructing Bob to pay Eve. If Eve intercepts the document and the signature, she can resend it later to get money again from Bob.

Process

Figure 16.13 shows the digital signature process. The sender uses a *signing algorithm* to sign the message. The message and the signature are sent to the receiver. The receiver receives the message and the signature and applies the **verifying algorithm** to the combination. If the result is true, the message is accepted; otherwise, it is rejected.

Figure 16.13 *Digital signature process*



A conventional signature is like a private 'key' belonging to the signer of the document. The signer uses it to sign documents; no one else has this signature. The copy of the signature on file is like a public key; anyone can use it to verify a document, to compare it to the original signature.

In a digital signature, the signer uses her private key, applied to a signing algorithm, to sign the document. The verifier, on the other hand, uses the public key of the signer, applied to the verifying algorithm, to verify the document.

Note that when a document is signed, anyone, including Bob, can verify it because everyone has access to Alice's public key. Alice must not use her public key to sign the document because then anyone could forge her signature.

Can we use a secret (symmetric) key to both sign and verify a signature? The answer is negative for several reasons. First, a secret key is known by only two entities (Alice and Bob, for example). So if Alice needs to sign another document and send it to Ted, she needs to use another secret key. Second, as we will see, creating a secret key for a session involves authentication, which uses a digital signature. We have a vicious cycle. Third, Bob could use the secret key between himself and Alice, sign a document, send it to Ted, and pretend that it came from Alice.

A digital signature needs a public-key system. The signer signs with her private key; the verifier verifies with the signer's public key.

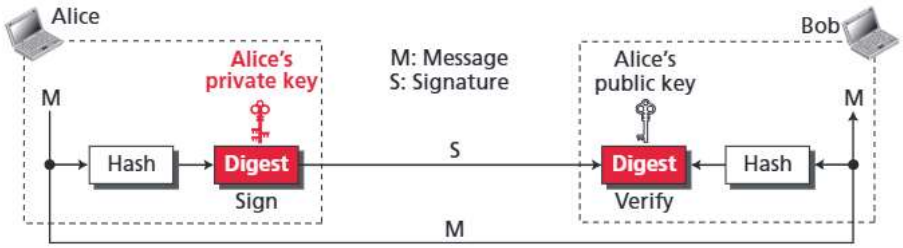
We should make a distinction between private and public keys as used in digital signatures and public and private keys as used in a cryptosystem for confidentiality. In the latter, the public and private keys of the receiver are used in the process. The sender uses the public key of the receiver to encrypt; the receiver uses his own private key to decrypt. In a digital signature, the private and public keys of the sender are used. The sender uses her private key; the receiver uses the sender's public key.

A cryptosystem uses the private and public keys of the receiver; a digital signature uses the private and public keys of the sender.

Signing the digest

We said before that the asymmetric-key cryptosystems are very inefficient when dealing with long messages. In a digital signature system, the messages are normally long, but we have to use asymmetric-key schemes. The solution is to sign a digest of the message, which is much shorter than the message. A carefully selected message digest has a one-to-one relationship with the message. The sender can sign the message digest and the receiver can verify the message digest. The effect is the same. Figure 16.14 shows signing a digest in a digital signature system.

Figure 16.14 *Signing the digest*



A digest is made out of the message at Alice's site. The digest then goes through the signing process using Alice's private key. Alice then sends the message and the signature to Bob.

At Bob's site, using the same public hash function, a digest is first created out of the received message. The verifying process is applied. If authentic, the message is accepted; otherwise, it is rejected.

Services

We discussed several security services in the beginning of the chapter including *message confidentiality*, *message authentication*, *message integrity*, and *non-repudiation*. A digital signature can directly provide the last three; for message confidentiality we still need encryption/decryption.

Message authentication

A secure digital signature scheme, like a secure conventional signature (one that cannot be easily copied) can provide message authentication (also referred to as data-origin authentication). Bob can verify that the message is sent by Alice because Alice's public key is used in verification. Alice's public key cannot verify the signature signed by Eve's private key.

Message integrity

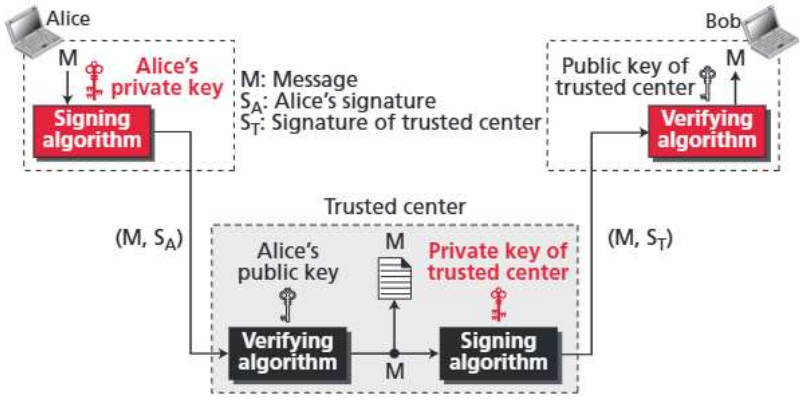
The integrity of the message is preserved if we sign the message or the digest of the message because we cannot get the same digest if any part of the message is changed. The digital signature schemes today use a hash function in the signing and verifying algorithms that preserves the integrity of the message.

Nonrepudiation

If Alice signs a message and then denies it, can Bob later prove that Alice actually signed it? For example, if Alice sends a message to a bank (Bob) and asks to transfer $10000 from her account to Ted's account, can Alice later deny that she sent this message? With the scheme we have presented so far, Bob might have a problem. Bob must keep the signature on file and later use Alice's public key to create the original message to prove the message in the file and the newly created message are the same. This is not feasible because Alice may have changed her private or public key during this time; she may also claim that the file containing the signature is not authentic.

One solution is a trusted third party. People can create an established trusted party among themselves. Later in the chapter, we will see that a trusted party can solve many other problems concerning security services and key exchange. Figure 16.15 shows how a trusted party can prevent Alice from denying that she sent the message.

Figure 16.15 *Using a trusted center for nonrepudiation*



Alice creates a signature from her message (S_A) and sends the message, her identity, Bob's identity, and the signature to the center. The center, after checking that Alice's

public key is valid, verifies through Alice's public key that the message came from Alice. The center then saves a copy of the message with the sender identity, recipient identity, and a timestamp in its archive. The center uses its private key to create another signature (S_T) from the message. The center then sends the message, the new signature, Alice's identity, and Bob's identity to Bob. Bob verifies the message using the public key of the trusted center.

If in the future Alice denies that she sent the message, the center can show a copy of the saved message. If Bob's message is a duplicate of the message saved at the center, Alice will lose the dispute. To make everything confidential, a level of encryption/decryption can be added to the scheme, as discussed in the next section.

Confidentiality

A digital signature does not provide confidential communication. If confidentiality is required, the message and the signature must be encrypted using either a symmetric-key or an asymmetric-key cipher.

Alice, the signer, first uses an agreed-upon hash function to create a digest from the message, $D = h(M)$. She then signs the digest, $S = D^d \bmod n$. The message and the signature are sent to Bob. Bob, the verifier, receives the message and the signature. He first uses Alice's public exponent to retrieve the digest, $D' = S^e \bmod n$. He then applies the hash algorithm to the message received to obtain $D = h(M)$. Bob now compares the two digests, D and D′. If they are equal (in modulo arithmetic), he accepts the message.

16.3.4 Entity authentication

Entity authentication is a technique designed to let one party verify the identity of another party. An *entity* can be a person, a process, a client, or a server. The entity whose identity needs to be proven is called the *claimant;* the party that tries to verify the identity of the claimant is called the *verifier*.

Entity versus *message authentication*

There are two differences between *entity authentication and message authentication (data-origin authentication)*.

1. Message authentication (or data-origin authentication) might not happen in real time; entity authentication does. In the former, Alice sends a message to Bob. When Bob authenticates the message, Alice may or may not be present in the communication process. On the other hand, when Alice requests entity authentication, there is no real message communication involved until Alice is authenticated by Bob. Alice needs to be online and to take part in the process. Only after she is authenticated can messages be communicated between Alice and Bob. Data-origin authentication is required when an email is sent from Alice to Bob. Entity authentication is required when Alice gets cash from an automatic teller machine.
2. Message authentication simply authenticates one message; the process needs to be repeated for each new message. Entity authentication authenticates the claimant for the entire duration of a session.

Verification categories

In entity authentication, the claimant must identify him- or herself to the verifier. This can be done with one of three kinds of witnesses: *something known*, *something possessed*, or *something inherent*.

- **Something known.** This is a secret known only by the claimant that can be checked by the verifier. Examples are a password, a PIN, a secret key, and a private key.
- **Something possessed.** This is something that can prove the claimant's identity. Examples are a passport, a driver's license, an identification card, a credit card, and a smart card.
- **Something inherent.** This is an inherent characteristic of the claimant. Examples are conventional signatures, fingerprints, voice, facial characteristics, retinal pattern, and handwriting.

In this section, we only discuss the first type of witness, *something known*, which is normally used for remote (online) entity authentication. The other two categories are normally used when the claimant is personally present.

Passwords

The simplest and oldest method of entity authentication is the use of a *password*, which is something that the claimant *knows*. A password is used when a user needs to access a system's resources (login). Each user has a user identification that is public, and a password that is private. Passwords, however, are very prone to attack. A password can be stolen, intercepted, guessed, and so on.

Challenge–response

In password authentication, the claimant proves her identity by demonstrating that she knows a secret, the password. However, because the claimant sends this secret, it is susceptible to interception by the adversary. In **challenge–response authentication,** the claimant proves that she *knows* a secret without sending it. In other words, the claimant does not send the secret to the verifier; the verifier either has it or finds it.

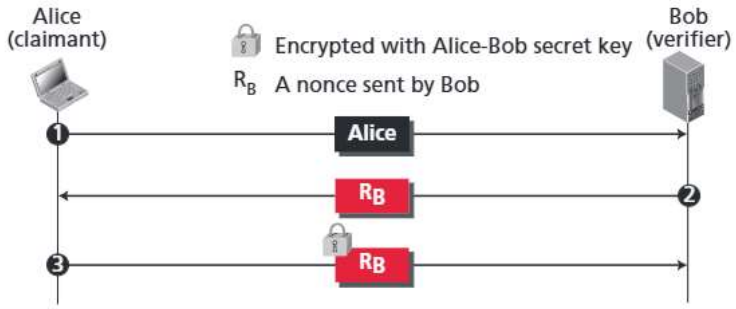
In challenge–response authentication, the claimant proves that she knows a secret without sending it to the verifier.

The *challenge* is a time-varying value such as a random number or a timestamp that is sent by the verifier. The claimant applies a function to the challenge and sends the result, called a *response,* to the verifier. The response shows that the claimant knows the secret.

Using a symmetric-key cipher

Several approaches to challenge–response authentication use **symmetric-key encryption**. The secret here is the shared secret key, known by both the claimant and the verifier. The function is the encrypting algorithm applied on the challenge. Although there are several approaches to this method, we just show the simplest one to give an idea. Figure 16.16 shows this first approach.

Figure 16.16 *Unidirectional, symmetric-key authentication*



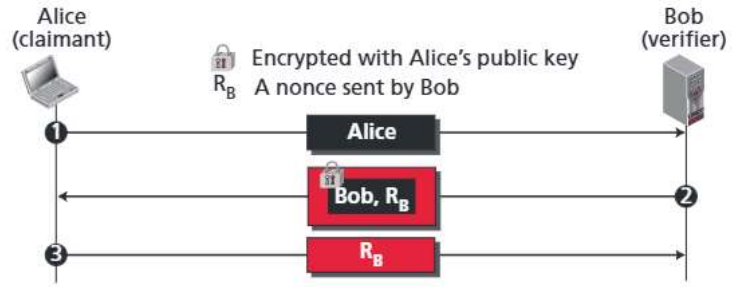
The first message is not part of challenge–response, it only informs the verifier that the claimant wants to be challenged. The second message is the challenge. R_B is the nonce (abbreviation for *number once*) randomly chosen by the verifier (Bob) to challenge the claimant. The claimant encrypts the nonce using the shared secret key known only to the claimant and the verifier and sends the result to the verifier. The verifier decrypts the message. If the nonce obtained from decryption is the same as the one sent by the verifier, Alice is granted access.

Note that in this process, the claimant and the verifier need to keep the symmetric key used in the process secret. The verifier must also keep the value of the nonce for claimant identification until the response is returned.

Using an asymmetric-key cipher

Figure 16.17 shows this approach. Instead of a symmetric-key cipher, we can use an asymmetric-key cipher for entity authentication. Here the secret must be the private key of the claimant. The claimant must show that she owns the private key related to the public key that is available to everyone. This means that the verifier must encrypt the challenge using the public key of the claimant; the claimant then decrypts the message using her private key. The response to the challenge is the decrypted message.

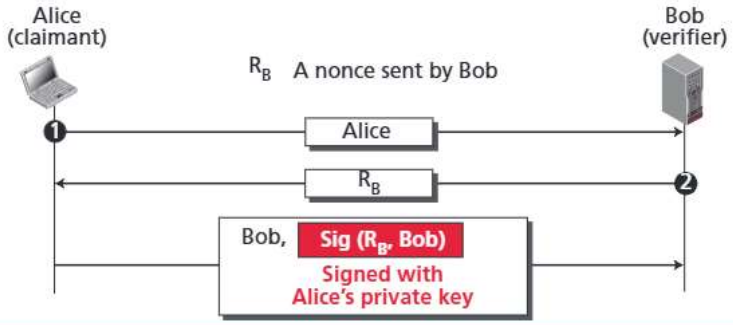
Figure 16.17 *Unidirectional, asymmetric-key authentication*



Using digital signatures

Entity authentication can also be achieved using a digital signature. When a digital signature is used for entity authentication, the claimant uses her private key for signing. In the first approach, shown in Figure 16.18, Bob uses a plaintext challenge and Alice signs the response.

Figure 16.18 *Digital signature, unidirectional authentication*



16.3.5 Key management

We discussed symmetric-key and asymmetric-key cryptography in the previous sections. However, we have not yet discussed how secret keys in symmetric-key cryptography, and public keys in asymmetric-key cryptography, are distributed and maintained. This section touches on these two issues.

Symmetric-key distribution

Symmetric-key cryptography is more efficient than asymmetric-key cryptography for enciphering large messages. Symmetric-key cryptography, however, needs a shared secret key between two parties.

If Alice needs to exchange confidential messages with N people, she needs N different keys. What if N people need to communicate with each other? A total of $N(N - 1)$ keys is needed if we require that two people use two keys for bidirectional communication; only $N(N - 1)/2$ keys are needed if we allow a key to be used for both directions. This means that if one million people need to communicate with each other, each person has almost one million different keys; in total, half a trillion keys are needed. This is normally referred to as the N^2 problem because the number of required keys for N entities is close to N^2.

The number of keys is not the only problem; the distribution of keys is another. If Alice and Bob want to communicate, they need a way to exchange a secret key; if Alice wants to communicate with one million people, how can she exchange one million keys

with one million people? Using the Internet is definitely not a secure method. It is obvious that we need an efficient way to maintain and distribute secret keys.

Key distribution center: KDC

A practical solution is the use of a trusted third party, referred to as a **key-distribution center (KDC)** . To reduce the number of keys, each person establishes a shared secret key with the KDC. A secret key is established between the KDC and each member. Now the question is how Alice can send a confidential message to Bob. The process is as follows:

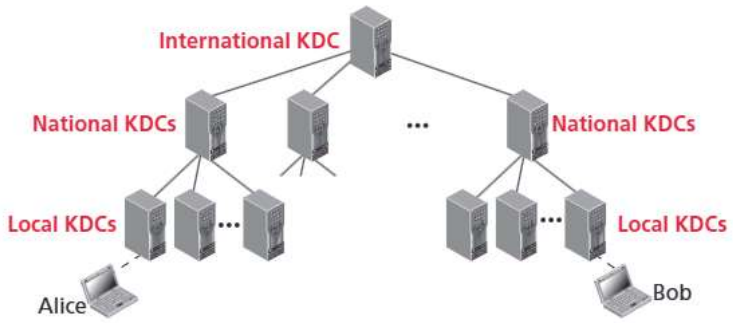
1. Alice sends a request to the KDC stating that she needs a session (temporary) secret key between herself and Bob.
2. The KDC informs Bob about Alice's request.
3. If Bob agrees, a session key is created between the two.

The secret key between Alice and Bob that is established with the KDC is used to authenticate Alice and Bob to the KDC and to prevent Eve from impersonating either of them.

Multiple KDCs

When the number of people using a KDC increases, the system becomes unmanageable and a bottleneck can result. To solve the problem, we need to have multiple KDCs. We can divide the world into domains. Each domain can have one or more KDCs (for redundancy in case of failure). Now if Alice wants to send a confidential message to Bob, who belongs to another domain, Alice contacts her KDC, which in turn contacts the KDC in Bob's domain. The two KDCs can create a secret key between Alice and Bob. There can be local KDCs, national KDCs, and international KDCs. When Alice needs to communicate with Bob, who lives in another country, she sends her request to a local KDC; the local KDC relays the request to the national KDC; the national KDC relays the request to an international KDC. The request is then relayed all the way down to the local KDC where Bob lives. Figure 16.19 shows a configuration of hierarchical multiple KDCs.

Figure 16.19 *Multiple KDCs*



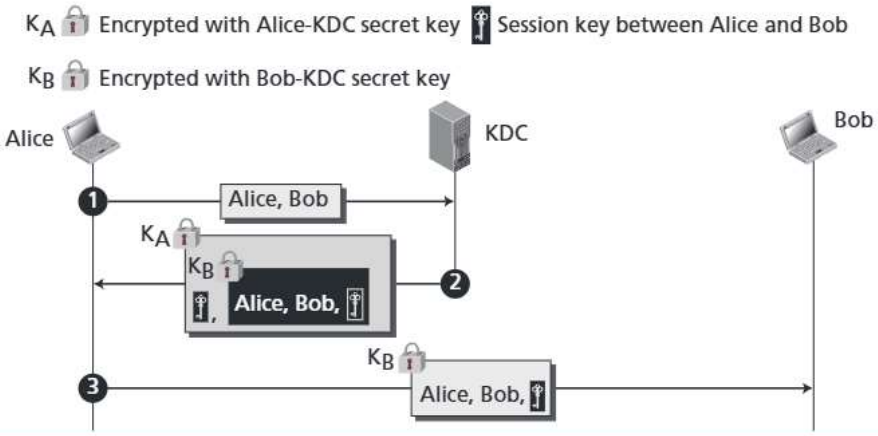
Session keys

A KDC creates a secret key for each member. This secret key can be used only between the member and the KDC, not between two members. If Alice needs to communicate secretly with Bob, she needs a secret key between herself and Bob. A KDC can create a *session key* between Alice and Bob, using their keys with the center. The keys of Alice and Bob are used to authenticate Alice and Bob to the center and to each other before the session key is established. After communication is terminated, the session key is no longer useful.

A session symmetric key between two parties is used only once.

Several different approaches have been proposed to create the session key using ideas discussed in the previous sections. We show the simplest approach in Figure 16.20. Although this approach is very rudimentary, it helps to understand more sophisticated approaches in the literature.

Figure 16.20 *Creating a session key using KDC*



1. Alice sends a plaintext message to the KDC to obtain a symmetric session key between Bob and herself. The message contains her registered identity (the word *Alice* in the figure) and the identity of Bob (the word *Bob* in the figure). This message is not encrypted, it is public. The KDC does not care.
2. The KDC receives the message and creates what is called a **ticket.** The ticket is encrypted using Bob's key (K_B). The ticket contains the identities of Alice and Bob and the session key. The ticket with a copy of the session key is sent to Alice. Alice receives the message, decrypts it, and extracts the session key. She cannot decrypt

Bob's ticket; the ticket is for Bob, not for Alice. Note that this message contains a double encryption—the ticket is encrypted, and the entire message is also encrypted. In the second message, Alice is actually authenticated to the KDC, because only Alice can open the whole message using her secret key with KDC.

3. Alice sends the ticket to Bob. Bob opens the ticket and knows that Alice needs to send messages to him using the session key. Note that in this message, Bob is authenticated to the KDC because only Bob can open the ticket. Because Bob is authenticated to the KDC, he is also authenticated to Alice, who trusts the KDC. In the same way, Alice is also authenticated to Bob, because Bob trusts the KDC and the KDC has sent Bob the ticket that includes the identity of Alice.

Public-key distribution

In asymmetric-key cryptography, people do not need to know a symmetric shared key. If Alice wants to send a message to Bob, she only needs to know Bob's public key, which is open to the public and available to everyone. If Bob needs to send a message to Alice, he only needs to know Alice's public key, which is also known to everyone. In public-key cryptography, everyone shields a private key and advertises a public key.

In public-key cryptography, everyone has access to everyone's public key; public keys are available to the public.

Public keys, like secret keys, need to be distributed to be useful. Let us briefly discuss the way public keys can be distributed.

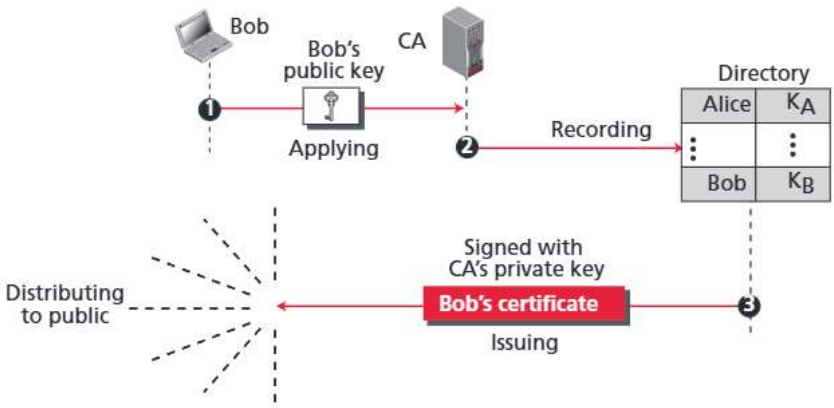
Public announcement

The naive approach is to announce public keys publicly. Bob can put his public key on his website or announce it in a local or national newspaper. When Alice needs to send a confidential message to Bob, she can obtain Bob's public key from his site or from the newspaper, or even send a message to ask for it. This approach, however, is not secure; it is subject to forgery. For example, Eve could make such a public announcement. Before Bob can react, damage could be done. Eve can fool Alice into sending her a message that is intended for Bob. Eve could also sign a document with a corresponding forged private key and make everyone believe it was signed by Bob. The approach is also vulnerable if Alice directly requests Bob's public key. Eve can intercept Bob's response and substitute her own forged public key for Bob's public key.

Certification authority

The common approach to distributing public keys is to create **public-key certificates.** Bob wants two things; he wants people to know his public key, and he wants no one to accept a forged public key as his. Bob can go to a **certification authority (CA)**, a federal or state organization that binds a public key to an entity and issues a certificate. Figure 16.21 shows the concept.

Figure 16.21 *Certification authority*



The CA itself has a well-known public key that cannot be forged. The CA checks Bob's identification (using a photo ID along with other proof). It then asks for Bob's public key and writes it on the certificate. To prevent the certificate itself from being forged, the CA signs the certificate with its private key. Now Bob can upload the signed certificate. Anyone who wants Bob's public key downloads the signed certificate and uses the authority's public key to extract Bob's public key.

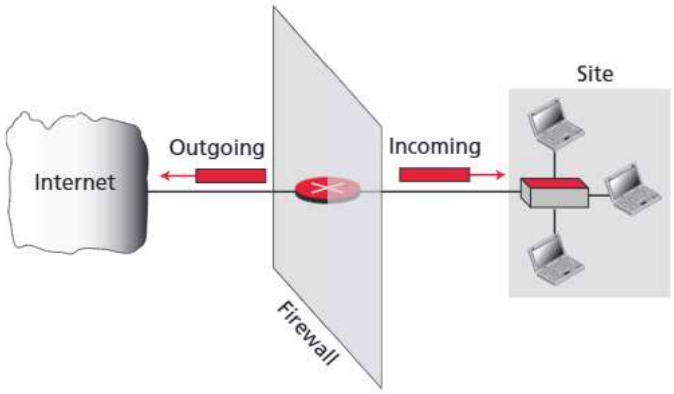
X.509

Although the use of a CA has solved the problem of public-key fraud, it has created a **side effect**. Each certificate may have a different format. If Alice wants to use a program to automatically download different certificates and digests belonging to different people, the program may not be able to do this. One certificate may have the public key in one format and another certificate may have it in a different format. The public key may be on the first line in one certificate and on the third line in another. Anything that needs to be used universally must have a universal format. To remove this side effect, the ITU has designed X.509, a recommendation that has been accepted by the Internet with some changes. X.509 is a way to describe the certificate in a structured way. It uses a well-known protocol called ASN.1 that defines fields familiar to computer programmers.

16.4 FIREWALLS

All previous security measures cannot prevent Eve from sending a harmful message to a system. To control access to a system we need firewalls. A **firewall** is a device (usually a router or a computer) installed between the internal network of an organization and the rest of the Internet. It is designed to forward some packets and filter (not forward) others. Figure 16.22 shows a firewall.

Figure 16.22 *Firewall*

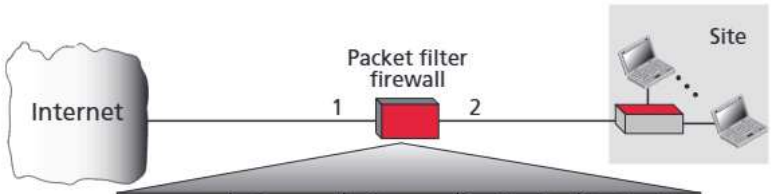


For example, a firewall may filter all incoming packets destined for a specific host or a specific server such as HTTP. A firewall can be used to deny access to a specific host or a specific service in the organization. A firewall is usually classified as a *packet-filter firewall* or a *proxy-based firewall*.

16.4.1 Packet-filter firewall

A firewall can be used as a packet filter. It can forward or block packets based on the information in the network-layer and transport-layer headers: source and destination IP addresses, source and destination **port addresses**, and type of protocol (TCP or UDP). A **packet-filter firewall** is a router that uses a filtering table to decide which packets must be discarded (not forwarded). Figure 16.23 shows an example of a filtering table for this kind of a firewall.

Figure 16.23 *Packet-filter firewall*



Interface	Source IP	Source port	Destination IP	Destination port
1	131.34.0.0	*	*	*
1	*	*	*	23
1	*	*	194.78.20.8	*
2	*	*	*	80

According to the figure, the following packets are filtered:

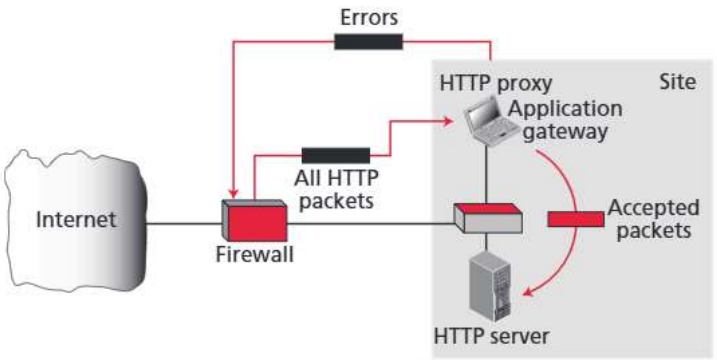
1. Incoming packets from network 131.34.0.0 are blocked (security precaution). Note that the * (asterisk) means 'any'.
2. Incoming packets destined for any internal TELNET server (port 23) are blocked.
3. Incoming packets destined for internal host 194.78.20.8. are blocked. The organization wants this host for internal use only.
4. Outgoing packets destined for an HTTP server (port 80) are blocked. The organization does not want employees to browse the Internet.

16.4.2 Proxy firewall

The packet-filter firewall is based on the information available in the network-layer and transport-layer headers (IP and TCP/UDP). However, sometimes we need to filter a message based on the information available in the message itself (at the application layer). As an example, assume that an organization wants to implement the following policies regarding its web pages: only those Internet users who have previously established business relations with the company can have access; access to other users must be blocked. In this case, a packet-filter firewall is not feasible because it cannot distinguish between different packets arriving at TCP port 80 (HTTP). Testing must be done at the application level (using URLs).

One solution is to install a proxy computer (sometimes called an **application gateway**), which stands between the customer computer and the corporation computer. When the user client process sends a message, the application gateway runs a server process to receive the request. The server opens the packet at the application level and finds out if the request is legitimate. If it is, the server acts as a client process and sends the message to the real server in the corporation. If it is not, the message is dropped and an error message is sent to the external user. In this way, the requests of the external users are filtered based on the contents at the application layer. Figure 16.24 shows an application gateway implementation for HTTP.

Figure 16.24 *Proxy firewall*



16.5 END-CHAPTER MATERIALS

16.5.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Bishop, M. *Computer Security*, Reading, MA: Addison-Wesley, 2002
- Forouzan, B. *Cryptography and Network Security*, New York: McGraw-Hill, 2007
- Kaufman, C., Perlman, R. and Speciner, M. *Network Security*, Upper Saddle River, NJ: Prentice-Hall, 2002
- Stallings, W. *Cryptography and Network Security*, Upper Saddle River, NJ: Prentice- Hall, 2006

16.5.2 Key terms

additive cipher 417	one-time pad 422
application gateway 444	packet-filter firewall 443
asymmetric-key cipher 423	plaintext 416
autokey cipher 419	polyalphabetic cipher 418
availability 412	port address 443
block cipher 421	private key 424
caesar cipher 418	proxy firewall 444
certification authority (CA) 441	public key 424
challenge–response authentication 436	public-key certificate 441
cipher 415	replaying 414
ciphertext 416	RSA cryptosystem 426
confidentiality 412	secret key 415
cryptographic hash function 429	Secure Hash Algorithm (SHA) 429
cryptography 415	security attack 413
decryption 415	security goal 412
decryption algorithm 416	shift cipher 417
denial of service (DoS) 414	side effect 442
digest 429	snooping 413
digital signature 431	spoofing 414
encryption 415	steganography 415
encryption algorithm 416	stream cipher 420
firewall 442	substitution cipher 417
hashed MAC (HMAC) 430	symmetric-key cipher 415

(Continued)

integrity 412
key 416
key-distribution center (KDC) 439
masquerading 414
message authentication code (MAC) 430
message digest (MD) 429
monoalphabetic cipher 417
symmetric-key encryption 436
ticket 440
traffic analysis 413
transposition cipher 419
verifying algorithm 432
X.509 442

16.5.3 Summary

- We mentioned three goals of security: *confidentiality*, *integrity*, and *availability*. We have divided attacks on security into three categories: attacks threatening confidentiality, attacks threatening integrity, and attacks threatening availability. To achieve security goals and prevent the corresponding attacks, the ITU (International Telecommunication Union) has defined several services: **data confidentiality**, **data integrity**, *authentication*, **nonrepudiation**, and *access control*. Two techniques are used to provide these services: *cryptography* and *steganography*.
- *Symmetric-key cryptography* uses a single key for encryption and decryption. Alice and Bob first agree upon a shared secret, which forms their secret key. To send a message to Bob, Alice encrypts her message using the secret key: to send a message to Alice, Bob encrypts his message using the same secret key. Traditional symmetric-key ciphers were character-oriented and used two techniques for hiding information from an intruder: *substitution* and *transposition*.
- Modern symmetric-key ciphers are bit-oriented and use very complex algorithms to encrypt and decrypt blocks of bits. *Asymmetric-key cryptography* uses two distinctive keys: a private key and a public key. Bob first creates a pair of keys. He keeps the private key and announces the public key. If anyone needs to send a message to Bob, they encrypt the message with Bob's public key. To read the message, Bob decrypts the message with his private key.
- *Integrity* means protecting a message from being modified. To preserve the integrity of a message, the message is passed through an algorithm called a *cryptographic hash function*. The function creates a compressed image of the message called a *message digest*. To provide message authentication, a message authentication code (MAC) is needed. A MAC includes a secret shared by the sender and the recipient. A digital signature is the process of signing a document electronically. It provides *message integrity*, *message authentication*, and *nonrepudiation*. Entity authentication is a technique designed to let one party prove the identity of another party. Entity authentication uses three verification categories: *something known*, *something possessed* and *something inherent*. We mentioned four authentication techniques: *password-based*, *challenge-response*, **zero-knowledge**, and **biometrics**.

5/1/25, 9:57 PM study viewer

- For symmetric-key or asymmetric-key cryptography, the two parties need to exchange keys. Key management methods allow us to do this without the need for face-to-face key exchange. In symmetric-key cryptography, a practical solution is the use of a key-distribution center (KDC). In asymmetric-key cryptography, a practical solution is the use of certificates issued by a certification authority (CA).

16.6 PRACTICE SET

16.6.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

16.6.2 Questions

Q16-1. Which of the following attacks is a threat to confidentiality?

a. snooping
b. masquerading
c. repudiation

Q16-2. Which of the following attacks is a threat to integrity?

a. modification
b. replaying
c. denial of service

Q16-3. Which of the following attacks is a threat to availability?

a. repudiation
b. denial of service
c. modification

Q16-4. Which of the following words mean 'secret writing'? Which one means 'covered writing'?

a. cryptography
b. steganography

Q16-5. When a sealed letter is sent from Alice to Bob, is this an example of using cryptography or steganography for confidentiality?

Q16-6. When a letter is sent from Bob to Alice in a language that only the two can understand, is this an example of cryptography or steganography?

Q16-7. Alice has found a way to write secretly to Bob. Each time, she takes a new text, such as an article from the newspaper, but inserts one or two spaces between the words. A single space means a binary digit 0; a double space means a binary

https://studylib.net/doc/26059254/foundations-computer-science-4th

digit 1. Bob extracts the binary digits and interprets them using ASCII code. Is this an example of cryptography or steganography? Explain.

Q16-8. Alice and Bob exchange confidential messages. They share a very large number as the encryption and decryption key in both directions. Is this an example of symmetric-key or asymmetric-key cryptography? Explain.

Q16-9. Alice uses the same key when she encrypts a message to be sent to Bob and when she decrypts a message received from Bob. Is this an example of symmetric-key or asymmetric-key cryptography? Explain.

Q16-10. Distinguish between a substitution cipher and a transposition cipher.

Q16-11. In a cipher, all the As in the plaintext have been changed to Ds in the ciphertext and all the Ds in the plaintext have been changed to Hs in the ciphertext. Is this a monoalphabetic or polyalphabetic substitution cipher? Explain.

Q16-12. Which cipher can be broken more easily, monoalphabetic or polyalphabetic?

Q16-13. Assume Alice and Bob use an additive cipher in modulo 26 arithmetic. If Eve, the intruder, wants to break the code by trying all possible keys (brute-force attack), how many keys should she try on average?

Q16-14. Assume we have a plaintext of 1000 characters. How many keys do we need to encrypt or decrypt the message in each of the following ciphers?

- **a.** additive
- **b.** monoalphabetic
- **c.** autokey

Q16-15. According to the definitions of stream and block ciphers, find which of the following ciphers is a stream cipher.

- **a.** additive
- **b.** monoalphabetic
- **c.** autokey

Q16-16. If the one-time pad cipher (Figure 16.7 in the text) is the simplest and most secure cipher, why is it not used all of the time?

Q16-17. Why do you think asymmetric-key cryptography is used only with small messages?

Q16-18. In an asymmetric public key cipher, which key is used for encryption? Which key is used for decryption?

- **a.** public key
- **b.** private key

Q16-19. In RSA, why can't Bob choose 1 as the public key e?

Q16-20. What is the role of the secret key added to the hash function in Figure 16.12 in the text (MAC)? Explain.

Q16-21. Distinguish between message authentication and entity authentication.

Q16-22. Alice signs the message she sends to Bob to prove that she is the sender of the message. Which of the following keys does Alice need to use?

- **a.** Alice's public key
- **b.** Alice's private key

Q16-23. Alice needs to send a message to a group of 50 people. If Alice needs to use message authentication, which of the following schemes do you recommend?

a. MAC

b. digital signature

Q16-24. Which of the following services are not provided by digital signature?

a. message authentication

b. confidentiality

c. nonrepudiation

16.6.3 Problems

P16-1. Define the type of attack in each of the following cases:

a. A student breaks into a professor's office to obtain a copy of the next test.

b. A student gives a check for $10 to buy a used book. Later the student finds out that the check was cashed for $100.

c. A student sends hundreds of emails per day to the school using a phony return email address.

P16-2. Use the additive cipher with k = 10 to encrypt the plaintext 'book'. Then decrypt the message to get the original plaintext.

P16-3. Encrypt the message 'this is an exercise' using additive cipher with key = 20. Ignore the space between words. Decrypt the message to get the original plaintext.

P16-4. Atbash was a popular cipher among Biblical writers. In Atbash, 'A' is encrypted as 'Z', 'B' is encrypted as 'Y', and so on. Similarly, 'Z' is encrypted as 'A', 'Y' is encrypted as 'B', and so on. Suppose that the alphabet is divided into halves and the letters in the first half are encrypted as the letters in the second and *vice versa*. Find the type of cipher and key. Encipher the plaintext 'an exercise' using the Atbash cipher.

P16-5. A substitution cipher does not have to be a character-to-character transformation. In a Polybius cipher, each letter in the plaintext is encrypted as two integers. The key is a 5 × 5 matrix of characters. The plaintext is the character in the matrix, the ciphertext is the two integers (each between 1 and 5) representing row and column numbers. Encipher the message 'An exercise' using the Polybius cipher with the following key:

	1	2	3	4	5
1	z	q	p	f	e
2	y	r	o	g	d
3	x	s	n	h	c
4	w	t	m	i / j	b
5	v	u	l	k	a

P16-6. Alice can use only the additive cipher on her computer to send a message to a friend. She thinks that the message is more secure if she encrypts the message two times, each time with a different key. Is she right? Defend your answer.

P16-7. One of the attacks an intruder can apply to a simple cipher like an additive cipher is called the *ciphertext* attack. In this type of attack, the intruder intercepts the cipher and tries to find the key and eventually the plaintext. One of the methods used in a ciphertext attack is called the *brute-force* approach, in which the intruder tries several keys and decrypts the message until the message makes sense. Assume the intruder has intercepted the ciphertext 'UVACLYZLJBYL'. Try to decrypt the message by using keys from 1 until a plaintext appears that makes sense.

P16-8. In a transposition cipher the encryption and decryption keys are often represented as two one-dimension tables (arrays) and the cipher is represented as a piece of software (a program).

a. Show the array for the encryption key in Figure 16.5 in the text. Hint: the value of each element can show the input-column number; the index can show the output-column number.

b. Show the array for the decryption key in Figure 16.5 in the text.

c. Explain, given the encryption key, how we can find the decryption key.

P16-9. Assume Bob, using the RSA cryptosystem, selects $p = 11$, $q = 13$, and $d = 7$. Which of the following can be the value of public key e?

a. 11

b. 103

c. 19

P16-10. In RSA, given $p = 107$, $q = 113$, $e = 13$, and $d = 3653$, encrypt the message 'THIS IS TOUGH' using 00 to 26 (A: 00 and space: 26) as the encoding scheme. Decrypt the ciphertext to find the original message.

P16-11. Explain why private–public keys cannot be used in creating a MAC.

CHAPTER 17

Theory of Computation

In Chapters 1 through 16, we considered a computer as a problem-solving machine. In this chapter, we answer some questions such as: which problems can be solved by a computer? Is one language superior to another? Before running a program, can it be determined whether the program will halt (terminate) or run forever? How long does it take to solve a problem using a particular language? To answer these questions, we turn to a discipline called the *theory of computation*.

Objectives

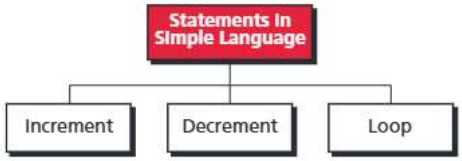
After studying this chapter, the student should be able to:

- Describe a programming language we call Simple Language and define its basic statements.
- Write macros in Simple Language using the combination of simple statements.
- Describe the components of a Turing machine as a computation model.
- Show how simple statements in Simple Language can be simulated using a Turing machine.
- Understand the Church–Turing thesis and its implication.
- Define the Gödel number and its application.
- Understand the concept of the halting problem and how it can be proved that this problem is unsolvable.
- Distinguish between solvable and unsolvable problems.
- Distinguish between polynomial and nonpolynomial solvable problems.

17.1 SIMPLE LANGUAGE

We can define a computer language with only three statements: the **increment statement**, the **decrement statement**, and the **loop statement** (Figure 17.1). In this language, the only data type we use is nonnegative integers. There is no need for any other data type, because the goal of the chapter is merely to demonstrate some ideas in computation theory. The language uses only a few symbols such as '{' and '}'.

Figure 17.1 *Statements in Simple Language*



17.1.1 Increment statement

The increment statement adds 1 to a variable. The format is shown in Algorithm 17.1.

Algorithm 17.1 The increment statement

```
incr (X)
```

17.1.2 Decrement statement

The decrement statement subtracts 1 from a variable. The format is shown in Algorithm 17.2.

Algorithm 17.2 The decrement statement

```
decr (X)
```

17.1.3 Loop statement

The loop statement repeats an action (or a series of actions) while the value of the variable is not 0. The format is shown in Algorithm 17.3.

Algorithm 17.3 The loop statement

```
while (X)
{
  decr (X)
  Body of the loop
}
```

17.1.4 The power of the simple language

It can be shown that this simple programming language with only three statements is as powerful—although not necessarily as efficient—as any sophisticated language in use today, such as C. To do so, we show how we can simulate several statements found in some popular languages.

Macros in Simple Language

We call each simulation a **macro** and use it in other simulations without the need to repeat code. A *macro* (short for *macroinstruction*) is an instruction in a high-level language that is equivalent to a specific set of one or more ordinary instructions in the same language.

First macro: X ← 0

Algorithm 17.4 shows how to use the statements in Simple Language to assign 0 to a variable X. It is sometimes called *clearing* a variable.

Algorithm 17.4 Macro X ← 0

```
while (X)
{
   decr (X)
}
```

Second macro: X ← n

Algorithm 17.5 shows how to use the statements in Simple Language to assign a positive integer *n* to a variable X. First clear the variable X, then increment X *n* times.

Algorithm 17.5 Macro X ← n

```
X ← 0
incr (X)
incr (X)
...
incr (X)
// The statement incr (X) is repeated n times.
```

Third macro: Y ← X

Algorithm 17.6 simulates the macro Y ← X in Simple Language. Note that we can use an extra line of code to restore the value of X.

Algorithm 17.6 Macro Y ← X

```
Y ← 0
while (X)
{
   decr (X)
   incr (Y)
}
```

Fourth macro: Y ← Y + X

Algorithm 17.7 simulates the macro Y ← Y + X in Simple Language. Again, we can use more code lines to restore the value of X to its original value.

Algorithm 17.7 Macro Y ← Y + X

```
while (X)
{
   decr (X)
   incr (Y)
}
```

Fifth macro: Y ← Y × X

Algorithm 17.8 simulates the macro Y ← Y × X in Simple Language. We can use the addition macro because integer multiplication can be simulated by repeated addition. Note that we need to preserve the value of X in a temporary variable, because in each addition we need the original value of X to be added to Y.

Algorithm 17.8 Macro Y ← Y × X

```
TEMP ← Y
Y ← 0
while (X)
{
      decr (X)
      Y ← Y + TEMP
}
```

Sixth macro: Y ← Y^X

Algorithm 17.9 simulates the macro Y ← Y^X in Simple Language. We do this using the multiplication macro because integer exponentiation can be simulated by repeated multiplication.

Algorithm 17.9 Macro Y ← Y^X

```
TEMP ← Y
Y ← 1
while (X)
{
    decr (X)
    Y ← Y × TEMP
}
```

Seventh macro: if X then A

Algorithm 17.10 simulates the seventh macro in Simple Language. This macro simulates the decision-making (*if*) statement of modern languages. In this macro, the variable X has only one of the two values 0 or 1. If the value of X is not 0, A (an action or a series of actions) is executed in the loop. However, the loop is executed only once because, after the first iteration, the value of X becomes 0 and we come out of the loop. If the value of X is originally 0, the loop is skipped.

Algorithm 17.10 Macro if X then A

```
while (X)
{
    decr (X)
    A
}
```

Other macros

It is obvious that we need more macros to make Simple Language compatible with contemporary languages. Creating other macros is possible, although not trivial.

Input and output

In this simple language the statement *read* X can be simulated using (X ← *n*). We also simulate the output by assuming that the last variable used in a program holds what should be printed. Remember that this is not a practical language, it is merely designed to prove some theorems in computer science.

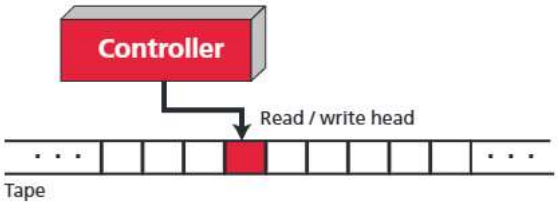
17.2 THE TURING MACHINE

The **Turing machine** was introduced in 1936 by Alan M. Turing to solve computable problems, and is the foundation of modern computers. In this section we introduce a very simplified version of the machine to show how it works.

17.2.1 Turing machine components

A Turing machine is made of three components: a tape, a controller, and a read/write head (Figure 17.2).

Figure 17.2 *The Turing machine*



Tape

Although modern computers use a random-access storage device with finite capacity, we assume that the Turing machine's memory is infinite. The tape, at any one time, holds a sequence of characters from the set of characters accepted by the machine. For our purpose, we assume that the machine can accept only two symbols: a blank (b) and digit 1. Figure 17.3 shows an example of data on a tape in this machine.

Figure 17.3 *The tape in the Turing machine*

The left-hand blank defines the beginning of the nonnegative integers stored on the tape. The integer is represented by a string of 1s, and the right-hand blank defines the end of the integer. The rest of the tape contains blank characters. If more than one integer are stored on the tape, they are separated by at least one blank character.

We also assume that the tape processes only positive integer data represented in unary arithmetic. In this arithmetic, a positive integer is made up only of 1s. For example, the

5/1/25, 9:57 PM study viewer

integer 4 is represented as 1111 (four 1s) and the integer 7 is represented as 1111111 (seven 1s). The absence of 1s represents 0.

Read/write head

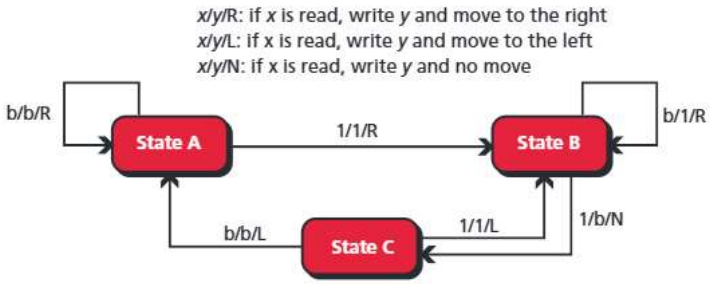
The **read/write head** at any moment points to one symbol on the tape. We call this symbol the *current* symbol. The read/write head reads and writes one symbol at a time from the tape. After reading and writing, it moves to the left or to the right. Reading, writing, and moving are all done under instructions from the controller.

Controller

The **controller** is the theoretical counterpart of the central processing unit (CPU) in modern computers. It is a **finite state automaton**, a machine that has a predetermined finite number of states and moves from one state to another based on the input. At any moment, it can be in one of these states.

Figure 17.4 shows the transition state diagram for a simple controller as a finite state automaton. In this figure, the automaton has only three states (A, B, and C), although a controller normally has many states. The diagram shows the change of state as a function of the character read. The expression on each line, *x*/*y*/L, *x*/*y*/R, and *x*/*y*/*N*, shows that if the controller has read the symbol *x*, it writes the symbol *y* (overwrites *x*), and the read/write head moves to the left (L), right (R), or does not move (N). Note that since the symbols on the tape can be only a blank or the digit 1, there should be two paths out of each state: one if the blank symbol is read and one if the digit 1 is read. The beginning of the line (called the *transition* line) shows the current state and the end of the line (arrow head) shows the next state.

Figure 17.4 *Transition state diagram for the Turing machine*



We can create a transition table (Table 17.1) in which each row relates to one state. The table will have five columns: current state, the symbol that is read, the symbol to write, the direction of movement of the head, and the next symbol. Since the machine can only go through a finite number of states, we can create an instruction set like the one we create for the Simple Computer in Chapter 5.

https://sanet.st/blogs/polatebooks/

https://studylib.net/doc/26059254/foundations-computer-science-4th 477/706

Table 17.1 Transition table

Current State	*Read*	*Write*	*Move*	*New State*
A	b	b	R	A
A	1	1	R	B
B	b	1	R	B
B	1	b	N	C
C	b	b	L	A
C	1	1	L	B

The instructions put together the value of five columns in each row. For this elementary machine, we have only six instructions:

1. (A, b, b, R, A)	3. (B, b, 1, R, B)	5. (C, b, b, L, A)
2. (A, 1, 1, R, B)	4. (B, 1, b, N, C)	6. (C, 1, 1, L, B)

For example, the first instruction says that if the machine is in state A and has read the symbol b, it overwrites the symbol with a new b, moves to the next symbol to the right, and the machine transitions to state A—that is, remains in the same state.

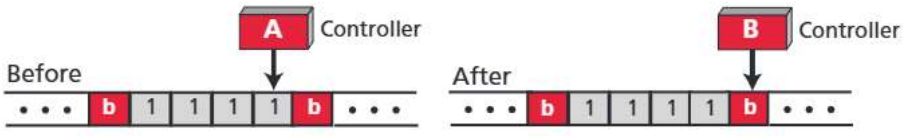
Example 17.1

A Turing machine has only two states and the following four instructions:

1. (A, b, b, L, A)	2. (A, 1, 1, R, B)	3. (B, b, b, L, A)	4. (B, 1, b, R, A)

If the machine starts with the configuration shown in Figure 17.5, what is the configuration of the machine after executing one of the above instructions? Note that the machine can only execute one of the instructions, the one that matches the current state and the current symbol.

Figure 17.5 *Example 17.1*



Solution

The machine is in state A and the current symbol is 1, which means that only the second instruction, (A, 1, 1, R, B) can be executed. The new configuration is also shown in Figure 17.5. Note that the state of the controller has been changed to B and the read/write head has moved one symbol to the right.

17.2.2 Simulating Simple Language

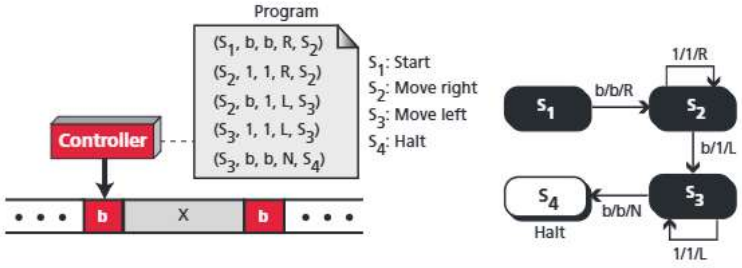
We can now write programs that implement the statements of Simple Language. Note that these statements can be written in many different ways: we have chosen the simplest or most convenient for our educational purpose, but they are not necessarily the best ones.

Increment statement

Figure 17.6 shows the Turing machine for the **incr** (X) statement. The controller has four states, S_1 through S_4. State S_1 is the starting state, state S_2 is the moving-right state, state S_3 is the moving-left state, and state S_4 is the halting state. If the machine reaches the halting state, it stops: there is no instruction that starts with this state.

Figure 17.6 also shows the program for the **incr** (X) statement. It has only five instructions. The process starts from the blank symbol at the left of X (data to be incremented), moves right over all 1s until it reaches the blank symbol at the right of X. It changes this blank to 1. It then moves left over all 1s until it reaches the blank at the left again. At this point it halts. Note that we have also written the program to move the read/write head back to the blank symbol to the left of X, which is necessary if more operations are to be performed on X.

Figure 17.6 *The Turing machine for the incr (X) statement*



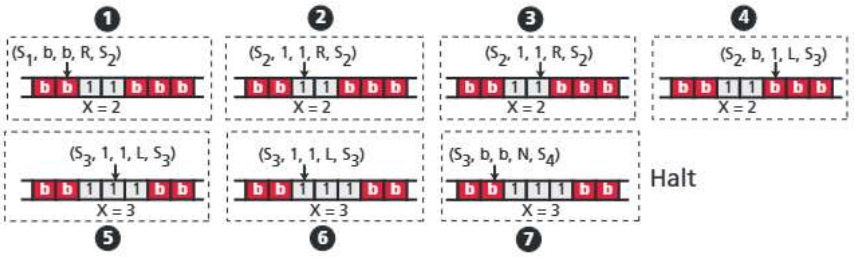
Example 17.2

Show how the Turing machine can increment X when X = 2.

Solution

Figure 17.7 shows the solution. The value of X (11 in the unary system) is stored between the two blank symbols. It takes seven steps for the machine to increment X and return the read/write head to its original position. Steps 1 to 4 move the read/write head to the end of X. Steps 5 to 7 change the blank at the end and move the read/write head back to where it was before.

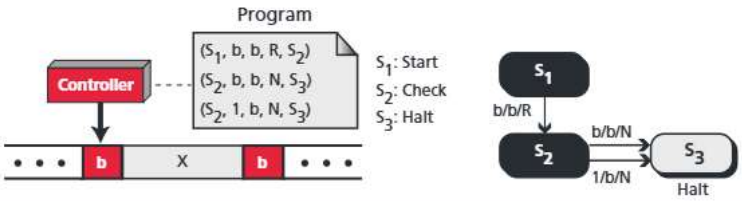
Figure 17.7 *Example 17.2*



Decrement statement

We implement the **decr** (X) statement using the minimum number of instructions. The reason is that we need to use this statement in the next statement, the *while* loop, which will also be used to implement all macros. Figure 17.8 shows the Turing machine for this statement. The controller has three states, S_1, S_2, and S_3. Statement S_1 is the starting state. State S_2 is the checking statement, which checks to see if the current symbol is 1 or b. If it is b, the statement goes to the halting state: if the next symbol is 1, the second statement changes it to b and goes to the halting state. Figure 17.8 also shows the program for this statement.

Figure 17.8 *The Turing machine for the decr (X) statement*



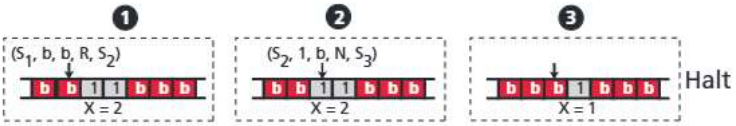
Example 17.3

Show how the Turing machine can decrement X when X = 2.

Solution

Figure 17.9 shows the situation. The machine starts at the blank to the left of the data and changes the next symbol to blank if it is 1. The read/write head stops over the blank character to the left of the resulting data. This is the same arrangement as with the increment statement. Note that we could have moved the read/write head to the end of the data and deleted the last 1 instead of the first one, but that program would be much longer than our version. Since we need this statement in every loop statement, we have used the shorter version to save the number of instructions. We use the short version of this statement in the *while* loop statement that we develop next.

Figure 17.9 *Example 17.3*



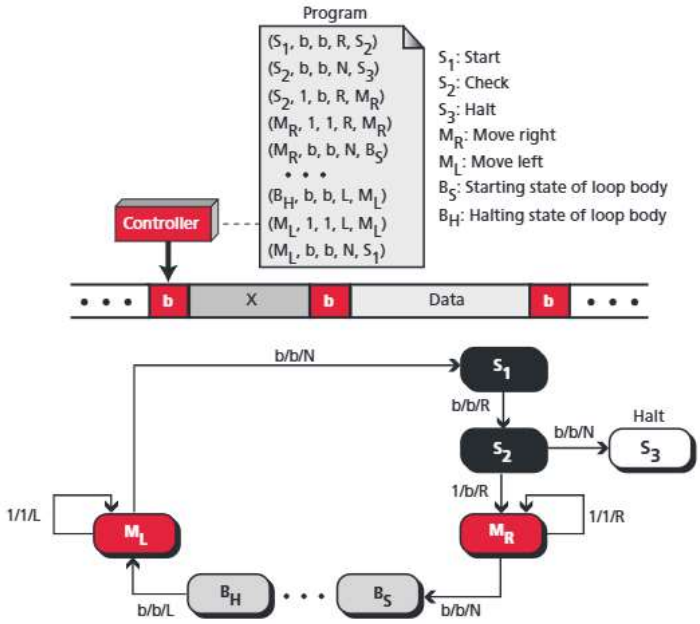
Loop statement

To simulate the loop, we assume that X and the data to be processed by the body of the loop are stored on the tape separated by a single blank symbol. Figure 17.10 shows the table, the program, and the state transition diagram for a general loop statement.

The three states S_1, S_2, and S_3 control the loops by determining X and exiting the loop if X = 0. Compare these three statements to the three statements used in the decrement statement in Figure 17.8. The state M_R moves the read/write head over the blank symbol that defines the start of the data at the beginning of processing data in each iteration, the state M_L moves the read/write head over the blank symbol defining the start of the X at the end of processing in each iteration. The state B_S (*body start*) defines the beginning state of the body of the loop, while the state B_H (*body halt*) defines the halting state for the body of the loop. The body of the loop may have several states between these two states.

Figure 17.10 also shows the repetitive nature of the statement. The state diagram itself is a loop that is repeated as long as the value of X is not zero. When the value of X becomes 0, the loop stops and state S_3, the halting state, is reached.

Figure 17.10 *The Turing machine for the while loop statement*



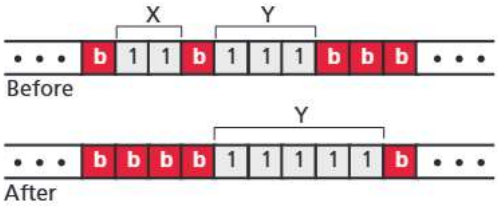
Example 17.4

Let us show a very simple example. Suppose we want to simulate the fourth macro, Y← Y + X (page 4). As we discussed before, this macro can be simulated using the while statement in Simple Language:

```
while (X)
{
   decr (X)
   incr (Y)
}
```

To make the procedure shorter, we assume that X = 2 and Y = 3, so the result is Y = 5. Figure 17.11 shows the state of the tape before and after applying the macro. Note that in this program we erase the value of X to make the process shorter, but the original value of X can be preserved if we allow other symbols on the tape.

Figure 17.11 *Configuration of the tapes for Example 17.4*



Since X = 2, the program goes through two iterations. At the end of the first iteration, the value of X = 1 and the value of Y = 4. At the end of the second iteration, the value of X = 0 and the value of Y = 5.

17.2.3 The Church–Turing thesis

We have shown that a Turing machine can simulate the three basic statements in Simple Language. This means that the Turing machine can also simulate all the macros we defined for Simple Language. Can the Turing machine therefore solve any problem that can be solved by a computer? The answer to this question can be found in the **Church–Turing thesis**.

The Church–Turing Thesis
If an algorithm exists to do a symbol manipulation task, then a Turing machine exists to do that task.

Based on this claim, any symbol-manipulation task that can be done by writing an algorithm to do so can also be done by a Turing machine. Note that this is only a *thesis*, not a

theorem. A theorem can be proved mathematically, a thesis cannot. Although this thesis probably can never be proved, there are strong arguments in its favor. First, no algorithms have been found that cannot be simulated using a Turing machine. Second, it has been proven that all computing models that *have* been mathematically proved are equivalent to the Turing machine model.

17.3 GÖDEL NUMBERS

In theoretical computer science, an unsigned number is assigned to every program that can be written in a specific language. This is usually referred to as the **Gödel number**, named after the Austrian mathematician Kurt Gödel.

This assignment has many advantages. First, programs can be used as a single data item as input to other programs. Second, programs can be referred to by just their integer representations. Third, the numbering can be used to prove that some problems cannot be solved by a computer, by showing that the total number of problems in the world is much larger than the total number of programs that can ever be written.

Different methods have been devised for numbering programs. We use a very simple transformation to number programs written in our Simple Language. Simple Language uses only fifteen symbols (Table 17.2).

Table 17.2 Code for symbols used in Simple Language

Symbol	*Hex code*	*Symbol*	*Hex code*
1	1	9	9
2	2	incr	A
3	3	decr	B
4	4	while	C
5	5	{	D
6	6	}	E
7	7	X	F
8	8		

Note that in this language we use only X, X_1, X_2, …, X_9 as variables. To encode these variables, we handle X_n as two symbols X and n (X_3 is X and 3). If we have a macro with other variables, they need to be changed to X_n.

17.3.1 Representing a program

Using the table, we can represent any program written in Simple Language by a unique positive integer by following these steps:

1. Replace each symbol with the corresponding hexadecimal code from the table.
2. Interpret the resulting hexadecimal number as an unsigned integer.

Example 17.5

What is the Gödel number for the program **incr** X?

Solution

Replace each symbol by its hexadecimal code.

$$\textbf{incr X} \rightarrow (\textbf{AF})_{16} \rightarrow \textbf{175}$$

So this program can be represented by the number 175.

17.3.2 Interpreting a number

To show that the numbering system is unique, use the following steps to interpret a Gödel number:

1. Convert the number to hexadecimal.
2. Interpret each **hexadecimal digit** as a symbol using Table 17.2 (ignore a 0).

Note that while any program written in Simple Language can be represented by a number, not every number can be interpreted as a valid program. After conversion, if the symbols do not follow the syntax of the language, the number is not a valid program.

Example 17.6

Interpret 3058 as a program.

Solution

Change the number to hexadecimal and replace each digit with the corresponding symbol:

$$3058 \rightarrow (\text{BF2})_{16} \rightarrow \textbf{decr X 2} \rightarrow \textbf{decr } (X_2)$$

This means that the equivalent code in Simple Language is decr (X_2). Note that in Simple Language, each program includes input and output. This means that the combination of a program and its inputs defines the Gödel number.

17.4 THE HALTING PROBLEM

Almost every program written in a programming language involves some form of repetition—loops or recursive functions. A repetition construct may never terminate (halt): that is, a program can run forever if it has an infinite loop. For example, the following program in Simple Language never terminates:

```
X → 1
while (X)
{
}
```

A classical programming question is:

Can we write a program that tests whether or not any program, represented by its Gödel number, will terminate?

The existence of this program would save programmers a lot of time. Running a program without knowing if it halts or not is a tedious job. Unfortunately, it has now been proven that such a program cannot exist—much to the disappointment of programmers!

17.4.1 The halting problem is not solvable

Instead of saying that the testing program does not exist and can never exist, the computer scientist says 'The **halting problem** is not solvable'.

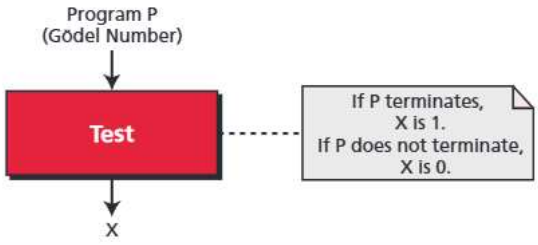
Proof

Let us give an informal proof about the nonexistence of this testing program. Our method, called *proof by contradiction*, is often used in mathematics: we assume that the program does exist, then show that its existence creates a contradiction—therefore, it cannot exist. We use three steps to show the proof in this approach.

Step 1

In this step, we assume that a program, called Test, exists. It can accept any program such as *P*, represented by its Gödel number, as input, and outputs either 1 or 0. If *P* terminates, the output of Test is 1: if *P* does not terminate, the output of Test is 0 (see Figure 17.12).

Figure 17.12 *Step 1 in the proof*



Step 2

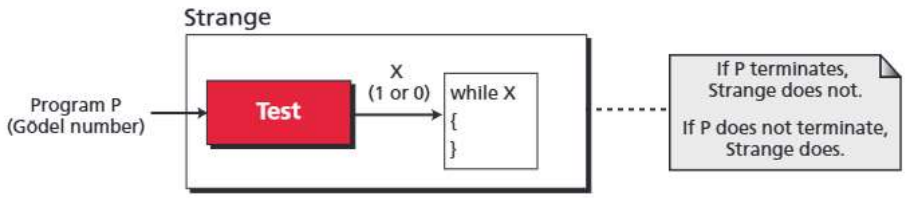
In this step, we create another program called Strange that is made of two parts: a copy of Test at the beginning and an empty loop—a loop with an empty body—at the end. The loop uses *X* as the testing variable, which is actually the output of the Test program. This program also uses *P* as the input. We call this program Strange for the following reason: if *P* terminates, the first part of Strange, which is a copy of Test, outputs 1. This 1 is input to the loop. The loop does not terminate—it's an infinite loop—and consequently Strange does not terminate. If *P* does not terminate, the first part of Strange, which is a copy of

Test, outputs 0. This 0 is input to the loop, so the loop does terminate—it's now a finite loop, the loop never iterates—and consequently, Strange does terminate. In other words, we have these strange situations:

If P terminates, Strange does not terminate.
If P does not terminate, Strange terminates.

Figure 17.13 shows Step 2 of the proof.

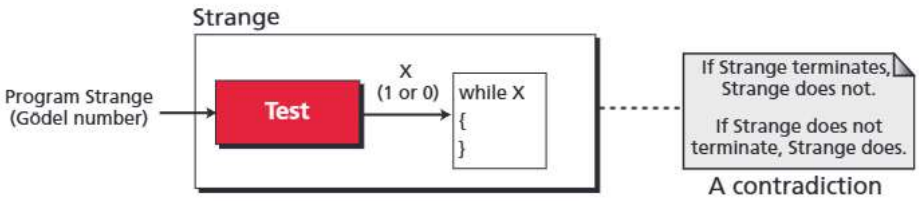
Figure 17.13 *Step 2 in the proof*



Step 3

Having written the program Strange, we test it with itself (its Gödel number) as input. This is legitimate because we did not put any restrictions on *P*. Figure 17.14 shows the situation.

Figure 17.14 *Step 3 in the proof*



Contradiction

If we assume that Test exists, we have the following contradictions:
Strange does not terminate if Strange terminates.
Strange terminates if Strange does not terminate.

This proves that the Test program cannot exist and that we should stop looking for it, so…

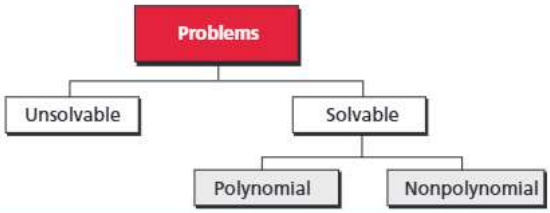
The halting problem is unsolvable.

The unsolvability of the halting program has proved that many other programs are also unsolvable, because if they are solvable, then the halting problem is solvable—which it is not.

17.5 THE COMPLEXITY OF PROBLEMS

Now that we have shown that at least one problem is unsolvable by a computer, we'll touch on this important issue a bit more. In computer science, we can say that, in general, problems can be divided into two categories: **solvable problems** and **unsolvable problems**. The solvable problems can themselves be divided into two categories: *polynomial* and *nonpolynomial* problems (Figure 17.15).

Figure 17.15 *Taxonomy of problems*



17.5.1 Unsolvable problems

There are an infinite number of problems that cannot be solved by a computer: one is the halting problem. One method to prove that a problem is not solvable is to show that if that problem is solvable, the halting problem is solvable too. In other words, prove that the solvability of a problem results in the solvability of the halting problem.

17.5.2 Solvable problems

There are many problems that *can* be solved by a computer. However, we often want to know how *long* it takes for the computer to solve that problem. In other words, how complex is the program?

The complexity of a program can be measured in several different ways, such as its run time, the memory it needs, and so on. One approach is the program's run time—how long does the program take to run?

Complexity of solvable problems

One way to measure the complexity of a solvable problem is to find the number of operations executed by the computer when it runs the program. In this way, the complexity measure is independent of the speed of the computer that runs the program. This measure

of complexity can depend on the number of inputs. For example, if a program is processing a list, such as sorting it, the complexity depends on the number of elements in the list.

Big-O notation

With the speed of computers today, we are not as concerned with exact numbers as with general orders of magnitude. For example, if the analysis of two programs shows that one executes 15 operations (or a set of operations) while the other executes 25, they are both so fast that we can't see the difference. On the other hand, if the numbers are 15 *versus* 1500, we should be concerned.

This simplification of efficiency is known as **big-O notation**. We present the idea of this notation without delving into its formal definition and calculation. In big-O notation, the number of operations—or a set of related operations—is given as a function of the number of inputs. The notation $O(n)$ means a program does n operations for n inputs, while the notation $O(n^2)$ means a program does n^2 operations for n inputs.

Example 17.7

Imagine we have written three different programs to solve the same problem. The first one has a complexity of $O(\log_{10} n)$, the second $O(n)$, and the third $O(n^2)$. Assuming 1 million inputs, how long does it take to execute each of these programs on a computer that executes one instruction in one microsecond, that is, one million instructions per second?

Solution

The following shows the analysis:

1st program:	$n = 1\,000\,000$	$O(\log_{10} n) \rightarrow 6$	Time $\rightarrow$ 6 μs
2nd program:	$n = 1\,000\,000$	$O(n) \rightarrow 1\,000\,000$	Time $\rightarrow$ 1 sec
3rd program:	$n = 1\,000\,000$	$O(n^2) \rightarrow 10^{12}$	Time $\rightarrow$ 277 h

Polynomial problems

If a program has a complexity of $O(\log n)$, $O(n)$, $O(n^2)$, $O(n^3)$, $O(n^4)$, or $O(n^k)$, where k is a constant, it is called *polynomial*. With the speed of computers today, we can get solutions to **polynomial problems** with a reasonable number of inputs, for example 1000 to 1 million.

Non polynomial problems

If a program has a complexity that is greater than a polynomial—for example, $O(10^n)$ or $O(n!)$—it can be solved if the number of inputs is very small, such as fewer than 100. If the number of inputs is large, one could sit in front of the computer for months to see the result of a **nonpolynomial problem**. But who knows? At the rate at which the speed of computers is increasing, we may be able to get a result for this type of problem in less time.

5/1/25, 9:57 PM study viewer

17.6 END-CHAPTER MATERIALS

17.6.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Hennie, F. *Introduction to Computability*, Reading, MA: Addison-Wesley, 1977
- Hofstadter, D. *Gödel, Escher, Bach: An Eternal Golden Braid*, St. Paul, MN: Vintage, 1980
- Hopcroft, J., Motwani, R. and Ullman, J. *Introduction to Automata Theory, Languages, and Computation*, Reading, MA: Addison-Wesley, 2006
- Kfoury, A., Moll, R. and Michael, A. *A Programming Approach to Computability*, New York: Springer, 1982
- Minsky, M. *Computation: Finite and Infinite Machines*, Engelwood Cliffs, NJ: Prentice-Hall, 1967
- Sipser, M. *Introduction to the Theory of Computation*, Boston, MA: Course Technology, 2005

17.6.2 Key terms

big-O notation 468	loop statement 452
Church–Turing thesis 462	macro 453
controller 457	nonpolynomial problem 468
decrement statement 452	polynomial problem 468
finite state automaton 457	read/write head 457
Gödel number 463	solvable problem 467
halting problem 465	Turing machine 456
hexadecimal digit 464	unsolvable problem 467
increment statement 452	

17.6.3 Summary

- We can define a computer language with only three statements: the *increment* statement, the *decrement* statement, and the *loop* statement. The increment statement adds 1 to a variable, the decrement statement subtracts 1 from a variable, and the loop statement repeats an action or a series of actions while the value of a variable is not 0.
- It can be shown that this simple programming language can simulate several statements found in some popular languages. We call each simulation a *macro* and use it in other simulations without the need to repeat code.

https://studylib.net/doc/26059254/foundations-computer-science-4th 489/706

- The Turing machine was designed to solve computable problems. It is the foundation of modern computers. A Turing machine is made of three components: a tape, a controller, and a read/write head.
- Based on the Church–Turing thesis, if an algorithm to do a symbol manipulation task exists, then a Turing machine to do that task also exists.
- In theoretical computer science, an unsigned number is assigned to every program that can be written in a specific language. This is usually referred to as the *Gödel number*.
- A classical programming question is whether a program that can determine if another program halts can be constructed. Unfortunately, it has now been proved that this program cannot exist: the halting problem is not solvable.
- In computer science, problems can be divided into two categories: solvable problems and unsolvable problems. The solvable problems can themselves be divided into two categories: polynomial and non polynomial problems.

17.7 PRACTICE SET

17.7.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

17.7.2 Review questions

Q17-1. Name and describe the functions of the three basic statements that are the foundation of other statements in Simple Language.

Q17-2. Show how assigning the value of one variable to another uses the three basic statements.

Q17-3. What is the relationship between the Turing machine and our Simple Language?

Q17-4. What are the components of the Turing machine and what is the function of each component?

Q17-5. Describe one way to delimit the data on a Turing machine's tape.

Q17-6. When a read/write head in a Turing machine finishes reading and writing a symbol, what are its next options?

Q17-7. How is a transition state diagram related to a Turing machine controller?

Q17-8. How is a transition state diagram related to a transition table? Do they have the same information? Which has more information?

Q17-9. What is a Gödel number? How do we use a Gödel number to prove that the halting problem is not solvable?

Q17-10. Compare and contrast the complexity of a polynomial solvable problem and a non polynomial solvable problem.

17.7.3 Problems

P17-1. Rewrite Algorithm 17.6 ($Y \leftarrow X$) so that it preserves the value of X.

P17-2. Rewrite Algorithm 17.7 so that it calculates $Z \leftarrow Y + X$ while preserving the values of X and Y.

P17-3. Rewrite Algorithm 17.8 so that it calculates $Z \leftarrow Y \times X$ while preserving the values of X and Y.

P17-4. Rewrite Algorithm 17.9 so that it calculates $Z \leftarrow Y^X$ while preserving the values of X and Y.

P17-5. Simulate the following macro using the previously defined statements or macros in Simple Language: $Y \leftarrow Y - X$.

P17-6. Simulate the following macro using the previously defined statements or macros in the Language (X can be only 0 or 1):

```
if (X) then
{
   A1
}
else
{
   A2
}
```

P17-7. Given a Turing machine with a single instruction (A, 1, b, R, B) and the tape configuration:

```
        ↓
...     b   1   1   1   b   ...
```

show the final configuration of the tape.

P17-8. Given a Turing machine with a single instruction (A, b, b, R, B) and the tape configuration:

```
        ↓
...     b   1   1   1   b   ...
```

show the final configuration of the tape.

P17-9. Given a Turing machine with five instructions (A, b, b, R, B), (B, 1, #, R, B), (B, b, b, L, C), (C, #, 1, L, C), (C, b, b, R, B) and the tape configuration:

```
        ↓
...     b   1   1   1   b   ...
```

show the final configuration of the tape.

P17-10. Show the state diagram of a Turing machine that increments a nonnegative integer represented in the binary system. For example, if the contents of the tape is $(101)_2$, it will be changed to $(110)_2$.

P17-11. Show that the simulation of **incr** (X) in the Turing machine, as defined in this chapter, gives the correct answer when $X = 0$.

P17-12. Show that the simulation of **decr** (X) in the Turing machine, as defined in this chapter, gives the correct answer when $X = 0$.

P17-13. Show how the simulation of a loop statement in the Turing machine, as defined in this chapter, can be changed to preserve the original value of X if we allow another symbol such as # to be used by the machine.

P17-14. Give the transition states and the program for the Turing machine that simulates the macro $X \leftarrow 0$.

P17-15. Give the transition states and the program for the Turing machine that simulates the macro $Y \leftarrow X$.

P17-16. A Turing machine uses a single 1 to represent the integer 0. Show how the integer *n* can be represented in this machine.

P17-17. What is the Gödel number for the macro $X_1 \leftarrow 0$?

P17-18. What is the Gödel number for the macro $X_2 \leftarrow 2$?

P17-19. What is the Gödel number for the macro $X_3 \leftarrow X_1 + X_2$?

CHAPTER 18

Artificial Intelligence

In this chapter of the book, we offer an introduction to artificial intelligence (AI). The first section is a brief history and an attempt to define artificial intelligence. *Knowledge representation*, a broad and well-developed area in AI, is discussed in the next section. We then introduce *expert systems*, systems that can replace human expertise when it is needed but not available. We then discuss how artificial intelligence can be used to simulate the normal (mundane) behavior of human beings in two areas: *image processing* and *language analysis*. We then show how expert systems and *mundane systems* can solve problems using different searching method. Finally, we discuss how *neural networks* can simulate the process of learning in an intelligent agent.

Objectives

After studying this chapter, the student should be able to:

- Define and give a brief history of artificial intelligence.
- Describe how knowledge is represented in an intelligent agent.
- Show how expert systems can be used when a human expert is not available.
- Show how an artificial agent can be used to simulate mundane tasks performed by human beings.
- Show how expert systems and mundane systems can use different search techniques to solve problems.
- Show how the learning process in humans can be simulated, to some extent, using neural networks that create the electronic version of a neuron called a *perceptron*.

18.1 INTRODUCTION

In this section we first try to define the term **artificial intelligence (AI)** informally and give a brief history of it. We also define an *intelligent agent* and its two broad categories. Finally, we mention two programming languages that are commonly used in artificial intelligence.

18.1.1 What is artificial intelligence?

Although there is no universally agreed definition of artificial intelligence, we accept the following definition that matches the topics covered in this chapter:

Artificial intelligence is the study of programmed systems that can simulate, to some extent, human activities such as perceiving, thinking, learning, and acting.

18.1.2 A brief history of artificial intelligence

Although artificial intelligence as an independent field of study is relatively new, it has some roots in the past. We can say that it started 2400 years ago when the Greek philosopher Aristotle invented the concept of logical reasoning. The effort to finalize the language of logic continued with Leibniz and Newton. George Boole developed Boolean algebra in the nineteenth century (Appendix E) that laid the foundation of computer circuits. However, the main idea of a thinking machine came from Alan Turing, who proposed the Turing test. The term 'artificial intelligence' was first coined by John McCarthy in 1956.

18.1.3 The Turing test

In 1950, Alan Turing proposed the **Turing Test**, which provides a definition of *intelligence* in a machine. The test simply compares the intelligent behavior of a human being with that of a computer. An interrogator asks a set of questions that are forwarded to both a computer and a human being. The interrogator receives two sets of responses, but does not know which set comes from the human and which set from the computer. After careful examination of the two sets, if the interrogator cannot definitely tell which set has come from the computer and which from the human, the computer has passed the Turing test for intelligent behavior.

18.1.4 Intelligent agents

An **intelligent agent** is a system that perceives its environment, learns from it, and interacts with it intelligently. Intelligent agents can be divided into two broad categories: *software agents* and *physical agents*.

Software agents

A **software agent** is a set of programs that are designed to do particular tasks. For example, some intelligent systems can be used to organize electronic mail (email). This type of

agent can check the contents of received emails and classify them into different categories (junk, less important, important, very important, and so on). Another example of software agents is a search engine used to search the World Wide Web and find sites that can provide information about a requested subject.

Physical agents

A **physical agent** (robot) is a programmable system that can be used to perform a variety of tasks. Simple robots can be used in manufacturing to do routine jobs such as assembling, welding, or painting. Some organizations use mobile robots that do delivery jobs such as distributing mail or correspondence to different rooms. There are mobile robots that are used underwater for prospecting for oil.

A humanoid robot is an autonomous mobile robot that is supposed to behave like a human. Although humanoid robots are prevalent in science fiction, there is still a lot of work to do before such robots will be able to interact properly with their surroundings and learn from events that occur there.

18.1.5 Programming languages

Although some all-purpose languages such as C, C++, and Java are used to create intelligent software, two languages are specifically designed for AI: LISP and PROLOG.

LISP

LISP (LISt Programming) was invented by John McCarthy in 1958. As the name implies, LISP is a programming language that manipulates lists. LISP treats data as well as programs as lists, which means that a LISP program can change itself. This feature matches the idea of an intelligent agent that can learn from its environment and improve its behavior.

However, one drawback of LISP is its sluggishness. It is slow if the list to be handled is long. Another drawback is the complexity of its syntax.

PROLOG

PROLOG (PROgramming in LOGic) is a language that can build a database of facts and a knowledge base of rules. A program in PROLOG can use logical reasoning to answer questions that can be inferred from the knowledge base. However, PROLOG is not a very efficient programming language. Some complex problems can be more efficiently solved using other languages, such as C, C++, or Java.

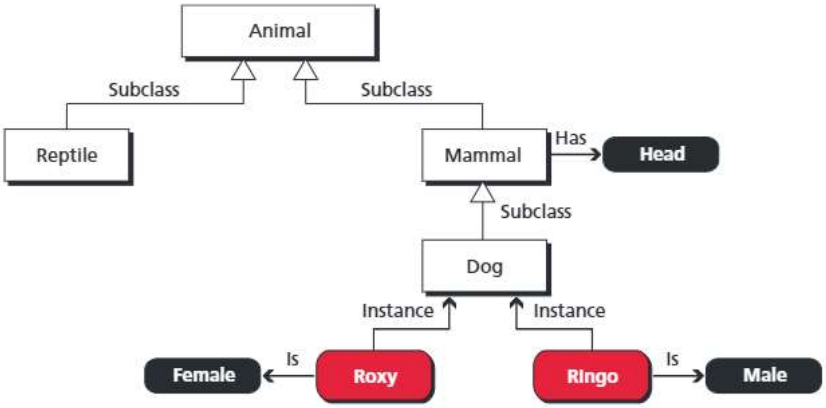
18.2 KNOWLEDGE REPRESENTATION

If an artificial agent is supposed to solve some problems related to the real world, it needs to be able to represent knowledge somehow. Facts are represented as data structures that can be manipulated by programs stored inside the computer. In this section, we describe four common methods for representing knowledge: *semantic networks*, *frames*, *predicate logic*, and *rule-based systems*.

18.2.1 Semantic networks

Semantic networks were developed in the early 1960s by Richard H. Richens. A semantic network uses directed graphs to represent knowledge. A directed graph, as discussed in Chapter 12, is made of vertices (nodes) and edges (arcs). Semantic networks use vertices to represent concepts, and edges (denoted by arrows) to represent the relation between two concepts (Figure 18.1).

Figure 18.1 *A simple semantic network*



Concepts

To develop an exact definition of a concept, experts have related the definition of concepts to the theory of sets. A concept, therefore, can be thought of as a set or a subset. For example, *animal* defines the set of all animals, *horse* defines the set of all horses and is a subset of the set *animal*. An object is a member (instance) of a set. Concepts are shown by vertices.

Relations

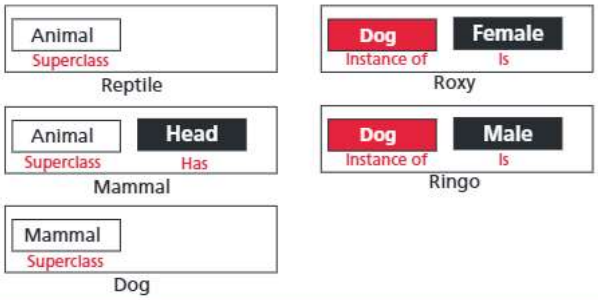
In a semantic network, relations are shown by edges. An edge can define a *subclass* relation—the edge is directed from the subclass to its superclass. An edge can also define an *instance* relation—the edge is directed from the instance to the set to which it belongs. An edge can also define an *attribute* of an object (color, size, …). Finally, an edge can define a property of an object, such as possessing another object. One of the most important relations that can be well defined in a semantic network is *inheritance*. An inheritance relation defines the fact that all the attributes of a class are present in an inherited class. This can be used to infer new knowledge from the knowledge represented by the graph.

18.2.2 Frames

Frames are closely related to semantic networks. In semantic networks, a graph is used to represent knowledge: in frames, data structures (records) are used to represent the same knowledge. One advantage of frames over semantic networks is that programs can handle

frames more easily than semantic networks. Figure 18.2 shows how the semantic network shown in Figure 18.1 can be implemented using frames.

Figure 18.2 *A set of frames representing semantic network*



Objects

A node in a semantic network becomes an object in a set of frames, so an object can define a class, a subclass, or an instance of a class. In Figure 18.2 reptile, mammal, dog, Roxy, and Ringo are objects.

Slots

Edges in semantic networks are translated into *slots*—fields in the data structure. The name of the slot defines the type of the relationship and the value of the slot completes the relationship. In Figure 18.2, for example, *animal* is a slot in the *reptile* object.

18.2.3 Predicate logic

The most common knowledge representation is **predicate logic**. Predicate logic can be used to represent complex facts. It is a well-defined language developed via a long history of theoretical logic. Although this section defines predicate logic, we first introduce **propositional logic**, a simpler language. We then discuss predicate logic, which employs propositional logic.

Propositional logic

Propositional logic is a language made up from a set of sentences that can be used to carry out logical reasoning about the world.

Operators

Propositional logic uses five operators, as shown below:

$\neg$	$\vee$	$\wedge$	$\rightarrow$	$\leftrightarrow$
(not)	(or)	(and)	(if … then)	(if and only if)

The first operator is unary—the operator takes only one sentence: the other four operators are binary—they take two sentences. The logical value (*true* or *false*) of each sentence depends on the logical value of the atomic sentences (sentences with no operators) of which the complex sentence is made. Figure 18.3 shows the truth table for each logical operator in propositional logic. Truth tables were introduced in Chapter 4 and explained in Appendix E.

Figure 18.3 *Truth table for five operators in propositional logic*

A	¬ A
F	T
T	F

A	B	A ∧ B
F	F	F
F	T	F
T	F	F
T	T	T

A	B	A ∨ B
F	F	F
F	T	T
T	F	T
T	T	T

A	B	A → B
F	F	T
F	T	T
T	F	F
T	T	T

A	B	A ↔ B
F	F	T
F	T	F
T	F	F
T	T	T

Sentence

A sentence in this language is defined recursively as shown below:

1. An uppercase letter, such as A, B, S, or T, that represents a statement in a natural languages, is a sentence.
2. Any of the two constant values (*true* and *false)* is a sentence.
3. If P is a sentence, then ¬P is a sentence.
4. If P and Q are sentences, then P ∨ Q, P ∧ Q, P → Q, and P ↔ Q are sentences.

Example 18.1

The following are sentences in propositional language:

a. Today is Sunday (S).
b. It is raining (R).
c. Today is Sunday or Monday (S ∨ M).
d. It is not raining (¬R)
e. If a dog is a mammal then a cat is a mammal (D → C)

Deduction

In AI we need to create new facts from the existing facts. In propositional logic, the process is called *deduction*. Given two presumably true sentences, we can deduce a new true sentence. The first two sentences are called *premisses*: the deduced sentence is called the *conclusion*. The whole is called an *argument*. For example:

Premiss 1:	**Either he is at home or at the office**
Premiss 2:	**He is not at home**
Conclusion	**Therefore, he is at the office**

If we use H for 'he is at home', O for 'he is at office', and the symbol |- for the 'therefore', then we can show the above argument as:

$$\{H \vee O, \neg H\} \vdash O$$

The question is how we can prove if a deductive argument is *valid*. A valid deductive argument is an argument whose conclusions follow necessarily from its premisses. In other words, in a valid deductive argument, it is impossible for the conclusion to be false while its premisses all are true.

One way to do this is to create a truth table for the premisses and the conclusion. A conclusion is invalid if we can find a *counterexample* case: a case in which both premisses are true, but the conclusion is false.

Example 18.2

The validity of the argument $\{H \vee O, \neg H\} \vdash O$ can be proved using the following truth table:

H	O	$H \vee O$	$\neg H$	O	
F	F	F	T	F	
F	T	T	T	T	OK
T	F	T	F	F	
T	T	T	F	T	
		Premiss	Premiss	Conclusion	

The only row to be checked is the second row. This row does not show a counterexample, so the argument is valid. There are however arguments that are not logically valid. For example:

Premiss 1:	**If she is rich, she has a car.**
Premiss 2:	**She has a car**
Conclusion	**Therefore, she is rich.**

It can be seen that even if the first two sentences are true, the conclusion can be false. We can show the above argument as $\{R \rightarrow C, C\} \vdash R$, in which R means 'She is rich', and C means 'she has a car'.

Example 18.3

The argument $\{R \rightarrow C, C\} \vdash R$ is not valid because a counterexample can be found:

R	C	$R \rightarrow C$	C	R
F	F	T	F	F
F	T	T	T	F
T	F	F	F	T
T	T	T	T	T
		Premiss	Premiss	Conclusion

Here row 2 and row 4 need to be checked. Although row 4 is ok, row 2 shows a counter example (two true premisses result in a false conclusion). The argument is therefore invalid.

An argument is valid if no counterexample can be found.

Predicate logic

In propositional logic, a symbol that represents a sentence is atomic: it cannot be broken up to find information about its components. For example, consider the sentences:

P_1: 'Linda is Mary's mother'	P_2: 'Mary is Anne's mother'

We can combine these two sentences in many ways to create other sentences, but we cannot extract any relation between Linda and Anne. For example, we cannot infer from the above two sentences that Linda is the grandmother of Anne. To do so, we need predicate logic: the logic that defines the relation between the parts in a proposition.

In predicate logic, a sentence is divided into a predicate and arguments. For example, each of the following propositions can be written as predicates with two arguments:

P_1: 'Linda is Mary's mother'	becomes	mother (Linda, Mary)
P_2: 'Mary is Anne's mother'	becomes	mother (Mary, Anne)

The relationship of motherhood in each of the above sentences is defined by the predicate *mother*. If the object *Mary* in both sentences refers to the same person, we can infer a new relation between Linda and Anne: grandmother (Linda, Anne). This is the whole purpose of predicate logic.

Sentence

A sentence in predicate language is defined as follows:

1. A predicate with n arguments such as *predicate_name* ($argument_1$, ..., $argument_n$) is a sentence. The *predicate_name* relates arguments to each other. Each argument can be:
   a. A constant, such as *human*, *animal*, *John*, *Mary*.
   b. A variable, such as x, y, and z.
   c. A function such as *mother (Anne)*. Note that a function is a predicate that is used as an argument: a function returns an object that can takes the place of an argument.
2. Any of the two constant values (*true* and *false*) is a sentence.
3. If P is a sentence, then $\neg$P is a sentence.
4. If P and Q are sentences, then P $\vee$ Q, P $\wedge$ Q, P $\rightarrow$ Q, and P $\leftrightarrow$ Q are sentences.

Example 18.4

1. The sentence 'John works for Ann's sister' can be written as:

   works [John, sister (Ann)]

   in which the function sister (Ann) is used as an argument.
2. The sentence 'John's father loves Ann's sister' can be written as:

   loves [father (John), sister (Ann)]

Quantifiers

Predicate logic allows us to use **quantifiers**. Two quantifiers are common in predicate logic: ∀ and ∃.

1. The first, ∀, which is read as 'for all', is called the *universal quantifier*: it states that something is true for every object that its variable represents.
2. The second, ∃, which is read as 'there exists', is called the *existential quantifier*: it states that something is true for one or more objects that its variable represents.

Example 18.5

The following shows how English sentences can be written as sentences in predicate logic (X is a placeholder):

1. The sentence 'All men are mortals' can be written as:

   $$\forall x\,[\text{man}\,(x) \rightarrow \text{mortal}\,(x)]$$

2. The sentence 'Frogs are green' can be written as:

   $$\forall x\,[\text{frog}\,(x) \rightarrow \text{green}\,(x)]$$

   because the sentence can be written as 'All frogs are green' or 'Any frog is green'. The predicate *greenness* is applied to all frogs.
3. The sentence 'Some flowers are red' can be written as:

   $$\exists x\,[\text{flower}\,(x) \wedge \text{red}(x)]$$

   Note that the operator inside the bracket is ∧ instead of →, but the reason for this is beyond the scope of this book.
4. The sentence 'John has a book' can be written as:

   $$\exists\, x\,[\text{book}\,(x) \wedge \text{has}\,(\text{John}, x)]$$

   In other words, the sentence is changed to 'There exists a book that belongs to John'.
5. The sentence 'No frog is yellow' can be written as:

   $$\forall x[\text{frog}\,(x) \rightarrow \neg\text{yellow}\,(x)] \text{ or as } \neg\exists x\,[\text{frog}\,(x) \wedge \text{yellow}\,(x)]$$

   which means that 'It is not the case that there exists a frog and it is yellow'.

Deduction

In predicate logic, if there is no quantifier, the verification of an argument is the same as that which we discussed in propositional logic. However, the verification becomes more complicated if there are quantifiers. For example, the following argument is completely valid:

Premiss 1:	All men are mortals.
Premiss 2:	Socrates is a man.
Conclusion	Therefore, Socrates is mortal.

Verification of this simple argument is not difficult. We can write this argument as:

$$\forall x\,[\text{man}\,(x) \rightarrow \text{mortal}\,(x)]\,,\ \text{man (Socrates)}\ \vdash\ \text{mortal (Socrates)}$$

Since the first premises talks about all men, we can replace one instance of the class man (Socrates) in that premiss to get the following argument:

$$\text{man (Socrates)} \rightarrow \text{mortal (Socrates)}\,,\ \text{man (Socrates)}\ \vdash\ \text{mortal (Socrates)}$$

which is reduced to $M_1 \rightarrow M_2$, $M_1 \vdash M_2$, in which M_1 is man (Socrates) and M_2 is mortal (Socrates). The result is an argument in propositional logic and can be easily validated. However, there are many arguments in predicate logic that cannot be validated so easily. We need a set of systematic proofs that are beyond the scope of this book.

Beyond predicate logic

There have been further developments in logic to include the need of logical reasoning. Some examples of these include **high-order logic**, **default logic**, **modal logic**, and **temporal logic**. We briefly mention these topics here only for interest: their discussion is beyond the scope of this book.

High-order logic

High-order logic extends the scope of quantifiers $\forall$ and $\exists$ in predicate logic. These quantifiers in predicate logic bind variables x and y to instances (when instantiating). In high-order logic we can use these quantifiers for binding variables that stand for properties and relations. In this case, during instantiation, these variables are replaced by predicates. For example, we can have $\forall P\,(Pj \wedge Pa)$, where the subscripts j and a denote John and Anne, which means that John and Anne have exactly the same properties.

Modal logic

One fast-growing trend in logic is **modal logic**, which includes expressions such as 'could', 'should', 'may', 'might', 'ought', and so on, to express the grammatical mood of a sentence. In this logic, we can have symbols to denote operators such as 'it is possible that'.

Temporal logic

Temporal logic, like modal logic, extends predicate logic with a set of temporal operators such as 'from now on' or 'at some point in time' to include the time factor in the validity of the argument.

Default logic

In **default logic**, we assume that the default conclusion of an argument is acceptable if it is consistent with the contents of the knowledge base. For example, we assume that all birds fly unless there is something in the knowledge base that annuls this general fact.

18.2.4 Rule-based systems

A **rule-based system** represents knowledge using a set of rules that can be used to deduce new facts from known facts. The rules express what is true if specific conditions are met. A rule-based database is a set of *if... then...* statements in the form

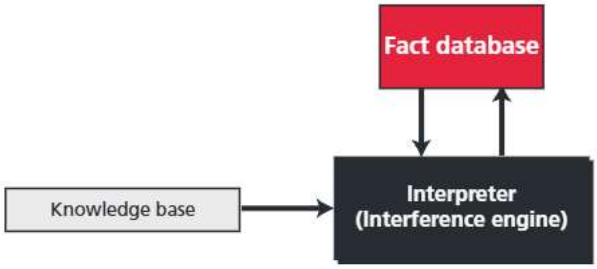
If A then B or A → B

in which A is called the *antecedent* and B is called the *consequent*. Note that in a rule-based system, each rule is handled independently without any connection to other rules.

Components

A rule-based system is made up of three components: an *interpreter* (or inference engine), a *knowledge base*, and a *fact database*, as shown in Figure 18.4.

Figure 18.4 *The components of a rule-based system*



Knowledge base

The knowledge base component in a rule-based system is a database (repository) of rules. It contains a set of pre-established rules that can be used to draw conclusions from the given facts.

Database of facts

The database of facts contains a set of conditions that are used by the rules in the knowledge base.

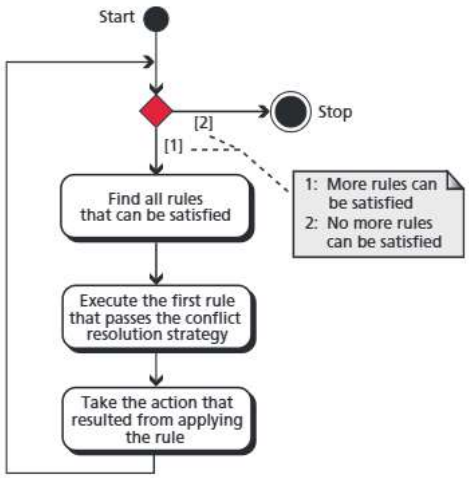
Interpreter

The interpreter (inference engine) is a processor or controller—a program, for example—that combines rules and facts. Interpreters are of two types: *forward chaining* and *backward chaining*, as we briefly explain.

Forward chaining

Forward chaining is the process in which an interpreter uses a set of rules and a set of facts to perform an action. The action can be just adding a new fact to the base of facts, or issuing some commands, such as start another program or a machine. The interpreter interprets and executes rules until no more rules can be interpreted. Figure 18.5 shows the basic algorithm.

Figure 18.5 *Flow diagram for forward chaining*



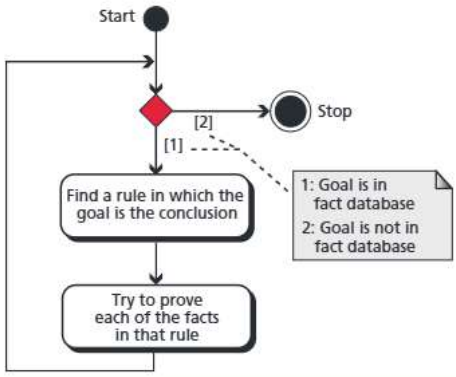
If there is any conflict in which two different rules can be applied to one fact or one rule can be applied to two facts, the system needs to call a conflict resolution procedure to solve the problem. This guarantees that only one of the outputs should be added to the database of facts or only one action should be taken. The discussion of conflict resolution is complex and beyond the scope of this book.

Backward chaining

Forward chaining is not very efficient if the system tries to prove a conclusion. All facts must be checked by all rules to come up with the given conclusion. In this case, it may be more efficient if backward chaining is used. Figure 18.6 shows the procedure for backward chaining.

5/1/25, 9:57 PM study viewer

Figure 18.6 *Flow diagram for backward chaining*



The process starts with the conclusion (goal). If the goal is already in the fact database, the process stops and the conclusion is proved. If the goal is not in the fact database, the system finds the rule that has the goal in its conclusion. However, instead of firing that rule, backward chaining is now applied to each fact in the rule (recursion). If all of the facts in that rule are found in the database fact, the original goal is proved.

18.3 EXPERT SYSTEMS

Expert systems use the knowledge representation languages discussed in the previous section to perform tasks that normally need human expertise. They can be used in situations in which that expertise is in short supply, expensive, or unavailable when required. For example, in medicine, an expert system can narrow down a set of symptoms to a likely subset of causes, a task normally carried out by a doctor.

18.3.1 Extracting knowledge

An expert system is built on predefined knowledge about its field of expertise. An expert system in medicine, for example, is built on the knowledge of a doctor specialized in the field for which the system is built: an expert system is supposed to do the same job as the human expert. The first step in building an expert system is therefore to extract the knowledge from a human expert. This extracted knowledge becomes the knowledge base we discussed in the previous section.

Extracting knowledge from an expert is normally a difficult task, for several reasons:

1. The knowledge possessed by the expert is normally heuristic: it is based on probability rather than certainty.
2. The expert often finds it hard to express their knowledge in such a way that it can be stored in a knowledge base as exact rules. For example, it is hard for an electrical

https://studylib.net/doc/26059254/foundations-computer-science-4th

engineer to show how, step by step, a faulty electric motor can be diagnosed. The knowledge is normally intuitive.

3. Knowledge acquisition can only be done via personal interview with the expert, which can be a tiring and boring task if the interviewer is not an expert in this type of interview.

The knowledge-extraction process is normally done by a *knowledge engineer*, who may not be expert in the field for which the expert system is to be built, but has the expertise to know how to do the interview and how to interpret the answers so that they can be used in building the knowledge base.

18.3.2 Extracting facts

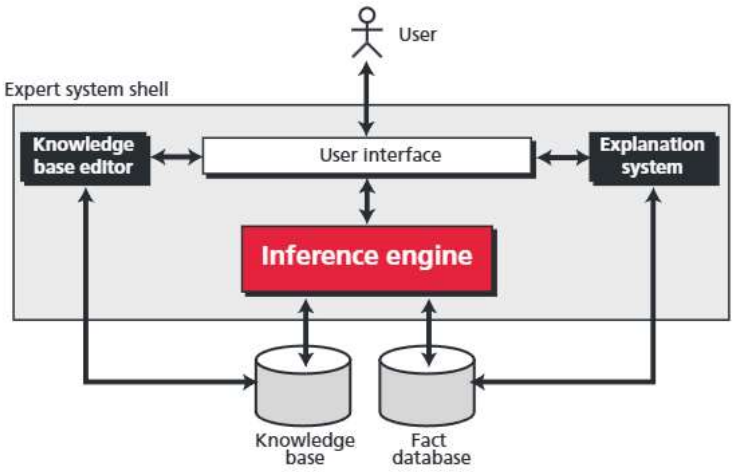
To be able to infer new facts or perform actions, a fact database is needed in addition to the knowledge base for a knowledge representation language. The fact database in an expert system is case-based, in which facts collected or measured are entered into the system to be used by the inference engine.

18.3.3 Architecture

Figure 18.7 shows the general idea behind the architecture of an expert system. As the figure shows, an expert system can have up to seven components: *user*, *user interface*, *inference engine*, *knowledge base*, *fact database*, *explanation system*, and *knowledge base editor*.

The inference engine is the heart of an expert system: it communicates with the knowledge base, fact database, and the user interface. Four of the seven components of an expert system—user interface, inference engine, explanation system, and knowledge base editor—can be made once and used for many applications, as they are not dependent on the particular knowledge base or fact database. The figure shows these components in the shaded box, normally called an *expert system shell*.

Figure 18.7 *The architecture of an expert system*



5/1/25, 9:57 PM study viewer

User

The user is the entity that uses the system to benefit from the expertise offered.

User interface

The user interface allows the user to interact with the system. The user interface can accept natural language from the user and interpret it for the system. Most user interfaces also offer a user-friendly menu system.

Inference engine

The inference engine is the heart of the system that uses the knowledge base and the fact database to infer the action to be taken.

Knowledge base

The knowledge base is a collection of knowledge based on interviews with experts in the relevant field of expertise.

Fact database

The fact database in an expert system is case-based. For each case, the user enters the available or measured data into the fact database to be used by the inference engine for that particular case.

Explanation system

The explanation system, which may not be included in all systems, is used to explain the rationale behind the decision made by the inference engine.

Knowledge base editor

The knowledge base editor, which may not be included in all systems, is used to update the knowledge base if new experience has been obtained from experts in the field.

18.4 PERCEPTION

One of the goals in artificial intelligence is to create a machine that behaves like an expert—an expert system. Another goal is to create a machine that behaves like an ordinary human. One of the meanings of the word 'perception' is understanding what is received through the senses—sight, hearing, touch, smell, taste. A human being sees a scene through the eyes, and the brain interprets it to extract the type of the objects in the scene. A human being hears a set of voice signals through the ears, and the brain interprets it as a meaningful sentence, and so on.

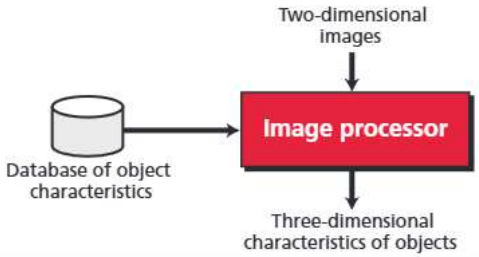
An intelligent agent should be able to perceive if it needs to act like a human being. AI has been particularly involved in two types of perception, sight and hearing, although other types of perception may be implemented in the future. In this section we briefly discuss these two areas of research.

https://studylib.net/doc/26059254/foundations-computer-science-4th 507/706

18.4.1 Image processing

Image processing or *computer vision* is an area of AI that deals with the perception of objects through the artificial eyes of an agent, such as a camera. An image processor takes a two-dimensional image from the outside world and tries to create a description of the three-dimensional objects present in the scene. Although this is an easy task for a human being, it turns out to be a difficult task for an artificial agent. The input presented to an image processor is one or more images from the scene, while the output is a description of the objects in the scene. The processor uses a database containing the characteristics of objects for comparison (Figure 18.8).

Figure 18.8 *The components of an image processor*

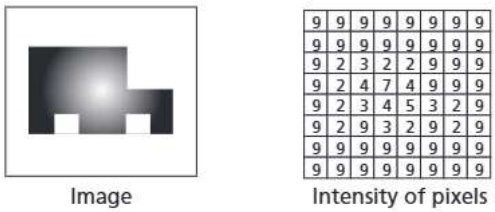


We need to emphasize that image acquisition uses photography and television technology to creates images. The concern of AI is how to interpret the images and extract the characteristics of the objects.

Edge detection

The first stage in image processing is **edge detection**: finding where the edges in the image are. Edges can define the boundaries between an object and its background in the image. Normally there is a sharp contrast between the surfaces belonging to an object and the environment, assuming that there is no camouflage. Edges show discontinuity in surface, in depth, or in illumination. For example, Figure 18.9 shows a very simple image and the intensity of pixels on a scale 0 to 9, where 0 is black and 9 is white. The edges can be detected by finding adjacent pixels with a large difference in intensity.

Figure 18.9 *The edge-detection process*



There are several mathematical methods that use the intensity of the pixels to find the boundary of the objects with respect to the background. The simplest method is to differentiate the matrix of intensities. The areas that have consistent intensity will produce low differentials (0 or 1): the edges will produce greatest differentials. Discussion of these methods is beyond the scope of this book: we recommend the reference books listed at the end of this chapter for further study.

Segmentation

Segmentation is the next stage in image analysis. Segmentation divides the image into homogeneous segments or areas. The definition of homogeneity differs in different methods, but in general a homogeneous area is an area in which the intensity of pixels varies smoothly. Segmentation is very similar to edge detection. In edge detection, the boundaries of the object and the background are found: in segmentation, the boundaries between different areas inside the object are found. After segmentation, the object is divided into different areas.

Several methods have been used for segmentation. One is called **thresholding**, in which a pixel with a specific intensity is selected and the process tries to find all the pixels with the same or very close intensity. All pixels found in this way make a segment. Another method is called splitting. Splitting takes an area that is not homogeneous and divides it into several homogeneous areas. Still another method is called merging, which can be used to merge areas with the same pixel intensity.

Finding depth

The next step in image analysis is to find the depth of the object or objects in the image. Depth finding can help the intelligent agent to gauge how far the object is from it. Two general methods have been used for this purpose: *stereo vision* and *motion*.

Stereo vision

Stereo vision (sometimes called *stereopsis*) uses the technique deployed by human eyes to find the depth of the object. To have good distance recognition, a human being needs two eyes. If the object is very close, the two images created in our eyes are different, but if the object is far away the two images are almost the same. Without delving into mathematical calculation and proof, we can say that one of the tools for recognizing the distance of objects is to use two eyes or two cameras. The picture created from two cameras can help the intelligent agent to gauge if the object is close or far away.

Motion

Another method that can help to find the distance of objects in an image is to create several images when one or more objects are moving. The relative position of a moving object with respect to other objects in the scene can give a clue to the distance of objects. For example, assume that a video shows a person moving in front of a house. The relative position of the person and house (a close object) will change, but the relative position of the person and a distant mountain will remain the same. The intelligent agent can conclude that the house is close but the mountain is far away.

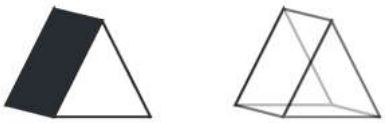
Finding orientation

Orientation of the object in the scene can be found using two techniques: *shading* and *texture*.

Shading

The amount of light reflected from a surface depends on several factors. If the optical properties of the different surfaces of an object are the same, the amount of reflection depends on the orientation of the surface (its relative position) which reflects the light source. Figure 18.10 shows two drawn objects. The one that is shaded definitely shows the orientation of the object's surfaces more accurately.

Figure 18.10 *The effect of shading on orientation finding*



Texture

Texture (a regularly repeated pattern) can also help in finding the orientation or the curvature of a surface. If an intelligent agent can recognize the pattern, it can help it to find an object's orientation or curvature.

Object recognition

The last step in image processing is object recognition. To recognize an object, the agent needs to have a model of the object in memory for comparison. However, creating and storing a model for each object in the view is an impossible task. One solution is to assume that the objects to be recognized are compound objects made of a set of simple geometric shapes. These primitive shapes can be created and stored in the intelligent agent's memory, then classes of object that we need the agent to recognize can be created from a combination of these objects, and stored.

When an agent 'sees' an object, it tries to decompose the object into a combination of the primitives. If the combination matches one of the classes already known to the object, the object is recognized. Figure 18.11 shows a small set of primitive geometric shapes.

Figure 18.11 *Primitive geometric shapes*

Applications

One of the areas in which image processing has found application is in manufacturing, particularly on assembly lines. A robot with image-processing capability can be used to determine the position of an object on the assembly line. In this environment, where the number of objects to be perceived is limited, an image processor can be very helpful.

18.4.2 Language understanding

One of the inherent capabilities of a human being is to understand—that is, interpret—the audio signals that they perceive. A machine that can understand natural language can be very useful in daily life. For example, it can replace—most of the time—a telephone operator. It can also be used on occasions when a system needs a predefined format of queries. For example, the queries given to a database must normally follow the format used by that specific system. A machine that can understand queries in natural language and translate them to formal queries can be very useful.

We can divide the task of a machine that understands natural language into four consecutive steps: *speech recognition*, *syntactic analysis*, *semantic analysis*, and *pragmatic analysis*.

Speech recognition

The first step in natural language processing is **speech recognition**. In this step, a speech signal is analyzed and the sequence of words it contains are extracted. The input to the speech recognition subsystem is a continuous (analog) signal: the output is a sequence of words. The signal needs to be divided into different sounds, sometimes called *phonemes*. The sounds then need to be combined into words. The detailed process, however, is beyond the scope of this book: we leave the task to specialized books in speech recognition.

Syntactic analysis

The **syntactic analysis** step is used to define how words are to be grouped in a sentence. This is a difficult task in a language like English, in which the function of a word in a sentence is not determined by its position in the sentence. For example, in the following two sentences:

Mary rewarded John.

John was rewarded by Mary.

it is always John who is rewarded, but in the first sentence John is in the last position and Mary is in the first position. A machine that hears any of the above sentences needs to interpret them correctly and come to the same conclusion no matter which sentence is heard.

Grammar

The first tool to correctly analyze a sentence is a well-defined grammar. A fully developed language like English has a very long set of grammatical rules. We assume a very small subset of the English language and define a very small set of rules just to show the idea.

The grammar of a language can be defined using several methods: we use a simple version of BNF (Backus–Naur Form) that is used in computer science to define the syntax of a programming language (Table 18.1).

Table 18.1 A simple grammar

	Rule		
1	Sentence	→	NounPhrase VerbPhrase
2	Noun-Phrase	→	Noun | Article Noun | Article Adjective Noun
3	Verb Phrase	→	Verb | Verb NounPhrase | Verb NounPhrase Adverb
4	Noun	→	[home] | [cat] | [water] | [dog] | [John] | [Mary] |
5	Article	→	[a] | [the]
6	Adjective	→	[big] | [small] | [tall] | [short] | [white] | [black] |
7	Verb	→	[goes] | [comes] | [eats] | [drinks] | [has] / [loves]

The first rule defines a sentence as a noun phrase followed by a verb phrase. The second rule defines three choices for a noun phrase: a single noun, an article followed by a noun, or an article followed by an adjective and a noun. The fourth rule explicitly defines what a noun can be. In our simple language, we have defined only seven nouns: in a language like English the list of nouns is defined in a dictionary. The sixth rule also defines a very small set of adjectives and the seventh rule a small set of verbs.

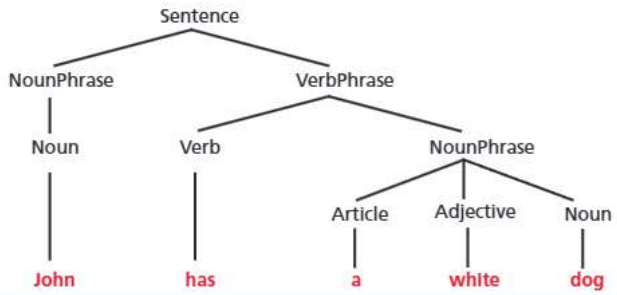
Although the syntax of our language is very primitive, we can make many sentences out of it. For example, we can have:

John comes home.
Mary drinks water.
John has a white dog.
John loves Mary.
Mary loves John.

Parser

It should be clear that even a simple grammar as defined in Table 18.1 uses different options. A machine that determines if a sentence is grammatically (syntactically) correct does not need to check all possible choices before rejecting a sentence as an invalid one. This is done by a **parser**. A parser creates a *parse tree* based on the grammar rules to determine the validity of a sentence. Figure 18.12 shows the parse tree for the sentence 'John has a white dog'. based on our rules defined in Table 18.1.

Figure 18.12 *Parsing a sentence*



Semantic analysis

The **semantic analysis** extracts the meaning of a sentence after it has been syntactically analyzed. This analysis creates a representation of the objects involved in the sentence, their relations, and their attributes. The analysis can use any of the knowledge representation schemes we discussed before. For example, the sentence 'John has a dog' can be represented using predicate logic as:

$$\exists\, x \text{ dog}(x) \text{ has (John, } x)$$

Pragmatic analysis

The three previous steps—speech recognition, syntax analysis, and semantic analysis—can create a knowledge representation of a spoken sentence. In most cases, another step, **pragmatic analysis**, is needed to further clarify the purpose of the sentence and to remove ambiguities.

Purpose

The purpose of the sentence cannot be found using the three steps listed above. For example, the sentence 'Can you swim a mile?' asks about the ability of hearer. However, the sentence 'Can you pass the salt?' is merely a polite request. An English language sentence can have many different purposes, such as informing, requesting, promising, inquiring, and so on. Pragmatic analysis is required to find the purpose of the sentence.

Removing ambiguity

Sometimes a sentence is ambiguous after semantic analysis. Ambiguity can manifest itself in different ways. A word can have more than one function—for example, the word 'hard' can be used both as an adjective and an adverb. A word can also have more than one meaning—for example, the word 'ball' can mean different things in 'football' and 'ball room'. Two words with the same pronunciation can have different spellings and meanings—a sentence may be syntactically correct, but be nonsense. For example, the sentence 'John ate the mountain' can be syntactically parsed as a valid sentence and be correctly analyzed by the semantic analyzer, but it is still nonsense. Another purpose of the pragmatic analyzer is to remove ambiguities from the knowledge representation sentence if possible.

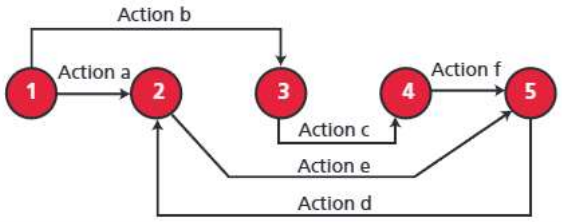
https://sanet.st/blogs/polatebooks/

18.5 SEARCHING

One of the techniques for solving problems in artificial intelligence is *searching*, which is discussed briefly in this section. Searching can be described as solving a problem using a set of states (a situation). A search procedure starts from an initial state, goes through intermediate states until finally reaching a target state. For example, in solving a puzzle, the initial state is the unsolved puzzle, the intermediate states are the steps taken to solve the puzzle, and the target state is the situation in which the puzzle is solved. The set of all states used by a searching process is referred to as the **search space**.

Figure 18.13 shows an example of a search space with five states. Any of the states can be the initial or the target space. The directed lines show how one can go from one state to another by taking the appropriate action. Note that it may not be possible to go from one state to another if there is no action or series of actions to do so.

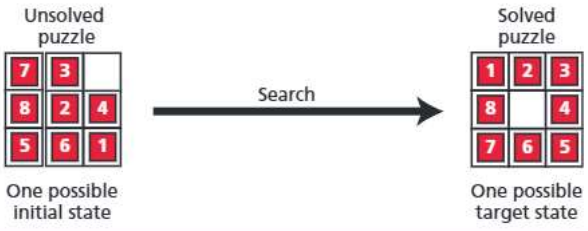
Figure 18.13 *An example of a search space*



Example 18.6

One example of a puzzle that shows the search space is the famous 8-puzzle. The puzzle is contained in a tray that can be thought of as a grid of nine squares. The tray contains only eight tiles, which means that one of the grids is always empty. The tiles are numbered from 1 to 8. Given an initial random arrangement of the tiles (the initial state), the goal is to rearrange the tiles until an ordered arrangement of the tiles is reached (the target state). The rule of the game is that a tile can be slid into an empty slot. Figure 18.14 shows an instance of the initial and the target states.

Figure 18.14 *The initial and possible states for Example 18.6*



18.5.1 Search methods

There are two general search methods: *brute-force* and *heuristic*. The brute-force method is itself either breadth-first or depth-first.

Brute-force search

We use **brute-force search** if we do not have any prior knowledge about the search. For example, consider the steps required to find our way through the maze in Figure 18.15 with points A and T as starting and finishing points respectively. The tree diagram for the maze is shown in Figure 18.16.

Figure 18.15 *A maze used to show brute-force search*

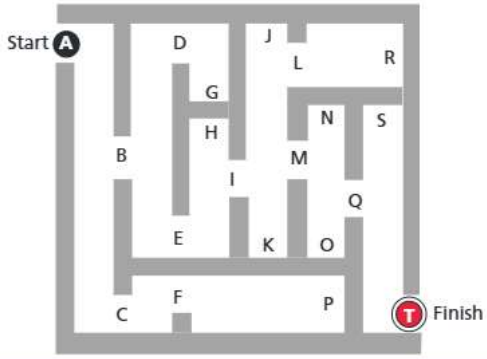
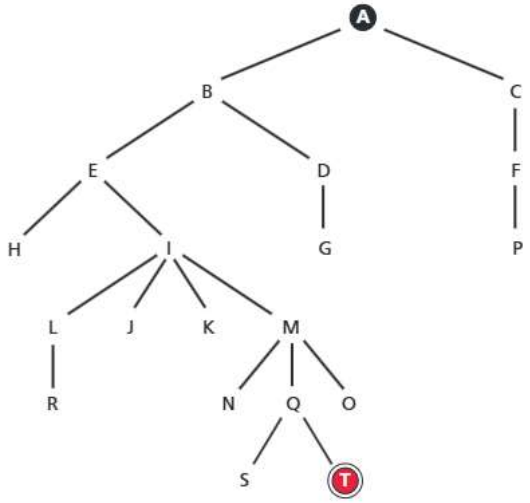


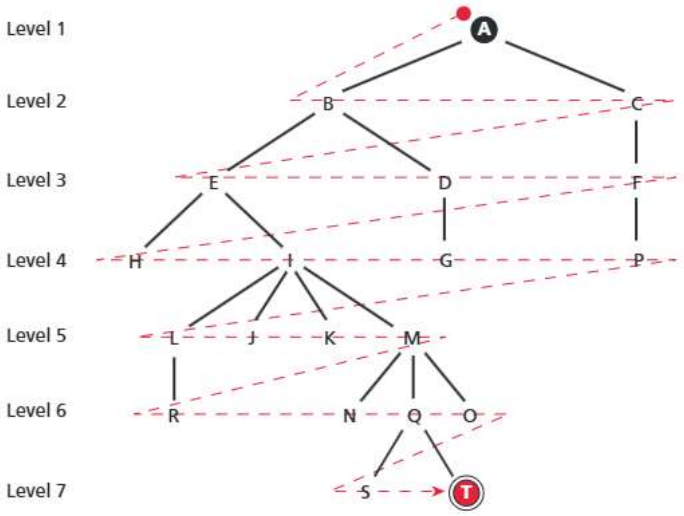
Figure 18.16 *The tree for the maze in Figure 18.15*



Breadth-first search

In this method we start from the root of the tree and examine all the nodes at each level before we move to the next level. The breadth-first search from left to right for the maze is shown in Figure 18.17. Note that we have to search all nodes before we reach the target state, so the method is very inefficient. If we search from right to left, the number of nodes we need to search may be different.

Figure 18.17 *Breadth-first search of the tree in Figure 18.16*



Depth-first search

In this method we start from the root of the tree and do a forward search until we hit the goal or arrive at a dead-end. If we hit a dead-end, we backtrack to the nearest branch and do a forward search again. We continue this process until we reach the goal (see Figure 18.18).

Figure 18.18 *Depth-first search of the tree in Figure 18.16*

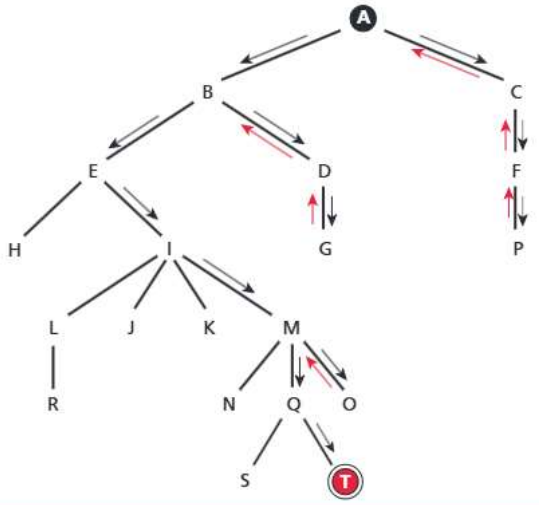


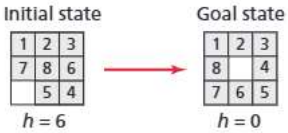
Figure 18.18 shows the depth-first search from the right for the maze shown in Figure 18.15. The search starts at the root node. The search path ACFP comes to a dead-end, so we

backtrack to A and continue the search along the path ABDG, which also comes to a dead-end. Backtracking to node B and searching along path BEIMO results in another dead-end. Backtracking to M and searching along the path MQT, we hit the goal. Note that this method is more efficient than the breadth-first method (Figure 18.17) for the maze problem.

Heuristic search

Using **heuristic search**, we assign a quantitative value called a *heuristic value* (h value) to each node. This quantitative value shows the relative closeness of the node to the goal state. For example, consider solving the 8-puzzle of Figure 18.19.

Figure 18.19 *Initial and goal states for heuristic search*



Assume the initial and goal states of the puzzle are as shown. The heuristic value for each tile is the minimum number of movements the tile must make to come to the goal state. The heuristic value for each state is the sum of the heuristic values of the tiles in that state.

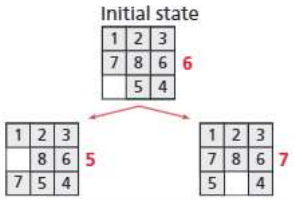
Table 18.2 shows the heuristic value for the initial and final state of the puzzle.

Table 18.2 Heuristic value

Tile number	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*Total*
Heuristic value of initial state	0	0	0	1	1	2	1	1	6
Heuristic value of goal state	0	0	0	0	0	0	0	0	0

To start the search, we consider all possible states of the next level and their corresponding heuristic values. For our puzzle a single move results only in two possible states, with the h values shown in Figure 18.20.

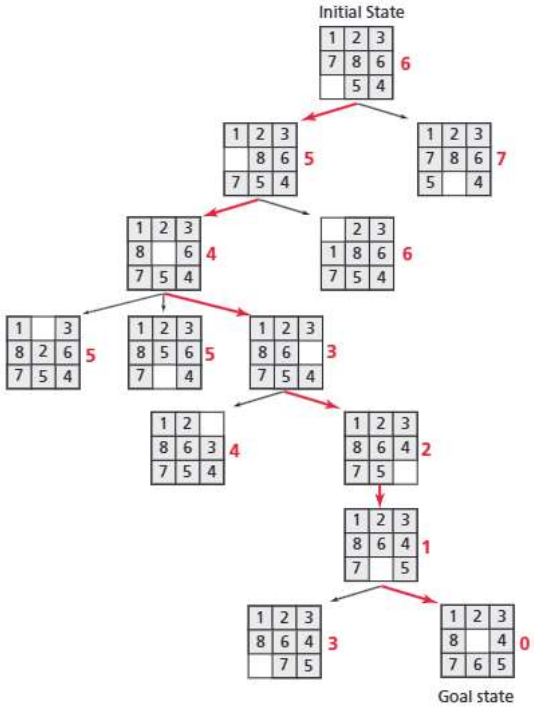
Figure 18.20 *The heuristic values for the first step*



Next we start with the state with the smaller h value and draw the possible states of the next level. We continue this way until we come to the state with an h value of zero (the

goal state), as shown in Figure 18.21. The route to the puzzle's solution is that with bold arrows.

Figure 18.21 *Heuristic search for solving the 8-puzzle*



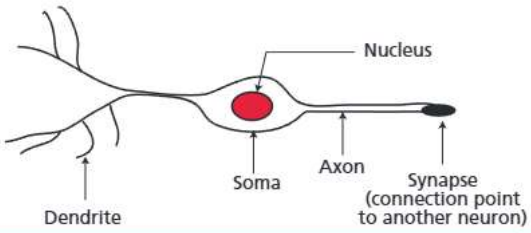
18.6 NEURAL NETWORKS

If an intelligent agent is supposed to behave like a human being, it may need to learn. Learning is a complex biological phenomenon that is not even totally understood in humans. Enabling an artificial intelligence agent to learn is definitely not an easy task. However, several methods have been used in the past that create hope for the future. Most of the methods use *inductive learning* or *learning by example*. This means that a large set of problems and their solutions are given to the machine from which to learn. In this section we discuss only one of these methods, which can be described without complex mathematical concepts: **neural networks**. Neural networks try to simulate the learning process of the human brain using a network of neurons.

18.6.1 Biological neurons

The human brain has billions of processing units, called **neurons**. Each neuron, on average, is connected to several thousand other neurons. A neuron is made of three parts: **soma**, **axon**, and **dendrites**, as shown in Figure 18.22.

Figure 18.22 *A simplified diagram of a neuron*



The soma (body) holds the nucleus of the cell: it is the processor. The dendrites act as input devices: each dendrite receives input from another neuron. The axon acts as an output device: it sends the output to other neurons. The **synapse** is the connecting point between the axon of a neuron and dendrites of other neurons. The dendrites collect electrical signals from the neighboring neurons and pass them to the soma. The job of the synapse is to apply a weight to the signal that passes to the neighboring neuron: it acts as strong or weak connection based on the amount of chemical material it produces.

A neuron can be in one of two states: *excited* or *inhibited*. If the sum of the received signals reaches a threshold, the body is excited and *fires* an output signal that passes to the axon and eventually to other neurons. If the sum of the received signals does not reach the threshold, the neuron remains in the inhibited state: it does not fire or produce an output.

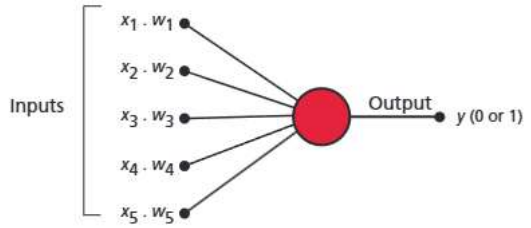
18.6.2 Perceptrons

A **perceptron** is an artificial neuron similar to a single biological neuron. It takes a set of weighted inputs, sums the inputs, and compares the result with a threshold value. If the result is above the threshold value, the perceptron fires, otherwise, it does not. When a perceptron fires, the output is 1: when it does not fire, the output is zero. Figure 18.23 shows a perceptron with five inputs (x_1 to x_5), and five weights (w_1 to w_5). In this perceptron, if T is the value of the threshold, the value of output is determined as:

$$S = (x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 + x_4 \cdot w_3 + x_5 \cdot w_5)$$

If $S > T$, then $y = 1$; else $y = 0$

Figure 18.23 *A perceptron*



Example 18.7

Assume a case study with three inputs and one output. There are already four examples with known inputs and outputs, as shown in the following table:

Inputs			Output
1	0	0	0
0	0	1	0
1	0	1	0
1	1	1	1

This set of inputs is used to train a perceptron with all equal weights ($w_1 = w_2 = w_3$). The threshold is set to 0.8. The original weight for all inputs is 50 per cent. The weights remain the same if the output produced is correct—that is, matches the actual output. The weights are increased by 10 per cent if the output produced is less than the output data: the weights are decreased by 10 per cent if the output produced is greater than the output data. The following table shows the process of applying the previous established examples to train the perceptron:

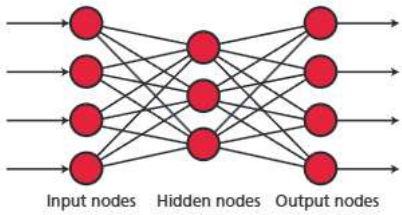
Inputs			Weight	Weighted sum	Output produced	Actual output	Action
1	0	0	50%	0.5	0	0	None
0	0	1	50%	1	1	0	Decrease
1	0	1	40%	8.0	0	0	None
1	1	1	40%	1.2	1	1	None

Note that the perceptron has been trained even with a small set of available data. In a real situation, a perceptron would be trained with a much larger set of data (100 or 1000). After training, it is ready to accept new input data and produce acceptably correct output.

18.6.3 Multilayer networks

Several layers of perceptions can be combined to create multilayer neural networks. The output from each layer becomes the input to the next layer. The first layer is called the *input* layer, the middle layers are called the *hidden* layers, and the last layer is called the *output* layer. The nodes in the input layer are not neurons, they are only distributors. The hidden nodes are normally used to impose the weight on the output from the previous layer. Figure 18.24 shows an example of a neural network with three layers.

Figure 18.24 *A multilayer neural network*



18.6.4 Applications

Neural networks can be used when enough pre-established inputs and outputs exist to train the network. Two areas in which neural networks have proved to be useful are optical character recognition (OCR), in which the intelligent agent is supposed to read any handwriting, and credit assignment, where different factors can be weighted to establish a credit rating, for example for a loan applicant.

18.7 END-CHAPTER MATERIALS

18.7.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Cawsey, A. *The Essence of Artificial Intelligence*, Upper Saddle River, NJ: Prentice-Hall, 1998
- Luger, G. *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, Reading, MA: Addison-Wesley, 2004
- Winston, P. *Artificial Intelligence*, Reading, MA: Addison-Wesley, 1993
- Coppin, B. *Artificial Intelligence Illuminated*, Sudbury, MA: Jones and Bartlett, 2004
- Russel, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*, Upper Saddle River, NJ: Prentice-Hall, 2003
- Dean, T. *Artificial Intelligence: Theory and Practice*, Redwood City, Reading, MA: Addison-Wesley, 2002

18.7.2 Key terms

artificial intelligence 474	pragmatic analysis 493
axon 498	predicate logic 477
brute-force search 495	PROLOG 475

default logic 482	propositional logic 477
dendrite 498	quantifier 481
edge detection 488	rule-based system 483
expert system 485	search space 494
frames 476	segmentation 489
heuristic search 497	semantic analysis 493
high-order logic 482	semantic network 476
image processing 488	software agent 474
intelligent agent 474	soma 498
LISP 475	speech recognition 491
modal logic 482	synapse 499
neural network 498	syntactic analysis 491
neuron 498	temporal logic 482
parser 492	thresholding 489
perceptron 499	Turing test 474
physical agent 475	

18.7.3 Summary

- Artificial intelligence is the study of programmed systems that can simulate, to some extent, human activities such as perceiving, thinking, learning, and acting. One way to define artificial intelligence is the Turing Test, which compares the intelligent behavior of a human being with that of a computer.
- An intelligent agent is a system that perceives its environment, learns from it, and interacts with it intelligently. Intelligent agents can be divided into two broad categories: software agents and physical agents.
- Although some all-purpose languages such as C, C++, and Java are used to create intelligent software, two languages are specifically designed for AI: LISP and PROLOG.
- Knowledge representation is the first step in creating an artificial agent. We discussed four common methods for representing knowledge: semantic networks, frames, predicate logic, and rule-based system. A semantic network uses a directed graph to represent knowledge. Frames are closely related to semantic networks, in which data structures (records) are used to represent the same knowledge. Predicate logic can represent a well-defined language developed during a long history of theoretical logic. A ruled-based system represents knowledge using a set of rules that can be used to deduce new facts from known facts.
- One of the goals of AI is to create expert systems to do tasks that normally need human expertise. It can be used in situations in which that expertise is in short supply, expensive, or unavailable.
- Another goal of AI is to create a machine that behaves like an ordinary human. The first part of this goal involves image processing or computer vision, which is an area of

5/1/25, 9:57 PM study viewer

AI that deals with the perception of objects. The second part of this goal is language processing, analyzing and interpreting a natural language.

- In artificial intelligence, one of the techniques for solving problems is searching. Searching can be described as solving a problem using a set of states (situations). Two broad categories of searching are brute-force search and heuristic search.
- If an intelligent agent is supposed to behave like a human being, it may need to learn. Several methods have been used that create hope for the future. Most of the methods use inductive learning or learning by example. One common method involves the use of neural networks that try to simulate the learning process of the human brain using a networks of neurons.

18.8 PRACTICE SET

18.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

18.8.2 Review questions

Q18-1. Describe the Turing test. Do you think this test can be used to define an intelligent system accurately?

Q18-2. Define an intelligent systems and list two broad categories of agents.

Q18-3. Compare and contrast LISP and PROLOG when they are used in artificial intelligence.

Q18-4. Describe the need for knowledge representation and list four different methods discussed in this chapter.

Q18-5. Compare and contrast predicate logic and propositional logic.

Q18-6. Compare and contrast frames and semantic networks.

Q18-7. Define a rule-base system and compare it with semantic networks.

Q18-8. Compare and contrast expert systems and mundane systems.

Q18-9. List different steps in image processing.

Q18-10. List different steps in language processing.

Q18-11. Define a neural network and how it can simulate the learning process in human beings.

Q18-12. Define a perceptron.

18.8.3 Problems

P18-1. Draw a semantic network to show the relations between the following: medical doctor, family practitioner, gynecologist, intern, engineer, accountant, Dr. Pascal who is a French family practitioner.

P18-2. Represent the semantic network of Problem P18-1 as a set of frames.

https://studylib.net/doc/26059254/foundations-computer-science-4th

P18-3. Using the symbol *R* for the sentence 'It is raining' and the symbol *S* for the sentence 'It is sunny', write each of the following English sentences in propositional logic:

a. It is not raining.
b. It is not sunny.
c. It is neither raining nor sunny.
d. It is raining and sunny.
e. If it is sunny, then it is not raining.
f. If it is raining, then it is not sunny.
g. It is sunny if and only if it is not raining.
h. It is not true that if it is not raining, it is sunny.

P18-4. If the symbols C, *W*, and *H* mean 'it is cold', 'it is warm', and 'it is hot', write the English statements corresponding to the following statements in propositional logic:

a. $\neg H$
b. $W \vee H$
c. $W \wedge H$
d. $W \wedge (\neg H)$
e. $\neg (W \wedge H)$
f. $W \rightarrow H$
g. $(\neg C) \rightarrow W$
h. $\neg (W \rightarrow H)$
i. $H \rightarrow (\neg W)$
j. $((\neg C) \wedge H) \vee (C \vee (\neg H))$

P18-5. Using the symbols *Wh*, *Re*, *Gr*, and *Fl* for the predicates 'is white', 'is red', 'is green', and 'is a flower' respectively, write the following sentences in predicate logic:

a. Some flowers are white.
b. Some flowers are not red.
c. Not all flowers are red.
d. Some flowers are either red or white.
e. There is not a green flower.
f. No flowers are green.
g. Some flowers are not white.

P18-6. Using the symbols *Has*, *Loves*, *Dog*, and *Cat* for the predicates 'has', 'loves', 'is a dog', and 'is a cat' respectively, write the following sentences in predicate logic:

a. John has a cat.
b. John loves all cats.
c. John loves Anne.

d. Anne loves some dogs.
e. Not everything John loves is a cat.
f. Anne does not like some cats.
g. If John loves a cat, Anne loves it.
h. John loves a cat if and only if Anne loves it.

P18-7. Using the symbols *Expensive*, *Cheap*, *Buys*, and *Sells* for the predicates 'is expensive', 'is cheap', 'buys', and 'sells' respectively, write the following sentences in predicate logic:
a. Everything is expensive.
b. Everything is cheap.
c. Bob buys everything that is cheap.
d. John sells something expensive.
e. Not everything is expensive.
f. Not everything is cheap.
g. If something is cheap, then it is not expensive.

P18-8. Using the symbols *Identical* for the predicate 'is identical to', write the following sentences in predicate logic. Note that the predicate 'equal' needs two arguments:
a. John is not Anne.
b. John exists.
c. Anne does not exist.
d. Something exists.
e. Nothing exists.
f. There are at least two things.

P18-9. Use a truth table to find whether the following argument is valid:

$$\{P \rightarrow Q, P\} \vdash Q$$

P18-10. Use a truth table to find whether the following argument is valid:

$$\{P \vee Q, P\} \vdash Q$$

P18-11. Use a truth table to find whether the following argument is valid:

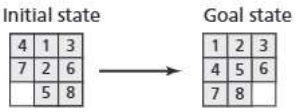
$$\{P \wedge Q, P\} \vdash Q$$

P18-12. Use a truth table to find whether the following argument is valid:

$$\{P \rightarrow Q, Q \rightarrow R\} \vdash (P \rightarrow R)$$

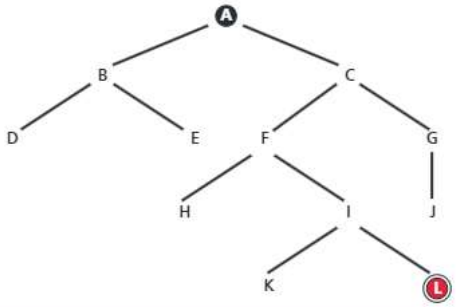
P18-13. Draw a neural network that can simulate an OR gate.
P18-14. Draw a neural network that can simulate an AND gate.
P18-15. The initial and goal states of an 8-puzzle are shown in Figure 18.25. Draw the heuristic search tree for solving the puzzle.

Figure 18.25 *Problem P18-15.*



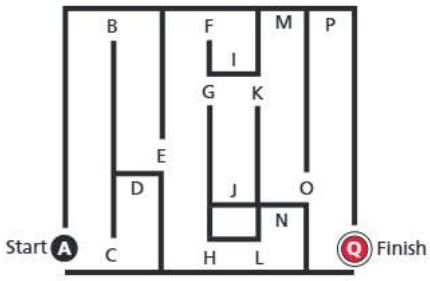
P18-16. Show the breadth-first search for the tree diagram shown in Figure 18.26.

Figure 18.26 *Problem P18-16.*



P18-17. Show the depth-first search for the tree diagram of Problem P18-16.
P18-18. Draw the tree diagram for the maze shown in Figure 18.27.

Figure 18.27 *Problem P18-18.*



P18-19. Draw the tree and show a breadth-first search for Problem P18-18.
P18-20. Draw the tree and show a depth-first search for Problem P18-18.

CHAPTER 19

Introduction to Social Media

In this chapter, we briefly touch on **social media** as one of the applications of computer science. Our goal is not to show how to use social media; many students know and use social media in their daily lives. Our goal is primarily to show the concepts behind social media and how these websites are designed for this purpose. We will discuss only two, **Facebook** and **Twitter**, each as an example of a particular social media type.

Objectives

After studying this chapter, the student should be able to

- Define the *friendship* relationship in Facebook.
- Define the two-way relationship between friends in Facebook.
- Understand the communication channels in Facebook.
- Know how to become a member and how to terminate membership in Facebook.
- Know how to log in and log out of Facebook.
- Know how to find friends in Facebook.
- Know how to communicate with friends in Facebook.
- Define the *following* relationship in Twitter.
- Define the one-way relationship between members and followers in Twitter.
- Understand the communication channels in Twitter.
- Know how to become a member and how to terminate membership in Twitter.
- Know how to log in and log out of Twitter.
- Know how to follow other members in Twitter.
- Know how to communicate with followers in Twitter.

19.1 INTRODUCTION

In the last century, when computer science started as a discipline, we could never have imagined that it would become a part of our daily life in such a short period of time. One of the areas in which computer science has helped all citizens is with the advent of *social media*. Today most people around the world are, to a greater or lesser extent, involved with one or more types of social media, which can be considered a computer science application. Social media is in fact the result of using several computer science disciplines including operating systems, computer programming, computer networks, and databases.

Social media platforms are websites on a large scale, designed to let people exchange their ideas, opinions and experiences. Some are designed primary for exchanging messages or pictures; some are designed to let job seekers find employers, and employers find employees.

A discussion of all types of social media would take a book by itself. In this chapter, therefore, we look at only two of them: Facebook and Twitter. We have chosen these two because the concepts behind Facebook or Twitter are duplicated, more or less, in several other sites.

19.2 FACEBOOK

Facebook is a **social media** that allows families and friends around the world to keep in touch with each other and share thoughts, pictures, comments, and so on.

Facebook allows members to share thoughts, pictures and comments.

19.2.1 General idea

Before explaining how to use Facebook, let us consider the general ideas behind it.

Friendship

In Facebook, sharing is done only between **friends**. **Friendship** is a one-to-one reciprocal relationship. If John is a friend of Lucie, Lucie is also a friend of John. However, the relationship does not propagate; if John is a friend of Lucie and Lucie is a friend of Ann, it does not necessarily mean that John is a friend of Ann. To be so, John or Ann needs to request friendship from the other.

In Facebook, a friend of a friend is not necessarily a friend.

Although there are billions of members in Facebook, we will assume for the purposes of this discussion that it has only eight members (Ml to M8), as shown in Figure 19.1.

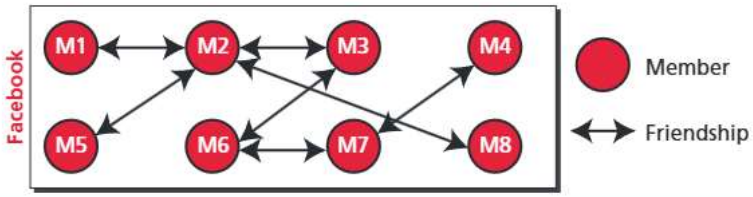
Figure 19.1 *A Facebook site with only eight members*

Now assume that the following requests for friendship are made and are all accepted by the other members:

1. Ml requests friendship from M2.
2. M2 requests friendship from M5.
3. M3 requests friendship from M2.
4. M4 requests friendship from M7.
5. M6 requests friendship from M3.
6. M7 requests friendship from M6.
7. M8 requests friendship from M2.

Figure 19.2 shows the friendship relationships between the eight members.

Figure 19.2 *Relationship between members after friendship requests accepted*



Note that Facebook holds the information about each member, but adds a link between two friends after the friendship has been requested and approved.

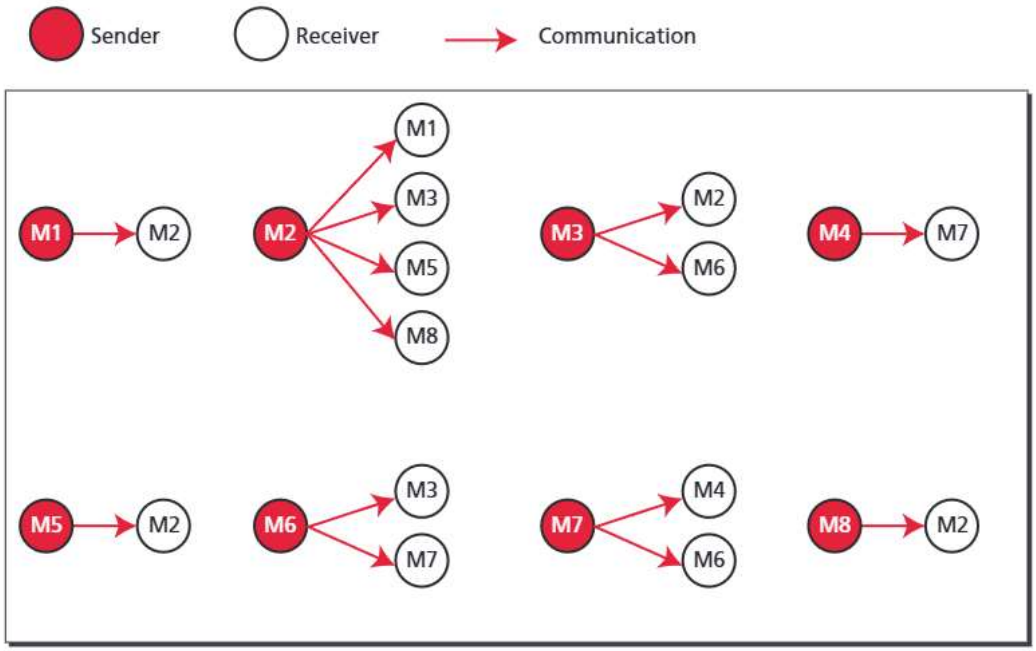
Communication

Although friendship between members is a two-way relationship, we can think of communication as a one-way (one-to-many) relationship: one member posts something, and all their friends can see it, as shown in Figure 19.3:

a. What is posted by Ml can be seen by M2.
b. What is posted by M2 can be seen by Ml, M3, M5, and M8.

c. What is posted by M3 can be seen by M2 and M6.
d. What is posted by M4 can be seen by M7.
e. What is posted by M5 can be seen M2.
f. What is posted by M6 can be seen by M3 and M7.
g. What is posted by M7 can be seen by M4 and M6.
h. What is posted by M8 can be seen by M2.

Figure 19.3 *One-to-many communication in Facebook*



19.2.2 Web pages

Facebook uses several pages, but the two used most often are the home page and the general page. Let us show these two pages before explaining how to use them.

Home page

The **home page** is used only for sign-up (enrolling) and log-in (accessing) Facebook. The general format is shown in Figure 19.4.

User page

The **user page** is the main page you will use on Facebook. The page is fairly complex, but the general format is shown in Figure 19.5. Note that your page always has a toolbar as

the first row. It has three columns, but the two columns on the left can be scrolled to show more options.

19.2.3 Membership

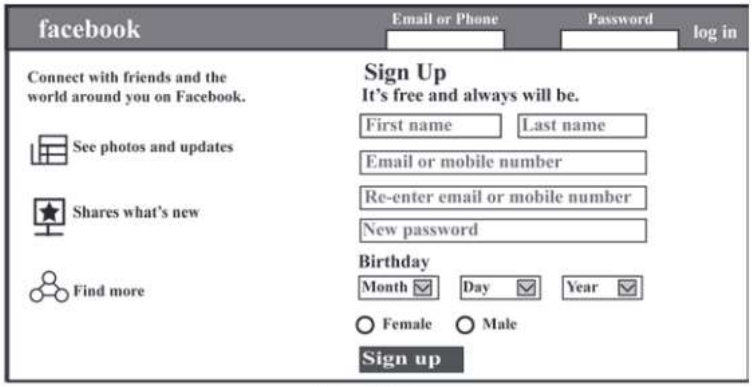
You need to be a member of Facebook to use it. To become a member, you need to sign up. To terminate your membership, you need to **sign out** or deactivate your account.

Sign up

To become a member of Facebook (which is free), you need to go to the Facebook home page (www.facebook.com) as shown in Figure 19.4. You can then use the following steps to sign up.

1. Ignore the *log-in* section (first row) and go to the *sign-up* section.
2. Enter your first and last name in the corresponding boxes.
3. Enter your email or mobile number in the corresponding box.
4. Re-enter your email or mobile number in the corresponding box.
5. Select your data of birth (Month, Day, Year).
6. Check your gender button.
7. Click on the *sign-up* button.

Figure 19.4 *Facebook home page*



Sign out (deactivation)

To permanently deactivate your account in Facebook, go to your home page (Figure 19.5) and do the following:

1. Click the down arrow on the far right of the tool bar.
2. In the submenu that appears, click on *Settings*.

3. Click on *Security*.
4. Choose *Deactivate your account* and follow the steps to confirm it.

Figure 19.5 *Facebook user page*

19.2.4 Accessing services of Facebook

Even if you have signed up as a member, whenever you want to use Facebook, you need to *log in*. When you do not want to use it for a while, you can *log out*.

Log in

To log in into your account, go to the Facebook home page at www.facebook.com (Figure 19.5). In the first row, type your email or mobile number, type your password, and click on the *log-in* button. You will see your page, which means you can now use Facebook.

Log out

Whenever you are not using your Facebook page for a while, you can log out. On the toolbar of user page (your page) (Figure 19.5) click on the down arrow to see the menu. Then click on the *log-out* button.

19.2.5 Friends

As described above, the whole idea of using Facebook is to keep in touch with *friends*. If you post something on the Facebook site, you need to have friends to see it. If you want to see what another member posts, you have to be one of her friends. In other words, you need to find friends and let other members find you as a friend before you can start to communicate through Facebook.

Finding friends

There are several ways of finding friends in Facebook.

Accept Facebook recommendation

You can accept a Facebook recommendation, based on the information you provided during sign-up. To accept a recommendation, go to the toolbar of your page (Figure 19.5) and click on the *find friend* button, which displays a list of people whom you may know.

The list is divided into several categories (eg people you may know, mutual friends, people from your home town, current city, high school, college or university, employer etc) that you can scroll through to choose from. In each category, you can select people with whom you want to be friends and click the *add friend* button in front of their name.

Follow email contacts

You can follow people who regularly contact you by email. On your home page toolbar (Figure 19.5) click on the *friend* icon (two faces next to *find friend* button). Facebook displays a page with different email icons. Choose the appropriate one and type your email and password to see a list of individuals who communicate with you by email. If you want someone on this list to be your Facebook friend, click on the *add friend* button.

Find people you know

At the left side of your home page toolbar (Figure 19.5) type the member name and click on the search icon. Facebook shows the list of members with that name. If you find someone you know, click on the name to see their profile page. If you want that person to be your Facebook friend, click on the *add friend* button.

Accept friendship from other members

Other members may want to add you as their friends. In this case, you may accept or decline friendship when you receive the request. On the toolbar, click on the *friend request* button. You will see the profile for every member who has sent you a request to be your friend. You can click the *confirm* or *decline* button in front of the member name to accept or reject the invitation.

Unfriend a friend

You can remove any member from your friends list at any time. To do so, click on your profile picture on your home page toolbar (Figure 19.5). Click the *friends* button under your name to see the list of your friends. Scroll down until you see the name of a member that you want to unfriend. Then click on the *unfriend* button.

19.2.6 Exchanging information

The whole purpose of Facebook is to allow friends to exchange news (texts, photos, videos etc). To receive your friends' posts, go to your *news feed* where they will appear. To send news to your friends, you need to *update your status*. We will briefly show these two activities.

Reading news

To see what news your friends have posted, go the toolbar (Figure 19.5) and click on the *home* button. You will see all the new posts made by your friends. Each post includes the name of your friend, the date it was posted, and the contents. They may also contain links to web pages. Posted photos will also appear there, but you may want to make them larger by clicking on them. If there are videos in the posts, a thumbnail appears in the body of the news feed with the *play* arrow; click it to play the video.

Commenting on posted updates

At the top of each item of posted news (called an update), there is a *like* button that you can click on to show that you like the post.

Sharing posted updates

Click the *share* button underneath the original post to see a new window. Enter any comments you want to make about the post in the new window and then click the *share* button in the new window. In this way, you can post what you have received to your own friends.

Posting news

Facebook allows you to post news (called updates) for your friends. This can be a long message (up to 60000 characters), a link to a web page, a photo, or a video.

Posting news

To post news, go to the toolbar (Figure 19.5) and click on the *home* button to see the posting window. The *update status* will be selected by default. Click on the *what's on your mind* button and type your news. Then click the *post* button to post the news for your friends.

Posting photos or videos

In the posting window (see Posting News paragraph), click on the *add photos/video* button and then select the right-hand button to post your photos or videos.

Tagging

If you want to mention a friend in your post, you can *tag* that friend. To do so, you need to click on the *tag* button (picture of a head) and then select the name of the friend from the list.

Limiting who can see your post

Generally, when you post something on Facebook, all Facebook members can see it. You can restrict this just to your friends or even just to yourself. In the *status update* window (see Posting News paragraph), click the button *public* to let anyone see the post, click the *friends* button so that only your friends see the post, or click the *only me* button so that no one except you sees the post.

19.3 TWITTER

Twitter is a social network that allows members to post a short message, called a **tweet**, of a maximum 140 characters, for their followers to see.

Twitter allows members to post a tweet for their followers.

19.3.1 General idea

Before showing how to use Twitter, let us discuss the general idea behind this social media site.

Member–followers relationship

In Twitter, the relationship is between members and their followers; a one-to-many relationship. A group of Twitter members follow the member they like; the followed member may not even know who their followers are. This is similar to the relationship between a celebrity and her followers: the followers are interested in what the celebrity does, but the celebrity probably does not know who these followers are.

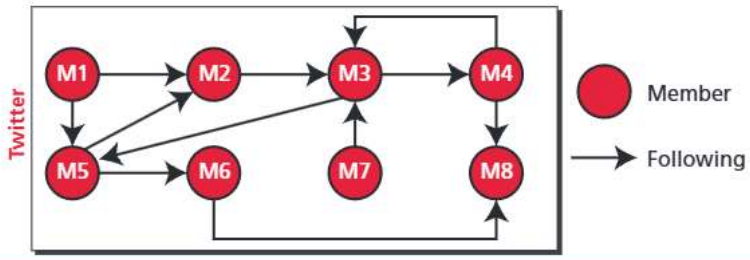
Let us assume that we have eight members registered on the Twitter site as shown in Figure 19.6. Now assume some members decide to follow other members as shown below:

1. M1 follows M2 and M5.
2. M2 follows M3.
3. M3 follows M4 and M5.
4. M4 follows M3 and M8.
5. M5 follows M2 and M6.
6. M6 follows M8.
7. M7 follows M3.
8. M8 follows M7.

Figure 19.6 *Twitter with only eight members registered*

The new one-way relationships are shown in Figure 19.7.

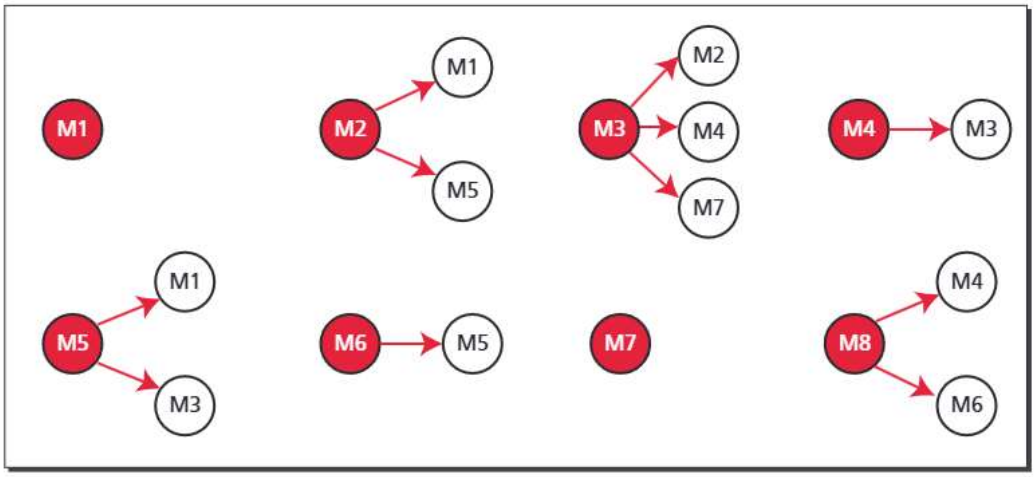
Figure 19.7 *The follower–followed relationship*



Communication

In Twitter, it is also easier to think of communication as a one-to-many relationship. A member posts a tweet for her followers. Figure 19.8 shows this scenario.

Figure 19.8 *One-to-many communication in Twitter*



a. M1 has no followers. This means that what is posted by M1 can be seen by no one.
b. M2 has M1 and M5 as followers. This means that what is posted by M2 can be seen by M1 and M5.
c. M3 has M2, M4, and M7 as followers. This means that what is posted by M3 can be seen by M2, M4, and M7.
d. M4 has M3 as a follower. This means that what is posted by M4 can be seen by M3.
e. M5 has M1 and M3 as followers. This means that what is posted by M5 can be seen by M1 and M3.
f. M6 only has M5 as a follower. This means that what is posted by M6 can be seen only by M5.
g. M7 has no followers. This means that what is posted by M7 can be seen by no one.
h. M8 has M4 and M6 as follower. This means what is posted by M8 can be seen by M4 and M6.

19.3.2 Pages

The Twitter website has several pages, but the two used primarily are the *web page* and the *home page*. Other pages can be reached from one of these two pages. Let us describe these two pages before explaining how to use them.

Web page

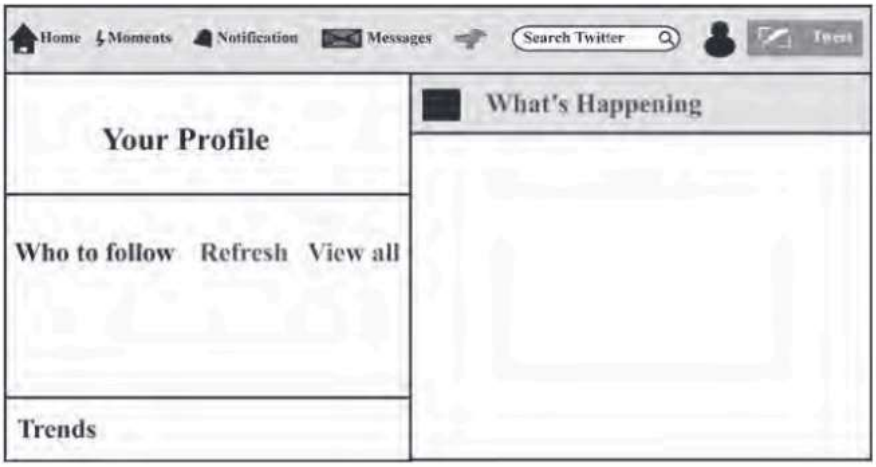
The **web page** is used for joining Twitter for the first time or when you want to log into your account (Figure 19.9)

Figure 19.9 *Twitter web page*

Home page

The home page is used when you have already have an account with Twitter and you want to use it. The general format of this page is shown in Figure 19.10.

Figure 19.10 *Twitter home page*



19.3.3 Membership

As with any social media, you need to sign up to become a member (Figure 19.9). If you want to deactivate your account you need to sign out (Figure 19.10).

Sign up

To become a member of Twitter, go to www.twitter.com to find the Twitter web page, as shown in Figure 19.9. Then click on the *sign-up* button to see the sign-up window as shown in Figure 19.11.

Figure 19.11 *Twitter sign-up window*

In the sign-up window give your full name, your phone number or email, and your password. Now click the *sign-up* button to join as a member.

Sign out (deactivation)

To permanently deactivate your account in Twitter, go to the Twitter home page (Figure 19.10) and do the following:

1. Click the *profile* and *settings* icons.
2. When the new window opens, click on the *settings* and *privacy* buttons.
3. When the new window opens, click on the *deactivate my account* button.

19.3.4 Accessing services of Twitter

Even if you are a member, whenever you want to use Twitter, you need to *log in*. When you do not want to use it for a while, you can *log out*.

Log in

When you are a member, you can log in to your account from any computer or smartphone. On the Twitter web page (Figure 19.9), click on the *log-in* button to see the log-in window as shown in Figure 19.12.

Figure 19.12 *Twitter log-in window*

In the log-in window, give your full name, your phone number or email, and your password. Now click the *log-in* button. You can now use Twitter.

Log out

To log out of your account in Twitter, go to the Twitter home page (Figure 19.10) and do the following:

1. Click the *profile* and *settings* icons.
2. When the new window opens, click on the *log-out* button.

19.3.5 Following and being followed

The whole idea of Twitter is to enable members and their followers to communicate. Members can receive tweets if they follow other members. If members have followers, they can send useful tweets which will be received by their followers.

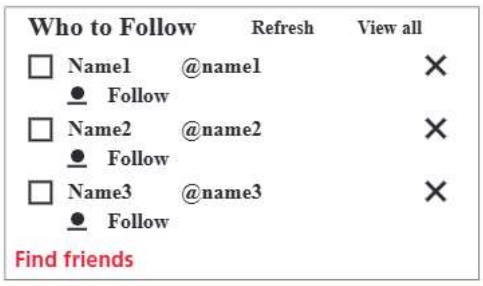
You need to follow other members

If you want to receive tweets from certain members, you need to let Twitter know that you want to follow them. There is no need to get permission from the member you want to follow (unless they explicitly ask Twitter to block you as one of their followers). If you enter the names of the members you want to follow, Twitter creates a list of them in your profile and every time any of these members sends a tweet, you will get a copy. The question is: how do you tell Twitter who you want to follow? There are several ways to do this.

Accept Twitter recommendation

You can accept Twitter's recommendation (based on your past activity and interests). To do this, go to the bottom left section of the Twitter home page, the *who to follow* window, as shown in Figure 19.13 and click the *view-all* button to see Twitter's recommendations. To learn more about any of these members, click on the @name. To follow a member, click the *follow* button.

Figure 19.13 *Who to follow page*



Follow email contacts

You can follow people who regularly contact you by email. On the *who to follow* page (Figure 19.13) click on the *find friends* button (at the bottom of the right-hand column).

A list of email providers appears on the next window, from which you can choose those friends who communicate with you via those email providers.

Search for specific people or organizations

On the *who to follow* section (Figure 19.13) click the *view all* button. When the new window opens, enter either the actual name or the Twitter name of the person or organization you are looking for. Now click the search button. If an individual or organization with that name is a Twitter member, you can choose to follow them.

Stop following a member

You may for some reason want to stop receiving tweets from a member. To do so, you must **unfollow** them. Go to the Twitter home page, click on the **following** link. Move the mouse over the *follow* button for that member and change it to *unfollow* button.

Other members following you

If you want your tweets to be read, you need to have followers to receive them. However, you cannot choose your followers; they have to find you and decide to follow you. They do not need to get your approval to follow you, but if you wish to, you can inform Twitter that you want to block a specific follower.

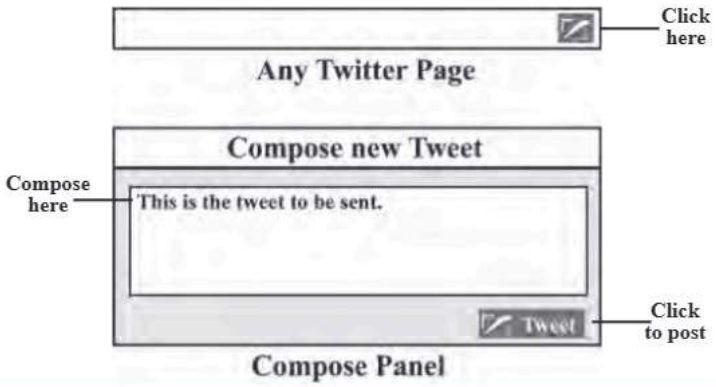
19.3.6 Sending tweets

Although you do not know all your followers personally—they have selected you, you have not selected them—and cannot communicate with them directly, Twitter has a list of your followers. To send a tweet to your followers, just post it on your Twitter page and the site will send it to all your followers.

Compose a new tweet

The first thing you need to know is how to compose a new tweet. This can be done as follows (see Figure 19.14).

Figure 19.14 *Composing a new tweet*



1. Click the *tweet* button on any Twitter web page to see the *compose new tweet* panel.
2. In the *compose new tweet* panel, type your message.
3. Click the *tweet* button at the bottom of the page to post your tweet.
4. Twitter takes your tweet and adds your profile, including your name, at the top of the tweet.
5. Twitter then sends a copy to all your followers.

Referring to other people's tweets

In any tweets you send, you can also refer to the tweets of other users. This can be done in two ways: by using **ampersands** and/or by using **hashtags.**

Using ampersands

When you are sending a tweet to your followers, if you want to refer to the tweets of another user, you can insert the name of that user preceded by an ampersand (@). Any of your followers can click on the name to see tweets posted by that user.

Using hashtags

Sometimes you want to refer to a set of tweets that contain a specific word or phrase originating from different senders. In this case you can use a hashtag (#) in front of the word. The word becomes a *keyword* representing a particular topic or issue, and any click on the word displays all recent tweets that contain that word.

19.3.7 Receiving tweets

After you identify and designate those members that you want to follow, any tweet posted by any one of them comes to your Twitter home page (also called your *Twitter feed*). To get to your home page, click on the *home* icon at the top-left of the Twitter toolbar (Figure 19.10) which enlarges the *what's happening* section. After reading any tweet, you can:

1. click on the arrow to compose a tweet in response to the sender of the tweet.
2. click on the double arrow to **retweet** the received tweet to their own followers.
3. click on the star to show that they like the tweet.

19.4 END-CHAPTER MATERIALS

19.4.1 Recommended reading

For more details about the subjects discussed in this chapter, the following book is recommended:

Russel Matthew A. *Mining the Social Web*, Sebastopol, CA: O'Reilly, 2014

19.4.2 Key terms

Facebook 507	news 514
follow 513	sign out 511
following 520	social media 507
friend 508	tweet 514
friendship 508	Twitter 507
hashtag 521	user page 510
home page 510	web page 517

19.4.3 Summary

- Facebook allows families and friends around the world to keep in touch with each other.
- In Facebook, each member needs to find friends to communicate with, but a friend of a friend is not a friend.
- Communication in Facebook is one-to-many. What is posted by one member can be seen by all their friends.
- To become a member of Facebook, one needs to sign up. To terminate membership, one needs to sign out.
- To use Facebook, a member needs to log in; to stop using Facebook for a while, a member needs to log out.
- Twitter allows a member to post a tweet (small message) for their followers to read.
- In Twitter, each member needs to find other members to follow.
- Communication in Twitter is one-to-many. Everything posted by one member can be seen by all their followers.
- To become a member of Twitter, one needs to sign up. To terminate membership, one needs to sign out.
- To use Twitter, a member needs to log in; to stop using Twitter for a while, a member needs to log out.
- A member needs to follow other members to receive tweets from them.

19.5 PRACTICE SET

19.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

19.5.2 True/false questions

Categorize the following statements as true or false.

T19-1. In Facebook, members post something for their followers.
T19-2. In Facebook, sharing is done between friends.
T19-3. In Facebook, a friend of a friend is a friend.
T19-4. Friendship in Facebook is a one-way relationship.
T19-5. Communication in Facebook is one-way.
T19-6. To sign up in Facebook you need to be on the user page.
T19-7. There is only one way to find friends in Facebook.
T19-8. In Facebook, what is posted by one member can be seen by all their friends.
T19-9. In Twitter communication takes place between a member and his/her friends.
T19-10. To become a member of Twitter, you need to log in.
T19-11. To discontinue membership in Twitter, you need to sign out.
T19-12. Before using Twitter, you need to look for followers.
T19-13. In Twitter, a post (message) can contain thousands of characters.
T19-14. To refer to another person named in your tweet, you need to use an ampersand.
T19-15. To refer to a word in all tweets, you need to use a hashtag.

19.5.3 Review questions

Q19-1. What is the difference between Facebook and Twitter when we consider the relationship between members?
Q19-2. What is the difference between Facebook and Twitter when we are thinking about the size of messages exchanged?
Q19-3. In Twitter, explain why a follower cannot send a message to the member that he/she follows.
Q19-4. In Facebook, if x and y are friends and z is only a friend of x but not a friend of y, can x send a message to z? Can z send a message to x?
Q19-5. In Twitter, if x is a follower of y and y is a follower of z, can x see a tweet sent by z?

19.5.4 Problems

P19-1. In Figure 19.2, assume that M2 and M5 terminate their friendships. Redraw Figure 19.3 for the new situation.
P19-2. In Figure 19.2, assume that M3 and M8 become friends. Redraw Figure 19.3 for the new situation.
P19-3. In Figure 19.7, assume that M2 decided to follow M6. Redraw Figure 19.7 using the new situation.
P19-4. In Figure 19.7, assume that M4 follows M7 and M7 follows M3. Redraw Figure 19.7 using the new situation.

CHAPTER 20

Social and Ethical Issues

In this chapter, we briefly focus on social and ethical issues related to the use of computers and to the Internet as a network of computers.

Objectives

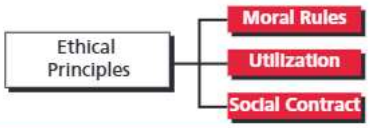
After studying this chapter, the student should be able to:

- Define three ethical principles related to the use of computers.
- Distinguish between physical and intellectual property and list some types of intellectual property.
- Define privacy as related to the use of computers.
- Give the definition of a computer crime and discuss types of attacks, motivation for attacks, and how to protect against attacks.
- Define hackers and the damage done by them.

20.1 ETHICAL PRINCIPLES

One of the ways to evaluate our responsibility towards the rest of the world when using a computer is to base our decisions on **ethics**. Ethics is a very complex subject that would take several books to describe in detail. In this chapter, we discuss only three principles that can be related to our goal, shown in Figure 20.1.

Figure 20.1 *Three main principles of ethics*



20.1.1 Moral rules

The first ethical principle states that when we make an ethical decision, we need to consider if the decision is made in accordance with a universally accepted principle of morality. For example, if we want to illegally access a computer to get some information, we need to ask ourselves if this act is moral. We know that most people in the world do not consider such actions to be moral, which means that we would be ignoring the first principle of ethics if we acted in this way.

The first principle of ethics says that we should avoid doing anything if it is against universal morality.

20.1.2 Utilization

The second theory of ethics is related to the consequences of the act. An act is ethical if it results in consequences which are useful for society. If a person accesses a bank's computer and erases customer records, is this act useful for society? Since this action may damage the financial status of the bank's customer, it is detrimental to society. It does not bring about a good result. It is not ethical.

The second principle of ethics says that an act is ethical if it brings about a good result.

20.1.3 Social contract

The social contract theory says that an act is ethical when a majority of people in society agrees with it. If someone breaks into somebody else's house and commits a robbery, does this act receive the approval of a majority of society? Since the answer is negative, this act is not ethical.

The third principle of ethics says an act is ethical if a majority of people in society agree with it.

20.2 INTELLECTUAL PROPERTY

Most ethical issues in the past were related to physical property. Physical property has been defined and physical property rights have been recognized by society throughout history. If a person has a physical object, such as a computer, the rights to this property are granted to the owner. It has been proved that ignoring physical property rights may affect the three ethical principles discussed above.

Modern societies have gone further and have recognized the right to **intellectual property**. For example, an author should be given the right to benefit from his/her written book. An artist should be given the right to benefit from his/her artwork. However, there are some differences between the two types of property:

1. A physical property cannot be copied; it needs to be manufactured or built. If we need another computer, we need to physically build it. On the other hand, intellectual property can be copied. We can make a copy of a book without rewriting it.
2. Copying intellectual property still leaves the owner with the original. Stealing a computer deprives the owner of its use.
3. The owner of intellectual property is only one person (or a group); many people can have the same physical property.

20.2.1 Types of intellectual property

Modern societies have recognized several types of intellectual property: *trademarks, trade secrets, patents,* and *copyright*.

Trademarks

A **trademark** identifies a company's product or service. A trademark is an intellectual property right granted by the government for a limited term, but which can be renewed. A trademark is considered intellectual property in that the corresponding product cannot legally be copied by other companies or individuals.

Trade secrets

A **trade secret** is the information about a product that is kept secret by the owner. For example, a company can create a product but keep the formula secret. A programmer can create a piece of software, but keep the program code secret. People can use the product or the software, but they do not own the formula or the code. Unlike trademarks, a trade secret does not have to be registered; the owner just has to keep it secret.

Patents

A **patent** is a right to a monopoly to use and commercially exploit a piece of intellectual property for a limited period of time. The owner has the right to give or not to give permission

to anyone who wishes to use the invention. However, the individual property needs to have certain characteristics such as novelty, usefulness, and the capability of being built.

Copyright

A **copyright** is a right to a written or created work. It gives the author the exclusive rights to copy, distribute, and display the work. Copyright arises automatically, and does not need to be applied for or formally registered, but a statement of the creator's copyright should be mentioned somewhere on the work.

20.3 PRIVACY

Today, a large amount of personal information about a citizen is collected by private and public agencies. Although in many cases the collection of this information is necessary, it may also pose some risks. Some of the information collected by government or private companies can be used commercially. In many countries, a citizen's right to **privacy** is, directly or indirectly, mentioned in the nation's constitution. However, there is a conflict between people's right to privacy and the need to collect information about them. Usually governments create a balance between the two through laws. Some countries have introduced codes of ethics related to the use of computers to collect data, as shown below:

1. Collect only data that are needed.
2. Be sure that the collected data are accurate.
3. Allow individuals to know what data have been collected.
4. Allow individuals to correct the collected data if necessary.
5. Be sure that collected data are used only for the original purpose.
6. Use encryption techniques (discussed in Chapter 16) to accomplish private communication.

20.4 COMPUTER CRIMES

For the purposes of this book, we give a simple definition of a computer crime. A computer crime is an illegal act, called an *attack*, involving any of the following:

1. A computer
2. A computer network
3. A computer-related device
4. Software
5. Data stored in a computer
6. Documentation related to the use of computers

20.4.1 Types of attacks

Attacks can be divided into two categories: *penetration* and *denial of service*.

Penetration attack

Penetration in this case means breaking into a system to get access to the data stored in a computer or in a computer network. Penetration can result in changing data directly or injecting **viruses**, **worms**, and **Trojan horses** to alter the data indirectly.

Viruses

Viruses are unwanted programs that are hidden within other programs (host). When the user executes the host program, the virus takes control and replicates itself by attaching itself to other programs. Eventually, the multitude of viruses may stop normal computer operations. Viruses can also be transferred to other machines through the network.

Worms

A worm is an independent program which can copy itself and which travels through the network. It is a self-replicating piece of software that can travel from one node to another. It tries to find weaknesses in the system to inflict harm. It can reproduce many copies of itself, thus slowing down access to the Internet or stopping communication altogether.

Trojan horses

A Trojan horse is a computer program that does perform a legitimate task, but which also contains code to carry out malicious attacks such as deleting or corrupting files. It can also be used to access user passwords or other secret information.

Denial of service attack

The **denial of service** is an attack on a computer connected to the Internet. These attacks reduce the capability of a computer system to function correctly or bring the system down altogether by exhausting its resources.

20.4.2 Motives

Attacks are made with many different motivations such as political reasons, a hacker's personal interpretation of computer ethics, terrorism, espionage, financial gain, or hate.

20.4.3 Attack protection

Although attacks cannot be avoided easily, there are some strategies that can be applied to reduce the number or impact of the attacks. We describe three strategies briefly.

Use physical protection

The computer can be physically protected to allow physical access only to trusted individuals.

Use protective software

Software can be used to protect your data, such as data encryption or the use of strong passwords to access the software.

Install strong anti-virus software

Strong anti-virus software can control access to the computer when installing new software or accessing Internet sites.

20.4.4 Cost

It is obvious that ordinary citizens usually bear the cost of these computer attacks. When private companies spend money on preventing attacks, consumers using their products pay through increased prices. When government organizations spend money to prevent attacks, citizens pay through increased taxes.

20.5 HACKERS

The word **hacker** today has a different meaning than when it was used in the past. Previously, a hacker was a person with a lot of knowledge who could improve a system and increase its capability. Today, a hacker is someone who gains unauthorized access to a computer belonging to someone else in order to copy secret information.

Although some of the infiltrations carried out by hackers can be harmless, most countries impose heavy penalties for both harmless and harmful hacking. In most countries, accessing government computers without authorization is a crime. Moreover, in many countries there is a heavy punishment for hackers who access the computers of private institutions, and the simple act of obtaining information from somebody else's computer is a crime, whether the information is used or not.

20.6 END-CHAPTER MATERIALS

20.6.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Kizza, J. M. *Ethical and Social Issues in the Information Age*, London: Springer, 2010
- Schneider, M and. Gersting, J. L. *Invitation to Computer Science*, 7th edition, Boston, MA: Cengage Learning, 2016
- Reynold, C. and Tymann, P. *Schaum's Outline of Principles of Computer Science*, New York: McGraw-Hill, 2008
- Long, L and Long, N. *Computers*, Upper Saddle River, NJ: Prentice-Hall, 1999

20.6.2 Key terms

This chapter has introduced the following key terms, which are listed here with the pages on which they first occur:

copyright 528	privacy 528
denial of service 529	social contract 526

ethical principle 526	trademark 527
hackers 530	trade secret 527
intellectual property 527	Trojan horse 529
moral rules 526	utilization 526
patent 527	virus 529
penetration attack 529	worm 529

20.6.3 Summary

- One of the ways to evaluate our responsibility towards the rest of the world when using a computer is to base our decisions on ethics using the three principles of moral rules, utilization, and social contract.
- Ethical issues nowadays deal not only with physical property but also with intellectual property.
- Four different types of intellectual property are trademarks, trade secrets, patents, and copyrights.
- One major ethical issue of our time is respect of privacy.
- Computer crime mostly involves penetration or denial of service attacks on computer systems.
- Computer attacks can be avoided using physical protection, protective software, and anti-virus software.
- The term *hacker* today refers to a person or organization that gains unauthorized access to a computer belonging to someone else to copy secret information.

20.7 PRACTICE SET

20.7.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book's website. It is strongly recommended that the student takes the quizzes to check his/her understanding of the materials before continuing with the practice set.

20.7.2 Review questions

Q20-1. What are the three principles of ethics discussed in this chapter?

Q20-2. What are some differences between a physical and an intellectual property?

Q20-3. What are some privacy codes of ethics related to using computers?

Q20-4. What is the difference between penetration and denial of service when talking about computer crimes?

Q20-5. What are the three ways a malicious person can penetrate a computer belonging to someone else?

20.7.3 Problems

P20-1. Assume you have created a piece of software that can be used by many vendors. Is this intellectual property protected by a copyright or a patent? Explain your answer.

P20-2. You have created a piece of software for which you want to keep the code secret. Do you need to register this intellectual property? Explain your answer.

P20-3. If someone collects data about you without informing you in advance, is this act against your right to privacy? Explain your answer.

P20-4. If someone sends you an email that carries a virus, what type of computer crime is committed here? Explain your answer.

P20-5. If someone renders an institution's computer system so busy that it cannot do any more work, what type of computer crime is committed? Explain your answer.

P20-6. Explain the difference between a virus and a worm.

P20-7. Explain the difference between a virus and a Trojan horse.

P20-8. Explain the difference between a worm and a Trojan horse.

P20-9. Describe different ways in which you can protect your computer from attacks.

P20-10. Explain the damage that a hacker can inflict on a computer system of a financial institution.

APPENDIX A

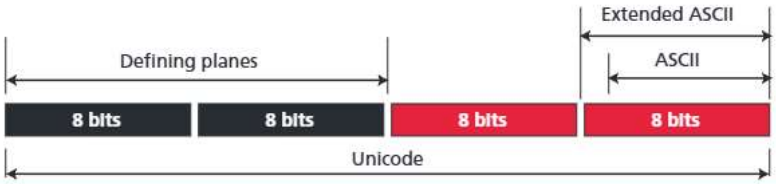
Unicode

Computers use numbers. They store characters by assigning a number to each one. The original coding system was called ASCII (American Standard Code for Information Interchange) and had 128 characters each stored as a 7-bit number. ASCII can satisfactorily handle lowercase and uppercase letters, digits, punctuation characters, and some control characters. An attempt was made to extend the ASCII character set to eight bits. The new code, which was called Extended ASCII, was never internationally standardized.

To overcome the difficulties inherent in ASCII and Extended ASCII—not enough bits to represent characters and other symbols needed for communication in other languages—the Unicode Consortium, a group of multilingual software manufacturers, created a universal encoding system to provide a comprehensive character set, called **Unicode**.

Unicode was originally a 2-byte character set. Unicode version 5, however, is a 4-byte code and is fully compatible with ASCII and Extended ASCII. The ASCII set, which is now called Basic Latin, is Unicode with the upper 25 bits set to zero. Extended ASCII, which is now called Latin-1, is Unicode with the 24 upper bits set to zero. Figure A.1 shows how the different systems are compatible.

Figure A.1 *Unicode compatibility*



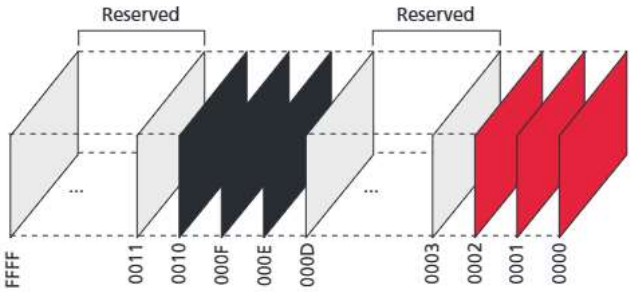
Each character or symbol in Unicode is defined by a 32-bit number. The code can define up to 2^{32} (4 294 967 296) characters or symbols. The description here uses hexadecimal digits in the following format, in which each X is a hexadecimal digit:

U+XXXXXXXX

A.1 PLANES

Unicode divides the whole code space into planes. The most significant 16 bits define the plane, which means we can have 65 536 (2^{16}) planes. For plane 0, the most significant 16 bits are 0s, $(0000)_{16}$, in plane 1 the bits are $(0001)_{16}$, in plane 2 they are $(0002)_{16}$, and so on until in the plane 65 536, they are $(FFFF)_{16}$. Each plane can define up to 65 536 characters or symbols. Figure A.2 shows the structure of Unicode code spaces and its planes.

Figure A.2 *Unicode planes*



0000: Basic Multilingual Plane (BMP)
0001: Supplementary Multilingual Plane (SMP)
0002: Supplementary Ideographic Plane (SIP)
000E: Supplementary Special Plane (SSP)
000F: Private Use Plane (PUP)
0010: Private Use Plane (PUP)

A.1.1 Basic multilingual plane (BMP)

The **basic multilingual plane**, plane 0, is designed to be compatible with the previous 16-bit Unicode. The most significant 16 bits in this plane are all zeros. The codes are normally shown as U+XXXX with the understanding that XXXX defines only the least significant 16 bits. This plane mostly defines character sets in different languages, with the exception of some codes used for control or other special characters (for more information, see the Unicode Web Page).

A.1.2 Other planes

Unicode had other planes:

- The **supplementary multilingual plane**, plane $(0001)_{16}$, is designed to provide more code for multilingual characters that are not included in the BMP plane.
- The **supplementary ideographic plane**, plane $(0002)_{16}$, is designed to provide code for ideographic symbols, any symbol that primarily denotes an idea or meaning in contrast to a sound or pronunciation.

- The **supplementary special plane,** plane $(000E)_{16}$, is used for special characters not found in the Basic Latin or Basic Latin-1 codes.
- **Private use planes,** planes $(000F)_{16}$ and $(0010)_{16}$, are reserved for private use.

A.2 ASCII

Today, ASCII or Basic Latin, is part of Unicode. It occupies the first 128 codes in Unicode (U-00000000 to U-0000007F). Table A.1 contains the hexadecimal codes and symbols. To find the actual code, we prepend $(000000)_{16}$ to the code.

Table A.1 ASCII

Code	*Symbol*	*Code*	*Symbol*	*Code*	*Symbol*	*Code*	*Symbol*
$(00)_{16}$	Null	$(20)_{16}$	Space	$(40)_{16}$	@	$(60)_{16}$	`
$(01)_{16}$	SOH	$(21)_{16}$	!	$(41)_{16}$	A	$(61)_{16}$	a
$(02)_{16}$	STX	$(22)_{16}$	"	$(42)_{16}$	B	$(62)_{16}$	b
$(03)_{16}$	ETX	$(23)_{16}$	#	$(43)_{16}$	C	$(63)_{16}$	c
$(04)_{16}$	EOT	$(24)_{16}$	$	$(44)_{16}$	D	$(64)_{16}$	d
$(05)_{16}$	ENQ	$(25)_{16}$	%	$(45)_{16}$	E	$(65)_{16}$	e
$(06)_{16}$	ACK	$(26)_{16}$	&	$(46)_{16}$	F	$(66)_{16}$	f
$(07)_{16}$	BEL	$(27)_{16}$	'	$(47)_{16}$	G	$(67)_{16}$	g
$(08)_{16}$	BS	$(28)_{16}$	(	$(48)_{16}$	H	$(68)_{16}$	h
$(09)_{16}$	HT	$(29)_{16}$	)	$(49)_{16}$	I	$(69)_{16}$	i
$(0A)_{16}$	LF	$(2A)_{16}$	*	$(4A)_{16}$	J	$(6A)_{16}$	j
$(0B)_{16}$	VT	$(2B)_{16}$	+	$(4B)_{16}$	K	$(6B)_{16}$	k
$(0C)_{16}$	FF	$(2C)_{16}$	,	$(4C)_{16}$	L	$(6C)_{16}$	l
$(0D)_{16}$	CR	$(2D)_{16}$	-	$(4D)_{16}$	M	$(6D)_{16}$	m
$(0E)_{16}$	SO	$(2E)_{16}$	.	$(4E)_{16}$	N	$(6E)_{16}$	n
$(0F)_{16}$	SI	$(2F)_{16}$	/	$(4F)_{16}$	O	$(6F)_{16}$	o
$(10)_{16}$	DLE	$(30)_{16}$	0	$(50)_{16}$	P	$(70)_{16}$	p

Table A.1 ASCII (continued)

Code	*Symbol*	*Code*	*Symbol*	*Code*	*Symbol*	*Code*	*Symbol*
$(11)_{16}$	DC1	$(31)_{16}$	1	$(51)_{16}$	Q	$(71)_{16}$	q
$(12)_{16}$	DC2	$(32)_{16}$	2	$(52)_{16}$	R	$(72)_{16}$	r
$(13)_{16}$	DC3	$(33)_{16}$	3	$(53)_{16}$	S	$(73)_{16}$	s
$(14)_{16}$	DC4	$(34)_{16}$	4	$(54)_{16}$	T	$(74)_{16}$	t
$(15)_{16}$	NAK	$(35)_{16}$	5	$(55)_{16}$	U	$(75)_{16}$	u
$(16)_{16}$	SYN	$(36)_{16}$	6	$(56)_{16}$	V	$(76)_{16}$	v
$(17)_{16}$	ETB	$(37)_{16}$	7	$(57)_{16}$	W	$(77)_{16}$	w
$(18)_{16}$	CAN	$(38)_{16}$	8	$(58)_{16}$	X	$(78)_{16}$	x
$(19)_{16}$	EM	$(39)_{16}$	9	$(59)_{16}$	Y	$(79)_{16}$	y
$(1A)_{16}$	SUB	$(3A)_{16}$	:	$(5A)_{16}$	Z	$(7A)_{16}$	z
$(1B)_{16}$	ESC	$(3B)_{16}$	;	$(5B)_{16}$	[	$(7B)_{16}$	{
$(1C)_{16}$	FS	$(3C)_{16}$	<	$(5C)_{16}$	\	$(7C)_{16}$	|
$(1D)_{16}$	GS	$(3D)_{16}$	=	$(5D)_{16}$	]	$(7D)_{16}$	}
$(1E)_{16}$	RS	$(3E)_{16}$	>	$(5E)_{16}$	^	$(7E)_{16}$	~
$(1F)_{16}$	US	$(3F)_{16}$	?	$(5F)_{16}$	_	$(7F)_{16}$	DEL

A.2.1 Some properties of ASCII

ASCII has some interesting properties that we need to briefly mention here:

1. The first code, $(00)_{16}$, which is non-printable, is the null character. It represents the absence of any character.
2. The last code, $(7F)_{16}$, is the delete character, which is also non-printable. It is used by some programs to delete the current character.
3. The space character, $(20)_{16}$, is a printable character. It prints a blank space.
4. Characters with code $(01)_{16}$ to $(1F)_{16}$ are control characters: they are not printable. Table A.2 shows their functions. Most of these characters were used in data communication in out-of-date protocols.

Table A.2 Explanation for control characters

Symbol	*Explanation*	*Symbol*	*Explanation*
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledgment
ACK	Acknowledgment	SYN	Synchronous idle
BEL	Ring bell	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator

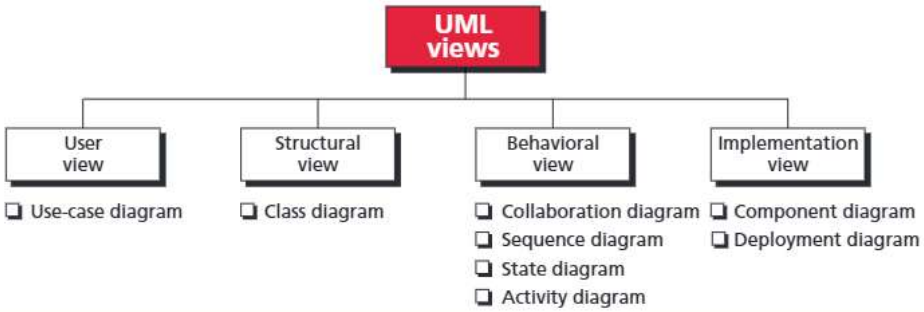
5. The uppercase letters start from $(41)_{16}$. The lowercase letters start from $(61)_{16}$. When numerically compared, uppercase letters are smaller than lowercase ones. This means that when we sort a list based on ASCII values, the uppercase letters show before the lowercase letters.
6. The uppercase and lowercase letters differ by only one bit in the 7-bit code. For example, character *A* is $(41)_{16}$ and character *a* is $(61)_{16}$. The difference is bit 6, which is 0 in uppercase letters and 1 in lowercase letters. If we know the code for one case, we can find the code for the other easily by adding or subtracting, $(20)_{16}$ in hexadecimal or flipping the sixth bit. In other words, the code for character *A* is $(41)_{16} = (1000001)_2$, but the code for character *a* is $(61)_{16} = (1100001)_2$: the sixth bit in binary notation is flipped from 0 to 1.
7. The uppercase letters are not immediately followed by lowercase letters—there are some punctuation characters in between.
8. Decimal digits (0 to 9) begin at $(30)_{16}$. This means that if we want to change a numeric character to its face value as an integer, we need to subtract $(30)_{16} = 48$ from it. For example, the code for 8 in ASCII is $(38)_{16} = 56$. To find the face value, we need to subtract 48 from this, or $56 - 48 = 8$.

APPENDIX B

Unified Modeling Language (UML)

Unified Modeling Language (UML) is a graphical language used for analysis and design. Through UML we can specify, visualize, construct, and document software and hardware systems using standard graphical notations. UML provides different levels of abstraction, called **views**, as shown in Figure B.1.

Figure B.1 *UML views*



As shown in Figure B.1 the four views are:

1. The **user view**, which shows the interaction of the user with the system. This view is represented by use-case diagrams.
2. The **structural view**, which shows the static structure of the system. This view is represented by class diagrams.
3. The **behavioral view**, which shows how the objects in the system behave. This view is represented by collaboration, sequence, state, and activity diagrams.
4. The **implementation view**, which shows how the system is implemented. It contains component and deployment diagrams.

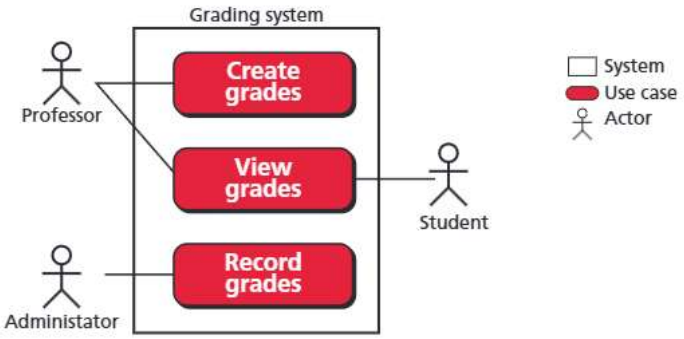
B.1 THE USER VIEW

The user view is a high level view of the whole system. It shows how a system is organized in general. There is only one type of diagram in user views, the use-case diagram.

B.1.1 Use-case diagrams

A project normally starts with a use-case diagram. A **use-case diagram** gives the user's view of a system: it shows how the users communicate with the system. Figure B.2 shows an example of a use-case diagram. A use-case diagram uses four main components: **system**, **use cases**, **actors**, and **relationships**. Each component is explained below.

Figure B.2 *A use-case diagram*



System

A system performs a function. We are interested only in a computer system. The computer system in a use-case diagram is shown by a rectangular box with the name of the system outside the box in the top-left corner.

Use cases

A system contains many actions represented as use cases. Each use case defines one of the actions that can be taken by the users of a system. A use case in a use-case diagram is shown by a rectangle with rounded corners.

Actors

An actor is someone or something that uses the system. Although actors are shown as stick figures, they do not necessarily represent human beings.

Relationships

Relationships are associations between actors and use cases. A relationship is shown as a line connecting actors to use cases. An actor can relate to multiple use cases and a use case can be used by multiple actors.

B.2 THE STRUCTURAL VIEW

The structural view shows the static nature of the system, classes and their relationships. The structural view uses only one type of diagram, class diagrams.

B.2.1 Class diagrams

A **class diagram** manifests the static structure of a system. It shows the characteristics of the classes and the relationship between them. The symbol for a class is a rectangle with the name of the class written inside. Figure B.3 shows three classes, Person, Fraction, and Elevator, belonging to three different systems—that is, there is no relationship between them.

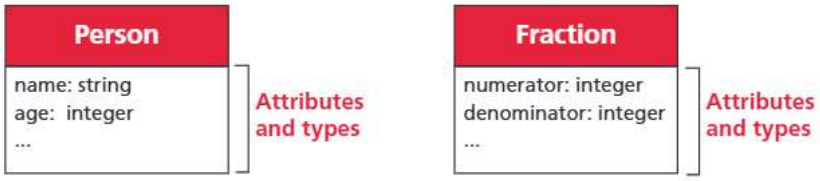
Figure B.3 *Symbol for a class*

Class diagrams are extended by adding attributes, types, and methods to the diagram. Relationships between classes are shown with association and generalization diagrams.

Attributes and types

A class symbol can include attributes and types in a separate compartment. An attribute is a property of a class and a type is the type of data used to represent that attribute. Figure B.4 shows some attributes of the classes Person and Fraction.

Figure B.4 *Attributes added to the class symbols*



Methods

A class can also be extended to include methods. A method is a procedure that can be used by an object (an instance of a class) or applied to an object. In other words, an object is either a doer or a receiver. Figure B.5 shows two classes with attributes and methods. The attributes and methods are listed in separate compartments.

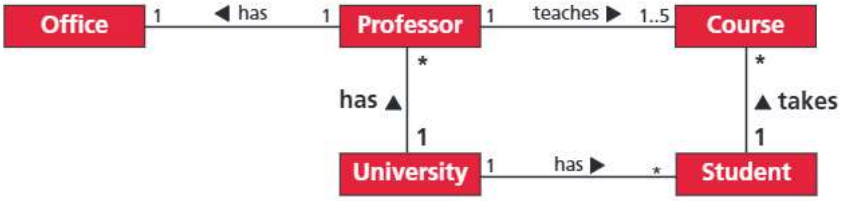
Figure B.5 *Attributes and methods added to the class symbols*

Association

An association is a conceptual relation between two classes. An association is shown by a solid line between two classes. If a name is given to the association, it is written next to the line with a solid arrow.

An association can be one-to-one, one-to-many, many-to-one or many-to-many. Figure B.6 shows four classes and some associations between them. It shows that one professor (an object of the Professor class) can teach from one to five courses (1…5). Conversely, in this example, a course can have only one professor. The university (an object of the University class) can have many professors and many students (objects of the Student class), as indicated by the asterisk (*) on the association line. The figure also shows that a student can take many courses.

Figure B.6 *Association between classes*

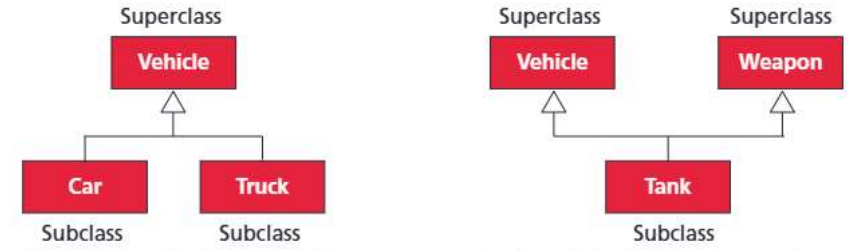


Generalization

Generalization organizes classes based on their similarities and differences. Generalization allows us to define **subclasses** and **superclasses**. A subclass inherits characteristics (attributes and methods) of all its superclasses, but it normally has some characteristics (attributes and methods) of its own. Figure B.7 shows single and multiple inheritance.

https://sanet.st/blogs/polatebooks/

Figure B.7 *Generalization (inheritance)*



B.3 THE BEHAVIORAL VIEW

A behavioral view looks at the behavior of objects in a system. Depending on the type of the behavior, we can have four different diagrams: collaboration diagrams, state diagrams, sequence diagrams, and activity diagrams.

B.3.1 Collaboration diagrams

A collaboration diagram is similar to a class diagram. The difference is that the class diagram shows the relationship between classes, whereas a collaboration diagram shows the relationship between objects (instances of classes).

Any object instantiated from the class can also be shown in a rectangle with the name of the object followed by a colon and the name of the class. For an anonymous object, the name of the object is left out. Figure B.8 shows three objects instantiated from the class Person.

Figure B.8 *Three objects instantiated from the same class*

a. An object named John b. An object named Anne c. An anonymous object

Attributes and values

An attribute is a property of a class, while a value is a property of an object corresponding to an attribute. An object symbol can include values. Figure B.9 shows some attributes of the classes Person and Fraction with values for attributes within the classes.

Figure B.9 *Examples of attributes and values*

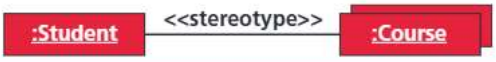
Methods and operations

Although an object symbol can also include methods and operations, it is not common in a collaboration diagram.

Links

A link in a collaboration diagram is an instance of an association in a class diagram. Objects can be related to each other using links. Two stereotype notations can be used for links: local and parameter. The first shows that one object uses another object as a local variable: the second shows that one object uses another object as a parameter. Multiplicity, as shown in the association between Student and Course in Figure B.6, can also be shown by multiple superimposed objects. Multiplicity can also be shown between objects of the same class. Figure B.10 shows that a Student's object uses multiple Course objects as parameters.

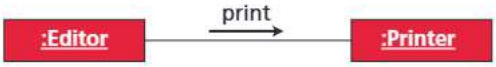
Figure B.10 *A link between objects*



Messages

An object can send a message to another object. A message can represent an event sent from the first object to the second. A message can also invoke a method in the second object. Finally, an object can create or destroy another object using a message. Messages are shown by an arrow pointing in the direction of the message and are shown over the link between objects. Figure B.11 shows how an Editor object sends a print message to a Printer object.

Figure B.11 *A message sent from one object to another*



B.3.2 State diagrams

A state diagram is used to show changes in the states of a single object. An object may change its state in response to an event. For example, a switch may change its state from **off** to **on** when it is turned on. A washing machine may change its state from **wash** to **rinse** in response to triggers from a timer.

Symbols

A state diagram uses three main symbols, as shown in Figure B.12.

Figure B.12 *Symbols used in a state diagram*

States

There are three symbols for states: the start state, the stop state, and the intermediate state. The start state, which is drawn as a black circle with its name next to the circle, is allowed only once in the diagram. The stop state, which is drawn as a solid black circle inside another circle, can be repeated in the diagram. The intermediate state is drawn as a rectangle with rounded corners with the name of the state inside the rectangle.

Transitions

In a state diagram, a transition is a movement between states. The transition symbol is an arrowed line between two states. The arrow shows the next state. One or more transitions can leave a state: only one transition can arrive at a state.

Decision point

A decision point is shown by a diamond. A transition can take several paths based on data or conditions in the object.

Events

In a state diagram, an object is triggered by an event, which can be external or internal. For example, a switch may move from an **off** state to an **on** state if it is turned on. An event is represented by a string which defines the operation in the class that handles the event. It may have parentheses containing the formal parameters to be passed to the operation. An event can also have a condition enclosed in brackets. The following shows an example of an event:

```
withdraw (amount) [amount < balance]
```

An object may or may not move to another state when triggered by an event.

Actions

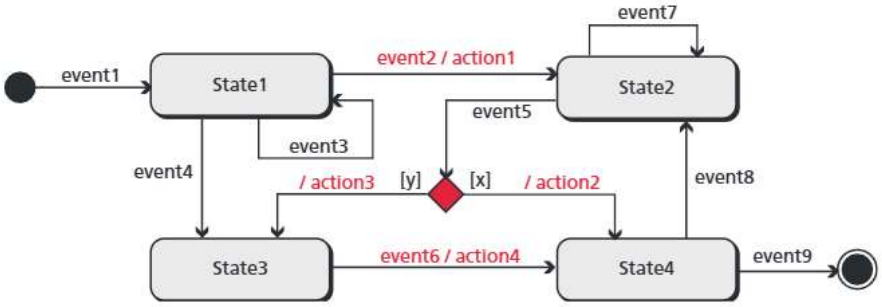
Although an action may be triggered in several ways, we only mention an action triggered by an event. An action is shown by a string, which normally defines another object and the event that should be invoked for that object. If parameters are needed for the target object they are included in parentheses. The action is separated from the event by using a forward slash. The following shows an example of an action:

```
deposit (amount)  /  add (balance)
      ↑                   ↑
    event               action
```

Example B.1

Figure B.13 shows a simple example of a state diagram. There are six states—a start and a stop state and four intermediate states—nine events, and four actions.

Figure B.13 *An example of a state diagram*



B.3.3 Sequence diagrams

A sequence diagram shows the interaction between objects (or actors) over time. In a sequence diagram, objects (or actors) are listed as columns, and time, which flows notionally downwards, is represented as a vertical broken line.

Symbols

A sequence diagram uses five main symbols, as shown in Figure B.14.

Figure B.14 *Symbols used in a sequence diagram*

Actor

The symbol for an actor is the same stick figure as we saw in use-case diagrams. Since actors can also communicate with objects, they can be part of a sequence diagram.

Object

Objects, as we saw before, are instances of classes. A sequence diagram represents the interaction between the objects.

Lifeline

A lifeline, shown by a solid or dashed vertical line, represents an individual participant in a sequence diagram. It is usually headed by a rectangle that contains the name of the object or actor. The vertical line, which represents the lifespan of the object, extends to the point where the object is no longer active.

Activation

Activation, represented by a solid narrow rectangle, shows the time when the object is involved in an activity, that is, when it is not idle. For example, if an object has sent a message to another object and is waiting for a response, the object is involved during this time.

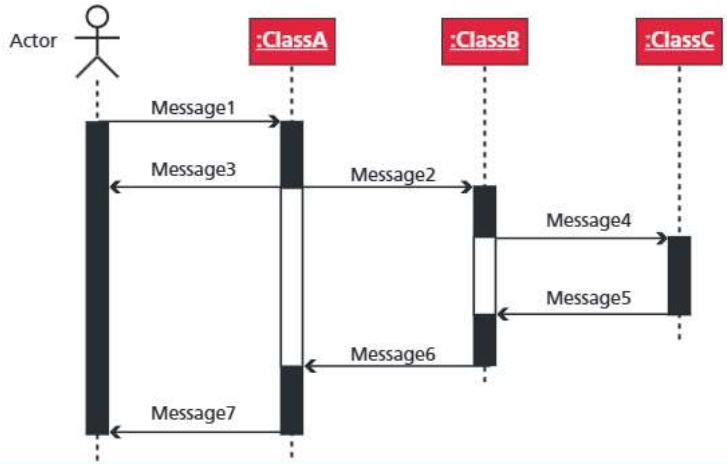
Message

Messages are shown as horizontal arrowed lines showing the interaction between objects (or actors).

Example B.2

Figure B.15 shows a simple example of a sequence diagram with one actor and three anonymous objects. The diagram also shows concurrency: the first object, after receiving the first message, concurrently sends two messages: one to the actor and one to the second object.

Figure B.15 *An example of a sequence diagram*



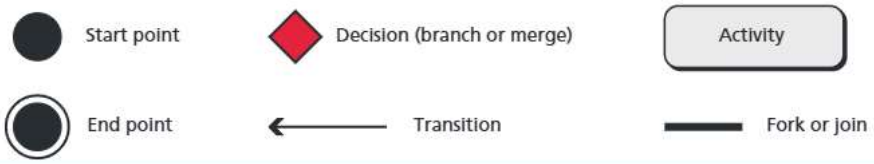
B.3.4 Activity diagrams

An activity diagram shows the break-down of a complex operation or a process into a set of simpler operations or processes. An activity diagram is more detailed than a sequence diagram. A sequence diagram emphasizes objects, while an activity diagram shows more detailed operations performed by one or more objects. An activity diagram in object-oriented programming replaces a traditional flowchart in procedural programming. However, a traditional flowchart shows only sequential flow control (serial), while an activity diagram can show both sequential and concurrent (parallel) flow control.

Symbols

An activity diagram uses six main symbols, as shown in Figure B.16.

Figure B.16 *Symbols used in an activity diagram*



Activities

An activity is a step in an activity diagram. We show an activity using a rectangle with rounded corners that contains the name of the activity. The level of detail in an activity should be consistent for the whole diagram. If more detail is needed for one of the activities, a new diagram should be drawn to show it.

Transitions

Similar to a state diagram, a transition in an activity diagram is shown by an arrowed line. The arrow shows the direction of the action.

Start and end points

The start point in an activity diagram is a solid circle with a single outgoing transition: the stop point is a solid circle surrounded by a hollow circle (bull's eye) with a single incoming transition. There can be only one start point. While logically there can be only one end point, multiple end points are allowed to make the diagram easier to read.

Decision and merge

A diamond shows a decision or a merge point. A transition can take several paths based on conditions. When used as a decision point, a diamond symbol can have only one entry, with two or more exits. When used as a merge point, a diamond symbol can have two or more entries but only one exit.

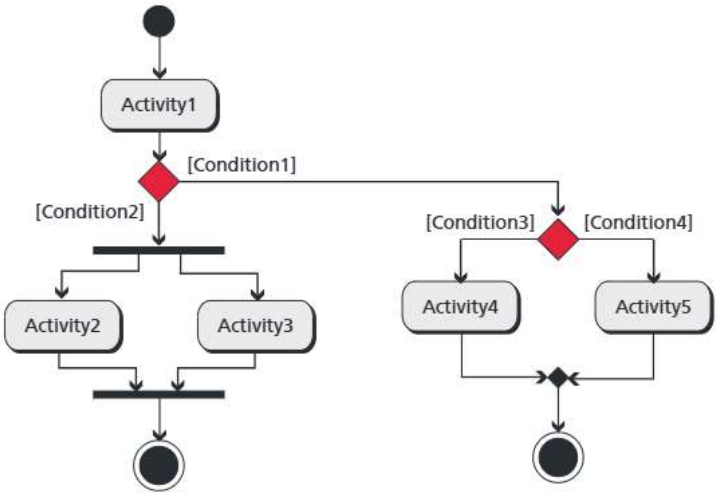
Fork or joint

A thick line shows a fork or join in parallel processing. A fork symbol shows the start of two or more threads of processes: a joint symbol shows the end of the threads.

Example B.3

Figure B.17 shows an example of an activity diagram. Activities 2 and 3 are done concurrently (parallel processing).

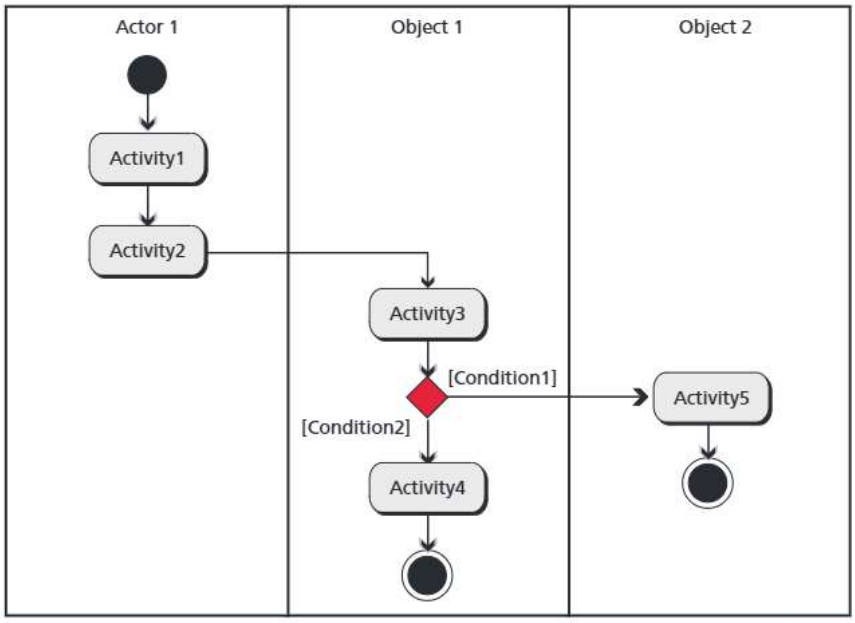
Figure B.17 *An example of an activity diagram*



B.3.5 Swimlanes

Sometimes operations in an activity diagram are performed by different objects or actors. To show that more than one object or actor is involved, **swimlanes** are added to an activity diagram, as shown in Figure B.18.

Figure B.18 *An activity diagram with swimlanes*



B.4 THE IMPLEMENTATION VIEW

An implementation view shows how the final product is implemented. Two types of diagrams are used to show the implementations: component diagrams and deployment diagrams.

B.4.1 Component diagrams

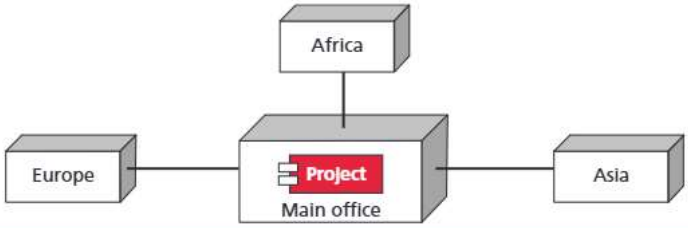
A component diagram shows the software components and the dependencies among them. The components are shown as rectangles with two small rectangles on their left edges. A dependency between the components is shown by a dashed line with an arrow on the end. We can also use stereotyping on the dependency line by including such stereotype relations such as **<<report>>**. Figure B.19 shows a component diagram.

Figure B.19 *An example of a component diagram*

B.4.2 Deployment diagrams

A deployment diagram shows nodes connected by communication links. A node is shown as a cuboid and the communication association (link) is shown as a line connecting two nodes. A node can also include one or more components. Figure B.20 shows a simple deployment diagram.

Figure B.20 *An example of a deployment diagram*



APPENDIX C

Pseudocode

One of the most common tools for defining algorithms is pseudocode. **Pseudocode** is an English-like representation of the code required for an algorithm. It is part English and part structured code. The English part provides a relaxed syntax that is easy to read. The code part consists of an extended version of the basic algorithmic constructs: *sequence*, *selection*, and *loop*. Algorithm C.1 shows an example of pseudocode. We briefly discuss each component in the next section.

Algorithm C.1 Example of pseudocode

```
Loop: FindingSmallest (list)
Purpose: Finds the smallest number among a list of numbers
Pre: List of numbers
Post: None
Return: The smallest number in the list
{
   smallest ← first number
   Loop (not end of list)
   {
        If (next number < smallest)
        {
             smallest ← second number
        }
   }
   Return value of smallest
}
```

C.1 COMPONENTS

An algorithm written in pseudocode can be decomposed into several elements and constructs.

C.1.1 Algorithm header

Each algorithm begins with a header that names it. For example, in Algorithm C.1, the header starts with the word *Algorithm*, which gives the algorithm's title as 'Finding Smallest'.

C.1.2 Purpose, conditions, and return

After the header, we normally mention the purpose, the precondition, postconditions, and the data returned from the algorithm.

Purpose

The purpose is a short statement about what the algorithm does. It needs to describe only the general algorithm processing. It should not attempt to describe all of the processing. The purpose starts with the word Purpose and continues with the goal of the algorithm.

Precondition

The precondition lists any precursor requirements. For example, we require that the list be available to the algorithm.

Postcondition

The postcondition identifies any effect created by the algorithm. For example, the algorithm may require the printing of data.

Return

We believe that every algorithm should show what is returned from the algorithm. If there is nothing to be returned, we advise that null be specified. The smallest value that is found is returned.

Statement

Statements are commands such as **assign, input, output, if-then-else, and loop**, as shown in Algorithms C.1, C.2, C.3, and C.4. Nested statements—statements inside another statement—are indented. The list of nested statements starts with the opening brace (curly bracket) and ends with a closing brace. The whole argument is a list of nested statements inside the algorithm itself. For this reason, we see an opening brace at the beginning and a closing brace at the end.

C.1.3 Statement constructs

When Niklaus Wirth first proposed the structured programming model, he stated that any algorithm could be written with only three programming constructs: *sequence*, *selection*,

and *loop*. Our pseudocode contains only these three basic constructs. The implementation of these constructs relies on the richness of the implementation language. For example, the loop can be implemented as a *while*, *do-while*, or *for* statement in the C language.

Sequence

A sequence is a series of statements that do not alter the execution path within an algorithm. Although it is obvious that statements such as **assign** and **add** are sequence statements, it is not so obvious that a call to other algorithms is also considered a sequence statement. The reason lies in the structured programming concept that each algorithm has only one entry and one exit. Furthermore, when an algorithm completes, it returns to the statement immediately after the call that invoked it. You can therefore properly consider the algorithm call a sequence statement. Algorithm C.2 shows a sequence.

Algorithm C.2 Example of a sequence

```
x ← first number
y ← second number
z ← x × y
call Argument X
```

Selection

Selection statements evaluate one or more alternatives. If true, one path is taken; if false, a different path is taken. The typical selection statement is the two-way selection (if-else). Whereas most languages provide for multi-way selections, we provide none in pseudocode. The alternatives of the selection are identified by indentation, as shown in Algorithm C.3.

Algorithm C.3 Example of a selection

```
If (x < y)
{
    Increment x
    Print x
}
Else
{
    Decrement y
    Print y
}
```

Loop

A loop iterates a block of code. The loop in our pseudocode most closely resembles the while loop. It is a pretest loop: that is, the condition is evaluated before the body of the loop is executed. If the condition is true, the body is executed. If the condition is false, the loop terminates. Algorithm C.4 shows an example of a loop.

Algorithm C.4 Example of a loop

```
Loop (more lines in the file File1)
{
   Read next line
   Delete the leading space
   Copy the line to File2
}
```

APPENDIX D

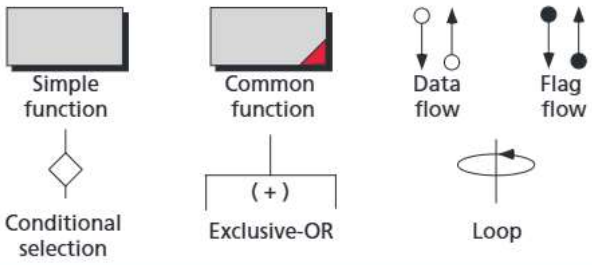
Structure Chart

The structure chart is the primary tool in a procedure-oriented software design phase. As a design tool, it is created before we start writing our program.

D.1 STRUCTURE CHART SYMBOLS

Figure D.1 shows the various symbols used in a structure chart.

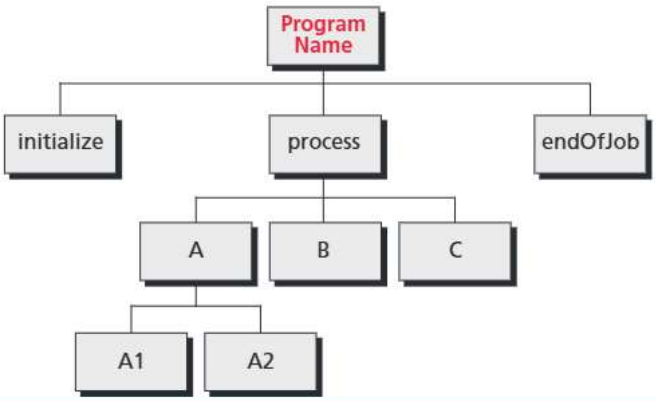
Figure D.1 *Structure chart symbols*



D.1.1 Module symbol

Each rectangle in a structure chart represents a module. The name in the rectangle is the name you give to the module (Figure D.2).

Figure D.2 *An example of a structure chart*



D.1.2 Selection in structure charts

Figure D.3 shows two symbols for a module that is called by a selection statement: the condition and the exclusive OR.

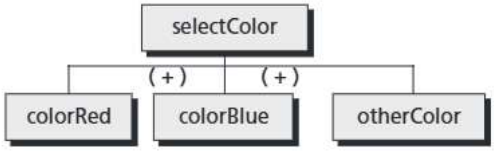
Figure D.3 *Selection in a structure chart*

In Figure D.3a, the module *A* contains a conditional call to a submodule, *fun*. If the condition is true, you call *fun*. If it is not true, we skip *fun*. This situation is represented in a structure chart as a diamond on the vertical line between the two module blocks.

Figure D.3b represents selection between two different modules. In this example the module *select* chooses between *A* and *B*. One and only one of them will be called each time the selection statement is executed. This is known as an exclusive OR: one of the two alternatives is executed to the exclusion of the other. The exclusive OR is represented by a plus sign between the modules.

Now consider the design of a series of modules that can be called exclusively. This occurs when a multi-way selection contains calls to several different modules. Figure D.4 contains an example of a selection statement that calls different modules based on color.

Figure D.4 *An example of a selection*



D.1.3 Loops in structure charts

Let's look at how loops are shown in a structure chart. The symbols are very simple. Loops go in circles, so the symbol used is a circle. Programmers use two basic looping symbols. The first is a simple loop, shown in Figure D.5a. The other is the conditional loop, shown in Figure D.5b. When the module is called unconditionally, as in a *while* loop, the circle flows around the line above the called module. On the other hand, if the call is conditional, as in a module called in an *if-else* statement inside a loop, then the circle includes a decision diamond on the line.

Figure D.5 *Loops in a structure chart*

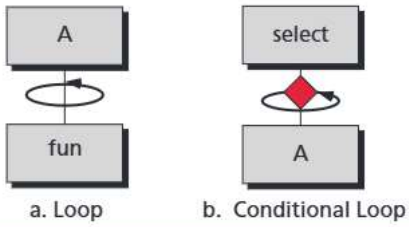
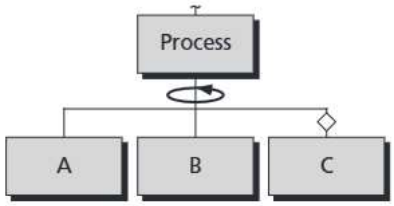


Figure D.6 shows the basic structure for a module called *process*. The circle is *below* the module that controls the loop. In this example, the looping statement is contained in *process*, and it calls three modules, *A*, *B*, and C. The exact nature of the loop cannot be determined from the structure chart. It could be any of the three basic looping constructs.

Figure D.6 *An example of a loop*



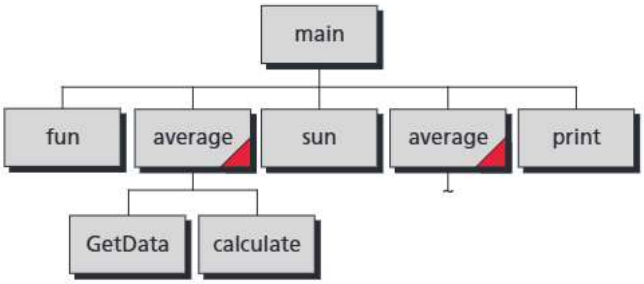
D.2 READING STRUCTURE CHARTS

Structure charts are read *top-down* and *left-to-right*. Referring to Figure D.2 (page 558), this rule says that the program (*main*) consists of three submodules: *initialize*, *process*, and *endOfJob*. According to the left-to-right rule, the first call in the program is to *initialize*. After *initialize* is complete, the program calls *process*. When *process* is complete, the program calls *endOfJob*. In other words, the modules on the same level of a structure chart are called in order from left to right.

The concept of top-down is demonstrated by *process*. When *process* is called, it calls *A*, *B*, and C in turn. Module *B* does not start running, however, until *A* is finished. While *A* is running, it calls *A1* and *A2* in turn. In other words, all modules in a line from *process* to *A2* must be called before module *B* can start.

Often a program will contain several calls to a common module. These calls are usually scattered throughout the program. The structure chart will show the call wherever it logically occurs in the program. To identify common structures, the lower right corner of the rectangle will contain crosshatching or will be shaded. If the common module is complex and contains submodules, these submodules need to be shown only once. An indication that the incomplete references contain additional structure should be shown. This is usually done with a line below the module rectangle and a cut (~) symbol. This idea is shown in Figure D.7, which uses a common module, *average*, in two different places in the program. Note, however, that you never show a module connected to two calling modules graphically.

Figure D.7 *Several calls to the same module*



D.3 RULES OF STRUCTURE CHARTS

We summarize the rules discussed in this section:

- Each rectangle in a structure chart represents a module.
- The name in the rectangle is the name that will be used in the coding of the module.
- The structure chart contains only module flow. No code is indicated.

- Common modules are indicated by crosshatching or shading in the lower right corner of the module rectangle.
- Data flows and flags are optional. When used, they should be named.
- Input flows and flags are shown to the left of the vertical line; output flows and flags are shown to the right.

APPENDIX E

Boolean Algebra and Logic Circuits

E.1 BOOLEAN ALGEBRA

Boolean algebra deals with variables and constants that take only one of two values: 1 or 0. This algebra is a suitable way to represent information in a computer, which is made of a collection of signals that can be in only one of the two states: on or off.

E.1.1 Constants, variables, and operators

We use constants, variables, and operators in Boolean algebra.

Constants

There are only two constants: 1 and 0. The value of 1 is associated with the logical value *true*: the value 0 is associated with the logical value *false*.

Variables

We use letters such as x, y, and z to represent variables. Boolean variables can take only the values 0 or 1.

Operators

We use three basic operators: NOT, AND, and OR. We use a prime to represent NOT, a dot to represent AND, and a plus sign to represent OR, as shown below:

$x' \rightarrow$ NOT x	$x \cdot y \rightarrow x$ AND y	$x + y \rightarrow x$ OR y

An operator takes one or two values and creates one output value. The first operator, NOT, is a unary operator that takes only one value: the other two, AND and OR, are binary operators that take two values. Note that the choice of operators is arbitrary. We can construct all gates from the NAND gate (explained later).

E.1.2 Expressions

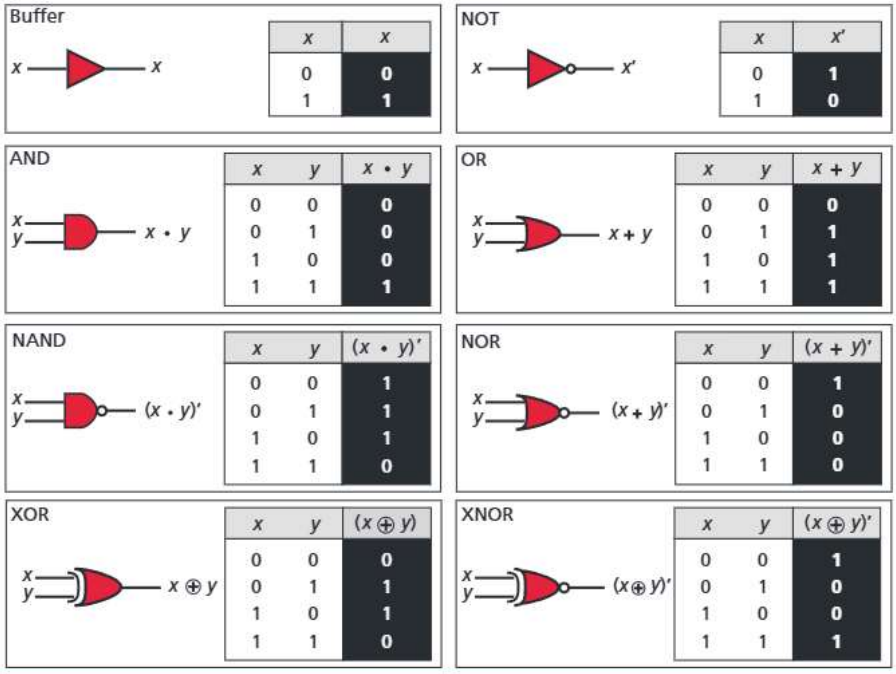
An expression is a combination of Boolean operators, constants, and variables. The following shows some Boolean expressions:

0	x	$x \,.\, 1$	$x + 0$
$x + 1 + y$	$x \,.\, (y + z)$	$x + y + z$	$x \,.\, y \,.\, z \,.\, t$

E.1.3 Logic gates

A **logic gate** is an electronic device that normally takes 1 to N inputs and creates one output. In this appendix, however, we use gates with only one or two inputs for simplicity. The logical value of the output is determined by the expression representing the gate and the input values. A variety of logic gates are commonly used in digital computers. Figure E.1 shows the symbols for the eight most common gates, their truth tables (see Chapter 4), and the expressions that can be used to find the output when the input or inputs are given.

Figure E.1 *Symbols and truth table for common gates*



- **Buffer**. The first gate is just a buffer, in which the input and the output are the same. If the input is 0, the output is 0: if the input is 1, the output is 1. The buffer only amplifies the input signal.

- **NOT**. The NOT gate is the implementation of the NOT operator. The output of this gate is the complement of the input. If the input is 1, the output is 0: if the input is 0, the output is 1.
- **AND**. The AND gate is the implementation of the AND operator. It takes two inputs and creates one output. The output is 1 if both inputs are 1s, otherwise it is 0. Sometimes the AND operator is referred to as *product*.
- **OR**. The OR gate is the implementation of the OR operator. It takes two inputs and creates one output. The output is 1 if any of the inputs, or both of them, is 1, otherwise it is 0. Sometimes the OR gate is referred to as *sum*.
- **NAND**. The NAND gate is a logical combination of an AND gate followed by a NOT gate. The reason for its existence can be explained when we discuss the actual implementation of these gates. The output of a NAND gate is the complement of the corresponding AND gate if the inputs to two gates are the same.
- **NOR**. The NOR gate is a logical combination of an OR gate followed by a NOT gate. The reason for its existence can also be explained when we discuss the actual implementation of these gates. The output of a NOR gate is the complement of the corresponding OR gate if the inputs to two gates are the same.
- **XOR**. The XOR (exclusive-OR) gate is defined by the expression $(x \,.\, y' + x' \,.\, y)$, which is normally represented as $(x \oplus y)$. The output of this gate is 1 when the two inputs are different and is 0 when the inputs are the same. One can say that this is a more restricted OR gate. The output of an XOR gate is the same as the OR gate except that, if the two inputs are 1s, the output is 0.
- **XNOR**. The XNOR (exclusive-NOR) gate is defined by the expression $(x \,.\, y' + x' \,.\, y)'$ which is normally represented as $(x \oplus y)'$. It is the complement of the XOR gate. The output of this gate is 1 when the two inputs are the same and 0 when the inputs are different. One can say that this represents the logical idea of equivalence: only if the two inputs are equal is the output 1.

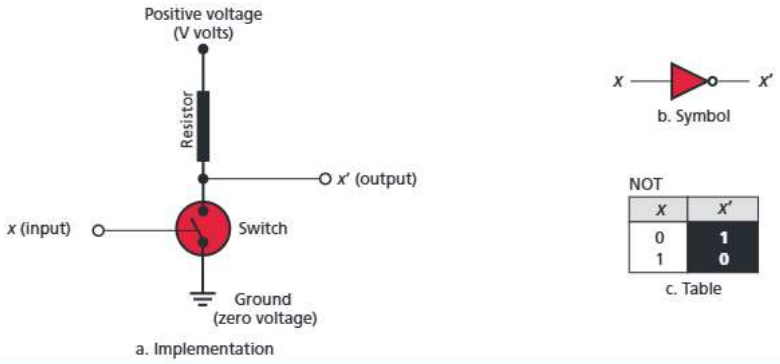
Implementation of gates

The logic gates discussed in the previous section can be physically implemented using electronic switches (transistors). The most common implementation uses only three gates: NOT, NAND, and NOR. A NAND gate uses fewer components than an AND gate. This is also true for the NOR gate *versus* the OR gate. As a result, NAND and NOR gates have become the common standard in the industry. We only discuss these three implementations. Although we show simple switches in this discussion, we need to know that, in practice, switches are replaced by transistors. A transistor, when used in gates, behaves like a switch. The switch can be opened or closed by applying the appropriate voltage to the input. Several different technologies are used to implement these transistors, but we leave this discussion to books on electronics.

Implementation of the NOT gate

The NOT gate can be implemented with an electronic switch, a voltage source, and a resistor as shown in Figure E.2.

Figure E.2 *Implementation of the NOT gate*



The input to the gate is a control signal that holds the switch open or closed. An input signal of 0 holds the switch open, while an input signal of 1 closes the switch. The output is the voltage at the point before the switch (output terminal). If the value of this voltage is positive (V volts), the output is interpreted as 1: if the voltage is 0 (or below a threshold), the output is interpreted as 0. When the switch is open, there is no current through the resistor, and therefore no voltage drop. The output voltage is V (interpreted as logic 1). Closing the switch grounds the output terminal and makes its voltage 0 (or almost 0), which is interpreted as logic 0. Note that the behavior of the circuit matches the values shown in the table.

To implement a NOT gate, we need only one electronic switch.

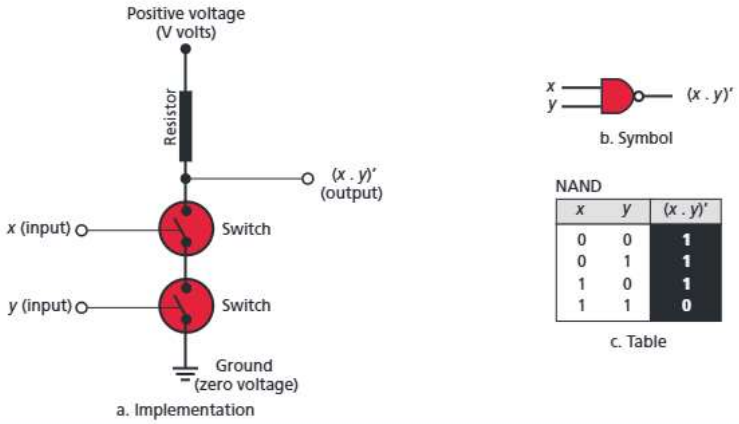
Implementation of the NAND gate

The NAND gate can be implemented using two switches in series (two inputs). For the current to flow through the circuit from the positive terminal to the ground, both switches must be closed—that is, both inputs must be 1s. In this case, the voltage of the output terminal is zero because it is grounded (logic 0). If one of the switches or both switches are open—that is, where the inputs are 00, 01, or 10—no current flows through the resistor. There is thus no voltage drop across the resistor and the voltage at the output terminal is V (logic 1).

Figure E.3 shows the implementation of the NAND gate. The behavior of the circuit matches the values shown in the table. Note that if an AND gate is needed, it can be made from a NAND gate followed by a NOT gate.

To implement a NAND gate, we need two electronic switches that are connected in series.

Figure E.3 *Implementation of the NAND gate*



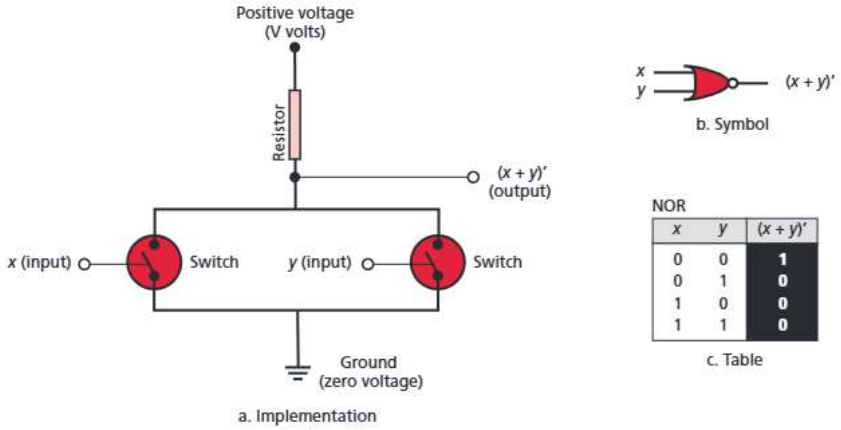
Implementation of the NOR gate

The NOR gate can also be implemented using two switches in parallel (two inputs). If both switches are open, then the current does not flow through the resistor. In this case, there is no voltage drop across the resistor, which means the output terminal holds the voltage V (logic 1). If either or both of the switches are closed, the output terminal is grounded and the output voltage is zero (logic 0).

Figure E.4 shows the implementation of the NOR gate. The behavior of the circuit matches the values in the table. Note that if an OR gate is needed, it can be simulated using a NOR gate followed by a NOT gate.

To implement a NOR gate, we need two electronic switches that are connected in parallel.

Figure E.4 *Implementation of the NOR gate*



E.1.4 Axioms, theorems, and Identities

To be able to work with Boolean algebra, we need to have some rules. The rules in Boolean algebra are divided into three broad categories: *axioms*, *theorems*, and *identities*.

Axioms

Boolean algebra, like any other algebra, uses some rules, called **axioms**: they cannot be proved. Table E.1 shows the axioms for Boolean algebra.

Table E.1 Axioms for Boolean algebra

	Related to NOT	*Related to AND*	*Related to OR*
1	$x = 0 \rightarrow x' = 1$		
2	$x = 1 \rightarrow x' = 0$		
3		$0 \bullet 0 = 0$	$0 + 0 = 0$
4		$1 \bullet 1 = 1$	$1 + 1 = 1$
5		$1 \bullet 0 = 0 \bullet 1 = 0$	$1 + 0 = 0 + 1 = 1$

Theorems

Theorems are rules that we prove using the axioms, although we must leave the proofs to textbooks on Boolean algebra. Table E.2 shows some theorems used in Boolean algebra.

Table E.2 Basic theorems for Boolean algebra

	Related to NOT	*Related to* AND	*Related to* OR
1	$(x')' = x$		
2		$0 \bullet x = 0$	$0 + x = x$
3		$1 \bullet x = x$	$1 + x = 1$
4		$x \bullet x = x$	$x + x = x$
5		$x \bullet x' = 0$	$x + x' = 1$

Identities

We can also drive many identities using the axioms and the theorems. We list only the most common in Table E.3, although we must leave the proofs to textbooks on Boolean algebra.

Table E.3 Basic Identities related to OR and AND operators

	Description	*Related to AND*	*Related to OR*
1	Commutativity	$x \bullet y = y \bullet x$	$x + y = y + x$
2	Associativity	$x \bullet (y \bullet z) = (x \bullet y) \bullet z$	$x + (y + z) = (x + y) + z$
3	Distributivity	$x \bullet (y + z) = (x \bullet y) + (y \bullet z)$	$x + (y \bullet z) = (x + y) \bullet (x + z)$
4	De Morgan's Rules	$(x \bullet y)' = x' + y'$	$(x + y)' = x' \bullet y'$
5	Absorption	$x \bullet (x' + y) = x \bullet y$	$x + (x' \bullet y) = x + y$

De Morgan's Rules play a very important role in logic design, as we will see shortly. They can be extended to more than one variable. For example, we can have the following two identities for three variables:

$$(x + y + z)' = x'.\ y'.\ z' \qquad (x.\ y.\ z)' = x' + y' + z'$$

E.1.5 Boolean functions

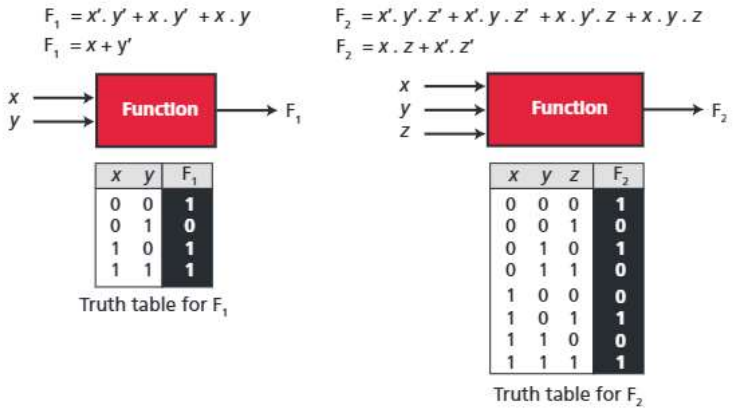
We define a **Boolean function** as a function with n Boolean input variables and one Boolean output variable, as shown in Figure E.5.

Figure E.5 *A Boolean function*

A function can be represented either by a truth table or an expression. The truth table for a function has 2^n rows and $n + 1$ columns, in which the first n columns define the possible values of the variables and the last column defines the value of the function's output for the combination of the values defined in the first n columns.

Figure E.6 shows the truth tables and expression representation for two functions F_1 and F_2. Although the truth table representation is unique, a function can be represented by different expressions. We have shown two of the expressions for each function. Note that the second expressions are shorter and simpler. Later we show that we need to simplify the expressions to make the implementation more efficient.

Figure E.6 *Examples of table-to-expression transformation*



x	y	F_1
0	0	1
0	1	0
1	0	1
1	1	1

Truth table for F_1

x	y	z	F_2
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Truth table for F_2

Table-to-expression transformation

The specification of a function is normally given by a truth table (see Chapter 4). To implement the function using logic gates (as discussed earlier), we need to find an expression for the truth table. This can be done in two ways.

Sum of products

The first method of changing a truth table into an expression is referred to as the *sum of products* method. A sum of products representation of a function is made of up to 2^n terms in which each term is called a **minterm**. A minterm is a product (ANDing) of all variables in a function in which each variable appears only once. For example, in a three-variable function, we can have eight minterms, such as $x'.y'.z'$ or $x.y'.z'$. Each term represents one row in the truth value. If the value of a variable is 0, the complement of the variable appears in the term: if the value of the variable is 1, the variable itself appears in the term. To transform a truth table to a sum of product representation, we use the following strategy:

1. Find the minterms for each row for which the function has a value of 1.
2. Use the sum (ORing) of the terms in step 1.

Product of sums

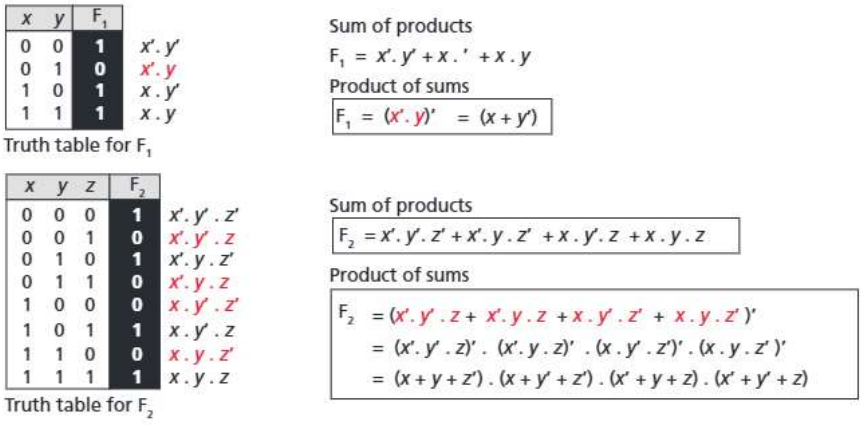
The second method of changing a truth table to an expression is referred to as the *product of sums* method. A product of sum representation of a function is made of up to 2^n terms in which each term is called a **maxterm**. A maxterm is a sum (ORing) of all variables in a function in which each variable appears only once. For example, in a three-variable function, we can have eight maxterms such as $x'+y'+z'$ or $x+y'+z'$. To transform a truth table to a product of sum representation, we use the following strategy:

1. Find the minterms for each row for which the function has a 0 value.
2. Find the complement of the sum of the terms in step 1.
3. Use De Morgan's rules to change minterms to maxterms.

Example E.1

Figure E.7 shows how we create the sum of products and product of sums for the functions F1 and F2 in Figure E.6.

Figure E.7 *Example E.1*



The sum of products is directly made from the table, but the product of sums needs the use of De Morgan's rules. Note that sometimes the first method gives the shorter expression and sometimes the second one.

E.1.6 Function simplification

Although we can implement a Boolean function using the logic gates discussed before, it is normally not efficient. The direct implementation of a function requires more gates. The number of gates could be reduced if we can carry out simplification. Traditionally two methods of simplification are used: the algebraic method using Karnaugh maps, and the Quine–McKluskey method.

Algebraic method

We can simplify a function using the axioms, theorems, and identities discussed before. For example, we can simplify the first function (F_1) in Figure E.7, as shown below:

F1	=	$x' \bullet y' + x \bullet y' + x \bullet y$	
	=	$(x' + x) \bullet y' + x \bullet y$	Identity 3 (distributivity) for AND
	=	$1 \bullet y' + x \bullet y$	Theorem 5 for OR
	=	$y' + x \bullet y$	Theorem 3 for AND
	=	$y' + y \bullet x$	Theorem 1 (commutativity) for AND
	=	$y' + x$	Identity 5 (absorption)
	=	$x + y'$	Theorem 1 (commutativity) for OR

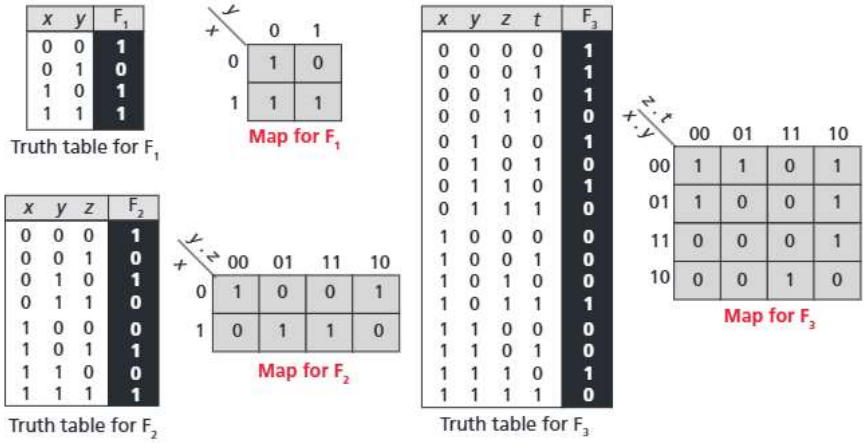
https://sanet.st/blogs/polatebooks/

This means that if the non-simplified version needs eight gates, the simplified version needs only two gates, one NOT and one OR.

Karnaugh map method

Another simplification method involves the use of a **Karnaugh map**. This method can normally be used for functions of up to four variables. A map is a matrix of 2^n cells in which each cell represents one of the values of the function. The first point that deserves attention is to fill up the map correctly. Contrary to expectations, the map is not always filled up row by row or column by column: it is filled up according to the value of variables as shown on the map. Figure E.8 shows an example where $n = 2$, 3, or 4.

Figure E.8 *Construction of Karnaugh maps*



In the truth table, we use the function values from the top to the bottom of the truth table. The map is filled up one by one, but the order of rows are 1, 2, 4, 3. In each row, the columns are filled up one by one, but the order of the columns are 1, 2, 4, 3. The fourth row comes before the third row: the fourth column comes before the third. This arrangement is needed to allow the maximum of simplification.

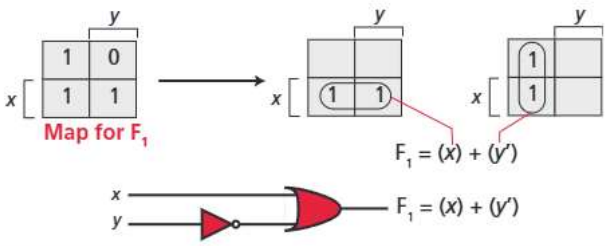
Sum of products

The simplification can be done to create sum of products terms. When we simplify a function in this way we use minterms with value of 1. To create an efficient expression, we first combine adjacent minterm cells. Note that adjacency can also include wrap-around of bits.

Example E.2

Figure E.9 shows the sum of products simplification for our first function. The 1s in the second row are the entire x domain. The 1s in the first column are the entire y' domain. The resulting simplified function is $F_1 = (x) + (y')$. The figure also shows the implementation using one OR gate and one NOT gate.

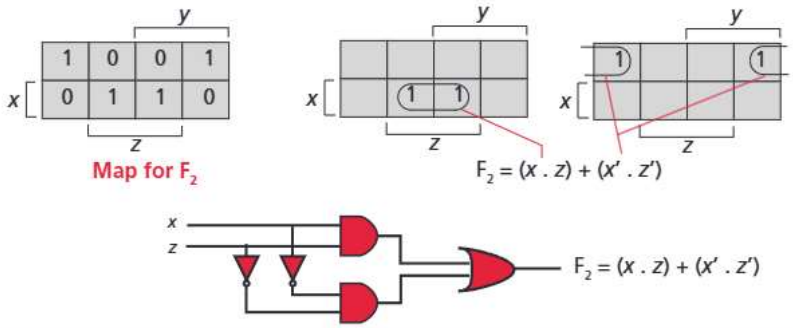
Figure E.9 *Example E.2*



Example E.3

Figure E.10 shows the sum of products simplification for our second function. The 1s in the second row are the intersection of x and z domains, which is represented as $(x \,.\, z)$. The 1s in the first row are the intersection of x' and z' domains, which is represented as $(x'.\, z')$. The resulting simplified function is $F_2 = (x \,.\, z) + (x'.\, z')$. The figure also shows the implementation using one OR gate, two AND gates, and two NOT gates.

Figure E.10 *Example E.3*



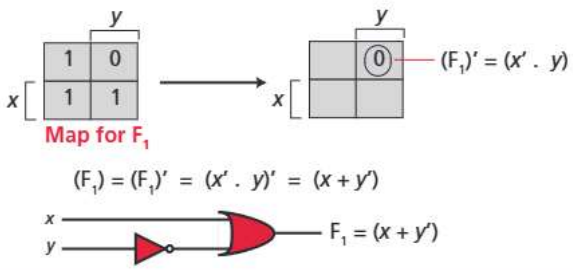
Product of sums

The simplification can be done using the product of sums methods. When we simplify a function in this way, we need to use maxterms. To create an efficient expression, we first combine the adjacent minterm cells. However, the function obtained in this way is the complement of the function we are looking for: we need to use the De Morgan's rules to find our function.

Example E.4

Figure E.11 shows a product of sums simplification for our first function. Note that in this case the implementation is exactly the same as Figure E.9, but this is not always the case. Also note that our function has only one term: we need no AND gate.

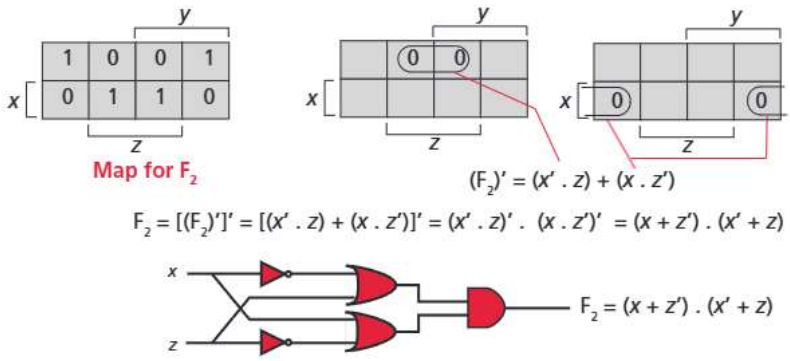
Figure E.11 *Example E.4*



Example E.5

Figure E.12 shows the product-of-sums simplification for our second function. Note that the process gives us $(F_2)'$, so we need to apply the De Morgan's rules to find F_2. The figure also shows the implementation using two NOT gates, two OR gates, and one AND gate. This implementation is less efficient than the one we found with minterms. We should always use the one which is more efficient.

Figure E.12 *Example E.5*



E.2 LOGIC CIRCUITS

A computer is normally built out of standard components that we collectively refer to as **logic circuits**. Logic circuits are divided into two broad categories, known as *combinational circuits* and *sequential circuits*. We briefly discuss each category here and give some examples.

E.2.1 Combinational circuits

A **combinational circuit** is a circuit made up of a combination of logic gates with n inputs and m outputs. Each output at any time entirely depends on all given inputs.

In a combinational circuit, each output at any time depends entirely on all inputs.

Figure E.13 shows the block diagram of a combinational circuit with n inputs and m outputs. Comparing Figure E.13 and Figure E.5, we can say that a combinational circuit with m outputs can be thought of as m functions, a function for each output.

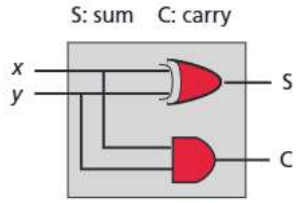
Figure E.13 *A combinational circuit*

The output of a combinational circuit is normally defined by a truth table. However, the truth table needs to have m outputs.

Half adder

A simple example of a combinational circuit is a **half adder**, an adder that can only add two bits. A half adder is a combinational circuit with two inputs and two outputs. The two inputs define the two bits to be added. The first output is the sum of the two bits, while the second output is the carry bit that needs to be propagated to the next adder. Figure E.14 shows a half-adder with its truth table and the logic gates used to the make the circuit.

Figure E.14 *Half adder*



a. Combinational circuit

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

b. Truth table

The sum of two bits can be achieved using an XOR gate: the carry can be achieved using and AND gate.

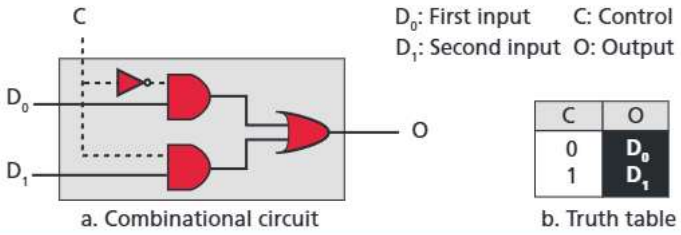
Multiplexer

A **multiplexer** is a combinational circuit with n inputs and only one output. The n inputs are made up of D data inputs and C control inputs ($n = D + C$). At any time, the multiplexer routes one of its D data inputs to its single data output. The selection is based on the value of control bits. To select one of the D data inputs, we need $C = \log_2 D$ control bits. If $D = 2$, at any time only one of the data inputs is routed to the output. The control input is only one

bit. If the control input is 0, the first data input is directed to the output: if the control input is 1, the second input is routed to the output.

Figure E.15 shows the truth table and the circuit for a 2 × 1 multiplexer. Note that the circuit actually has three inputs and one output: the control input is considered one of the inputs.

Figure E.15 *Multiplexer*



C	O
0	D_0
1	D_1

Note that the truth table here is very simplified: the output depends only on the control input but the value of the output, however, is one of the two data inputs.

E.2.2 Sequential circuits

A combinational circuit is memoryless: it does not remember its previous output. At any moment the output depends on the current input. A **sequential circuit**, on the other hand, includes the concept of memory in the logic. The memory enables the circuit to remember its current state to be used in the future; the future state can be dependent on the current state.

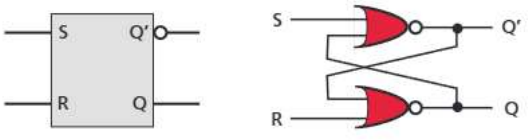
Flip-flops

To add the idea of memory to the combinational circuit, a storage element called a **flip-flop** was invented that can hold one bit of information. A set of flip-flops can be used to hold a set of bits.

SR flip-flops

The simplest type of flip-flop is called an **SR flip-flop**, in which there are two inputs S (set) and R (reset) and two outputs Q and Q', which are always complements of each other. Figure E.16 shows the symbol, the circuit, and the characteristic table of an SR flip-flop. Note that the characteristic table is different from the truth tables we have used for combinational circuits. The characteristic table shows the next output, Q (t + 1) based on the current output, Q (t) and the input.

Figure E.16 *SR flip-flop*



S	R	Q(t + 1)
0	0	Q(t)
0	1	0
1	0	1
1	1	undefined

c. Characteristic table

5/1/25, 9:57 PM study viewer

The characteristic table shows that if both S and R are zero, Q $(t+1)$ = Q (t). The next output will be the same as the current output. If S is 0 and R is 1, Q $(t+1) = 0$, which means the output will be reset (R = 1). If S is 1 and R is 0, Q $(t+1) = 1$, which means that the output will be set. However, if both S and R are 1s, the next output is unpredictable (undefined). Note that we have not shown the value of Q' in the characteristic table, because it is always the complement of Q.

An SR flip-flop can be used as a set–reset device. For example, if the output is connected to an electric sounder, the alarm can be set by letting R = 0 and S = 1. After setting, the alarm continues sounding until it is reset by setting R = 1 and S = 0. The only flaw in this design is that R and S should not simultaneously be 1s.

To understand the behavior of the SR flip-flop we need to create its truth table. However, note that we now have three inputs and one output (Q and Q' are independent). Table E.4 shows the truth table for this flip-flop.

D flip-flop

The SR flip-flop cannot be used as a 1-bit memory, as it needs two inputs instead of one. A small modification to the SR flip-flop can create a **D flop** (D stands for data). Figure E.17 shows the symbol and characteristics of a D flip-flop.

Note that the output of D flip-flop is the same as its input. However, the output remains as it is until the new input is given. This means that it memorizes its input states.

JK flip-flop

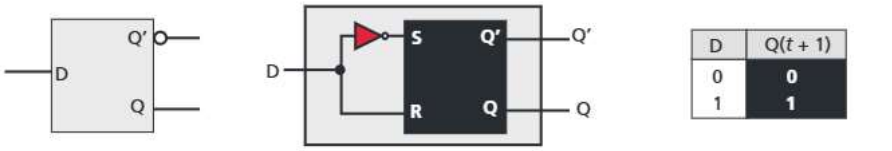
To remove the undefined state from the SR flip-flop, the **JK flip-flop** was invented (JK stands for Jack Kilby, who invented integrated circuits). Adding two AND gates to an SR flip-flop creates a JK flip-flop that has no undefined state. Figure E.18 shows the JK flip-flop and its characteristic table.

Table E.4 Truth table for an SR flip-flop

S	*R*	*Q (t)*	*Q (t + 1)*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	undefined
1	1	1	undefined

https://studylib.net/doc/26059254/foundations-computer-science-4th

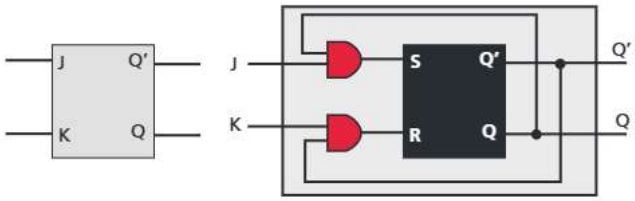
Figure E.17 *D flip-flop*



D	Q(t + 1)
0	0
1	1

c. Characteristic table

Figure E.18 *JK flip-flop*



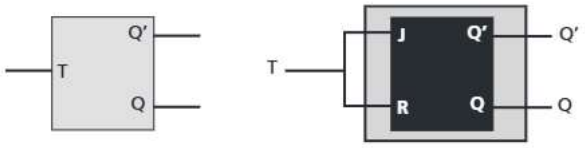
J	K	Q(t + 1)
0	0	Q(t)
0	1	0
1	0	1
1	1	Q'(t)

c. Characteristic table

T flip-flop

Another common type of flip-flop is the T **flip-flop** (T stands for *toggle*). This flip-flop can be made by connecting the two inputs of an JK flip-flop together and calling it the T input. This input toggles the state of the flip-flop: if the input is 0, the next state is the same as the current state. If the input is 1, the next state is the complement of the current state. Figure E.19 shows the symbol, circuit, and characteristic table of the T flip-flop.

Figure E.19 *T flip-flop*



T	Q(t + 1)
0	Q(t)
1	Q'(t)

c. Characteristic table

Synchronous versus *asynchronous*

The flip-flops we have discussed so far are all referred to as **asynchronous devices**: the transition from one state to another can happen only when there is a change in the input.

Digital computers, on the other hand, are **synchronous devices**. A central clock in the computer controls the timing of all logic circuits. The clock creates a signal—a series of pulses with an exact pulse width—that coordinates all events. A simple event takes place only at the 'tick' of this clock signal.

Figure E.20 shows an abstract idea of a clock signal. We call it *abstract* because in reality no electronic circuit can generate a signal with perfectly sharp impulses, but the signal shown here is sufficient for our discussion.

Figure E.20 *Clock pulses*

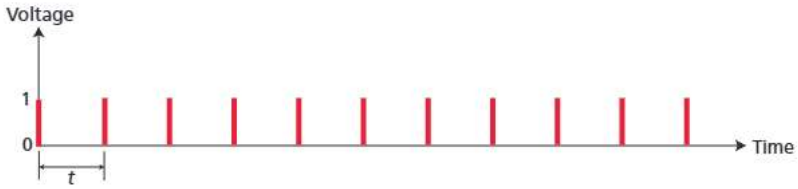
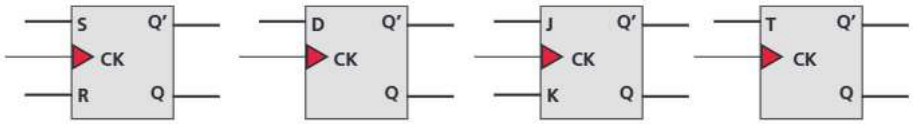


Figure E.21 *Clocked flip-flops*



a. Clocked SR a. Clocked D a. Clocked JK a. Clocked T

A flip-flop can be synchronous if we add one more input to the circuit: the clock input. The clock input can be ANDed with every input to gate the input so that it is effective only when the clock pulse is present. Figure E.21 shows the symbols for the clocked versions of all four flip-flop types we discussed. Figure E.22 shows the circuit of an SR flip-flop with a clock signal. The other flip-flops have the same additional circuitry.

Register

As the first application of a synchronous (clocked) sequential circuit, we will introduce a simplified version of a **register**. A register is an *n*-bit storage device that stores its data between consecutive clock pulses. At the trigger of the clock, the old data is discarded and replaced by the new data.

Figure E.23 shows a 4-bit register in which each cell is composed of a D flip-flop. Note that the clock input is common for all cells. We have rotated our previous symbols to make the connections simpler.

Figure E.22 *Circuit of clocked SR flip-flop*

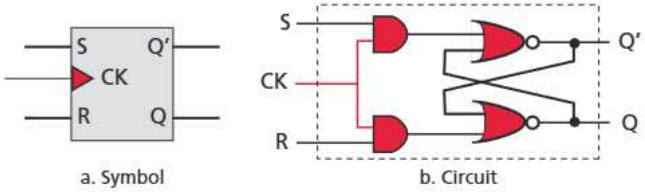
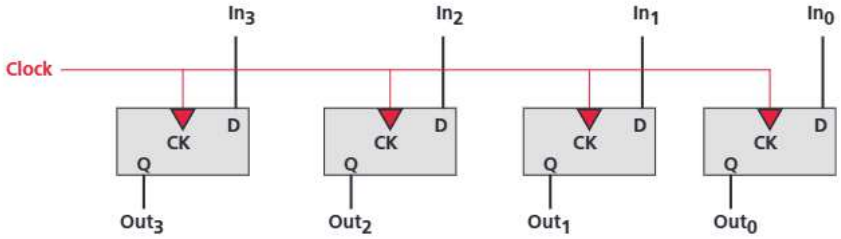


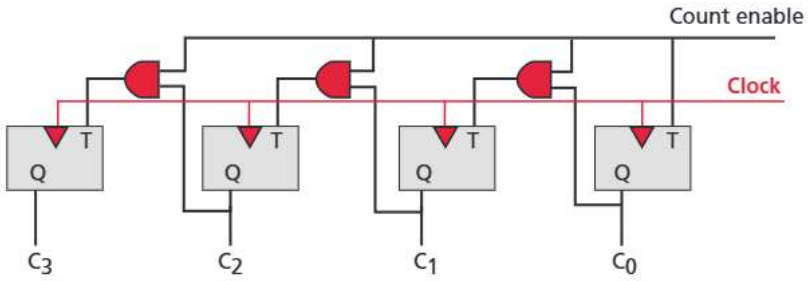
Figure E.23 *A 4-bit register*



Digital counter

An n-bit **digital counter** counts from 0 to $2^n - 1$. For example, when $n = 4$, the output of the counter is 0000, 0001, 0010, 0011, …, 1111, so it counts from 0 to 15. An n-bit counter can be made out of n T flip-flops. At the start, the counter represents 0000. The count enable line—see Figure E.24—carries a sequence of 1s: the data (pulse) to be counted. Looking at the sequence of events, we can see that the rightmost bit is complemented with each positive transition of the count enable connection, simulating the arrival of a data item. When the rightmost bit changes from 1 to 0, the next leftmost bit is complemented. The process is repeated for all bits. This observation gives us a clue to the use of a T flip-flop. The characteristic table of this flip-flop shows that each input of value 1 complements the output. Note that this counter can count only up to 15 or $(1111)_2$. The arrival of the sixteenth data item resets the counter back to $(0000)_2$. Figure E.24 shows the circuit of a 4-bit counter.

Figure E.24 *A 4-bit counter*



APPENDIX F

Examples of Programs in C, C++, and Java

In this appendix we present some examples of programs written in three languages, C, C++, and Java, to give a general idea about the structure of these three common languages. Note that the line number at the left of each program is not part of the program; it is added to make references easier. Also note that the text in color are comments and ignored by the compiler when the program is compiled into machine language.

F.1 PROGRAMS IN C LANGUAGE

In this section, we show three simple programs in C. The goal is not to teach the language; it is to give an idea what programs in C look like.

Example F.1

Program F-1 is the simplest program written in the C language that prints the message 'Hello World!'. It is an example that uses only the sequence construct, which means that the code is executed line by line without branching or repeating some sections.

Program F-1 First program in C

```
1  /*
2    This program shows how we can use only sequence construct
3    to achieve a simple goal.
4  */
5
6  #include <stdio.h>
7
8  int main ()
9  {
```

Program F-1 First program in C (Continued)

```
10      // Statement
11      printf ("Hello World\n");
12      return 0;
13
14  } // End of main
```

Run:
Hello World

Example F.2

Program F-2 is an example of a simple program in C that uses both sequence and branching construct. If a condition is met, the program executes some lines; if not, other lines are executed. We run the program twice to show the two different cases.

Program F-2 Second program in C

```
1   /*
2     This program shows how to make a decision in a program written in C.
3     The program gets an integer and finds if it is divisible by 7.
4   */
5
6   #include <stdio.h>
7
8   int main ()
9   {
10      // Declaration
11      int num;
12      // Statement
13      printf ("Enter an integer: ");
14      scanf ("%d", &num);
15      // Selection
16      if (num % 7 == 0)
17      {
18          printf ("The number %d", num);
19          printf (" is divisible by 7.\n");
20      }
21      else
22      {
```

Program F-2 Second program in C (Continued)

```
23            printf ("The number %d", num);
24            printf (" is not divisible by 7.\n");
25        }
26        return 0;
27    } // End of main
```

```
Run:
Enter an integer: 24
The number 24 is not divisible by 7.
```

```
Run:
Enter an integer: 35
The number 35 is divisible by 7.
```

Example F.3

Program F-3 shows the combination of sequence and repetition construct. We use a loop to repeatedly print a number, but the number is changing in each repetition. We run the program twice: the first time, the limit is 6; the second time the limit is 9.

Program F-3 Third program in C

```
1   /*
2     This program shows how to use repetition in C.
3     The program prints number from 1 to n, in which n is given by the user.
4   */
5
6   #include <stdio.h>
7
8   int main ()
9   {
10    // Declaration
11    int n;
12    int i;
13    // Statement
14    printf ("Enter the upper limit: ");
15    scanf ("%d", &n);
16    // Repetition
17    for (i = 1; i <= n; i++)
18    {
19      printf ("%d\n", i);
```

Program F-3 Third program in C (continued)

```
20    }
21    return 0;
22  } // End of main
```

```
Run:
Enter the upper limit: 6
1
2
3
4
5
6
```

```
Run:
Enter the upper limit: 9
1
2
3
4
5
6
7
8
9
```

F.2 PROGRAMS IN C++ LANGUAGE

In this section, we show how to write the same three previous programs using the C++ language. The point is to show the similarity and differences between the languages. The C language is a procedural language in which there are no classes and objects. On the other hand, C++ is an object-oriented language in which we can have classes and objects.

Example F.4

Program F-4 accomplishes the same purpose as Program F-1, but it is written in C++ instead of C. We can see the main difference in line 14. To print data in C++, we need to use an object. The term *cout* defines an object that is responsible to output data.

Program F-4 First program in C++

```
1  /*
2    This program demonstrates some of the components of a simple
3    program written in C++
4  */
5
6  #include <iostream>
7  #include <iomanip>
8
```

Program F-4 First program in C++ (Continued)

```
 9  using namespace std;
10
11  int main ()
12  {
13    // Statement
14    cout << "Hello World!" << endl;
15    return 0;
16  } // End of main
```

Run:
Hello World

Example F.5

Program F-5 accomplishes the same purpose as Program F-2, but it is written in C++ instead of C. The main difference between this program and its C version is in lines 16, 17, 21, 22, 26, and 27 in which we need to use input object (*cin*) and output objects (*cout*) for input and output.

Program F-5 Second program in C++

```
 1  /*
 2    This program shows how to make a decision in a program written in C++.
 3    The program gets an integer and prints it if it is less than 50.
 4  */
 5
 6  #include <iostream>
 7  #include <iomanip>
 8
 9  using namespace std;
10
11  int main ()
12  {
13    // Declaration
14    int num;
15    // Statement
16    cout << ("Enter an integer: ");
17    cin >> num;
18    // Decision
19    if (num % 7 == 0)
```

Program F-5 Second program in C++ (Continued)

```
20   {
21       cout << "The number " << num;
22       cout << " is divisible by 7." << endl;
23   }
24   else
25   {
26      cout << "The number " << num;
27      cout << " is not divisible by 7." << endl;
28   }
29   return 0;
30 } // End of main
```

```
Run:
Enter an integer: 22
The number 22 is not divisible by 7.
Run:
Enter an integer: 21
The number 21 is divisible by 7.
```

Example F.6

Program F-6 accomplishes the same purpose as Program F-3, but it is written in C++ instead of C. The main difference between this program and its C version is in lines 17, 18, and 23 in which we need to use input object (*cin*) and output objects (*cout*) for input and output.

Program F-6 Third program in C++

```
1  /*
2    This program shows how to use repetition in C++.
3    The program prints number from 1 to n, in which n is given by the user.
4  */
5
6  #include <iostream>
7  #include <iomanip>
8
9  using namespace std;
10
11 int main ()
12   {
13   // Declaration
14   int n;
```

Program F-6 Third program in C++ (Continued)

```
15
16    // Statement
17    cout << "Enter the upper limit: ";
18    cin >> n;
19
20    // loop
21    for (int i = 1; i <= n; i++)
22    {
23       cout << i << endl;
24    }
25
26    return 0;
27 } // End of main
```

```
Run:
Enter the upper limit: 4
1
2
3
4
```

```
Run:
Enter the upper limit: 8
1
2
3
4
5
6
7
8
```

F.3 PROGRAMS IN JAVA LANGUAGE

In this section, we show how to write the same three programs in Java language. The point is to show the similarity and differences between the languages. The first difference we encounter is in the *main* function in C++ and *main* method in Java. In C++, the main function is a stand-alone program; in Java the main method should be part of a class. We call these classes First, Second, and Third respectively in these programs.

Example F.7

Program F-7 accomplishes the same purpose as Program F-4, but it is written in Java instead of C++. We need a class to host the *main* method. Another difference is in line10 where we use a predefined object (System.out) for output.

Program F-7 First program in Java

```
1  /*
2    This program demonstrates some of the components of a simple
3    program written in Java
4  */
5
6  public class First
7  {
8    public static void main (String[] args)
9    {
10         System.out.println ("Hello World!");
11   } // End main
12 } // End class
```

Example F.8

Program F-8 accomplishes the same purpose as Program F-5, but it is written in Java instead of C++. We need a class to host the *main* method. Other differences are in lines 13, 14, 15, 20, 21, 26, and 27 where we use an object of class Scanner for input and a predefined object (System.out) for output.

Program F-8 Second program in Java

```
1  /*
2    This program shows how to make a decision in a program written in Java.
3    The program gets an integer and checks if it is divisible by 7.
4  */
5
6  import java.util.*;
7
8  public class Second
9  {
10   public static void main (String[] args)
11   {
12       // Declaration
13       Scanner input = new Scanner (System.in);
14       System.out.print ("Enter an integer: ");
15       int num = input.nextInt ();
16
```

Program F-8 Second program in Java (Continued)

```
17        // Decision
18        if (num % 7 == 0)
19        {
20             System.out.print ("The number " + num);
21             System.out.println (" is divisible by 7");
22
23        }
24        else
25        {
26             System.out.print ("The number " + num);
27             System.out.println (" is not divisible by 7.");
28
29        }
30    } // End main
31
32 } // End class
```

```
Run:
Enter an integer: 25
The number 25 is not divisible by 7.
```

```
Run:
Enter an integer: 42
The number 42 is divisible by 7.
```

Example F.9

Program F-9 accomplishes the same purpose as Program F-6, but it is written in Java instead of C++. We need a class to host the *main* method. Other differences are in lines 13, 14, 15, and 20 where we use an object of class Scanner for input and a predefined object (System.out) for output.

Program F-9 Third program in Java

```
1  /*
2    This program shows how to use a loop in Java.
3    The program prints number from 1 to n, in which n is given by the user.
4  */
5
6  import java.util.*;
7
8  public class Third
```

Program F-9 Third program in Java (Continued)

```
1  {
2      public static void main (String[] args)
3      {
4          // Statements to get the value of n
5          Scanner input = new Scanner (System.in);
6          System.out.print("Enter the upper limit: ");
7          int n = input.nextInt ();
8
9          // Loop
10         for (int i = 1 ; i <= n; i++)
11         {
12             System.out.println (i);
13         }
14     } // End main
15
16 } // End class
```

```
Run:
Enter the upper limit: 3
1
2
3
```

```
Run:
Enter the upper limit: 7
1
2
3
4
5
6
7
```

APPENDIX G

Mathematical Review

In this appendix we review some mathematical concepts that may help with understanding the topics covered in the book. We first give a brief treatment of exponential and logarithmic functions. We then discuss modular arithmetic. Finally, we give the formulas for the discrete cosine transforms that are used in data compression.

G.1 EXPONENT AND LOGARITHM

In solving some problems in this book, we often need to know how to handle exponential and logarithmic functions. This section briefly reviews these two concepts.

G.1.1 Exponential functions

The exponential function with base a is defined as a^x. If x is an integer, this is interpreted as multiplying a by itself x times. Normally we can use a calculator to find the value of y.

Example G.1

Calculate the value of the following exponential functions.

a. 3^2

b. 5.2^6

Solution

Using the interpretation of exponentiation, we can find:

a. $3^2 = 3 \times 3 = 9$

b. $5.2^6 = 5.2 \times 5.2 \times 5.2 \times 5.2 \times 5.2 \times 5.2 = 19\,770.609664$

Example G.2

Calculate the value of the following exponential functions:

a. $3^{2.2}$

b. $5.2^{6.3}$

Solution

These problems can be done more easily using a calculator—we can find:

a. $3^{2.2} \approx 11.212$

b. $5.2^{6.3} \approx 32424.60$

Three common bases

In the expression a^b, we call a the base and b the exponent. Three bases are very common: base 10, base e, and base 2.

- Base 10 is the base of decimal system. Most calculators have a 10^x key.
- The base used in science and mathematics is the **natural base e**, which has the value 2.71828183... Most calculators have an e^x key. This base is used in science because some phenomena, such as radioactive decay, can be best described using this base.
- The base which we normally need in computer science is base 2. Most calculators have no 2^x key, but we can always use the general x^y key, in which $x = 2$.

Example G.3

Calculate the value of the following exponential functions:

a. e^4

b. $e^{6.3}$

c. $10^{3.3}$

d. $2^{6.3}$

e. 2^{10}

Solution

a. $e^4 \approx 54.60$

b. $e^{6.3} \approx 544.57$

c. $103.3 \approx 1995.26$

d. $26.3 \approx 78.79$

e. $2^{10} = 1024$

Example G.4

In computer science the dominant base is 2. It is a good practice for us to know the powers of 2 for some common exponents. We often need to remember that:

$2^0 = 1$	$2^1 = 2$	$2^2 = 2$	$2^3 = 8$	$2^4 = 16$	$2^5 = 32$	$2^6 = 64$
$2^7 = 128$	$2^8 = 256$	$2^9 = 512$	$2^{10} = 1024$			

Properties of the exponential function

Exponential functions have several properties, and some are useful to us:

1. $a^0 = 1$	2. $a^1 = a$	3. $a^{-x} = 1 / (a^x)$
4. $a^{x+y} = a^x \times a^y$	5. $a^{x-y} = a^x / a^y$	6. $(a^x)^y = a^{x \times y}$

Example G.5

Examples using these properties are:

a. $5^0 = 1$
b. $6^1 = 6$
c. $2^{-4} = 1/2^4 = 1/16 = 0.0625$
d. $2^{5+3} = 2^5 \times 2^3 = 32 \times 8 = 256$
e. $3^{2-3} = 3^2/3^3 = 9 \times 27 = 1/3 \approx 0.33$
f. $(10^4)^2 = 10^{4 \times 2} = 10^8 = 100000000$

G.1.2 Logarithmic function

A logarithmic function is the inverse of an exponential function, as shown below:

$$y = a^x \quad \leftrightarrow \quad x = \log_a y$$

Just as in the exponential function, a is called the **base** of the logarithmic function. In other words, if x is given, we can calculate y by using the exponential function: if y is given, we can calculate x by using the logarithmic function.

Exponential and logarithmic functions are the inverse of each other.

Logarithms facilitate calculations in arithmetic because they convert multiplication to addition and exponentiation to multiplication.

Example G.6

Calculate the value of the following logarithmic functions:

a. $\log_3 9$
b. $\log_2 16$
c. $\log_{10} 0$
d. $\log_2 (-2)$

Solution

We have not yet shown how to calculate the log function in different bases, but we can solve this problem intuitively.

a. Because $3^2 = 9$, $\log_3 9 = 2$, using the fact that the two functions are the inverse of each other.
b. Similarly, because $2^4 = 16$, then $\log_2 16 = 4$.
c. Since there is no finite number x such that $10^x = 0$, then $\log_{10} 0 =$ is undefined or mathematically negative infinity.
d. A negative number in real number mathematics does not have a logarithm. However, in the domain of complex numbers we can have the logarithm of a negative number, but we leave this to books on complex number theory.

Three common bases

As in the case of exponentiation, there are three common bases in logarithms: base 10, base e, and base 2. Logarithms in base e are normally shown as *ln* (natural logarithm), and

logarithms in base 10 as **log** (omitting the base). Not all calculators have logarithms in base 2. We show how to handle this base shortly.

Example G.7

Calculate the value of the following logarithmic functions:

a. log 233
b. ln45

Solution

For these two bases we can use a calculator:

a. $\log 233 \approx 2.367$
b. $\ln 45 \approx 3.81$

Base transformation

We often need to find the value of a logarithmic function in a base other than e or 10. If the available calculator cannot give the result in our desired base, we can use a fundamental property of the logarithm, base transformation, as shown:

$$\log_a y = \frac{\log_b y}{\log_b a}$$

Note that the right-hand side shows two log functions with base b, which is different from the base a at the left-hand side. This means that we can choose a base that is available in our calculator (base b) and find the log of a base that is not available (base a).

Example G.8

Calculate the value of the following logarithmic functions:

a. $\log_3 810$
b. $\log_5 600$
c. $\log_2 1024$
d. $\log_2 600$

Solution

These bases are normally not available on most calculators, but we can use base 10, which is available.

a. $\log_3 810 = \log 810/\log 3 = 2.908/0.477 \approx 6.095$
b. $\log_5 600 = \log 600/\log 5 = 2.778/0.699 \approx 3.975$
c. $\log_2 1024 = \log 1024/\log 2 = 3.01/0.301 = 10$
d. $\log_2 600 = \log 600/\log 2 \approx 2.778/0.301 \approx 9.223$

Example G.9

Base 2 is very common in computer science. Since we know that $\log_{10} 2 \approx 0.301$, it is very easy to calculate (approximately) the log of this base. We find the log of the corresponding

number in base 10 and divide it by 0.310. Alternatively, we can multiply the corresponding log in base 10 by 3.332 (≈ 1/0.301).

a. $\log_2 600 \approx 3.322 \times \log_{10} 600 \approx 3.322 \times 2.778 \approx 9.228$

b. $\log_2 2048 \approx 3.322 \times \log_{10} 2048 \approx 3.322 \times 2.778 = 11$

Properties of logarithmic functions

Logarithmic functions have six useful properties, each related to the corresponding property of the exponential function (mentioned earlier).

1. $\log_a 1 = 0$	4. $\log_a (x \times y) = \log_a x + \log_a y$
2. $\log_a a = 1$	5. $\log_a (x/y) = \log_a x - \log_a y$
3. $\log_a (1/x) = -\log_a x$	6. $\log_a x^y = y \times \log_a x$

Example G.10

Calculate the value of the following logarithmic functions.

a. $\log_3 1$

b. $\log_3 3$

c. $\log_{(1/10)}$

d. $\log_a (x \times y)$ if we know that $\log_a x = 2$ and $\log_a y = 3$

e. $\log_a (x/y)$ if we know that $\log_a x = 2$ and $\log_a y = 3$

f. $\log_2 (1024)$ without using a calculator

Solution

We use the property of log functions to solve the problems:

a. $\log_3 1 = 0$

b. $\log_3 3 = 1$

c. $\log_{(1/10)} = \log 10 - 1 = -\log 10 = -1$

d. $\log_a (x \times y) = \log_a x + \log_a y = 2 + 3 = 5$

e. $\log_a (x / y) = \log_a x - \log_a y = 2 - 3 = -1$

f. $\log_2 (1024) = \log_2 (2^{10}) = 10 \times \log_2 2 = 10 \times 1 = 10$

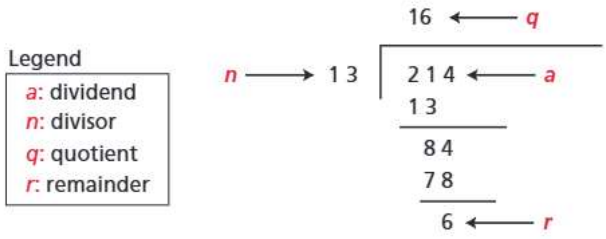
G.2 MODULAR ARITHMETIC

In integer arithmetic, if we divide a by n, we can get q and r. The relationship between these four integers can be shown as $a = q \times n + r$. In this relation, a is called the dividend, q the quotient, n the divisor, and r the residue. Since an operation is normally defined with one single output, this is not an operation. We can call it the **division relation**.

Example G.11

Assume that $a = 214$ and $n = 13$. We can find $q = 16$ and $r = 6$ using the division algorithm we have learned in arithmetic, as shown in Figure G.1.

Figure G.1 *Integer division*

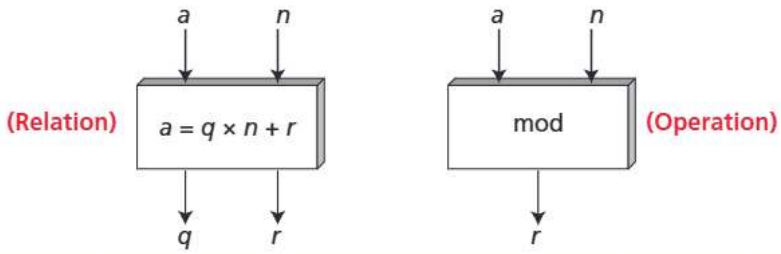


Most computer languages can find the quotient and the residue using language-specific operators. For example, in the C language, the division operator (/) can find the quotient and the modulo operator (%) can find the residue.

G.2.1 The modulo operator

In modular arithmetic we are interested in only one of the outputs, the remainder, r. We don't care about the quotient, q. In other words, we want to know what is the value of r when we divide a by n. This implies that we can change the above relation into a binary operator with two inputs a and n and one output r. The binary operator is then called the **modulo operator** and is shown as *mod*. The second input (n) is called the **modulus** and the output r is called the **residue**. Figure G.2 shows the division relation compared with the modulo operator.

Figure G.2 *Division relation* versus *modulo operator*



The modulo operator (**mod**) takes an integer (a) a modulus (n). The operator creates a residue (r). Although a and r can be any integer, n cannot be 0 because it implies division by zero, which yields an undefined value or infinity. However, in practice we need the value of n to be non-negative. For this reason, the values of a and r should be between 0 and $n - 1$.

Example G.12

A very good example of the use of modular arithmetic is our clock system. The clock is based on modulo 12 arithmetic. However, the integer 12 in our clock should actually be 0 to make it conformant with the modulo arithmetic.

Example G.13

Find the result of the following operations:

a. 28 mod 6
b. 32 mod 12
c. 19 mod 15
d. 7 mod 11

Solution

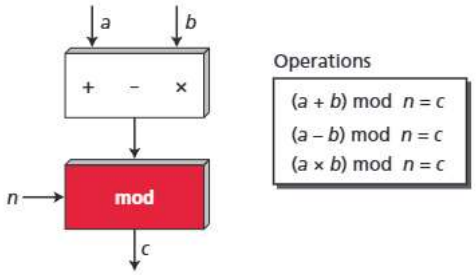
We are looking for the residue r. We can divide a by n and find q and r. We can then disregard q and keep r.

a. Dividing 28 by 6 results in $r = 4$. This means that 28 mod 6 = 4.
b. Dividing 32 by 12 results in $r = 8$. This means that 32 mod 12 = 8.
c. Dividing 19 by 15 results in $r = 4$. This means that 19 mod 15 = 4.
d. Dividing 7 by 11 results in $r = 7$. This means that 7 mod 11 = 4.

G.2.2 Arithmetic operations

The three binary operations (addition, subtraction, and multiplication) that we discussed for integers can also be defined for modulo arithmetic. We may need to normalize the result (apply the mod operation and use the residue) if the result is greater than $n - 1$, as shown in Figure G.3.

Figure G.3 *Three operations in modular arithmetic*



Actually, two sets of binary operators are used here. The first set is one of the binary operators (+, −, ×) and the second is the **mod** operator. We need to use parentheses to emphasize the order of operations. If at any time during calculation we find a negative value for r, the value should be normalized. We need to add the modulus to the result as many times as is necessary to make it positive.

Example G.14

Perform the following operations:

a. Add 7 to 14 using modulo 15.
b. Subtract 11 from 7 using modulo 13.
c. Multiply 11 by 7 using modulo 20.

Solution

The following shows the two steps involved in each case:

(14 + 7) mod 15	→	(21) mod 15 = 6
(7 − 11) mod 13	→	(−4) mod 13 = −4 + 13 = 9
(7 × 11) mod 20	→	(77) mod 20 = 17

Example G.15

Perform the following operations:

a. Add 17 to 27 using modulo 14.
b. Subtract 43 from 12 using modulo 13
c. Multiply 123 by −10 using modulo 19.

Solution

Note that the integers in these examples are sometimes out of the range of 0 to $n - 1$. We can normalize them either before applying the operation or after applying the operation. We show the second choice, you try the first choice. The result should be the same.

(17 + 27) mod 14	→	(44) mod 14 = 2
(12 – 43) mod 15	→	(–31) mod 15 = –1 + 15 = 14
(123 × –10) mod 20	→	(–1230) mod 19 = –14 + 19 = 5

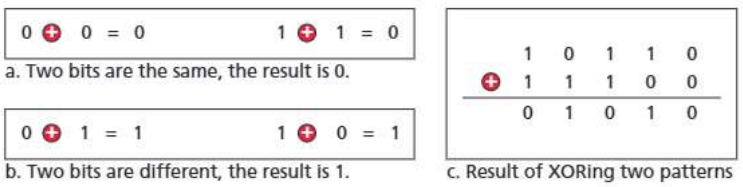
Modulo-2 arithmetic

Modulo-2 arithmetic is of particular interest. As the modulus is 2, we can use only the values 0 and 1. Operations in this arithmetic are very simple. The following shows how we can add or subtract 2 bits:

Adding:	(0 + 0) mod 2 = 0	(0 + 1) mod 2 = 1
	(1 + 0) mod 2 = 1	(1 + 1) mod 2 = 0
Subtracting:	(0 − 0) mod 2 = 0	(0 − 1) mod 2 = 1
	(1 − 0) mod 2 = 1	(1 − 1) mod 2 = 0

Notice particularly that addition and subtraction give the same results. In this arithmetic we use the XOR (exclusive OR) operation for both addition and subtraction. The result of an XOR operation is 0 if two bits are the same and 1 if two bits are different. Figure G.4 shows this operation.

Figure G.4 *XORing of two single bits or two words*



a. Two bits are the same, the result is 0.
b. Two bits are different, the result is 1.
c. Result of XORing two patterns

G.3 DISCRETE COSINE TRANSFORM

In this section we give the mathematical background for the discrete cosine and inverse discrete cosine transforms that are used for data compression, as discussed in Chapter 15.

G.3.1 The discrete cosine transform

The discrete cosine transform (DCT) changes each block of 64 pixels so that the relative relationship between pixels is preserved but redundancies are revealed. The formula follows. $P(x, y)$ defines one particular value in the picture block, while $T(m, n)$ defines one value in the transformed block.

$$T(m, n) = 0.25\, c(m)\, c(n) \sum_{x=0}^{7} \sum_{y=0}^{7} P(x, y) \cos\left[\frac{(2x+1)m\pi}{16}\right] \cos\left[\frac{(2y+1)n\pi}{16}\right]$$

$$\text{where} \qquad c(i) = \begin{cases} \dfrac{1}{(2)^{1/2}} & \text{if } i = 0 \\ 1 & \text{otherwise} \end{cases}$$

G.3.2 The inverse discrete cosine transform

The inverse transform is used to create the $P(x, y)$ table from the $T(m, n)$ table.

$$P(x, y) = 0.25\, c(x)\, c(y) \sum_{m=0}^{7} \sum_{n=0}^{7} T(m, n) \cos\left[\frac{(2m+1)x\pi}{16}\right] \cos\left[\frac{(2n+1)y\pi}{16}\right]$$

$$\text{where} \qquad c(i) = \begin{cases} \dfrac{1}{(2)^{1/2}} & \text{if } i = 0 \\ 1 & \text{otherwise} \end{cases}$$

Example G.16

Evaluate $T(0, 0)$ and $T(0, 1)$ if $P(x, y) = 20$ for all x and y.

Solution

Using sum-to-product identity $\cos x + \cos y = 2[\cos(x + y)/2]\ [\cos(x - y)/2]$, we can show that the sum of all cosine terms is 0.

APPENDIX H

Error Detection and Correction

When data is transferred from one place to another, or moved from one device to another, the accuracy of the data must be checked. For most applications, a system must guarantee that the data received is identical to the data transmitted. Some applications, on the other hand, can tolerate a small level of error. For example, random errors in audio or video transmissions may be tolerable, but when we transfer text, we expect a very high level of accuracy. We only discuss error in transition: errors due to data corruption in storage are treated in the same way.

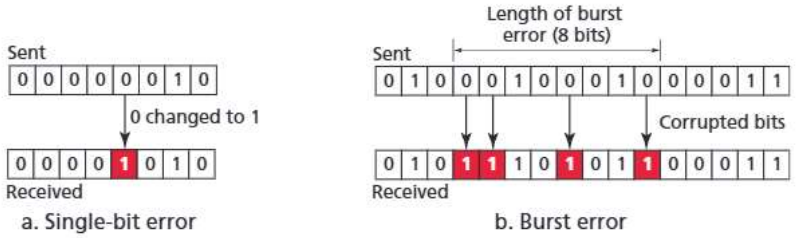
H.1 INTRODUCTION

We first discuss some issues related to error detection and correction.

H.1.1 Types of errors

Whenever bits flow from one place to another they are subject to unpredictable changes because of interference in the transmission medium, such as crosstalk, external electromagnetic fields, and so on. This is illustrated by Figure H.1.

Figure H.1 *Single-bit* versus *burst errors*



In a **single-bit error**, a 0 is changed to a 1 or a 1 to a 0. In a **burst error** multiple bits are changed. The term **single-bit error** means that only 1 bit of a given data unit, such as a byte, character, or packet, is changed from 1 to 0 or from 0 to 1. The term **burst error** means that two or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

H.1.2 Redundancy

The central concept in correcting errors is **redundancy**. To be able to correct errors, we need to send extra bits with our data. These redundant bits are added by the sender and removed by the receiver. Their presence allows the receiver to correct corrupted bits.

To correct errors, we need to send extra (redundant) bits with data.

H.1.3 Detection *versus* correction

The correction of errors is more difficult than the detection. In **error detection**, we are looking only to see if an error has occurred. The answer is a simple yes or no. We are not even interested in the number of errors: a single-bit error is the same for us as a burst error.

In **error correction**, we need to know the exact number of bits that are corrupted—and more importantly, their location in the message. The number of errors and the size of the message are important factors. If we need to correct a single error in an 8-bit data unit, we need to consider eight possible error locations: if we need to correct two errors in a data unit of the same size, we need to consider 28 (7 + 6 + … + 1) possibilities. You can imagine the receiver's difficulty in finding ten errors in a data unit of 1000 bits.

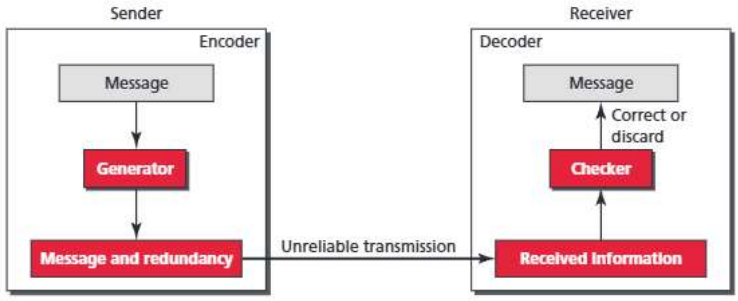
H.1.4 Forward error correction *versus* retransmission

There are two main methods of error correction. **Forward error correction** is the process in which the receiver tries to guess the message by using redundant bits. This is possible, as we will see later, if the number of errors is small. Correction by **retransmission** is a technique in which the receiver detects the occurrence of an error and asks the sender to resend the message. Re-sending is repeated until a message arrives that the receiver believes is error-free: usually, not all errors can be detected.

H.1.5 Coding

Redundancy is achieved through various coding schemes. The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits. The receiver checks the relationships between the two sets of bits to detect or correct the errors. The ratio of redundant bits to data bits and the robustness of the process are important factors in any coding scheme. Figure H.2 shows the general idea of coding.

Figure H.2 *The structure of an encoder and decoder*



We can divide coding schemes into two broad categories: **block coding** and **convolution coding**. In this appendix, we concentrate on block coding: convolution coding is more complex and beyond the scope of this book.

Block coding uses modular arithmetic, as discussed in Appendix G.

We only concentrate on block codes:
we leave convolution codes to advanced texts.

H.2 BLOCK CODING

In block coding we divide a message into blocks, each of k bits, called **datawords**. We add r redundant bits to each block to make the length $n = k + r$. The resulting n-bit blocks are called **codewords**. How the extra r bits are chosen or calculated is something we discuss later. For the moment, it is important to know that we have a set of datawords, each of size k, and a set of codewords, each of size of n.

With k bits, we can create a combination of 2^k datawords: with n bits, we can create a combination of 2^n codewords. Since $n > k$, the number of possible codewords is larger than the number of possible datawords. The block coding process is one-to-one: the same dataword is always encoded as the same codeword. This means that we have $2^n - 2^k$ codewords that are not used. We call these codewords **invalid** or **illegal**. Figure H.3 shows the situation.

Figure H.3 *Datawords and codewords in block coding*

Example H.1

Let us assume our message is made up of a single block of 8 bits ($k = 8$). There are $2^8 = 256$ possible combination of datawords. If we add two redundant bits ($r = 1$),

then each possible codeword is 10 bits ($n = 10$) and the total number of possible codewords is $2^{10} = 1024$. This means that we have $1024 - 256 = 768$ codewords are invalid. If one of these invalid codewords is received, the receiver knows that the codeword is corrupted.

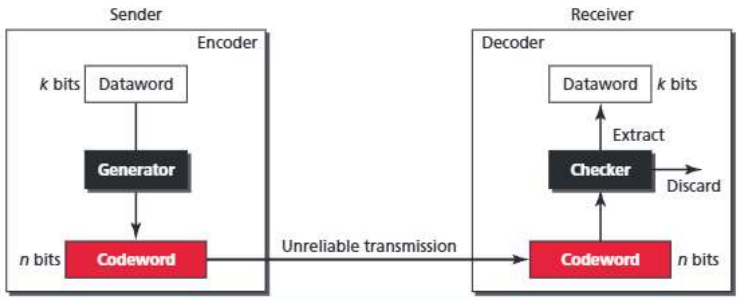
H.2.1 Error detection

How can errors be detected by using block coding? If the following two conditions are met, the receiver detects a change in the original codeword.

1. The receiver has (or can find) a list of valid codewords.
2. The original codeword has changed to an invalid one.

Figure H.4 shows the role of block coding in error detection.

Figure H.4 *The process of error detection in block coding*



The sender creates codewords out of datawords by using a generator that applies the rules and procedures of encoding (discussed later). Each codeword sent to the receiver may change during transmission. If the received codeword is the same as one of the valid codewords, the word is accepted and the corresponding dataword is extracted for use. If the received codeword is not valid, it is discarded.

However, if the codeword is corrupted during transmission, but the received word still matches a valid codeword, the error remains undetected. This type of coding can therefore detect only single errors: two or more errors in the same codeword may remain undetected.

Example H.2

Let us assume that $k = 2$ and $n = 3$. Table H.1 shows the list of datawords and defined codewords, which is agreed between the sender and the receiver. Later we will see how to derive a codeword from a dataword.

Table H.1 A code for error detection (Example H.2)

Datawords	*Codewords*
00	000
01	011
10	101
11	110

Assume that the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

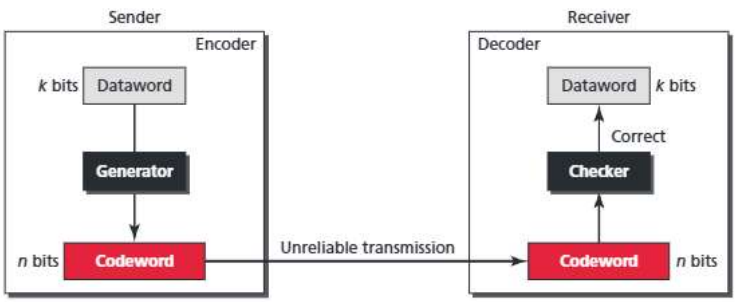
1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.
2. The codeword is corrupted during transmission, and 111 is received—that is, the leftmost bit is corrupted. This is not a valid codeword, so it is discarded.
3. The codeword is corrupted during transmission, and 000 is received—that is, the right two bits are corrupted. This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

An error-detecting code can detect only the types of errors for which it is designed: other types of errors may remain undetected.

H.2.2 Error correction

Error correction is much more difficult than error detection. In error detection, the receiver needs to know only that the received codeword is invalid: in error correction, the receiver needs to find (or guess) the original codeword sent. We need more redundant bits for error correction than for error detection. Figure H.5 shows the role of block coding in error correction. We can see that the idea is the same as error detection, but the generator and checker functions are much more complex.

Figure H.5 *The structure of encoder and decoder in error correction*



Example H.3

Let us add more redundant bits to Example H.2 to see if the receiver can correct an error without knowing what was actually sent. We add three redundant bits to the 2-bit dataword to make 5-bit codewords. Again, later we will show how we choose the redundant bits. For the moment let us concentrate on the error correction concept. Table H.2 shows the datawords and codewords.

Table H.2 A code for error correction (Example H.3)

Dataword	*Codeword*	*Dataword*	*Codeword*
00	00000	10	10101
01	01011	11	11110

Assume the dataword is 01. The sender consults the table (or uses an algorithm) to create the codeword 01011. The codeword is corrupted during transmission, and 01001 is received—an error in the second bit from the right. First, the receiver finds that the received codeword is not in the table. This means an error has occurred. (Detection must come before correction.) The receiver, assuming that only 1 bit is corrupted, uses the following strategy to guess the correct dataword.

1. Comparing the received codeword with the first codeword in the table (01001 *versus* 00000), the receiver decides that the first codeword is not the one that was sent because there are two different bits.
2. By the same reasoning, the original codeword cannot be the third or fourth one in the table.
3. The original codeword must be the second one in the table, because this is the only one that differs from the received codeword by 1 bit. The receiver replaces 01001 with 01011 and consults the table to find the dataword 01.

H.3 LINEAR BLOCK CODES

Almost all block codes used today belong to a subset called **linear block codes**. The use of nonlinear block codes for error detection and correction is not as widespread, because their structure makes theoretical analysis and implementation difficult. We therefore concentrate on linear block codes.

The formal definition of linear block codes requires a knowledge of abstract algebra (particularly Galois fields) which is beyond the scope of this book. We therefore give an informal definition. For our purposes, a linear block code is a code in which the exclusive OR (modulo-2 addition, discussed in Appendix G) of two valid codewords creates another valid codeword.

In a linear block code, the exclusive OR (XOR) of any two valid codewords creates another valid codeword.

Example H.4

Let us see if the two codes we defined in Table H.1 and Table H.2 belong to the class of linear block codes.

1. The scheme in Table H.1 is a linear block code because the result of XORing any codeword with any other codeword is a valid codeword. For example, XORing of the second and third codewords creates the fourth one.
2. The scheme in Table H.2 is also a linear block code. We can create all four codewords by XORing two other codewords.

H.3.1 Some linear block codes

Let us now show some linear block codes. These codes are trivial because we can easily find the encoding and decoding algorithms and check their performances.

Simple parity-check code

Perhaps the most familiar error-detecting code is the **simple parity-check code**. In this code, a k-bit dataword is changed to an n-bit codeword, where $n = k + 1$. The extra bit, called the **parity bit**, is added to a pre-defined position. It is selected to make the total number of 1s in the codeword even. Although some implementations specify an odd number of 1s, we discuss the even number case.

A simple parity-check code is a single-bit error-detecting code in which $n = k + 1$.

Our first code (Table H.3) is a parity-check code with $k = 2$ and $n = 3$. The code in Table H.3 is also a parity-check code with $k = 4$ and $n = 5$.

Table H.3 Simple parity-check code C(5, 4)

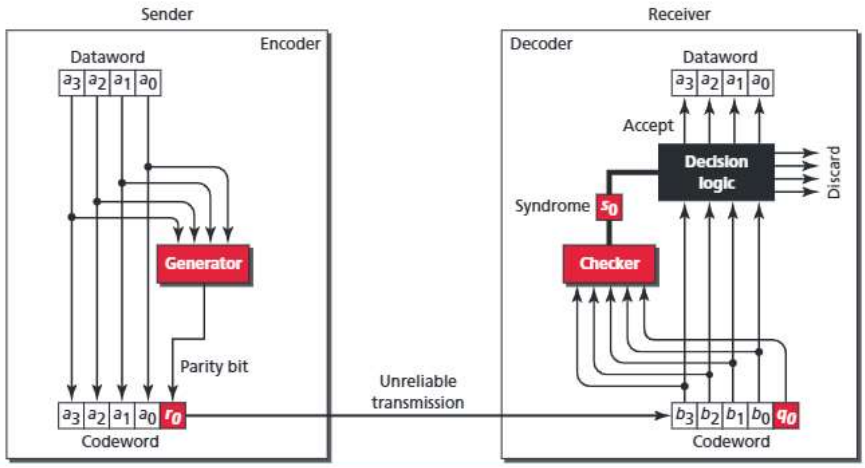
Datawords	*Codewords*	*Datawords*	*Codewords*
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Figure H.6 shows a possible structure of an encoder (at the sender) and a decoder (at the receiver).

The encoder uses a generator that takes a copy of a 4-bit dataword (a_0, a_1, a_2, and a_3) and generates a parity bit r_0. The dataword bits and the parity bit create the 5-bit codeword. The parity bit that is added makes the number of 1s in the codeword even. This is normally done by adding the 4 bits of the dataword (modulo-2): the result is the parity bit. In other words:

$$r_0 = a_3 + a_2 + a_1 + a_0 \qquad \text{(modulo-2)}$$

Figure H.6 *Encoder and decoder for simple parity-check code*



If the number of 1s is even, the result is 0: if the number of 1s is odd, the result is 1. In both cases, the total number of 1s in the codeword is even.

The sender sends the codeword, which may be corrupted during transmission. The receiver receives a 5-bit word. The checker at the receiver does the same thing as the generator in the sender with one exception: the addition is done over all five bits. The result, which is called the **syndrome**, is just 1 bit. The syndrome is 0 when the number of 1s in the received codeword is even; otherwise, it is 1.

$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \qquad \text{(modulo-2)}$$

A syndrome is the output of a checking process that is fed into the decision logic of a receiver in order to decide what to do with the data portion of the code word. The decision may be to accept it, to reject it, or (for correcting code) to modify it before accepting it. In this case, the syndrome is passed to the **decision logic analyzer**. If there is no error in the codeword, the syndrome is 0 and the decision logic accepts the data portion of the codeword as the actual dataword. If the syndrome is 1, there must be an error in the

codeword, and so the decision logic discards the data portion of the codeword: the dataword is not created.

Example H.5

Let us look at some transmission scenarios. Assume that the sender sends the dataword 1011. The parity bit is (1 + 0 + 1 + 1) mod 2 = 1, which is appended to the right of the dataword. The codeword created from this dataword is therefore 10111, which is sent to the receiver. We examine five cases:

1. No error occurs: the received codeword is 10111. The syndrome is 0. The dataword 1011 is created.
2. A single-bit error changes a_1. The received codeword is 10011. The syndrome is 1. No dataword is created.
3. A single-bit error changes the parity bit r_0. The received codeword is 10110. The syndrome is 1. No dataword is created. Note that although none of the dataword bits are corrupted, no dataword is created because the code is not sophisticated enough to show the position of the corrupted bit.
4. An error changes r_0 and a second error changes a_3. The received codeword is 00110. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here the dataword is wrongly created due to the syndrome value: the simple parity-check decoder cannot detect an even number of errors. The errors cancel each other out and give the syndrome a value of 0.
5. Three bits—a_3, a_2, and a_1—are changed by errors. The received codeword is 01011. The syndrome is 1. The dataword is not created. This shows that the simple parity check, guaranteed to detect a single error, can also find any odd number of errors.

A simple parity-check code can detect an odd number of errors.

Hamming codes

Hamming codes are a subset of linear block codes that follow two criteria:

$$n = k + r \quad \text{and} \quad n = 2^r - 1$$

in which k is the number of bits in dataword, r is the number of redundant bits, and n is the number of bits in the codeword. These codes can detect up to $r - 1$ bits of error and can correct up to $(r - 1) / 2$ bits of error.

Example H.6

A code with $k = 4$, $r = 3$, and $n = 7$ satisfies the two conditions of a Hamming code, because we have $7 = 4 + 3$ and $7 = 2^3 - 1$. This code can detect only $(3 - 1) = 2$ bits of error and correct only $(3 - 1)/2 = 1$ bit of error.

The theory of Hamming codes in general is beyond the scope of this book. For more information, see *Data Communication and Networking*, by Behrouz Forouzan, McGraw-Hill, New York, 2006. In the next section, we discuss a subset of Hamming code called cyclic codes.

H.4 CYCLIC CODES

Cyclic codes are special linear block codes with one extra property. In a **cyclic code**, if a codeword is cyclically shifted (rotated), the result is another codeword. For example, if 1011000 is a codeword and we cyclically left-shift it, then 0110001 is also a codeword.

H.4.1 Cyclic redundancy check

We can create cyclic codes to correct errors. However, the theoretical background required is beyond the scope of this appendix. In this section, we simply discuss a category of cyclic codes called the **cyclic redundancy checks (CRC)** that are used in networks such as LANs and WANs.

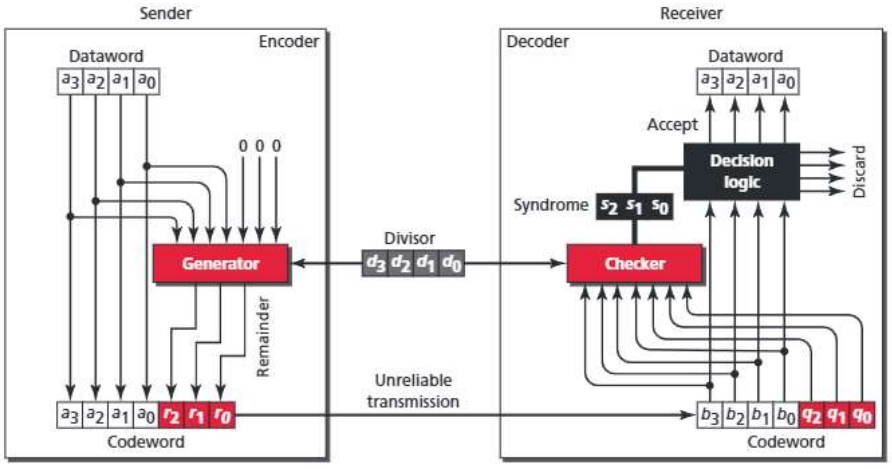
Table H.4 shows an example of a CRC code. We can see both the linear and cyclic properties of this code.

Table H.4 A CRC code with $k = 4$, $n = 7$, and $r = 3$

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

Figure H.7 shows one possible design for the encoder and decoder.

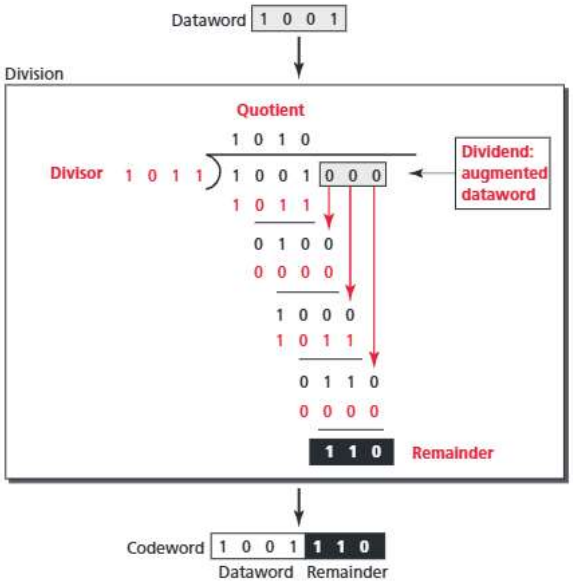
Figure H.7 *CRC encoder and decoder*



In the encoder in Figure H.7, the dataword has k bits (4 here) and the codeword has n bits (here 7). The size of the dataword is augmented by adding $n - k$ (here 3) 0s to the right-hand side of the word. The n-bit result is fed into the generator. The generator uses a predefined divisor of size $n - k + 1$ (here 4). The generator divides the augmented dataword by the divisor using modulo-2 division. The quotient of the division is discarded: the remainder ($r_2 r_1 r_0$) is appended to the dataword to create the codeword.

The decoder receives the codeword, which could be corrupted. A copy of all n bits is fed to the checker, which is a replica of the generator. The remainder produced by the checker is a syndrome of $n - k$ (here 3) bits, which is fed to the decision logic analyzer. The analyzer has a simple function: if the syndrome bits are all 0s, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error), otherwise, the 4 bits are discarded (error).

Figure H.8 *Division in a CRC encoder*



Encoder

Let us take a closer look at the encoder. The encoder takes the dataword and augments it with $n - k$ number of 0s. It then divides the augmented dataword by the divisor, as shown in Figure H.8.

Note that this is not the regular binary **division**—obviously the result of dividing 72 by 11 is not the quotient of 10 and the remainder of 6. This is binary division in modulo 2 arithmetic, as we discussed in Appendix G. In this division, adding and subtracting is the same (it is the XOR operation discussed in Appendix E), which means that we do not subtract, but add. A better explanation of this division is that we treat the binary word as a polynomial with a coefficient in modulo 2 arithmetic—only 0 or 1. For more information,

we refer the interested reader to *finite field theory* (*Galois fields*) and books on error detection and correction.

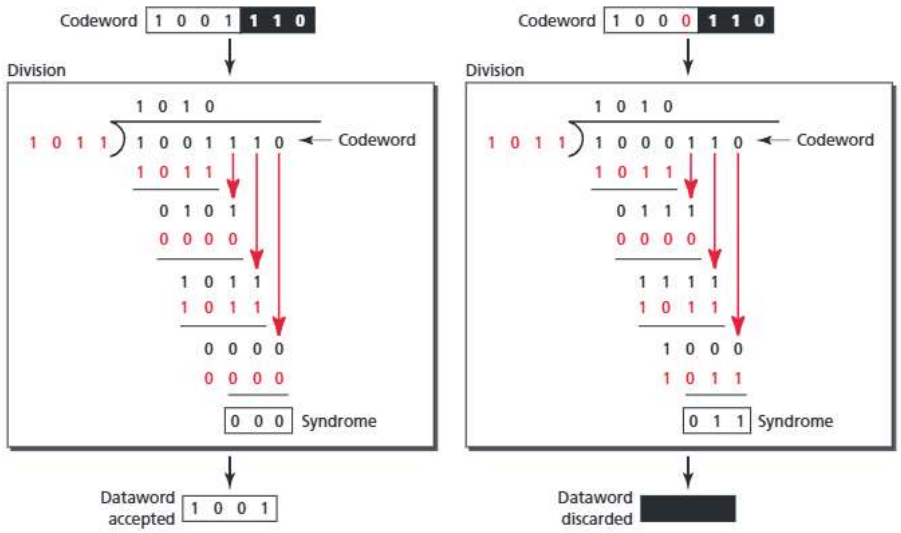
In each step, a copy of the divisor is XORed with the 4 bits of the dividend. The result of the XOR operation (remainder) is 3 bits (in this case), which is used for the next step after 1 extra bit is carried down—see Figure H.8—to make it 4 bits long. There is one important point we need to remember in this type of division. If the leftmost bit of the dividend (or the part used in each step) is 0, the corresponding bit in the quotient is 0.

When there are no bits left to pull down, we have a result. The 3-bit remainder forms the check bits (r_2, r_1, and r_0). They are appended to the dataword to create the codeword. Note also that we are not interested in the quotient, as only the remainder is used in cyclic codes.

Decoder

The codeword can change during transmission. The decoder does the same division process as the encoder. The remainder of the division is the syndrome. If the syndrome is all 0s, there is no error: the dataword is separated from the received codeword and accepted. Otherwise, everything is discarded. Figure H.9 shows two cases. The left-hand figure shows the value of syndrome when no error has occurred: the syndrome is 000. The right-hand part of the figure shows the case in which there is a single error: the syndrome is not all 0s (it is 011).

Figure H.9 *Division in the CRC decoder for two cases*



Divisor

You may be wondering how the divisor 1011 is chosen. This needs abstract algebra and theory of finite field to explain, which we leave this to books specialized in this area.

H.4.2 Performance of cyclic codes

We have seen that cyclic codes have a very good performance in detecting single-bit errors, double errors, an odd number of errors, and burst errors. They can easily be implemented in hardware and software. They are especially fast when implemented in hardware. This has made cyclic codes a good candidate for many networks.

H.5 CHECKSUM

The last error detection method we discuss here is called the **checksum**. The checksum is used in the Internet by several protocols. Like linear and cyclic codes, the checksum is based on the concept of **redundancy**.

H.5.1 Checksum concept

The concept of the checksum is not difficult. Let us illustrate it with a few examples.

Example H.7

Suppose our data is a list of five 4-bit numbers that we want to send to some destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the data is not accepted.

Example H.8

We can make the job of the receiver easier if we send the negative (complement) of the sum, called the *checksum*. In this case, we send (7, 11, 12, 0, 6, −36). The receiver can add all the numbers received (including the checksum). If the result is 0, it assumes no error, otherwise there is an error.

H.5.2 One's complement

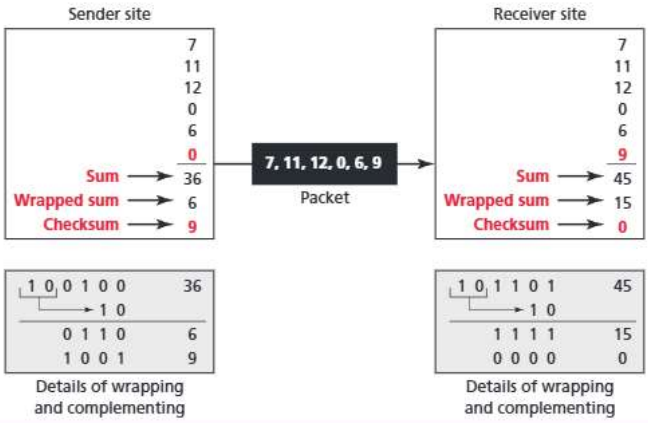
The previous example has one major drawback. Our data can be written as 4-bit words (they are all less than 15) except for the checksum. One solution is to use **one's complement** arithmetic. as discussed in Chapter 3.

Example H.9

Let us redo Example H.8 using one's complement arithmetic. Figure H.10 shows the process at the sender and at the receiver.

https://sanet.st/blogs/polatebooks/

Figure H.10 *Example H.9*



The sender initializes the checksum to 0 and adds all data items and the checksum (the checksum is considered as one data item and is shown in color). The result is 36. However, 36 cannot be expressed in 4 bits. The extra two bits are wrapped and added with the sum to create the wrapped sum value 6. In the figure, we have shown the details in binary. The sum is then complemented, resulting in the checksum value 9 (15 − 6 = 9). The sender now sends six data items to the receiver, including the checksum 9. The receiver follows the same procedure as the sender. It adds all data items (including the checksum): the result is 45. The sum is wrapped and becomes 15. The wrapped sum is complemented and becomes 0. Since the value of the checksum is 0, this means that the data is not corrupted. The receiver drops the checksum and keeps the other data items. If the checksum is not zero, the entire packet is dropped and must be retransmitted.

H.5.3 Internet checksum

Traditionally, the Internet (IP protocol) has used a 16-bit checksum. The sender and receiver use the following procedures:

Sender side

- A 16-bit checksum is set to zero and added to the message.
- The new message is divided into 16-bit words.
- All words are added using one's complement addition.
- The sum is complemented and replaces the previous checksum.

Receiver side

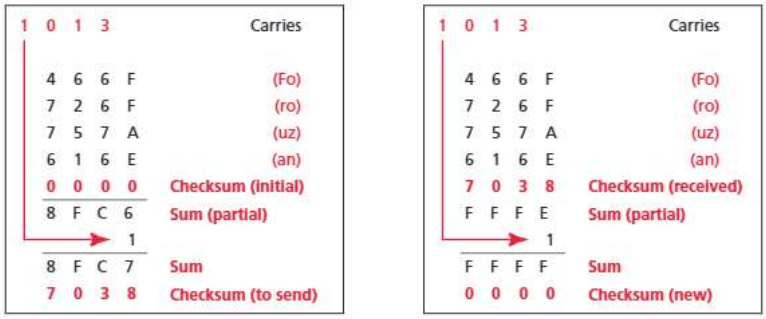
- The received message (including the checksum) is divided into 16-bit words.
- All words are added using one's complement addition.

- The sum is complemented.
- If the complemented sum is 0, the message is accepted, otherwise it is rejected.

Example H.10

Let us calculate the checksum for a text word of eight characters ('Forouzan') as shown in Figure H.11.

Figure H.11 *An example of a checksum calculation*



a. Checksum at the sender site b. Checksum at the receiver site

The text needs to be divided into 2-byte (16-bit) words. We use ASCII encoding (see Appendix A) to change each byte to a two-digit hexadecimal number. For example, 'F' is represented as $(46)_{16}$ and 'o' is represented as $(6F)_{16}$. In Figure H.11.a, the value of the partial sum for the first column is $(36)_{16}$. We keep the rightmost digit (6) and insert the leftmost digit (3) as the carry in the second column. The process is repeated for each column. The checksum is calculated and transmitted with the data to the receiver. The receiver performs the same operations, Figure H.11.b. If there is any corruption, the checksum recalculated by the receiver is not all 0s.

Performance

The traditional checksum uses a small number of bits (16) to detect errors in a message of any size (sometimes thousands of bits). However, it is not as strong as the CRC in its error-checking capability. For example, if the value of one word is incremented and the value of another word is decremented by the same amount, the two errors cannot be detected because the sum and checksum remain the same. Also if the values of several words are incremented but the total change is a multiple of 65 535 ($2^{16}-1$), the sum and the checksum remain the same and the error goes undetected.

APPENDIX I

Addition and Subtraction for Sign-and-Magnitude Integers

In Chapter 4 we showed how to operate on data. Addition and subtraction for sign-and-magnitude integers are a little more involved and are discussed in this appendix.

I.1 OPERATIONS ON INTEGERS

Addition and subtraction for integers in sign-and-magnitude representation look very complex. We have four different combination of signs (two signs, each of two values) for addition, and four different conditions for subtraction. This means that we need to consider eight different situations. However, if we examine the signs in more detail, we can reduce the number of cases, as shown in Figure I-1.

Let us first explain the diagram:

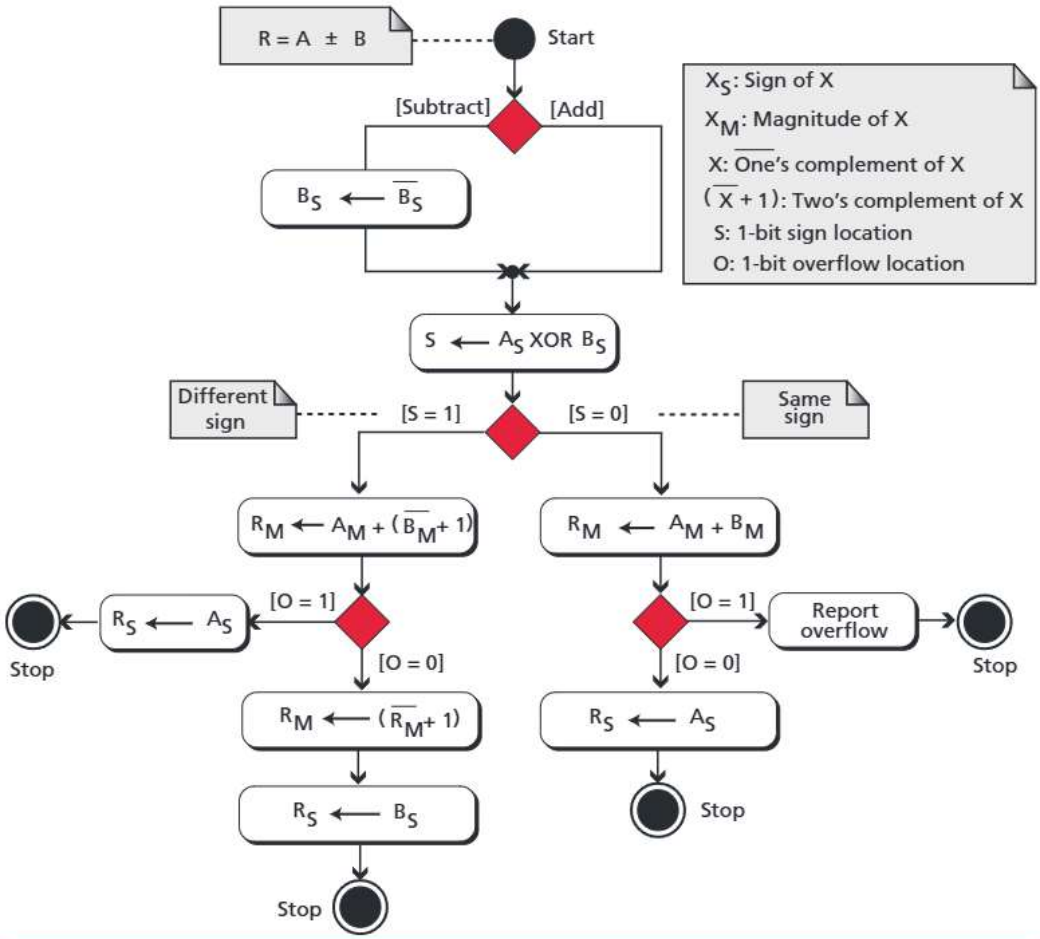
1. We check the operation. If the operation is subtraction, we change the sign of the second integer (B). This means we now only have to worry about the addition of two signed integers.
2. We apply the XOR operation to the two signs. If the result (stored in temporary location S) is 0, it means that the signs are the same (either both signs are positive or both are negative).
3. If the signs are the same, $R = \pm (A_M + B_M)$. We need to add the magnitude and the sign of the result is the common sign. So, we have:

$$R_M = (A_M) + (B_M) \text{ and } R_S = A_S$$

where the subscript M means magnitude and subscript S means sign. In this case, however, we should be careful about the overflow. When we add the two magnitudes, an overflow may occur that must be reported and the process aborted.

4. If the signs are different, $R = \pm (A_M - B_M)$. So we need to subtract B_M from A_M and then make a decision about the sign. Instead of subtracting bit by bit, we take the two's complement of the second magnitude (B_M) and add them. The sign of the result is the sign of the integer with larger magnitude.
   a. It can be shown that if $A_M \geq B_M$, there is an overflow and the result is a positive number. Therefore, if there is an overflow, we discard the overflow and let the sign of the result be the sign of A.
   b. It can be shown that if $A_M < B_M$, there is no overflow, but the result is a negative number. So if there is no overflow, we make the two's complement of the result and let the sign of the result be the sign of B.

Figure I.1 *Addition and subtraction of integers in sign-and-magnitude format*



Example I.1

Two integers A and B are stored in sign-and-magnitude format (we have separated the sign from the magnitude for clarity). Show how B is added to A.

$$A = (0\ 0010001)^2 \quad B = (0\ 0010110)^2$$

Solution

The operation is adding: the sign of B is not changed. Since $S = A_S$ XOR $B_S = 0$, $R_M = A_M + B_M$ and $R_S = A_S$. There is no overflow.

		No overflow				1					Carry
A_S	0			0	0	1	0	0	0	1	A_M
B_S	0		+	0	0	1	0	1	1	0	B_M
R_S	0			0	1	0	0	1	1	1	R_M

Checking the result in decimal, (+17) + (+22) = (+39).

Example I.2

Two integers A and B are stored in sign-and-magnitude format. Show how B is added to A.

$$A = (0\ 0010001)_2 \quad B = (1\ 0010110)_2$$

Solution

The operation is adding: the sign of B $S = A_S$ XOR $B_S = 1$; $R_M = A_M + (\overline{B}_M + 1)$. is not changed. Since there is no overflow, we need to take the two's complement of R^M. The sign of R is the sign of B.

		No overflow								Carry
A_S	0		0	0	1	0	0	0	1	A_M
B_S	1	+	1	1	0	1	0	1	0	$(\overline{B}_M + 1)$
			1	1	1	1	0	1	1	R_M
R_S	1		0	0	0	0	1	0	1	$R_M = (\overline{R}_M + 1)$

Checking the result in decimal, (+17) + (−22) = (−5).

Example I.3

Two integers A and B are stored in sign-and-magnitude format. Show how B is subtracted from A.

$$A = (1\ 1010001)^2 \quad B = (1\ 0010110)^2$$

Solution

The operation is subtracting: $S_B = S_B S. = A_S$ XOR $B_S = 1$, $R_M = A_M + (\overline{B}_M + 1)$. Since there is an overflow, the value of R_M is final. The sign of R is the sign of A.

		Overflow →	1							
										Carry
A_S	**1**			1	0	1	0	0	0	1 A_M
B_S	**1**		+	1	1	0	1	0	1	0 $(\overline{B}_M + 1)$
R_S	**1**			0	1	1	1	0	1	1 R_M

Checking the result in decimal, $(-81) - (-22) = (-59)$.

APPENDIX J

Addition and Subtraction for Reals

In Chapter 4 we showed how to operate on data. Addition and subtraction for reals are a little more involved and are discussed in this appendix.

J.1 OPERATIONS ON REALS

All arithmetic operations such as addition, subtraction, multiplication, and division can be applied to reals stored in floating-point format. Multiplication of two reals involves multiplication of two integers in sign-and-magnitude representation. Division of two reals involves division of two integers in sign-and-magnitude representations. Since we did not discuss the multiplication or division of integers in sign-and-magnitude representation, we will not discuss the multiplication and division of reals, and only show addition and subtraction for reals.

Addition and subtraction of real numbers stored in floating-point numbers is reduced to addition and subtraction of two integers stored in sign-and-magnitude (combination of sign and mantissa) after the alignment of decimal points. Figure J.1 shows a simplified version of the procedure (there are some special cases that we have ignored).

The simplified procedure works as follows:

1. If any of the two numbers (A or B) is zero, we let the result be 0 and stop.
2. If the operation is subtraction, we change the sign of the second number (B) to simulate addition.
3. We denormalize both numbers by including the hidden 1 in the mantissa and incrementing the exponents. The mantissa is now is treated as an integer.
4. We then align the exponents, which means that we increment the lower exponent and shift the corresponding mantissa until both have the same exponent. For example, if we have:

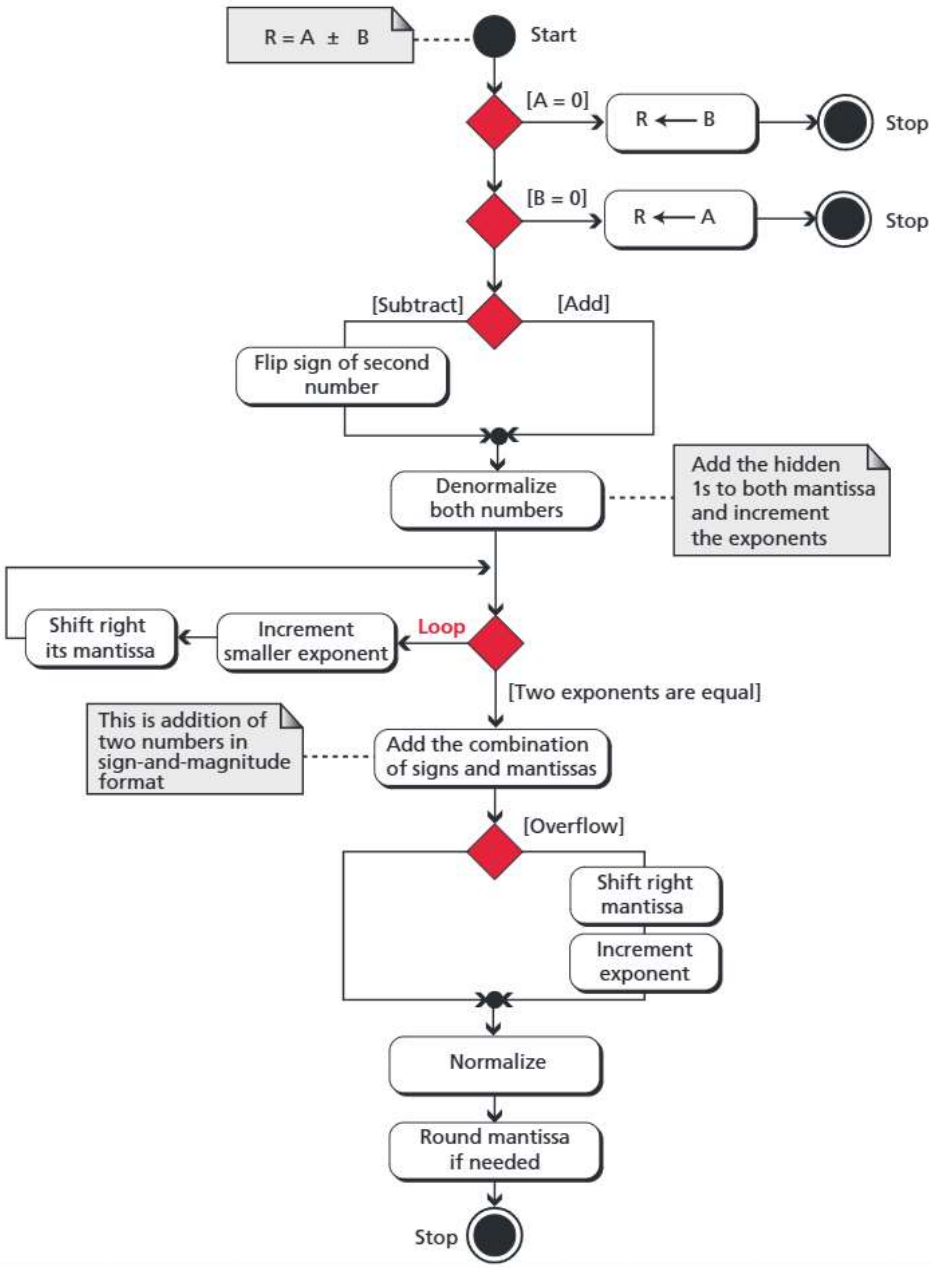
$$1.11101 \times 2^4 + 1.01 \times 2^2$$

we need to make both exponents 4:

$$1.11101 \times 2^4 + 0.00101 \times 2^4$$

5. Now, we treat the combination of the sign and mantissa of each number as an integer in sign-and-magnitude format. We add these two integers, as explained earlier in appendix I
6. Finally, we normalized the number again to 1.000010×2^5.

Figure J.1 *Addition and subtraction of reals in floating-point format*



Example J.1

Show how the computer finds the result of (+5.75) + (+161.875) = (+167.625).

Solution

As we saw in Chapter 3, these two numbers are stored in floating-point format, as shown below, but we need to remember that each number has a hidden 1 (which is not stored, but assumed):

	S	E	M
A	0	10000001	01110000000000000000000
B	0	10000110	01000011110000000000000

The first few steps in the UML diagram (Figure J.1) are not needed. We move to denormalization and denormalize the numbers by adding the hidden 1s to the mantissa and incrementing the exponent. Now both denormalized mantissas are 24 bits and include the hidden 1s. They should be stored in a location that can hold all 24 bits. Each exponent is incremented:

	S	E	Denormalized M
A	0	10000010	101110000000000000000000
B	0	10000111	101000011110000000000000

Now we need to align the mantissas. We need to increment the first exponent and shift its mantissa to the right. We change the first exponent to $(10000111)^2$, so we need to shift the first mantissa right by five positions:

	S	E	Denormalized M
A	0	10000111	000001011100000000000000
B	0	10000111	101000011110000000000000

Now we do sign-and-magnitude addition, treating the sign and the mantissa of each number as one integer stored in sign-and-magnitude representation:

	S	E	Denormalized M
R	0	10000111	101001111010000000000000

There is no overflow in the mantissa, so we normalize:

	S	E	M
R	0	10000110	01001111010000000000000

The mantissa is only 23 bits, no rounding is needed. $E = (10000110)_2 = 134$ M=0100111101. In other words, the result is $(1.0100111101)_2 \times 2^{134-127} = (10100111.101)_2 = 167.625$.

Example J.2

Show how the computer finds the result of (+5.75) + (−7.0234375) = −1.2734375.

Solution

These two numbers can be stored in floating-point format, as shown below:

	S	E	M
A	0	10000001	01110000000000000000000
B	1	10000001	11000001100000000000000

Denormalization results in:

	S	E	Denormalized M
A	0	10000010	10111000000000000000000
B	1	10000010	11100000110000000000000

Alignment is not needed (both exponents are the same), so we apply addition operation on the combinations of sign and mantissa. The result is shown below, in which the sign of the result is negative:

	S	E	Denormalized M
R	1	10000010	00101000110000000000000

Now we need to normalize. We decrement the exponent three times and shift the denormalized mantissa to the left three positions:

	S	E	M
R	1	01111111	01000110000000000000000

The mantissa is now 24 bits, so we round it to 23 bits:

	S	E	M
R	1	01111111	01000110000000000000000

The result is $R = -2^{127-127} \times 1.0100011 = -1.2734375$, as expected.

ACRONYMS

ADT	Abstract data type
AES	Advanced Encryption Standard
ALU	Arithmetic logic unit
ANSI	American National Standards Institute
ASCII	American Standard Code for Information Interchange
B-frame	Bidirectional frame
bit	Binary digit
BST	Binary search tree
CA	Certification authority
CD-R	Compact disc recordable
CD-ROM	Compact disc read-only memory
CD-RW	Compact disc rewritable
CGI	Common Gateway Interface
CISC	Complex instruction set computer
COBOL	Common Business-Oriented Language
CPU	Central processing unit
DBMS	Database management system
DCT	Discrete cosine transform
DES	Data Encryption Standard
digraph	Directed graph
DMA	Direct memory access
DRAM	Dynamic RAM
DVD	Digital versatile disk
EBCDIC	Extended Binary Coded Decimal Interchange Code
EEPROM	Electronically erasable programmable read-only memory
EPROM	Erasable programmable read-only memory

E-R	Entity–relation
FIFO	First in, first out
FORTRAN	FORmula TRANslation
FTP	File Transfer Protocol
GIF	Graphic Interchange Format
GUI	Graphical user interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
I-frame	Intracoded frame
IMAP	Internet Mail Access Protocol
IP	Internet Protocol
ISO	International Organization for Standardization
ISP	Internet service provider
JPEG	Joint Photographic Experts Group
KDC	Key distribution center
LAN	Local area network
LIFO	Last in, first out
LZ	Lempel Ziv
LZW	Lempel Ziv Welch
MAC	Media access control
MAC	Message authentication code
MAN	Metropolitan area network
MIME	Multipurpose Internet Mail Extension
MP3	MPEG audio layer 3
MPEG	Motion Pictures Experts Group
MS-DOS	Microsoft Disk Operating System
MTA	Message transfer agent
NF	Normal Form
NTFS	NT file system
P-frame	Predicted frame
POP	Post Office Protocol
PROM	Programmable read-only memory
RAM	Random access memory
RDBMS	Relational database management system
RGB	Red, green, blue
RISC	Reduced instruction set computer
ROM	Read-only memory
RSA	Rivest–Shamir–Adleman
SCSI	Small computer system interface
SCTP	Stream Control Transmission Protocol

SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SRAM	Static RAM
TCP	Transmission Control Protocol
TCP/IP	Transmission Control Protocol/Internet Protocol
TELNET	Terminal Network
UDP	User Datagram Protocol
UML	Unified Modeling Language
URL	Uniform Resource Locator
USB	Universal serial bus
WAN	Wide area network
WORM	Write once, read many
WWW	World Wide Web
XML	Extensible Markup Language
XOR	Exclusive OR

GLOSSARY

10-Gigabit Ethernet An implementation of Ethernet operating at 10 Gigabits per second.

absolute pathname In UNIX or Linux, a path name that starts from the root.

abstract data type (ADT) A data declaration packaged together with operations that are meaningful on the data type.

AC value A value that changes with time.

access method A technique for reading data from a secondary (auxiliary) storage device.

actual parameters The parameters in the function calling statement that contain the values to be passed to the function. Contrast with *formal parameters*.

Ada A high-level concurrent programming language developed by the US Department of Defense.

additive cipher A type of cipher that key defines shifting of a character toward the end of the alphabet (shift cipher).

additive cipher Simplest monoalphabetic cipher in which each ciphertext character is plaintext character plus the key value.

address bus The part of the system bus used for address transfer.

address space A range of addresses.

algorithm The logical steps necessary to solve a problem with a computer.

American National Standards Institute (ANSI) An organization that creates standards in programming languages, electrical specifications, communication protocols, and so on.

American Standard Code for Information Interchange (ASCII)	An encoding scheme that defines control and printable characters for 128 values.
ampersand	The at-sign (@) used in email addressing.
analog	A continuously varying entity.
analog data	Data that are continuous and smooth and not limited to specific value.
analog signal	A type of waveform that changes smoothly over time.
analog-to-analog conversion	Representation of analog data by analog signal.
analog-to-digital	Representation of analog data by digital signal.
analysis phase	A phase in the software system lifecycle that defines requirements that specify what the proposed system is to accomplish.
ancestor	Any node in the path from the current node to the root of a tree.
AND	One of the bit-level operations: the result of the operation is 1 only if both bits are 1s, otherwise it is 0.
applet	A computer program written in Java that creates an active Web document.
application gateway	A computer that stands between the firewall and the organization computer to filter unwanted messages.
application layer	The seventh layer in the TCP/IP model: provides access to network services.
application programs	In DBMS terminology, the programs that access and process data.
arc	A directed line in a graph. Contrast with *edge*.
arithmetic logic unit (ALU)	The part of a computer system that performs arithmetic and logic operations on data.
arithmetic operation	An operation that takes two numbers and creates another number.
arithmetic operator	The operator used in an arithmetic operation.
arithmetic shift operation	A shift operation in which the sign of the number is preserved.
array	A fixed-sized, sequenced collection of elements of the same data type.
artificial intelligence	The study of computer systems that simulate the intelligence of the human mind.
assembler	System software that converts a source program into executable object code. Traditionally associated with an assembly language program. See also *compiler*.

assembly language	A programming language in which there is a one-to-one correspondence between the computer's machine language and the symbolic instruction set of the language.
assignment statement	The statement that assigns a value to a variable.
asymmetric-key cipher	A type of cryptography that uses two different keys: a public key for encryption and a private key for decryption.
asymmetric-key encryption	Encryption using a public key.
attribute	In a relational database, each column in a relation.
attribute	In relational database terminology the name of the column heading.
audio	Recording or transmission of sound or music.
authentication	Verification of the sender of a message.
autokey cipher	A cipher that key is automatically created from the plaintext cipher during encryption.
auxiliary storage	Any storage device outside main memory: permanent data storage; external storage; secondary storage.
availability	The component of information security that requires that information created and stored by an organization be available to authorized entities.
axon	The part of a neuron in the human body that provides output to other neurons via a synapse.
base	the number of digits used in a numbering system. Base of binary is 2, of octal is 8, of decimal is 10, and of hexadecimal is 16.
basis path testing	A white-box test method that creates a set of test cases that executes every statement in the software at least once.
batch operating system	The operating system used in early computers, in which jobs were grouped before being served.
bidirectional frame (B-frame)	In MPEG, a frame that is related to both the preceding and following frames.
big-O notation	A measure of the efficiency of an algorithm with only the dominant factor considered.
binary digit (bit)	The smallest unit of information (0 or 1).
binary file	A collection of data stored in the internal format of the computer. Contrast with *text file*.
binary operation	An operation that needs two input operands.
binary search	A search algorithm in which the search value is located by repeatedly dividing the list in half.

binary search tree (BST) A binary tree in which the keys of the left subtrees are all less than the root key, the keys of all right subtrees are greater than or equal to the root key, and each subtree is itself a binary search tree.

binary system A numbering system that uses two symbols (0 and 1).

binary tree A tree in which each node has zero, one, or two subtrees.

bit Acronym for *binary digit*. In a computer, the basic storage unit with a value of either 0 or 1.

bit depth The number of bits representing a sample in a sampling process.

bit pattern A sequence of bits (0s and 1s).

bit rate The number of bits transmitted per second.

bitmap graphic A graphic representation in which a combination of pixels defines the image.

black-box testing Testing based on the system requirements rather than a knowledge of the program.

block cipher A cipher in which a group of characters are encrypted or decrypted at a time.

Bluetooth A wireless technology designed to connect devices in a small area.

Boolean algebra An algebra for manipulation of objects that can take one of only two values: true or false.

bootstrap The process in which the operating system is loaded into main memory when the computer is turned on.

breadth-first traversal A graph traversal method in which nodes adjacent to the current node are processed before their descendants.

browser An application program that displays a WWW document.

brute-force search A search method that examines every path in a search tree until it finds the goal.

bubble sort A sort algorithm in which each pass through the data moves (bubbles) the lowest element to the beginning of the unsorted portion of the list.

bucket In a hashing algorithm, a location that can accommodate multiple data units.

bucket hashing A hashing method that uses buckets to reduce collision.

bus The physical channel that links hardware components in a computer: the shared physical medium used in a bus-topology network.

byte	A unit of storage, usually 8 bits.
bytecode	A machine language into which a Java source program is compiled.
C language	A procedural language developed by Dennis Ritchie.
C++ language	An object-oriented language developed by Bjarne Stroustrup.
cache memory	A small, fast memory used to hold data items that are being processed.
Caesar cipher	A shift cipher used by Julius Caesar.
cardinality	In relational database, the total number of rows in each relation.
cellular telephony	A wirless communication technique in which one area is divided into cells. Each cell is served by a transmitter.
central processing unit (CPU)	The part of a computer that contains the control components to interpret instructions. In a personal computer, a microchip containing a control unit and an arithmetic logic unit.
certification authority (CA)	An organization that binds a public key to an entity and issues a certificate.
challenge–response authentication	A process in which the claimant proves that she knows a secret without sending it.
child	A node in a tree or graph that has a predecessor.
Church–Turing thesis	In computability theory, a combined hypothesis about the nature of computable functions by recursion (Church's Thesis) and by mechanical devices equivalent to a Turing machine.
cipher	An encryption decryption algorithm.
ciphertext	Encrypted data.
circular shift operation	A shift operation in which dropped bits from one end of a binary word are inserted at the other end.
circular waiting	A condition in an operating system in which all processes and resources involved form a loop.
class	The combination of data and functions joined to form a type.
class diagram	A diagram that manifests the relationship between classes in a system.
client–server paradigm	A paradigm in which two computers connected by an internet run a program: one to provide a service and one to request a service.

coaxial cable	A cable made of one conductor media and a sheath that serves as the second conductor.
code	A set of bit patterns designed to represent text symbols.
code generator	A process in a compiler or interpreter that creates the machine language code.
cohesion	The attribute of a module that describes how closely the processes in a module are related to one another.
collision	In hashing, an event that occurs when a hashing algorithm produces an address for an insertion, and that address is already occupied.
collision resolution	An algorithmic process that determines an alternative address after a collision.
color depth	The number of bits used to represent the color of a pixel.
column-major storage	A method of storing two-dimensional arrays in which the elements are stored column by column.
COmmon Business-Oriented Language (COBOL)	A business programming language developed by Grace Hopper.
compact disk (CD)	A direct access optical storage medium.
compact disk read-only memory (CD-ROM)	A compact disc in which data is written to the disc by the manufacturer and can only be read by the user.
compact disk recordable (CD-R)	A compact disc that a user can write to only once, but read from many times.
compact disk rewritable (CD-RW)	A compact disc that can be written to many times and read from many times.
compilation	The process of translating the whole source program written in a high-level language into machine language before executing the program.
compiler	System software that converts a source program into executable object code: traditionally associated with high-level languages. See also *assembler*.
complex instruction set computer (CISC)	A computer that defines an extensive set of instructions, even those that are used less frequently.
composite type	A data type that is composed of two or more simple types.
compound statement	In some programming languages, a collection of statements (instructions) treated as one by the language.

computer language	Any of the syntactical languages used to write programs for computers, such as machine language, assembly language, C, COBOL, and FORTRAN.
conceptual level	Relating to the logical structure of a database. It deals with the meaning of the database, not its physical implementation.
confidentiality	A security goal that defines procedures to hide information from an unauthorized entity.
connecting device	A device that connect computers or networks.
connectionless protocol	A protocol for data transfer without connection establishment or termination.
constant	A data value that cannot change during the execution of the program. Contrast with *variable*.
control bus	The bus that carries information between computer components.
control statement	A statement that alters the sequential flow of control in a source program.
control structure testing	A white-box test method that uses different categories of tests: conditional testing, dataflow testing, and loop testing.
control unit	The component of a CPU that interprets the instructions and controls the flow of data.
controller	A component of a Turing machine that is equivalent to a computer's CPU.
copyright	A copyright is a right to a written or created work. It gives the author the exclusive rights to copy, distribute, and display the work.
copyright	The right to written or created work.
country domain	A subdomain in the Domain Name System that uses two characters (representing a country) as the last suffix.
coupling	A measure of the interdependence between two separate functions.
cryptographic hash function	A function that creates a message digest from a message.
cryptography	The science and art of transforming messages to make them secure and immune to attack.
current directory	In UNIX and Linux, the directory that a user is in at the present time.

cycle	A graph path with a length greater than 1 that starts and ends at the same vertex.
data bus	The bus inside a computer used to carry data between components.
data compression	Reduction of the volume of data without significant loss.
data confidentiality	A security service designed to protect data from disclosure attacks, snooping, and traffic analysis.
data file	A file that contains only data, not programs.
data flow diagram	A diagram that shows the movement of data in the system.
data integrity	A security service designed to protect data from modification, insertion, deletion, and replaying.
data link layer address	The address used in the data link layer, sometimes called the MAC address, sometimes the physical address.
data processor	An entity that inputs data, processes it, and outputs the result.
data structure	The syntactical representation of data organized to show the relationship among the individual elements.
data type	A named set of values and operations defined to manipulate them, such as character and integer.
data-link layer	The second layer of the TCP/IP protocol.
database	A collection of organized information.
database management system (DBMS)	A program or a set of programs that manipulates a database.
database model	A model that defines the logical design of data.
datagram	An independent data unit.
DC value	A value that does not change with time.
deadlock	A situation in which the resources needed by one job to finish its task are held by other jobs.
decimal digit	A symbol in the decimal system.
decimal system	A method of representing numbers using ten symbols (0 to 9).
decision	In programing a two-way path that one of them should be chosen by the program.
declarative language	A computer language that uses the principle of logical reasoning to answer queries.

declarative paradigm	A paradigm that uses the principle of logical reasoning to answer queries.
decode	Interpretation of an instruction by the control unit.
decrement statement	A statement that subtracts 1 from the value of a variable.
decryption	Recovery of the original message from encrypted data. See *encryption*.
decryption algorithm	An algorithm that decrypts an encrypted message to create the plain text.
default logic	A logic in which the default conclusion of an argument is acceptable if it is consistent with the contents of knowledge base.
delete operation	Operation that deletes a tuple based on the criterion given in the call.
delete operation	In a relational database, the operation that deletes a tuple from the relation.
demand paging	A memory allocation method in which a page of a program is loaded into memory only when it is needed.
demand paging and segmentation	A memory allocation method in which a page or a segment of a program is loaded into memory only when it is needed.
demand segmentation	A memory allocation method in which a segment of a program is loaded into memory only when it is needed.
demodulator	A device that converts a signal to data.
dendrite	In a neuron, the section that acts as the input device.
denial of service	The only attack on the availability goal of security that may slow down or interrupt the system.
depth-first traversal	A traversal method in which all of a node's descendants are processed before any adjacent nodes (siblings).
dequeue	Deleting an element from a queue.
descendant	Any node in the path from the current node to a leaf.
design phase	A phase in the software system lifecycle that defines how the system will accomplish what was defined in the analysis phase.
development process	The process of creating software that is outside the system lifecycle.
device manager	A component of an operating system that controls access to the input/output devices.

dictionary-based encoding	A compression method in which a dictionary is created during the session.
difference operation	An operation on two sets, the result of which is the first set minus the common elements in the two sets, or an operator in a relational database that is applied to two relations with the same attributes. The tuples in the resulting relation are those that are in the first relation but not the second.
digest	A compressed image of a message.
digit extraction hashing	The process of extracting selected digits from a key and use them as an address.
digit extraction method	A hashing method that uses digit extraction.
digital	A discrete (noncontinuous) entity.
Digital data	Data represented by discrete values.
digital signal	A signal with a number of distinct values.
digital signature	A method used to authenticate the sender of a message and to preserve the integrity of its data.
digital subscriber line (DSL)	A technology that supports high-speed communication over existing telephone line.
digital versatile disk (DVD)	A direct access optical storage medium.
digital-to-analog conversion	Representation of digital data by an analog signal.
digital-to-digital conversion	Representation of digital data by digital signal.
digraph	A directed graph.
direct hashing	A hashing method in which the key is obtained without algorithmic modification.
direct memory access (DMA)	A form of I/O in which a special device controls the exchange of data between memory and I/O devices.
directed graph	A graph in which the direction is indicated on the lines (arcs).
directory	A file that contains the names and addresses of other files.
discrete cosine transform (DCT)	A mathematical transformation used in JPEG encoding.
distributed database	A database in which data is stored on several computers.
distributed system	An operating system that controls resources located in computers at different sites.
division remainder method	A type of hashing in which the key is divided by a number and the remainder is used as the address.

domain name	In DNS, a sequence of labels separated by dots.
Domain Name Server (DNS)	A computer that holds information about Internet domain names.
doman name space	A method for organizing the name space in which the names are defined in an inverted-tree structure with the root at the top.
dotted-decimal notation	The notation devised to make IP addresses easier to read: each byte is converted to a decimal number; numbers are separated by a dot.
DSL	Digital subscriber line.
dynamic RAM (DRAM)	RAM in which the cells use capacitors. DRAM must be refreshed periodically to retain its data.
edge	A graph line that has no direction.
edge detection	A method of image processing that finds the edges in an image by looking at areas with changes in color or texture.
electrically erasable programmable read-only memory (EEPROM)	Programmable read-only memory that can be programmed and erased using electronic impulses without being removed from the computer.
electronic mail (email)	A method of sending messages electronically based on a mailbox address rather than host-to-host exchange.
emacs	A text editor in Unix.
encryption	Converting a message into a form that is unreadable unless decrypted.
encryption algorithm	An algorithm that encrypts a plain text message.
end system	A sender or receiver of data.
end users	In DBMS terminology, users that access the database directly.
enqueue	Inserting an element into a queue.
entity authentication	A technique designed to let one party prove the identity of another party.
entity–relation (E-R) model	A model that defines the entities and their relationship in a relational database.
entity–relationship (E-R) diagram	A diagram used in Entity–Relation model.
ephemeral port number	A port number used by the client

erasable programmable read-only memory (EPROM)	Programmable read-only memory that can be programmed. Erasing EPROM requires removing it from the computer.
error report file	In a file update process, a report of errors detected during the update.
ethical principle	One of the ways to evaluate our responsibility towards the rest of the world when using a computer is to base our decisions on ethics.
Excess representation	A number representation method used to store the exponential value of a fraction.
Excess_1023	The high-precision IEEE standard for representation of floating point numbers.
Excess_127	The low-precision IEEE standard for representing floating-point numbers.
execute	Sending commands to different section of the computer by the Control Unit.
expert system	A system that uses knowledge representation to perform tasks that normally need human expertise.
expression	A sequence of operators and operands that reduces to a single value.
expression tree	An upside down tree in which the root and nodes are operators and leaves are operands.
external level	The part of the database that interacts with the user.
Facebook	A social network site with more than one billion users.
Fast Ethernet	An implementation of Ethernet with a data rate of 100 Mps.
fetch	The part of the instruction cycle in which the instruction to be executed is brought in from memory.
fiber-optic cable	A medium that carries signals in the form of pulses. It consists of a thin cylinder of glass or plastic (core) inside another cylinder of glass or plastic (cladding).
field	The smallest named unit of data that has meaning in describing information. A field may be either a variable or a constant.
File Transfer Protocol (FTP)	An application-layer service in TCP/IP for transferring files from and to a remote site.
finite state automaton	A machine that has predefined number of states.
firewall	A device installed between the internal network of an organization and the rest of the Internet to provide security.

FireWire	An I/O device controller with a high-speed serial interface that transfers data in packets.
first in, first out (FIFO)	An algorithm in which the first data item that is added to a list is removed from the list first.
flat-file	A stand-alone file not related to any other file in an organization.
floating-point representation	A number representation in which the position of the decimal point is floated to create better precision. Normally used to represent real numbers in a computer.
follow	How to connect to other users on Twitter.
following	The many-to-one relationship in Twitter in which many users follow a user.
follwing	Follow people you know to see their tweets on your home page.
formal parameters	The parameter declaration in a function to describe the type of data to be passed to the function.
FORmula TRANslation (FORTRAN)	A high-level procedural language used for scientific and engineering applications.
fragmented distributed database	A distributed database in which data is localized.
frame	A data unit at the data-link layer.
frames	A method similar to semantic networks for knowledge representation.
frequency masking	The process that a frequency totally mask another frequency.
friend	In Facebook, a user who can receive posts made by another user.
friendship	In Facebook, sharing is done only between friends. Friendship is a one-to-one reciprocal relationship.
friendship	The one-to-one relationship between users in Facebook.
front	The next element in a queue that is deleted by the dequeue operation.
function	In a functional paradigm, a black box that maps a list of input to a list of output.
functional language	A programming language in which a program is considered to be a mathematical function.
functional paradigm	A paradigm in which a program is considered a mathematical function.

https://sanet.st/blogs/polatebooks/

general linear list	A list in which data can be inserted or deleted anywhere in the list.
generic domain	A subdomain in the domain name space that uses generic suffixes.
Gigabit Ethernet	An Etherenet with on gigabit (1000 Mbps) data rate.
glass-box testing	See *white-box testing*.
Gödel number	A number assigned to every program that can be written in a specific language.
graph	A collection of nodes, called vertices, and line segments, called edges or arcs, connecting pairs of nodes.
Graphic Interchange Format (GIF)	An 8-bit per pixel bitmap image.
graphical user interface (GUI)	A user interface that defines icon and operations on icons.
Guided media	Cable.
hacker	An astute computer specialist who may do harm.
halting problem	Writing a program that tests whether or not any program, represented as its Gödel number, will terminate.
hardware	Any of the physical components of a computer system, such as a keyboard or a printer.
hardware abstraction layer (HAL)	The lowest level program in the Windows system that hides the differences from upper layer.
hashed file	A file that is searched using one of the hashing methods.
hashed MAC (HMAC)	A recommendation by ITU to describe a certificate in public-key distribution using a well-known standard called ASN.1.
hashing method	A method to access a hashed file.
hashtag	A way to indicate an important word in a tweet. It starts with a hash (#) sign.
HDMI (High-Definition Multimedia Interface)	HDMI is a digital replacement for existing analog video standards.
header	The information added to the beginning of a packet for routing and other purposes.
heuristic search	A search in which a rule or a piece of information is used to make the search more efficient.
hexadecimal digit	A symbol in the hexadecimal system.
hexadecimal system	A numbering system with base 16. Its digits are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.

hierarchical model	A database model that organizes data in a treelike structure that can be searched from top to bottom.
high-level language	A (portable) programming language designed to allow the programmer to concentrate on the application rather than the structure of a particular computer or operating system.
high-order logic	A logic that extends the scope of quantifiers $\forall$ and $\exists$ in predicate logic to bind predicates as well as variables.
hold state	The state of a job that is waiting to be loaded into memory.
home address	In a hashed list, the first address produced by the hashing algorithm.
home directory	In UNIX or Linux, a directory that a user is in when first logged in.
home page	The main page of a hypertext document available on the Web.
host	A station or node node on the network.
host identifier	The identifier of a station.
hub	A device that connects other devices in a network.
Huffman coding	A statistical compression method using variable-length code.
hypertext	A document containing embedded links to other documents.
Hypertext Markup Language (HTML)	The computer language for specifying the contents and format of a Web document: allows text to include fonts, layouts, embedded graphics, and links to other documents.
Hypertext Transfer Protocol (HTTP)	The protocol that is used to retrieve Web pages on the Internet.
identifier	The name given to an object in a programming language.
image processing	An area of artificial intelligence that deals with the perception of objects—the artificial eyes of an agent.
imperative language	Another name for a procedural language.
imperative paradigm	Another name for a procedural paradigm.
implementation phase	A phase in the software system lifecycle in which the actual programs are created.
increment statement	In C or C++, the statement that adds 1 to an integer value.

incremental model	A model in software engineering in which the entire package is constructed with each module consisting of just a shell: modules gain complexity with each iteration of the package.
index	The address of an element in an array.
indexed color	A technique in raster graphic that uses only a portion of True-Color to encode colors in each application.
indexed file	A file that uses an index for random access.
infix	An arithmetic notation in which the operator is placed between two operands.
infrared waves	Part of light spectrum from 300 GHz to 400 THz used for short-range communication.
inheritance	The ability to extend a class to create a new class while retaining the data objects and methods of the base class and adding new data objects and methods.
inorder traversal	A binary tree traversal method in which the root is traversed after the left subtree and before the right subtree.
input	The given data to an operation.
input data	User information that is submitted to a computer to run a program.
input/output (I/O) controller	A device that controls access to input/output devices.
input/output subsystem	The part of the computer's organization that receives data from the outside and sends data to the outside.
insert operation	An operation in a relational database that inserts a tuple in a relation.
insertion sort	A sort algorithm in which the first element from the unsorted portion of the list is inserted into its proper position in the sorted portion of the list.
instruction	A command that tells a computer what to do.
instruction register	A register in the CPU that holds the instruction before being interpreted by the control unit.
integer	An integral number, a number without a fractional part.
integrated circuit	Transistors, wiring, and other components on a single chip.
integrity	A security goal to protect data from modification, insertion, deletion, or replaying.
intellectual property	Intangible things such as ideas, inventions, technologies, artworks, music and literature over which one can claim ownership.

intelligent agent	An agent that perceives its environment, learns from it, and interacts with it intelligently.
interface	A list of public operations or data that are to be passed or returned from an operation.
internal level	The part of the database that defines where data is actually stored.
internal node	Any tree node except the root and the leaves: a node in the middle of a tree.
International Organization for Standardization (ISO)	A worldwide organization that defines and develops standards for a variety of topics.
internet	Abbreviation for *internetwork*.
Internet	The global Internet that uses the TCP/IP protocol suite.
Internet address	A 32-bit address used to define a computer uniquely on the Internet.
Internet Protocol (IP)	The network-layer protocol in the TCP/IP protocol responsible for transmitting packets from one computer to another across the Internet.
Internet Protocol version 6 (IPv6)	The new IP with augmented address space and redesigned packets and protocols.
Internet service provider (ISP)	An organization that provides Internet services.
internetwork	A network of networks.
interpretation	The process of translating each line of a source program into the corresponding object program and executing the line.
interpretation	This is repeated term to be deleted
interpreter	A program that translates a source program in a high-level language line by line and executes each line immediately.
interrupt driven I/O	A form of I/O in which the CPU, after issuing an I/O command, continues serving other processes until it receives an interrupt signal that the I/O operation is completed.
intersection operation	An operation on two sets in which the result is a set with the elements common to the two sets.
intersector gap	The gap between sectors on a disk.
intertrack gap	The gap between tracks on a tape.
intracoded frame (I-frame)	In MPEG, an independent frame.
inverted file	A file sorted according to a second key.
IP address	See *Internet address*.

IP datagram	The data unit in the network layer.
IP new generation (IPng)	Another name for IPv6.
isolated I/O	A method of addressing an I/O module in which the instructions used to read/write memory are totally different than the instructions used to read/write to input/output devices.
Java	An object-oriented programming language for creating stand-alone programs or dynamic documents on the Internet.
job	A program becomes a job when it is selected for execution.
job scheduler	A scheduler that selects a job for processing from a queue of jobs waiting to be moved to memory.
join operation	An operation in a relational database that takes two relations and combines them based on common attributes.
Joint Photographic Experts Group (JPEG)	A standard for compressing images.
kernel	The main part of an operating system.
key	One or more fields used to identify a record (structure).
key-distribution center (KDC)	A trusted third party that establishes a shared secret key between two parties.
keyboard	An input device providing character-by-character input.
land	On an optical disc, an area not hit by the laser in the translation of a bit pattern. Usually represents a bit.
last in, first out (LIFO)	An algorithm in which the last data item that is added to a list is removed from the list first.
leaf	A graph or tree node with one incoming arc and no outgoing arcs.
Lempel Ziv (LZ) encoding	A compression algorithm that uses a dictionary.
Lempel Ziv Welch (LZW) encoding	An enhanced version of LZ encoding.
lexical analyzer	A program used in a translation process that reads the source code symbol by symbol and creates a list of tokens.
linear list	A list structure in which each element except the last has a unique successor.
link	In a list structure, the field that identifies the next element in the list.
linked list	A linear list structure in which the ordering of the elements is determined by link fields.

linked list resolution	A collision resolution method in hashing that uses a separate area for synonyms, which are maintained in a linked list.
Linux	An operating system developed by Linus Torvalds to make UNIX more efficient when run on an Intel microprocessor.
LISP	A list processing programming language in which everything is considered a list.
list	An ordered set of data contained in main memory.
literal	A constant used in a program.
local area network (LAN)	A network connecting devices inside a limited area.
local variable	A variable defined within a block or a module.
logical operation	An operation in which the result is a logical value (true or false).
logical operator	An operator that combines Boolean values to get a new Boolean values.
logical shift operation	A shift operation that does not preserve the sign of the number.
loop	In a program, a structured programming construct that causes one or more statements to be repeated. In a graph, a line that starts and ends with the same vertex.
loop statement	A statement that causes the program to iterate a set of statements.
lossless data compression	Data compression in which no data is lost. Used for compressing text or programs.
lossy data compression	Data compression in which some data is allowed to be lost. Used for image, audio, or video compression.
MAC addresses	Address of a device at the data-link layer.
machine cycle	A repeated collection of fetch, decode, and execute operations.
machine language	Instructions native to the central processor of a computer that are executable without assembly or compilation.
macro	A custom-designed procedure that can be used repeatedly.
magnetic disk	A storage medium with random access capability.
magnetic tape	A storage medium with sequential access capability.
Mail Transfer Agent	The client-server program used in email communication.

main memory	The primary memory of a computer, consisting of medium speed. random access memory. Contrast with *cache memory*.
maintainability	A quality that refers to keeping a system running correctly and up to date.
mantissa	The part of a floating-point number that shows the number's significant digits.
mask	A variable or constant that contains a bit configuration used to control the setting of bits in a bitwise operation.
masquerading	A type of attack on integrity of information in which the attacker impersonates somebody else.
master disk	The first component created in a CD-ROM
master file	A permanent file that contains the most current data regarding an application.
medium access control (MAC) address	See *data link layer address*.
memory	The main memory of a computer consisting of random access memory (RAM) and read-only memory (ROM), used to store data and program instructions.
memory management	The component of the operating system that controls the use of main memory.
memory-mapped I/O	A method of addressing an I/O module in a single address space, used for both memory and I/O devices.
Message Access Agent (MAA)	A client-server program that pulls the stored email message.
message authentication code (MAC)	A message digest that includes a secret between two parties.
message digest	The fixed-length string created from applying a hash function to a message.
message transfer agent (MTA)	An SMTP program that transfers a message across the Internet.
method	A function in an object-oriented language.
metropolitan area network (MAN)	A network that can span a city or a town.
microcomputer	A computer small enough to fit on a desktop.
Microsoft Disk Operating System (MS-DOS)	The operating system based on DOS and developed by Microsoft.
modal logic	An extension to logic that includes certainty and possibility.

modem	A modulator–demodulator combined.
modularity	Breaking a large project into small parts that can be understood and handled easily.
modulator	A device that converts data to a signal.
module	A small part created from applying modularity to a project.
modulo division	Dividing two numbers and keeping the remainder.
monitor	A nonstorage device that provides output.
Monitor	The monitor displays output and at the same time echoes input typed on the keyboard.
monoalphabetic cipher	A cipher in which the same character is always encrypted the same regardless of its position in the text.
monoprogramming	The technique that allows only one program to be in memory at a time.
moral rule	A principle in ethics dictating that we should avoid doing anything that is against universal morality.
moral rules	The first ethical principle states that when we make an ethical decision, we need to consider if the decision is made in accordance with a universally accepted principle of morality.
Moving Picture Experts Group (MPEG)	A lossy compression method for compressing video (and audio).
MPEG audio layer (MP3)	A standard used for compression audio based on MPEG.
multidimensional array	An array with elements having more than one level of indexing.
Multiple Instruction Stream, Single Data Stream (MISD)	A computer in which several streams operates on one single data.
multiple instruction_stream, multiple data stream (MIMD)	A computer with several CUs, several ALUs, and several memory unit.
multiprogramming	A technique that allows more than one program to reside in memory while being processed.
multithreading	Parallel processing supported by some languages such as Java.
mutual exclusion	A condition imposed by an operating system in which only one process can hold a resource.
name	In relational database terminology, the identifier of a relation.
name space	All the name assigned to a machine on an internet
network	A system of connected nodes that can share resources.

network layer	The third layer in the TCP/IP model, responsible for delivery of packets from the original host to the final destination.
network model	A database model in which a record can have more than one parent record.
neural network	A network of neurons which is modeled on the human brain.
neuron	The individual cells that are responsible for transmitting information in the human brain and nervous system.
new master file	The master file that is created from an old master file when the file is updated.
news	Facebook allows you to post news (called updates) for your friends. This can be a long message (up to 60 000 characters), a link to a web page, a photo, or a video.
news (remove)	I do not think that this item should be in glossary.
no preemption	A condition in which the operating system cannot temporarily allocate a resource.
node	In a data structure, an element that contains both data and structural elements used to process the data structure.
node-to-node	The communication at the data-link layer.
nonpolynomial problem	A problem that cannot be solved with polynomial complexity.
nonpositional number system	A number system in which the position of symbols does not define the value of the symbol.
Nonstorage device	Nonstorage devices allow the CPU/memory to communicate with the outside world, but they cannot store information.
nonstorage device	A device that allows communication between CPU and memory without storing information.
normal form (NF)	A step in the normalization process of a relational database.
normalization	In a relational database, the process of applying normal forms to a relational model.
NOT operator	The operation that changes a 0 bit to 1 or a 1 bit to 0.
null pointer	A pointer that points to nothing.
number system	A system that uses a set of symbols to define a value.
object program	The machine language code created from a compiler or an interpreter.

object-oriented analysis	The analysis phase of a developmental process in which the implementation uses an object-oriented language.
object-oriented database	A database in which data is treated as structures (objects).
object-oriented design	Given the details and codes for each class in object-orienting programming.
object-oriented language	A programming language in which the objects and the operations to be applied to them are tied together.
object-oriented paradigm	A paradigm in which a program acts on active objects.
octal digit	A digit in base 8.
octal system	A numbering system with a base of 8: the octal digits are 0 to 7.
old master file	The master file that is processed in conjunction with the transaction file to create the new master file.
one-dimensional array	An array with only one level of indexing.
one-time pad	A type of cipher in which the key is randomly chosen for each encipherment.
one's complement	A bitwise operation that reverses the value of the bits in a variable.
open addressing resolution	A collision resolution method in which the new address is in the home area.
operability	The quality factor that addresses the ease with which a system can be used.
operand	An object in a statement on which an operation is performed. Contrast with *operator*.
operating system	The software that controls the computing environment and provides an interface to the user.
operator	The syntactical token representing an action on data (the operand). Contrast with *operand*.
optical storage device	An I/O device that uses (laser) light to store and retrieve data.
OR operator	A binary operation resulting in an output of 0 only if the two inputs are 0s, otherwise 1.
output	The resulting data from an operation.
output data	The results of running a computer program.
output device	A device that can be written to but not read from.
overflow	The condition that results when there are insufficient bits to represent a number in binary.

packet-filter firewall	A firewall that uses a filtering table to protect the organization.
packetizing	Encapsulating data in a packet.
page	One of a number of equally sized sections of a program.
paging	A multiprogramming technique in which memory is divided into equally sized sections called *frames*.
palette color	See *indexed color*.
parallel processing	A form of multiprocessing that speeds processing by allowing several processors to operate at the same time.
parallel system	An operating system with multiple CPUs on the same machine.
parameter	A value passed to a function.
parent	A tree or graph node with one or more child nodes.
parent directory	The directory immediately above the current directory.
parser	An entity that does parsing.
partitioning	A technique used in multiprogramming that divides the memory into variable-length sections.
Pascal	A programming language designed with the goal of teaching programming to novices by emphasizing the structured programming approach.
pass by reference	A parameter passing technique in which the called function refers to a passed parameter using an alias name.
pass by value	A parameter passing technique in which the value of a variable is passed to a function.
patent	A right to an intellectual property.
path	A sequence of nodes in which each vertex is adjacent to the next.
peer-to-peer paradigm	A paradigm in which two peer computers can communicate with each other to exchange services.
penetration attack	Penetration means breaking into a system to get access to the data stored in a computer or in a computer network.
penetration attack	Unlawful accessing of a computer to get information or to cause damage.
perceptron	A simple neuron-like element that is used in neural networks.
perceptual encoding	Type of encoding used in audio.

physical agent A programmable system (robot) that can be used to perform a variety of tasks.

physical layer The first layer in the TCP/IP model, responsible for signaling and transmitting bits across the network.

pico Another text editor in Unix.

picture element (pixel) The smallest unit of an image.

pipelining A technique used by modern computers to improve the throughput by combining different phases of on instruction with the next.

Pipelining Modern computers use a technique called pipelining to improve the throughput (the total number of instructions performed in each period of time). The idea is that if the control unit can do two or three of these phases simultaneously, the next instruction can start before the previous one is finished.

pit On an optical disc, an area struck by the laser in the translation of a bit pattern, which usually represents a 0 bit.

pixel See *picture element*.

place value The value related to a position in the positional number system.

plaintext Text before being encrypted.

point-to-point A type of network that connects two communication devices through a transmission media.

pointer A constant or variable that contains an address that can be used to access data stored elsewhere.

polyalphabetic cipher A type of cipher that a character in the plaintext will change to a different character in the ciphertext base on its position in the text.

polycarbonate resin In CD-ROM production, a material injected into a mold.

polymorphism In C++, defining several operations with the same name that can do different things in related classes.

polynomial problem A problem that can be solved in an acceptable time by a computer.

pop The stack delete operation.

port address See *port number*.

port number	The address used in TCP and UDP to distinguish one process from another.
portability	The quality factor relating to the ease with which a system can be moved to other hardware environments.
portability process scheduler	I cannot find this term. We have process scheduler, but not portability process scheduler.
positional number system	A number system in which the position of a symbol in a number defines its value.
postfix	An arithmetic notation in which the operator is placed after its operands.
postorder traversal	A binary tree traversal method in which the left subtree is processed first, then the right subtree, then the root.
pragmatic analysis	The analysis of a sentence to find the real meaning of words by removing the ambiguities.
predicate logic	A logic system in which quantifiers can be applied to terms but not to predicates.
predicted frame (P-frame)	In MPEG, a frame that is related to the preceding I-frame or B-frame.
Predictive encoding	Encoding the difference between two samples instead of encoding the sample itself.
prefix	An arithmetic notation in which the operator is placed before the operands.
preorder traversal	A binary tree traversal in which the left subtree is traversed first, the root is traversed next, and the right subtree is traversed last.
prime area	In a hashed list, the memory that contains the home address.
printer	An output device that creates hard copy.
privacy	The right of individual to keep some information secret.
privacy	Today, a large amount of personal information about a citizen is collected by private and public agencies. Some countries have ethical rules about the use of this information.
private key	One of the two keys used in public key encryption.
procedural language	A computer language in a procedural paradigm.
procedural paradigm	A paradigm in which a program acts on passive objects using procedures.
procedure-oriented analysis	The analysis phase of a developmental process in which the implementation uses a procedural language.

procedure-oriented design	The design phase of developmental process when the implementation uses a procedural language.
process	A program in execution.
process manager	An operating system component that controls the processes.
process scheduler	An operating system mechanism that dispatches the processes waiting to get access to the CPU.
product	The result of multiplying a list of numeric data and finding the result.
program	A set of instructions.
program counter	A register in the CPU that holds the address of the next instruction in memory to be executed.
programmable read-only memory (PROM)	Memory with contents electrically set by the manufacturer that may be reset by the user.
programmed I/O	A form of I/O in which the CPU must wait for the I/O operation to be completed.
programming language	A language with limited words and limited rules designed to solve problems on a computer.
project operation	An operation in a relational database in which a set of columns is selected based on a criterion.
PROLOG	A programming language that can build a database of facts and a knowledge base of rules.
propositional logic	A logic system based on logical operators and propositional term.
protocol	A set of rules for data exchange between computers.
protocol layering	The idea of using a set of protocols to create a hierarchy of rules for handling a difficult task.
proxy firewall	A firerwall that stands between the customer computer and the corporation computer. It accepts only legitimate messages from the customer computer.
proxy firewall	A proxy computer (sometimes called an application gateway) that stands between the customer computer and the corporation computer.
pseudocode	English-like statements that follow a loosely defined syntax and are used to convey the design of an algorithm or function.
public key	One of the keys in a public key encryption, revealed to the public.
public-key certificate	A certificate that binds an entity to its public key.

push	The stack insert operation.
quantifier	Two operators used in predicate logic: ∀ and ∃.
quantization	Assigning a value from a finite set of values.
queue	A linear list in which data can only be inserted at one end, called the rear, and deleted from the other end, called the front.
radix	The base in a positional number system.
random access	A storage method that allows data to be retrieved in an arbitrary order.
random access memory (RAM)	The main memory of the computer that stores data and programs.
raster graphic	See *bitmap graphic*.
read-only memory (ROM)	Permanent memory with contents that cannot be changed.
read/write head	The device in a hard disk that reads or writes data.
ready state	In process management, the state of processing in which the process is waiting to get the attention of the CPU.
real	A number with both integral and fractional parts.
real-time system	An operating system that is expected to do a task within specific time constraints.
rear	The last element inserted into a queue using the enqueue operation.
record	Information related to one entity.
recursion	A function design in which the function calls itself.
reduced instruction set computer (RISC)	A computer that uses only frequently used instructions.
register	A fast stand-alone storage location that holds data temporarily.
relation	A table in a relational database.
relational database	A database model in which data is organized in related tables called relations.
relational database management system (RDBMS)	A set of programs that handles relations in a relational database model.
relational model	See *relational database*.
relational operator	An operator that compares two values.
relative pathname	The path of a file defined in terms of the working directory.

reliability	The quality factor that addresses the confidence or trust in a system's total operation.
remote login	Logging on to a remote computer that is connected to the local computer.
repetition	One of the three constructs in structural programming.
replaying	A type of attack on information integrity in which the attacker intercepts the message and resends it again.
replicated distributed database	A database in which each site holds a replica of another site.
resolution	The scanning rate in image processing: the number of pixels per unit measure.
resource holding	A condition in which a process holds a resource but cannot use it until all other resources are available.
retrieval	The location and return of an element in a list.
retweet	A tweet forwarded to other Twitter users.
RGB	A color system in which tints are represented by a combination of red, green, and blue primary colors.
Roman number system	The nonpositional number system used by the Romans.
root	The first node of a tree.
root directory	The highest level in a file hierarchy.
rotational speed	The spin rate of a magnetic disk.
router	A device operating at the first three TCP/IP layers that connects independent networks. A router routes a packet based on its destination address.
routing	The process performed by a router.
row-major storage	A method of storing array elements in memory in which the elements are stored row by row.
RSA cryptosystem	A common public cryptosystem designed by River, Shamir, and Adelman. It uses two exponents, e and d, in which the the first is public and the second is private.
RSA cryptosystem	RSA cryptosystem is one of the common public-key algorithms.
rule-based system	A knowledge representation system that uses a set of rules that can be used to deduce new facts from known facts.
run-length encoding	A lossless compression method in which a sequence of the same symbols is replaced by the symbol and the number of repetitions.

running state	In process management, a state in which a process is using the CPU.
sampling	Taking measurements at equal intervals.
sampling rate	The number of samples obtained per second in the sampling process.
scanning	Converting an image into digital data by sampling its density and color at evenly-spaced points.
scheduler	A program to move a job from one state to another.
scheduling	Allocating the resources of an operating system to different programs and deciding which program should use which resource, and when.
scheme	The de facto standard of the LISP language.
scientific notation	Representation of a number with one digit at the left of the decimal point and the power of 10 defines the shifting of the decimal point.
search space	The set of possible situations that can be examined by a search method to find a solution.
searching	The process that examines a list to locate one or more elements containing a designated value known as the search argument.
secondary storage device	See *auxiliary storage*.
secret key	A key that is shared by two participants in secret key encryption.
sector	A part of a track on a disk.
Secure Hash Algorithm (SHA)	A standard hash function developed by NIST.
Secure Shell (SSH)	A client-server program that provide secure logging.
security	The quality factor that addresses the ease or difficulty with which an unauthorized user can access data.
security attack	An attack threatening the security goals of a system.
security goal	One of the three goals of information security: confidentiality, integrity, and availability.
seek time	In disk access, the time required to move the read/write head over the track that holds the required data.
segment	Part of a packet.
segmentation	A step in image processing that divides the image into homogeneous segments or areas.
select operation	An operation in a relational database that selects a set of tuples.

selection	One of the three constructs in structure programming.
selection sort	A sort algorithm in which the smallest value in the unsorted portion of the list is selected and placed at the end of the sorted portion of the list.
semantic analysis	Analysis of the meaning of words in a sentence or tokens in a statement.
semantic network	A graph in which the node represents objects and the edges represent relationships between objects.
sequence	One of the three constructs in structure programming.
sequential access	An access method in which the records in a file are accessed serially beginning with the first element.
sequential file	A file structure in which data must be processed sequentially from the first element in the file.
sequential search	A search technique used with a linear list in which searching begins at the first element and continues until the value of an element equal to the value being sought is located, or until the end of the list is reached.
server	In a client–server system, the centralized computer that provides auxiliary services (server programs).
shell	A user interface in some operating systems, such as UNIX.
shift cipher	A type of substitution cipher in which the key defines shifting of characters toward the end of the alphabet.
siblings	Nodes in a tree with the same parent.
side effect	A change in a variable that results from the evaluation of an expression: any input/output performed by a called function.
sign out	To terminate membership in a social media.
sign out	You need to be a member of Facebook to use it. To become a member, you need to sign up. To terminate your membership, you need to sign out or deactivate your account.
sign-and-magnitude representation	A method of integer representation in which 1 bit represents the sign of the number and the remaining bits represent the magnitude.
simple type	An atomic data type such as integer or real.
single instruction stream, multiple data stream (SIMD)	A computer with one control unit, multiple ALU, and one memory unit.

single-user operating system	An operating system in which only one program can be in memory at a time.
small computer system interface (SCSI)	An I/O device controller with a parallel interface.
snooping	Unauthorized access to confidential information.
social contract	A principle of ethics that says an act is ethical if a majority of people in society agree with it.
social contract	The social contract theory says that an act is ethical when a majority of people in society agrees with it.
social media	Websites used by people to share ideas and messages.
social network	Another name for social media.
software	The application and system programs necessary for computer hardware to accomplish a task.
software agent	In artificial intelligence applications, a set of programs that are designed to do particular tasks.
software engineering	The design and writing of structured programs.
software lifecycle	The life of a software package.
software quality	Three characteristics of a software (operability, maintainability, and transferability).
solvable problem	A problem that can be solved by a computer.
soma	The body of a neuron that holds the nucleus of the cell.
something inherent	A characteristic of the claimant, such as conventional signature, fingerprint, voice, and so on, used for entity authentication.
something known	A secret known by the claimant that can be used by the verifier in entity authentication.
something possessed	Something belonging to the claimant that can prove the claimant's identity.
sort pass	One loop during which all elements are tested by a sorting program.
sorting	The process that orders a list or file.
source program	The file that contains program statements written by a programmer before they are converted into machine language: the input file to an assembler or compiler.
source-to-destination delivery	The delivery of a data packet from the source to the destination.
spatial compression	Compression done on a frame by the JPEG encoding process.

speech recognition	The first step in natural language processing (in AI) in which the speech signal is analyzed and the sequence of words it contains are extracted.
spoofing	See *masquerading*.
stack	A restricted data structure in which data can be inserted and deleted only at one end, called the top.
Standard Ethernet	The original Ethernet operating at 10 Mbps.
starvation	A problem in the operation of an operating system in which processes cannot get access to the resources they need.
state chart	A diagram, similar to a state diagram, but used in object-oriented software engineering.
state diagram	A diagram that shows the different states of a process.
statement	A syntactical construct in C that represents one operation in a function.
static RAM (SRAM)	A technology that uses traditional flip-flop gates (a gate with two states, 0 and 1) to hold data.
steganography	A security technique in which a message is concealed by covering it with something else.
storage device	An I/O device that can store large amounts of information for retrieval at a later time.
stream cipher	A cipher that encryption and decryption are done a character at a time.
String	A correction of characters considered as a single unit of data.
string	A string, as a set of characters, is treated in different languages differently. In C, a string is anarray of characters. In C++, a string can be an array of characters, but there is a type named string. In Java, a string is a type.
structure chart	A design and documentation tool that represents a program as a hierarchical flow of functions.
structured program	A program written according to the rules of Software Engineering.
Structured Query Language (SQL)	A database language that includes statements for database definition, manipulation, and control.
subalgorithm	A part of an algorithm that is independently written and is executed when called inside the algorithm.
subprogram	A smaller program called by the main program.

subroutine	See *subalgorithm*.
substitution cipher	A cipher that replaces one symbol with another.
subtree	Any connected structure below the root of a tree.
summation	Addition of a series of numbers.
switch	A device that connects components in a network together.
switched WAN	A complex WAN made of several medium and several switches.
symbolic language	A computer language, one level removed from machine language, that has a mnemonic identifier for each machine instruction and can use symbolic data names.
symmetric-key cipher	A type of cryptography in which a single secret key is used for both encryption and decryption.
symmetric-key encryption	Encryption using a symmetric-key cipher.
synapse	A connection between two neuron in the human nervous system.
synonym	In a hashed list, two or more keys that hash to the same home address.
syntactic analysis	The analysis of a sentence to check for grammar.
syntax	The grammatical rules of a language.
syntax analyzer	The process that checks the grammar of a sentence.
system documentation	A formal structured record of a software package.
TCP/IP protocol suite	A five-layer protocol suite that defines the exchange of transmission across the Internet
technical documentation	Documenting the procedures for installation and servicing of the software system.
TELNET (terminal network)	A general-purpose client-server program that allows remote login.
temporal compression	Compression done by MPEG on frames.
temporal logic	A type of logic that includes change and the effect of time in reasoning.
temporal masking	A process that a loud sound eliminates the effect of less loud sound.
terminated state	In process management, a state in which a process has finished executing.
testability	An attribute of software that measures the ease with which the software can be tested as an operational system.

testing phase	A phase in the software lifecycle in which experiments are carried out to prove that a software package works.
text	Data stored as characters.
text editor	Software that creates and maintains text files, such as a word processor or a source program editor.
text file	A file in which all data is stored as characters. Contrast with *binary file*.
thresholding	A method used in image segmentation in which a pixel with a specific intensity is selected and the method tries to find all the pixels with the same intensity.
throughput	The number of data units that can be passed through a point in one unit of time.
ticket	In session key distribution, an encrypted message containing the session key intended for Bob, but sent to Alice to be deliver to Bob.
time sharing	An operating system concept in which more than one user has access to a computer at the same time.
token	A syntactical construct that represents an operation, a flag, or a piece of data.
topology	The structure of a network, including the physical arrangement of devices.
track	A part of a disk.
trade secret	Information about a product that is kept secret.
trademark	A sign or name that identifies a company product.
traffic analysis	A type of attack on confidentiality in which the attacker obtains some information by monitoring online traffic.
transaction file	A file containing relatively transient data that are used to change the contents of a master file.
transfer time	The time to move data from the disk to the CPU/memory.
transferability	A quality in software system that refers to the ability to move the system from one platform to another.
translator	A generic term for any of the language conversion programs. See also *assembler* and *compiler*.
Transmission Control Protocol (TCP)	One of the transport-layer protocols in the TCP/IP protocol suite.
Transmission Control Protocol/ Internet Protocol (TCP/IP)	The official protocol of the Internet, composed of five layers.

transmission medium	The physical path linking two communication devices.
transmission rate	The number of bits sent per second.
transposition cipher	A cipher that transposes symbols in the plaintext to create the ciphertext and vice versa.
traversal	An algorithmic process in which each element in a structure is processed once and only once.
tree	A set of connected nodes structured so that each node has only one predecessor.
Trojan horse	A program that can do malicious acts such as deleting or corrupting files.
True-Color	A technique in raster graphic that uses 24 bits to represent a color.
truncation error	The error that occurs when a number is stored using floating-point representation. The value of the number stored may not be exactly as expected.
truth table	A table listing all the possible logical input combinations with the corresponding logical output.
tuple	In a relational database, a record (a line) in a relation.
Turing machine	A computer model with three components (tape, controller, and read/write head) that can implement statements in a computer language.
Turing model	A computer model based on Alan Turing's theoretical definition of a computer.
Turing test	A test devised by Alan Turing to determine whether a computer can be said to be truly intelligent.
tweet	A short (maximum 140 characters) message exchanged by users in Twitter. Sending a tweet.
twisted-pair cable	Two insulated cable twisted together in a cover.
Twitter	A popular social network where users can post short messages called tweets.
two-dimensional array	An array with elements having two levels of indexing. See also *multidimensional array.*
two's complement	A representation of binary numbers in which the complement of a number is found by complementing all bits and adding a 1 after that.
two's complement representation	A method of integer representation in which a negative number is represented by leaving all the rightmost 0s and the first unchanged and complementing the remaining bits.

unary operation	An operation that needs only one input operand.
underflow	An event that occurs when an attempt is made to delete data from an empty data structure.
undirected graph	A graph consisting only of edges—that is, a graph in which there is no indication of direction on the lines.
unfollow	A process whereby a user stops following another user on Twitter.
unguided media	Transmission media with no physical boundries.
Unicode	A 32-bit code that includes the symbols and alphabets from most languages in the world.
Unified Modeling Language (UML)	A graphical language used for analysis and design.
Uniform Resource Locator (URL)	A string of characters that defines a page on the Internet.
union operation	An operation on two sets in which the result contains all the elements from both sets without duplicates.
universal serial bus (USB)	A serial I/O device controller that connects slower devices such as the keyboard and mouse to a computer.
UNIX	A popular operating system among computer programmers and computer scientists.
unsigned integer	An integer without a sign whose value ranges between and positive infinity.
unsolvable problem	A problem that cannot be solved by a computer.
update operation	An operation in a relational database in which the operation about one tuple is changed.
use-case diagram	A diagram showing the user view of a system in UML.
user agent (UA)	An SMTP component that prepares the message, creates the envelope, and puts the message in the envelope.
user datagram	The data unit used by the UDP protocol.
User Datagram Protocol (UDP)	One of the transport-layer protocols in the TCP/IP protocol suite.
user interface	A program that accepts requests from users (processes) and interprets them for the rest of the operating system.
user page	The main page in Facebook.
user page	The user page is the main page you will use on Facebook.
users	In DBMS terminology, every entity that access the database.
utility	An application program in UNIX.

utilization	A principle in ethics that says an act is ethical if it creates a good result.
utilization	The second theory of ethics, related to the consequences of the act. An act is ethical if it results in consequences which are useful for society.
variable	A memory storage object whose value can be changed during the execution of a program. Contrast with *constant*.
vector graphic	A type of graphics file format in which lines and curves are defined using mathematical formulas.
verifying algorithm	The algorithm that verifies the validity of a digital signature.
vertex	A node in a graph.
vi	A screen text editor available in Unix operating system.
video	A representation of images (called *frames*) in time.
virtual memory	A form of memory organization that allows swapping of programs between memory and magnetic storage to give the impression of a larger main memory than really exists.
virus	Unwanted program hidden inside another program, and which can replicate itself.
von Neumann model	A computer model (consisting of memory, arithmetic logic unit, control unit, and input/output subsystems) upon which the modern computer is based.
waiting state	A state in which a process is waiting to receive the attention of the CPU.
waterfall model	A software development model in which each module is completely finished before the next module is started.
Web	See *World Wide Web*.
Web page	A unit of hypertext or hypermedia available on the Web.
well-known port number	A port number that defines a process on the network.
white-box testing	Program testing in which the internal design of the program is considered. Also known as *glass-box testing*. Contrast with *black-box testing*.
wide area network (WAN)	A network that spans a large geographical distance.
WiMax	Worldwide Interoperability Access.
Windows	The operating system designed by Microsoft.
Windows 2000	A version of the Windows NT operating system.

Windows NT	An operating system devised by Microsoft to replace MS-DOS.
Windows XP	A version of the Windows NT operating system.
working directory	The directory that is currently being used.
World Wide Web (WWW)	A multimedia Internet service that allows users to traverse the Internet by moving from one document to another via links.
Worldwide Interoperability Access	Wireless version of DSL or cable connection to the Internet.
worm	An independent program that travels through the network and can copy itself.
write once, read many (WORM)	Another name for a CD-R.
X.509	A technique used by modern computers to improve the throughput by combining different phases of on instruction with the next.
X.509	X.509 is a way to describe a certificate in a structured way. It uses a well-known protocol called ASN.1 that defines fields familiar to computer programmers.
XOR operator	A bitwise operation in which the result of the operation is 1 only if one of the operands is 1.
zero-knowledge authentication	An entity authentication method in which the claimant does notreveal anything that might endanger the confidentiality of the secret.

INDEX

A

10 Gigabit Ethernet 168, 169
ABC (Atanasoff Berry Computer) 10
absolute pathname 361, 629
abstract data type (ADT) 629
  background 318–20
  binary search tree 343
  complex 318
  definition 319
  general linear lists 331–7
  graphs 343–4
  implementation 320
  model 319
  queues 326–31
  simple 318
  stacks 320–6
  trees 337–43
AC value 402, 629
access method 629
acronyms 625–7
actions 546
activation 547
active agent 248
active document 629
activity 548
activity diagram 548–9
actors 540, 547
actual parameters 264, 629
ad hoc network 170
Ada 246, 251, 629
additive cipher 417, 629
address
  as bit pattern 95
  bus 105, 629
  space 94, 629
addressing 44, 142–3
ADSL *see* asymmetric DSL
ADT *see* abstract data type
Advanced Encryption Standard (AES) 629
AES *see* Advanced Encryption Standard
Aiken, Howard 10
algebraic method 571–2
algorithm 8, 11, 326, 330–1, 337, 629
  basic 224–33
  constructs 218
  defining actions 216
  encryption 639
  formal definition 223–4
  generalization 217
  header 554
  informal definition 214–16
  produce a result 223
  recursion 234–6
  refinement 216–17
  representation 219–23
  smallest/largest 225–6
  subalgorithms 233–4
  terminate in finite time 224
  unambiguous steps 223
  well-defined 223
ALU *see* arithmetic logic unit
American National Standards Institute (ANSI) 61, 375, 629
American National Standards Institute/ Standards Planning and Requirements Committee (ANSI/SPARC) 372
American Standard Code for Information Interchange (ASCII) 61, 535–8, 630
analog 61, 630
  data 175
  signal 175
  transmission 176–7
analog-to-analog conversion 177, 630
analog-to-digital conversion 176, 630
analysis phase 276, 630
  object-oriented 277–9
  procedure-oriented 276–7
Analytical Engine 9, 251
ancestor 337, 630
AND operator 75, 565, 630
ANSI *see* American National Standards Institute
ANSI/SPARC *see* American National Standards Institute/ Standards Planning and Requirements Committee
anti-virus software 530
APL language 247
applet 254, 630
application gate-way 444, 630
application layer 141, 143–4, 630
  domain name systems 152–4
  electronic mail 150–1

application layer (*Continued*)
File Transfer Protocol 149–50
paradigms 144–6
peer-to-peer paradigm 154–6
providing services 144
Secure Shell 151–2
standard client-server 146–9
TELNET 151
application programs 204–5, 372, 630
applications 11
arc 630
architecture
computer 11, 113–17
database 372–3
email 150–1
expert system 486–7
layered 141–2
Windows 206–7
Aristotle 474
arithmetic logic unit (ALU) 5, 9, 93, 630
arithmetic operations 82, 93, 597–8, 630
on integers 82–5
on reals 86
arithmetic operator 260, 630
arithmetic shift operation 81–2, 630
array 257, 292–4, 630
application 298
array name vs element name 294
arrays vs array of records 301
comparison with records 299
memory layout 295–6
multidimensional 294–5
operations 296–8
strings 298
traversal 297–8
artificial intelligence 11, 630
definition 474
expert systems 485–7
history 474
intelligent agents 474–5
knowledge representation 475–85
neural networks 498–501
perception 487–93
programming languages 475
searching 494–8
Turing test 474
ASCII *see* American Standard Code for Information Interchange
assembler 630
assembly language 245, 631
assign 554
assign and add 555
assignment statement 631
association 542
asychronous devices 578
asymmetric DSL (ADSL) 171–2
asymmetric-key cipher 423–4, 437, 631
general idea 424–5
RSA cryptosystem 426–8
Atanasoff, John V. 10
attacks, security
availability 414–15
confidentiality 413
integrity 413–14
attribute 374–5, 541, 543, 631
audio 631
storage 61–3
audio compression 405
perceptual encoding 405–6
predictive encoding 405
authentication 631
challenge-response 436–8
data-origin/message 435
digital signature 434, 438
entity 435–8
security 430
autokey cipher 631
auxiliary storage device 350, 631
availability 412, 631
axioms 568
axon 498, 631

B

B-frame *see* bidirectional frame
Babbage, Charles 9, 251
backbones 136
backward chaining 484–5
base 631
BASIC language 246, 247
basic multilingual plane 534
basic service point (BSS) 170
basis path testing 283–4, 631
batch operating systems 189, 631
behavioral view 539, 543
activity diagrams 548–9
collaboration diagrams 543–4
sequence diagrams 546–8
state diagrams 545–6
swimlanes 549–50
Bell, Wayne 154
Berry, Clifford 10
bidirectional frame (B-frame) 404, 631
big-O notation 468, 631
binary 5
digit 40, 631
file 362, 631
operation 631
search 230, 231–3, 631
binary search tree (BST) 341–2, 632
ADTs 343
implementation 343
binary system 18–20, 632
conversions 29–30
binary tree 338
applications 341
implementation 341
recursive definition 338–9
traversals 339–40
biological neurons 498–9
biometrics 632
bit 40, 632
depth 632
logic operations 74–6
pattern 40–1, 95, 632
per sample 62
rate 63, 171, 632
bit-oriented cipher 421
bitmap graphics 63–5, 632
black-box testing 285, 632
block cipher 421, 632
block coding 603–6
Bluetooth 170, 632
body halt 461
body start 461
Boole, George 74, 474
Boolean algebra 74, 563, 632
axioms 568
constants 563
expressions 564
function simplification 571–4
identities 568–9
logic gates 564–7
operators 563
table-to-expression transformation 570–1
theorems 568
variables 563
Boolean function 569–71
Boolean type 257
bootstrap process 188–9, 632
boundary-value testing 285
breadth-first search 495–6
breadth-first traversal 340, 632
browser 148, 632
brute-force search 495–7, 632
BSS *see* basic service point
BST *see* binary search tree
bubble sort 228–9, 632

bucket 359, 632
bucket hashing 359, 632
buffer 564
burst error 602
bus 104–5, 633
  topology 633
Byron, Ada 251
byte 633
bytecode 247, 633

C
C++ language 246, 254, 584–7, 633
C language 246, 251, 581–4, 633
CA *see* certification authority
cable network 172
cache memory 97, 633
Caesar cipher 418, 633
calls 249
cardinality 375, 633
case statement 262
CD *see* compact disk
CD-R *see* compact disk recordable
CD-ROM *see* compact disk read-only memory
CD-RW *see* compact disk rewritable
CDS *see* Content Distributed Systems
cellular telephony network 173
central processing unit (CPU) 92–4, 118, 633
  connecting memory 104–5
centralized networks 155
certification authority (CA) 441–2, 633
CGI (Common Gateway Interface) 634
chaining
  backward 484–5
  forward 484
challenge-response authentication 436, 633
  asymmetric-key cipher 437
  digital signature 438
  symmetric-key cipher 436–7
character type 257
character-oriented cipher 421
chatting 633
checksum 613, 614–15
  concept 613
  Internet 614–15
  one's complement 613–14
child 337, 633
Church-Turing thesis 462–3, 633
cipher 633
  asymmetric-key 423–8
  symmetric-key 415–23
ciphertext 325, 633
circular shift operation 80–1, 633
circular waiting 201, 633
CISC *see* complex instruction set computer
Clarke, Ian 154
class 252–3, 633
  diagram 278, 541, 634
  library 254
client-server paradigm 144–5, 146, 634
coaxial cable 178, 179, 634
COBOL (COmmon Business-Oriented Language) 11, 246, 251, 634
code 61, 634
code generator 248
codewords 603
coding 602–3
cohesion 280, 634
cold fusion 634
collaboration diagram 543–4
collision 356, 358, 634
  resolution 358–60, 634
color depth 63, 634
Colossus 10
column-major storage 295, 634
combination
  approaches 360
  ciphers 421
  statements 381
combinational circuits 574–5
  half adder 575
  multiplexer 575–6
compact disk (CD) 99–100, 634
compact disk read-only memory (CD-ROM) 100–1, 634
compact disk recordable (CD-R) 102, 634
compact disk rewritable (CD-RW) 102–3, 634
compatibility 206
compilation 246, 634
compiler 246, 321, 635
complex ADTs 318
complex instruction set computer (CISC) 113–14
complexity of problems
  solvable 467–8
  unsolvable 467
component diagrams 550
components
  data 6–7
  hardware 6
  Linux 205
  operating systems 191–203
  software 7–9
  survey of 203–7
composite data type 257, 635
compound condition 284
compound statement 261, 635
compression 397–8, 403
compression function 429
computer
  architecture 11, 113–17
  generations 10–11
  history 9–11
  language 8, 244, 635
  simple 117–26
  vision 488
computer crime 528
  attack protection 529
  cost 530
  denial of service 529
  install strong anti-virus software 530
  motives 529
  penetration attack 529
  physical protection 529
  software protection 529
computer science 11
  applications 11
  systems 11
conceptual level 372, 635
condition testing 284
confidentiality 370, 635
  asymmetric-key ciphers 423–8
  digital signature 435
  security goal 412
  symmetric-key ciphers 415–23
congestion control 635
connecting device 134, 635
connection-oriented protocol 635
connectionless delivery 162
connectionless protocol 635
constant 259, 635
Content Distributed Systems (CDS) 155
control
  bus 105, 635
  statement 261–3, 635
  structure testing 284–5, 635
  unit 5, 94, 118, 635
controller 105–8, 457–9, 635

conversion 23
  any base to decimal 23–4
  binary-hexadecimal 29
  binary-octal 29–30
  decimal to any base 24–8
  fractional part 26–8
  integral part 24–6
  number of digits 28
  octal-hexadecimal 30–1
convolution coding 603
copy 249
copyright 538
counting 44
country domain 154, 635
coupling 280, 635
CPU *see* central processing unit
cryptographic hash function 428–9, 636
cryptography 415, 636
cryptosystem 636
cur pointer 304, 305
current directory 636
current symbol 457
customer networks 136
cycles 122–6
cyclic code 610–13
cyclic redundancy checks (CRC) 610–13
cyclomatic complexity 284

D

D flip-flop 577
data 6, 249, 371
  access 99
  bus 104, 636
  confidentiality 636
  file 636
  integrity 370, 636
  items 249
  organization 7
  registers 93, 118
  storage 6, 122
  types 40–2, 257, 636
data compression 41–2, 363, 392
  lossless compression methods 392–400
  lossy compression methods 400–6
data encryption standard (DES) 636
data flow diagram 276, 636
data flow testing 285
data items
  pairing 324–5
  reversing 321–2
data processor 2, 636
  programmable 2–4
data structure 319, 636
  arrays 292–8
  linked lists 301–13
  records 298–301
data-link layer 166, 636
  local area networks 168–70
  nodes/links 166–8
  wide area networks 171–3
data-origin authentication 435
database 11, 636
  advantages 370
  architecture 372–3
  definition 370
  design 381–5
  facts 483
  models 373–4, 636
database management systems (DBMS) 371, 636
  data 371
  hardware 371
  procedures 372
  software 371
  users 371–2
datagram 636
datawords 603
DBMS *see* database management systems
DC value 402, 636
DCT *see* discrete cosine transform
DDS *see* Distributed Data Structure
deadlock 199, 200–1, 637
decentralized networks 155–6
decimal digit 16, 637
decimal system base 16–18, 637
decision 218, 549
decision logic analyzer 608
decision point 545
declarative paradigm 256, 637
decode 110, 114, 637
decoder 612
decoding 396
decompression 399–400
decr (X) statement 460
decrement statement 452, 460, 637
decryption 325, 637
decryption algorithm 637
deduction 478–9, 482
default logic 483, 637
delete 249
  node 309–11
  operation 332–3
  transactions 352
delete operation 376, 637
demand paging 194, 637
demand paging and segmentation 195, 637
demand segmentation 195, 637
demodulator 637
demultiplexing 637
dendrite 498, 637
denial of service (DoS) 414, 529, 637
deployment diagrams 551
depth-first search 496–7
depth-first traversal 339, 637
dequeue 327, 638
DES *see* data encryption standard
descendant 337, 638
design phase 279, 638
  object-oriented design 280
  procedure-oriented design 279–80
development process 274, 638
device manager 202, 638
DHT *see* Distributed Hash Table
dial-up service 171
dictionary-based encoding 638
Difference Engine 9
difference operation 380–1, 638
digest 429, 638
digit 16
  conversions 28, 30–1
  extraction hashing 357, 638
digital 638
  counter 580
  data 175
  divide 638
  signal 175
digital signature 431, 638
  comparison 431–2
  confidentiality 435
  duplicity 432
  inclusion 431
  message authentication 434, 438
  message integrity 434
  nonrepudiation 434–5
  process 432–3
  relationship 431
  services 433–5
  signing the digest 433
  verification method 431
digital subscriber line (DSL) 171–2, 638, 639
digital versatile disk (DVD) 103–4, 638
digital-to-analog conversion 177, 638
digital-to-digital conversion 175–6, 638

digraph 343, 638
direct hashing 356, 638
direct memory access (DMA) 112–13, 638
directed graph 343, 638
directory 360–1, 639
  paths/pathnames 361
  special 361
discrete cosine transform (DCT) 401–3, 599, 639
Distributed Data Structure (DDS) 155
distributed database 385, 639
  fragmented 385–6
  replicated 386
Distributed Hash Table (DHT) 155–6
distributed system 190, 639
division 611
division remainder hashing 356–7, 639
divisor 613
DMA *see* direct memory access
DNS *see* domain name server
do-while loop 263
documentation 285
  system 286
  technical 286
  user 286
domain name server (DNS) 152–4, 155–6, 639
domain name space 639
DoS *see* denial of service
dotted-decimal notation 639
DRAM *see* dynamic RAM
DSL *see* digital subscriber line
duplicity 432
DVD *see* digital versatile disk
dynamic document 639
dynamic RAM (DRAM) 96, 639

E

Eckert, J. Presper 10
edge 343, 639
edge detection 488–9, 639
EDSAC 10
EDVAC 10
EEPROM *see* electrically erasable programmable read-only memory
electrically erasable programmable read-only memory (EEPROM) 96, 639
electronic computer 10
electronic mail (email) 150–1, 474–5, 639
  categories 475
  Facebook 513
  retrieving 297
elements
  deletion 297
  insertion 296–7
  retrieving 297
  searching 296
emacs 204, 639
email *see* electronic mail
empty operation 321–2, 328, 334
encapsulation 639
encoder 611–12
encoding 62, 395
encryption 325, 639
encryption algorithm 639
end point 549
end system 134, 639
end users 371, 640
ENIAC (Electronic Numerical Integrator and Calculator) 10
enqueuque operation 327, 640
entity authentication 640
  challenge-response 436–8
  entity vs message 435
  passwords 436
  verification categories 436
entity-relation model (ERM) 381–2
entity-relationship (E-R) diagram 276, 382–3, 640
environmental subsystems 207
ephemeral port number 158, 640
EPROM *see* erasable programmable read-only memory
erasable optical disk 102
erasable programmable read-only memory (EPROM) 96, 640
error detection/correction 42
  block coding 603–6
  checksum 613–15
  coding 602–3
  cyclic codes 610–13
  detection vs correction 602
  forward error correction vs retransmission 602
  linear block codes 606–9
  redundancy 602
  types of errors 601–2
error report file 353, 640
ESS *see* extended service set
ethical principles 526
  moral rules 526
  social contract 526
  utilization 526
events 545
excess representation 640
excess system 55
execute 110, 114, 640
executive 207
exhaustive testing 285
expert system 485, 640
  architecture 486–7
  extracting facts 486
  extracting knowledge 485–6
explanation system 487
exponent 54
exponential functions 591–3
expression 260–3, 640
expression tree 341, 640
Extended Binary Coded Decimal Interchange Code (EBCDIC) 640
extended service set (ESS) 170
extensibility 206
external level 373, 640

F

Facebook 640
  accessing services 512
  commenting on posted updates 514
  communication 509–10
  deactivate 512
  exchanging information 513–14
  following email contacts 513
  friends 512–13
  friendship 508–9, 513
  limiting who can *see* posts 514
  log in 512
  log out 512
  membership 511–12
  posting news, photos, videos 514
  reading news 513–14
  sharing posted updates 514
  sign out 512
  sign up 511–12
  tagging 514
  web pages 510–11
facts 483, 486
  database 487
false value 304
Fanning, Shawn 154
Fast Ethernet 168
feedback shift register (FSR) 422–3
fetch 110, 114, 640

fiber-optic cable 178, 179, 641
field 298, 641
FIFO *see* first in, first out
file 249
file manager 202–3
file structure 350
  directories 360–2
  hashed 355–60
  indexed 354–5
  sequential 350–3
  text vs binary 362
file transfer protocol (FTP) 145, 149–50, 641
fingerprint 428
finite state automaton 641
firewall 442–3, 641
  packet-filter 443–4
  proxy 444
FireWire 106, 641
first in, first out (FIFO) 199, 326, 641
fixed-point representation 42, 641
flat-file 370, 641
flip-flop 576–8
flip-flop gates 95
floating-point representation 52–4, 641
  IEEE standards 56–8
following 641
  Facebook 513
  Twitter 520
for loop 263
fork 549
formal parameters 264, 641
FORTRAN (FORmula TRANslation) 11, 250, 641
forward chaining 484
forward error correction 602
fraction 298
fragmented distributed database 385–6, 641
frames 65, 168, 194, 476–7, 641
frequency masking 641
friends 508, 512, 641
  accepting 513
  finding 512–13
  unfriending 513
friendship 508
front 641
FSR *see* feedback shift register
FTP *see* file transfer protocol
function 254
functional language 255–6, 642
functional paradigm 254–6, 642

G

general linear list 331, 642
  ADT 334
  applications 334–6
  implementation 336–7
  operations 331–4
generalization 542
generic domain 153–4, 642
GIF *see* Graphic Interchange Format
Gigabit Ethernet 168, 169, 642
glass-box testing 283–5, 642
go to statement 261
Gödel, Kurt 463
Gödel numbers 463, 642
  interpreting a number 464
  representing a program 463–4
graph 343–4, 642
graph theory 284
Graphic Interchange Format (GIF) 642
graphical user interface (GUI) 642
GUI *see* graphical user interface
guided media 178–9, 642

H

hacker 530, 642
HAL *see* hardware abstraction layer
half adder 575
halting problem 465–7
halting program 464–5
Hamming codes 609
hardware 6, 137, 188, 371, 642
hardware abstraction layer (HAL) 206, 642
hash function 429
hashed file 355, 642
hashing method 355, 642
hashtag 521, 642
HDNI *see* High-Definition Multimedia Interface
header 158, 642
healting problem 642
heuristic search 497–8, 642
heuristic value 497
hexadecimal digit 464, 642
hexadecimal system 20–1, 642
  binary conversion 29
  octal conversion 30–1
hidden layers 500
hierarchical model 373, 643
High-Definition Multimedia Interface (HDMI) 108, 643
high-level language 244, 245–6, 643
high-order logic 482, 643
HMAC (hashed MAC) 430
hold state 197, 643
Hollerith, Herman 9
home address 358, 643
home directory 643
home page 510, 516, 517, 643
host 134, 643
host identifier 148, 643
hot-swappable 107
HTML *see* HyperText Markup Language
HTTP *see* HyperText Transfer Protocol
hub 107, 643
Huffman coding 341, 393–6, 643
HyperText Markup Language (HTML) 148, 643
HyperText Transfer Protocol (HTTP) 145, 149, 643

I

I-frame *see* intracoded frame
identifier 257, 643
identities 568
IEEE *see* Institute of Electrical and Electronics Engineers
if-else statement 262
if-then-else 554
illegal 603
image processing 488, 643
  applications 491
  edge detection 488–9
  finding depth 489
  finding orientation 490
  object recognition 490
  segmentation 489
images
  standards for encoding 65
  storage 63–5
IMAP *see* Internet Mail Access Protocol
imperative language 257, 643
imperative paradigm 248, 643
implementation 267
implementation phase 280, 643
  choice of language 281
  software quality 281–3
inconsistency avoidance 370
incr (X) statement 459
increment statement 452, 459–60, 644
incremental model 275, 644

index 292, 354, 644
indexed color 64, 644
indexed file 354–5, 644
inductive learning 498
inference engine 487
infix 341, 644
infrared waves 180, 644
inheritance 253, 644
inorder traversal 340, 644
input 5, 215, 259, 455, 554
  data 2–4, 644
  layer 500
  subsystem 5
input/output 5
  controllers 105, 644
  direct memory access 112–13
  interrupt-driven 111–12
  isolated 108
  memory-mapped 109
  programmed 110–11
input/output (I/O) subsystem 97–8, 119, 644
  connecting 105–9
  nonstorage devices 98
  storage devices 98–104
insert operation 332, 375–6, 644
inserting a node 306
  at the beginning 306–7
  at the end 307
  empty list 306
  in the middle 308
insertion of elements 296
  at the beginning/middle 296–7
  at the end 296
insertion sort 229, 644
Institute of Electrical and Electronics Engineers (IEEE) standards 55–60
instruction 644
  processing 120–1
  register 93, 644
  sequence 6, 7–8
  set 119–20
integer 17, 257, 644
  arithmetic operations 82–5
  binary system 19
  decimal system 17–18
  hexadecimal system 20–1
  maximum value 18, 19, 21
  octal system 21–2
  operations 617–20
  storing 42–50
integrated circuit 644
integrity 412, 644
intellectual property 527, 645
  types 527–8
intelligent agent 474–5, 645
interface 319, 644
internal level 372, 645
internal node 337, 645
International Organization for Standardization (ISO) 375, 645
Internet 136–7, 645
  address 645
  checksum 613–15
  domain name space (tree) 153–4
  static document 645
Internet Mail Access Protocol (IMAP) 645
Internet Protocol (IP) 163, 645
  Version 4 (IPv4) 163–4
  Version 6 (IPv6) 164–6, 645
internet service provider (ISPs) 136, 645
internetwork 135–6, 645
interpretation 246, 246–7
  first approach 247
  second approach 247
interpreter 246, 484, 644
interrupt-driven I/O 111–12, 645
intersection operation 380, 645
intersector gap 99, 645
intertrack gap 99, 645
intracoded frame (I-frame) 404, 645
invalid 603
inverse discrete cosine transform 599
inverted file 355, 645
IP datagram 158, 646
IP new generation (IPng) 164, 646
ISO *see* International Organization for Standardization
isolated I/O 1–8, 646
iterated cryptographic hash function 429
iterative definition 234
iterative solution 235

J

Jacquard, Joseph-Marie 9
Jacquard loom 9
Java 246, 254, 587–90, 646
Java Virtual machine (JVM) 247
JK flip-flop 577–8
job 196–7, 646
  control block 199
  scheduler 198, 646
join operation 646
joint 549
JPEG (Joint Photographic Experts Group) 400–1, 646
  compression 403
  DCT 401–3
  quantization 403
jump instruction 261
JVM *see* Java Virtual machine

K

Karnaugh map 572–4
KDC *see* key distribution center
kernel 204, 205, 206, 646
key 352, 646
key distribution center (KDC) 439, 646
  multiple 439
  session keys 440–1
key management 438
  public-key distribution 441–2
  symmetric-key distribution 438–41
keyboard 98, 646
knowledge
  base 483, 487
  base editor 487
  extracting 485–6
knowledge representation 475
  frames 476–7
  predicate logic 477–83
  rule-based systems 483–5
  semantic networks 476

L

LAN *see* local area network
land 100, 646
language understanding 491
  pragmatic analysis 493
  semantic analysis 493
  speech recognition 491
  syntactic analysis 491–3
languages 8, 11
  assembly 245
  choice of 281
  high-level 245–6
  machine 24405
  programming 244–67
  symbolic 245
last in, first out (LIFO) 320, 646
layered architecture 141–2
leaf 646
learning by example 498
Leibniz, Gottfried 9, 474

Leibniz Wheel 9
Lempel, Abraham 306
Lempel Ziv (LZ) encoding 396–400, 646
lexical analyzer 247, 646
lifeline 547
LIFO *see* last in, first out
linear block codes 606–9
linear list 320, 646
link 166, 544, 646
linked list 301–2, 647
  applications 313
  arrays vs linked lists 302–3
  nodes names vs linked list names 303
  operations 303–13
  resolution 359, 647
  searching 303–6
  traversal 647
linking tool 302
Linux 205, 647
LISP (LISt Programming) 255, 475, 647
list operation 332, 647
listserv 647
literal 258, 647
loader 647
local area network (LAN) 134, 168–70, 647
local login 647
local variable 264, 647
log in 512
log out 512
logarithmic function 593–5
logic
  developments 482–3
  predicated 477, 480–2
logic circuits 574
  combinational 574–6
  sequential 576–80
logic gate 564–5
  implementation 565–7
logic operations 74, 93
  at bit level 74–6
  at pattern level 76–9
  complementing 77
  flipping specific bits 78–9
  setting specific bits 78
  unsetting specific bits 77–8
logical
  address 647
  connections 139–40
  operation 5, 647
  operators 260–1
  shift operations 79–82, 647
loop 647
  statement 452, 461–2, 554, 556, 647
  testing 285
lossless data compression 392, 647
  Huffman coding 393–6
  Lempel Ziv encoding 396–400
  run-length encoding 392–3
LZ *see* Lempel Ziv encoding

M

MAA *see* Message Access Agent
MAC *see* message authentication code
McCabe, Tom 283
McCarthy, John 474, 475
machine cycle 109–10, 647
machine language 8, 244–5, 647
macro 453–5, 648
magnetic disk 98, 648
magnetic storage devices 98–9
magnetic tape 99, 648
main memory 94, 118–19, 648
  address space 94–5
  cache 97
  hierarchy 96–7
  types 95–6
maintainability 282, 648
  changeable 282
  correctable 282
  flexible 282
  testable 282
MAN *see* metropolitan area network
mantissa 54–5, 648
mask 648
masquerading 414, 648
master disk 100, 648
master file 648
Mauchly, John 10
mechanical machines 9
memory 5, 648
  cache 97
  connecting CPU 104–5
  hierarchy 96–7
  layout 295–6
  management 191, 648
  types 95–6
memory-mapped I/O 109, 648
merge 549, 648
mesh topology 648
message 544, 547
  authentication 434, 435
  digest 428–9, 648
  integrity 428–9, 434
Message Access Agent (MAA) 648
message authentication code (MAC) 430, 648
message transfer agent (MTA) 648
method 253, 541, 544
metropolitan area network (MAN) 649
microcomputer 11, 649
micromemory 114
microoperations 114
microprogramming 114
Microsoft Disk Operating System (MS-DOS) 649
microwaves 180
MIMD *see* multiple instruction-stream, multiple data-stream
MIME *see* Multipurpose Internet Mail Extension
minicomputer 11
mintern 570
MISD *see* multiple instruction-stream, single data-stream
mod 596
modal logic 482, 649
modem 649
modern block cipher 421
modern stream cipher 422
modification 414
modular arithmetic 595–8
modularity 279–80, 649
modulator 649
module 557–8, 649
modulo division hashing 356–7, 649
modulo operator 596
modulus 596
monitor 98
monoalphabetic cipher 417–18, 649
monoprogramming 191–2, 649
moral rule 526, 649
motion 489
MP3 (MPEG audio layer 3) 63, 400, 404, 649
  perceptual encoding 405–6
  predictive encoding 405
  spatial compression 404
  temporal compression 404
  versions 405
MPEG (Motion Picture Experts Group) 63, 400, 649
MS DOS *see* Microsoft Disk Operating System

MTA *see* message transfer agent
multidimensional array 649
multilayer networks 500–1
multiple instruction-stream, multiple data-stream (MIMD) 117, 649
multiple instruction-stream, single data-stream (MISD) 116–17, 649
multiplexer 575–6, 649
multipoint connection 649
multiprogramming 190, 191, 192–5, 649
Multipurpose Internet Mail Extension (MIME) 649
multithreading 254, 650
mutual exclusion 201, 650

N

name 374, 650
name space 153, 650
NAND gate 565
  implementation 566–7
Napster 154–5
National Institute of Standards and Technology (NIST) 429, 430
network 134–6, 650
network layer 159–60, 650
  protocols 163–6
  services 160–2
network model 373, 650
networking 11
  capabilities 205
neural network 498, 650
  applications 500–1
  biological neurons 498–9
  multilayer networks 500–1
  perceptrons 499–500
neuron 498, 650
new master file 352, 650
Newton, Sir Isaac 474
NIST *see* National Institute of Standards and Technology
no preemption 201, 650
node 166, 302, 337, 650
  deleting 309–11
  inserting 306–8
  internal 337
  retrieving 311–12
node-to-node 166, 650
nonpolynomial problems 468, 650
nonpositional number system 650
nonrepudiation 434–5, 650
nonstorage devices 98
nonswapping 193
NOR gate 565
  implementation 567
normal form (NF) 650
normalization 54, 383–4, 650
NOT operator 75, 565–6, 650
NT file system (NTFS) 650
NTFS *see* NT file system
null
  binary tree 338
  pointer 301, 651
  tree 651
number system 651

O

object program 246, 651
object recognition 490
object-oriented
  analysis 277–9, 651
  database 386, 651
  design 280, 651
  language 651
  paradigm 252–4, 651
objects 477, 547
octal system 21–3, 651
  binary conversion 29–30
  hexadecimal conversion 30–1
off state 545
old master file 352, 651
on state 545
one-dimensional arrays 294, 651
one-time pad 422, 651
one's complement 47, 613–14, 651
open addressing resolution 358–9, 651
operability 281, 651
  accurate 281
  efficient 282
  reliable 282
  secure 282
  timely 282
  usability 282
operand 261, 651
operating system 9, 11, 188, 651
operations 319, 544
  general linear lists 331–4
operator 260, 651
optical storage devices 99–104, 651
OR operator 75, 565, 652
output 5, 259, 455, 554
  data 2
  date 652
  device 98, 652
  layer 500
  subsystem 5
overflow 59, 652
  sign-and-magnitude representation 46
  two's complement notation 50
  unsigned integers 43–4

P

P-frame *see* predicted frame
P2P *see* peer-to-peer
packet delivery 161–2
packet names 142–3
packet-filter firewall 443–4, 652
packetizing 160–1, 652
paging 194–5, 652
paradigm
  functional 254–7
  object-oriented 252–4
  procedural 248–52
parallel processing 115, 652
parallel system 190, 652
parameter 264, 652
parent 337, 652
parent directory 361, 652
parity bit 607
parser 492–3, 652
partitioning 193–4, 652
Pascal 246, 251, 652
Pascal, Blaise 9
pass by reference 266, 652
pass by value 264, 652
passive hub 652
passive objects 248–9
password 436
password-based authentication 652
patent 527–8, 653
path 148, 337, 361, 653
pathname 361
peer-to-peer (P2P) paradigm 145–6, 154–6, 653
peering points 136
perception 487, 653
  image processing 488–91
  language understanding 491–3
perceptron 499–500, 653
perceptual encoding 405–6
performance 99, 206
PERL 653
personal system 190
photoreceptor 63

physical address 653
physical agent 475, 653
physical layer 141, 174, 653
  analog transmission 176–7
  data/signals 175
  digital transmission 175–6
pico 204, 653
picture elements 63
pipelining 114–15
pit 100, 653
pixels (picture element) 63, 653
place value 653
plaintext 325, 653
point-to-point 135, 653
point-to-point wireless WANs 171
pointer 653
polyalphabetic cipher 418–19
polycarbonate resin 100, 171
polymorphism 171, 253
polynomial problem 253, 468
pop operation 321, 653
port addresses 43
port number 142, 148, 157–8, 654
portability 206, 654
positional number systems 16–31, 654
postfix 341, 654
postorder traversal 340, 654
pragmatic analysis 654
pre pointer 304, 305
predicate logic 477, 480–2, 654
predicted frame (P-frame) 404, 654
predictive encoding 405, 654
prefix 341, 654
preorder traversal 339, 654
pretest loop 263
prime area 358, 654
print 249
printer 98, 654
privacy 538
private key 424
private use planes 535
probe 654
problems
  big-O notation 468
  complexity 467–8
  halting problem not solvable 465–7
  nonpolynomial 468
  polynomial 468
  solvable 467–8
  unsolvable 467, 665
procedural language 654
procedural paradigm 248–52, 654
procedure 249, 372, 654
procedure-oriented analysis 276–7, 654
procedure-oriented design 279–80, 654
process 197, 655
  control block 199
  manager 196–202, 655
  scheduler 198
  synchronization 199–202, 655
process-to-process communication 157, 655
processing
  algorithm 215–16
  instructions 120–1
product 224–5, 655
product of sums 570
program 2, 196, 655
  counter 93, 655
  execution 109–13
  storage 5, 7, 122
programmable data processor 655
programmable read-only memory (PROM) 96, 655
programmed I/O 110–11, 655
programming language 11, 244, 475, 655
  common concepts 257–67
  evolution 244–6
  paradigms 248–57
  translation 246–8
project operation 378–9, 655
PROLOG (PROgramming in LOGic) 256–7, 475, 655
PROM *see* programmable read-only memory
proof by contradiction 465
propositional logic 477–80, 655
protocol 148, 655
protocol layering 137–40, 655
  addressing/packet names 142–3
  architecture 141–2
  logical connections 139–40
  principles 139
provider networks 136
proxy firewall 444
pseudocode 219–23, 553, 655
pseudocode components 554
  algorithm header 554
  postcondition 554
  precondition 554
  purpose 554
  return 554
  statement 554–6
public key 424, 655
  certificate 656
  encryption 655
public-key distribution 441
  certification authority 441–2
  public announcement 441
  X.509 442
push operation 321, 656

Q

quantifier 481, 656
  existential 481
  universal 481
quantization 62, 403, 656
queue 199, 326, 656
  applications 328–9
  implementation 330–1
  operation 326–8

R

radio waves 180
radix 656
RAM *see* random access memory
random access 350, 656
random access memory (RAM) 95, 656
random testing 285
raster graphic 63–5, 656
read-only memory (ROM) 96, 656
read/write head 98, 457, 656
ready state 197, 656
real 18, 51, 656
  arithmetic operations 86
  binary system 19–20
  decimal system 18
  hexadecimal system 21
  octal system 22
  operations 621–8
  storing 51–60
  type 257
real-time system 190, 656
rear 656
record 298–9, 656
  array 299–301
  arrays vs array of records 301
  record name vs field name 299
  records/arrays comparison 299
recursion 234–6, 656
recursive definition 234–5
recursive solution 236
reduced instruction set computer (RISC) 114, 656

redundancy 370, 602, 613
register 93, 579, 656
  data 93
  instruction 93
  program counter 93
relation 374–5, 656
relational database model (RDBMS) 374–5, 656
  operations 375–81
relational model 374, 656
relational operator 260, 656
relationship sets 383
  first normal form (1NF) 384
  second normal form (2NF) 384–5
relationships 540
relative pathname 361, 656
reliability 206, 657
remote logic 657
repetition 218, 262, 657
replaying 414, 657
replicated distributed database 386, 657
repudiation 414, 657
reserved words 657
residue 596
resolution 63, 657
resource holding 201, 657
restricted list 657
retransmission 602
retrieve 333, 657
  elements 297
  IEEE standards 58
  node 311–12
  two's complement format 48–9
  unsigned integers 43
returning values 267
RGB (red, green, blue) 63, 657
Richens, Richard H. 476
ring topology 657
RISC *see* reduced instruction set computer
Rivest, Ron 429, 657
Rivest-Shamir-Adleman (RSA) 426–8, 657
ROM *see* read-only memory
Roman number system 657
root 337, 657
  directory 361, 657
  hub 107
rotate operation 80
rotational speed 657
router 134, 657
routing 141, 162, 657
routing protocols 162
row-major storage 295, 658
RSA *see* Rivest-Shamir-Adleman
rule-based system 483, 658
  backward chaining 484–5
  components 483–4
  forward chaining 484
run-length encoding 392–3, 658
running state 197, 658

S

sampling 62, 658
  rate 62, 658
satellite network 173
scanning 63, 658
scenario 137–9
scheduler 198–9, 658
scheduling 658
Scheme 255, 658
scientific notation 658
scores 301
SCSI *see* small computer system interface
SCTP *see* Stream Control Transmission Protocol
search space 494, 658
searching 230, 494, 658
  breadth-first 495–6
  brute-force 495–7
  depth-first 496–7
  heuristic 497–8
  methods 494–8
secondary storage device 350
secret key 424, 658
sector 658
Secure Hash Algorithm (SHA) 429, 658
Secure Shell (SSH) 151–2, 658
security 412, 658
  attack 413–14, 658
  availability 412
  confidentiality 412, 415–28
  digital signature 431–5
  entity authentication 435–8
  firewall 442–4
  goal 412, 658
  integrity 412, 428–9
  issues 11, 205
  key management 438–42
  message authentication 430
  services/techniques 414–15
seek time 99, 659
segment 159, 659
segmentation 195, 489, 659
select operation 377, 659
selection 218, 262, 555, 659
selection sort 226–7, 659
self-referential record 659
semantic analysis 248, 493, 659
  purpose 493
  removing ambiguity 493
semantic network 476, 659
  concepts 476
  relations 476
sentence 478, 480–1
sequence 218, 262, 555, 659
sequence diagram 546–8
sequential
  access 659
  execution of instructions 6, 7–8
  search 230–1, 659
sequential circuits 576
  digital counter 580
  flip-flops 576–8
  register 579–80
  synchronous vs asynchronous 578–9
sequential file 350–1, 659
  updating 352–3
server 659
SHA *see* Secure Hash Algorithm
shading 490
shell 204, 659
shift cipher 417, 659
shift operations 79–82, 93
siblings 337, 659
side effect 442, 659
sign 54
sign out 511, 512
sign up 511–12, 517
sign-and-magnitude representation 44–6, 659
  addition/subtraction 85
  applications 46
  operations on integers 617–20
  overflow 46
SIMD *see* single instruction-stream, multiple data-stream
simple
  ADTs 318
  computer 117–26
  condition 284
  parity-check code 607–9
  shift 79–80
  type 257, 660
simple language 452
  input/output 455
  macros 453–5
  simulating 459–62

Simple Mail Transfer Protocol (SMTP) 660
single instruction-stream, multiple data-stream (SIMD) 116, 660
single instruction-stream, single data-stream (SISD) 115–16
single-bit error 602
single-user operating system 660
SISD *see* single instruction-stream, single data-stream
slots 477
small computer system interface (SCSI) 106, 660
SMTP *see* Simple Mail Transfer Protocol
snooping 413, 660
social contract 526, 660
social media 508, 660
  Facebook 508–14
  Twitter 514–21
social network 660
software 7, 137, 188, 371, 660
  agent 474–5, 660
  algorithms 8
  anti-virus 530
  languages 8
  operating systems 9
  program storage 7
  protective 529
  sequence of instructions 7–8
software engineering 9, 11, 274, 660
  analysis 276–9
  design 279–80
  documentation 285–6
  implementation 280–3
  lifecycle 274–5
  testing 283–5
software lifecycle 274, 660
  development process models 274–5
software quality 281, 660
  maintainability 282
  operability 281–2
  transferability 282–3
solvable problem 660
soma 498, 660
something inherent 436, 660
something known 436, 660
something possessed 436, 660
sort pass 226, 660
sorting 226–30, 660
sound encoding 63
source program 246, 660
source-to-destination delivery 660
spatial compression 404, 661
speech recognition 491, 661
spoofing 661
SQL *see* Structured Query Language
SR flip-flops 576–7
SRAM *see* static RAM
SSH *see* Secure Shell
stack 326, 661
  ADT 322
  applications 322–5
  implementation 325–6
  operations 320–2
stackName 320
Standard Ethernet 168, 168–9
standards
  IEEE 55–60
  image encoding 65
  sound encoding 63
star topology 661
start point 549
starvation 199, 201–2, 661
state 545
  chart 279, 661
  diagram 197–8, 277, 545–6, 661
statements 261–3
  combination 381
  constructs 554–5
  control 261–3, 635
  decrement 452, 460
  increment 452, 459–60
  loop 452, 461–2, 556
  pseudocode 554–6
  selection 555
  sequence 555
static document 661
static RAM (SRAM) 95, 661
statistical compression 661
steganography 415, 661
stereo vision 489
stereopsis 489
storage
  audio 61–3
  data 6, 122
  IEE standards 56–7
  images 63–5
  numbers 42–60
  other data types 44
  program 5, 122
  text 60–1
  video 65–6
storage device 98, 661
  auxiliary/secondary 350
  magnetic 98–9
  optical 99–104
stream cipher 420, 661
Stream Control Transmission Protocol (SCTP) 661
string 298
string of bits 40
structural view 539, 541–3
structure chart 234, 279, 661
  loops 559
  reading 560
  rules 560–1
  selection 558–9
  symbols 557–9
structure program 661
Structured Query Language (SQL) 375, 661
student 295, 298, 299–301
subalgorithm 233–4, 662
subclass 542
subprogram 263–7
substitution cipher 417, 662
subsystem interconnection 104
  CPU/memory 104–5
  I/O devices 105–9
subsystems 4–5
  arithmetic logic unit 5
  control unit 5
  environmental 207
  input/output 5
  memory 5
subtree 662
sum of products 570
summation 224, 255, 662
superclass 542
SuperSpeed 108
supplementary
  ideographic plane 534
  multilingual plane 534
  special plane 535
surface organization 99
swapping 193
swimlane 549–50
switch 134
switch statement 262
switched WAN 135, 173, 662
sychronous devices 579
symbolic language 245, 662
symbols 545, 546, 548
symmetric-key cipher 415–17, 436–7, 662
  modern 421–3
  stream/block ciphers 420–1
  traditional 417–20
symmetric-key distribution 438–41
symmetric-key encryption 662
synapse 662
syndrome 662
synonym 356, 662

syntactic analysis 491, 662
  grammar 491–2
  parser 492–3
syntax 244, 662
syntax analyzer 248, 662
system 11, 540
  documentation 662
  libraries 205
  utilities 205
systems development lifecycle 662

T

T flip-flop 578
table-to-expression transformation 570–1
TCP *see* Transmission Control Protocol
TCP/IP *see* Transmission Control Protocol/Internet Protocol
technical documentation 286, 662
TELNET (TErminaL NETwork) 151, 662
temporal
  compression 404, 662
  logic 483, 662
  masking 662
terminal state 663
terminated state 198, 663
testability 282, 663
testing phase 283, 663
  black-box testing 285
  glass-box testing 283–5
text 663
  editor 41, 663
  file 362
  storage 60–1
texture 490
theorems 568
threshholding 489, 663
throughput 114, 663
ticket 440, 663
time-sharing system 190, 663
token 247, 663
topology 107, 603
track 99, 663
trade secret 527, 663
trademark 527, 663
traffic analysis 413, 663
trailer 663
transaction file 352, 663
transfer time 99, 663
transferability 282, 663
  interoperable 283
  portable 283
  reusable 282
transister 664
transitions 457, 545, 549
translation process 247–8
translator 248
Transmission Control Protocol (TCP) 159, 664
Transmission Control Protocol/Internet Protocol (TCP/IP) 140, 662, 664
transmission media 177–8, 664
transmission rate 664
transport layer 156–7, 664
  protocols 158–9
  services 157–8
transposition cipher 419–20, 664
traversal 333–4, 664
  array 297–8
  binary tree 339–40
  linked list 312–13
tree 337–43, 664
Trojan horse 529, 664
true value 304
True-Color 63–4, 664
truncation error 59–60, 664
truth table 74, 664
tuple 375, 664
Turing, Alan M. 2, 10, 456, 474
Turing machine 2, 456–63, 664
  Church-Turing thesis 462–3
  simulating simple language 459–62
  universal 4
Turing machine components 456
  controller 457–9
  read/write head 457
  tape 456–7
Turing model 2–4, 664
Turing Test 474, 664
tweet 514, 664
  composing 520–1
  receiving 521
  referring to other people's tweets 521
  retweet 657
  sending 520–1
  using ampersands 521
  using hashtags 521
twisted-pair cable 178–9, 664
Twitter 514, 664
  accessing 518–19
  being followed 520
  communication 516
  following other members 519–20
  home page 517
  log in 518
  log out 519
  member-followers relationship 515
  membership 517–18
  receiving tweets 521
  referring to other people's tweets 521
  searching 520
  sending tweets 520–1
  sign out/deactivation 518
  sign up 517–18
  stop following a member 520
  web pages 517
two operations 47
two-dimensional array 294, 665
two's complement 665
two's complement representation 47–8, 665
  addition/subtraction 82–5
  application 50
  overflow 50
  retrieving 48–9
  storing 48
type 541, 665

U

UA *see* user agent
UDP *see* User Datagram Protocol
UML *see* Unified Modeling Language
unary operation 375–6, 665
underflow 59, 665
undirected graph 343
unfollow 520
unguided media 179–80
Unicode 61, 533, 665
  ASCII 535–8
  planes 534–5
Unified Modeling Language (UML) 219, 539, 665
  behavioral view 539, 543–9
  implementation view 539, 550–1
  structural view 539, 541–3
  user view 539, 540
uniform resource locator (URL) 148–9, 665
union operation 379, 665
Universal Serial Bus (USB) 107–8, 665
universal Turing machine 4
UNIX 203, 665

directories 360–1
structure 203–5
unreliable delivery 161–2
unsigned integer 42–3, 665
applications 44
overflow 43–4
retrieving 43
storing 43
unsolvable problem 467, 665
update operation 376–7, 665
URL *see* uniform resource locator
USB *see* Universal Serial Bus
use-case diagram 277–8, 540, 665
user
datagram 158, 665
documentation 286
interface 191, 487, 666
view 539, 540
user agent (UA) 665
User Datagram Protocol (UDP) 158, 666
users 371, 487, 666
utility 204, 666
utilization 666

V

values 543
variable 258, 292, 666
clearing 453
declarations 258
initialization 258
vector graphic 65, 666
verification
algorithm 666
digital signature 431
verification categories
something inherent 436, 660
something known 436, 660
something possessed 436, 660
vertex 337, 666
vi 204, 666
video 666
storage 65–6
videoconferencing 666
virtual memory 196, 666
virus 529, 666
von Neuman, John 4, 10
von Neuman model 4–6, 666
four subsystems 4–5
sequential execution of instructions 6
stored program concept 5

W

waiting state 666
walking order 340
WAN *see* wide area network
waterfall model 274–5, 666
Web 666
client 148
server 148
web page 147, 510, 510–11, 516, 666
home page 510, 516
user page 510–11
well-known port numbers 158, 666
while loop 262–3, 305, 460
white-box testing 666
wide area network (WAN) 134–5, 171, 667
wired 171–3
wireless 173
WiFi *see* wireless Ethernet
Wilkes, Maurice 10
WiMax *see* Worldwide Interoperability Access
Windows 205, 667
architecture 206–7
design goals 205–6
Windows NT 667
wireless communication 179–80
wireless Ethernet (WiFi) 170
Wirth, Niklaus 9
words 94
working directory 361, 667
World Wide Web (WWW) 145, 147–9, 667
Worldwide Interoperability Access (WiMax) 173, 667
worm 529, 667
WORM *see* write once, read many
write once, read many (WORM) 102, 667
WWW *see* World Wide Web

X

X.509 442
XML (Extensible Markup Language) 386, 640
XNOR gate 565
XOR (exclusive OR) 640
XOR operator 75–6, 565, 667

Z

zero storage 59
zero-knowledge authentication 667
Ziv, Jacob 306
Zuse, Konrad 10